

PrimeSim™ Custom Fault User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	16
Related Products, Publications, and Trademarks	16
Conventions	17
Customer Support	18
Statement on Inclusivity and Diversity	18

1. Introducing Custom Fault	20
Overview of Fault Simulation	20
Setting Up the Custom Fault Tool	22
Configuring the Custom Fault License	22
License Queuing	23
Setting Up the Path to the Executable	26
Defect Types	27
Comparison of Fault Campaign Methods	29

2. Running Custom Fault	30
Setting Up Fault Simulation	30
Setting Up Defect Detection Criteria	30
Using a Vector Check	31
Using a .MEASURE Statement	32
Using a Circuit Check Command	34
Using Simulation Output Messages	35
Creating a Configuration File	35
Defining Defect Models	36
Running Fault Simulation	37
Command-Line Options	37
Rerunning an Interrupted Fault Simulation	40
Single Run Example	41
Multi-Testbench Flow Example	41
Merging Fault Databases (FDBs)	42
Methodology for Merging FDBs	42
Merged FDB Command Syntax	43

Merged FDB Examples	43
Importing IP Simulations	44
IP Import Methodology	44
IP Import Command Syntax	45
Output Report File	46
IP Import Examples	46
Common DP Library Information	47
3. Defining and Scoping Defect Models	49
Supported Built-in Defect Models	49
Device Instance Defect Model Diagrams	50
Customizing User-defined Defect Models	54
Resistor Divider Gate Open Defect Model	56
Resistor Divider Syntax	57
Resistor Divider Examples	58
Pin Open and Net Open Defect Models	60
Pin Open and Net Open Defect Model Sample Diagrams	61
Pin Open and Net Open Defect Model Syntax	63
Pin Open and Net Open Defect Model Examples	68
Instance Parameter Variation Defect Models	70
MOS Primitive Variation Defects	73
Defining Custom SPICE Defect Models	74
Custom SPICE Defect Model Definition in the SPICE Netlist File	75
Custom SPICE Defect Model Definition in the Configuration File	75
Custom SPICE Defect Model Example	75
Applying Built-in and Customized Defect Models to a Circuit	77
4. Weighting and Sampling for Defects	78
Defect Weights	78
Defect Sampling	79
Specifying Weights for Defects	79
bhv_cap_open_weight and bhv_cap_short_weight Parameters	82
bhv_cap_pinopen_weight and bhv_cap_netopen_weight Parameters	83
bhv_res_open_weight and bhv_res_short_weight Parameters	83
bhv_res_pinopen_weight and bhv_res_netopen_weight Parameters	84

Contents

bjt_open_weight and bjt_short_weight Parameters	85
bjt_pinopen_weight and bjt_netopen_weight Parameters	86
cap_open_weight and cap_short_weight Parameters	86
cap_pinopen_weight and cap_netopen_weight Parameters	87
diode_open_weight and diode_short_weight Parameters	88
diode_pinopen_weight and diode_netopen_weight Parameters	88
ind_open_weight and ind_short_weight Parameters	89
ind_pinopen_weight and ind_netopen_weight Parameters	90
jfet_open_weight and jfet_short_weight Parameters	90
jfet_pinopen_weight and jfet_netopen_weight Parameters	91
mos_open_weight and mos_short_weight Parameters	92
mos_pinopen_weight and mos_netopen_weight Parameters	93
net_to_net_short_weight Parameter	94
res_open_weight and res_short_weight Parameters	94
res_pinopen_weight and res_netopen_weight Parameters	95
subckt_open_weight and subckt_short_weight Parameters	96
subckt_pinopen_weight and subckt_netopen_weight Parameters	96
Weight File Example	97
Assigning Weight Based on Hierarchical Instance Names	99
Estimating Error and Sample Size	100
Sampling Error Configuration Options	100
Estimated Coverage Error Bound Report Output	101
Estimated Coverage Error Bound Report Examples	103
Using Adaptive Weighted Random Sampling (AWRS)	106
Specifying a New Seed for Adaptive Weighted Random Sampling (AWRS)	108
Weighted Coverage Report With Adaptive Weighted Random Sampling (AWRS)	108
Using Incremental Sampling	111
Using Weight Sorted Sampling (WSS)	112
Test Coverage With Weight Sorted Sampling (WSS)	113
Diagnostic Coverage of Residual Faults With Weight Sorted Sampling (WSS)	114
Adaptive Weighted Random Sampling (AWRS) Tutorial	114
Generating a Template Weight File	116
Running an Exhaustive Fault Campaign With a Weight File	118
Generating an Error Bound Report	121
Running Different Sample Sizes With a Weight File	123
Weighted Sorted Sampling (WSS) Tutorial	125

Running the WSS Tutorial With PrimeSim XA	127
Running the WSS Tutorial With FineSim	129
Running the WSS Tutorial With PrimeSim	131
IEEE P2427 Weight Tutorial	133
IEEE P2427 Recommendations	134
Defect Models Definition	135
Weight Definition	136
Running the Tutorial With PrimeSim XA	136
Running the Tutorial With FineSim	137
Running the Tutorial With PrimeSim	139
Adding printf Statements in the Weight File	140
5. Custom Fault Stimulus Quality Assessment	141
Overview of Stimulus Quality Assessment	141
Active Stimulus Quality Assessment	141
ASQA Limitations	142
Concurrent Stimulus Quality Assessment	142
CSQA Limitations	142
Running Active SQA	142
Using Active SQA Output Files	143
Running Active SQA with Multiple Testbenches	144
Running Concurrent SQA	145
Concurrent SQA Output Files	146
6. Custom Fault Automatic Injection Flow	148
Overview of the Automatic Injection Flow	148
Automatic Injection Flow Tutorial	148
Running the Custom Fault Automatic Injection Flow With PrimeSim XA	150
Creating the Configuration File to Run Custom Fault With PrimeSim XA	150
Running the Custom Fault Tool With PrimeSim XA	151
Generating Fault Simulation Output Using PrimeSim XA	151
Running the Custom Fault Automatic Injection Flow With FineSim	152
Creating the Configuration File To Run Custom Fault With FineSim	153
Running the Custom Fault Tool With FineSim	154
Generating Fault Simulation Output Using FineSim	154
Running the Custom Fault Automatic Injection Flow With PrimeSim	155
Creating the Configuration File To Run Custom Fault With PrimeSim	156

Running the Custom Fault Tool With PrimeSim	157
Generating Fault Simulation Output Using PrimeSim	157
7. Custom Fault Manual Injection Flow	159
Overview of the Manual Defect Injection Flow	159
Defect List File Format	160
Device Instance Open Defect	161
Device Instance Short Defect	162
Subcircuit Open Defect	163
Subcircuit Short Defect	163
Behavioral Resistor Open Defect	164
Behavioral Resistor Short Defect	165
Behavioral Capacitor Open Defect	165
Behavioral Capacitor Short Defect	166
MOSFET Resistor Divider Gate Open Defect	166
Subcircuit MOSFET Resistor Divider Gate Open Defect	167
Net to Net Short Defect	167
Examples	168
Parametric Defect	169
Parametric Defect Examples	170
Manual Injection Flow Tutorial	171
Providing the Defect List File for the Manual Injection Flow	172
Running the Custom Fault Manual Injection Flow With PrimeSim XA	173
Creating the Configuration File to Run Custom Fault With PrimeSim XA	173
Running the Custom Fault Tool With PrimeSim XA	174
Generating Fault Simulation Output Using PrimeSim XA	175
Running the Custom Fault Manual Injection Flow With FineSim	176
Creating the Configuration File to Run Custom Fault With FineSim	176
Running the Custom Fault Tool With FineSim	177
Generating Fault Simulation Output Using FineSim	177
Running the Custom Fault Manual Injection Flow With PrimeSim	179
Creating the Configuration File to Run Custom Fault With PrimeSim	179
Running the Custom Fault Tool With PrimeSim	180
Generating Fault Simulation Output Using PrimeSim	180
8. Custom Fault Multi-Testbench Flow	182
Overview of the Multi-Testbench Flow	182
Multi-Testbench Flow Tutorial	183

Running the Custom Fault Multi-Testbench Flow With PrimeSim XA	185
Creating the Configuration File to Run Custom Fault With PrimeSim XA	185
Running the Custom Fault Tool With PrimeSim XA	187
Generating Multi-Testbench Simulation Output Using PrimeSim XA	187
Running the Custom Fault Multi-Testbench Flow With FineSim	189
Creating the Configuration File to Run Custom Fault With FineSim	190
Running the Custom Fault Tool With FineSim	191
Generating Multi-Testbench Simulation Output Using FineSim	191
Running the Custom Fault Multi-Testbench Flow With PrimeSim	194
Creating the Configuration File to Run Custom Fault With PrimeSim	194
Running the Custom Fault Tool With PrimeSim	195
Generating Multi-Testbench Simulation Output Using PrimeSim	195
9. Custom Fault Mixed-Signal Flow	199
Overview of the Mixed-Signal Fault Simulation Flow	199
Digital Waveform Comparison in the Mixed-Signal Flow	200
Signal File Format	200
Signal File Example	203
Conditional Comparison of Signals	203
Comparison of Real Signals	204
Time Window Based FSDB Comparison	204
Defining a Time Window	205
Specifying the Time Window in a Signal	205
Time Window Examples	205
Mixed-Signal Flow Tutorial	206
Running Mixed-Signal Fault Simulation for Verilog Top Configuration	208
Creating the Configuration File to Run Mixed-Signal Fault Simulation With FSDB Comparison Check	209
Running Mixed-Signal Fault Simulation	211
Generating Mixed-Signal Fault Simulation Output	212
Running Mixed-Signal Fault Simulation for SPICE Top Configuration	213
10. Custom Fault Post-Layout Flow	217
Overview of the Post-Layout Flow	217
Post-Layout Flow Tutorial	218
Running the Custom Fault Post-Layout Flow With PrimeSim XA	219
Running the Custom Fault Post-Layout Flow With FineSim	221
Running the Custom Fault Post-Layout Flow With PrimeSim	222

Post-Layout Mixed-Signal Flow Tutorial	223
Running the Custom Fault Post-Layout Mixed-Signal Flow Tutorial With PrimeSim XA	225
Running the Custom Fault Post-Layout Mixed-Signal Flow Tutorial With FineSim	227
Running the Custom Fault Post-Layout Mixed-Signal Flow Tutorial With PrimeSim	229
11. Custom Fault Parametric Defect Injection	231
Parametric Defect Injection Flow	231
Preparing the Netlist for PDI	232
Generating a Template File for PDI	232
Defining Parameter Variations for PDI	233
12. Using Save-and-Restore With Custom Fault	234
When to Use Save-and-Restore With Custom Fault	234
Save-and-Restore Using the PrimeSim XA Simulator	235
Saving the PrimeSim XA Simulation when the Circuit is Stable	235
Running the Fault Campaign With PrimeSim XA Save Results	236
Example — Save-and-Restore With PrimeSim XA	236
Save-and-Restore Using the FineSim Simulator	237
Saving the FineSim Simulation when the Circuit is Stable	238
Running the Fault Campaign With FineSim Save Results	238
Example — Save-and-Restore With FineSim	238
Save-and-Restore Using the PrimeSim Simulator	239
Saving the PrimeSim Simulation when the Circuit is Stable	240
Running the Fault Campaign With PrimeSim Save Results	240
Example — Save-and-Restore With PrimeSim	240
Save-and-Restore Known Limitations	241
Save-and-Restore With the Custom Fault Mixed-Signal Flow	241
Running and Saving Standalone Mixed-Signal Simulation Until the Circuit is Stable or Test Conditions Are Met	242
Restoring the Saved Simulation and Running the Fault Campaign	242
13. Custom Fault Output Files	243

Test Coverage Report (Single Testbench)	243
Header	243
Defect Universe Summary	244
Coverage Summary	244
Detailed Coverage Listing	245
Simple Format Report	246
Detailed Format Report	247
Test Coverage Report (Multi-Testbench)	248
Diagnostic Coverage Report	250
Not-covered Defect Analysis	253
14. Using Custom Fault for Functional Safety	256
Overview of Functional Safety	256
ISO 26262 Basics	257
Automotive Safety Integrity Level (ASIL)	257
Single Point Fault Metric (SPFM)	257
Single Point Faults and Residual Faults	258
Diagnostic Coverage (DC)	259
Calculating Diagnostic Coverage	259
Failure Mode	260
Latent Fault Metric	260
Calculating Diagnostic Coverage for Latent Faults	262
Measuring the Diagnostic Coverage of Residual Faults	263
Defining Observation Points	263
Defining Thresholds for Fault Detection	264
Diagnostic Coverage Command	264
Diagnostic Coverage Command Example	266
Analyze Fault Campaign Results	267
Diagnostic Coverage Output Example	267
Diagnostic Coverage of Residual Faults Tutorial	269
Tutorial Files and Commands for the PrimeSim XA Simulator	269
Observation Points and Thresholds for Fault Detection	270
Diagnostic Coverage Commands	270
Running the Custom Fault Tutorial With PrimeSim XA	271
Tutorial Files and Commands for the FineSim Simulator	272
Running the Custom Fault Tutorial With FineSim	273
Tutorial Files and Commands for the PrimeSim Simulator	274
Running the Custom Fault Tutorial With PrimeSim	274

Contents

Output Results	275
Measuring the Diagnostic Coverage of Latent Faults	278
Defining Observation Points and Thresholds for Fault Detection	279
Diagnostic Coverage Latent Faults Command Example	279
Analyzing Fault Campaign Results	280
Example of Diagnostic Coverage of Latent Faults Report	280
Diagnostic Coverage of Latent Faults Tutorial	281
Tutorial Overview	282
Input, Netlist, and Script Files	282
Circuit Description	283
Observation Points and Thresholds for Fault Detection	283
Running the Diagnostic Coverage of Latent Faults Tutorial	283
Performing Diagnostic Coverage of Residual Faults	284
Analyzing Defects in the Safety Mechanism	285
Dual-Fault Injection	286
Running Dual-Fault Injection	288
Diagnostic Coverage of Latent Faults	288
Direct SPFM Calculation	290
SPFM Definition	291
Limitations of Direct SPFM Calculation	292
Command Specification	292
SPFM Calculation	292
SPFM Output	293
15. Using Custom Fault for Detectable Fault Analysis	294
Overview of Detectable Fault Analysis	294
Invoking DFA	295
Recommended Use Flow	295
Running static analysis	296
Using DFA With Mixed-Signal Simulation	296
DFA Output Results	297
How to Read the .dfa_report File	297
DFA Waveform Comparison Utility (cfutil)	299
DFA Tutorial	300
Running the Tutorial Using the PrimeSim XA Simulator	301
DFA Report	302
Coverage Report	304
Electrical Collapsing	305

Improving Coverage	305
Using New Stimuli	305
Using the DFA Waveform Comparison Utility	306
DFA Mixed-Signal Tutorial	310
Running the Tutorial Using the VCS PrimeSim AMS Simulator	311
DFA Report	312
Coverage Report	314
Electrical Collapsing	315
Using the DFA Waveform Comparison Utility	316
16. Running the PrimeSim Custom Fault Application	320
Tool Setup	320
Launching the Custom Fault App	321
PrimeSim Custom Fault App Entry Points	322
Starting a New Session	324
Environment Settings	325
Functional Safety Settings	326
Importing Testbenches	328
Importing Testbenches in the PrimeWave Design Environment Flow	328
Importing Analog-Only Testbenches in the Netlist Flow	331
Importing Mixed-Signal Testbenches in the Netlist Flow	335
Manufacturing Test Testbench Setup	337
Test Options	337
Defect Detection Expressions	338
Mixed-Signal Digital Waveform Compare Settings	339
Functional Safety Analysis Testbench Setup	341
Failure Modes	341
Expression	342
Mixed-Signal Functional Safety Setup for Digital Waveform Comparison	342
Silicon Failure Analysis Testbench Setup	343
Requirements	344
Launching the Custom Fault App	344
Selecting the Netlist and Measurement Expression	347
Creating the Defect Models	348
Running Simulations	349
Creating More Measurements and Running What-if Analysis	351
Deleting original measurements	354

Contents

Limitations	355
Defect Model Settings	356
Defect Weights	359
Defect Universe Definition	360
Manual Injection of Defects	363
Launching the Defect Campaign	363
Defect Sampling	363
Launching Simulations	364
Coverage Reports	365
Analyzing Defect Campaign Results	366
Loading an Existing Fault Database	370
Viewing Defect Simulation Waveforms	371
Schematic Highlighting	372
Analyzing Coverage With the Fault Database Viewer	374
Fault Database Viewer	374
Invoking the FDV	375
Information and Help	376
Zooming In and Out	377
Changing the Colors	377
Displaying Defects	379
Highlighting Covered or Non-covered Defects	382
Using an Expression to Display Defects	387
Highlighting Defects for Functional Safety	389
Displaying a Heat Map of Weights (or Weight Map)	390
Outputting Results	392
Screenshot	392
Text File	392
Windows Manipulation	393
Other Display Options	393
Hide Nodes and Devices	393
Hide Depth	394
Display Instance Names	395
Displaying Waveforms	396
17. Configuration File Command Reference	398
Input Commands	398
analog_setup	399
case_dir	402

Contents

circuit_device_count	402
cleanup_level	408
collapsing	408
collapsing	409
cosim_setup	410
cpulimit	414
create_fault_files	415
dc_operating_point	415
defect_weight	416
detectability	417
diagnostic_coverage (for Latent Faults)	418
diagnostic_coverage (for Residual Faults and Direct SPFM Calculation)	419
dpconfig_file	423
enable_fault_collapse	424
enable_stackmos_fault_collapse	424
fault_list_file	427
exclude_fault_list_file	427
fault_sim_output	428
generate_dual_fault	434
grouped_waveform	435
identify_only	436
nc_analysis	436
outdir	436
parameter_variation	437
report	438
resume_data	446
sampling	447
simulate_only	449
sqa	452
what_if	455
Defect Model Commands	458
apply_fault_model	459
faults	467
A. Frequently Asked Questions (FAQ)	477
General	477
License	479

Contents

Defect Models and Simulation	479
Weight and Sampling	483
Mixed-Signal Simulation	490
Manufacturing Test	491
Functional Safety	492

About the PrimeSim Custom Fault User Guide

The *PrimeSim Custom Fault User Guide* describes how to use the Synopsys® PrimeSim Custom Fault tool, an analog fault simulator that helps identify hardware issues to improve reliability.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the *PrimeSim Custom Fault Release Notes* on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the PrimeSim Custom Fault tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

- PrimeSim™ XA
- FineSim®
- PrimeSim™ (PrimeSim™ Pro and PrimeSim™ SPICE)
- VCS®
- VCS® PrimeSim™ AMS
- WaveView™

- PrimeWave™ Design Environment WaveView
- Avalon™

To run the Custom Fault tool, certain UNIX/Linux platform requirements and compatibility with other Synopsys tools (such as PrimeSim XA, FineSim, PrimeSim, VCS, VCS PrimeSim AMS, WaveView, or PrimeWave Design Environment WaveView) must be respected. Those requirements are listed in the *PrimeSim Custom Fault Tool and Platform Compatibility Table* on SolvNetPlus:

<https://solvnetplus.synopsys.com/s/article/TestMAX-CustomFault-Tool-and-Platform-Compatibility-Table>

Note:

You must be logged in to SolvNetPlus for the preceding link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.

Convention	Description
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working

environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introducing Custom Fault

This chapter introduces the concept of analog fault simulation and the Custom Fault tool.

- [Overview of Fault Simulation](#)
- [Setting Up the Custom Fault Tool](#)
- [Defect Types](#)
- [Comparison of Fault Campaign Methods](#)

Overview of Fault Simulation

The Custom Fault tool calls the Synopsys FastSPICE tools (PrimeSim XA, FineSim Pro, and PrimeSim Pro tools) for transient simulation. It supports technologies to improve the throughput of fault simulation, such as a defect reduction algorithm to reduce the number of simulated defects. The weighting and sampling technologies can minimize the number of defects to be simulated, while increasing the defect coverage percentage. It uses the distributed processing capability to run many parallel simulations.

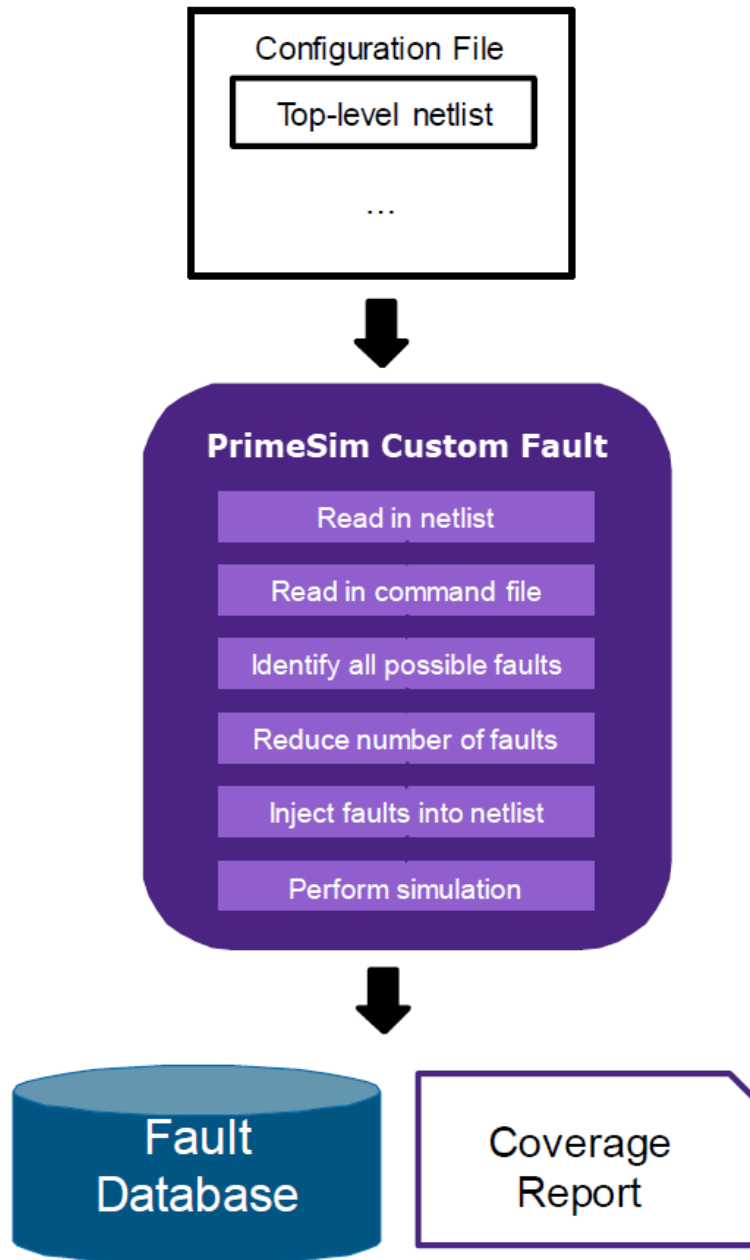
The Custom Fault tool also supports mixed-signal simulation, which improves the simulation throughput.

The three main applications for analog fault simulation are:

- Functional safety verification in the automotive Industry
- Manufacturing test coverage
- Post-silicon failure analysis

Figure 1 shows an overview of analog fault simulation.

Figure 1 Analog Fault Simulation Overview



The tool reads in the netlist and configuration file and identifies all the possible defects based on the netlist. Then it reduces them by merging equivalent defects. It also automatically injects one defect model at a time into the netlist to perform analog fault simulation based on the reduced list of the possible defects. The data of the circuit

condition is analyzed to determine whether the defect in the circuit is detected (or covered). The defect information and simulation data is stored in the fault database file.

The tool supports a multi-processing capability to launch many analog fault simulations in parallel to speed up the total runtime.

Setting Up the Custom Fault Tool

Before running the Custom Fault tool, you must complete the following setup:

- [Configuring the Custom Fault License](#)
- [Setting Up the Path to the Executable](#)

Configuring the Custom Fault License

You must set up one of the following environment variables to point to the license paths:

- `SNPSLMD_LICENSE_FILE` — A pointer to the Synopsys license paths
- `LM_LICENSE_FILE` — A pointer to all license paths

Note:

If you specify both environment variables, `SNPSLMD_LICENSE_FILE` takes precedence over `LM_LICENSE_FILE`.

You can specify the pointer to the license paths using the full path of the license file, the `port@host` syntax, or a combination of the two syntax types in a colon-separated list.

C-Shell:

```
setenv SNPSLMD_LICENSE_FILE pointer_to_license_paths
```

Bourne, Korn, or Bash shells:

```
export SNPSLMD_LICENSE_FILE=pointer_to_license_paths
```

Custom Fault operates in one of the following licensing modes.

- Standalone license mode (default)
- PrimeSim Reliability Analysis token license mode

Mixed-signal fault simulation is supported in standalone and token license modes and requires a VCS license and a license to enable cosimulation.

License Queuing

The `CF_WAIT_LICENSE` environment variable controls license queuing. By default, it is set to 1 to enable license queuing. You can disable license queuing as follows:

C-Shell:

```
setenv CF_WAIT_LICENSE 0
```

Bourne, Korn, or Bash shells:

```
export CF_WAIT_LICENSE=0
```

If `CF_WAIT_LICENSE_TIMEOUT` is set, the Custom Fault tool times out after the specified number of minutes with a timeout error message.

For example, the following setting causes queuing to time out after 120 minutes (two hours):

C-Shell:

```
setenv CF_WAIT_LICENSE_TIMEOUT 120
```

Bourne, Korn, or Bash shells:

```
export CF_WAIT_LICENSE_TIMEOUT=120
```

Configuring Custom Fault Standalone License Mode

This is the default license mode.

In standalone license mode, the maximum number of cores that can be used by the fault simulator's `-mt` command-line option is limited to 8.

In this mode, the Custom Fault tool queues for a license indefinitely unless the environment variable `CF_WAIT_LICENSE_TIMEOUT` is set.

The fault simulator license timeout settings in the user shell are kept by the Custom Fault tool. This allows the fault simulator license wait time to be set separately from Custom Fault. If the fault simulator license wait environment variables are not set in the user shell, the Custom Fault tool automatically sets them to be consistent with the `CF_WAIT_LICENSE` and `CF_WAIT_LICENSE_TIMEOUT` environment variables.

For example, to set the Custom Fault wait license timeout to 5 minutes and the fault simulator license wait timeout to 10 minutes, the following settings can be used:

```
%> setenv CF_WAIT_LICENSE 1
%> setenv CF_WAIT_LICENSE_TIMEOUT 5
%> setenv XA_WAIT_LICENSE 1
%> setenv XA_WAIT_LICENSE_TIMEOUT 10
%> setenv FINESIM_LICENSE_WAIT_TIMEOUT 600
%> setenv PRIMESIM_WAIT_LICENSE 1
%> setenv PRIMESIM_WAIT_LICENSE_TIMEOUT 10
```

Note:

The FineSim value is in seconds, while PrimeSim XA and PrimeSim values are in minutes.

Configuring PrimeSim Reliability Analysis Token License Mode

Use one of the following methods to enable PrimeSim Reliability Analysis token license mode:

- Define the shell environment variable `PRIMESIM` to 1:

- `csh`: `setenv PRIMESIM 1`
- `bash`: `export PRIMESIM=1`

For example:

```
%> setenv PRIMESIM 1
%> customfault -c fc.json -o cfs/cfs
```

- Use the `--primesim` command-line argument:
 - A value of 0 disables the token license mode
 - A value of 1 enables the token license mode
 - A value of 2 enables standalone and token-based license mode. By default, the standalone keys are attempted first (see the `PRIMESIM_ORDER` environment variable or the `--primesim_order` command-line option).

For example:

```
%> customfault -c fc.json --primesim 1
```

Note:

If both the shell environment variable is set and the `--primesim` command-line option is used, the command-line option takes precedence and overrides the shell environment variable setting.

[Table 1](#) lists the shell environment variables that control the token-based license mode and license queuing.

Table 1 *Variables for Controlling Token-Based License Mode and License Queuing*

Environment Variable	Description
<code>PRIMESIM 0 1 2</code>	This variable enables PrimeSim Reliability Analysis token-based license mode. Valid values: • 0: Disables token-based license mode. • 1: Enables token-based license mode. • 2: (Default) Enables standalone and token-based license mode. By default, the standalone keys are attempted first (see <code>PRIMESIM_ORDER</code>). This variable can also be enabled via the <code>-primesim</code> command-line option.
<code>PRIMESIM_ORDER 0 1</code>	This variable controls whether standalone keys or token-based keys are attempted first when the <code>PRIMESIM</code> variable is set to 2. • 0: (Default) Standalone keys are attempted first. • 1: Token-based keys are attempted first.

In token license mode, PrimeSim Reliability Analysis tokens are used in conjunction with PrimeSim Continuum tokens to execute a fault simulation.

In this mode, there is no limit on the number of cores used by the fault simulator's `-mt` command-line option. It is not limited to 8 or fewer as it is with standalone license mode.

In token license mode, the maximum number of concurrent fault simulations is not fixed to 4 times the `-k|--keys` command-line setting. The maximum number of concurrent fault simulations can be set with the command-line argument `--dp` instead of `-k`. For example:

```
%> customfault -c cfg1.json -k 10
```

The preceding example runs up to $10 \times 4 = 40$ parallel fault simulations.

```
%> customfault -c cfg1.json --primesim 1 --dp 25
```

The preceding example runs up to 25 parallel fault simulations. See [Command-Line Options](#) for additional details on the `--dp` option.

In PrimeSim Reliability Analysis token license mode, the Custom Fault tool queues for 2 minutes unless the environment variable `CF_WAIT_LICENSE_TIMEOUT` is set.

As in standalone license mode, the fault simulator license timeout settings in the user shell are kept by the Custom Fault tool. This allows the fault simulator license wait time to be set separately from Custom Fault. If the fault simulator license wait environment variables are not set in the user shell, the Custom Fault tool automatically sets them to be consistent with the `CF_WAIT_LICENSE` and `CF_WAIT_LICENSE_TIMEOUT` environment variables.

Setting Up the Path to the Executable

The Custom Fault executable is called by the launch script path. The launch script is in the `CUSTOMFAULT_INSTALL_DIR/bin` directory. You must set up the shell path to include the path to the Custom Fault launch script using the following syntax:

C-Shell:

```
set path = (CUSTOMFAULT_INSTALL_DIR/bin $path)
```

Bourne, Korn, or Bash shells:

```
export PATH=CUSTOMFAULT_INSTALL_DIR/bin:$path
```

For example, the following sets the path to the launch script at `/tools/synopsys/customfault-2020.03/bin`.

C-Shell:

```
set path = (/tools/synopsys/customfault-2020.03/bin $path)
```

Bourne, Korn, or Bash shells:

```
export PATH=/tools/synopsys/customfault-2020.03/bin:$path
```

The Custom Fault tool may call the following tools:

- PrimeSim XA
- FineSim
- PrimeSim (PrimeSim Pro and PrimeSim SPICE)
- WaveView
- PrimeWave Design Environment WaveView
- VCS
- VCS PrimeSim AMS

Follow the instructions in the documentation for the specific tools to set up the paths to these executables.

Defect Types

The Custom Fault tool supports the following types of defect models:

- *Open defect* is a type of defect that occurs where a terminal connection is disconnected where it should be connected. The simulation of an open defect can be modeled by injecting a large resistor and/or a capacitor to that terminal. Open defect is supported for the following devices:
 - MOSFET
 - Drain open defect
 - Gate open defect
 - Source open defect
 - Bulk open defect
 - Resistor divider gate open defect
 - Diode
 - BJT
 - JFET
 - Subcircuit
 - Resistor
 - Capacitor
 - Inductor
 - Custom SPICE defect
- *Short defect* is a type of defect that occurs when an incorrect connection exists between two terminals. The simulation of short defect can be modeled by injecting a small resistor and/or a capacitor connecting the two terminals. Short defect is supported for the following devices:
 - MOSFET
 - Bulk-gate short defect
 - Bulk-drain short defect
 - Bulk-source short defect
 - Drain-gate short defect

- Drain-source short defect
 - Source-gate short defect
- Diode
- BJT
- JFET
- Subcircuit
- Resistor
- Capacitor
- Inductor
- Custom SPICE defect
- *Net-to-Net short defect* is a short between two nets. A net is defined as resistively connected nodes.
- *Soft defect* is a type of defect that is modeled as a change in instance or model parameter values. This defect type changes a parameter of an instance in the simulation database and does not change any circuit connectivity.

For more information, see [Supported Built-in Defect Models](#) and [Defect Model Commands](#).

Comparison of Fault Campaign Methods

The Custom Fault tool supports the following methods of conducting a fault campaign:

- Exhaustive Fault Simulation

In this method, all identified defects in the defect universe are simulated and there is no uncertainty in the resulting coverage. However, the combined defect universe size and per-defect simulation time can make this method impractical for many circuits.

Note:

Exhaustive fault simulation is equivalent to Weight Sorted Sampling with a maximum error target of zero.

- Adaptive Weighted Random Sampling (AWRS)

This method provides a practical means to estimate the coverage results. The Custom Fault tool provides a 95% confidence interval, meaning that there is a 5% chance that the true coverage is outside of the confidence interval. There is always some residual uncertainty in the coverage result, but this method is the most efficient in terms of the number of simulations required to meet a required accuracy.

For more information, see [Using Adaptive Weighted Random Sampling \(AWRS\)](#).

- Weight Sorted Sampling (WSS)

The primary benefit of this method is that there is an absolute bound on the error in the coverage result. Some savings in terms of simulations can be achieved over exhaustive fault simulation. However, to get small margins of error, the simulation requirement approaches that of exhaustive fault simulation. This method is best suited to applications where the absolute error in the result must be a known quantity and for scenarios where the defect universe size combined with per-defect simulation time remains practical.

For more information, see [Using Weight Sorted Sampling \(WSS\)](#).

2

Running Custom Fault

This chapter outlines how to run the Custom Fault tool.

- [Setting Up Fault Simulation](#)
- [Running Fault Simulation](#)
- [Rerunning an Interrupted Fault Simulation](#)
- [Merging Fault Databases \(FDBs\)](#)
- [Importing IP Simulations](#)
- [Common DP Library Information](#)

Setting Up Fault Simulation

To set up analog fault simulation, perform the following steps.

1. [Setting Up Defect Detection Criteria](#)
2. [Creating a Configuration File](#)
3. [Defining Defect Models](#)

Setting Up Defect Detection Criteria

The defect detection criteria helps to determine if the defect in the circuit is detected. If the defect detection result meets the checking criteria, the defect in the circuit is not detected. Otherwise, it is detected.

You set defect detection criteria through one of the following:

- [Using a Vector Check](#)
- [Using a .MEASURE Statement](#)

- [Using a Circuit Check Command](#)
- [Using Simulation Output Messages](#)

Using a Vector Check

The tool supports many vector formats. See your simulator's user guide for more information. The vector file must contain an output vector for the defect detection check. The defect in the circuit is detected if output vector checking encounters a comparison error.

The PrimeSim XA `set_vector_option` command, FineSim `finesim_vector_errmsg` option, and PrimeSim `primesim_vector_errmsg` option specify to stop the simulation when a comparison error occurs.

You can include the `set_vector_option` command in the PrimeSim XA command file, or in the netlist file using the `xa_cmd` option. The syntax for the `set_vector_option` command is:

```
set_vector_option -error 1

.option xa_cmd="set_vector_option -error 1"
```

You can include the `finesim_vector_errmsg` FineSim command in the netlist file. The syntax for the `finesim_vector_errmsg` option is:

```
.option finesim_vector_errmsg=2
```

You can include the `primesim_vector_errmsg` PrimeSim command in the netlist file. The syntax for the `primesim_vector_errmsg` option is:

```
.option primesim_vector_errmsg=2
```

Note:

When running [Diagnostic Coverage \(DC\)](#), you must either not set the vector option for your simulator or set it to finish the simulation (see first row of [Table 2](#)).

Table 2 Vector Option Settings

PrimeSim XA	FineSim or PrimeSim	Result
No <code>set_vector_option</code> command or <code>set_vector_option -error 0</code>	No <code>finesim_vector_errmsg</code> option or <code>finesim_vector_errmsg=0</code> or No <code>primesim_vector_errmsg</code> option or <code>primesim_vector_errmsg=0</code>	Simulation finishes transient.

Table 2 Vector Option Settings (Continued)

PrimeSim XA	FineSim or PrimeSim	Result
<code>set_vector_option -error 1</code>	<code>finesim_vector_errmsg=2</code> or <code>primesim_vector_errmsg=2</code>	Simulation stops at the first comparison error.

Using a .MEASURE Statement

The `.MEASURE` statement evaluates and reports user-defined functions. See your simulator's user guide for details about how to specify the measure statements. If the measurement result is less than the low threshold value or greater than the high threshold value, the defect in the circuit is detected.

set_sim_stop / .set_sim_stop Syntax for .MEASURE Statement

The PrimeSim XA `set_sim_stop` command and the FineSim or PrimeSim `.set_sim_stop` statement provide options to control and specify the defect detection checking criteria, such as the low threshold value and high threshold value. Multiple commands are supported in one simulation. You apply the command only to the measurements related to checking criteria.

You can include the `set_sim_stop` command in the PrimeSim XA command file, or inside the netlist file using the `xa_cmd` option. The `set_sim_stop` syntax is:

```
set_sim_stop -check meas -name name \
[-min low_threshold] [-max high_threshold] [-stop 1|0] \
[-logic safe|violation] [-id id_name]
```

or

```
.option xa_cmd="set_sim_stop -check meas -name name \
[-min low_threshold] [-max high_threshold] [-stop 1|0]" \
[-logic safe|violation] [-id id_name]
```

You can include the `.set_sim_stop` FineSim or PrimeSim statement in the netlist file. The `.set_sim_stop` syntax is:

```
.set_sim_stop check=meas name=name
+ [min=low_threshold] [max=high_threshold] [stop=1|0]
+ [logic=safe|violation] [id=id_name]
```

Argument	Description
<code>-check meas</code> <code>check=meas</code>	Specifies that you are using a <code>.MEASURE</code> statement to set defect detection criteria.
<code>-name name</code> <code>name=name</code>	This option is mandatory. It checks only the measurement result of the specified measuring statement against the low threshold value and high threshold value specified with <code>-min /min=</code> and <code>-max /max=</code> arguments. Note: In the PrimeSim XA <code>.MEASURE</code> documentation, the measuring statement is identified using the argument, <i>mname</i> . In the FineSim and PrimeSim <code>.MEASURE</code> documentation, the measuring statement is identified using the argument, <i>Result</i> .
<code>-min low_threshold</code> <code>-max high_threshold</code> <code>min=low_threshold</code> <code>max=high_threshold</code>	You must specify either a <code>-min /min=</code> or <code>-max /max=</code> argument. By default (see <code>-logic, logic=</code>), the defect in the circuit is detected if the measurement result is less than <i>low_threshold</i> or larger than <i>high_threshold</i> .
<code>-stop [1 0]</code> <code>stop=[1 0]</code>	Enables on-the-fly error checking to boost the simulation throughput, which you can set to: • 1 (default) to stop simulation if the measurement result exceeds the specified tolerance. • 0 to disable on-the-fly error checking so that each simulation continues until the end of the simulation before error checking.
<code>-logic safe violation</code> <code>logic=[safe violation]</code>	Sets the logic for how the minimum (<code>-min, min=</code>) and maximum (<code>-max, max=</code>) arguments are used. • <i>safe</i> (default) specifies that there is <i>no</i> violation when the measurement results are within the specified minimum and maximum range. • <i>violation</i> specifies that there is a violation when the measurement results are within the specified minimum and maximum range.
<code>-id id_name</code> <code>id=id_name</code>	Allows you to specify a unique ID name for the <code>set_sim_stop</code> statement if you are using multiple statements with the same measurement name and different minimum and maximum thresholds. Note: The <i>id_name</i> is always converted and processed in lowercase. The <code>diagnostic_coverage</code> command also converts and processes the <i>id_name</i> in lowercase.

Using a Circuit Check Command

You can use circuit check commands, such as the PrimeSim XA dynamic CCK commands, `cck_signal` or `cck_soa`, or the FineSim or PrimeSim command, `.CHKEXPR`, to define defect detection criteria. See your simulator's user guide for details about how to specify the circuit check commands.

set_sim_stop / .set_sim_stop Syntax for Circuit Check Command

The PrimeSim XA `set_sim_stop` command and FineSim or PrimeSim `.set_sim_stop` statement provide options to control and specify the defect detection checking criteria. Multiple commands are supported in one simulation. You apply the command only to the circuit checks related to checking criteria.

You can include the `set_sim_stop` command in the PrimeSim XA command file, or inside the netlist file using the `xa_cmd` option. The `set_sim_stop` syntax is:

```
set_sim_stop -check check -name name [-stop 1|0]
```

or

```
.option xa_cmd="set_sim_stop -check check -name name [-stop 1|0]"
```

You can include the `.set_sim_stop` FineSim or PrimeSim statement in the netlist file. The `.set_sim_stop` syntax is:

```
.set_sim_stop check=check name=name [stop=1|0]
```

Argument	Description
<code>-check <i>check</i></code>	Specify one of the following options: <code>cck_signal</code> – Specifies that you are using the PrimeSim XA dynamic CircuitCheck (CCK) command, <code>cck_signal</code>, to set defect detection criteria. <code>cck_soa</code> – Specifies that you are using the PrimeSim XA dynamic CircuitCheck (CCK) command, <code>cck_soa</code>, to set defect detection criteria. <code>chk</code> – Specifies that you are using the FineSim or PrimeSim circuit check command, <code>.CHKEXPR</code>, to set defect detection criteria.
<code>check=<i>check</i></code>	
<code>-name <i>name</i></code>	This option is mandatory. It checks only the result of the specified circuit check command.
<code>name=<i>name</i></code>	Note: For the PrimeSim XA tool, the circuit check command is identified using the argument, <code>-label <i>lname</i></code> . For the FineSim or PrimeSim tools, the circuit check command is identified using the argument, <code>tag=<i>tagname</i></code> .

Argument	Description
<code>-stop [1 0]</code>	Enables on-the-fly error checking to boost the simulation throughput, which you can set to:
<code>stop=[1 0]</code>	• 1 (default) to stop simulation if the circuit check command exceeds the specified tolerance. • 0 to disable on-the-fly error checking so that each simulation continues until the end of the simulation before error checking.

Using Simulation Output Messages

In some cases, defect detection can be achieved through automatic checkers embedded into the testbench and simulation. This is a common methodology in mixed-signal cosimulation. For example, a Verilog-A model or the Verilog testbench can use a `$display()` statement to print user-defined messages to the simulation output.

You can configure the Custom Fault tool to scan the STDOUT and STDERR streams for specific messages. Furthermore, you can extract numeric values from these messages and compare them against user-defined thresholds. Such detected messages can be used to treat the effect of a defect as being observed and detected.

See the `detection_message` argument of the `fault_sim_output` command for details on using this feature.

Creating a Configuration File

The configuration file contains a list of commands to run analog fault simulation. The format of the configuration file is based the JavaScript Object Notation (JSON) format. Here are the basic JSON syntax rules:

- Commands are represented in name/value pairs, separated by a comma.
- A name/value pair consists of a field name (in double quotation marks), followed by a colon, followed by a value.
- A value must be one of the following data types: a string, a number, an object, an array, a Boolean (`true` or `false`), or `null`.
- Curly braces (`{ }`) hold objects, and each name is followed by colon.
- Square brackets (`[ ]`) hold arrays.

The syntax of the configuration file is:

```
{
    command1,
    command2,
```

```
...
    commandn,
}

```

You can add comments in the configuration file. To add single line comments, use double slash characters (`//`). Any text written to the right side of the double slash is considered a comment and is ignored during a simulation. You can add multi-line comments by placing `/*` at the beginning of the comments. You use the character `*/` to end the multi-line comments.

For example:

```
{
  "cpulimit" : 3600,
  "fault_list_file" : "manual_fault_list_file",
  "case_dir" : "./",
  "outdir" : "./FAULT_RESULT_RUNDIR/",
  "dpconfig_file" : "./xhost/xhost_sge_launch",
  "identify_only" : false,
  "cleanup_level" : 0,
  "analog_setup" : {
    "sim_cmdline" : "xa input1.sp -c xa.cfg"
  },
  "faults" : { // This is a single line comment.
    "defaults" : {
      "short" : {
        "res" : 0.01,
        "cap" : null
      },
      "open" : {
        "res" : 1e6,
        "cap" : null
      }
    }
  }
}
/* This is a
multi-line comment */
}

```

For information about the configuration file commands, see [Configuration File Command Reference](#).

Defining Defect Models

The Custom Fault tool supports several built-in defect models for many kinds of SPICE primitives. These built-in defect models, except for subcircuit short/open defect, are applied by default unless some user-defined defect models are defined; see [Supported Built-in Defect Models](#) for more information.

Most modern PDKs do not use SPICE primitives to define the actual design devices. Real design devices are usually modeled as macro devices defined as SPICE subcircuits within a technology PDK. These PDK devices usually require user-defined defect models to ensure that defects are injected at the appropriate locations within the circuit. You can define these custom defect models within the configuration file; see [Defining and Scoping Defect Models](#) for more information.

Running Fault Simulation

You run the tool with the following command at the UNIX command prompt:

```
$> customfault -c fault_config_file
[--|--ignore_rest]
[--dp #sims]
[-f|--fdb fault_list_file]
[-h|--help]
[-i|--incremental]
[-k|--keys #keys]
[-n|--nominal no|only|yes]
[--nca]
[-o|--out outdir/outfile]
[--primesim 0|1|2]
[--primesim_order 0|1]
[-r|--resume -f fdb_file_name]
[-s|--size #sample]
[-t|--tag TagName]
[--version]
[-w|--webhelp]
```

Command-Line Options

The following table describes the Custom Fault command-line options.

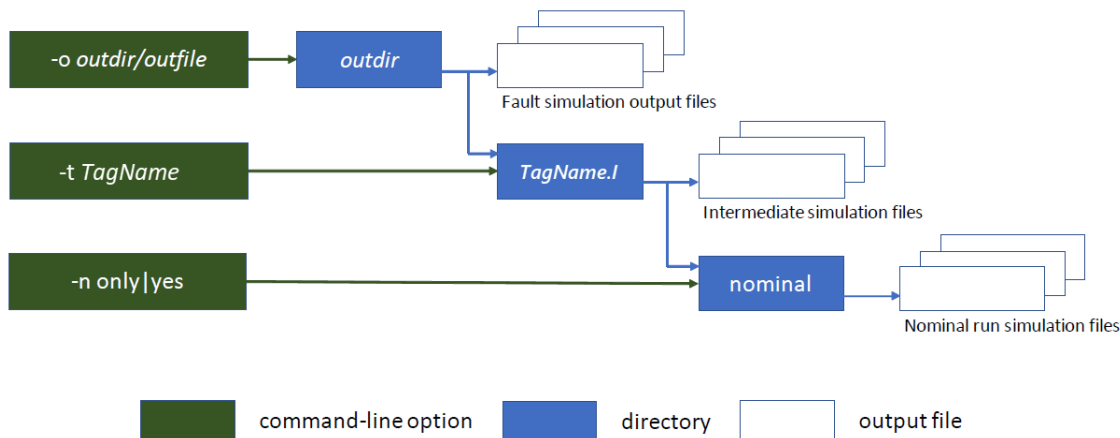
Command-Line Option	Description
<code>-c fault_config_file</code>	Specifies the name of configuration file that contains a list of commands to execute an analog fault simulation. See Creating a Configuration File for more information.
<code>-- --ignore_rest</code>	Ignores all the command-line arguments that follow this one.
<code>--dp #sims</code>	Sets the maximum number of parallel fault simulations that are run. It cannot be used at the same time as the <code>-k --keys</code> option. Note: This option can only be used in PrimeSim Reliability Analysis token license mode.

Command-Line Option	Description
<code>-f --fdb <i>fault_list_file</i></code>	Specifies the user-defined defect list file name or the fault database file name. See Overview of the Manual Defect Injection Flow and Overview of the Multi-Testbench Flow for details about the <code>fault_list_file</code> command.
<code>-h --help</code>	Prints usage information.
<code>-i --incremental</code>	Enables the incremental sampling feature. See Using Incremental Sampling for more information.
<code>-k --keys <i>#keys</i></code>	Specifies the number of Custom Fault keys to use. The maximum number of parallel fault simulations is four times the number of keys requested. It cannot be used at the same time as the <code>--dp</code> option.
<code>-n --nominal <i>no only yes</i></code>	Enables a nominal (defect-free) run. You can enable a nominal run to check the setup before you run a fault simulation. You can set it to: <code>no</code>: (Default) Disables a nominal run. <code>only</code>: Enables a nominal run and exits after the nominal run completes. <code>yes</code>: Enables a nominal run. If the nominal run passes, it continues to fault simulation. Otherwise, it exits after the nominal run. When a nominal run is enabled, a directory named <code>nominal</code> is created under the <code>TagName.I</code> directory, which contains the simulation output files of the nominal run. See Figure 2 .
<code>--nca</code>	Runs a Not-covered Defect Analysis which produces <code>.nc_analysis.dat</code> and <code>.nc_analysis.csv</code> files in the output directory. Note: This command-line option must be used with the <code>-f --fdb</code> option to provide a fault database of a completed fault campaign.

Command-Line Option	Description
<code>-o --out outdir/outfile</code>	Specifies the name of the output directory (see Figure 2) and output files. The default for <code>outdir</code> is the current working directory and the default for <code>outfile</code> is <code>cf</code>. Note: If the specified output files already exist, the Custom Fault tool stops and reports an error. You can either specify a different output name or delete the existing output files. The output files are: • <code>*.fdb</code> is the fault database file that contains the defect information and simulation results. • <code>*.log</code> is the analog simulation log file. • <code>*.report</code> is the analog simulation report file. See Test Coverage Report (Single Testbench) for more information.
<code>--primesim 0 1 2</code>	Specifies the license mode as follows: • 0: Enables Custom Fault standalone license mode. • 1: Enables PrimeSim Reliability Analysis token license mode. • 2: (Default) Enables standalone and token-based license mode. By default, the standalone keys are attempted first (see command-line option <code>--primesim_order</code>). This command-line setting overrides the <code>PRIMESIM</code> shell environment variable setting. See Configuring PrimeSim Reliability Analysis Token License Mode for more information.
<code>--primesim_order 0 1</code>	Specifies whether standalone keys or token-based keys are attempted first when the <code>--primesim</code> option is set to 2. • 0: (Default) Standalone keys are attempted first. • 1: Token-based keys are attempted first. This command-line setting overrides the <code>PRIMESIM_ORDER</code> shell environment variable setting. See Configuring PrimeSim Reliability Analysis Token License Mode for more information.
<code>-r --resume -f fdb_file_name</code>	Resume an incomplete fault simulation run. See Rerunning an Interrupted Fault Simulation for more information.
<code>-s --size #sample</code>	Specifies the sample size. See Using Adaptive Weighted Random Sampling (AWRS) for more information.

Command-Line Option	Description
<code>-t --tag TagName</code>	Specifies the tag name of the simulation. This argument is mandatory. It is important to specify a representative tag name that shows the corner and the temperature for each simulation. The tag name can then be used to identify the simulation and help to extract the information from the fault database file. A directory <code>TagName.I</code>, where <i>I</i> is an incremental integer starting at 1, is created sequentially under the output directory specified by <code>outdir</code> that contains the intermediate files and simulation results of all fault simulations (see Figure 2). You can use the <code>cleanup_level</code> command to choose the files to keep.
<code>--version</code>	Prints the version information.
<code>-w --webhelp</code>	Opens a full online help collection in a web browser. Click “Using Help” in the navigation menu of the Help window for details about how to use Help features such as the Help icons, Search, and Feedback.

Figure 2 Simulation Output Files



Rerunning an Interrupted Fault Simulation

When a fault simulation is interrupted for any reason (`kill` command, machine crash or reset, and so on), you can resume the simulations if and only if they could not complete in the original run. You can use the `-r` or `--resume` command-line option to resume a simulation.

To resume the interrupted simulation, you also must use:

- The name of the FDB file from the interrupted simulation, with the `-f` command-line option.
- The same configuration files used to run the interrupted simulation, with the `-c` command-line option.
- The `-t` command option with the exact same tag value as in the interrupted run.

Note that you need also to specify the number of licenses you want to use.

If the interrupted run was using sampling with the `-s` command-line option, it is not needed again because the sampling information is present in the FDB file.

You can change the number of licenses to use when resuming a simulation (with the `-k` command-line option). If you do not specify the `-k` option, the tool uses one license. It is also recommended to specify the `-t` option with the tag name you want to resume.

You can specify a new output directory when resuming a simulation. If you do not specify the `-o` option, the output results get the `cfs` prefix and are generated in the working directory.

Note:

If you change the original configuration file, the Custom Fault tool might or might not be able to resume. When you specify the `-r|--resume` option, the `-s` option is ignored (the sampling number should be available in the FDB file).

In addition, the `cleanup_level` command must be set to 1 (default) or 0. If `cleanup_level` is set to 2, the fault campaign may not resume properly.

Single Run Example

Suppose that you started the following Custom Fault simulation:

```
customfault -c config.json -o AF/xa -t fault
```

If the simulation stopped accidentally, the command line to resume the simulation is:

```
customfault -r -c config.json -f AF/xa.fdb -t fault
```

Multi-Testbench Flow Example

Suppose that you started the following Custom Fault multi-testbench simulation:

```
customfault -c config_base.json tb1.json -o AF/tb1 -t tb1
customfault -c config_base.json tb2.json -o AF/tb2 -t tb2 -f AF/tb1.fdb
customfault -c config_base.json tb3.json -o AF/tb3 -t tb3 -f AF/tb2.fdb
```

```
customfault -c config_base.json last.json -o AF/last -t last -f
AF/tb3.fdb
```

If the simulation stopped accidentally while the `tb3` testbench was running, the command lines to resume the simulation are:

```
customfault --resume -c config.json tb3.json -f AF/tb3.fdb -t tb3
customfault -c config_base.json last.json -o AF/last -t last -f
AF/tb3.fdb
```

If the simulation stopped accidentally while the `tb2` testbench was running, the command lines to resume the simulation are:

```
customfault --resume -c config.json tb2.json -f AF/tb2.fdb -t tb2
customfault -c config_base.json tb3.json -o AF/tb3 -t tb3 -f AF/tb2.fdb
customfault -c config_base.json last.json -o AF/last -t last -f
AF/tb3.fdb
```

Merging Fault Databases (FDBs)

To get faster throughput for analog fault simulation, you can replace a SPICE netlist for a functional block in a circuit with Verilog-D as described in [Methodology for Merging FDBs](#). This means that fault simulation is run only for the SPICE netlists. You replace the rest of netlists with Verilog-D to accelerate the simulation.

Methodology for Merging FDBs

This flow allows you to merge the individual FDBs into one target FDB and generates the report file.

Figure 3 Merging FDBs



Note that:

- The FDB merge feature changes the internal information to create the report file, so the detail coverage part in report file may not match.
- The merge does not check duplicated defects, so you must use the proper scope feature for common circuit defect injection.

In the following example the first unit has 48.16% coverage with 380 defects. The second unit has 35.92% coverage with 760 defects.

First Unit:

Type	merge_test
mos_open	34/164
mos_short	149/216
TOTAL	183/380

Weighted test coverage:
merge_test 48.16%

Second Unit:

Type	merge_test
mos_open	45/328
mos_short	228/432
TOTAL	273/760

Weighted test coverage:
merge_test 35.92%

Merge report:

Type	merge_test
mos_open	79/492
mos_short	377/648
TOTAL	456/1140

Weighted test coverage:
merge_test 40.00%

Merged FDB Command Syntax

You specify the `customfault` command on the command line.

```
customfault -m[|--merge] fdb_file[|list_file]
[-m[|--merge] fdb_file|list_file] -o[|--output] output_file
```

Argument	Description
-m[--merge] fdb_file[list_file]	Specifies an FDB file to merge or a list file that contains multiple FDBs to merge. The FDBs in the list file must have the correct paths.
-o[--output] output_file	Specifies the output report file for the merged FDB. The default prefix of <code>cfs</code> is used in the working directory.

Merged FDB Examples

Merges the `fdb1`, `fdb2`, and `fdb3` FDBs and regenerates a merged FDB. The report file name is `aaa.report`.

```
customfault -m fdb1 -m fdb2 -m fdb3 -o aaa.report
```

Merges the `fdb1`, `fdb2`, and `fdb3` FDBs and regenerates a merged FDB. The report file name is `aaa.report`. The list file contains `fdb1`, `fdb2`, and `fdb3`.

```
customfault -m file_list.txt -o aaa.report
```

Importing IP Simulations

Many designs consist of IP blocks provided from third-party vendors or other internal design groups. For these designs it is important to remove the duplicated effort to re-simulate instances of previously simulated instances.

The Custom Fault tool allows you to import existing results and skip the fault simulation that is already covered. The tool simulates for uncovered defects and skips simulation for covered defects. The output report includes the imported IP results.

The Custom Fault tool supports multiple instances and multiple fault databases (FDBs) in one simulation.

Note:

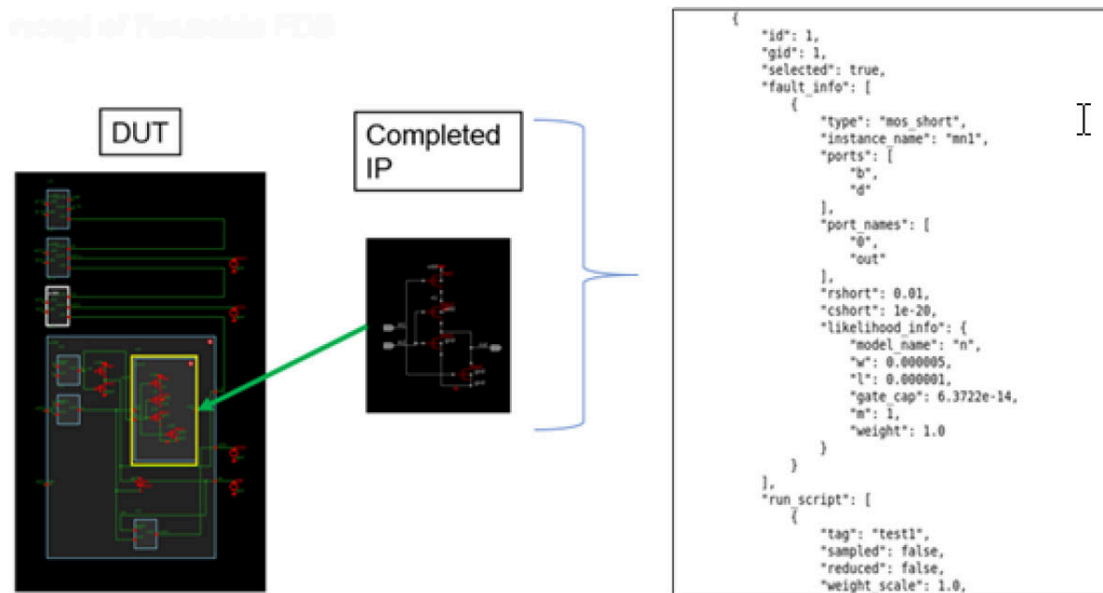
Diagnostic coverage and sampling coverage are not supported.

IP Import Methodology

By importing an FDB file, the Custom Fault tool automatically changes the defect information for new defect universe. For example, the Fault ID, hierarchy information, and results of fault campaign are automatically modified to get the correct report file.

The following figure illustrates the FDB reuse concept.

Figure 4 Reusing FDBs



IP Import Command Syntax

The syntax to import IP in a Custom Fault simulation is:

```

"fault_ip_import": [{
  "instance": ["ins1", "ins2",... "insN"],
  "fault_db_file": "fault_list.fdb"
}]

```

"instance" specifies the imported IP instance to include in the Custom Fault simulation. You must provide the correct hierarchy.

"fault_db_file" specifies a list file that contain the IP Custom Fault FDBs to import. You must provide the FDB list file.

You can define multiple FDBs for different instances:

```

"fault_ip_import": [{
  "instance": ["ins1", "ins2",... "insN"],
  "fault_db_file": "fault_list.fdb"
},
{
  "instance": ["ins_z1", "ins_z2",... "ins_zN"],
  "fault_db_file": "fault_list_z.fdb"
}]

```

Output Report File

Coverage for the imported IP is shown in the report file. For example:

Figure 5 Sample Output Report File

***COVERAGE SUMMARY**

Defects covered - unweighted (covered/simulated):

Type	ip_nor	top
mos_open	24/32	35/632
mos_short	28/42	178/836
TOTAL	52/74	213/1468

Defects covered - weighted (covered/simulated):

Type	ip_nor	top
mos_open	24/32	35/632
mos_short	28/42	178/836
TOTAL	52/74	213/1468

Weighted test coverage:

ip_nor	70.27%	Imported IP Coverage
top	14.51%	Top block + non-covered retest Coverage

TOTAL Coverage:

Non-Sampled	17.43%	Total Coverage
-------------	--------	----------------

***DETAILED COVERAGE LISTING**

Id	Defect	ip_nor	top	Insta
A1.1	mos_short	NS	CU	x1.x1
A5.5	mos_short	NS	NC	x1.x3
A9.9	mos_short	NS	NC	x1.x5.mp1
A13.13	mos_short	NS	CU	x1.x7.mp1
A15.15	mos_short	NS	NC	x1.x5.mn1
A1446.1446	mos_open	NS	NC	x4.x9.mn1
A1447.1447	mos_short	NC	NC	x1.x8.mn1
A1449.1449	mos_short	NC	NC	x1.x8.mn2
A1451.1451	mos_short	CL	NS	x1.x8.mn2
A1455.1455	mos_short	CU	NS	x1.x8.mp2
A1456.1456	mos_short	CL	NS	x1.x8.mp2
A1458.1458	mos_short	CL	NS	x1.x8.mp2
A1459.1459	mos_short	CL	NS	x1.x8.mn1
A1480.1480	mos_open	CU	NS	x1.x8.mn1
A1481.1481	mos_open	CL	NS	x1.x8.mn1
A1482.1482	mos_open	CU	NS	x1.x8.mn1
A1483.1483	mos_open	NC	NC	x1.x8.mn1
A1484.1484	mos_short	NC	CU	x1.x9.mn1
A1486.1486	mos_short	NC	CU	x1.x9.mn2
A1488.1488	mos_short	CL	NS	x1.x9.mn2
A1492.1492	mos_short	CU	NS	x1.x9.mp2

Non-covered retested in top block

Covered skipped in top block simulation

Non-covered retested, result updated

IP Import Examples

Imports "x1.x8" and "x1.x9", which are NOR Instances and identical, so they share same FDB data.

```
"fault_ip_import": [{
  "instance": ["x1.x8", "x1.x9"],
  "fault_db_file": "/slowfs/amscae19/CustomFault_DATA/Panasonic/af-346-sample/nor/nor.fdb"
}],
```

Imports multiple instances with multiple FDBs. This is the typical usage for this feature.

```
"fault_ip_import": [{
  "instance": ["x1.x8", "x1.x9"],
  "fault_db_file": "/slowfs/amscae19/CustomFault_DATA/Panasonic/af-346-sample/nor/nor.fdb"
},
{
  "instance": ["x2.x1", "x2.x2"],
  "fault_db_file": "/slowfs/amscae19/CustomFault_DATA/Panasonic/af-346-sample/xor/xor.fdb"
}],
```

Note:

You must provide instances for each FDB. You cannot define multiple FDBs for a single instance.

Common DP Library Information

Common Distributed Processing Library (CDPL) is an internal library used by Synopsys products to enable distributed parallel processing of compute and data intensive applications. It provides a uniform "Master-Worker" model across multiple distributed process launch mechanisms available under the UNIX environment, and provides features that enable applications to offload the distributed processing functionality to the CDPL framework. This enables applications to run on multiple platforms with identical distributed processing functionality.

The Hosts Information file specifies information about the machines (and infrastructure). This information is needed for setting up the way workers are launched, as well as the capacity of workers available for the distributed run.

The following are some examples of specifying the Hosts Information file.

- Using the Host Information file to specify "SGE" runs:

```
# HOSTS INFORMATION FILE (SGE)
# Each valid host definition occupies an entire line.
# Line with '#' as the first character is a comment line.
# Format of each line:
# flag| hostname| slots|tmpDir| protocol| command
# The end-user environment should be properly configured and capable of
# starting SGE jobs
1| |3|/remote/users/tmp|SGE| qsub -P bnormal -l arch=glinux
```

- Using the Host Information File to specify "LSF" runs:
 - # HOSTS INFORMATION FILE (LSF)
 - # Each valid host definition occupies an entire line.
 - # Line with '#' as the first character is a comment line.
 - # Format of each line:
flag| hostname| slots|tmpDir| protocol| command
 - # The end-user environment should be properly configured and capable of
starting LSF jobs
 - 1| **-1**|/remote/users/tmp|LSF| bsub -P bnormal

3

Defining and Scoping Defect Models

This chapter describes how to define defect models and apply the defect models to the circuit.

- [Supported Built-in Defect Models](#)
- [Device Instance Defect Model Diagrams](#)
- [Customizing User-defined Defect Models](#)
- [Resistor Divider Gate Open Defect Model](#)
- [Pin Open and Net Open Defect Models](#)
- [Instance Parameter Variation Defect Models](#)
- [Defining Custom SPICE Defect Models](#)
- [Applying Built-in and Customized Defect Models to a Circuit](#)

Supported Built-in Defect Models

The Custom Fault tool supports many built-in defect models. [Table 3](#) shows the supported keywords and the built-in defect models. The properties of the built-in defect models cannot be overridden.

Table 3 *Keywords and Built-In Defect Models*

Keyword	Built-In Defect Model Type	<code>inject_fault</code> Command Syntax
<code>mos_open</code>	MOSFET open defect model Sample diagram: Figure 6	Device Instance Open Defect
<code>mos_short</code>	MOSFET short defect model Sample diagram: Figure 10	Device Instance Short Defect
<code>jfet_open</code>	JFET open defect model	Device Instance Open Defect
<code>jfet_short</code>	JFET short defect model	Device Instance Open Defect

Table 3 *Keywords and Built-In Defect Models (Continued)*

Keyword	Built-In Defect Model Type	<code>inject_fault</code> Command Syntax
<code>bjt_open</code>	BJT open defect model	Device Instance Open Defect
<code>bjt_short</code>	BJT short defect model	Device Instance Open Defect
<code>diode_open</code>	Diode open defect model Sample diagram: Figure 7	Device Instance Open Defect
<code>diode_short</code>	Diode short defect model Sample diagram: Figure 11	Device Instance Open Defect
<code>res_open</code>	Resistor open defect model Sample diagram: Figure 8	Device Instance Open Defect
<code>res_short</code>	Resistor short defect model Sample diagram: Figure 12	Device Instance Open Defect
<code>cap_open</code>	Capacitor open defect model Sample diagram: Figure 9	Device Instance Open Defect
<code>cap_short</code>	Capacitor short defect model Sample diagram: Figure 13	Device Instance Open Defect
<code>ind_open</code>	Inductor open defect model Sample diagram: Figure 7	Device Instance Open Defect
<code>ind_short</code>	Inductor short defect model Sample diagram: Figure 11	Device Instance Open Defect
<code>bhv_res_open</code>	Behavioral resistor open defect model	Behavioral Resistor Open Defect
<code>bhv_res_short</code>	Behavioral resistor short defect model	Behavioral Resistor Short Defect
<code>bhv_cap_open</code>	Behavioral capacitor open defect model	Behavioral Capacitor Open Defect
<code>bhv_cap_short</code>	Behavioral capacitor short defect model	Behavioral Capacitor Short Defect
<code>subckt_open</code>	Subcircuit open defect model	Subcircuit Open Defect
<code>subckt_short</code>	Subcircuit short defect model	Subcircuit Short Defect
<code>net_to_net_short</code>	Net-to-net short defect model	Net to Net Short Defect

Device Instance Defect Model Diagrams

This section includes sample diagrams of the following [Supported Built-in Defect Models](#).

Figure 6 Source Open Defect Modeled With Capacitor and Resistor on MOS Device

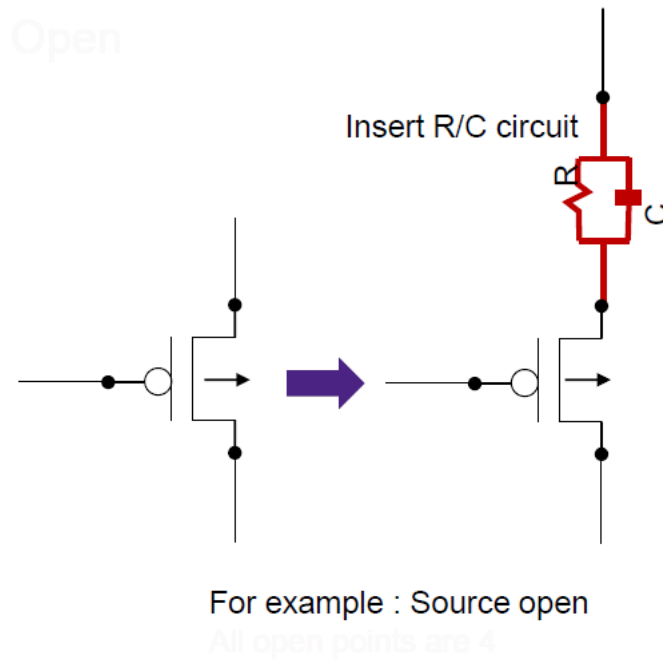


Figure 7 Diode/Inductor Open Defect Modeled With Resistor and Capacitor

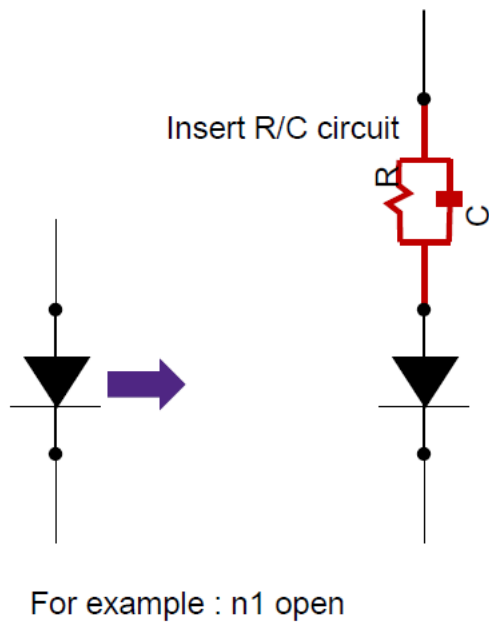


Figure 8 Resistor Open Defect Modeled With Resistor and Capacitor

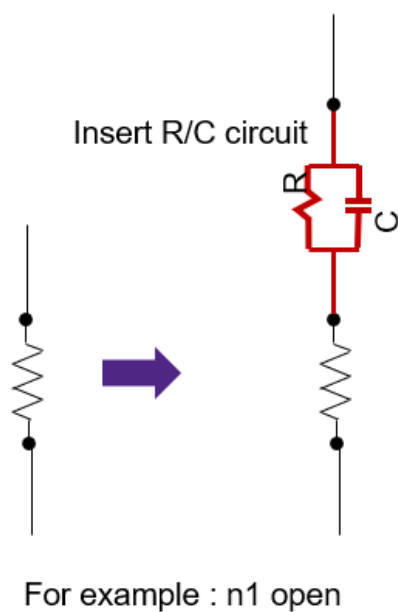


Figure 9 Capacitor Open Defect Modeled With Resistor and Capacitor

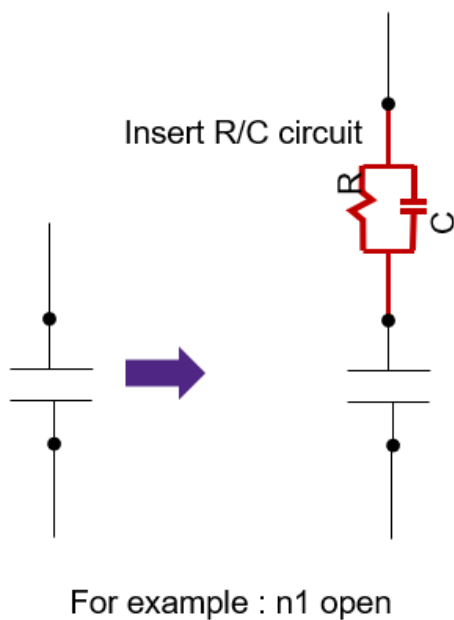
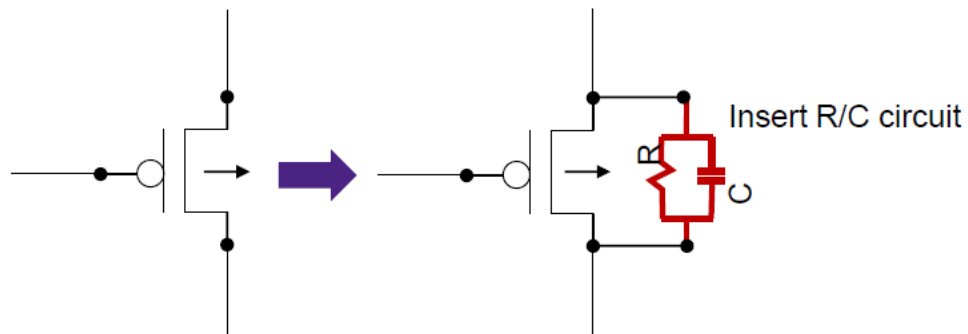


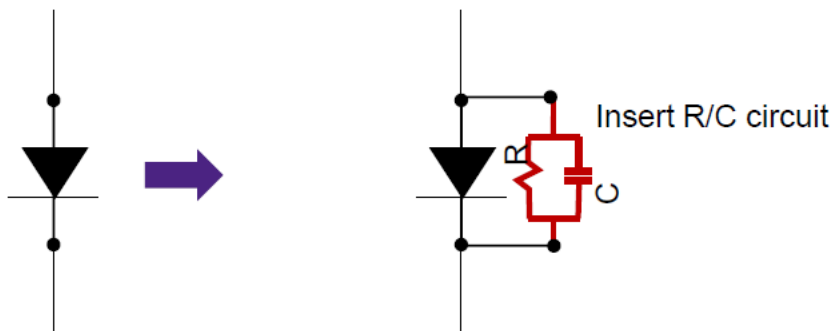
Figure 10 Source-Drain Short Defect Modeled With Resistor and Capacitor



For example : Drain Source short

All short combinations are 6

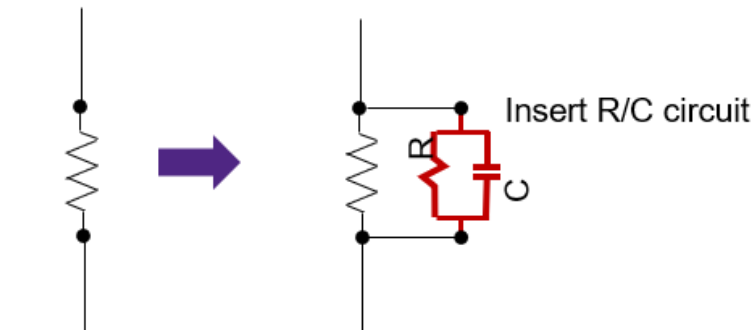
Figure 11 Diode/Inductor Short Defect Modeled With Resistor and Capacitor



For example : n1 n2 short

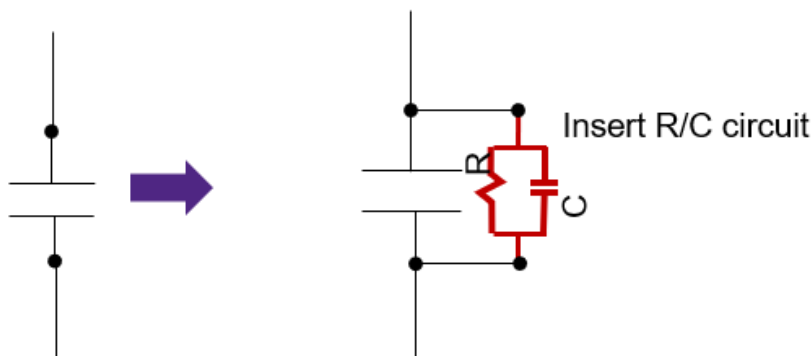
All short combinations are 1

Figure 12 Resistor Short Defect Modeled With Resistor and Capacitor



For example : n1 n2 short

Figure 13 Capacitor Short Defect Modeled With Resistor and Capacitor



For example : n1 n2 short

Customizing User-defined Defect Models

The Custom Fault tool allows you to customize the defect models. The customized defect models must be based on the defect models supported by the tool. You can customize the defect model in the fault definition block using the `faults` command.

The fault definition block allows you to:

- Set the resistor value and the capacitor value modeling the short defect and open defect.
- Define custom defect models. These customized models can be applied to a specific scope in the circuit using `apply_fault_model` command. Customizations include:
 - User-defined defect model names
 - Resistor and capacitor values modeling the short and open defects
 - Defining a macromodel
 - Defining a generic subcircuit defect model
 - Customizing the list of shorted and opened terminals or ports

The following example defines:

- Resistor modeling short defect is 200 ohms
- Resistor modeling open defect is 1e10 ohms
- The `mos_stuck_on` as a customized `mos_short` defect model that applies only to drain and source terminals.
- The `mos_stuck_off` as a customized `mos_short` defect model that applies only to gate and bulk terminals.

```
"faults" : {
  "defaults" : {
    "short" : {
      "res" : 200,
      "cap" : null
    },
    "open" : {
      "res" : 1e10,
      "cap" : null
    }
  },
  "models" : [ {
    "name" : "mos_stuck_on",
    "base_model" : "mos_short",
    "terminals" : ["d","s"]
  },
  {
    "name" : "mos_stuck_off",
    "base_model" : "mos_short",
    "terminals" : ["g","b"]
  } ]
}
```

The following example defines:

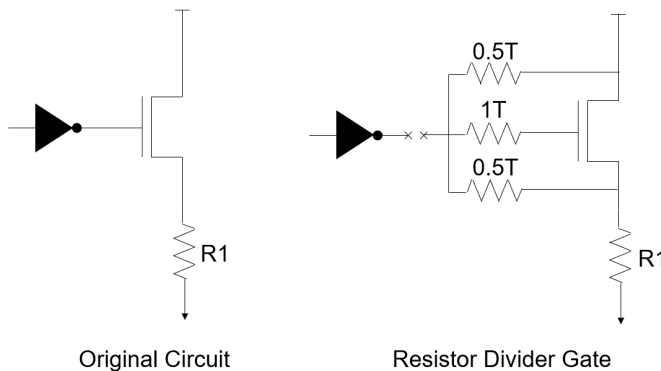
- Resistor modeling short defect is 200 ohms
- Resistor modeling open defect is 1e10 ohms
- The `macromos_nch_stuck_on` as a customized `subckt_short` defect model that applies to the subcircuit named `nch` and only on the `d` and `s` ports of the subcircuit.
- The `macromos_nch_stuck_off` as a customized `subckt_short` defect model that applies to the subcircuit named `nch` and only on the `g` and `b` ports of the subcircuit.

```
"faults" : {
  "defaults" : {
    "short" : {
      "res" : 200,
      "cap" : null
    },
    "open" : {
      "res" : 1e10,
      "cap" : null
    }
  },
  "models" : [ {
    "name" : "macromos_nch_stuck_on",
    "base_model" : "subckt_short",
    "subckt" : ["nch" ],
    "terminals" : ["d","s"]
  },
  {
    "name" : "macromos_nch_stuck_off",
    "base_model" : "subckt_short",
    "subckt" : ["nch"],
    "terminals" : ["g","b"]
  } ]
}
```

Resistor Divider Gate Open Defect Model

The Custom Fault tool supports a special resistor divider gate open defect model. This defect model completely disconnects the gate node of a MOSFET, but instead of leaving the gate node floating, it is biased with an automatically inserted resistor divider circuit. The resistor values in this model are fixed and cannot be adjusted. This model can also be applied to subcircuit macro models. This defect model is not applied by default. It can only be applied as a user-defined customized defect model.

Figure 14 Resistor Divider Gate Open Defect Model



The main alternatives for the MOSFET gate open model are as follows:

- **Large R** – This defect can be achieved with the built-in open defect model or a customized open defect model with a large resistance, 1e9 ohms or larger. The gate voltage still changes with the input signal if the gate capacitance is sufficiently small. The gate is already a high impedance node, so a large resistance defect model has minimal impact.
- **Disconnect** – This defect can be achieved with the built-in open defect model or a customized open defect model with the resistance value set to null. It models a real physical disconnect where the gate floats without a direct DC bias. The simulated circuit behavior is dependent on the MOS model accuracy and the DC operating point solution of the floating gate node.
- **Resistor Divider** – Provides a gate voltage bias that is a function of the MOSFET drain and source voltages. The defect free gate input signal is decoupled from the gate. Relative to the disconnect model, numerical convergence should be better and consistent across various simulators. This model is recommended when the MOSFET does not have significant gate leakage currents.

Resistor Divider Syntax

The resistor divider model can be specified for a primitive MOSFET by creating a user-defined defect model and adding "gate_rdiv" : true.

```
"faults" : {
...
  "models" : [
    {
      "name" : "user_defined_model_name",
      "base_model" : "mos_open",
      "gate_rdiv" : true
    }
  ]
}
```

```
]
}
```

The resistor divider model can also be applied to MOSFET subcircuit macro models by creating a user-defined defect model and adding "gate_rdiv" : true and then specifying the gate, drain, and source ports of the subcircuit. The gate drain and source ports can be specified by name or by index.

```
"faults" : {
...
  "models" : [
    {
      // by port name
      "name" : "user_defined_name1",
      "base_model" : "subckt_open",
      "subckt" : ["subckt_name"],
      "gate_rdiv" : true,
      "port_name" : {
        "drain" : "d",
        "gate" : "g",
        "source" : "s"
      }
    },
    {
      // by port index
      "name" : "user_defined_name2",
      "base_model" : "subckt_open",
      "subckt" : ["nch_mac"],
      "gate_rdiv" : true,
      "port_index" : {
        "drain" : 1,
        "gate" : 2,
        "source" : 3
      }
    }
  ]
}
```

Resistor Divider Examples

Example 1: Primitive MOSFETs

Netlist:

```
mn1 a b c bulk nch w=20n l=10n
```

Fault definition:

```
"faults" : {
  "models" : [
    {
      "name" : "gate_rdiv",
```

Chapter 3: Defining and Scoping Defect Models

Resistor Divider Gate Open Defect Model

```

        "base_model" : "mos_open",
        "gate_rdiv" : true
    }
]
}

```

Example 2: Macro MOSFET by Port Name

Netlist:

```

.subckt nch_mac d c b a w=20n l=10n
main a b c d nch w=w l=l
.model nch nmos level=54
.ends

```

Fault definition:

```

"faults" : {
  "models" : [
    {
      "name" : "nch_mac_rdiv",
      "base_model" : "subckt_open",
      "subckt" : ["nch_mac"],
      "gate_rdiv" : true,
      "port_name" : {
        "drain" : "a",
        "gate" : "b",
        "source" : "c"
      }
    }
  ]
}

```

Example 3: MOSFET by Port Index

Netlist:

```

.subckt nch_mac d c b a w=20n l=10n
main a b c d nch w=w l=l
.model nch nmos level=54
.ends

```

Fault definition:

```

"faults" : {
  "models" : [
    {
      "name" : "nch_mac_rdiv",
      "base_model" : "subckt_open",
      "subckt" : ["nch_mac"],
      "gate_rdiv" : true,
      "port_index" : {
        "drain" : 4,
        "gate" : 3,

```

```
        "source" : 2
    }
}
]
```

Pin Open and Net Open Defect Models

The Custom Fault tool supports both pin open and net open defect models.

[Table 4](#) lists the supported keywords and defect models.

Table 4 *Keywords and Defect Models*

Keyword	Defect Model Type
bhv_cap_pinopen	Parallel behavioral capacitor pin open defect model.
bhv_cap_netopen	Parallel behavioral capacitor net open defect model.
bhv_res_pinopen	Parallel behavioral resistor pin open defect model.
bhv_res_netopen	Parallel behavioral resistor net open defect model.
bjt_pinopen	Parallel BJT pin open defect model.
bjt_netopen	Parallel BJT net open defect model.
cap_pinopen	Parallel capacitor pin open defect model.
cap_netopen	Parallel capacitor net open defect model. Sample diagram: Figure 15
diode_pinopen	Parallel diode pin open defect model.
diode_netopen	Parallel diode net open defect model.
ind_pinopen	Parallel inductor pin open defect model.
ind_netopen	Parallel inductor net open defect model.
jfet_pinopen	Parallel JFET pin open defect model.
jfet_netopen	Parallel JFET net open defect model.
mos_pinopen	Parallel MOSFET pin open defect model. Sample diagram: Figure 16
mos_netopen	Parallel MOSFET net open defect model. Sample diagram: Figure 17
res_pinopen	Parallel resistor pin open defect model.
res_netopen	Parallel resistor net open defect model.

Table 4 *Keywords and Defect Models (Continued)*

Keyword	Defect Model Type
subckt_pinopen	Parallel subcircuit pin open defect model. Sample diagram: Figure 18
subckt_netopen	Parallel subcircuit net open defect model. Sample diagram: Figure 19

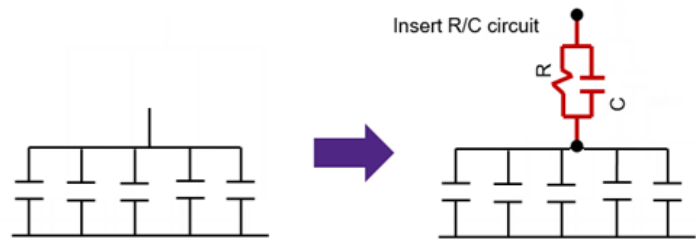
The device open net defect model is described further in the following sections:

- [Pin Open and Net Open Defect Model Sample Diagrams](#)
- [Pin Open and Net Open Defect Model Syntax](#)
- [Pin Open and Net Open Defect Model Examples](#)

Pin Open and Net Open Defect Model Sample Diagrams

[Figure 15](#) shows a `cap_netopen` defect model on parallel capacitor devices. A defect (defaults for R and C are null) is injected to open the connection of a net of a group of parallel capacitors. The default weight of the defect is equal to 1. Weight is adjustable using the `defect_weight` command.

Figure 15 *cap_netopen on Terminal of Capacitor Devices*



[Figure 16](#) shows a `mos_pinopen` defect model on the drain terminal of a MOSFET. A defect is injected to open a connection of a pin of a group of parallel MOSFETs. The

default weight of the defect is equal to the number of parallel MOSFETs. Weight is adjustable using the `defect_weight` command.

Figure 16 *mos_pinopen Defect Model on Drain Terminal of MOS Devices*

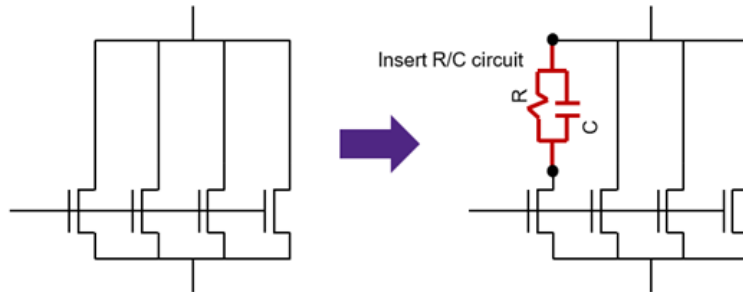


Figure 17 shows a `mos_netopen` defect model on the drain terminal of a MOSFET. A defect is injected to open the connection of a net of a group of parallel MOSFETs. The default weight of the defect is equal to 1. Weight is adjustable using the `defect_weight` command.

Figure 17 *mos_netopen Defect Model on Drain Terminal of MOS Devices*

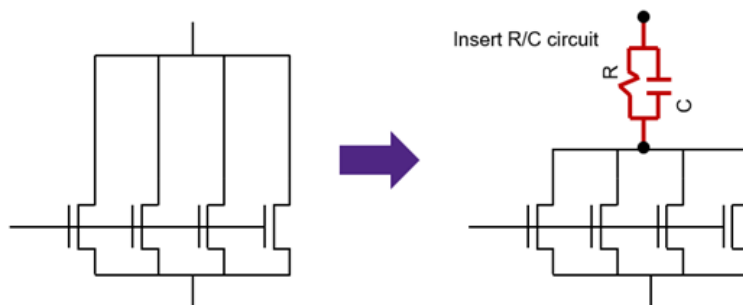


Figure 18 shows a `subckt_pinopen` defect model on the port of a subcircuit. A defect is injected to open the connection of a pin of a group of parallel subcircuits (often used as a macromodel). The default weight of the defect is equal to the number of parallel subcircuits, but you can use the `m_param` argument of the `faults` command to specify which instance parameter is the multiplying factor and thus control the number of parallel

subcircuits. The default for `m_param` is `m`. Weight is adjustable using the `defect_weight` command.

Figure 18 *subckt_pinopen for one of the Ports of the Subcircuit or Macromodel*

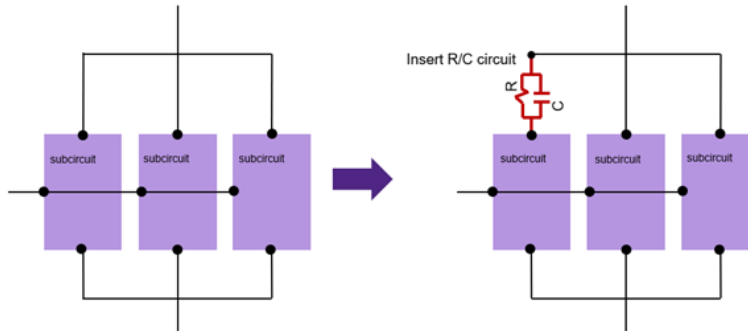
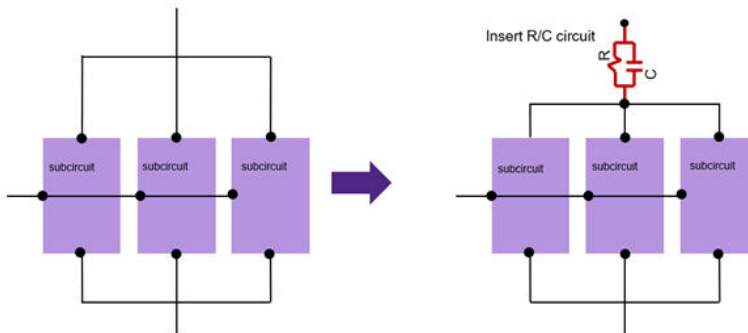


Figure 19 shows a `subckt_netopen` defect model on the port of a subcircuit. A defect is injected to open the connection of a net of a group of parallel subcircuits (often used as a macromodel). The default weight of the defect is equal to 1. Weight is adjustable using the `defect_weight` command.

Figure 19 *subckt_netopen for one of the Ports of the Subcircuit or Macromodel*



Pin Open and Net Open Defect Model Syntax

Apply the pin open and net open defect models using the `faults` command and the following syntax:

```
{
  "name" : "user_defined_name01",
  "base_model" : "bhv_cap_pinopen",
  "m_param" : "parameter",
  "res" : rval
  "cap" : cval
  ...
},
{
```

Chapter 3: Defining and Scoping Defect Models

Pin Open and Net Open Defect Models

```

    "name" : "user_defined_name02",
    "base_model" : "bhv_cap_netopen",
    "res" : rval
    "cap" : cval
    ...
  },
  {
    "name" : "user_defined_name03",
    "base_model" : "bhv_res_pinopen",
    "m_param" : "parameter",
    "res" : rval
    "cap" : cval
    ...
  },
  {
    "name" : "user_defined_name04",
    "base_model" : "bhv_res_netopen",
    "res" : rval
    "cap" : cval
    ...
  },
  {
    "name" : "user_defined_name05",
    "base_model" : "bjt_pinopen",
    "terminals" : [list_of_terminals],
    "m_param" : "parameter",
    "res" : rval
    "cap" : cval
    ...
  },
  {
    "name" : "user_defined_name06",
    "base_model" : "bjt_netopen",
    "terminals" : [list_of_terminals],
    "res" : rval
    "cap" : cval
    ...
  },
  {
    "name" : "user_defined_name07",
    "base_model" : "cap_pinopen",
    "m_param" : "parameter",
    "res" : rval
    "cap" : cval
    ...
  },
  {
    "name" : "user_defined_name08",
    "base_model" : "cap_netopen",
    "res" : rval
    "cap" : cval
    ...
  },
  {

```

```
{
  "name" : "user_defined_name09",
  "base_model" : "diode_pinopen",
  "m_param" : "parameter",
  "res" : rval
  "cap" : cval
  ...
},
{
  "name" : "user_defined_name10",
  "base_model" : "diode_netopen",
  "res" : rval
  "cap" : cval
  ...
},
{
  "name" : "user_defined_name11",
  "base_model" : "ind_pinopen",
  "m_param" : "parameter",
  "res" : rval
  "cap" : cval
  ...
},
{
  "name" : "user_defined_name12",
  "base_model" : "ind_netopen",
  "res" : rval
  "cap" : cval
  ...
},
{
  "name" : "user_defined_name13",
  "base_model" : "jfet_pinopen",
  "terminals" : [list_of_terminals],
  "m_param" : "parameter",
  "res" : rval
  "cap" : cval
  ...
},
{
  "name" : "user_defined_name14",
  "base_model" : "jfet_netopen",
  "terminals" : [list_of_terminals],
  "res" : rval
  "cap" : cval
  ...
},
{
  "name" : "user_defined_name15",
  "base_model" : "mos_pinopen",
  "terminals" : [list_of_terminals],
  "m_param" : "parameter",
  ...
}
```

```
    },
    {
      "name" : "user_defined_name16",
      "base_model" : "mos_netopen",
      "terminals" : [list_of_terminals],
      "res" : rval
      "cap" : cval
      ...
    },
    {
      "name" : "user_defined_name17",
      "base_model" : "res_pinopen",
      "m_param" : "parameter",
      "res" : rval
      "cap" : cval
      ...
    },
    {
      "name" : "user_defined_name18",
      "base_model" : "res_netopen",
      "res" : rval
      "cap" : cval
      ...
    },
    {
      "name" : "user_defined_name19",
      "base_model" : "subckt_pinopen",
      "subckt" : [list_of_subcircuit_macromodel],
      "terminals" : [list_of_terminals],
      "m_param" : "parameter",
      ...
    },
    {
      "name" : "user_defined_name20",
      "base_model" : "subckt_netopen",
      "subckt" : [list_of_subcircuit_macromodel],
      "terminals" : [list_of_terminals],
      "res" : rval
      "cap" : cval
      ...
    }
  }
```

Argument	Description
"name" : "user_defined_name"	Defines the name of the customized model.

Argument	Description
<pre>"base_model" : "base_model"</pre>	Specifies one of the following base defect models: • <code>bhv_cap_pinopen</code>: Parallel behavioral capacitor pin open defect model. • <code>bhv_cap_netopen</code>: Parallel behavioral capacitor net open defect model. • <code>bhv_res_pinopen</code>: Parallel behavioral resistor pin open defect model. • <code>bhv_res_netopen</code>: Parallel behavioral resistor net open defect model. • <code>bjt_pinopen</code>: Parallel BJT pin open defect model. • <code>bjt_netopen</code>: Parallel BJT net open defect model. • <code>cap_pinopen</code>: Parallel capacitor pin open defect model. • <code>cap_netopen</code>: Parallel capacitor net open defect model. • <code>diode_pinopen</code>: Parallel diode pin open defect model. • <code>diode_netopen</code>: Parallel diode net open defect model. • <code>ind_pinopen</code>: Parallel inductor pin open defect model. • <code>ind_netopen</code>: Parallel inductor net open defect model. • <code>jfet_pinopen</code>: Parallel JFET pin open defect model. • <code>jfet_netopen</code>: Parallel JFET net open defect model. • <code>mos_pinopen</code>: Parallel MOSFET pin open defect model. • <code>mos_netopen</code>: Parallel MOSFET net open defect model. • <code>res_pinopen</code>: Parallel resistor pin open defect model. • <code>res_netopen</code>: Parallel resistor net open defect model. • <code>subckt_pinopen</code>: Parallel subcircuit pin open defect model. • <code>subckt_netopen</code>: Parallel subcircuit net open defect model.
<pre>"subckt" : [list_of_subcircuit_macro model]</pre>	Scopes the defect models to the specified subcircuits. All instances within these subcircuits are included for defect identification. The syntax for <code>list_of_subcircuit_macro</code> is as follows: <pre>"subckt_name", "subckt_name", ...</pre> More than one subcircuit name is allowed. Every subcircuit name must be inside quotation marks (") and separated by a comma (,). For example: <pre>"subckt" : ["nch_mac", "pch_mac", "nanch_biso_mac"]</pre>
<pre>"terminals" : [list_of_terminals]</pre>	Specifies the terminal names. If no terminal is provided, it applies to all terminals of the MOSFET and subcircuits. The syntax for <code>list_of_terminals</code> is: <pre>"terminal_name", "terminal_name", ...</pre> More than one terminal name is allowed. Every terminal name must be inside quotation marks (") and separated by a comma (,). For example: <pre>"terminals" : ["d", "g"]</pre>
<pre>"res" : rval</pre>	Specifies a resistor of <code>rval</code> ohm injected for the customized model. You can set <code>rval</code> to null to omit the injection of a resistor. If this value is not specified, the default value is the same as the value specified in the <code>"defaults" : {...}</code> command.

Argument	Description
"cap" : <i>cval</i>	Specifies a capacitor of <i>cval</i> Farad injected for the customized model. You can set <i>cval</i> to null to omit the injection of a capacitor. If this value is not specified, the default value is the same as the value specified in the "defaults" : {...} command.
"m_param" : " <i>parameter</i> "	Specifies the name of the instance parameter that is used for specifying the number of parallel subcircuits in the macromodel. If this argument is not specified, it is set to <i>m</i> . By default, <i>m</i> is equal to 1.

Pin Open and Net Open Defect Model Examples

Example 1: mos_pinopen and mos_netopen

The following example shows how to enable the `mos_pinopen` and `mos_netopen` defect models.

```
.subckt u_nl_n d g s
mn2a d g s 0 nch w=0.12U l=10U m=2
mn2b d g s 0 nch w=0.12U l=10U m=5
.ends u_nl_n

.subckt u_nl_ds_p d g s
mp2 d g s VDD pch w=0.12U l=10U m=1
.ends u_nl_ds_p

.subckt INV_STD_CELL invin invout
xinv1 invout invin 0 u_nl_n
xinv2 invout invin VDD u_nl_ds_p
.ends INV_STD_CELL

xchain cin cout INV_STD_CELL
cout cout 0 1.572f
```

Apply the `mos_pinopen` and `mos_netopen` defect models to the preceding netlist using the following configuration file:

```
"faults" : {
  ...
  "models" : [ {
    "name" : "mos_pin_open",
    "base_model" : "mos_pinopen",
    "terminals" : ["d"]
  },
  {
    "name" : "mos_net_open",
    "base_model" : "mos_netopen",
    "terminals" : ["d"]
  }
]
```


Chapter 3: Defining and Scoping Defect Models

Pin Open and Net Open Defect Models

```

    } ]
  },
  "apply_fault_model" : [ {
    "fault_model" : ["mos_pin_open", "mos_net_open"],
    "inst" : ["*"]
  } ]

```

The following defects are identified:

- mos_pinopen for xchain.xinv2.mp2 (weight=1)
- mos_pinopen for xchain.xinv1.mn2a + xchain.xinv1.mn2b (weight=7 due to m=7)
- mos_netopen for xchain.xinv2.mp2 (weight=1)
- mos_netopen for xchain.xinv1.mn2a + xchain.xinv1.mn2b (weight=1)

You see the following at the report file:

A1.1	mos_net_open	CL	fault_mos_netopen_1_1_1/mos_netopen_1.*	1.000000e+00
	xchain.xinv2.mp2		d cout	
A2.2	mos_pin_open	CL	fault_mos_pinopen_2_2_2/mos_pinopen_2.*	1.000000e+00
	xchain.xinv2.mp2		d cout	
A3.3	mos_pin_open	NC	fault_mos_pinopen_3_3_3/mos_pinopen_3.*	7.000000e+00
	xchain.xinv1.mn2b		d cout	
A5.5	mos_net_open	NC	fault_mos_netopen_5_5_5/mos_netopen_5.*	1.000000e+00
	xchain.xinv1.mn2b		d cout	

Example 2: subckt_pinopen and subckt_netopen

The following example shows how to enable the subckt_pinopen and subckt_netopen defect models. In this example, u_nl_n_macro and u_nl_ds_p_macro are subcircuits for macromodel.

```

.subckt u_nl_n_macro d g s
mn2 d g s 0 nch w=0.12U l=10U m=simm
.ends u_nl_n

.subckt u_nl_ds_p_macro d g s
mp2 d g s VDD pch w=0.12U l=10U m=1
.ends u_nl_ds_p

.subckt INV_STD_CELL invin invout
xinv1a invout invin 0 u_nl_n_macro simm=2
xinv1b invout invin 0 u_nl_n_macro simm=5

xinv2 invout invin VDD u_nl_ds_p_macro simm=1
.ends INV_STD_CELL

xchain cin cout INV_STD_CELL
cout cout 0 1.572f

```

Apply the `subckt_pinopen` and `subckt_netopen` defect models to the preceding netlist using the following configuration file:

```
"faults" : {
  ...
  "models" : [ {
    "name" : "subckt_pin_open",
    "base_model" : "subckt_pinopen",
    "subckt": [
      "u_nl_n_macro", "u_nl_ds_p_macro"
    ],
    "terminals" : ["d"],
    "m_param" : "simm"
  },
  {
    "name" : "subckt_net_open",
    "base_model" : "subckt_netopen",
    "subckt": [
      "u_nl_n_macro", "u_nl_ds_p_macro"
    ],
    "terminals" : ["d"]
  } ]
},
"apply_fault_model" : [ {
  "fault_model" : ["subckt_pin_open", "subckt_net_open"],
  "inst" : ["*"]
} ]
```

The following defects are identified:

- `subckt_pinopen` for `xchain.xinv2` (weight=1)
- `subckt_pinopen` for `xchain.xinv1a + xchain.xinv1b` (weight=7 due to `simm=7`)
- `subckt_netopen` for `xchain.xinv2` (weight=1)
- `subckt_netopen` for `xchain.xinv1a + xchain.xinv1b` (weight=1)

You see the following at the report file:

```
A1.1  subckt_net_open      CL  fault_subckt_netopen_1_1_1/subckt_netopen_1.*
1.000000e+00 xchain.xinv2      d      cout
A2.2  subckt_pin_open      CL  fault_subckt_pinopen_2_2_2/subckt_pinopen_2.*
1.000000e+00 xchain.xinv2      d      cout
A3.3  subckt_net_open      NC  fault_subckt_netopen_3_3_3/subckt_netopen_3.*
1.000000e+00 xchain.xinv1a     d      cout
A5.5  subckt_pin_open      NC  fault_subckt_pinopen_5_5_5/subckt_pinopen_5.*
7.000000e+00 xchain.xinv1a     d      cout
```

Instance Parameter Variation Defect Models

The Custom Fault tool supports a soft defect flow allowing instance parameter variation defect models. These defect models are not applied by default and can be applied only

as a user-defined customized defect model. You can apply the instance parameter defect model to any defined SPICE instance parameter for primitive or subcircuit elements. You can set the instance parameters to specific values or apply absolute deltas or relative deltas. You can also apply the absolute and relative values through a value sweep.

The instance parameter base model types are as follows:

- mos_param
- jfet_param
- bjt_param
- diode_param
- res_param
- cap_param
- ind_param
- subckt_param

Use the `base_model` and `inst_param` arguments and one of the following instance parameter change arguments of the `faults` command to apply parameter variations.

Command	Description
"inst_param_abs_values" : [absval1,absval2,...]	List of actual parameter values
"inst_param_abs_sweep" : [start,stop,step]	Sweep of actual parameter values
"inst_param_abs_delta" : [absdel1,absdel2,...]	List of absolute parameter delta values
"inst_param_rel_delta" : [relval1,relval2,...]	List of percentage change values
"inst_param_rel_sweep" : [start,stop,step]	Sweep of percentage change values

Example

This example shows how to apply each of the parameter change commands to a primitive capacitor.

```
"faults" : {
...
  "models" : [
    {
      "name" : "cap_param_abs_values",
      "base_model" : "cap_param",
      "inst_param" : "c",
      "inst_param_abs_values" : [8e-12,12e-12]
```

Chapter 3: Defining and Scoping Defect Models

Instance Parameter Variation Defect Models

```

    },
    {
      "name" : "cap_param_abs_sweep",
      "base_model" : "cap_param",
      "inst_param" : "c",
      "inst_param_abs_sweep" : [8e-12,12e-12,1e-12]
    },
    {
      "name" : "cap_param_abs_delta",
      "base_model" : "cap_param",
      "inst_param" : "c",
      "inst_param_abs_delta" : [-1e-12,1e-12]
    },
    {
      "name" : "cap_param_rel_sweep",
      "base_model" : "cap_param",
      "inst_param" : "c",
      "inst_param_rel_sweep" : [-10,10,20]
    },
    {
      "name" : "cap_param_rel_delta",
      "base_model" : "cap_param",
      "inst_param" : "c",
      "inst_param_rel_delta" : [-10,10]
    }
  ] // end models
} //end faults

```

Assume the netlist contains the following capacitor:

```
Cap1 a c =10p
```

The following table shows which defects are injected for each defect model.

Command	Capacitance Value for Injected Defect
"inst_param_abs_values" : [8e-12,12e-12]	8pF and 12pF
"inst_param_abs_sweep" : [8e-12,12e-12,1e-12]	8pF, 9pF, 10pF, 11pF, and 12pF
"inst_param_abs_delta" : [-1e-12,1e-12]	9pF and 11pF
"inst_param_rel_sweep" : [-10,10,20]	9pF and 11pF
"inst_param_rel_delta" : [-10,10]	9pF and 11pF

MOS Primitive Variation Defects

The Custom Fault tool supports a soft defect flow allowing MOSFET model parameter variation defects. These defect models are not applied by default and can only be applied as a user-defined customized defect model. You can apply the MOSFET model parameter defects to primitive MOSFET instances. You cannot apply this variation to subcircuit instances or macro MOSFETs. Only the `vth`, `ids`, and `ig` parameters can be varied as shown in the following table..

MOSFET Model Parameter	Allowed Variation Method
<code>vth</code>	<code>inst_param_abs_delta</code>
<code>ids</code>	<code>inst_param_rel_delta</code> , <code>inst_param_rel_sweep</code>
<code>ig</code>	<code>inst_param_rel_delta</code> , <code>inst_param_rel_sweep</code>

[Table 5](#) lists which models support MOSFET parameter variation when using the PrimeSim XA simulator.

Table 5 MOSFET Model Support (PrimeSim XA)

	PrimeSim HSPICE Format	Eldo Format	3rd-Party Format
BSIM3v3	Level 49/53	Level 53	bsim3 / bsim3v3
BSIM4	Level 54	Level 60	bsim4
BSIM-CMG	Level 72	Level 77	bsimcmg / bsimmg / ssim_mbc
BSIM-IMG	Level 78	Level 81	bsimimg
MOS9	Level 50	Level 59	MOS902 / MOS903

[Table 6](#) lists which models support MOSFET parameter variation when using the FineSim or PrimeSim simulator.

Table 6 MOSFET Model Support (FineSim and PrimeSim)

	PrimeSim HSPICE Format	Eldo Format	3rd-Party Format
BSIM3v3	Level 49/53	Level 53	bsim3v3
BSIM4	Level 54	Level 60	bsim4

Table 6 MOSFET Model Support (FineSim and PrimeSim) (Continued)

	PrimeSim HSPICE Format	Eldo Format	3rd-Party Format
BSIM-CMG	Level 72	Level 77	bsimcmg / ssim_mbc

Example

In this example, the `vth` parameter defect is applied by changing `vth` by -50mV and +50mV.

```
"faults" : {
...
  "models" : [
    {
      "name" : "mos_vth_abs_delta",
      "base_model" : "mos_param",
      "inst_param" : "vth",
      "inst_param_abs_delta" : [-0.05,0.05]
    },
  ],
} //end models
} // end faults
"apply_fault_model" : [
  {
    "fault_model" : ["mos_vth_abs_delta"],
    "inst" : ["xtest.xinv2.mn"]
  }
] // end apply fault model
```

Defining Custom SPICE Defect Models

The Custom Fault tool allows you to define your own fully custom SPICE defect models using the following steps:

1. Define a SPICE `.SUBCKT` or Verilog-A module definition of the defect model (see [Custom SPICE Defect Model Definition in the SPICE Netlist File](#)).
2. Define the use of the SPICE defect model in the `faults` and `apply_fault_model` commands (see [Custom SPICE Defect Model Definition in the Configuration File](#)).

Note:

This feature has the following limitation:

- You can define custom SPICE defect models only for device open and short type defects. You cannot define custom SPICE defect models for net-to-net short types.

Custom SPICE Defect Model Definition in the SPICE Netlist File

You can define the SPICE defect model as either a SPICE subcircuit or a Verilog-A module in the source netlist file. The defect model must have two ports and the defect subcircuit can have any legal SPICE content. You must ensure that the custom defect model has good numerical convergence properties during simulation.

The defect model does not have to be symmetric, which means that the port order connection of the defect model to the circuit matters. The following rules are used:

- For short type defects, port1 is connected to the first terminal in the configuration file's terminal definition and port2 is connected to the second terminal in the configuration file's terminal definition.
- For open type defects, port1 is connected to the net-side and port2 is connected to the device terminal where the open defect is being injected.

Custom SPICE Defect Model Definition in the Configuration File

Define a custom SPICE defect model using the `faults` command, naming the SPICE subcircuit name with the `defect_subckt` argument. For short type defects, you must also define the `terminals` argument in array of array format.

```
"faults" : {  
  "models" : [  
    {  
      "name"           : "custom_mos_short_gds",  
      "base_model"     : "mos_short",  
      "defect_subckt"   : "custom_short_defect",  
      "terminals"      : [[ "d", "g" ], [ "d", "s" ], [ "g", "s" ] ]  
    }  
  ]  
}
```

You can then apply the custom SPICE defect model to a specific scope in the circuit using the `apply_fault_model` command.

Custom SPICE Defect Model Example

The following example uses the SPICE `.SUBCKT` definitions `myshort` and `myopen`.

Netlist:

```
.subckt myshort n1 n2  
r1 n1 a r=200  
c1 a 0 c=10p  
r2 a n2 r=1k  
.ends
```

```
.subckt myopen n1 n2
r1 n1 a r=50k
c1 a 0 c=10p
r2 a n2 r=450k
.ends

mptest d s vdd vdd pch w=1u l=1u
mntest d g vss vss nch w=w1u l=1u
```

Fault definition:

```
"faults" : {
  "defaults" : {
    "short" : {
      "res" : 200,
      "cap" : null
    },
    "open" : {
      "res" : 1e6,
      "cap" : null
    }
  },
  "models" : [
    {
      "name" : "my_short_gd",
      "base_model" : "mos_short",
      "defect_subckt" : "myshort",
      "terminals" : [["g", "d"]]
    },
    {
      "name" : "my_open_d",
      "base_model" : "mos_open",
      "defect_subckt" : "myopen",
      "terminals" : ["d"]
    }
  ]
},
"apply_fault_model" : [
  {
    "fault_model" : ["my_short_gd", "my_open_d"],
    "inst" : ["mntest"]
  }
]
```

The resulting effective circuit for the short defect is as follows:

```
mptest d g vdd vdd pch w=1u l=1u
mntest d g vss vss nch w=w1u l=1u
xmyshort g d myshort
```


The resulting effective circuit for the open defect is as follows:

```
mpctest d g vdd vdd pch w=1u l=1u
mntest dl g vss vss nch w=w1u l=1u
xmyopen d dl myopen
```

Applying Built-in and Customized Defect Models to a Circuit

The `apply_fault_model` command enables you to customize where defects are applied in a circuit by scoping the defect models to specific blocks. This command lets you:

- Specify which defect models to apply to a specific circuit scope
- Scope by subcircuit
- Scope by instances
- Scope by models

See `apply_fault_model` for details on syntax and examples for using this command.

4

Weighting and Sampling for Defects

This chapter describes the use of weighted sampling by the Custom Fault tool.

This chapter contains the following topics:

- [Defect Weights](#)
- [Defect Sampling](#)
- [Specifying Weights for Defects](#)
- [Estimating Error and Sample Size](#)
- [Using Adaptive Weighted Random Sampling \(AWRS\)](#)
- [Using Weight Sorted Sampling \(WSS\)](#)
- [Adaptive Weighted Random Sampling \(AWRS\) Tutorial](#)
- [Weighted Sorted Sampling \(WSS\) Tutorial](#)
- [IEEE P2427 Weight Tutorial](#)

Defect Weights

All Custom Fault defect analyses use the concept of weighted defects. Each defect is assigned a weight. The weight of a defect represents the defect's likelihood of occurrence and is used to calculate weighted defect coverage.

The default weight of each defect is 1. You can set the weight of each defect. See [Specifying Weights for Defects](#) and the tutorials in this chapter for detailed instructions.

The Custom Fault tool collapses equivalent defects unless the `"enable_fault_collapse" : false` command is used. When two or more defects are collapsed into a group, only one simulation is run for that group. The weight of the group is the sum of the weights of each individual defect.

For short defects, if the same pair of nodes is being shorted, the defects can be collapsed. An example of a collapsed defect is the gate-to-drain short of an inverter. The gate-to-drain short of the PMOS device is electrically equivalent to the gate-to-drain short of the

NMOS device. Only one simulation is required to assess the circuit behavior under this short defect condition.

Open defects are collapsed for equivalent parallel devices. The devices must have identical port connections. For example, if a current mirror output has two transistors with identical gate, drain, source, and bulk connections, the opening of a port on one transistor is equivalent to opening the same port on the other transistor.

The report files always report the weighted coverage. The per-defect details in the report files always report the group weight. If a defect is not collapsed, its weight is the group weight. If two or more defects are collapsed together, the reported group weight is the sum of the weight of each defect in the group.

Defect Sampling

A circuit design may contain a prohibitive number of defects that make an exhaustive defect simulation impractical in terms of time and/or computing resources. Defect sampling is the primary technique to estimate coverage in these situations. To simulate potential defects efficiently in a reasonable amount of time, the Custom Fault tool offers the following sampling methods:

- Adaptive Weighted Random Sampling (AWRS)

The AWRS method requires you to request a specific number of samples. The AWRS algorithm then selects samples and simulates those defects to estimate the coverage value of the entire population of defects. A 95% confidence interval value is reported. The accuracy of the coverage depends on the number of samples selected. The more samples the Custom Fault tool simulates, the more accurate the coverage estimation.

- Weight Sorted Sampling (WSS)

The WSS method requires you to specify a target maximum absolute error. The WSS algorithm then dispatches each defect simulation in defect weight order. The absolute error or uncertainty in the coverage result is calculated from the remaining number of defect simulations. The defect campaign stops as soon as the error target is achieved.

Both sampling methods work regardless of whether you have defined weights.

Specifying Weights for Defects

Weight is equivalent to the relative likelihood of a potential defect occurring and is user-adjustable. Some defects are more likely to happen than others. A different weight is assigned to a different defect to emphasize the importance of that defect. If you do not specify weight to the defects, all defects are assigned the same weight, and therefore

all defects have an equal potential of occurring. This is also referred to non-weighted coverage.

The Custom Fault tool uses C++ functions to define weight for each defect type. The C++ functions are specified in the weight file and read in by the Custom Fault tool to calculate the weight for each defect.

The weight file supports the following predefined C++ functions to define weight for each defect type.

Table 7 *Weight Definitions by Defect Type*

C++ Function	Defines the weight for ...	Parameters
<code>double bhv_cap_open_weight()</code>	<code>bhv_cap_open_weight</code> defect	See bhv_cap_open_weight and bhv_cap_short_weight Parameters
<code>double bhv_cap_short_weight()</code>	<code>bhv_cap_short_weight</code> defect	
<code>double bhv_cap_pinopen_weight()</code>	<code>bhv_cap_pinopen</code> defect	See bhv_cap_pinopen_weight and bhv_cap_netopen_weight Parameters
<code>double bhv_cap_netopen_weight()</code>	<code>bhv_cap_netopen</code> defect	
<code>double bhv_res_open_weight()</code>	<code>bhv_res_open_weight</code> defect	See bhv_res_open_weight and bhv_res_short_weight Parameters
<code>double bhv_res_short_weight()</code>	<code>bhv_res_short_weight</code> defect	
<code>double bhv_res_pinopen_weight()</code>	<code>bhv_res_pinopen</code> defect	See bhv_res_pinopen_weight and bhv_res_netopen_weight Parameters
<code>double bhv_res_netopen_weight()</code>	<code>bhv_res_netopen</code> defect	
<code>double bjt_open_weight()</code>	<code>bjt_open</code> defect	See bjt_open_weight and bjt_short_weight Parameters.
<code>double bjt_short_weight()</code>	<code>bjt_short</code> defect	
<code>double bjt_pinopen_weight()</code>	<code>bjt_pinopen</code> defect	See bjt_pinopen_weight and bjt_netopen_weight Parameters
<code>double bjt_netopen_weight()</code>	<code>bjt_netopen</code> defect	
<code>double cap_open_weight()</code>	<code>cap_open</code> defect	See cap_open_weight and cap_short_weight Parameters.
<code>double cap_short_weight()</code>	<code>cap_short</code> defect	
<code>double cap_pinopen_weight()</code>	<code>cap_pinopen</code> defect	See cap_pinopen_weight and cap_netopen_weight Parameters
<code>double cap_netopen_weight()</code>	<code>cap_netopen</code> defect	

Table 7 *Weight Definitions by Defect Type (Continued)*

C++ Function	Defines the weight for ...	Parameters
<code>double diode_open_weight()</code>	diode_open defect	See diode_open_weight and diode_short_weight Parameters
<code>double diode_short_weight()</code>	diode_short defect	
<code>double diode_pinopen_weight()</code>	diode_pinopen defect	See diode_pinopen_weight and diode_netopen_weight Parameters
<code>double diode_netopen_weight()</code>	diode_netopen defect	
<code>double ind_open_weight()</code>	inductor_open defect	See ind_open_weight and ind_short_weight Parameters.
<code>double ind_short_weight()</code>	inductor_short defect	
<code>double ind_pinopen_weight()</code>	ind_pinopen defect	See ind_pinopen_weight and ind_netopen_weight Parameters
<code>double ind_netopen_weight()</code>	ind_netopen defect	
<code>double jfet_open_weight()</code>	jfet_open_weight defect	See jfet_open_weight and jfet_short_weight Parameters
<code>double jfet_short_weight()</code>	jfet_short_weight defect	
<code>double jfet_pinopen_weight()</code>	jfet_pinopen defect	See jfet_pinopen_weight and jfet_netopen_weight Parameters
<code>double jfet_netopen_weight()</code>	jfet_netopen defect	
<code>double mos_open_weight()</code>	mos_open defect	See mos_open_weight and mos_short_weight Parameters.
<code>double mos_short_weight()</code>	mos_short defect	
<code>double mos_pinopen_weight()</code>	mos_pinopen defect	See mos_pinopen_weight and mos_netopen_weight Parameters
<code>double mos_netopen_weight()</code>	mos_netopen defect	
<code>double net_to_net_short_weight()</code>	net_to_net_short defect	See net_to_net_short_weight Parameter.
<code>double res_open_weight()</code>	res_open defect	See res_open_weight and res_short_weight Parameters.
<code>double res_short_weight()</code>	res_short defect	
<code>double res_pinopen_weight()</code>	res_pinopen defect	See res_pinopen_weight and res_netopen_weight Parameters
<code>double res_netopen_weight()</code>	res_netopen defect	
<code>double subckt_open_weight()</code>	subckt_open defect	See subckt_open_weight and subckt_short_weight Parameters.
<code>double subckt_short_weight()</code>	subckt_short defect	

Table 7 *Weight Definitions by Defect Type (Continued)*

C++ Function	Defines the weight for ...	Parameters
<code>double subckt_pinopen_weight()</code>	<code>subckt_pinopen</code> defect	See subckt_pinopen_weight and subckt_netopen_weight Parameters
<code>double subckt_netopen_weight()</code>	<code>subckt_netopen</code> defect	

Each defect type supports many parameters (such as length and width) that let you adjust the weight based on these parameters.

bhv_cap_open_weight and bhv_cap_short_weight Parameters

The supported parameters for `bhv_cap_open_weight()` and `bhv_cap_short_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for instance name.
<code>GET_STRING_VAR(model_name);</code>	<code>model_name</code> is the parameter for model name.

The following example shows that the weight is 0.5 for all `bhv_cap_open` defects.

```
double bhv_cap_open_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    GET_DOUBLE_VAR(m);
    GET_STRING_VAR(fault_port0);
    GET_STRING_VAR(fault_port_name0);
    GET_STRING_VAR(fault_type);
    GET_STRING_VAR(instance_name);
    GET_STRING_VAR(model_name);
    GET_STRING_VAR(port_name0);
    GET_STRING_VAR(port_name1);
    GET_STRING_VAR(port_name2);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;

        weight = 0.5;

    return weight;
}
```

bhv_cap_pinopen_weight and bhv_cap_netopen_weight Parameters

The supported parameters for `bhv_cap_pinopen_weight()` and `bhv_cap_netopen_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for instance name.
<code>GET_STRING_VAR(model_name);</code>	<code>model_name</code> is the parameter for model name.
<code>inst_count</code>	Total count of parallel behavioral capacitors.

The following example assigns a weight equal to the sum of all behavioral capacitor instances for all `bhv_cap_netopen` defects:

```
double bhv_cap_netopen_weight(int inst_count,
    int * sacount, char *** sakey, char *** saval,
    int * dacount, char *** dakey, double ** daval)
{
    GET_ARRAY_DOUBLE_VAR(m);
    GET_ARRAY_STRING_VAR(instance_name);
    GET_ARRAY_STRING_VAR(model_name);
    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;
    weight = inst_count;
    ...
    return weight;
}
```

bhv_res_open_weight and bhv_res_short_weight Parameters

The supported parameters for `bhv_res_open_weight()` and `bhv_res_short_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for instance name.
<code>GET_STRING_VAR(model_name);</code>	<code>model_name</code> is the parameter for model name.

The following example shows that the weight is 0.5 for all `bhv_res_open` defects.

```
double bhv_res_open_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    GET_DOUBLE_VAR(m);
    GET_STRING_VAR(fault_port0);
    GET_STRING_VAR(fault_port_name0);
    GET_STRING_VAR(fault_type);
    GET_STRING_VAR(instance_name);
    GET_STRING_VAR(model_name);
    GET_STRING_VAR(port_name0);
    GET_STRING_VAR(port_name1);
    GET_STRING_VAR(port_name2);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;

        weight = 0.5;

    return weight;
}
```

bhv_res_pinopen_weight and bhv_res_netopen_weight Parameters

The supported parameters for `bhv_res_pinopen_weight()` and `bhv_res_netopen_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for instance name.
<code>GET_STRING_VAR(model_name);</code>	<code>model_name</code> is the parameter for model name.
<code>inst_count</code>	Total count of parallel behavioral resistors.

The following example assigns a weight equal to the sum of all behavioral resistors instances for all `bhv_res_netopen` defects:

```
double bhv_res_netopen_weight(int inst_count,
    int * sacount, char *** sakey, char *** saval,
    int * daccount, char *** dakey, double ** daval)
{
    GET_ARRAY_DOUBLE_VAR(m);
    GET_ARRAY_STRING_VAR(instance_name);
    GET_ARRAY_STRING_VAR(model_name);
```



```
////////// CUSTOM WEIGHT CODE GOES HERE //////////  
double weight=1.0;  
weight = inst_count;  
...  
return weight;  
}
```

bjt_open_weight and bjt_short_weight Parameters

The supported parameters for `bjt_open_weight()` and `bjt_short_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for the instance name.
<code>GET_STRING_VAR(model_name);</code>	<code>model_name</code> is the parameter for the model name.

The following example shows that if the multiplier `m` is larger than 1, the tool assigns the weight of the `bjt_open` defect to be 2.0. Otherwise, it assigns 0.5.

```
double bjt_open_weight(int scount, char ** skey, char ** sval,  
    int dcount, char ** dkey, double * dval)  
{  
    GET_DOUBLE_VAR(m);  
    GET_STRING_VAR(instance_name);  
    GET_STRING_VAR(model_name);  
  
    ////////// CUSTOM WEIGHT CODE GOES HERE //////////  
    double weight;  
  
    if (m>1) {  
        weight = 2.0;  
    } else {  
        weight = 0.5;  
    }  
  
    return weight;  
}
```

bjt_pinopen_weight and bjt_netopen_weight Parameters

The supported parameters for `bjt_pinopen_weight()` and `bjt_netopen_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for the instance name.
<code>GET_STRING_VAR(model_name);</code>	<code>model_name</code> is the parameter for the model name.
<code>inst_count</code>	Total count of parallel BJTs.

The following example assigns a weight equal to the sum of all BJT instances for all `bjt_netopen` defects:

```
double jfet_netopen_weight(int inst_count,
    int * sacount, char *** sakey, char *** saval,
    int * dacount, char *** dakey, double ** daval)
{
    GET_ARRAY_DOUBLE_VAR(m);
    GET_ARRAY_STRING_VAR(instance_name);
    GET_ARRAY_STRING_VAR(model_name);
    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;
    weight = inst_count;
    ...
    return weight;
}
```

cap_open_weight and cap_short_weight Parameters

The supported parameters for `cap_open_weight()` and `cap_short_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_DOUBLE_VAR(c);</code>	<code>c</code> is the parameter for capacitance value.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for instance name.

In the following example, if the value of the capacitor is larger than $1e-6$, then the tool assigns the weight of `cap_short` defect to be 2.0. If the value of the capacitor is larger than $1e-10$, then it assigns the weight of 1.5. Otherwise, it assigns 1.0.

```
double cap_short_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    GET_DOUBLE_VAR(m);
    GET_DOUBLE_VAR(c);
    GET_STRING_VAR(instance_name);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight;

    if (c > 1e-6) {
        weight = 2.0;
    } else if (c > 1e-10) {
        weight = 1.5;
    } else {
        weight = 1.0;
    }

    return weight;
}
```

cap_pinopen_weight and cap_netopen_weight Parameters

The supported parameters for `cap_pinopen_weight()` and `cap_netopen_weight()` are:

Parameter	Description
<code>GET_ARRAY_DOUBLE_VAR(c)</code>	<code>c</code> is the capacitance value.
<code>inst_count</code>	Total count of parallel capacitors.

The following example assigns a weight equal to the sum of all capacitance values, where the sum is normalized by a $1e12$ factor:

```
double cap_netopen_weight(int inst_count,
    int * saccount, char *** sakey, char *** saval,
    int * dacount, char *** dakey, double ** daval)
{
    GET_ARRAY_DOUBLE_VAR(c);
    ...
    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=0.0;
    // loop over all the parallel instances to access individual parameters
    for (int i=0; i< int(inst_count); i++)
    {
        weight += c[i]*1e12;
        // printf("weight=%f\n",weight);
    }
    return weight;
}
```

diode_open_weight and diode_short_weight Parameters

The supported parameters for `diode_open_weight()` and `diode_short_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_DOUBLE_VAR(area);</code>	<code>area</code> is parameter of the area of the diode.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter of an instance name.
<code>GET_STRING_VAR(model_name);</code>	<code>model_name</code> is the parameter of a model name.

The following example assigns the weight of 1.0 to all the `diode_short` defects.

```
double diode_short_weight(int scout, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    GET_DOUBLE_VAR(area);
    GET_DOUBLE_VAR(m);
    GET_STRING_VAR(instance_name);
    GET_STRING_VAR(model_name);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;
    return weight;
}
```

diode_pinopen_weight and diode_netopen_weight Parameters

The supported parameters for `diode_pinopen_weight()` and `diode_netopen_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_DOUBLE_VAR(area);</code>	<code>area</code> is parameter of the area of the diode.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter of an instance name.
<code>GET_STRING_VAR(model_name);</code>	<code>model_name</code> is the parameter of a model name.
<code>inst_count</code>	Total count of parallel diodes.

The following example assigns a weight equal to the sum of all m factor values for all `diode_netopen` defects:

```
double diode_netopen_weight(int inst_count,
    int * sacount, char *** sakey, char *** saval,
    int * dacount, char *** dakey, double ** daval)
{
    GET_ARRAY_DOUBLE_VAR(m);
    GET_ARRAY_STRING_VAR(instance_name);
    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=0.0;

    // loop over all the parallel instances to access individual parameters
    for (int i=0; i< int(inst_count); i++)
    {
        // access i-th element as: digital[i]
        // etc.
        weight += m[i];
    }
    return weight;
}
```

ind_open_weight and ind_short_weight Parameters

The supported parameters for `ind_open_weight()` and `ind_short_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	m is the parameter for multiplier.
<code>GET_DOUBLE_VAR(l);</code>	l is the parameter for inductance value.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for instance name.

In the following example, if the value of the inductor is larger than $1e-3$, then the tool assigns the weight of `ind_short` defect to be 2.0. Otherwise, it assigns 1.0.

```
double ind_short_weight(int scout, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    GET_DOUBLE_VAR(m);
    GET_DOUBLE_VAR(l);
    GET_STRING_VAR(instance_name);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight;

    if (l > 1e-3) {
        weight = 2.0;
    } else {
```

```
    weight = 1.0;

    return weight;
}
```

ind_pinopen_weight and ind_netopen_weight Parameters

The supported parameters for `ind_pinopen_weight()` and `ind_netopen_weight()` are :

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_DOUBLE_VAR(l);</code>	<code>l</code> is the parameter for inductance value.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for instance name.
<code>inst_count</code>	Total count of parallel inductors.

The following example assigns a weight equal to the sum of all `m` factor values for all `ind_netopen` defects:

```
double ind_netopen_weight(int inst_count,
    int * saount, char *** sakey, char *** saval,
    int * dacount, char *** dakey, double ** daval)
{
    GET_ARRAY_DOUBLE_VAR(m);
    GET_ARRAY_STRING_VAR(instance_name);
    ////////////////////////////////// CUSTOM WEIGHT CODE GOES HERE //////////////////////////////////
    double weight=0.0;

    // loop over all the parallel instances to access individual parameters
    for (int i=0; i< int(inst_count); i++)
    {
        // access i-th element as: digital[i]
        // etc.
        weight += m[i];
    }
    return weight;
}
```

jfet_open_weight and jfet_short_weight Parameters

The supported parameters for `jfet_open_weight()` and `jfet_short_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.

Parameter	Description
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for instance name.
<code>GET_STRING_VAR(model_name);</code>	<code>model_name</code> is the parameter for model name.

The following example shows that if the multiplier `m` is larger than 1, the tool assigns the weight of the `jfet_open` defect to be 2.0. Otherwise, it assigns 0.5.

```
double jfet_open_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    GET_DOUBLE_VAR(m);
    GET_STRING_VAR(fault_port0);
    GET_STRING_VAR(fault_port_name0);
    GET_STRING_VAR(fault_type);
    GET_STRING_VAR(instance_name);
    GET_STRING_VAR(model_name);
    GET_STRING_VAR(port_name0);
    GET_STRING_VAR(port_name1);
    GET_STRING_VAR(port_name2);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;
    if (m>1) {
        weight = 2.0;
    }
    else {
        weight = 0.5;
    }
    return weight;
}
```

jfet_pinopen_weight and jfet_netopen_weight Parameters

The supported parameters for `jfet_pinopen_weight()` and `jfet_netopen_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for instance name.
<code>GET_STRING_VAR(model_name);</code>	<code>model_name</code> is the parameter for model name.
<code>inst_count</code>	Total count of parallel JFETs.

The following example assigns a weight equal to the sum of all JFET instances for all `jfet_netopen` defects:

```
double jfet_netopen_weight(int inst_count,
    int * sacount, char *** sakey, char *** saval,
    int * dacount, char *** dakey, double ** daval)
{
    GET_ARRAY_DOUBLE_VAR(m);
    GET_ARRAY_STRING_VAR(instance_name);
    GET_ARRAY_STRING_VAR(model_name);
    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;
    weight = inst_count;
    ...
    return weight;
}
```

mos_open_weight and mos_short_weight Parameters

The supported parameters for `mos_open_weight()` and `mos_short_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_DOUBLE_VAR(gate_cap);</code>	<code>gate_cap</code> is the parameter of gate capacitance of the MOSFET.
<code>GET_DOUBLE_VAR(w);</code>	<code>w</code> is the parameter of the width of a MOSFET.
<code>GET_DOUBLE_VAR(l);</code>	<code>l</code> is the parameter of the length of a MOSFET.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for instance name.
<code>GET_STRING_VAR(model_name);</code>	<code>model_name</code> is the parameter for model name.
<code>GET_STRING_VAR(nfin);</code>	<code>nfin</code> is the parameter that specifies the number of fins per finger.

In the following example if the model name of the MOSFET is `pch3`, then the tool assigns the weight of `mos_short` to be `gate_cap*1e-6`. if the model name of the MOSFET is `nch3`, then the tool assigns the weight of `0.5*gate_cap*1e-6`. Otherwise, it assigns 0.0.

```
double mos_short_weight(int scount, char ** skey, char ** sval,
    int dcoun, char ** dkey, double * dval)
{
    GET_DOUBLE_VAR(gate_cap);
    GET_DOUBLE_VAR(l);
    GET_DOUBLE_VAR(m);
    GET_DOUBLE_VAR(w);
```



```

GET_STRING_VAR(instance_name);
GET_STRING_VAR(model_name);

////////// CUSTOM WEIGHT CODE GOES HERE //////////
double weight;

if (!strcmp(model_name,"pch3")) {
    weight = gate_cap * 1e-6;
} else if (!strcmp(model_name,"nch3")) {
    weight = 0.5 * gate_cap * 1e-6;
} else {
    weight = 0.0;
}

return weight;
}

```

mos_pinopen_weight and mos_netopen_weight Parameters

The supported parameters for `mos_pinopen_weight()` and `mos_netopen_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_DOUBLE_VAR(gate_cap);</code>	<code>gate_cap</code> is the parameter of gate capacitance of the MOSFET.
<code>GET_DOUBLE_VAR(w);</code>	<code>w</code> is the parameter of the width of a MOSFET.
<code>GET_DOUBLE_VAR(l);</code>	<code>l</code> is the parameter of the length of a MOSFET.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for instance name.
<code>GET_STRING_VAR(model_name);</code>	<code>model_name</code> is the parameter for model name.
<code>GET_STRING_VAR(nfin);</code>	<code>nfin</code> is the parameter that specifies the number of fins per finger.
<code>inst_count</code>	Total count of parallel MOSFETs.

The following example assigns a weight equal to the sum of all MOSFET instances for all `mos_netopen` defects:

```

double mos_netopen_weight(int inst_count,
    int * sacount, char *** sakey, char *** saval,
    int * dacount, char *** dakey, double ** daval)
{
    GET_ARRAY_DOUBLE_VAR(l);
    GET_ARRAY_DOUBLE_VAR(w);

```

```
GET_ARRAY_DOUBLE_VAR(m);
GET_ARRAY_STRING_VAR(instance_name);
GET_ARRAY_STRING_VAR(model_name);
////////// CUSTOM WEIGHT CODE GOES HERE //////////
double weight=1.0;
weight = inst_count;
...
return weight;
}
```

net_to_net_short_weight Parameter

The supported parameter for `net_to_net_short_weight()` is:

Parameter	Description
<code>GET_DOUBLE_VAR(coupling_cap);</code>	<code>coupling_cap</code> is parameter of coupling capacitance value connecting two nets.

The following example assigns the weight of 1.0 to all the `net_to_net_short` defects:

```
double net_to_net_short_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    GET_DOUBLE_VAR(coupling_cap);

    ////////// CUSTOM WEIGHT CODE GOES HERE //////////
    double weight=1.0;
    return weight;
}
```

res_open_weight and res_short_weight Parameters

The supported parameters for `res_open_weight()` and `res_short_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_DOUBLE_VAR(r);</code>	<code>r</code> is the parameter for resistance value.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for instance name.

In the following example, if the value of the resistor is larger than 10000, then the tool assigns the weight of the `res_short` defect to be 2.0. If the value of the resistor is larger than 1000, it assigns the weight of 1.5. Otherwise, it assigns 1.0.

```
double res_short_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    GET_DOUBLE_VAR(m);
    GET_DOUBLE_VAR(r);
    GET_STRING_VAR(instance_name);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;

    if (r > 10000) {
        weight = 2.0;
    } else if (r > 1000) {
        weight = 1.5;
    } else {
        weight = 1.0;
    }

    return weight;
}
```

res_pinopen_weight and res_netopen_weight Parameters

The supported parameters for `res_pinopen_weight()` and `res_netopen_weight()` are:

Parameter	Description
<code>GET_DOUBLE_VAR(m);</code>	<code>m</code> is the parameter for multiplier.
<code>GET_DOUBLE_VAR(r);</code>	<code>r</code> is the parameter for resistance value.
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter for instance name.
<code>inst_count</code>	Total count of parallel resistors.

The following example assigns a weight equal to the sum of all resistance value for all `res_netopen` defects:

```
double res_netopen_weight(int inst_count,
    int * sacount, char *** sakey, char *** saval,
    int * daccount, char *** dakey, double ** daval)
{
    GET_ARRAY_DOUBLE_VAR(m);
    GET_ARRAY_STRING_VAR(instance_name);
    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=0.0;

    // loop over all the parallel instances to access individual parameters
    for (int i=0; i< int(inst_count); i++)
    {
```

```
// access i-th element as: digital[i]
// etc.
weight += r[i]
} return weight;
}
```

subckt_open_weight and subckt_short_weight Parameters

The supported parameters for `subckt_open_weight()` and `subckt_short_weight()` are:

Parameter	Description
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter of an instance name.
<code>GET_STRING_VAR(subckt_name);</code>	<code>subckt_name</code> is the parameter of a subcircuit name.

The following example assigns the weight of 1.0 to all the `subckt_short` defects:

```
double subckt_short_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    GET_STRING_VAR(instance_name);
    GET_STRING_VAR(subckt_name);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;
    return weight;
}
```

subckt_pinopen_weight and subckt_netopen_weight Parameters

The supported parameters for `subckt_pinopen_weight()` and `subckt_netopen_weight()` are:

Parameter	Description
<code>GET_STRING_VAR(instance_name);</code>	<code>instance_name</code> is the parameter of an instance name.
<code>GET_STRING_VAR(subckt_name);</code>	<code>subckt_name</code> is the parameter of a subcircuit name.
<code>inst_count</code>	Total count of parallel subcircuits.

The following example assigns a weight equal to the sum of all subcircuits instances for all subckt_netopen defects:

```
double subckt_netopen_weight(int inst_count,
    int * sacount, char *** sakey, char *** saval,
    int * dacount, char *** dakey, double ** daval)
{
    GET_ARRAY_DOUBLE_VAR(l);
    GET_ARRAY_DOUBLE_VAR(w);
    GET_ARRAY_DOUBLE_VAR(m);
    GET_ARRAY_STRING_VAR(instance_name);
    ////////////////////////////////// CUSTOM WEIGHT CODE GOES HERE //////////////////////////////////
    double weight=1.0;
    weight = inst_count;
    ...
    return weight;
}
```

Weight File Example

Figure 20 shows an example of the weight file containing the weight for mos_short and mos_open.

Figure 20 Weight File Example

```

#include <string>
#include <cmath>
#include <iostream>
#include <limits>
#include <string.h>

using namespace std;
#define UNUSED(expr) do {(void)(expr);} while(0)
#define QNaN std::numeric_limits<double>::quiet_NaN()
#define GET_DOUBLE_VAR(_vv_) \
double _vv_=QNaN; \
for (int i=0; i<dcount; i++) \
    if (!strcmp(dkey[i],#_vv_)) \
    { \
        _vv_=dval[i]; \
        break; \
    } \
UNUSED(_vv_); \
#define GET_STRING_VAR(_vv_) \
char _vv_[4096]=""; \
for (int i=0; i<scount; i++) \
    if (!strcmp(skey[i],#_vv_)) \
    { \
        strcpy(_vv_,sval[i]); \
        break; \
    } \
UNUSED(_vv_);
extern "C" {
double mce_open_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    // (model: n) model_name, u, l, gate_cap, n, instance_name
    // (model: p) model_name, u, l, gate_cap, n, instance_name
    GET_DOUBLE_VAR(gate_cap);
    GET_DOUBLE_VAR(l);
    GET_DOUBLE_VAR(n);
    GET_DOUBLE_VAR(u);
    GET_STRING_VAR(instance_name);
    GET_STRING_VAR(model_name);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;

    return weight;
}

double mce_short_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    // (model: n) model_name, u, l, gate_cap, n, instance_name
    // (model: p) model_name, u, l, gate_cap, n, instance_name
    GET_DOUBLE_VAR(gate_cap);
    GET_DOUBLE_VAR(l);
    GET_DOUBLE_VAR(n);
    GET_DOUBLE_VAR(u);
    GET_STRING_VAR(instance_name);
    GET_STRING_VAR(model_name);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;

    return weight;
}
} // extern C

```

The Custom Fault tool also generates a design-specific template for the weight file. The weight file contains the parameter definition used in the design. The template can be found in the output directory with a suffix of *_weights.cpp.

Figure 21 shows a portion of the design-specific template. You can use this template to create the weight file for this specific design. The template shows the list of parameters supported in your design, and you can use these parameters to construct equations to calculate the weight of the device. In the `ind_open_weight()` function, the supported parameters are `L`, `m`, `instance_name`. In the `mos_open_weight()` function, the supported parameters are `dtemp`, `gate_cap`, `ids0mult`, `l`, `nf`, `nfin`, `ngcon`, `pdej`, `psej`, `u0mult`, `instance_name`, and `model_name`.

Figure 21 Design-Specific Template Example

```
double inductor_open_weight(int scout, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    // L, m, instance_name
    GET_DOUBLE_VAR(L);
    GET_DOUBLE_VAR(m);
    GET_STRING_VAR(instance_name);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;

    return weight;
}

double mos_open_weight(int scout, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    // (model: egvpfet) model_name, dtemp, ngcon, ids0mult, u0mult, psej, pdej, nfin, nf, l, gate_cap, m, instance_name
    // (model: egvpfet) model_name, dtemp, ngcon, ids0mult, u0mult, psej, pdej, nfin, nf, l, gate_cap, m, instance_name
    // (model: lvtnfet) model_name, dtemp, ngcon, ids0mult, u0mult, psej, pdej, nfin, nf, l, gate_cap, m, instance_name
    // (model: lvtpfet) model_name, dtemp, ngcon, ids0mult, u0mult, psej, pdej, nfin, nf, l, gate_cap, m, instance_name
    // (model: slvtnfet) model_name, dtemp, ngcon, ids0mult, u0mult, psej, pdej, nfin, nf, l, gate_cap, m, instance_name
    // (model: slvtpfet) model_name, dtemp, ngcon, ids0mult, u0mult, psej, pdej, nfin, nf, l, gate_cap, m, instance_name
    // gate_cap, m, instance_name
    GET_DOUBLE_VAR(dtemp);
    GET_DOUBLE_VAR(gate_cap);
    GET_DOUBLE_VAR(ids0mult);
    GET_DOUBLE_VAR(l);
    GET_DOUBLE_VAR(m);
    GET_DOUBLE_VAR(nf);
    GET_DOUBLE_VAR(nfin);
    GET_DOUBLE_VAR(ngcon);
    GET_DOUBLE_VAR(pdej);
    GET_DOUBLE_VAR(psej);
    GET_DOUBLE_VAR(u0mult);
    GET_STRING_VAR(instance_name);
    GET_STRING_VAR(model_name);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;

    return weight;
}
```

You use the `defect_weight` command to read in the weight file.

Assigning Weight Based on Hierarchical Instance Names

The general recommendation for defect weights is to define a weight function that calculates the weight based on a device's physical instance parameters such as width, length, area, fingers, and so on. However, in some scenarios it may be necessary to

define weight as a function of the hierarchical instance name. This can be achieved using C functions for string manipulation and matching.

The following is an example that can be used as a basis for such comparisons.

```
////////// CUSTOM WEIGHT CODE GOES HERE ////////////
double weight=1.0;
//// Set high weight to all instances in xtop.x1, low weight to all others
//// Create a string object from the instance_name
std::string inst_name_str = std::string(instance_name);

//// Use string rfind function to see if initial instance name pattern matches
//// Instances under xtop.x1 will get weight x 1000 otherwise weight x 5
if (inst_name_str.rfind("xtop.x1.", 0) == 0)
    weight=width*1000;
else
    weight=width*5;
//// print statement to verify weight at fault id time; can be commented out
printf("%s %8.2f\n",instance_name,weight);
return weight;
```

Estimating Error and Sample Size

When either Adaptive Weighted Random Sampling (AWRS) or Weight Sorted Sampling (WSS) is used, it is helpful to predict the number of samples required to achieve an error target. You can use the Custom Fault tool to generate an estimated coverage error bound (%) report for various sampling sizes and coverage targets.

Sampling error estimation is run for test coverage. If the test case is set up for diagnostic coverage, then sampling error estimation is run for diagnostic coverage, as well. An estimate is produced simultaneously for both AWRS and WSS.

This is described further in the following topics:

- [Sampling Error Configuration Options](#)
- [Estimated Coverage Error Bound Report Output](#)
- [Estimated Coverage Error Bound Report Examples](#)

Sampling Error Configuration Options

Use the `sampling` command to enable sampling error estimation:

```
"sampling" :
{
    "estimate_error" : true
}
```


By default for AWRS, 10 sample sizes are chosen based on a percentage of the total number of defects: 2%, 4%, 8%, 16%, 32%, 50%, 60%, 70%, 80%, and 90%. So if there are 1000 defects, the sample sizes are: 20, 40, 80, 160, 320, 500, 600, 700, 800, and 900.

Optionally, you can specify your own array of at most 10 sample sizes. For example:

```
"sampling" :  
{  
  "estimate_error" : true,  
  "error_sample_sizes" : [100, 200, 400, 800, 1600, 3200]  
}
```

If you do not specify your own sample sizes, then some of the default generated sample sizes may be skipped based on the following conditions:

- For test coverage, if the maximum error is over 10% for a given sample size, then it is skipped.
- For both test and diagnostic coverage, if the maximum error is less than 1% for a given sample size, then larger sample sizes are skipped.

If you do specify your own sample sizes, then sampling error estimation is run for all of them.

Lastly, there is an option in the `diagnostic_coverage` command to allow safe defects. If this option is set to `false`, then all defects are considered dangerous and the sampling error estimation results are the same for both test and diagnostic coverage.

```
"diagnostic_coverage" :  
{  
  "allow_safe_fault" : false  
}
```

For more information, see [diagnostic_coverage \(for Residual Faults and Direct SPFM Calculation\)](#).

Estimated Coverage Error Bound Report Output

The report is in the form of a `.png` image and a subdirectory for `.dat` files. The `.png` image provides you with a way to visualize the error, while the `.dat` files list error margins for each sample size and coverage percentage. This report helps guide what sample size to select for simulation in a design.

The report files are produced simultaneously for both AWRS and WSS and stored as follows:

- Test coverage error estimation image:

out/out_prefix.awrs.tc_err.png

out/out_prefix.sorted.tc_err.png

- Test coverage error estimation data:

out/out_prefix.err_est/awrs.err_tc_sample_size.dat

out/out_prefix.err_est/sorted.err_tc_sample_size.dat

- Diagnostic coverage error estimation image:

out/out_prefix.awrs.dc_err.png

out/out_prefix.sorted.dc_err.png

- Diagnostic coverage error estimation data:

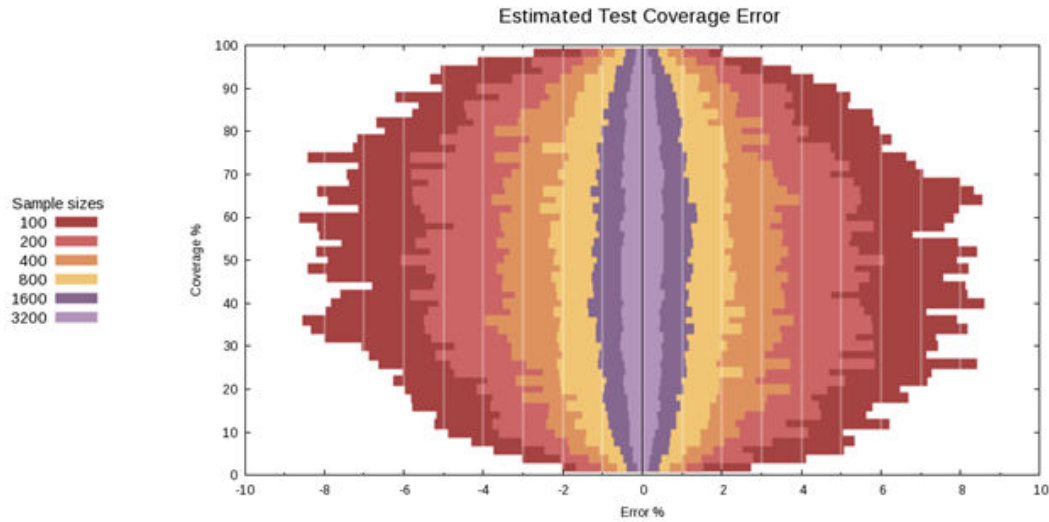
out/out_prefix.err_est/awrs.err_dc_sample_size_test_coverage.dat

out/out_prefix.err_est/sorted.err_dc_sample_size_test_coverage.dat

Estimated Coverage Error Bound Report Examples

The Adaptive Weighted Random Sampling (AWRS) test coverage example in [Figure 22](#) shows the error margins (x-axis) for each sample size (key) and coverage percentage (y-axis):

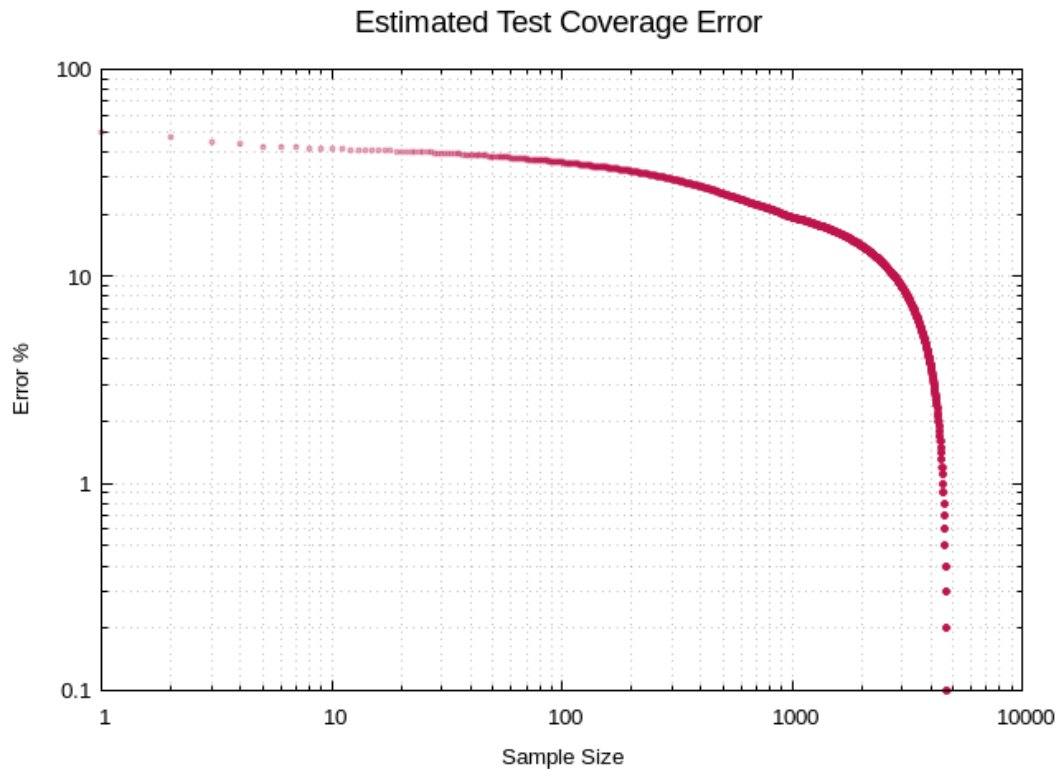
Figure 22 *AWRS Test Coverage Example*



This example shows that considering the coverage percentage, the estimated error bounds for a sample size of 200 fall within $\pm 6\%$. If you want a tighter error bound, you could choose a sample size of 3200, for which the error falls within $\pm 1\%$. Having a way to visualize the error also lets you see what the error is for intermediate sample sizes. In this case, a sample size of 1600 gives an accurate estimated coverage percentage. For faster throughput, consider a sample size of 400. To see the exact error bounds for each coverage percentage and sample size, refer to the `.dat` files.

The Weight Sorted Sampling (WSS) test coverage example in [Figure 23](#) shows the Error versus Sample size as a scatter plot:

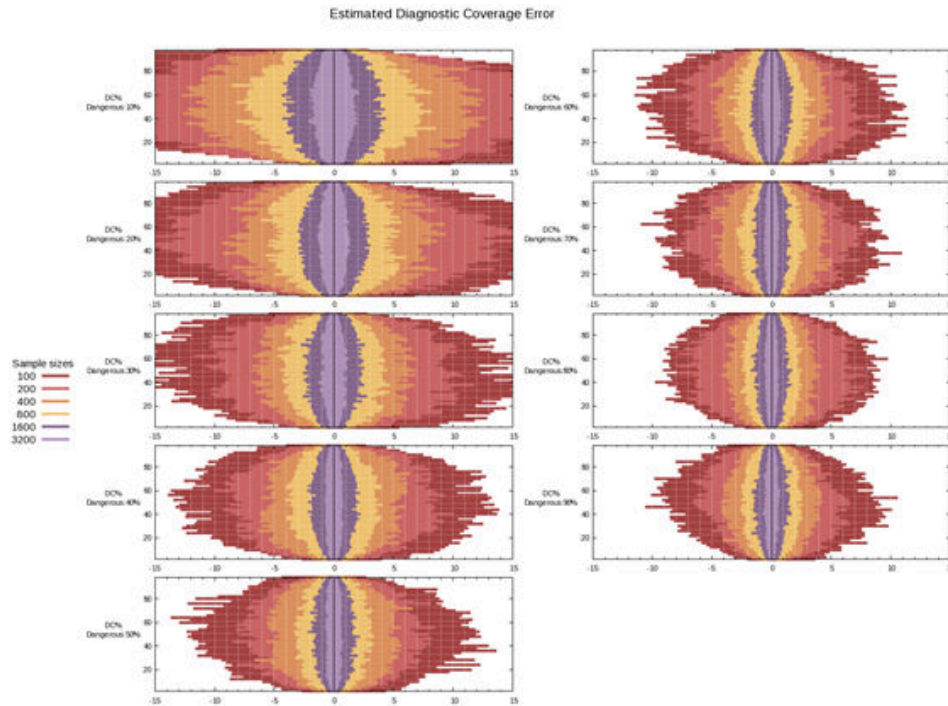
Figure 23 WSS Test Coverage Example



This example shows the estimated absolute error percentage versus the sample size for a WSS test coverage campaign. It can be seen that to achieve 10% error, about 2800 samples are required. To achieve 1% error requires about 4500 samples, and about 4800 samples yields 0.1% error.

The AWRS diagnostic coverage example in Figure 24 shows the error margins (x-axis) for each sample size (key) and coverage percentage (y-axis):

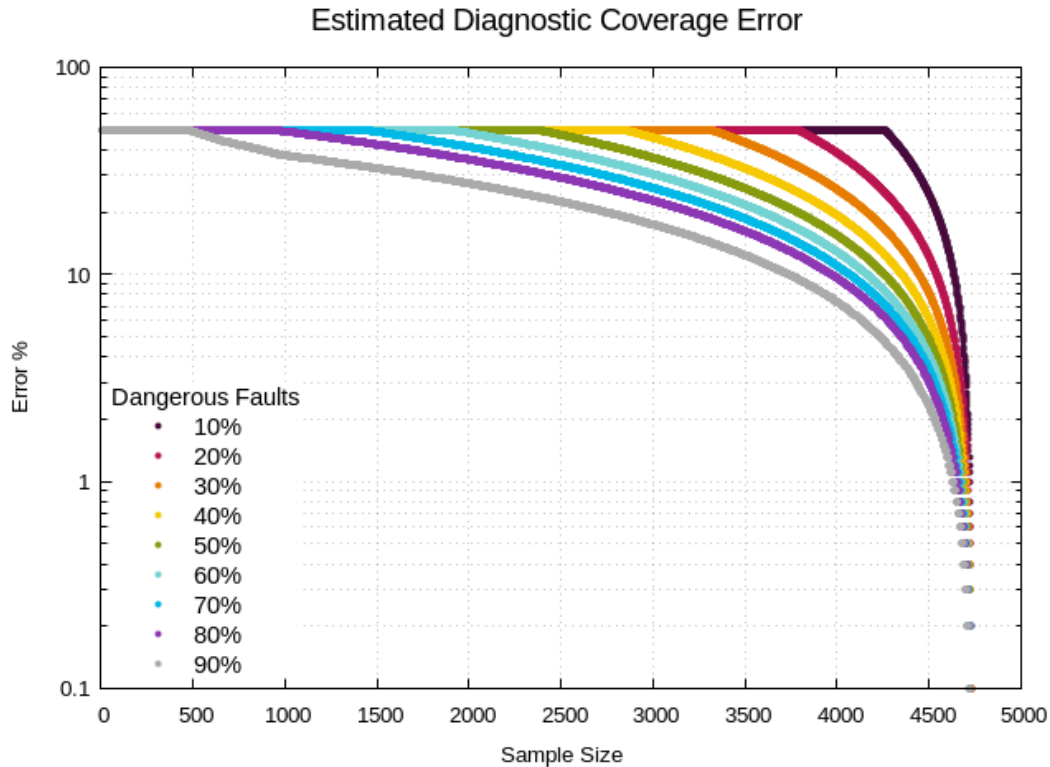
Figure 24 AWRS Diagnostic Coverage Example



For diagnostic coverage, multiple plots are generated on 10% intervals of dangerous defect percentages. Each subplot functions in the same way as the test coverage plot, but now you are provided with the additional dimension of dangerous defect percentage to consider. This example shows that for 10% dangerous defects, the estimated error bounds for a sample size of 1600 fall within $\pm 5\%$. Alternatively for 80% dangerous defects, a sample size of 400 is sufficient to fall within $\pm 5\%$ error. To see the exact error bounds for each coverage percentage, sample size, and dangerous defect percentage, refer to the .dat files.

The WSS test coverage example in [Figure 25](#) shows the Error versus Sample size for different dangerous defect percentages:

Figure 25 WSS Diagnostic Coverage Example



For diagnostic coverage, multiple lines are generated at 10% intervals of dangerous defect percentages. For example, if 90% of defects are dangerous, a 1% error can be achieved with about 4600 samples. However, if only 10% of defects are dangerous, about 4800 samples are required.

Using Adaptive Weighted Random Sampling (AWRS)

Sampling is a method of collecting a portion of data points from the population to draw conclusions of the characteristics of that population. The objective of the sampling is to accurately obtain the best possible estimated coverage (closest to actual coverage) of the population characteristics in a reasonable amount of time.

The accuracy of estimated coverage is based on the number of samples you selected. The more samples you select, the more accurate the estimated coverage is. If all samples are selected for simulation, it is referred to as an exhaustive fault campaign, and that gives

you the actual coverage. Any sampling less than an exhaustive fault campaign gives you an estimated coverage.

The Custom Fault tool uses an Adaptive Weighted Random Sampling (AWRS) technology for selecting defect samples. The likelihood of a defect being selected is determined by its relative weight, which is the weight of the defect with respect to the total weight of all defects. The advantage of AWRS is it can take input from the circuit designer to identify the most likely defects and adapt the weights automatically to meet the sampling requirements.

You can specify the weight for each individual defect or defect type. See [Specifying Weights for Defects](#) for details about how to define weight for each defect type. Based on the weight file, the Custom Fault tool calculates the weight for each defect ($W_1, W_2, \dots, W_n$), where n is the total number of defects. The weight of each defect is printed in the coverage report.

The Custom Fault tool supports fault collapsing using the `enable_fault_collapse` command. Fault collapsing is an approach to group the equivalent defects that produce the same behavior for all input patterns. Only one defect from the group must be simulated, so this approach significantly decreases the number of defects to simulate. In the case of fault collapsing, the group weight is considered. Each defect group carries the total weight of all equivalent defects in that group ($W_1, W_2, \dots, W_m$), where m is the total number of defect groups and is always less than or equal to n . The weight of each defect group is printed in the coverage report.

You must also provide the sample size (`#sample`), where `#sample` is less than n . The probability of sampling the i -th defect is calculated based on the following equation:

$$P_i = \frac{\#sample \times W_i}{\sum_{i=1}^{n \text{ or } m} W_i}$$

The Custom Fault tool uses AWRS to scale the weights of the defect universe and the defect selection probabilities to ensure that the number of simulated defects is close to but may not match exactly the number requested. This scaling is dependent on the weight distribution and the needed sample size. When the weight of a defect or a defect group is significantly larger than others, the defect or defect group might always be selected during sampling.

This is described further in the following topics:

- [Specifying a New Seed for Adaptive Weighted Random Sampling \(AWRS\)](#)
- [Weighted Coverage Report With Adaptive Weighted Random Sampling \(AWRS\)](#)
- [Using Incremental Sampling](#)

Specifying a New Seed for Adaptive Weighted Random Sampling (AWRS)

The Custom Fault tool uses a constant fixed seed for Adaptive Weighted Random Sampling (AWRS) sampling. When rerunning a simulation with the same number of samples, the same defects are selected and fault campaign results can be reproduced.

You can change the seed value to a different integer or to a random value using the `sampling` command:

```
"sampling" : {  
    "seed" : seed_value  
}
```

By default, the seed value is 123456789. You can specify another integer to use as the seed value, or specify the string "random" (including the quotation marks) to set a random seed value.

For example, you can change the seed value to 876540 as follows:

```
"sampling" : {  
    "seed" : 876540  
}
```

In this example, the log file prints the seed number used for the fault simulation as:

```
Sampling Seed : 876540
```

You can also set a random seed value as follows:

```
"sampling" : {  
    "seed" : "random"  
}
```

Weighted Coverage Report With Adaptive Weighted Random Sampling (AWRS)

Coverage percentage is calculated based on the following equation:

$$\text{Coverage(\%)} = \frac{\sum_{i=1}^n W_i \times C_i}{\sum_{i=1}^n W_i} \times 100$$

$C_i=0$ if i th is uncovered and $C_i=1$ if i th is covered

The coverage percentage is printed in the Coverage Summary section of the coverage report. It contains the weighted summary of the simulated and covered defects.

If an exhaustive fault campaign is applied, the coverage percentage is the actual coverage percentage. If any sample taken is less than an exhaustive fault campaign, the coverage percentage is an estimated coverage. If an estimated coverage is reported, an estimated error bound is also reported. The estimated coverage reported by the Custom Fault tool is based on a 95% confidence interval. A confidence interval is the probability that a value falls between an upper and lower bound of a probability distribution. A 95% confidence interval means that if you were to repeat the sampling method 100 times on the same case, 95 times the reported confidence intervals is expected to include the correct coverage.

Figure 26 shows an example of the Defect Universe Summary section of the coverage report for an exhaustive fault campaign (with tag name of `sim_with_weight`). It shows the type and number of defects. The Coverage Summary section shows the weighted coverage of covered and simulated defects. The total coverage in the exhaustive fault campaign (1520 samples) is 73.54% based on the weight specified in the weight file.

Figure 26 Defect Universe Summary Section of the Coverage Report for an Exhaustive Fault Campaign

```
*DEFECT UNIVERSE SUMMARY

mos_open      656
mos_short     864

TOTAL         1520

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
Type           sim_with_weight

mos_open      4545/7760
mos_short     8545/10040
TOTAL        13090/17800

Weighted test coverage:
sim_with_weight      73.54%

TOTAL Coverage:
Non-Sampled          73.54%

Total weight of fault universe: 17800.00
```

Figure 27 shows an example of the Defect Universe Summary section of the coverage report when sampling and weighting are enabled without fault collapsing (`"enable_fault_collapse" : false`). A sample size of 800 (with tag name of `sim_with_weight_s800`) is taken in this simulation. The Covered Summary section reports the scaled weight of the covered defect over simulated defects. This scaling factor depends on the weight distribution and the needed sample size. The estimated coverage with 800 samples is 73.73% with 95% confidence interval between 71.82% and 75.54%.

Figure 27 Defect Universe Summary Section of the Coverage Report for 800 Samples Without Fault Collapsing

```
*DEFECT UNIVERSE SUMMARY

      mos_open      656
      mos_short     864
      TOTAL         1520

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
      Type      sim_with_weight_s800

      mos_open      240,305/416,888
      mos_short     493,696/578,61
      TOTAL         734/995,497

Weighted test coverage:
      sim_with_weight_s800      73.73% (sample size: 800) (95% Confidence Interval: (71.82%,75.54%))

TOTAL Coverage:
      Sampled      73.73% (sample size: 800) (95% Confidence Interval: (71.82%,75.54%))

Total weight of fault universe: 17800.00
```

Figure 28 shows an example of the Defect Universe Summary section of the coverage report when sampling and weighting are enabled with fault collapsing ("enable_fault_collapse" : true). In this example, sample size of 800 is specified but only 796 samples are simulated. See [Using Adaptive Weighted Random Sampling \(AWRS\)](#) for an explanation of why the simulated sample is less than requested sample. When fault collapsing is enabled, the Custom Fault tool groups all the similar defects. If the grouped defect is sampled, the sample taken is 1 but the simulated defect is the total number of defects in the group. The estimated coverage with 796 samples is 73.24% with 95% confidence interval between 72.09% and 74.44%.

Figure 28 Defect Universe Summary Section of the Coverage Report for 800 Samples With Fault Collapsing

```
*DEFECT UNIVERSE SUMMARY

      mos_open      656
      mos_short     864
      TOTAL        1520

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
      Type      sim_with_weight_s800

      mos_open      441,823/762,317
      mos_short     837,639/984,642
      TOTAL        1279,46/1746,96

Weighted test coverage:
sim_with_weight_s800      73.24% (sample size: 796) (95% Confidence Interval: (72.09%,74.44%))

TOTAL Coverage:
      Sampled      73.24% (sample size: 796) (95% Confidence Interval: (72.09%,74.44%))

Total weight of fault universe: 17800.00
```

Using Incremental Sampling

A common use model for the Custom Fault tool is to first run a small number of samples to test your setup, then when you are sure the setup is correct and the results make sense, you can run more samples. The Custom Fault tool supports *incremental sampling* which allows you to save time by reusing the previous test run results, combining them with your later campaigns, as follows:

1. For your initial test, run the Custom Fault tool with a small sample size using the `-s|--size` command-line option.
2. For subsequent campaigns, use the `-s|--size` command-line option with a larger sample size, along with the `-i|--incremental` command-line option to enable incremental sampling (see [Command-Line Options](#)).

If the sample size for the second campaign is greater than the sample size of the first one, the second campaign only runs the difference in samples and combines the results with those of the first.

In the following example, the Custom Fault tool is run the first time with 100 samples. The second campaign uses incremental sampling, so even though 150 samples are specified, only 50 are run because the results of the first run's 100 sample results are merged with the second run's 50 sample results for a total of 150 results.

```
customfault -c config.json -t fault -o CF/fault_s100 -k 5 -s 100
customfault -c config.json -t fault -o CF/fault_s150 -k 2 -s 150 -f
CF/fault_s100.fdb -i
```

You can use incremental sampling with multiple campaigns so long as the sample size for subsequent campaigns keeps getting progressively larger.

In the following example, the Custom Fault tool is run the first time with 100 samples. As in the previous example, the second campaign uses incremental sampling, so only 50 more samples are run for a total of 150 results. The third campaign also uses incremental sampling, so even though 250 samples are specified, only 100 are run because the results of the first and second run's combined 150 sample results are merged with the third run's 100 sample results for a total of 250 results.

```
customfault -c config.json -t fault -o CF/fault_s100 -k 5 -s 100
customfault -c config.json -t fault150 -o CF/fault_s150 -k 2 -s 150 -f
CF/fault_s100.fdb -i
customfault -c config.json -t fault250 -o CF/fault_s250 -k 2 -s 250 -f
CF/fault_s150.fdb -i
```

If the sample size for a later campaign is smaller than the earlier sample size, then the Custom Fault tool does not combine the results.

In the following example, the Custom Fault tool is run the first time with 100 samples. The second campaign only specifies a sample size of 50, so even though incremental sampling is indicated and the first campaign has 100 results, the coverage data are recalculated for 50 samples and CF/fault_s50.fdb and CF/fault_s50.report files are created with 50 sample results.

```
customfault -c config.json -t fault100 -o CF/fault_s100 -k 5 -s 100
customfault -c config.json -t fault50 -o CF/fault_s50 -k 2 -s 50 -f
CF/fault_s100.fdb -i
```

Using Weight Sorted Sampling (WSS)

The primary reason for using Weight Sorted Sampling (WSS) is to have an absolute error bound on the coverage result. The defects with the highest weight contribute the most to the coverage result and error in the result. In this method, the defects are sorted by weight and the highest weight defects are simulated first. You must provide a maximum error to the tool via the [sampling](#) command. As the simulation results accumulate, the coverage and absolute error is calculated. As soon as the error is less than your requested maximum error, the fault campaign is halted, and the reports are produced.

WSS is supported for test coverage and diagnostic coverage of residual faults. It is not supported for diagnostic coverage of latent faults.

Note:

A fault campaign conducted with WSS cannot be guaranteed to be reproducible. While the defects are sorted by weight, and high weight defects are sent for simulation first, the defects are normally analyzed in parallel on a compute farm with many defects being evaluated simultaneously.

Therefore, due to variation and load on the compute farm, there are race conditions between different defects as the error reduces to your error target.

This is described further in the following topics:

- [Test Coverage With Weight Sorted Sampling \(WSS\)](#)
- [Diagnostic Coverage of Residual Faults With Weight Sorted Sampling \(WSS\)](#)

Test Coverage With Weight Sorted Sampling (WSS)

For test coverage, the Custom Fault tool determines coverage and error calculation as follows:

C = weight covered so far
N = weight not covered so far, that is faults with a result of NC or F
L = weight left to be evaluated
T = total weight of defect universe; $T = C + N + L$

$\text{MaxCoverage} = (C + L) / T$
 $\text{MinCoverage} = C / T$
 $\text{Coverage} = 0.5 * (\text{MaxC} + \text{MinC}) = (C + L/2) / T$
 $\text{Error} = 0.5 * (\text{MaxC} - \text{MinC}) = 0.5 * L / T$

Note:

Coverage is reported as the midpoint between the maximum and minimum possible coverage.

When using the [Custom Fault Multi-Testbench Flow](#) in test coverage, the following additional rules apply to determine whether a fault is covered, not covered, or contributing to the error:

- If in any testbench, the fault status is covered, then the fault is covered.
- If in all testbenches evaluated so far, the fault status is not covered, then the fault is not covered.
- If the fault has NS (not simulated) or F (simulation failed) status, then it cannot be conclusively shown that the fault is covered or not covered and therefore contributes to the error.

Diagnostic Coverage of Residual Faults With Weight Sorted Sampling (WSS)

For diagnostic coverage of residual fault analysis, the Custom Fault tool determines diagnostic coverage and error calculation as follows:

In diagnostic coverage (K), the fault status can be Safe Detected, Safe Undetected, Dangerous Detected, or Dangerous Undetected.

K is a function only of dangerous faults:

```
L = weight of faults not yet simulated
D = weight of Dangerous Detected so far
U = weight of Dangerous Undetected so far
K = D / (D + U)
```

Therefore, K is maximum when all remaining weight is D (dangerous detected):

$$K_{\max} = (D + L) / (D + L + U)$$

K is minimum when all remaining weight is U (dangerous undetected):

$$K_{\min} = D / (D + L + U)$$

With sorted sampling, K is defined to be the midpoint of the minimum and maximum:

$$K = 0.5 * (K_{\max} + K_{\min})$$
$$K = 0.5 * (2D + L) / (D + L + U)$$

Therefore, the absolute error bound is:

$$E = 0.5 * (K_{\max} - K_{\min})$$
$$E = 0.5 * L / (D + L + U)$$

Note:

Faults evaluated to have a status of safe do not contribute to the diagnostic coverage nor the error term.

In diagnostic coverage analysis, there may be more than one failure mode being analyzed in a fault campaign. When Weight Sorted Sampling (WSS) is used, the diagnostic coverage and error is calculated for each failure mode. The fault campaign is stopped only when the error of each failure mode is less than your specified maximum error.

Adaptive Weighted Random Sampling (AWRS) Tutorial

This tutorial demonstrates how to enable the Adaptive Weighted Random Sampling (AWRS) weighting and sampling feature in the Custom Fault tool. It disables fault

collapsing with the `"enable_fault_collapse" : false` command. You can enable fault collapsing by commenting out the command as follows:

```
// "enable_fault_collapse" : false,
```

When fault collapsing is enabled, group weight is considered rather than individual weight.

This tutorial covers:

- [Generating a Template Weight File](#)
- [Running an Exhaustive Fault Campaign With a Weight File](#)
- [Generating an Error Bound Report](#)
- [Running Different Sample Sizes With a Weight File](#)

Before you start this tutorial, make sure that all tools are installed correctly. [Table 8](#) lists the files needed for this tutorial. These files are located in the `install_directory/doc/tutorials/weight_and_sample` directory.

Table 8 *Weighting and Sampling Tutorial Files*

File	Description
<code>adder.vec</code>	Vector file containing both input and output vectors.
<code>config_identify_only_xa</code>	Configuration file to run the Custom Fault tool with the PrimeSim XA tool to generate the template weight file.
<code>config_identify_only_fsim</code>	Configuration file to run the Custom Fault tool with the FineSim tool to generate the template weight file.
<code>config_identify_only_psim</code>	Configuration file to run the Custom Fault tool with the PrimeSim tool to generate the template weight file.
<code>config_with_weight_xa</code>	Configuration file to include weight file and run the Custom Fault tool with the PrimeSim XA tool.
<code>config_with_weight_fsim</code>	Configuration file to include weight file and run the Custom Fault tool with the FineSim tool.
<code>config_with_weight_psim</code>	Configuration file to include weight file and run the Custom Fault tool with the PrimeSim tool.
<code>config_with_weight_errbound_xa</code>	Configuration file to run the Custom Fault tool with the PrimeSim XA tool to generate the error bound report.
<code>config_with_weight_errbound_fsim</code>	Configuration file to run the Custom Fault tool with the FineSim tool to generate the error bound report.
<code>config_with_weight_errbound_psim</code>	Configuration file to run the Custom Fault tool with the PrimeSim tool to generate the error bound report.

Table 8 *Weighting and Sampling Tutorial Files (Continued)*

File	Description
<code>run_identify_only</code>	Run script to generate the template weight file. It contains the command lines to run the Custom Fault tool with the PrimeSim XA, FineSim, and PrimeSim tools.
<code>run_with_weight</code>	Run script to run an exhaustive fault campaign with a weight file. It contains command lines to run the Custom Fault tool with the PrimeSim XA, FineSim, and PrimeSim tools.
<code>run_with_weight_errbound</code>	Run script to generate the error bound report. It contains command lines to run the Custom Fault tool with the PrimeSim XA, FineSim, and PrimeSim tools.
<code>run_with_weight_sample</code>	Run script to run with different sample sizes with a weight file. It contains command lines to run the Custom Fault tool with the PrimeSim XA, FineSim, and PrimeSim tools.
<code>top.spi</code>	Top-level netlist of a 4-bit adder.
<code>weight/weight_file.cpp</code>	Weight file.
<code>xhost/xhost_sh_launch</code>	Host file for distributing processing.

The defect detection checking criteria runs a vector check. The `adder.vec` file contains the output vectors of `COUT` and `S[3:0]`. If the simulation output matches with the expected states, then no violation is reported, and the defect is not covered. Otherwise, a violation is reported, and the defect is covered.

Generating a Template Weight File

The Custom Fault tool, by default, generates a design-specific template weight file when you run a simulation. The template weight file contains C++ functions to define weights for each defect. You can use the template weight file to specify the weight for each defect type using the parameters supported in your design. The supported parameters are listed in the template weight file.

To speed up the generation of the template weight file, use the following command to exit after defect identification:

```
"identify_only" : true
```

To invoke the Custom Fault tool to generate the template weight file, run the `run_identify_only` script:

```
$> run_identify_only
```


The script runs the following simulations:

- The Custom Fault tool with the PrimeSim XA simulator with the output files directed to the `RESULT_XA` directory.
- The Custom Fault tool with the FineSim simulator with the output files directed to the `RESULT_FSIM` directory.
- The Custom Fault tool with the PrimeSim simulator with the output files directed to the `RESULT_PSIM` directory.

The Custom Fault tool generates the `identify_only_weights.cpp` template weight file as:

```
double mos_open_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    // (model: n) model_name, w, l, gate_cap, m, instance_name
    // (model: p) model_name, w, l, gate_cap, m, instance_name
    GET_DOUBLE_VAR(gate_cap);
    GET_DOUBLE_VAR(l);
    GET_DOUBLE_VAR(m);
    GET_DOUBLE_VAR(w);
    GET_STRING_VAR(instance_name);
    GET_STRING_VAR(model_name);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;

    return weight;
}

double mos_short_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    // (model: n) model_name, w, l, gate_cap, m, instance_name
    // (model: p) model_name, w, l, gate_cap, m, instance_name
    GET_DOUBLE_VAR(gate_cap);
    GET_DOUBLE_VAR(l);
    GET_DOUBLE_VAR(m);
    GET_DOUBLE_VAR(w);
    GET_STRING_VAR(instance_name);
    GET_STRING_VAR(model_name);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;

    return weight;
}
```

This template weight file only contains the weight functions for `mos_open` and `mos_short` because the design contains only MOSFET devices. If a design contains other types of

devices, the template weight file contains the weight functions of other defects. By default, all defects have a weight of 1. You can modify the functions to set the weight based on the supported parameters. The supported parameters are design-specific. In this case, the supported parameters for MOSFET are:

```
GET_DOUBLE_VAR(gate_cap);  
GET_DOUBLE_VAR(l);  
GET_DOUBLE_VAR(m);  
GET_DOUBLE_VAR(w);  
GET_STRING_VAR(instance_name);  
GET_STRING_VAR(model_name);
```

You can modify the functions to calculate the weight using the supported parameters (gate_cap, w, l, m, instance_name and model_name). The following example calculates the weight of mos_open to be $1e12*(w*l)$. If the calculated weight of mos_open is larger than 200, set the weight to 200. A return statement is mandatory to return the weight value.

```
double mos_open_weight(int scount, char ** skey, char ** sval,  
    int dcount, char ** dkey, double * dval)  
{  
    // (model: n) model_name, w, l, gate_cap, m, instance_name  
    // (model: p) model_name, w, l, gate_cap, m, instance_name  
    GET_DOUBLE_VAR(gate_cap);  
    GET_DOUBLE_VAR(l);  
    GET_DOUBLE_VAR(m);  
    GET_DOUBLE_VAR(w);  
    GET_STRING_VAR(instance_name);  
    GET_STRING_VAR(model_name);  
  
    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////  
    double weight=1.0;  
  
    weight = 1e12*(w*l);  
    if (weight > 200.0) weight=200.0;  
  
    return weight;  
}
```

Running an Exhaustive Fault Campaign With a Weight File

In this section, you run an exhaustive fault campaign using a weight file. An exhaustive fault campaign is always recommended to provide the actual coverage percentage. A sample of the weight file using width and length to calculate the weight of each defect is in weight/weight_file.cpp.

To include the weight file, specify the following line in the configuration file.

```
"defect_weight" : "weight/weight_file.cpp"
```

The content of the `weight/weight_file.cpp` configuration file is:

```
double mos_open_weight(int scout, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    // (model: n) model_name, w, l, gate_cap, m, instance_name
    // (model: p) model_name, w, l, gate_cap, m, instance_name
    GET_DOUBLE_VAR(gate_cap);
    GET_DOUBLE_VAR(l);
    GET_DOUBLE_VAR(m);
    GET_DOUBLE_VAR(w);
    GET_STRING_VAR(instance_name);
    GET_STRING_VAR(model_name);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;

    weight = 1e12*(w*l);
    if (weight > 200.0) weight=200.0;

    return weight;
}

double mos_short_weight(int scout, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    // (model: n) model_name, w, l, gate_cap, m, instance_name
    // (model: p) model_name, w, l, gate_cap, m, instance_name
    GET_DOUBLE_VAR(gate_cap);
    GET_DOUBLE_VAR(l);
    GET_DOUBLE_VAR(m);
    GET_DOUBLE_VAR(w);
    GET_STRING_VAR(instance_name);
    GET_STRING_VAR(model_name);

    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;

    weight = 1e12*(w*l);
    if (weight > 200.0) weight=200.0;

    return weight;
}
```

To invoke the Custom Fault tool for an exhaustive fault campaign with a weight file, run `run_with_weight`:

```
$> run_with_weight
```

This script runs two simulations:

- The Custom Fault tool with the PrimeSim XA tool and the output files in the `RESULT_XA` directory.
- The Custom Fault tool with FineSim tool and the output files in the `RESULT_FSIM` directory.
- The Custom Fault tool with PrimeSim tool and the output files in the `RESULT_PSIM` directory.

Figure 29 is the Detect Universe Summary section. It shows there are 1520 defects in total (656 `mos_open` and 864 `mos_short`).

Figure 29 Detect Universe Summary

```
*DEFECT UNIVERSE SUMMARY

mos_open          656
mos_short         864
TOTAL             1520
```

Figure 30 shows the actual weighted test coverage percentage of 73.54% in Coverage Summary section.

Figure 30 Weighted Test Coverage

```
Defects covered - weighted (covered/simulated):
Type          sim_with_weight

mos_open      4545/7760
mos_short     8545/10040
TOTAL         13090/17800

Weighted test coverage:
sim_with_weight          73.54%

TOTAL Coverage:
Non-Sampled              73.54%
```

Figure 31 shows a portion of the Detailed Coverage Listing section. It contains the detail of each defect including the defect id, defect type, status, weight and so on.

Figure 31 Detailed Coverage Listing

CU: Covered by Upper boundary
CL: Covered by Lower boundary
CM: Covered by Waveform comparison
CV: Covered by Vector comparison
CF: Covered by Failed measure - e.g. flat-lined signal
CE: Covered by WV expression
CX: Covered - not categorizable
NI: Not Covered - Invalid measure statement
NC: Not Covered
NS: Not Simulated
F: Not Covered - Simulation failed

Id	Defect	sim_with_weight	Simulation Data	Weight	Instance/Node(s)	Port Name(s)
R1.1	mos_short	CV	fault_mos_short_1.1.1.*	1,000000e+01	x1.x1.x2.mp1	x1.x1.x2.vdd, x1.x1.x2.in
R1.2	mos_short	CV	fault_mos_short_2.2.1.*	1,000000e+01	x1.x8.mp2	x1.x8.vdd, x1.x8.in2
R1.3	mos_short	CV	fault_mos_short_3.3.1.*	4,000000e+01	x1.x6.mp1	x1.x6.vdd, x1.x6.out
R1.4	mos_short	CV	fault_mos_short_4.4.1.*	4,000000e+01	x1.x6.mp1	x1.x6.vdd, x1.x6.out
R1.5	mos_short	CV	fault_mos_short_5.5.1.*	1,000000e+01	x1.x1.x2.mp1	x1.x1.x2.vdd, x1.x1.x2.in
R6.6	mos_short	CV	fault_mos_short_6.6.6.*	4,000000e+01	x1.x3.mp1	x1.x3.vdd, x1.x3.out
R6.7	mos_short	CV	fault_mos_short_7.7.6.*	4,000000e+01	x1.x4.mp1	x1.x4.vdd, x1.x4.in
R6.8	mos_short	CV	fault_mos_short_8.8.6.*	4,000000e+01	x1.x4.mp1	x1.x4.vdd, x1.x4.in
R6.9	mos_short	CV	fault_mos_short_9.9.6.*	4,000000e+01	x1.x3.mp1	x1.x3.vdd, x1.x3.out
A10.10	mos_short	NC	fault_mos_short_10.10.10.*	4,000000e+01	x1.x5.mp1	x1.x5.vdd, x1.x5.out
A10.11	mos_short	NC	fault_mos_short_11.11.10.*	4,000000e+01	x1.x6.mp1	x1.x6.vdd, x1.x6.in
A10.12	mos_short	NC	fault_mos_short_12.12.10.*	4,000000e+01	x1.x6.mp1	x1.x6.vdd, x1.x6.in
A10.13	mos_short	NC	fault_mos_short_13.13.10.*	4,000000e+01	x1.x5.mp1	x1.x5.vdd, x1.x5.out
A14.14	mos_short	CV	fault_mos_short_14.14.14.*	1,000000e+01	x1.x7.mp1	x1.x7.vdd, x1.x7.in
A14.15	mos_short	CV	fault_mos_short_15.15.14.*	1,000000e+01	x1.x9.mp2	x1.x9.vdd, x1.x9.out
A14.16	mos_short	CV	fault_mos_short_16.16.14.*	1,000000e+01	x1.x7.mp1	x1.x7.vdd, x1.x7.in
A17.17	mos_short	NC	fault_mos_short_17.17.17.*	2,000000e+01	x1.x5.mn1	x1.x5.0, x1.x5.in
A17.18	mos_short	NC	fault_mos_short_18.18.17.*	2,000000e+01	x1.x5.mn1	x1.x5.0, x1.x5.in
A19.19	mos_short	NC	fault_mos_short_19.19.19.*	2,000000e+01	x1.x3.mn1	x1.x3.0, x1.x3.in
A19.20	mos_short	NC	fault_mos_short_20.20.19.*	2,000000e+01	x1.x3.mn1	x1.x3.0, x1.x3.in
A21.21	mos_short	NC	fault_mos_short_21.21.21.*	1,000000e+01	x1.x1.mp1	x1.x1.vdd, x1.x1.n3

Generating an Error Bound Report

The Custom Fault tool can generate an estimated coverage error bound (%) report for various sampling sizes in the log file and standard output (see [Estimating Error and Sample Size](#)). This error bound report is important to guide you to the sample size to select for simulation in a specific design. This report shows you the estimated error bound for an estimated coverage percentage when a certain sample size is selected.

The command to enable error bound reporting is:

```
"sampling" : {
  "estimate_error" : true
}
```

The "identify_only" : true command is set to exit the simulation after defect identification.

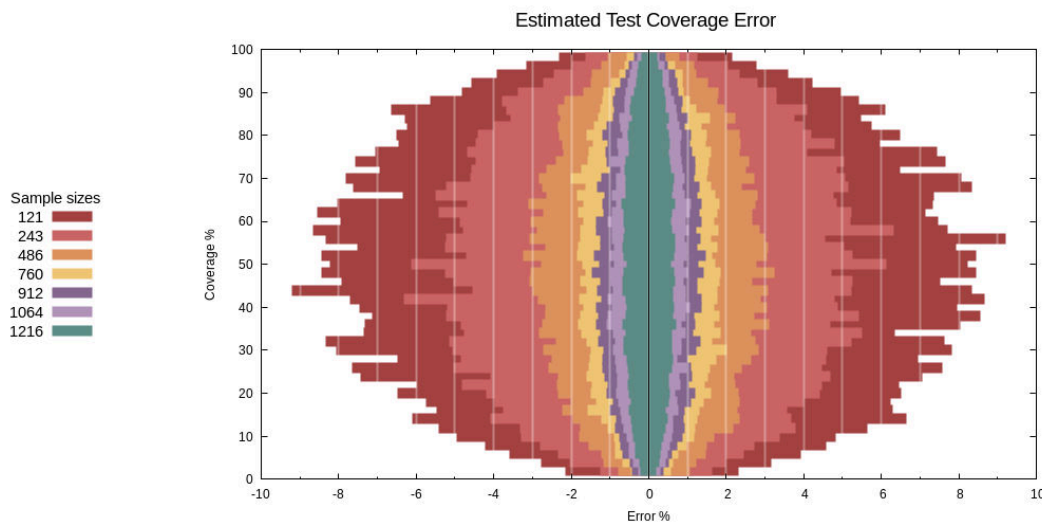
To invoke the Custom Fault tool to generate an error bound report, run the `run_with_weight_errbound` script:

```
$> run_with_weight_errbound
```

The report is in the form of a .png image and a subdirectory for .dat files. The .png image provides you with a way to visualize the error, while the .dat files list error margins for each sample size and coverage percentage. This report helps guide what sample size to select for simulation in a design.

Figure 32 shows the image produced by the Adaptive Weighted Random Sampling (AWRS) error bound report, a graph with a `.tc_err.png` extension: `with_weight_errbound.tc_err.png`. The image provides you the estimation of the error bound percentage and might be slightly different for a different binary.

Figure 32 Error Bound Report Image



The coverage error estimation data is output to a directory with a `.err.est` extension:

- PrimeSim XA: `RESULT_XA/with_weight_errbound.err_est`
- FineSim: `RESULT_FSIM/with_weight_errbound.err_est`
- PrimeSim: `RESULT_PSIM/with_weight_errbound.err_est`

In the `with_weight_errbound.err_est` directory, the report produces one `.dat` file per sample size:

```
err_tc_121.dat
err_tc_243.dat
err_tc_486.dat
err_tc_760.dat
err_tc_912.dat
err_tc_1064.dat
err_tc_1216.dat
```

This report shows the different error bound percentage for different sample sizes. This error bound report is only an estimation. The actual error bound percentage can only be obtained from the actual simulation.

Running Different Sample Sizes With a Weight File

In this section, you run the Custom Fault tool with different sample sizes by including a weight file. The sampling can be enabled with the `-s` command-line option or with the following command in the configuration file:

```
"sampling" : {  
    "size" : sample_size  
}
```

To invoke the Custom Fault tool using sampling with weight, run the `run_with_weight_sample` script:

```
$> run_with_weight_sample
```

This script run the following simulations:

- The Custom Fault tool with the PrimeSim XA tool with different sample sizes of 800, 400, and 200. The output files are in the `RESULT_XA` directory.
- The Custom Fault tool with the FineSim tool with different sample sizes of 800, 400, and 200. The output files are in the `RESULT_FSIM` directory.
- The Custom Fault tool with the PrimeSim tool with different sample sizes of 800, 400, and 200. The output files are in the `RESULT_PSIM` directory.

The content of `run_with_weight_sample` is as follows:

```
customfault -c config_with_weight_xa -o RESULT_XA/with_weight_s800 -t  
sim_with_weight_s800 -k 10 -s 800  
customfault -c config_with_weight_xa -o RESULT_XA/with_weight_s600 -t  
sim_with_weight_s600 -k 10 -s 600  
customfault -c config_with_weight_xa -o RESULT_XA/with_weight_s400 -t  
sim_with_weight_s400 -k 10 -s 400  
  
customfault -c config_with_weight_fsim -o RESULT_FSIM/with_weight_s800 -t  
sim_with_weight_s800 -k 10 -s 800  
customfault -c config_with_weight_fsim -o RESULT_FSIM/with_weight_s600 -t  
sim_with_weight_s600 -k 10 -s 600  
customfault -c config_with_weight_fsim -o RESULT_FSIM/with_weight_s400 -t  
sim_with_weight_s400 -k 10 -s 400  
  
customfault -c config_with_weight_psim -o RESULT_PSIM/with_weight_s800 -t  
sim_with_weight_s800 -k 10 -s 800  
customfault -c config_with_weight_psim -o RESULT_PSIM/with_weight_s600 -t  
sim_with_weight_s600 -k 10 -s 600  
customfault -c config_with_weight_psim -o RESULT_PSIM/with_weight_s400 -t  
sim_with_weight_s400 -k 10 -s 400
```

The coverage summary section of the RESULT_XA/with_weight_s800.report shows:

```
sim_with_weight_s800          73.73% (sample size: 800) (95%  
Confidence Interval: (71.82%,75.54%))
```

The coverage summary section of the RESULT_XA/with_weight_s600.report shows:

```
sim_with_weight_s600          73.98% (sample size: 564) (95%  
Confidence Interval: (71.1%,76.54%))
```

The coverage summary section of the RESULT_XA/with_weight_s400.report shows:

```
sim_with_weight_s400          70.85% (sample size: 398) (95%  
Confidence Interval: (66.33%,75.13%))
```

The coverage summary section of the RESULT_FSIM/with_weight_s800.report or
RESULT_PSIM/with_weight_s800.report shows:

```
sim_with_weight_s800          73.63% (sample size: 800) (95%  
Confidence Interval: (71.82%,75.54%))
```

The coverage summary section of the RESULT_FSIM/with_weight_s600.report or
RESULT_PSIM/with_weight_s600.report shows:

```
sim_with_weight_s600          73.12% (sample size: 586) (95%  
Confidence Interval: (70.37%,75.71%))
```

The coverage summary section of the RESULT_FSIM/with_weight_s400.report or
RESULT_PSIM/with_weight_s400.report shows:

```
sim_with_weight_s400          77.50% (sample size: 400) (95%  
Confidence Interval: (73.25%,81.5%))
```

Table 9 shows the Custom Fault with PrimeSim XA test coverage percentage and confidence interval for different sample sizes.

Table 9 Custom Fault With PrimeSim XA Test Coverage

Sample Size	Test Coverage (%)	95% Confidence Interval (%)
800	73.73	71.82-75.54
600	73.98	71.10-76.54
400	70.85	66.33-75.13

Table 10 shows the Custom Fault with FineSim or PrimeSim test coverage percentage and confidence interval for different sample sizes.

Table 10 Custom Fault With FineSim or PrimeSim Test Coverage

Sample Size	Test Coverage (%)	95% Confidence Interval (%)
800	73.63	71.82-75.54
600	73.98	70.37-75.71
400	70.85	73.25-81.50

Different sample sizes are set to show you the following:

- The error bound percentage closely matches with the estimated error bound percentage generated in the previous test. The estimated error bound percentage is around 2.51% for a sample size of 800. The actual error bound percentage for a sample size 800 is about 2%.
- The 95% confidence interval at different sample sizes. The margin gets larger as the sample size decreases.
- The test coverage percentage at different sample sizes. The estimated test coverage percentage gets more accurate (closer to the actual test coverage percentage when an exhaustive fault campaign is used).

Weighted Sorted Sampling (WSS) Tutorial

This tutorial shows how to use the Custom Fault tool to run weighted sorted sampling to achieve faster fault campaigns.

The tutorial described in this section demonstrates the following:

- [Running the WSS Tutorial With PrimeSim XA](#)
- [Running the WSS Tutorial With FineSim](#)
- [Running the WSS Tutorial With PrimeSim](#)

Before starting the tutorial, make sure that all tools are installed correctly.

[Table 11](#) lists the files needed for this tutorial. These files are in the `install_directory/doc/tutorials/wss_sampling` directory.

Table 11 Weighted Sorted Sampling (WSS) Tutorial Files

File	Description
<code>adder.vec</code>	Vector file containing both input and output vectors.

Table 11 *Weighted Sorted Sampling (WSS) Tutorial Files (Continued)*

File	Description
<code>top_no_vec_xa.spi</code>	Top-level netlist of a 4-bit adder for the PrimeSim XA tool.
<code>top_fsim.spi</code>	Top-level netlist of a 4-bit adder for the FineSim tool.
<code>top_psim.spi</code>	Top-level netlist of a 4-bit adder for the PrimeSim tool.
<code>xhost/xhost_sh_launch</code>	Host file for distributed processing.
<code>cf_config_est_xa.json</code>	Custom Fault configuration file for error estimation for the PrimeSim XA simulator.
<code>cf_config_est_fs.json</code>	Custom Fault configuration file for error estimation for the FineSim simulator.
<code>cf_config_est_ps.json</code>	Custom Fault configuration file for error estimation for the PrimeSim simulator.
<code>cf_config_xa.json</code>	Custom Fault configuration file for the PrimeSim XA simulator.
<code>cf_config_fs.json</code>	Custom Fault configuration file for the FineSim simulator.
<code>cf_config_ps.json</code>	Custom Fault configuration file for the PrimeSim simulator.
<code>run_cf_est_xa</code>	Run script to run the error estimation with PrimeSim XA simulator.
<code>run_cf_est_fs</code>	Run script to run the error estimation with the FineSim simulator.
<code>run_cf_est_ps</code>	Run script to run the error estimation with the PrimeSim simulator.
<code>run_cf_xa</code>	Run script to run WSS with the PrimeSim XA simulator.
<code>run_cf_fs</code>	Run script to run WSS with the FineSim simulator.
<code>run_cf_ps</code>	Run script to run WSS with the PrimeSim simulator.
<code>README</code>	README file with some information on the tutorial.
<code>weight.cpp</code>	Weight file.

Note:

You can edit the `xhost/xhost_sh_launch` file and modify it to use your own Distributing Processing.

Running the WSS Tutorial With PrimeSim XA

The WSS flow is invoked using the `sampling` command. You must specify "sorted" for sampling method and a value for "max_error".

Estimating Error

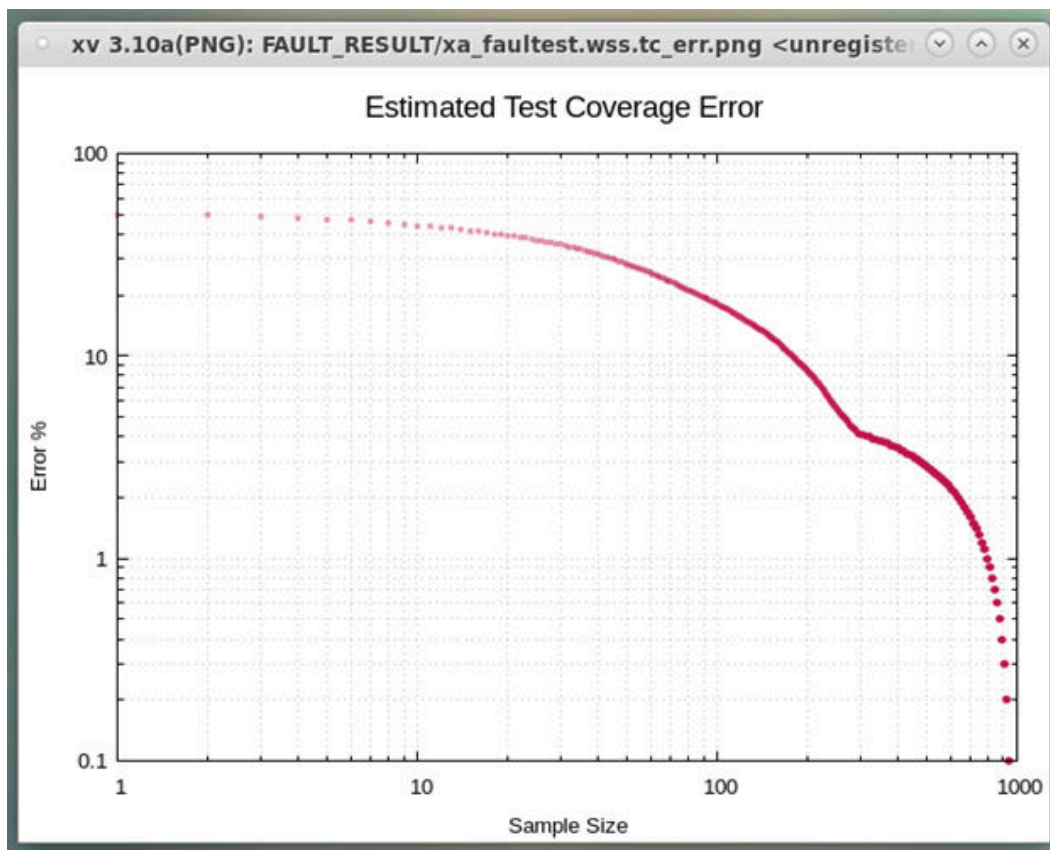
To estimate the error, you must also add "estimate_error" : true in the configuration file. The following is an excerpt of `cf_config_est_xa.json`:

```
"sampling" : {  
  "method" : "sorted",  
  "max_error" : 5,  
  "estimate_error" : true  
},
```

Run the error estimation using the run script `run_cf_est_xa`.

The Custom Fault tool runs the WSS error estimation and generates the error graph in `RESULT/xa_defectest.wss.tc_err.png`.

Figure 33 WSS Tutorial With PrimeSim XA: Error Estimation



Running the Fault Campaign

Run the fault campaign using the script `run_cf_xa`.

The configuration file specifies the sorted sampling and an error of 5%. The following is an excerpt of `cf_config_xa.json`:

```
"sampling" : {  
  "method" : "sorted",  
  "max_error" : 5  
},
```

The simulations stop after the error is below 5%; the log file shows the following:

Using wss sampling method with error upper bound of 5.00%

```
Done: 0 Running: 0 Queued: 20 Pending: 934  
Printing intermediate fdb and report files ...  
Warning: Stopping fault simulation as coverage error is 4.99183% which is  
less than 5.00%
```

The `RESULT/xa_defect.report` file shows the following:

Tags	Sample-Size
xa_defect	WSS Max Error = 5%


```
*DEFECT UNIVERSE SUMMARY
```

mos_open	656
mos_short	864
TOTAL	1520


```
*COVERAGE SUMMARY
```

Defects covered - weighted (covered/simulated):

Type	xa_defect
mos_open	0/0
mos_short	4454.5/7159
TOTAL	4454.5/7159

Weighted test coverage:

xa_defect	61.00% Error = +/- 4.99%
-----------	--------------------------

TOTAL Coverage:

WSS-Sampled	61.00% Error = +/- 4.99%
-------------	--------------------------

Running the WSS Tutorial With FineSim

The WSS flow is invoked using the `sampling` command. You must specify "sorted" for sampling method and a value for "max_error".

Estimating Error

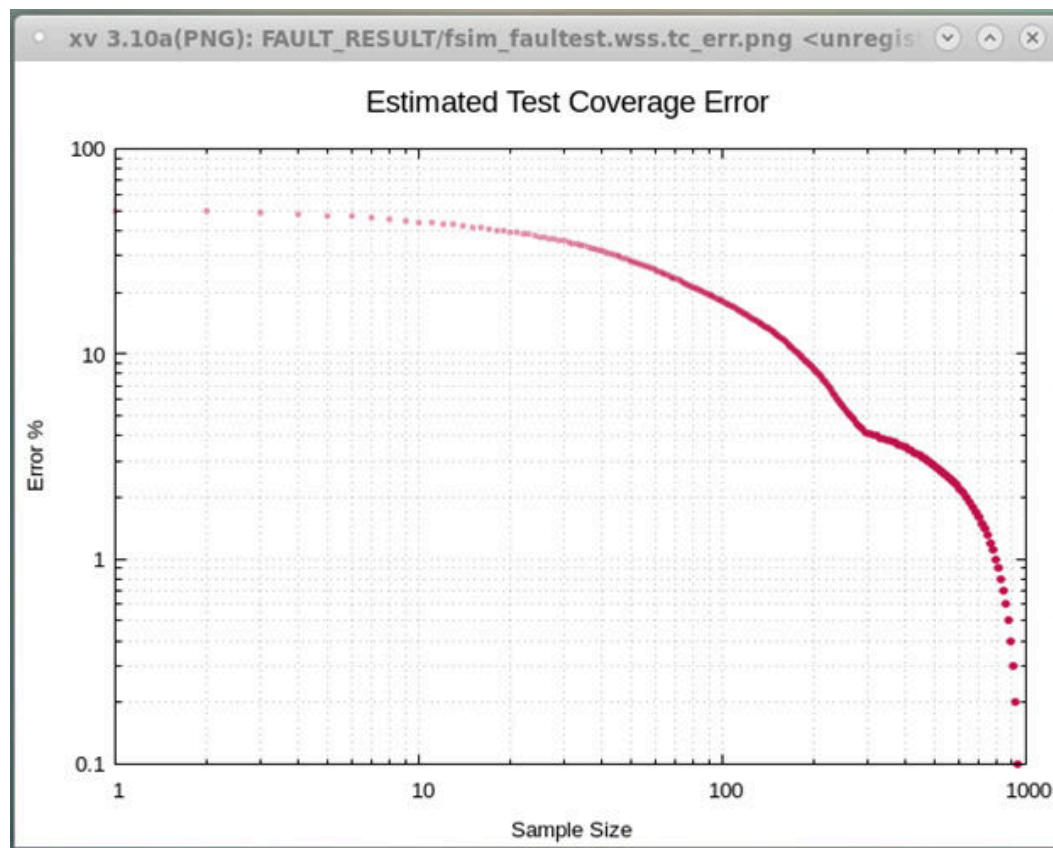
To estimate the error, you must also add "estimate_error" : true in the configuration file. The following is an excerpt of `cf_config_est_fsim.json`:

```
"sampling" : {  
  "method" : "sorted",  
  "max_error" : 5,  
  "estimate_error" : true  
},
```

Run the error estimation using the run script `run_cf_est_fsim`.

The Custom Fault tool runs the WSS error estimation and generates the error graph in `RESULT/fsim_defectest.wss.tc_err.png`.

Figure 34 WSS Tutorial With FineSim: Error Estimation



Running the Fault Campaign

Run the fault campaign using the script `run_cf_fsim`.

The configuration file specifies the sorted sampling and an error of 5%. The following is an excerpt of `cf_config_fsim.json`:

```
"sampling" : {
  "method" : "sorted",
  "max_error" : 5
},
```

The simulations stop after the error is below 5%; the log file shows the following:

Using wss sampling method with error upper bound of 5.00%

Done: 0 Running: 0 Queued: 20 Pending: 934

Printing intermediate fdb and report files ...

Warning: Stopping fault simulation as coverage error is 4.99183% which is less than 5.00%

The `RESULT/fsim_defect.report` file shows the following:

Tags	Sample-Size
fsim_defect	WSS Max Error = 5%


```
*DEFECT UNIVERSE SUMMARY
```

mos_open	656
mos_short	864
TOTAL	1520


```
*COVERAGE SUMMARY
```

Defects covered - weighted (covered/simulated):

Type	fsim_defect
mos_open	0/0
mos_short	4438/7159
TOTAL	4438/7159

Weighted test coverage:

fsim_defect	60.79% Error = +/- 4.99%
-------------	--------------------------

TOTAL Coverage:

WSS-Sampled	60.79% Error = +/- 4.99%
-------------	--------------------------

Running the WSS Tutorial With PrimeSim

The WSS flow is invoked using the `sampling` command. You must specify "sorted" for sampling method and a value for "max_error".

Estimating Error

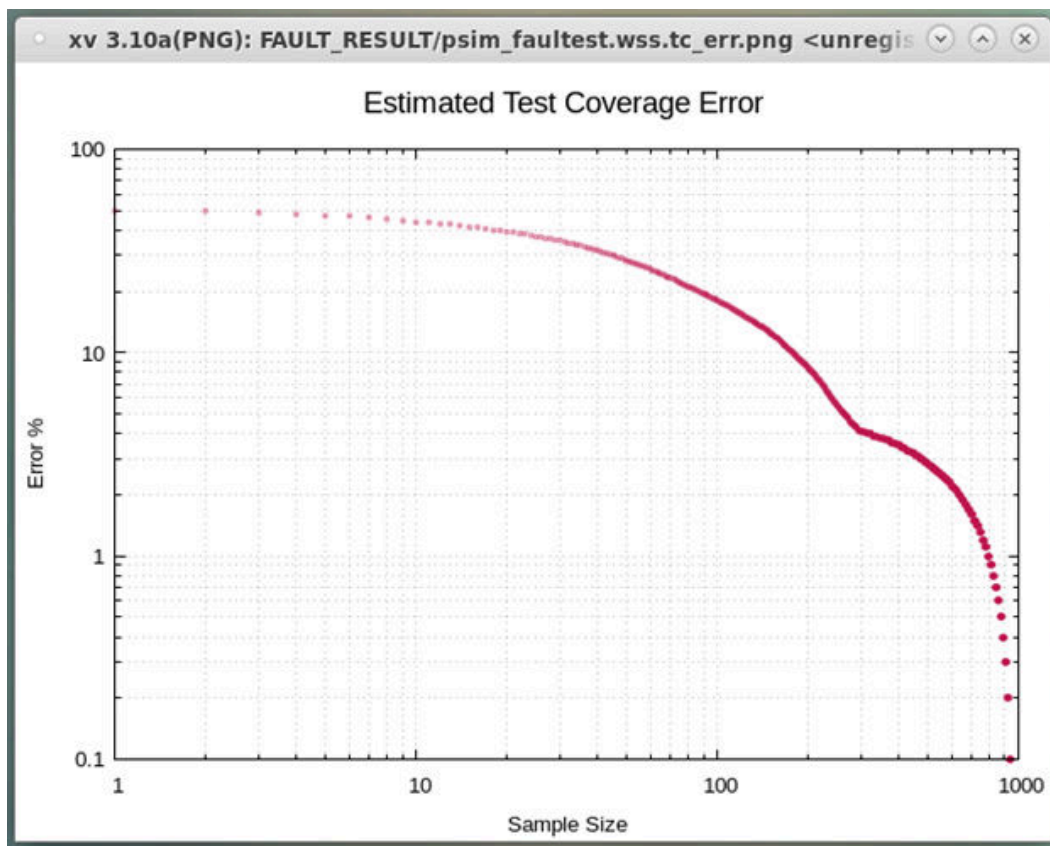
To estimate the error, you must also add "estimate_error" : true in the configuration file. The following is an excerpt of `cf_config_est_psim.json`:

```
"sampling" : {  
  "method" : "sorted",  
  "max_error" : 5,  
  "estimate_error" : true  
},
```

Run the error estimation using the run script `run_cf_est_psim`.

The Custom Fault tool runs the WSS error estimation and generates the error graph in `RESULT/psim_defectest.wss.tc_err.png`.

Figure 35 WSS Tutorial With PrimeSim: Error Estimation



Running the Fault Campaign

Run the fault campaign using the script `run_cf_psim`.

The configuration file specifies the sorted sampling and an error of 5%. The following is an excerpt of `cf_config_psim.json`:

```
"sampling" : {  
  "method" : "sorted",  
  "max_error" : 5  
},
```

The simulations stop after the error is below 5%; the log file shows the following:

```
Using wss sampling method with error upper bound of 5.00%
```

```
Done: 0 Running: 0 Queued: 20 Pending: 934  
Printing intermediate fdb and report files ...  
Warning: Stopping fault simulation as coverage error is 4.99183% which is  
less than 5.00% is less than 5.00%
```

The `RESULT/psim_defect.report` file shows the following:

Tags	Sample-Size
psim_defect	WSS Max Error = 5%


```
*DEFECT UNIVERSE SUMMARY
```

mos_open	656
mos_short	864
TOTAL	1520


```
*COVERAGE SUMMARY
```

Defects covered - weighted (covered/simulated):

Type	psim_defect
mos_open	0/0
mos_short	4438/7159
TOTAL	4438/7159

Weighted test coverage:

psim_defect	60.79% Error = +/- 4.99%
-------------	--------------------------

TOTAL Coverage:

WSS-Sampled	60.79% Error = +/- 4.99%
-------------	--------------------------

IEEE P2427 Weight Tutorial

The IEEE P2427 group has proposed some recommendations to calculate the weight for each kind of defect. This tutorial provides an example on how to apply P2427 weight recommendations with the Custom Fault tool.

Before starting the tutorial, make sure that all tools are installed correctly.

[Table 12](#) lists the files needed for this tutorial. These files are located in the `install_directory/doc/tutorials/P2427_Weight` directory.

Table 12 *P2427_weight Tutorial Files*

File	Description
netlist.sp	SPICE netlist of the circuit
dpconfig	Host file for distributed processing
xa.cfg	PrimeSim XA configuration file
config_xa.json	Custom Fault configuration file for the PrimeSim XA simulator
config_fs.json	Custom Fault configuration file for the FineSim simulator
config_ps.json	Custom Fault configuration file for the PrimeSim simulator
run_fault_xa	Run script to run the tutorial with the PrimeSim XA simulator
run_fault_fs	Run script to run the tutorial with the FineSim simulator
run_fault_ps	Run script to run the tutorial with the PrimeSim simulator
weight_template_schemeA.cpp	Weight file using the P2427 Scheme A recommendations
README	README file with some information on the tutorial
IEEE-P2427-Weight-Guidance.pdf	Document including the P2427 Guidance for defects weight

Edit the `dpconfig` file and modify it to use your own Distributing Processing.

The tutorial is described further in the following topics:

- [IEEE P2427 Recommendations](#)
- [Defect Models Definition](#)

- [Weight Definition](#)
- [Running the Tutorial With PrimeSim XA](#)
- [Running the Tutorial With FineSim](#)
- [Running the Tutorial With PrimeSim](#)
- [Adding printf Statements in the Weight File](#)

IEEE P2427 Recommendations

The IEEE group proposed three possible schemes to apply weight on analog defects (see [Table 13](#)).

Table 13 IEEE P2427 Schemes

Device		Scheme A	Scheme B	Scheme C
MOSFET	Open G	$P_{inter_open} * S_1$	1	$P_{inter_open} * S_1$
	Open D or S	$P_{inter_open} * S_1$	2	$P_{inter_open} * S_1$
	Short DS	$W * S_2 / (L * S_3)$	$0.35 * W / L$	$W * S_2 / (L * S_3)$
	Short GD or GS	$W * S_4$	W	$W * S_4$
Resistor	Open	$R * S_6$	$0.35 * L / W$	$R * S_6$
	Short	$P_{inter_short} * S_7$	$0.35 * W / L$	$P_{inter_short} * S_7$
Bipolar and Diode	Open B or C or E	$P_{inter_open} * S_1$	$P_{inter_open} * S_1$	$P_{inter_open} * S_1$
	Short BC or BE or CE	$P_{inter_short} * S_7$	$P_{inter_short} * S_7$	$P_{inter_short} * S_7$
Capacitor	Open	$P_{inter_open} * S_1$	1	$P_{inter_open} * S_1$
	Short	$C * S_5$	$W * L * 1e12$	$C * S_5$

Any of the three schemes can be used, however values from different schemes cannot be mixed together. When you pick one scheme to apply weight, you must stick to the values proposed in the chosen scheme (see [Table 14](#)).

Table 14 Variable Values for Schemes A and C

Variable	Scheme A	Scheme C
S_1	1	25

Table 14 Variable Values for Schemes A and C (Continued)

Variable	Scheme A	Scheme C
S ₂	3.5	2.857142857
S ₃	10	0.5
S ₄	1000000	1428571.429
S ₅	1e15	1.65e13
S ₆	0.00614	0.001
S ₇	1	15
P _{inter_open}	1	0.1
P _{inter_short}	10	0.9

For more information, see the [IEEE-P2427-Weight-Guidance.pdf](#) document in the `install_directory/doc/tutorials/P2427_Weight` directory (see [Table 12](#)).

Defect Models Definition

Open the `config_xa.json`, `config_fs.json`, or `config_ps.json` file. The defect models are defined in the "models" block inside the "fault" block.

MOS defect models:

- `mos_open_dgs` for open defect on terminals D, G, and S.
- `mos_short_dgs` for short defects on terminals D, G, and S.

BJT defect models:

- `bjt_open_cbe` for open defects on terminals C, B, and E.
- `bjt_short_cbe` for short defects on terminals C, B, and E.

MACRO device defect models are created for devices defined with a `.SUBCKT` description. There is one defect model per defect type (open or short) and per device type:

- `mos_mac_open_g` for open defect on terminal G of macro MOS
- `mos_mac_open_ds` for open defect on terminal D and S of macro MOS
- `mos_mac_short` for short defect on macro MOS
- `res_mac_open` for open defect on macro Resistors

- `res_mac_short` for short defect on macro Resistors
- `bjt_mac_open` for open defect on macro Bipolars
- `bjt_mac_short` for short defect on macro Bipolars
- `diode_mac_open` for open defect on macro Diodes
- `diode_mac_short` for short defect on macro Diodes
- `cap_mac_open` for open defect on macro Capacitors
- `cap_mac_short` for short defect on macro Capacitors

The defect models defined in the "models" blocks and basic defect models are applied using the `apply_fault_model` command:

```
"apply_fault_model" : [  
  {  
    "fault_model": [  
      "mos_mac_short", "mos_mac_open_ds", "mos_mac_open_g",  
      "res_mac_short", "res_mac_open",  
      "bjt_mac_open", "bjt_mac_short",  
      "diode_mac_open", "diode_mac_short",  
      "cap_mac_short", "cap_mac_open",  
      "mos_open_dgs", "mos_short_dgs",  
      "res_open", "res_short",  
      "bjt_short_cbe", "bjt_open_cbe",  
      "diode_open", "diode_short",  
      "cap_open", "cap_short"  
    ]  
  } ]
```

Weight Definition

The weight per defect type is defined in the weight file, `weight_template_schemeA.cpp`. To apply the weights defined in the weight file, use the `defect_weight` command in the Custom Fault configuration file:

```
"defect_weight" : "../weight_template_schemeA.cpp",
```

The weights in the `weight_template_schemeA.cpp` file use the P2427 Scheme A as shown in [Table 13](#).

Running the Tutorial With PrimeSim XA

Run the `run_fault_xa` script to run the Custom Fault tool with the PrimeSim XA simulator.

After the fault campaign has completed, you can verify the weight on each defect by looking at the `CF_Results_xa/xa.report` file. For example:

Id	Defect	xa	Simulation Data	Group Weight ...
A1.1	bjt_mac_open	NC	fault_subckt_open_1_1_1/subckt_open_1.*	1.000000e+00 ...

The weight is 1 for the `bjt_mac_open` defect. In the `weight_template_schemeA.cpp` file, check how the weight is defined for `bjt_mac_open`:

```
double s_1=1.0;
double p_inter_open=1.0;
double s_6=0.00614;

if STREQ(fault_type, "res_mac_open")
    weight = r * s_6 ;
if STREQ(fault_type, "diode_mac_open")
    weight = p_inter_open * s_1;
if STREQ(fault_type, "bjt_mac_open")
    weight = p_inter_open * s_1;
```

The weight for `bjt_mac_open` is $\text{weight} = \text{p_inter_open} * \text{s_1} = 1.0 * 1.0 = 1$, the `CF_Results_xa/xa.report` file result matches the `weight_template_schemeA.cpp` file calculation.

Look at another example in the `CF_Results_xa/xa.report` file:

A14.14	res_mac_open	CL	fault_subckt_open_14_14_14/subckt_open_14.*
6.140000e-01	xrb1		

The weight is 0.614 for the `res_mac_open` defect. The defect is injected on `xrb1` and this resistor has a resistance value of 100 Ohm, as shown in `netlist.sp`:

```
xrb1 inb b res_mac r=100
```

In the `weight_template_schemeA.cpp` file, check how the weight is defined for `res_mac_open`:

```
double s_6=0.00614;

if STREQ(fault_type, "res_mac_open")
    weight = r * s_6 ;
```

The weight for `res_mac_open` is $\text{r} * \text{s_6} = 100 * 0.00614 = 0.614$, the `CF_Results_xa/xa.report` file result matches the `weight_template_schemeA.cpp` file calculation.

Another solution to verify the weight file is correctly applied consists of [Adding printf Statements in the Weight File](#).

Running the Tutorial With FineSim

Run the `run_fault_fs` script to run the Custom Fault tool with the FineSim simulator.

After the fault campaign has completed, you can verify the weight on each defect by looking at the `CF_Results_fs/fs.report` file. For example,

Id	Defect	fs	Simulation Data	Group Weight ...
A1.1	bjt_mac_open	NC	fault_subckt_open_1_1_1/subckt_open_1.*	1.000000e+00 ...

The weight is 1 for the `bjt_mac_open` defect.

In the `weight_template_schemeA.cpp` file, check how the weight is defined for

`bjt_mac_open`:

```
double s_1=1.0;
double p_inter_open=1.0;
double s_6=0.00614;

if STREQ(fault_type, "res_mac_open")
    weight = r * s_6 ;
if STREQ(fault_type, "diode_mac_open")
    weight = p_inter_open * s_1;
if STREQ(fault_type, "bjt_mac_open")
    weight = p_inter_open * s_1;
```

The weight for `bjt_mac_open` is $\text{weight} = \text{p_inter_open} * \text{s_1} = 1.0 * 1.0 = 1$, the `CF_Results_fs/fs.report` file result matches the `weight_template_schemeA.cpp` file calculation.

Look at another example in the `CF_Results_fs/fs.report` file:

A14.14	res_mac_open	CL	fault_subckt_open_14_14_14/subckt_open_14.*
6.140000e-01	xrb1		

The weight is 0.614 for the `res_mac_open` defect. The defect is injected on `xrb1`, and this resistor has a resistance value of 100 Ohm, as shown in `netlist.sp`:

```
xrb1 inb b res_mac r=100
```

In the `weight_template_schemeA.cpp` file, check how the weight is defined for

`res_mac_open`:

```
double s_6=0.00614;

if STREQ(fault_type, "res_mac_open")
    weight = r * s_6 ;
```

The weight for `res_mac_open` is $r * \text{s_6} = 100 * 0.00614 = 0.614$, the `CF_Results_fs/fs.report` file result matches the `weight_template_schemeA.cpp` file calculation.

Another solution to verify the weight file is correctly applied consists of [Adding printf Statements in the Weight File](#).

Running the Tutorial With PrimeSim

Run the `run_fault_ps` script to run the Custom Fault tool with the PrimeSim simulator.

After the fault campaign has completed, you can verify the weight on each defect by looking at the `CF_Results_ps/ps.report` file. For example,

Id	Defect	fs	Simulation Data	Group Weight ...
A1.1	bjt_mac_open	NC	fault_subckt_open_1_1_1/subckt_open_1.*	1.000000e+00 ...

The weight is 1 for the `bjt_mac_open` defect.

In the `weight_template_schemeA.cpp` file, check how the weight is defined for `bjt_mac_open`:

```
double s_1=1.0;
double p_inter_open=1.0;
double s_6=0.00614;

if STREQ(fault_type, "res_mac_open")
    weight = r * s_6 ;
if STREQ(fault_type, "diode_mac_open")
    weight = p_inter_open * s_1;
if STREQ(fault_type, "bjt_mac_open")
    weight = p_inter_open * s_1;
```

The weight for `bjt_mac_open` is $\text{weight} = p_inter_open * s_1 = 1.0 * 1.0 = 1$, the `CF_Results_ps/ps.report` file result matches the `weight_template_schemeA.cpp` file calculation.

Look at another example in the `CF_Results_ps/ps.report` file:

A14.14	res_mac_open	CL	fault_subckt_open_14_14_14/subckt_open_14.*
6.140000e-01	xrb1		

The weight is 0.614 for the `res_mac_open` defect. The defect is injected on `xrb1`, and this resistor has a resistance value of 100 Ohm, as shown in `netlist.sp`:

```
xrb1 inb b res_mac r=100
```

In the `weight_template_schemeA.cpp` file, check how the weight is defined for `res_mac_open`:

```
double s_6=0.00614;

if STREQ(fault_type, "res_mac_open")
    weight = r * s_6 ;
```

The weight for `res_mac_open` is $r * s_6 = 100 * 0.00614 = 0.614$, the `CF_Results_ps/ps.report` file result matches the `weight_template_schemeA.cpp` file calculation.

Another solution to verify the weight file is correctly applied consists of [Adding printf Statements in the Weight File](#).

Adding printf Statements in the Weight File

You can add the `printf` command after each occurrence of `weight=`. For example, you can print the weight of the `res_mac_open` defect models and the weight of the `mos_short` defect models as follows:

```
if STREQ(fault_type, "res_mac_open")
{
    weight = r * s_6 ;
    printf("%s %f %f \n", instance_name, r, weight);
}
...
// DS weight
{
    weight = ((w * s_2) / (l * s_3)) * m * nfin ;
    printf("%s %s %s %e %e %f %f %f \n", instance_name,
    fault_port_name0, fault_port_name1, w, l, m, nfin, weight);
}
```

When the fault campaign is run, the values are displayed on your screen:

```
Calculating weights ...
xrb1 100.000000 0.614000
xrb1 100.000000 0.614000
xrl 1.000000 0.006140
xrl 1.000000 0.006140
xm2 0 out_inv 2.020000e-07 2.000000e-07 1.000000 5.000000 1.767500
xm1 out_inv vdd 2.020000e-07 2.000000e-07 1.000000 5.000000 1.767500
```


5

Custom Fault Stimulus Quality Assessment

This chapter describes how to use the Stimulus Quality Assessment (SQA) feature of the Custom Fault tool.

- [Overview of Stimulus Quality Assessment](#)
- [Running Active SQA](#)
- [Running Concurrent SQA](#)

Overview of Stimulus Quality Assessment

Running fault simulation can be expensive based on the number of defects being simulated. Stimulus Quality Assessment (SQA) simulations using the Custom Fault tool compliment defect simulation by assessing the quality of a test vector before running a simulation intensive fault campaign.

SQA is enabled using the [sqa](#) command and operates in the following modes:

- [Active Stimulus Quality Assessment](#)
- [Concurrent Stimulus Quality Assessment](#)

Active Stimulus Quality Assessment

The Active Stimulus Quality Assessment (ASQA) feature determines the quality of the stimulus/vector based on the circuit activity when the stimulus/vector is applied. Only one simulation is run.

Running Active SQA in the Custom Fault tool enables the transient simulator to run the nominal simulation and tracks the toggling activity between three operating regions (cutoff, linear, and saturation) for each MOSFET in the circuit. The toggling activity is used to determine the activity of the MOSFET. The more a MOSFET is toggled between the operating regions, the more active the MOSFET is.

For details, see [Running Active SQA](#).

Note:

No defect is injected with ASQA, only the nominal fault-free simulation is run.

ASQA Limitations

Active SQA is limited to MOSFET devices.

Active SQA cannot run on SUSE OS machines.

Concurrent Stimulus Quality Assessment

The Concurrent Stimulus Quality Assessment (CSQA) feature measures the sensitization of each defect. The Custom Fault tool tries to determine whether the provided stimulus causes a local reaction to the defect. When there is a local reaction, that means there is some significant difference to the defect free circuit.

The CSQA feature adds voltage probes and compares internal voltages from a fault-free simulation with the faulty simulation voltages. As a result, it provides the list of defects that cannot be sensitized by the input stimuli.

The Concurrent SQA feature injects multiple defects in the same simulation, each defect being injected in portions of the circuit that are totally independent.

For details, see [Running Concurrent SQA](#).

Note:

CSQA, unlike ASQA, requires defect models and a defect universe.

CSQA Limitations

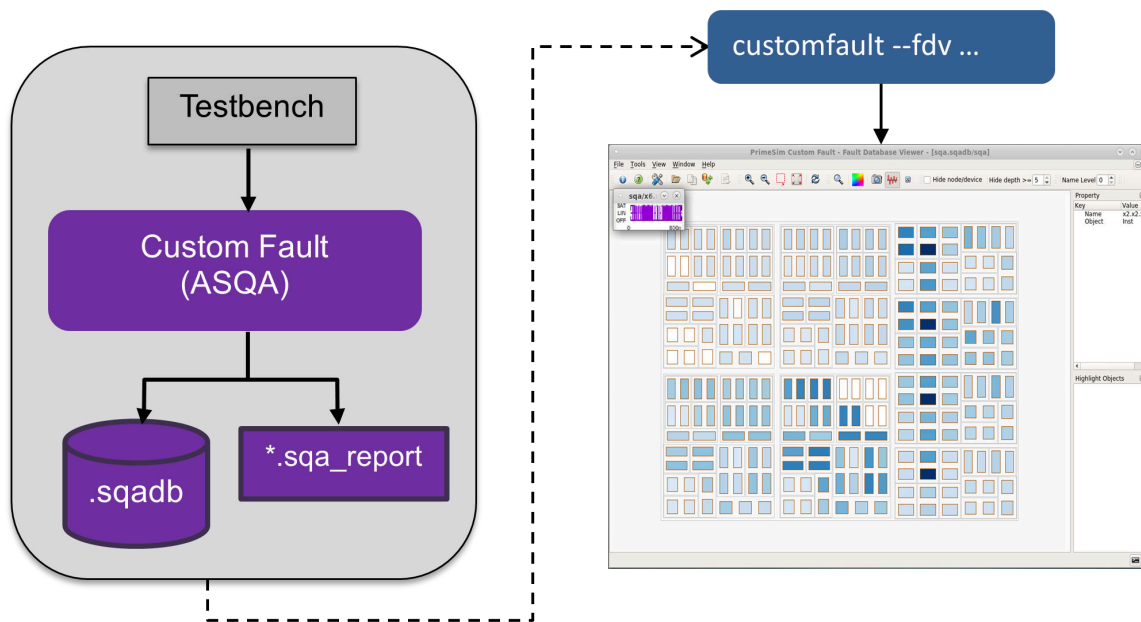
The Multi-Testbench (MTB) flow is not supported with CSQA.

Running Active SQA

The Active SQA (ASQA) flow helps you improve your vectors before running the defect campaign, so run ASQA before launching the defect campaigns.

[Figure 36](#) shows the Active SQA flow in the Custom Fault tool.

Figure 36 Active Stimulus Quality Assessment Flow



For example, the following command line reads in the `use_vec.json` and `asqa.json` configuration files to run the Active SQA simulation. The Active SQA database file is generated at `ASQA_RESULT/sim_vec.sqadb`.

```
$> customfault -c use_vec.json -c asqa.json -o ASQA_RESULT/sim_vec -t sim_vec -k 1
```

The following sections provide more information on running ASQA:

- [Using Active SQA Output Files](#)
- [Running Active SQA with Multiple Testbenches](#)

Using Active SQA Output Files

Active SQA (ASQA) generates the `*.sqa_report` and `*.sqadb` output files.

The `*.sqa_report` file contains the report of the toggling activity for all the MOSFET devices in the circuit.

An example of `*.sqa_report` is as follows:

```

PrimeSim (TM) Custom Fault

Version T-2022.06-SP2

Tags
csim_faultsim

```

```
*ELEMENT STATISTIC
      mos          164
      TOTAL        164

*ACTIVITY SUMMARY
              csim_faultsim
      mos      8383
      TOTAL    8383

*DETAIL OF ACTIVITY
              type  csim_faultsim
x1.x5.mn1    mos      0
x1.x5.mp1    mos      0
x1.x4.mn1    mos     18
x1.x4.mp1    mos     18
x1.x3.mn1    mos     18
x1.x3.mp1    mos     18
x3.mn2       mos     31
x4.mn2       mos     58
x2.mn2       mos     22
x1.mn2       mos     16
x3.mp1       mos     30
x4.mp1       mos     68
x2.mp1       mos      4
x1.mp1       mos      0
x2.mn1       mos     36
x3.mn1       mos     68
x1.mn1       mos     15
x4.mn1       mos    124
...
x4.x1.x2.mp1 mos     12
x4.x1.x1.mp1 mos    211
x4.x1.x3.mn1 mos    148
x4.x1.x3.mp1 mos    148
```

ASQA also generates the *.sqadb file that contains the database of ASQA simulation.

Running Active SQA with Multiple Testbenches

The Custom Fault tool supports running Active SQA (ASQA) with multiple testbenches sequentially.

After the first iteration of ASQA simulation, the tool generates an ASQA database file (*.sqadb) and a report file (*.sqa_report).

The ASQA database file (*.sqadb) from the first iteration is reused in the second iteration of ASQA simulation using the -f command line option with a change in the testbench.

The output files of the second iteration of ASQA simulation are a new ASQA database file (*.sqadb) that contains the ASQA simulation data from both the first and second iterations, and a combined report (*.sqa_report) from the first and second iterations.

The iterations can continue for as many testbenches as you require to run ASQA simulation.

Example

The first command line reads in the `1st_vec.json` and `asqa.json` configuration files to run the first iteration of the ASQA simulation. The ASQA database file is generated at `ASQA_RESULT/sim_1st_vec.sqadb`.

```
$> customfault -c 1st_vec.json -c asqa.json -o ASQA_RESULT/sim_1st_vec -t 1st_vec -k 1
```

The second command line reads in the `2nd_vec.json` and `asqa.json` configuration files, as well as the first ASQA database file (`ASQA_RESULT/sim_1st_vec.sqadb`) using the `-f` command line option to run the second iteration of the ASQA simulation.

```
$> customfault -c 2nd_vec.json -c asqa.json -o ASQA_RESULT/sim_2nd_vec -t 2nd_vec -f ASQA_RESULT/sim_1st_vec.sqadb -k 1
```

The second ASQA database file (`ASQA_RESULT/sim_2nd_vec.sqadb`) and report file (`ASQA_RESULT/sim_2nd_vec.sqa_report`) generated from this iteration contain the results from both simulations.

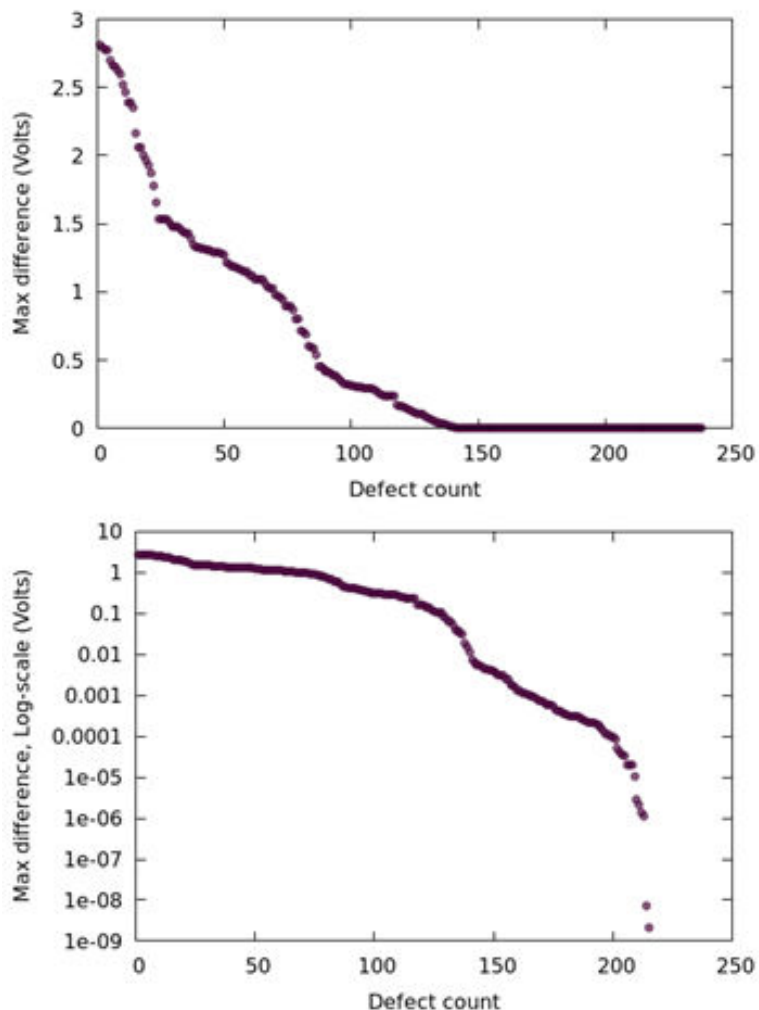
Running Concurrent SQA

The Concurrent SQA (CSQA) flow helps you figure out the portions of the circuit that are exercised but the defects are not sensitized with your stimuli, so run CSQA before launching the defect campaigns.

When Concurrent SQA is run, a plot file is generated with the maximum difference detected between a fault-free simulation and faulty simulations. This file is created with a `.csqa.csqasense.png` extension (see [Figure 37](#)).

The chart can help you to modify the threshold value using the `"defect_sensitization_threshold"` argument of the `sqa` command if you want a tighter or looser sensitization.

Figure 37 Example of .csqa.csqasense.png



Concurrent SQA Output Files

When Concurrent SQA (CSQA) is completed, the CSQA report has the `.sqa_report` extension. The following is an excerpt of a `.sqa_report` file:

```

PrimeSim (TM) Custom Fault

Version T-2022.06-SP2

Tags          SQA Method      Threshold
csqa          Concurrent SQA      0.1

*DEFECT UNIVERSE SUMMARY

```

mos_open	164
mos_short	214
TOTAL	378

*CONCURRENCY SUMMARY

Defects Per Simulation	Simulations
2	43
1	152
TOTAL	195

*DEFECT SENSITIZATION SUMMARY

Defect Type	Sensitized/Total
mos_open	0/0
mos_short	165/214
TOTAL	165/214

*DETAILED LISTING

Id	Defect	Sensitized	Simulation Data	Weight	Instance
Port (s)					
A1.1	mos_open	true	fault_multi_1_1_1/multi_1.*	0.50	x1.x4.mn1 d
A1.1	mos_open	false	fault_multi_1_1_1/multi_1.*	0.50	x1.x6.mn1 d
A2.2	mos_open	true	fault_multi_2_2_2/multi_2.*	0.50	x1.x4.mn1 g
A2.2	mos_open	false	fault_multi_2_2_2/multi_2.*	0.50	x1.x6.mn1 g
...					
A25.25	mos_short	true	fault_multi_25_25_25/multi_25.*	3.00	x1.x8.mp1 s d
A25.25	mos_short	true	fault_multi_25_25_25/multi_25.*	4.00	x1.x1.mn1 b g

As you can see in the `.sqa_report` file, multiple defects are injected at the same time. The two defects in A1.1 are independent, so the `mos_open` defect in `x1.x4.mn1` can be injected at the same time as the `mos_open` defect in `x1.x6.mn1` without perturbing the results.

If a defect is reported "`sensitized: false`", that means it does not propagate to the output and cannot be detected with your current stimuli. Such a defect can be removed from the final coverage, or you can modify your test vectors so that the defect is sensitized. Some defects are always undetectable even if the test vectors are modified, so they always show up as "`sensitized: false`" and must be ignored in the final coverage.

6

Custom Fault Automatic Injection Flow

This chapter covers the automatic injection flow for analog fault simulation.

- [Overview of the Automatic Injection Flow](#)
- [Automatic Injection Flow Tutorial](#)

Overview of the Automatic Injection Flow

In the automatic injection flow, you enable the tool to identify all the possible defects in the netlist, reduce the number of defects by merging similar defects, and inject one defect model at a time to perform analog fault simulation. To enable automatic injection flow, follow the steps in [Setting Up Fault Simulation](#).

Automatic Injection Flow Tutorial

The tutorial described in this section demonstrates the following:

- [Running the Custom Fault Automatic Injection Flow With PrimeSim XA](#)
- [Running the Custom Fault Automatic Injection Flow With FineSim](#)
- [Running the Custom Fault Automatic Injection Flow With PrimeSim](#)

Before starting these tutorials, make sure that all tools are installed correctly.

[Table 15](#) lists the files needed for this tutorial. These files are located in the `install_directory/doc/tutorials/automatic_flow` directory.

Table 15 *Automatic Injection Flow Tutorial Files*

File	Description
<code>adder.vec</code>	Vector file containing both input and output vectors.
<code>cfauto_configfile_xa</code>	Configuration file for the Custom Fault tool with the PrimeSim XA tool.

Table 15 *Automatic Injection Flow Tutorial Files (Continued)*

File	Description
<code>cfauto_configfile_fsim</code>	Configuration file for the Custom Fault tool with the FineSim tool.
<code>cfauto_configfile_psim</code>	Configuration file for the Custom Fault tool with the PrimeSim tool.
<code>cmd_load_vec_file</code>	Command file for the PrimeSim XA tool.
<code>top_no_vec_xa.spi</code>	Top-level netlist of a 4-bit adder for the PrimeSim XA tool.
<code>top_fsim.spi</code>	Top-level netlist of a 4-bit adder for the FineSim tool.
<code>top_psim.spi</code>	Top-level netlist of a 4-bit adder for the PrimeSim tool.
<code>xhost/xhost_sh_launch</code>	Host file for distributing processing.
<code>run_auto_xa</code>	Run script to run the automatic defect injection flow with the Custom Fault and PrimeSim XA tools.
<code>run_auto_fsim</code>	Run script to run the automatic defect injection flow with the Custom Fault and FineSim tools.
<code>run_auto_psim</code>	Run script to run the automatic defect injection flow with the Custom Fault and PrimeSim tools.

The defect detection checking criteria is done by vector check. The `adder.vec` file contains the output vectors of `COUT` and `S[3:0]`. If the simulation output matches with the expected states, then no violation is reported, and the defect is not covered. Otherwise, a violation is reported, and the defect is covered.

The `load_vector_file` command in `cmd_load_vec_file` loads in the `adder.vec` file. To speed up the throughput, the `set_vector_option -error 1` command in `cmd_load_vec_file` forces the simulation to exit when it encounters a comparison error.

The content of `cmd_load_vec_file` is:

```
# set command to probe voltage signals
probe_waveform_voltage *
# set command for speed-accuracy tradeoff
set_sim_level 3
# command to load the vector file
load_vector_file -file adder.vec
# set option to stop simulation when there is a comparison error
set_vector_option -error 1
```

Running the Custom Fault Automatic Injection Flow With PrimeSim XA

This section describes how to run the automatic defect injection flow with the Custom Fault and PrimeSim XA tools in batch mode.

- [Creating the Configuration File to Run Custom Fault With PrimeSim XA](#)
- [Running the Custom Fault Tool With PrimeSim XA](#)
- [Generating Fault Simulation Output Using PrimeSim XA](#)

Creating the Configuration File to Run Custom Fault With PrimeSim XA

To run a Custom Fault simulation, you must create a configuration file. The configuration file supplies all the information to the Custom Fault tool, including the top-level netlist name, simulator used, and so on.

The configuration file contains the following information:

- The simulator used for transient simulation is the PrimeSim XA tool (`xa`).
- The directory in which to find the data and run the simulation (`case_dir`).
- The top-level netlist (`top_no_vec_xa.spi`) in the current working directory.
- The maximum runtime allowed per simulation (`cpulimit`) is 100s.
- The path to the host file (`dpconfig_file`) for distributing processing is `xhost/xhost_sh_launch`.
- The command-line prefix to run the PrimeSim XA tool (`sim_cmdline`) is `-hspice` and the command-line suffix is `-c cmd_load_vec_file`.
- The short defect is modeled by a resistor with value of 200 ohms.
- The open defect is modeled by a resistor with value of 10e6 ohms.

The content of the `cfauto_configfile_xa` file is:

```
{
  "case_dir" : "./",
  "cpulimit" : 100,
  "dpconfig_file" : "xhost/xhost_sh_launch",
  "analog_setup" : {
    "sim_cmdline": "xa -hspice top_no_vec_xa.spi -c
cmd_load_vec_file"
  },
  "faults" : {
    "defaults" : {
```

```
        "short" : {
            "res" : 200,
            "cap" : null
        },
        "open"   : {
            "res" : 10e6,
            "cap" : null
        }
    }
}
```

Running the Custom Fault Tool With PrimeSim XA

The `run_auto_xa` file contains the command to execute the Custom Fault tool:

```
customfault -c cfauto_configfile_xa -o RESULT/xa_defect
\ -t xa_defect -k 5
```

To invoke the Custom Fault tool, run `run_auto_xa`:

```
$> run_auto_xa
```

Generating Fault Simulation Output Using PrimeSim XA

After the simulation completes, the Custom Fault tool generates the following output files and directory inside the `RESULT` directory shown in [Table 16](#).

Table 16 Custom Fault Output Files for the PrimeSim XA Tool

Output File	Description
<code>xa_defect/*</code>	Output directory containing all the intermediate files. The name of the directory is determined by the simulation tag name specified by the <code>-t</code> command-line option.
<code>xa_defect.config.json</code>	Configuration file to run individual fault simulation.
<code>xa_defect.fdb</code>	Fault simulation database.
<code>xa_defect.log</code>	Log file.
<code>xa_defect.report</code>	Coverage report of the fault simulation.

The following example shows the summary section of the coverage report (`xa_defect.report`). Note that the coverage percentage might be slightly different based on the version of the PrimeSim XA tool you use.

Tags	Sample-Size
xa_defect	Exhaustive

```
*DEFECT UNIVERSE SUMMARY

      mos_open                656
      mos_short              864

      TOTAL                  1520

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
      Type                    xa defect

      mos_open                397/656
      mos_short              700/864
      TOTAL                  1097/1520

Weighted test coverage:
      xa_defect                72.17%

TOTAL Coverage:
      Non-Sampled              72.17%

Total weight of defect universe:    1520.00
```

The coverage report shows the following information:

- The simulation tag name is `xa_defect`.
- There are 656 `mos_open` defects and 864 `mos_short` defects identified.
- 397 out of 656 `mos_open` defects are covered.
- 700 out of 864 `mos_short` defects are covered.
- The test coverage is 72.17%.

The `*DETAILED COVERAGE LISTING` section shows the detail of each fault simulation. The log file of each individual fault simulation can be found in `RESULT/xa_defect`.

Running the Custom Fault Automatic Injection Flow With FineSim

This section describes how to run the automatic defect injection flow with the Custom Fault and FineSim tools in batch mode.

- [Creating the Configuration File To Run Custom Fault With FineSim](#)
- [Running the Custom Fault Tool With FineSim](#)
- [Generating Fault Simulation Output Using FineSim](#)

Note:

The `.opt finesim_vector="adder.vec"` option is specified in the top-level netlist (`top_fsim.spi`) to load in the `adder.vec` file.

Creating the Configuration File To Run Custom Fault With FineSim

To run the Custom Fault tool, you must specify a configuration file. The configuration file supplies all the information to the tool, including the top-level netlist name, simulator used, and so on.

The configuration file contains the following information:

- The simulator used for transient simulation is the FineSim tool (`finesim`).
- The directory in which to find the data and run the simulation (`case_dir`).
- The maximum runtime allowed per simulation (`cpulimit`) is 100s.
- The path to the host file (`dpconfig_file`) for distributing processing is `xhost/xhost_sh_launch`.
- The top-level netlist (`top_fsim.spi`) in the current working directory.
- There is no command-line prefix and suffix.
- The short defect is modeled by a resistor with value of 200 ohms.
- The open defect is modeled by a resistor with value of 10e6 ohms.

The content of `cfauto_configfile_fsim` file is:

```
{
  "case_dir" : "./",
  "cpulimit" : 100,
  "dpconfig_file" : "xhost/xhost_sh_launch",
  "analog_setup" : {
    "sim_cmdline": "finesim top_fsim.spi"
  },
  "faults" : {
    "defaults" : {
      "short" : {
        "res" : 200,
        "cap" : null
      },
      "open" : {
        "res" : 10e6,
        "cap" : null
      }
    }
  }
}
```

```
    }  
}
```

Running the Custom Fault Tool With FineSim

The `run_auto_fsim` file contains the command to execute the Custom Fault tool:

```
customfault -c cfauto_configfile_fsim -o RESULT/fsim_defect  
\ -t fsim_defect -k 5
```

To invoke the Custom Fault tool, run `run_auto_fsim`:

```
$> run_auto_fsim
```

Generating Fault Simulation Output Using FineSim

After the simulation completes, the Custom Fault tool generates the following output files and directory inside the `RESULT` directory shown in [Table 17](#).

Table 17 Custom Fault Output Files for the FineSim Tool

Output File	Description
<code>fsim_defect/*</code>	Output directory containing all the intermediate files. The name of the directory is determined by the simulation tag name specified by the <code>-t</code> command-line option.
<code>fsim_defect.config.json</code>	Configuration file to run individual fault simulation.
<code>fsim_defect.fdb</code>	Fault simulation database.
<code>fsim_defect.log</code>	Log file.
<code>fsim_defect.report</code>	Coverage report of the fault simulation.

The following example shows the summary section of the coverage report (`fsim_defect.report`). Note that the coverage percentage might be slightly different based on the version of the FineSim tool you use.

```
                Tags          Sample-Size  
                fsim_defect    Exhaustive  
  
*DEFECT UNIVERSE SUMMARY  
  
    mos_open                656  
    mos_short               864  
  
    TOTAL                   1520  
  
*COVERAGE SUMMARY
```

```
Defects covered - weighted (covered/simulated):
      Type                                fsim defect

      mos_open                            397/656
      mos_short                           699/864
      TOTAL                               1096/1520

Weighted test coverage:
      fsim_defect                          72.11%

TOTAL Coverage:
      Non-Sampled                          72.11%

Total weight of defect universe:           1520.00
```

The coverage report shows the following information:

- The simulation tag name is `fsim_defect`.
- There are 656 `mos_open` defects and 864 `mos_short` defects identified.
- 397 out of 656 `mos_open` defects are covered.
- 699 out of 864 `mos_short` defects are covered.
- The test coverage is 72.11%.

The `*DETAILED COVERAGE LISTING` section shows the detail of each fault simulation. The log file of each individual fault simulation can be found in `RESULT/fsim_defect`.

Running the Custom Fault Automatic Injection Flow With PrimeSim

This section describes how to run the automatic defect injection flow with the Custom Fault and PrimeSim tools in batch mode.

- [Creating the Configuration File To Run Custom Fault With PrimeSim](#)
- [Running the Custom Fault Tool With PrimeSim](#)
- [Generating Fault Simulation Output Using PrimeSim](#)

Note:

The `.opt primesim_vector="adder.vec"` option is specified in the top-level netlist (`top_psim.spi`) to load in the `adder.vec` file.

Creating the Configuration File To Run Custom Fault With PrimeSim

To run the Custom Fault tool, you must specify a configuration file. The configuration file supplies all the information to the tool, including the top-level netlist name, simulator used, and so on.

The configuration file contains the following information:

- The simulator used for transient simulation is the PrimeSim tool (`primesim`).
- The directory in which to find the data and run the simulation (`case_dir`).
- The maximum runtime allowed per simulation (`cpulimit`) is 100s.
- The path to the host file (`dpconfig_file`) for distributing processing is `xhost/xhost_sh_launch`.
- The top-level netlist (`top_psim.spi`) in the current working directory.
- There is no command-line prefix and suffix.
- The short defect is modeled by a resistor with value of 200 ohms.
- The open defect is modeled by a resistor with value of 10e6 ohms.

The content of `cfauto_configfile_psim` file is:

```
{
  "case_dir" : "./",
  "cpulimit" : 100,
  "dpconfig_file" : "xhost/xhost_sh_launch",
  "analog_setup" : {
    "sim_cmdline": "primesim top_psim.spi"
  },
  "faults" : {
    "defaults" : {
      "short" : {
        "res" : 200,
        "cap" : null
      },
      "open" : {
        "res" : 10e6,
        "cap" : null
      }
    }
  }
}
```


Running the Custom Fault Tool With PrimeSim

The `run_auto_psim` file contains the command to execute the Custom Fault tool:

```
customfault -c cfauto_configfile_psim -o RESULT/psim_defect \
-t psim_defect -k 5
```

To invoke the Custom Fault tool, run `run_auto_psim`:

```
$> run_auto_psim
```

Generating Fault Simulation Output Using PrimeSim

After the simulation completes, the Custom Fault tool generates the following output files and directory inside the `RESULT` directory shown in [Table 18](#).

Table 18 Custom Fault Output Files for the PrimeSim Tool

Output File	Description
psim_defect/*	Output directory containing all the intermediate files. The name of the directory is determined by the simulation tag name specified by the <code>-t</code> command-line option.
psim_defect.config.json	Configuration file to run individual fault simulation.
psim_defect.fdb	Fault simulation database.
psim_defect.log	Log file.
psim_defect.report	Coverage report of the fault simulation.

The following example shows the summary section of the coverage report (`psim_defect.report`). Note that the coverage percentage might be slightly different based on the version of the PrimeSim tool you use.

```

                Tags          Sample-Size
psim_defect      Exhaustive

*DEFECT UNIVERSE SUMMARY

mos_open          656
mos_short         864
TOTAL             1520

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
                Type          psim defect
```

mos_open	397/656
mos_short	699/864
TOTAL	1096/1520
Weighted test coverage:	
psim_defect	72.11%
TOTAL Coverage:	
Non-Sampled	72.11%
Total weight of defect universe:	1520.00

The coverage report shows the following information:

- The simulation tag name is `psim_defect`.
- There are 656 `mos_open` defects and 864 `mos_short` defects identified.
- 397 out of 656 `mos_open` defects are covered.
- 699 out of 864 `mos_short` defects are covered.
- The test coverage is 72.11%.

The `*DETAILED COVERAGE LISTING` section shows the detail of each fault simulation. The log file of each individual fault simulation can be found in `RESULT/psim_defect`.

7

Custom Fault Manual Injection Flow

This chapter covers the manual injection flow for analog fault simulation.

- [Overview of the Manual Defect Injection Flow](#)
- [Defect List File Format](#)
- [Manual Injection Flow Tutorial](#)

Overview of the Manual Defect Injection Flow

In the manual injection flow, you use the Custom Fault tool to bypass defect identification and defect reduction and manually inject defect models. To enable manual defect injection flow, you provide a defect list file with user-defined defect models to be simulated. The defect list file can be included in configuration file using the [fault_list_file](#) command or `-f` command-line option. [Figure 38](#) shows an overview of the manual injection flow.

```
graph TD
    subgraph Config [Configuration File]
        direction TB
        T1[Top-level netlist]
        T2["fault_list_file" : "manual_fault_file"]
        T3[...]
    end
    Config --> PrimeSim
    subgraph PrimeSim [PrimeSim Custom Fault]
        direction TB
        P1[Read in netlist]
        P2[Read in command file]
        P3[Identify all possible faults]
        P4[Reduce number of faults]
        P5[Inject faults into netlist]
        P6[Perform simulation]
    end
    PrimeSim --> DB[(Fault Database)]
    PrimeSim --> CR[Coverage Report]
```

Configuration File

Top-level netlist

"fault_list_file" : "manual_fault_file"

⋮

PrimeSim Custom Fault

- Read in netlist
- Read in command file
- Identify all possible faults
- Reduce number of faults
- Inject faults into netlist
- Perform simulation

manual_fault_file

```
inject_fault -inst I0.D5.dipwnd -type open -node b -res_val 1.000000e+06 -cap_val 1.000000e-20
inject_fault -inst I0.D5.dipwnd -type open -node e -res_val 1.000000e+06 -cap_val 1.000000e-20
inject_fault -inst I0.D4.dipwnd -type open -node c -res_val 1.000000e+06 -cap_val 1.000000e-20
inject_fault -inst I0.D4.dipwnd -type open -node b -res_val 1.000000e+06 -cap_val 1.000000e-20
inject_fault -inst I0.D4.dipwnd -type open -node e -res_val 1.000000e+06 -cap_val 1.000000e-20
inject_fault -inst I0.XIOSC.MN1 -type short -node d g -res_val 1.000000e-02 -cap_val 1.000000e-20
inject_fault -inst I0.XIOSC.MN4 -type short -node g d -res_val 1.000000e-02 -cap_val 1.000000e-20
inject_fault -inst I0.XIOSC.MP6 -type short -node g s -res_val 1.000000e-02 -cap_val 1.000000e-20
inject_fault -inst I0.XIOSC.MN7 -type short -node d g -res_val 1.000000e-02 -cap_val 1.000000e-20
inject_fault -type short -node I0.FRSEL0 net11 -res_val 1.000000e+06 -cap_val 1.000000e-20
inject_fault -type short -node I0.FRSEL1 net11 -res_val 1.000000e+06 -cap_val 1.000000e-20
inject_fault -type short -node I0.FRSEL2 net11 -res_val 1.000000e+06 -cap_val 1.000000e-20
inject_fault -type short -node I0.FRSEL3 net11 -res_val 1.000000e+06 -cap_val 1.000000e-20
inject_fault -type short -node I0.XIOSC.AA1 I0.XIOSC.BB1 -res_val 1.000000e+06 -cap_val 1.000000e-20
inject_fault -type short -node I0.XIOSC.BB4 I0.XIOSC.N7 -res_val 1.000000e+06 -cap_val 1.000000e-20
```

Fault Database

Coverage Report

The defect list file contains the list of defect models that you can inject. You define the defect models using the `inject_fault` command. For each `inject_fault` command specified in the defect list file, the tool injects one defect model into the netlist and runs an analog fault simulation.

The `inject fault` command does not support wildcard characters.

- Device Instance Open Defect
- Device Instance Short Defect
- Subcircuit Open Defect
- Subcircuit Short Defect

- [Behavioral Resistor Open Defect](#)
- [Behavioral Resistor Short Defect](#)
- [Behavioral Capacitor Open Defect](#)
- [Behavioral Capacitor Short Defect](#)
- [MOSFET Resistor Divider Gate Open Defect](#)
- [Subcircuit MOSFET Resistor Divider Gate Open Defect](#)
- [Net to Net Short Defect](#)
- [Parametric Defect](#)

Device Instance Open Defect

```
inject_fault -inst inst -type open -node n1 -res_val rval  
[-cap_val cval] [-weight weight_value] [-fault_type type_name]
```

Argument	Description
-inst <i>inst</i>	Specifies the name of the device instance. It supports MOSFET, diode, JFET, BJT, resistor, capacitor, or inductor.
-type open	Specifies to inject an open defect.
-node <i>n1</i>	Specifies the terminal name for the device instance open defect where an open defect is injected. The supported terminal names for each device are: • MOSFET: <i>d</i> (drain), <i>g</i> (gate), <i>s</i> (source), and <i>b</i> (bulk) • Diode: <i>p</i> (positive), and <i>n</i> (negative) • JFET: <i>d</i> (drain), <i>g</i> (gate), <i>s</i> (source), and <i>b</i> (bulk) • BJT: <i>c</i> (collector), <i>b</i> (base), <i>e</i> (emitter), and <i>s</i> (substrate) • Resistor: <i>p</i> (positive), <i>n</i> (negative) • Capacitor <i>p</i> (positive), <i>n</i> (negative) • Inductor <i>p</i> (positive), <i>n</i> (negative)
-res_val <i>rval</i>	Specifies the resistance for the open defect model.
-cap_val <i>cval</i>	Specifies the capacitance for the open defect model.
-weight <i>weight_value</i>	Specifies the value of the weight. See Specifying Weights for Defects for more information.

Argument	Description
<code>-fault_type type_name</code>	Specifies the name of the defect type (see Supported Built-in Defect Models): • MOSFET: <code>mos_open</code> (for example, see Figure 6) • Diode: <code>diode_open</code> (for example, see Figure 7) • JFET: <code>jfet_open</code> • BJT: <code>bjt_open</code> • Resistor: <code>res_open</code> (for example, see Figure 8) • Capacitor: <code>cap_open</code> (for example, see Figure 9) • Inductor: <code>ind_open</code> (for example, see Figure 7) If you do not specify this argument, the default is <code>user_defined</code>. You can specify it to any name representative of the defect type.

Device Instance Short Defect

```
inject_fault -inst inst -type short -node n1 n2 -res_val rval
[-cap_val cval] [-weight weight_value] [-fault_type type_name]
```

Argument	Description
<code>-inst inst</code>	Specifies the name of the device instance. It supports MOSFET, diode, JFET, BJT, resistor, capacitor, or inductor.
<code>-type short</code>	Specifies to inject a short defect.
<code>-node n1 n2</code>	Specifies the terminal names for the device instance short defect where a short defect is injected. The supported terminal names for each device are: • MOSFET: <code>d</code> (drain), <code>g</code> (gate), <code>s</code> (source), and <code>b</code> (bulk) • Diode: <code>p</code> (positive), and <code>n</code> (negative) • JFET: <code>d</code> (drain), <code>g</code> (gate), <code>s</code> (source), and <code>b</code> (bulk) • BJT: <code>c</code> (collector), <code>b</code> (base), <code>e</code> (emitter), and <code>s</code> (substrate) • Resistor: <code>p</code> (positive), <code>n</code> (negative) • Capacitor <code>p</code> (positive), <code>n</code> (negative) • Inductor <code>p</code> (positive), <code>n</code> (negative)
<code>-res_val rval</code>	Specifies the resistance for the short defect model.
<code>-cap_val cval</code>	Specifies the capacitance for the short defect model.
<code>-weight weight_value</code>	Specifies the value of the weight. See Specifying Weights for Defects for more information.

Argument	Description
<code>-fault_type type_name</code>	Specifies the name of the defect type (see Supported Built-in Defect Models): • MOSFET: <code>mos_short</code> (for example, see Figure 10) • Diode: <code>diode_short</code> (for example, see Figure 11) • JFET: <code>jfet_short</code> • BJT: <code>bjt_short</code> • Resistor: <code>res_short</code> (for example, see Figure 12) • Capacitor: <code>cap_short</code> (for example, see Figure 13) • Inductor: <code>ind_short</code> (for example, see Figure 11) If you do not specify this argument, the default is <code>user_defined</code>. You can specify it to any name representative of the defect type.

Subcircuit Open Defect

```
inject_fault -subckt_inst sinst -type open -node n1 -res_val rval
[-cap_val cval] [-weight weight_value] [-fault_type subckt_open]
```

Argument	Description
<code>-subckt_inst sinst</code>	Specifies the name of the subcircuit instance.
<code>-type open</code>	Specifies to inject an open defect.
<code>-node n1</code>	Specifies the port name for the subcircuit open defect where an open defect is injected.
<code>-res_val rval</code>	Specifies the resistance for the open defect model.
<code>-cap_val cval</code>	Specifies the capacitance for the open defect model.
<code>-weight weight_value</code>	Specifies the value of the weight. See Specifying Weights for Defects for more information.
<code>-fault_type subckt_open</code>	Specifies that the defect type is Subcircuit Open Defect (see Supported Built-in Defect Models). If you do not specify this argument, the default is <code>user_defined</code>. You can specify it to any name representative of the defect type.

Subcircuit Short Defect

```
inject_fault -subckt_inst sinst -type short -node n1 n2 -res_val rval
[-cap_val cval] [-weight weight_value] [-fault_type subckt_short]
```

Argument	Description
<code>-subckt_inst <i>sinst</i></code>	Specifies the name of the subcircuit instance.
<code>-type short</code>	Specifies to inject a short defect.
<code>-node <i>n1 n2</i></code>	Specifies the port names for the subcircuit instance short defect where a short defect is injected.
<code>-res_val <i>rval</i></code>	Specifies the resistance for the short defect models.
<code>-cap_val <i>cval</i></code>	Specifies the capacitance for the short defect model.
<code>-weight <i>weight_value</i></code>	Specifies the value of the weight. See the Specifying Weights for Defects section for more information.
<code>-fault_type subckt_short</code>	Specifies that the defect type is Subcircuit Short Defect (see Supported Built-in Defect Models). If you do not specify this argument, the default is <code>user_defined</code> . You can specify it to any name representative of the defect type.

Behavioral Resistor Open Defect

```
inject_fault -inst inst -type open -node n1 -res_val rval
[-cap_val cval] -weight weight_value -fault_type bhv_res_open
```

Argument	Description
<code>-inst <i>inst</i></code>	Specifies the name of the behavioral resistor instance.
<code>-type open</code>	Specifies to inject an open defect.
<code>-node <i>n1 n2</i></code>	Specifies the terminal name for the behavioral resistor open defect where an open defect is injected.
<code>-res_val <i>rval</i></code>	Specifies the resistance for the open defect model.
<code>-cap_val <i>cval</i></code>	Specifies the capacitance for the open defect model.
<code>-weight <i>weight_value</i></code>	Specifies the value of the weight. See Specifying Weights for Defects for more information.
<code>-fault_type bhv_res_open</code>	Specifies that the defect type is Behavioral Resistor Open Defect (see Supported Built-in Defect Models). If you do not specify this argument, the default is <code>user_defined</code> . You can specify it to any name representative of the defect type.

Behavioral Resistor Short Defect

```
inject_fault -inst inst -type short -node n1 n2 -res_val rval
[-cap_val cval] -weight weight_val -fault_type bhv_res_short
```

Argument	Description
-inst <i>inst</i>	Specifies the name of the behavioral resistor instance.
-type short	Specifies to inject a short defect.
-node <i>n1 n2</i>	Specifies the terminal names for the behavioral resistor short defect where a short defect is injected.
-res_val <i>rval</i>	Specifies the resistance for the short defect model.
-cap_val <i>cval</i>	Specifies the capacitance for the short defect model.
-weight <i>weight_value</i>	Specifies the value of the weight. See Specifying Weights for Defects for more information.
-fault_type bhv_res_short	Specifies that the defect type is Behavioral Resistor Short Defect (see Supported Built-in Defect Models). If you do not specify this argument, the default is <code>user_defined</code> . You can specify it to any name representative of the defect type.

Behavioral Capacitor Open Defect

```
inject_fault -inst inst -type open -node n1 [-res_val rval]
-cap_val cval -weight weight_value -fault_type bhv_cap_open
```

Argument	Description
-inst <i>inst</i>	Specifies the name of the behavioral capacitor instance.
-type open	Specifies to inject an open defect.
-node <i>n1 n2</i>	Specifies the terminal names for the behavioral capacitor open defect where an open defect is injected.
-res_val <i>rval</i>	Specifies the resistance for the open defect model.
-cap_val <i>cval</i>	Specifies the capacitance for the open defect model.
-weight <i>weight_value</i>	Specifies the value of the weight. See Specifying Weights for Defects for more information.

Argument	Description
<code>-fault_type bhv_cap_open</code>	Specifies that the defect type is Behavioral Capacitor Open Defect (see Supported Built-in Defect Models). If you do not specify this argument, the default is <code>user_defined</code> . You can specify it to any name representative of the defect type.

Behavioral Capacitor Short Defect

```
inject_fault -inst inst -type short -node n1 n2 [-res_val rval]
-cap_val cval -weight weight_val -fault_type bhv_cap_short
```

Argument	Description
<code>-inst inst</code>	Specifies the name of the behavioral capacitor instance.
<code>-type short</code>	Specifies to inject a short defect.
<code>-node n1 n2</code>	Specifies the terminal names for the behavioral capacitor short defect where a short defect is injected.
<code>-res_val rval</code>	Specifies the resistance for the short defect model.
<code>-cap_val cval</code>	Specifies the capacitance for the short defect model.
<code>-weight weight_value</code>	Specifies the value of the weight. See Specifying Weights for Defects for more information.
<code>-fault_type bhv_cap_short</code>	Specifies that the defect type is Behavioral Capacitor Short Defect (see Supported Built-in Defect Models). If you do not specify this argument, the default is <code>user_defined</code> . You can specify it to any name representative of the defect type.

MOSFET Resistor Divider Gate Open Defect

```
inject_fault -inst inst -type open -node g -res_val rval -rdiv 1
```

Argument	Description
<code>-inst inst</code>	Specifies the name of the MOSFET resistor divider gate instance.
<code>-type open</code>	Specifies to inject an open defect.
<code>-node g</code>	Specifies the terminal name for the MOSFET resistor divider gate open defect where an open defect is injected.

Argument	Description
<code>-res_val rval</code>	Specifies the resistance for the open defect model.
<code>-rdiv 1</code>	Specifies that the defect is a MOSFET resistor divider gate open defect.

Subcircuit MOSFET Resistor Divider Gate Open Defect

```
inject_fault -subckt_inst sinst -type open -node n1 n2 n3 -res_val rval
-rdiv 1 [-port_index 1]
```

Argument	Description
<code>-subckt_inst <i>sinst</i></code>	Specifies the name of the MOSFET resistor divider gate subcircuit instance.
<code>-type open</code>	Specifies to inject an open defect.
<code>-node <i>n1 n2 n3</i></code>	Specifies the port names for the MOSFET resistor divider gate open defect where an open defect is injected.
<code>-res_val <i>rval</i></code>	Specifies the resistance for the open defect model.
<code>-rdiv 1</code>	Specifies that the defect is a MOSFET resistor divider gate open defect.
<code>-port_index 1</code>	Specifies that arguments for <code>-node</code> are interpreted as indexes rather than names.

Net to Net Short Defect

```
inject_fault -type short -node n1 n2 -res_val rval
[-cap_val cval] [-weight weight_value] [-fault_type net_to_net_short]
```

Argument	Description
<code>-type short</code>	Specifies to inject a short defect.
<code>-node <i>n1 n2</i></code>	Specifies the node names for the net-to-net short defect where a short defect is injected.
<code>-res_val <i>rval</i></code>	Specifies the resistance for the short defect model.
<code>-cap_val <i>cval</i></code>	Specifies the capacitance for the short defect model.

Argument	Description
<code>-weight weight_value</code>	Specifies the value of the weight. See Specifying Weights for Defects for more information.
<code>-fault_type net_to_net_short</code>	Specifies that the defect type is Net-to-Net Short Defect (see Supported Built-in Defect Models). If you do not specify this argument, the default is <code>user_defined</code> . You can specify it to any name representative of the defect type.

Examples

The following examples show multiple entries in a defect list file.

```
# mos_short
inject_fault -inst x1.x1.x2.mp1 -type short -node s g -res_val
1.000000e-02 -cap_val 1.000000e-20
inject_fault -inst x2.x9.mp1 -type short -node s d -res_val 1.000000e-02
-cap_val 1.000000e-20
inject_fault -inst x2.mn1 -type short -node b g -res_val 1.000000e-02
-cap_val 1.000000e-20
inject_fault -inst x3.x2.x3.mn1 -type short -node s g -res_val
1.000000e-02 -cap_val 1.000000e-20
inject_fault -inst x4.x5.mp1 -type short -node s g -res_val 1.000000e-02
-cap_val 1.000000e-20

# mos_open
inject_fault -inst x1.x2.x1.mn1 -type open -node b -res_val 1.000000e+06
-cap_val 1.000000e-20
inject_fault -inst x2.x8.mp1 -type open -node d -res_val 1.000000e+06
-cap_val 1.000000e-20
inject_fault -inst x3.x1.x3.mn1 -type open -node b -res_val 1.000000e+06
-cap_val 1.000000e-20
inject_fault -inst x4.x8.mn2 -type open -node g -res_val 1.000000e+06
-cap_val 1.000000e-20
inject_fault -inst x4.x9.mn1 -type open -node g -res_val 1.000000e+06
-cap_val 1.000000e-20

# net_to_net_short
inject_fault -type short -node x1.x1.x2.vdd x1.x1.x2.in -res_val
1.000000e-02 -cap_val 1.000000e-20
inject_fault -type short -node x2.x1.x1.0 x2.x1.x1.in -res_val
1.000000e-02 -cap_val 1.000000e-20
inject_fault -type short -node x3.x1.x4.gp x3.x2.n4 -res_val 1.000000e-02
-cap_val 1.000000e-20
inject_fault -type short -node x4.k x4.x8.n1 -res_val 1.000000e-02
-cap_val 1.000000e-20
inject_fault -type short -node x4.x8.out x4.x8.in2 -res_val 1.000000e-02
-cap_val 1.000000e-20
```

In the previous three examples the tool reads in the defect list file and runs 15 analog fault simulations with one defect model injected for each simulation.

```
"fault_list_file" : "my_fault_file",
```

Multiple `inject_fault` commands can be specified on the same line separated by a comma (,). When multiple `inject_fault` commands are specified on the same line, the defects are injected at the same time. For example:

```
#to simulate an open via, inject multiple open defects:
inject_fault -inst xtop.xinv1.m1 -type open -node d -res_val 1.000000e+07
-cap_val 1.000000e-20 , inject_fault -subckt_inst xtop.xbuf.xma -type
open -node d -res_val 1.000000e+07 -cap_val 1.000000e-20 , inject_fault
-subckt_inst xtop.xdec.xmpa -type open -node d -res_val 1.000000e+07
-cap_val 1.000000e-20 , inject_fault -subckt_inst xtop.xctrl.xmnb -type
open -node g -res_val 1.000000e+07 -cap_val 1.000000e-20 , inject_fault
-subckt_inst xtop.xctrl.xmpb -type open -node g -res_val 1.000000e+07
-cap_val 1.000000e-20
```

Parametric Defect

Parametric defect injection addresses circumstances when a design might fail not only because of a hardware defect, but also because of a process parameter excursion that occurred during high-volume manufacturing. The Custom Fault tool provides a way to inject process parameter excursion with the manual injection flow using width (w), length (L), Vth, Ids, and Ig.

To define a parametric defect you provide the instance name and the variation of parameter. Only the MOS element is currently supported.

```
inject_fault -inst inst_name -param_name param_name -del_val param_dval
[-fault_type defect_name_in_report]
```

Argument	Description
-inst inst_name	Instance in which to inject the parametric defect.
-param_name param_name	Parameter name for the parametric defect.
-del_val param_dval	Percentage change of the parameter value.
-fault_typedefect_name_in_report	Defect name in the report file.

Table 19 and Table 20 show the supported MOS parameters, models, and supported parameter values.

Table 19 Supported MOS Parameters and Model

MOS Parameter	Model
W, L, Vth, Ids, Ig	BSIM3v3
W, L, Vth, Ids, Ig	BSIM4
W, L, Vth, Ids, Ig	CustomCMI
L, Vth, Ids, Ig	BSIMCMG

Table 20 Supported Parameter Values

Parameter	Absolute Value ¹	Relative Value ²
W	Yes	Yes
L	Yes	Yes
Vth	Yes	No
Ids	No	Yes
Ig	No	Yes

Parametric Defect Examples

The following example shows a single `inject_fault` command that sets a relative width value increase of 30%.

```
inject_fault -inst xbias.xbiascore.xnm7.main -param_name w -del_val 30%
-fault_type PARAM_REL_Width
```

The resulting report has the `-fault_type` value in Defect location:

```
A97.97  PARAM_REL_Width  NC  fault_PARAM_REL_Width_96_97_97.*  1.000000e+00
```

The following example shows multiple `inject_fault` commands. You can specify a file that contains these commands in a command-line option or configuration file.

Command line:

```
-f fault_list.txt
```

1. Value has an absolute value and is a rational number.
2. Value has a relative value change that requires a percentage variation. If you do not provide a percentage variation the simulation errors out.

Configuration file:

```
"fault_list_file" : "fault_list_file.txt"
```

For example:

```
inject_fault -inst xbias.xbiascore.xnm7.main -param_name ids -del_val 30% -fault_type PARAM_IDS
inject_fault -inst xbias.xbiascore.xnm7.main -param_name ig -del_val 200% -fault_type PARAM_IG
inject_fault -inst xbias.xbiascore.xnm7.main -param_name vth -del_val 0.1 -fault_type PARAM_VTH
inject_fault -inst xbias.xbiascore.xnm7.main -param_name l -del_val 1e-6 -fault_type PARAM_Length
inject_fault -inst xbias.xbiascore.xnm7.main -param_name l -del_val 30% -fault_type PARAM_REL_Length
inject_fault -inst xbias.xbiascore.xnm7.main -param_name w -del_val 30% -fault_type PARAM_REL_Width
inject_fault -inst xbias.xbiascore.xnm7.main -param_name w -del_val 1e-6 -fault_type PARAM_Width
```

Id	Defect multi_list		Simulation Data	Group Weight	In
A1.1	PARAM_IDS	NC	fault_PARAM_IDS_0_1_1.*	1.000000e+00	
A2.2	PARAM_IG	NC	fault_PARAM_IG_1_2_2.*	1.000000e+00	
A3.3	PARAM_VTH	NC	fault_PARAM_VTH_2_3_3.*	1.000000e+00	
A4.4	PARAM_Length	NC	fault_PARAM_Length_3_4_4.*	1.000000e+00	
A5.5	PARAM_REL_Length	CU	fault_PARAM_REL_Length_4_5_5.*	1.000000e+00	
A6.6	PARAM_REL_Width	NC	fault_PARAM_REL_Width_5_6_6.*	1.000000e+00	
A7.7	PARAM_Width	CU	fault_PARAM_Width_6_7_7.*	1.000000e+00	

Manual Injection Flow Tutorial

The tutorial described in this section demonstrates the following:

- [Providing the Defect List File for the Manual Injection Flow](#)
- [Running the Custom Fault Manual Injection Flow With PrimeSim XA](#)
- [Running the Custom Fault Manual Injection Flow With FineSim](#)
- [Running the Custom Fault Manual Injection Flow With PrimeSim](#)

Before starting the tutorial, make sure that all tools are installed correctly.

[Table 21](#) lists the files needed for this tutorial. These files are located in the `install_directory/doc/tutorials/manual_flow` directory.

Table 21 Manual Injection Flow Tutorial Files

File	Description
<code>adder.vec</code>	Vector file containing both input and output vectors.
<code>cf_configfile_xa</code>	Configuration file for the Custom Fault tool with the PrimeSim XA tool.
<code>cf_configfile_fsim</code>	Configuration file for the Custom Fault tool with the FineSim tool.
<code>cf_configfile_psim</code>	Configuration file for the Custom Fault tool with the PrimeSim tool.

Table 21 *Manual Injection Flow Tutorial Files (Continued)*

File	Description
<code>cmd_load_vec_file</code>	Command file for the PrimeSim XA tool.
<code>top_no_vec_xa.spi</code>	Top-level netlist of a 4-bit adder for the PrimeSim XA tool.
<code>top_fsm.spi</code>	Top-level netlist of a 4-bit adder for the FineSim tool.
<code>top_psim.spi</code>	Top-level netlist of a 4-bit adder for the PrimeSim tool.
<code>xhost/xhost_sh_launch</code>	Host file for distributing processing.
<code>run_manual_xa</code>	Run script to run the manual defect injection flow with the Custom Fault and PrimeSim XA tools.
<code>run_manual_fsm</code>	Run script to run the manual defect injection flow with the Custom Fault and FineSim tools.
<code>run_manual_psim</code>	Run script to run the manual defect injection flow with the Custom Fault and PrimeSim tools.

The defect detection checking criteria is done by vector check. The `adder.vec` file contains the output vectors of `COUT` and `S[3:0]`. If the simulation output matches with the expected states, then no violation is reported, and the defect is not covered. Otherwise, a violation is reported, and the defect is covered.

The `load_vector_file` command is specified in `cmd_load_vec_file` to load in the `adder.vec` file. To speed up the throughput, the `set_vector_option -error 1` command forces the simulation to exit when it encounters a comparison error.

The content of `cmd_load_vec_file` is:

```
# set command to probe voltage signals
probe_waveform voltage *
# set command for speed-accuracy tradeoff
set_sim_level 3
# command to load the vector file
load_vector_file -file adder.vec
# set option to stop simulation when there is a comparison error
set_vector_option -error 1
```

Providing the Defect List File for the Manual Injection Flow

To enable manual injection flow, you must provide a defect list file that contains the types of defect injections.

The `manual_fault_list` file for this tutorial is:

```
# mos_short
inject_fault -inst x1.x1.x2.mp1 -type short -node s g -res_val
1.000000e-02
inject_fault -inst x2.x9.mp1 -type short -node s d -res_val 1.000000e-02
inject_fault -inst x2.mn1 -type short -node b g -res_val 1.000000e-02
inject_fault -inst x3.x2.x3.mn1 -type short -node s g -res_val
1.000000e-02
inject_fault -inst x4.x5.mp1 -type short -node s g -res_val 1.000000e-02

# mos_open
inject_fault -inst x1.x2.x1.mn1 -type open -node b -res_val 1.000000e+06
inject_fault -inst x2.x8.mp1 -type open -node d -res_val 1.000000e+06
inject_fault -inst x3.x1.x3.mn1 -type open -node b -res_val 1.000000e+06
inject_fault -inst x4.x8.mn2 -type open -node g -res_val 1.000000e+06
inject_fault -inst x4.x9.mn1 -type open -node g -res_val 1.000000e+06

# net_to_net_short
inject_fault -type short -node x1.x1.x2.vdd x1.x1.x2.in -res_val
1.000000e-02
inject_fault -type short -node x2.x1.x1.0 x2.x1.x1.in -res_val
1.000000e-02
inject_fault -type short -node x3.x1.x4.gp x3.x2.n4 -res_val 1.000000e-02
inject_fault -type short -node x4.k x4.x8.n1 -res_val 1.000000e-02
inject_fault -type short -node x4.x8.out x4.x8.in2 -res_val 1.000000e-02
```

Running the Custom Fault Manual Injection Flow With PrimeSim XA

This section describes how to run the manual defect injection flow with the Custom Fault and PrimeSim XA tools in batch mode.

- [Creating the Configuration File to Run Custom Fault With PrimeSim XA](#)
- [Running the Custom Fault Tool With PrimeSim XA](#)
- [Generating Fault Simulation Output Using PrimeSim XA](#)

Creating the Configuration File to Run Custom Fault With PrimeSim XA

To run a Custom Fault simulation, a configuration file is mandatory. The configuration file supplies all the information to the Custom Fault tool, including the top-level netlist name, simulator used, and so on.

The configuration file contains the following information:

- The simulator used for transient simulation is the PrimeSim XA tool (`xa`).
- The maximum runtime allowed per simulation (`cpulimit`) is 200s.
- The directory in which to find the data and run the simulation (`case_dir`).
- The path to the host file (`dpconfig_file`) for distributing processing is `xhost/xhost_sh_launch`.
- The top-level netlist (`top_no_vec_xa.spi`) is in the current working directory.
- The command-line prefix (`sim_cmdline`) to run the PrimeSim XA tool is `-hspice` and the command-line suffix is `-c cmd_load_vec_file`.
- The short defect is modeled by a resistor with value of 200 ohms.
- The open defect is modeled by a resistor with value of 10e6 ohms.

The content of the `cf_configfile_xa` file is:

```
{
  "case_dir" : "./",
  "cpulimit" : 600,
  "analog_setup" : {
    "sim_cmdline": "xa -hspice top_no_vec_xa.spi -c
cmd_load_vec_file"
  },
  "dpconfig_file" : "./xhost/xhost_sh_launch",
  "faults" : {
    "defaults" : {
      "short" : {
        "res" : 200,
        "cap" : null
      },
      "open" : {
        "res" : 10e6,
        "cap" : null
      }
    }
  }
}
```

Running the Custom Fault Tool With PrimeSim XA

The `run_manual_xa` file contains the command to run the Custom Fault tool. The `-f` command-line option reads in the `manual_fault_list` file to only run the list of defects specified in the file.

```
customfault -c cf_configfile_xa -o RESULT/xa_defect
\ -t manual_cf_xa -f manual_fault_list -k 5
```

To invoke the Custom Fault tool, run `run_manual_xa`:

```
$> run_manual_xa
```

Generating Fault Simulation Output Using PrimeSim XA

After the simulation completes, the Custom Fault tool generates the following output files and directory inside the `RESULT` directory shown in [Table 22](#).

Table 22 Custom Fault Output Files for the PrimeSim XA Tool

Output File	Description
<code>manual_cf_xa/*</code>	Output directory containing all the intermediate files. The name of the directory is determined by the simulation tag name specified by the <code>-t</code> command-line option.
<code>xa_defect.config.json</code>	Configuration file to run individual fault simulation.
<code>xa_defect.fdb</code>	Fault simulation database.
<code>xa_defect.log</code>	Log file.
<code>xa_defect.report</code>	Coverage report of the fault simulation.

The following example shows the summary section of the coverage report (`xa_defect.report`). Note that the coverage percentage might be slightly different based on the version of the PrimeSim XA tool you use.

```

                Tags                Sample-Size
manual_cf_xa                Exhaustive

*DEFECT UNIVERSE SUMMARY

    user_defined                15
                TOTAL                15

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
    Type                manual_cf_xa

    user_defined                12/15
                TOTAL                12/15

Weighted test coverage:
    manual_cf_xa                80.00%
```

```
TOTAL Coverage:
Non-Sampled      80.00%

Total weight of defect universe: 15.00
```

The coverage report shows the following information:

- The simulation tag name is `manual_cf_xa`.
- There are 15 user-defined defects.
- 12 out of 15 user-defined defects are covered.
- The test coverage is 80.00%.

The `*DETAILED COVERAGE LISTING` section shows the detail of each fault simulation. The log file of each individual fault simulation can be found in `RESULT/manual_cf_xa`.

Running the Custom Fault Manual Injection Flow With FineSim

This section describes how to run the manual defect injection flow with the Custom Fault and FineSim tools in batch mode.

- [Creating the Configuration File to Run Custom Fault With FineSim](#)
- [Running the Custom Fault Tool With FineSim](#)
- [Generating Fault Simulation Output Using FineSim](#)

Creating the Configuration File to Run Custom Fault With FineSim

To run a Custom Fault simulation, a configuration file is mandatory. The configuration file supplies all the information to the Custom Fault tool, including the top-level netlist name, simulator used, and so on.

The configuration file contains the following information:

- The simulator used for transient simulation is the FineSim tool (`finesim`).
- The maximum runtime allowed per simulation (`cpulimit`) is 200s.
- The directory in which to find the data and run the simulation (`case_dir`).
- The path to the host file (`dpconfig_file`) for distributing processing is `xhost/xhost_sh_launch`.
- The top-level netlist (`top_fsim.spi`) is in the current working directory.
- There is no command-line prefix and suffix.

- The short defect is modeled by a resistor with value of 200 ohms.
- The open defect is modeled by a resistor with value of 10e6 ohms.

The content of the `cf_configfile_fsim` file is:

```
{
  "case_dir" : "./",
  "cpulimit" : 600,
  "analog_setup" : {
    "sim_cmdline": "finesim top_fsim.spi"
  },
  "dpconfig_file" : "./xhost/xhost_sh_launch",
  "faults" : {
    "defaults" : {
      "short" : {
        "res" : 200,
        "cap" : null
      },
      "open" : {
        "res" : 10e6,
        "cap" : null
      }
    }
  }
}
```

Running the Custom Fault Tool With FineSim

The `run_manual_fsim` file contains the command to run the Custom Fault tool. The `-f` command-line option reads in the `manual_fault_list` file to only run the list of defects specified in the file.

```
customfault -c cf_configfile_fsim -o RESULT/fsim_defect
\ -t manual_cf_fsim -f manual_fault_list -k 5
```

To invoke the Custom Fault tool, run `run_manual_fsim`:

```
$> run_manual_fsim
```

Generating Fault Simulation Output Using FineSim

After the simulation completes, the Custom Fault tool generates the following output files and directory inside the `RESULT` directory shown in [Table 23](#).

Table 23 Custom Fault Output Files for the FineSim Tool

Output File	Description
<code>manual_cf_fsim/*</code>	Output directory containing all the intermediate files. The name of the directory is determined by the simulation tag name specified by the <code>-t</code> command-line option.

Table 23 Custom Fault Output Files for the FineSim Tool (Continued)

Output File	Description
<code>fsim_defect.config.json</code>	Configuration file to run individual fault simulation.
<code>fsim_defect.fdb</code>	Fault simulation database.
<code>fsim_defect.log</code>	Log file.
<code>fsim_defect.report</code>	Coverage report of the fault simulation.

The following example shows the summary section of the coverage report (`fsim_defect.report`). Note that the coverage percentage might be slightly different based on the version of the FineSim tool you use.

```

                Tags                Sample-Size
manual_cf_fsim                Exhaustive

*DEFECT UNIVERSE SUMMARY

    user_defined                15
                TOTAL                15

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
    Type                manual_cf_fsim

    user_defined                13/15
                TOTAL                13/15

Weighted test coverage:
    manual_cf_fsim                86.67%

TOTAL Coverage:
    Non-Sampled                86.67%

Total weight of defect universe: 15.00

```

The coverage report shows the following information:

- The simulation tag name is `manual_cf_fsim`.
- There are 15 user-defined defects.

- 13 out of 15 user-defined defects are covered.
- The test coverage is 86.67%.

The `*DETAILED COVERAGE LISTING` section shows the detail of each fault simulation. The log file of each individual fault simulation can be found in `RESULT/manual_cf_fsim`.

Running the Custom Fault Manual Injection Flow With PrimeSim

This section describes how to run the manual defect injection flow with the Custom Fault and PrimeSim tools in batch mode.

- [Creating the Configuration File to Run Custom Fault With PrimeSim](#)
- [Running the Custom Fault Tool With PrimeSim](#)
- [Generating Fault Simulation Output Using PrimeSim](#)

Creating the Configuration File to Run Custom Fault With PrimeSim

To run a Custom Fault simulation, a configuration file is mandatory. The configuration file supplies all the information to the Custom Fault tool, including the top-level netlist name, simulator used, and so on.

The configuration file contains the following information:

- The simulator used for transient simulation is the PrimeSim tool (`primesim`).
- The maximum runtime allowed per simulation (`cpulimit`) is 200s.
- The directory in which to find the data and run the simulation (`case_dir`).
- The path to the host file (`dpconfig_file`) for distributing processing is `xhost/xhost_sh_launch`.
- The top-level netlist (`top_psim.spi`) is in the current working directory.
- There is no command-line prefix and suffix.
- The short defect is modeled by a resistor with value of 200 ohms.
- The open defect is modeled by a resistor with value of 10e6 ohms.

The content of the `cf_configfile_psim` file is:

```
{
  "case_dir" : "./",
  "cpulimit" : 600,
  "analog_setup" : {
    "sim_cmdline": "primesim top_psim.spi"
```

```

    },
    "dpconfig_file" : "./xhost/xhost_sh_launch",
    "faults" : {
        "defaults" : {
            "short" : {
                "res" : 200,
                "cap" : null
            },
            "open" : {
                "res" : 10e6,
                "cap" : null
            }
        }
    }
}

```

Running the Custom Fault Tool With PrimeSim

The `run_manual_psim` file contains the command to run the Custom Fault tool. The `-f` command-line option reads in the `manual_fault_list` file to only run the list of defects specified in the file.

```

customfault -c cf_configfile_psim -o RESULT/psim_defect
\ -t manual_cf_psim -f manual_fault_list -k 5

```

To invoke the Custom Fault tool, run `run_manual_psim`:

```
$> run_manual_psim
```

Generating Fault Simulation Output Using PrimeSim

After the simulation completes, the Custom Fault tool generates the following output files and directory inside the `RESULT` directory shown in [Table 24](#).

Table 24 Custom Fault Output Files for the PrimeSim Tool

Output File	Description
<code>manual_cf_psim/*</code>	Output directory containing all the intermediate files. The name of the directory is determined by the simulation tag name specified by the <code>-t</code> command-line option.
<code>psim_defect.config.json</code>	Configuration file to run individual fault simulation.
<code>psim_defect.fdb</code>	Fault simulation database.
<code>psim_defect.log</code>	Log file.
<code>psim_defect.report</code>	Coverage report of the fault simulation.

The following example shows the summary section of the coverage report (`psim_defect.report`). Note that the coverage percentage might be slightly different based on the version of the PrimeSim tool you use.

```

                Tags                Sample-Size
manual_cf_psim           Exhaustive

*DEFECT UNIVERSE SUMMARY

    user_defined                15
                TOTAL                15

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
    Type                manual_cf_psim

    user_defined                13/15
                TOTAL                13/15

Weighted test coverage:
    manual_cf_psim                86.67%

TOTAL Coverage:
    Non-Sampled                86.67%

Total weight of defect universe: 15.00
```

The coverage report shows the following information:

- The simulation tag name is `manual_cf_psim`.
- There are 15 user-defined defects.
- 13 out of 15 user-defined defects are covered.
- The test coverage is 86.67%.

The `*DETAILED COVERAGE LISTING` section shows the detail of each fault simulation. The log file of each individual fault simulation can be found in `RESULT/manual_cf_psim`.

8

Custom Fault Multi-Testbench Flow

This chapter covers the multiple testbenches flow for analog fault simulation.

- [Overview of the Multi-Testbench Flow](#)
- [Multi-Testbench Flow Tutorial](#)

Overview of the Multi-Testbench Flow

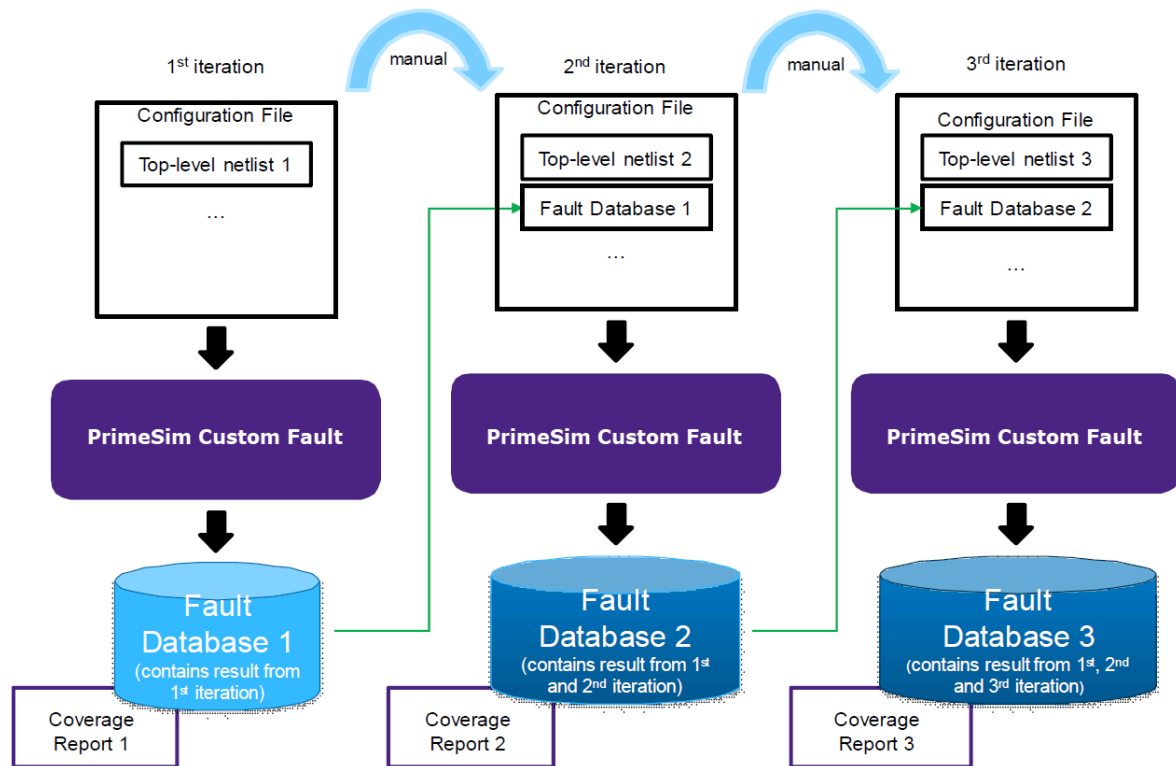
To increase the coverage, multiple test patterns can be simulated for the same design. The tool supports a methodology that allows you to run the same design with multiple testbenches and merge the analog fault simulation results into one coverage report.

[Figure 39](#) shows the methodology of multi-testbench flow.

After the first iteration of analog fault simulation a fault database file that contains the analog fault simulation data and the coverage report is generated. The fault database file (*.fdb) from first iteration is reused in the second iteration of analog fault simulation using the `fault_list_file` command or `-f` command-line option with a change in the testbench.

The outputs of second iteration of analog fault simulation are a new fault database file that contains the analog fault simulation data from both first and second iterations, and a combined coverage report from the first and second iterations. The iterations can continue for as many testbenches as you need to run analog fault simulation to achieve the needed coverage.

Figure 39 Multi-Testbench Flow



If a fault database file is used as an input to analog fault simulation, you can use the [simulate_only](#) command to run the simulation on the non-covered defect model to improve throughput.

Multi-Testbench Flow Tutorial

The tutorial described in this section demonstrates the following:

- [Running the Custom Fault Multi-Testbench Flow With PrimeSim XA](#)
- [Running the Custom Fault Multi-Testbench Flow With FineSim](#)
- [Running the Custom Fault Multi-Testbench Flow With PrimeSim](#)

Before starting these tutorials, make sure that all tools are installed correctly.

[Table 25](#) lists the files needed for this tutorial. These files are located in the `install_directory/doc/tutorials/multi-testbench_flow` directory.

Table 25 *Multi-Testbench Flow Tutorial Files*

File	Description
adder_1st.vec	Vector file used for the first analog fault simulation.
adder_2nd.vec	Vector file used for the second analog fault simulation.
cmd_load_vec_file_1st	PrimeSim XA command file for the first analog simulation.
cmd_load_vec_file_2nd	PrimeSim XA command file for the second analog simulation.
configfile_1st_xa	Configuration file for the first analog fault simulation for the Custom Fault tool with the PrimeSim XA tool.
configfile_1st_fsim	Configuration file for the first analog fault simulation for the Custom Fault tool with the FineSim tool.
configfile_1st_psim	Configuration file for the first analog fault simulation for the Custom Fault tool with the PrimeSim tool.
configfile_2nd_xa	Configuration file for the second analog fault simulation for the Custom Fault tool with the PrimeSim XA tool..
configfile_2nd_fsim	Configuration file for the second analog fault simulation for the Custom Fault tool with the FineSim tool.
configfile_2nd_psim	Configuration file for the second analog fault simulation for the Custom Fault tool with the PrimeSim tool.
run_cf_xa	Run script for both first and second analog simulation using the Custom Fault and PrimeSim XA tools.
run_cf_fsim	Run script for both first and second analog simulation using the Custom Fault and FineSim tools.
run_cf_psim	Run script for both first and second analog simulation using the Custom Fault and PrimeSim tools.
top_xa.spi	Top-level netlist for the PrimeSim XA tool.
top_fsim_1st_vec.spi	Top-level netlist for the first testbench and the FineSim tool.
top_fsim_2nd_vec.spi	Top-level netlist for the second testbench and the FineSim tool.
top_psim_1st_vec.spi	Top-level netlist for the first testbench and the PrimeSim tool.
top_psim_2nd_vec.spi	Top-level netlist for the second testbench and the PrimeSim tool.
xhost/xhost_sh_launch	Host file for distributed processing.

This tutorial runs two analog fault simulations with two testbenches.

- The first analog fault simulation uses the `adder_1st.vec` testbench. The database and the result of the first analog fault simulation is saved. The simulation tag name is `sim_1st_vec_<xa|fsim|psim>`.
- The second analog fault simulation, with the simulation tag name of `sim_2nd_vec_<xa|fsim|psim>`, uses the database and the result of the first analog fault simulation as an input. It also uses a different testbench, `adder_2nd.vec`.

In `sim_2nd_vec_<xa|fsim|psim>`, only the defects that are not covered (NC) in `sim_1st_vec_<xa|fsim|psim>` are re-simulated. The output of the second analog fault simulation is a combined coverage report from `sim_1st_vec_<xa|fsim|psim>` and `sim_2nd_vec_<xa|fsim|psim>`.

Running the Custom Fault Multi-Testbench Flow With PrimeSim XA

This section describes the files and commands needed to run the multi-testbench flow with the Custom Fault and PrimeSim XA tool in batch mode.

- [Creating the Configuration File to Run Custom Fault With PrimeSim XA](#)
- [Running the Custom Fault Tool With PrimeSim XA](#)
- [Generating Multi-Testbench Simulation Output Using PrimeSim XA](#)

Creating the Configuration File to Run Custom Fault With PrimeSim XA

The first analog fault simulation has the tag name `sim_1st_vec_xa`. It reads in the configuration file `configfile_1st_xa`. The output files are directed to the `RESULT` directory with the prefix `sim_1st_vec_xa.*`.

The second analog fault simulation has the simulation tag name `sim_2nd_vec_xa`. It reads in the configuration file `configfile_2nd_xa`. It also reads in the database file `RESULT/sim_1st_vec_xa.fdb` from `sim_1st_vec_xa` using the `-f` command-line option. The output files are directed to `RESULT` directory with the prefix `sim_2nd_vec_xa.*`. The coverage report contains the results of `sim_1st_vec_xa` and `sim_2nd_vec_xa`.

The `configfile_1st_xa` file reads in the PrimeSim XA command file `cmd_load_vec_file_1st` that contains the `load_vector_file -file adder_1st.vec` command.

The content of the `configfile_1st_xa` file is:

```
{  
    "cpulimit" : 3600,  
    "cpulimit" : 3600,
```

```

    "case_dir" : "./",
    "analog_setup" : {
        "sim_cmdline": "xa -hspice top_xa.spi -c cmd_load_vec_file_1st"
    },
    "dpconfig_file" : "./xhost/xhost_sh_launch",
    "faults" : {
        "defaults" : {
            "short" : {
                "res" : 0.01,
                "cap" : null
            },
            "open" : {
                "res" : 1e6,
                "cap" : null
            }
        }
    }
}

```

The `configfile_2nd_xa` file reads in the PrimeSim XA command file `cmd_load_vec_file_2nd` that contains the `load_vector_file -file adder_2nd.vec` command. The `"simulate_only" : { "covered" : false }` command is specified in `configfile_2nd_xa` to simulate only the defects not covered in `sim_1st_vec_xa`.

The content of the `configfile_2nd_xa` file is:

```

{
    "cpulimit" : 3600,
    "case_dir" : "./",
    "analog_setup" : {
        "sim_cmdline": "xa -hspice top_xa.spi -c cmd_load_vec_file_2nd"
    },
    "simulate_only" : {
        "covered" : false
    },
    "dpconfig_file" : "./xhost/xhost_sh_launch",
    "faults" : {
        "defaults" : {
            "short" : {
                "res" : 0.01,
                "cap" : null
            },
            "open" : {
                "res" : 1e6,
                "cap" : null
            }
        }
    }
}

```

Running the Custom Fault Tool With PrimeSim XA

The contents of the `run_cf_xa` run script file are:

```
customfault -c configfile_1st_xa -o RESULT/sim_1st_vec_xa  
\ -t sim_1st_vec_xa -k 5  
customfault -c configfile_2nd_xa -o RESULT/sim_2nd_vec_xa  
\ -t sim_2nd_vec_xa -f RESULT/sim_1st_vec_xa.fdb -k 5
```

To run the multi-testbench tutorial, type:

```
$> run_cf_xa
```

Generating Multi-Testbench Simulation Output Using PrimeSim XA

After the simulation completes, the Custom Fault tool generates the following output files and directory inside the `RESULT` directory shown in [Table 26](#).

Table 26 Custom Fault Output Files for the PrimeSim XA Tool

Output File	Description
<code>sim_1st_vec_xa/*</code>	Output directory containing all the intermediate files from the first analog fault simulation, <code>sim_1st_vec_xa</code> . The name of the directory is determined by the simulation tag name specified by the <code>-t</code> command-line option.
<code>sim_1st_vec_xa.config.json</code>	Configuration file to run the first analog fault simulation, <code>sim_1st_vec_xa</code> .
<code>sim_1st_vec_xa.fdb</code>	Fault simulation database of the first analog fault simulation, <code>sim_1st_vec_xa</code> .
<code>sim_1st_vec_xa.log</code>	Log file of the first analog fault simulation, <code>sim_1st_vec_xa</code> .
<code>sim_1st_vec_xa.report</code>	Coverage report of the first analog fault simulation, <code>sim_1st_vec_xa</code> .
<code>sim_1st_vec_xa.covered.list</code>	List of defects covered successfully in the first analog fault simulation, <code>sim_1st_vec_xa</code> .
<code>sim_1st_vec_xa.uncovered.list</code>	List of defects NOT covered by the input stimulus in the first analog fault simulation, <code>sim_1st_vec_xa</code> .
<code>sim_1st_vec_xa_weights.cpp</code>	Weight file associating each defect with its corresponding assigned weight value in the first analog fault simulation, <code>sim_1st_vec_xa</code> .

Table 26 Custom Fault Output Files for the PrimeSim XA Tool (Continued)

Output File	Description
<code>sim_2nd_vec_xa/*</code>	Output directory containing all the intermediate files from the second analog fault simulation, <code>sim_2nd_vec_xa</code> . The name of the directory is determined by the simulation tag name specified by the <code>-t</code> command-line option.
<code>sim_2nd_vec_xa.config.json</code>	Configuration file to run the second analog fault simulation, <code>sim_2nd_vec_xa</code> .
<code>sim_2nd_vec_xa.fdb</code>	Fault simulation database of the second analog fault simulation, <code>sim_2nd_vec_xa</code> .
<code>sim_2nd_vec_xa.log</code>	Log file of the second analog fault simulation, <code>sim_2nd_vec_xa</code> .
<code>sim_2nd_vec_xa.report</code>	Coverage report of the second analog fault simulation, <code>sim_2nd_vec_xa</code> .
<code>sim_2nd_vec_xa.covered.list</code>	List of defects covered successfully in the second analog fault simulation, <code>sim_2nd_vec_xa</code> .
<code>sim_2nd_vec_xa.uncovered.list</code>	List of defects NOT covered by the input stimulus in the second analog fault simulation, <code>sim_2nd_vec_xa</code> .
<code>sim_2nd_vec_xa_weights.cpp</code>	Weight file associating each defect with its corresponding assigned weight value in the second analog fault simulation, <code>sim_2nd_vec_xa</code> .

The following example shows a summary section of the coverage report (`sim_2nd_vec_xa.report`). Note that the coverage percentage might be slightly different based on the version of the PrimeSim XA tool you use.

```

                Tags          Sample-Size
sim_1st_vec_xa    Exhaustive
sim_2nd_vec_xa    Exhaustive

*DEFECT UNIVERSE SUMMARY

    mos_open                656
    mos_short               864
    TOTAL                   1520

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
    Type          sim_1st_vec_xa          sim_2nd_vec_xa          Total
(Union)
```



```

mos_open          405/656          7/251
  412/656
mos_short          788/864          1/76
  789/864
TOTAL             1193/1520        8/327
  1201/1520

Weighted test coverage:
  sim_1st_vec_xa      78.49%
+ sim_2nd_vec_xa      79.01%

TOTAL Coverage:
  Non-Sampled         79.01%

Total weight of defect universe: 1520.00

```

The coverage report shows the following information:

- The simulation tag names are `sim_1st_vec_xa` and `sim_2nd_vec_xa`.
- There are 656 `mos_open` defects and 864 `mos_short` defects.
- 405 out of 656 `mos_open` defects and 788 out of 864 `mos_short` defects are covered in `sim_1st_vec_xa`.
- 7 out of 251 `mos_open` defects and 1 out of 76 `mos_short` defects are covered in `sim_2nd_vec_xa`.
- 1193 out of 1520 defects are covered in `sim_1st_vec_xa` with the test coverage of 78.49%.
- 8 out of 327 defects are covered in `sim_2nd_vec_xa` with the test coverage of 2.45%.
- The combined test coverage of `sim_1st_vec_xa` and `sim_2nd_vec_xa` is 79.01%.

The `*DETAILED COVERAGE LISTING` section shows the detail of each fault simulation. The log file of each individual fault simulation can be found in `RESULT/sim_1st_vec_xa` and `RESULT/sim_2nd_vec_xa`.

Running the Custom Fault Multi-Testbench Flow With FineSim

This section describes the files and commands needed to run the multi-testbench flow with the Custom Fault and FineSim tool in batch mode.

- [Creating the Configuration File to Run Custom Fault With FineSim](#)
- [Running the Custom Fault Tool With FineSim](#)
- [Generating Multi-Testbench Simulation Output Using FineSim](#)

Creating the Configuration File to Run Custom Fault With FineSim

The first analog fault simulation has the tag name `sim_1st_vec_fsim`. It reads in the configuration file `configfile_1st_fsim`. The output files are directed to the `RESULT` directory with the prefix `sim_1st_vec_fsim.*`.

The second analog fault simulation has the simulation tag name `sim_2nd_vec_fsim`. It reads in the configuration file `configfile_2nd_fsim`. It also reads in the database file `RESULT/sim_1st_vec_fsim.fdb` from `sim_1st_vec_fsim` using the `-f` command-line option. The output files are directed to `RESULT` directory with the prefix `sim_2nd_vec_fsim.*`. The coverage report contains the results of `sim_1st_vec_fsim` and `sim_2nd_vec_fsim`.

The content of the `configfile_1st_fsim` file is:

```
{
  "cpulimit" : 3600,
  "case_dir" : "./",
  "analog_setup" : {
    "sim_cmdline": "finesim top_fsim_1st_vec.spi"
  },
  "dpconfig_file" : "./xhost/xhost_sh_launch",
  "faults" : {
    "defaults" : {
      "short" : {
        "res" : 200,
        "cap" : null
      },
      "open" : {
        "res" : 10e6,
        "cap" : null
      }
    }
  }
}
```

The `"simulate_only" : { "covered" : false }` command is specified in `configfile_2nd_fsim` to simulate only the defects not covered in `sim_1st_vec_fsim`.

The content of the `configfile_2nd_fsim` file is:

```
{
  "cpulimit" : 3600,
  "case_dir" : "./",
  "analog_setup" : {
    "sim_cmdline": "finesim top_fsim_2nd_vec.spi"
  },
  "simulate_only" : {
    "covered" : false
  },
  "dpconfig_file" : "./xhost/xhost_sh_launch",
  "faults" : {
```

```
        "defaults" : {
            "short" : {
                "res" : 200,
                "cap" : null
            },
            "open" : {
                "res" : 10e6,
                "cap" : null
            }
        }
    }
}
```

Running the Custom Fault Tool With FineSim

The contents of the `run_cf_fsim` run script file are:

```
customfault -c configfile_1st_fsim -o RESULT/sim_1st_vec_fsim
\ -t sim_1st_vec_fsim -k 5
customfault -c configfile_2nd_fsim -o RESULT/sim_2nd_vec_fsim
\ -t sim_2nd_vec_fsim -f RESULT/sim_1st_vec_fsim.fdb -k 5
```

To run the multi-testbench tutorial, type:

```
$> run_cf_fsim
```

Generating Multi-Testbench Simulation Output Using FineSim

After the simulation completes, the Custom Fault tool generates the following output files and directory inside the `RESULT` directory shown in [Table 27](#).

Table 27 Custom Fault Output Files for the FineSim Tool

Output File	Description
<code>sim_1st_vec_fsim/*</code>	Output directory containing all the intermediate files from the first analog fault simulation, <code>sim_1st_vec_fsim</code> . The name of the directory is determined by the simulation tag name specified by the <code>-t</code> command-line option.
<code>sim_1st_vec_fsim.config.json</code>	Configuration file to run the first analog fault simulation, <code>sim_1st_vec_fsim</code> .
<code>sim_1st_vec_fsim.fdb</code>	Fault simulation database of the first analog fault simulation, <code>sim_1st_vec_fsim</code> .
<code>sim_1st_vec_fsim.log</code>	Log file of the first analog fault simulation, <code>sim_1st_vec_fsim</code> .
<code>sim_1st_vec_fsim.report</code>	Coverage report of the first analog fault simulation, <code>sim_1st_vec_fsim</code> .
<code>sim_1st_vec_fsim.covered.list</code>	List of defects covered successfully in the first analog fault simulation, <code>sim_1st_vec_fsim</code> .

Table 27 Custom Fault Output Files for the FineSim Tool (Continued)

Output File	Description
<code>sim_1st_vec_fsim.uncovered.list</code>	List of defects NOT covered by the input stimulus in the first analog fault simulation, <code>sim_1st_vec_fsim</code> .
<code>sim_1st_vec_fsim_weights.cpp</code>	Weight file associating each defect with its corresponding assigned weight value in the first analog fault simulation, <code>sim_1st_vec_fsim</code> .
<code>sim_2nd_vec_fsim/*</code>	Output directory containing all the intermediate files from the second analog fault simulation, <code>sim_2nd_vec_fsim</code> . The name of the directory is determined by the simulation tag name specified by the <code>-t</code> command-line option.
<code>sim_2nd_vec_fsim.config.json</code>	Configuration file to run the second analog fault simulation, <code>sim_2nd_vec_fsim</code> .
<code>sim_2nd_vec_fsim.fdb</code>	Fault simulation database of the second analog fault simulation, <code>sim_2nd_vec_fsim</code> .
<code>sim_2nd_vec_fsim.log</code>	Log file of the second analog fault simulation, <code>sim_2nd_vec_fsim</code> .
<code>sim_2nd_vec_fsim.report</code>	Coverage report of the second analog fault simulation, <code>sim_2nd_vec_fsim</code> .
<code>sim_2nd_vec_fsim.covered.list</code>	List of defects covered successfully in the second analog fault simulation, <code>sim_2nd_vec_fsim</code> .
<code>sim_2nd_vec_fsim.uncovered.list</code>	List of defects NOT covered by the input stimulus in the second analog fault simulation, <code>sim_2nd_vec_fsim</code> .
<code>sim_2nd_vec_fsim_weights.cpp</code>	Weight file associating each defect with its corresponding assigned weight value in the second analog fault simulation, <code>sim_2nd_vec_fsim</code> .

The following example shows a summary section of the coverage report (`sim_2nd_vec_fsim.report`). Note that the coverage percentage might be slightly different based on the version of the FineSim tool you use.

Tags	Sample-Size
<code>sim_1st_vec_fsim</code>	Exhaustive
<code>sim_2nd_vec_fsim</code>	Exhaustive
*DEFECT UNIVERSE SUMMARY	
<code>mos_open</code>	656
<code>mos_short</code>	864

```

TOTAL                                1520

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
      Type      sim_1st_vec_fsim      sim_2nd_vec_fsim      Total
(Union)
      mos_open      397/656
      6/259      403/656
      mos_short      699/864
      30/165      729/864
      TOTAL      1096/1520
      36/424      1132/1520

Weighted test coverage:
      sim_1st_vec_fsim      72.11%
+ sim_2nd_vec_fsim      74.47%

TOTAL Coverage:
      Non-Sampled      74.47%

Total weight of defect universe: 1520.00

```

The coverage report shows the following information:

- The simulation tag names are `sim_1st_vec_fsim` and `sim_2nd_vec_fsim`.
- There are 656 `mos_open` defects and 864 `mos_short` defects.
- 397 out of 656 `mos_open` defects and 699 out of 864 `mos_short` defects are covered in `sim_1st_vec_fsim`.
- 6 out of 259 `mos_open` defects and 30 out of 165 `mos_short` defects are covered in `sim_2nd_vec_fsim`.
- 1096 out of 1520 defects are covered in `sim_1st_vec_fsim` with the test coverage of 72.11%.
- 36 out of 424 defects are covered in `sim_2nd_vec_fsim` with the test coverage of 8.49%.
- The combined test coverage of `sim_1st_vec_fsim` and `sim_2nd_vec_fsim` is 74.47%.

The `*DETAILED COVERAGE LISTING` section shows the detail of each fault simulation. The log file of each individual fault simulation can be found in `RESULT/sim_1st_vec_fsim` and `RESULT/sim_2nd_vec_fsim`.

Running the Custom Fault Multi-Testbench Flow With PrimeSim

This section describes the files and commands needed to run the multi-testbench flow with the Custom Fault and PrimeSim tool in batch mode.

- [Creating the Configuration File to Run Custom Fault With PrimeSim](#)
- [Running the Custom Fault Tool With PrimeSim](#)
- [Generating Multi-Testbench Simulation Output Using PrimeSim](#)

Creating the Configuration File to Run Custom Fault With PrimeSim

The first analog fault simulation has the tag name `sim_1st_vec_psim`. It reads in the configuration file `configfile_1st_psim`. The output files are directed to the `RESULT` directory with the prefix `sim_1st_vec_psim.*`.

The second analog fault simulation has the simulation tag name `sim_2nd_vec_psim`. It reads in the configuration file `configfile_2nd_psim`. It also reads in the database file `RESULT/sim_1st_vec_psim.fdb` from `sim_1st_vec_psim` using the `-f` command-line option. The output files are directed to `RESULT` directory with the prefix `sim_2nd_vec_psim.*`. The coverage report contains the results of `sim_1st_vec_psim` and `sim_2nd_vec_psim`.

The content of the `configfile_1st_psim` file is:

```
{
  "cpulimit" : 3600,
  "case_dir" : "./",
  "analog_setup" : {
    "sim_cmdline": "primesim top_psim_1st_vec.spi"
  },
  "dpconfig_file" : "./xhost/xhost_sh_launch",
  "faults" : {
    "defaults" : {
      "short" : {
        "res" : 200,
        "cap" : null
      },
      "open" : {
        "res" : 10e6,
        "cap" : null
      }
    }
  }
}
```

The `"simulate_only" : { "covered" : false }` command is specified in `configfile_2nd_psim` to simulate only the defects not covered in `sim_1st_vec_psim`.

The content of the `configfile_2nd_psim` file is:

```
{
  "cpulimit" : 3600,
  "case_dir" : "./",
  "analog_setup" : {
    "sim_cmdline": "primesim top_psim_2nd_vec.spi"
  },
  "simulate_only" : {
    "covered" : false
  },
  "dpconfig_file" : "./xhost/xhost_sh_launch",
  "faults" : {
    "defaults" : {
      "short" : {
        "res" : 200,
        "cap" : null
      },
      "open" : {
        "res" : 10e6,
        "cap" : null
      }
    }
  }
}
```

Running the Custom Fault Tool With PrimeSim

The contents of the `run_cf_psim` run script file are:

```
customfault -c configfile_1st_psim -o RESULT/sim_1st_vec_psim
\ -t sim_1st_vec_psim -k 5
customfault -c configfile_2nd_psim -o RESULT/sim_2nd_vec_psim
\ -t sim_2nd_vec_psim -f RESULT/sim_1st_vec_psim.fdb -k 5
```

To run the multi-testbench tutorial, type:

```
$> run_cf_psim
```

Generating Multi-Testbench Simulation Output Using PrimeSim

After the simulation completes, the Custom Fault tool generates the following output files and directory inside the `RESULT` directory shown in [Table 28](#).

Table 28 Custom Fault Output Files for the PrimeSim Tool

Output File	Description
<code>sim_1st_vec_psim/*</code>	Output directory containing all the intermediate files from the first analog fault simulation, <code>sim_1st_vec_psim</code> . The name of the directory is determined by the simulation tag name specified by the <code>-t</code> command-line option.

Table 28 Custom Fault Output Files for the PrimeSim Tool (Continued)

Output File	Description
<code>sim_1st_vec_psim.config.json</code>	Configuration file to run the first analog fault simulation, <code>sim_1st_vec_psim</code> .
<code>sim_1st_vec_psim.fdb</code>	Fault simulation database of the first analog fault simulation, <code>sim_1st_vec_psim</code> .
<code>sim_1st_vec_psim.log</code>	Log file of the first analog fault simulation, <code>sim_1st_vec_psim</code> .
<code>sim_1st_vec_psim.report</code>	Coverage report of the first analog fault simulation, <code>sim_1st_vec_psim</code> .
<code>sim_1st_vec_psim.covered.list</code>	List of defects covered successfully in the first analog fault simulation, <code>sim_1st_vec_psim</code> .
<code>sim_1st_vec_psim.uncovered.list</code>	List of defects NOT covered by the input stimulus in the first analog fault simulation, <code>sim_1st_vec_psim</code> .
<code>sim_1st_vec_psim_weights.cpp</code>	Weight file associating each defect with its corresponding assigned weight value in the first analog fault simulation, <code>sim_1st_vec_psim</code> .
<code>sim_2nd_vec_psim/*</code>	Output directory containing all the intermediate files from the second analog fault simulation, <code>sim_2nd_vec_psim</code> . The name of the directory is determined by the simulation tag name specified by the <code>-t</code> command-line option.
<code>sim_2nd_vec_psim.config.json</code>	Configuration file to run the second analog fault simulation, <code>sim_2nd_vec_psim</code> .
<code>sim_2nd_vec_psim.fdb</code>	Fault simulation database of the second analog fault simulation, <code>sim_2nd_vec_psim</code> .
<code>sim_2nd_vec_psim.log</code>	Log file of the second analog fault simulation, <code>sim_2nd_vec_psim</code> .
<code>sim_2nd_vec_psim.report</code>	Coverage report of the second analog fault simulation, <code>sim_2nd_vec_psim</code> .
<code>sim_2nd_vec_psim.covered.list</code>	List of defects covered successfully in the second analog fault simulation, <code>sim_2nd_vec_psim</code> .
<code>sim_2nd_vec_psim.uncovered.list</code>	List of defects NOT covered by the input stimulus in the second analog fault simulation, <code>sim_2nd_vec_psim</code> .
<code>sim_2nd_vec_psim_weights.cpp</code>	Weight file associating each defect with its corresponding assigned weight value in the second analog fault simulation, <code>sim_2nd_vec_psim</code> .

The following example shows a summary section of the coverage report (sim_2nd_vec_psim.report). Note that the coverage percentage might be slightly different based on the version of the PrimeSim tool you use.

```

                Tags          Sample-Size

sim_1st_vec_psim    Exhaustive
sim_2nd_vec_psim    Exhaustive

*DEFECT UNIVERSE SUMMARY

    mos_open                656
    mos_short              864

        TOTAL                1520

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
    Type          sim_1st_vec_psim    sim_2nd_vec_psim    Total
(Union)

    mos_open                397/656
    6/259                403/656
    mos_short              699/864
    30/165              729/864
        TOTAL                1096/1520
    36/424              1132/1520

Weighted test coverage:
    sim_1st_vec_psim                72.11%
+ sim_2nd_vec_psim                74.47%

TOTAL Coverage:
    Non-Sampled                74.47%

Total weight of defect universe: 1520.00
```

The coverage report shows the following information:

- The simulation tag names are `sim_1st_vec_psim` and `sim_2nd_vec_psim`.
- There are 656 `mos_open` defects and 864 `mos_short` defects.
- 397 out of 656 `mos_open` defects and 699 out of 864 `mos_short` defects are covered in `sim_1st_vec_psim`.
- 6 out of 259 `mos_open` defects and 30 out of 165 `mos_short` defects are covered in `sim_2nd_vec_psim`.

- 1096 out of 1520 defects are covered in `sim_1st_vec_psim` with the test coverage of 72.11%.
- 36 out of 424 defects are covered in `sim_2nd_vec_psim` with the test coverage of 8.49%.
- The combined test coverage of `sim_1st_vec_psim` and `sim_2nd_vec_psim` is 74.47%.

The `*DETAILED COVERAGE LISTING` section shows the detail of each fault simulation. The log file of each individual fault simulation can be found in `RESULT/sim_1st_vec_psim` and `RESULT/sim_2nd_vec_psim`.

9

Custom Fault Mixed-Signal Flow

This chapter covers analog fault simulation for mixed-signal designs.

- [Overview of the Mixed-Signal Fault Simulation Flow](#)
- [Digital Waveform Comparison in the Mixed-Signal Flow](#)
- [Mixed-Signal Flow Tutorial](#)

Overview of the Mixed-Signal Fault Simulation Flow

You can use the Custom Fault tool to simulate a design partly modeled in analog and partly modeled in digital. A defect model is only injected in the design modeled in analog. Mixed-signal analog fault simulation supports all the flows: [Overview of the Automatic Injection Flow](#), [Overview of the Manual Defect Injection Flow](#), and [Overview of the Multi-Testbench Flow](#).

The `cosim_setup` command is applicable to mixed-signal analog fault simulation.

The following example demonstrates the usage of the `cosim_setup` command.

```
"cosim_setup" : {
  "cosim_info" : {
    "init" : "./vcsAD_xa.init",
    "compilation" : "vcs -nc -full64 -sverilog testbench.v adder.v
-debug_access+all -ad=vcsAD_xa.init -l vcs.log",
    "simv_cmdline" : "simv -nc +COMPARE -l simv.log",
    "fsdb_check" : {
      "signal_file" : "signal_list",
      "golden_file" : "golden_waveform_xa.fsdb"
    }
  }
}
```

The contents of the `vcsAD.init` file are:

```
bus_format %d;
use_spice -cell addr4;
choose xa -n addr4.spi -c xa.cfg -o results/xa/xa -ssl2 ;
```

Digital Waveform Comparison in the Mixed-Signal Flow

This section describes the following:

- [Signal File Format](#)
- [Signal File Example](#)
- [Conditional Comparison of Signals](#)
- [Comparison of Real Signals](#)
- [Time Window Based FSDB Comparison](#)

Signal File Format

The signal file contains the list of digital signals to be checked when running a mixed-signal analog fault simulation. It also provides control globally or locally to each specified signal.

The signal file contains the following sections:

- (Optional) The global control setting for all specified signals.
- The signal list and local control setting for each signal.

Note:

All comments begin with # or //.

The syntax of the signal file is:

```
['tdelay time[unit]]
['count num]
['x 0|1|x]
['twin window_name tstart1[unit] tstop1[unit] tstart2[unit] tstop2[unit]
...]
signal_name1 [,arg1 [, arg2 [, arg3 [, arg4 [, arg5 [, arg6]]]]]]
signal_name2 [,arg1 [, arg2 [, arg3 [, arg4 [, arg5 [, arg6]]]]]]
...
signal_nameN [,arg1 [, arg2 [, arg3 [, arg4 [, arg5 [, arg6]]]]]]
```

Argument	Description
<code>'tdelay time[unit]</code>	Specifies the threshold for time difference during accuracy comparison between the reference signal and the simulated signal. If the time difference is less than <i>time[unit]</i> , there is no violation. If the time difference is equal to or larger than <i>time[unit]</i> , then a violation is reported. The default for time and unit are 0 and s (second) respectively. The keywords for <i>unit</i> are: • s - second • ms - millisecond • us - microsecond • ns - nanosecond • ps - picosecond • fs - femtosecond
<code>'count num</code>	Specifies the maximum number of violations allowed per signal before the simulation is terminated. The default is 1. If it is specified as 0, the tool issues a warning on violation and continues the simulation.
<code>'x 0 1 x</code>	Treats x state as 0, 1 or x. The default is x.
<code>'twin window_name tstart1[unit] tstop1[unit] tstart2[unit] tstop2[unit]</code>	Defines a time window for waveform comparison (see Time Window Based FSDB Comparison).
<code>signal_nameN</code>	Specifies the name of the digital signals. This is mandatory. A wildcard character is not supported. Note: The complete hierarchy of the signal name must be used. That is, if it is a SPICE-top design, the default top-level SPICE instance name is <code>snps_sptop</code> , so the signal name must start with <code>snps_sptop</code> .

Argument	Description
<i>arg1</i>	Specifies the logic states of digital signals when the checking starts and has the Verilog-sized format as: <pre>size ['base number]</pre> where: <i>size</i> specifies the number of bits. <i>base</i> can be <i>b</i> or <i>B</i> for binary, <i>o</i> or <i>O</i> for octal, and <i>h</i> and <i>H</i> for hexadecimal. <i>number</i> is combinations of 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F characters. Depending on the <i>base</i> format, only a subset of these characters might be legal. You can also use unknown values (x) and high-impedance (z). An x or z sets four bits in the hexadecimal base, three bits in the octal base, and one bit in the binary base. If the most significant bit of a number is 0, x, or z, the tool automatically extends the number (if necessary) to fill the remaining bits with 0, x and z, respectively. If the most significant bit is 1, the tool uses 0 to extend it. This argument can also specify a conditional signal name with a condition such that if the condition is satisfied, FSDB comparison takes place (see Conditional Comparison of Signals).
<i>arg2</i>	Specifies the maximum number of violations allowed for the specified signal. It takes precedence over 'count.
<i>arg3</i>	Specifies the threshold to use in case of a Comparison of Real Signals .
<i>arg4</i>	Specifies on what basis to perform a Comparison of Real Signals: • <i>blank</i> - Instantaneous Difference (ID) • <i>rms</i> - Root Mean Square Difference (RMSD)
<i>arg5</i>	Specifies the time window name if you are using the time window based FSDB comparison feature (see Time Window Based FSDB Comparison).
<i>arg6</i>	Specifies the tags associated with the functional output and detection output signals as defined using the "id" : "tagN" argument under diagnostic_coverage (for Residual Faults and Direct SPFM Calculation)

If a duplicate setting applies to the same signal, the last entry overrides all the previous entries.

Signal File Example

In the following example, the Custom Fault tool does the following:

1. Starts to compare `xa1.xadd.a` when it reaches 1 and gives a warning for a violation.
2. Starts to compare `xa1.xadd.q[1:7]` when it reaches 1111111 and stops the simulation at the fourth violation.
3. Starts to compare `xa1.fusa` when it reaches (x) and stops the simulation at the eighth violation.
4. Compares `xa1.fusb` at the beginning of simulation and stops the simulation at the fourth violation.
5. Compares all four bits of the `snps_sptop.x_addr_dig.s` bus from the beginning of the simulation and stops the simulation at the second violation.
6. Compares bits 1 and 2 of the `snps_sptop.x_addr_dig.s` bus from the beginning of the simulation and stops the simulation at the second violation.
7. Compares only bit 1 of the `snps_sptop.x_addr_dig.s` bus from the beginning of the simulation and stops the simulation at the second violation.

```
'count 4
xa1.xadd.a,1,0
xa1.xadd.q[1:7],7'b1
xa1.fusa,x,8
xa1.fusb
snps_sptop.x_addr_dig.s[3:0],,2
snps_sptop.x_addr_dig.s[2:1],,2
snps_sptop.x_addr_dig.s[1],,2
```

Conditional Comparison of Signals

The Custom Fault mixed-signal flow supports conditional comparison of signals, such as when certain signals reach a particular value the comparison for some other signals begins. The `sig.txt` file has been enhanced to handle conditional comparison of signals. The start condition has been enhanced to handle conditional comparisons.

For example, if the `sig.txt` entry is:

```
sigA,cntrl==1'b1
```

The tool compares the `sigA` signal when the `cntrl` value is equal to `1'b1` and automatically stops/starts comparison if the `cntrl` value changes.

Comparison of Real Signals

The Custom Fault mixed-signal flow supports comparison of real signals. The comparison of real signals is based on two different measures:

- Instantaneous Difference (ID)

If a real signal is specified for observation such as `sigA`, then the value of this signal during simulation is compared against that in the golden FSDB at the same time stamp or within an infinitesimally small window around it if they do not match exactly. You can specify this value with `'tdelay <time>`. The absolute value of the difference is measured and is known as the instantaneous difference, or ID.

If the ID is more than a user-specified threshold, then the defect injected is considered observed. ID is the default mechanism for comparison of real signals. The user-specified threshold is provided as the third argument. For example, if the `sig.txt` has the following entry:

```
sigA,,,0.5mV
```

The user-specified threshold is `0.5mV`. If the instantaneous difference between the golden FSDB and the simulated FSDB is more than `0.5mV`, a violation is reported

- Root Mean Square Difference (RMSD)

The RMSD is calculated by summing up the squared value of instantaneous differences and then normalizing the value by dividing it with the number of IDs considered during that period, and taking the square root.

The duration of accumulation is the period of mismatch that is the time interval in which IDs are non-zero. If RMSD is more than a user-specified threshold, then the defect injected is considered observed.

The user-specified threshold is provided as the third argument, and the mechanism is enforced through `rms` in the optional fourth argument. For example, if you specify:

```
sigA,,,0.5mV,rms
```

The user-specified threshold is `0.5mV`. If the RMSD between the golden FSDB and the simulated FSDB is more than `0.5mV`, a violation is reported.

Time Window Based FSDB Comparison

The time window based FSDB comparison feature allows you to detect defects within a specific time frame using the following steps:

1. [Defining a Time Window](#)
2. [Specifying the Time Window in a Signal](#)

Defining a Time Window

The time window must be defined with a new name ('twin) as follows:

```
['twin window_name tstart1[unit] tstop1[unit] tstart2[unit] tstop2[unit]  
...]
```

Argument	Description
'twin	Tells the tool that a time window is defined.
window_name	Specifies the identifier name for defining the time window.
tstart* and tstop*	Define the start and stop time windows for comparison.
unit	Refers to the time unit in one of the following: • s - second • ms - millisecond • us - microsecond • ns - nanosecond • ps - picosecond • fs - femtosecond

Specifying the Time Window in a Signal

The time window can be used as an optional argument **arg5** in the signal file as:

```
signal_name1 [,arg1 [, arg2 [, arg3 [, arg4 [, arg5 ]]]]]
```

Time Window Examples

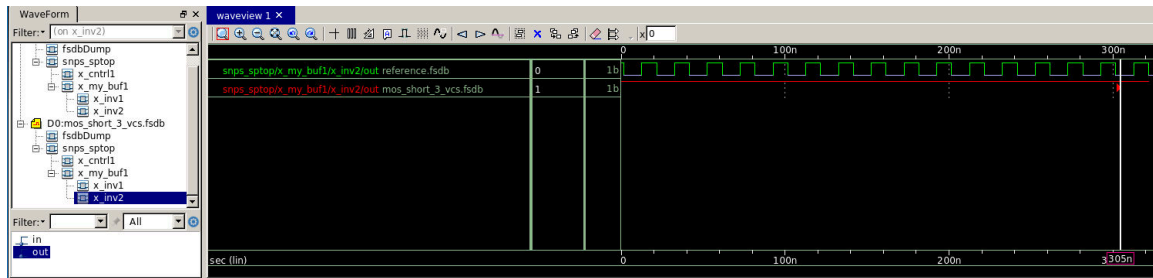
You can define the time window from a starting time until the end of simulation. In the following example, the comparison of the signal starts from 305ns and continues until the end of simulation because *tstop1* has not been specified.

```
'twin window1 305ns  
snps_sptop.x_my_buf1.x_inv2.out,,,,,window1
```

You can define the start and stop time of the window. In the following example, the comparison of the signal happens in the time window from 305ns to 310ns.

```
'twin window2 305ns 310ns  
snps_sptop.x_my_buf1.x_inv2.out,,,,,window2
```

The following figure shows that the signal *snps_sptop.x_my_buf1.x_inv2.out* in *mos_short_3_vcs.fldb* does not match the corresponding signal from *reference.fldb*. Therefore, a violation gets reported at 305ns.



You can define multiple start and stop time windows. In the following example, the comparison of the signal happens in the time windows from 315ns to 320ns and from 330ns to 350ns.

```
'twin win3 315ns 320ns 330ns 350ns
snps_sptop.x_my_buf1.x_inv2.out,,,,,win3
```

You can use the same time window for detecting defects for multiple signals. In the following example, the comparison starts from 10ns and continues until the end of simulation for both signals.

```
'twin tw2 10ns
snps_sptop.x_my_buf1.out1,,,,,tw2
snps_sptop.x_my_buf1.out2,,,,,tw2
```

You can use a time window along with other comparison conditions for defect detection. In the following example, the comparison of the signal starts when its value is 1'b1 in the time window from 10ns to 250ns.

```
'twin tw3 10ns 250ns
snps_sptop.x_my_buf1.out1,1'b1,,,,tw3
```

You can use a time window along with conditional comparison conditions for defect detection. In the following example, the comparison of the signal starts when the cntnl signal's value is 1'b1 in the time window from 10ns to 260ns.

```
'twin tw4 10ns 260ns
snps_sptop.x_my_buf1.out1,cntnl==1'b1,,,,tw4
```

Mixed-Signal Flow Tutorial

For a mixed-signal fault simulation, although defects can only be injected in the design modeled in analog, for defect observations, it can be in both analog as well as in digital.

- If the defect observation point is in analog, then vector check and/or `.MEASURE` are used.
- If the defect observation point is in digital, then FSDB waveform comparison is used as the defect detection criteria.

The mixed-signal fault simulation flow supports both Verilog top and SPICE top configurations. There are four tutorials that are described in this section and they all use FSDB waveform comparison for defect detection.

The tutorials are in subdirectories of the `install_directory/doc/tutorials/mixed-signal_flow` directory, as follows:

Subdirectory	Design Type and Configuration	Runs with:
<code>cf_mixed-signal_verilogTop</code>	Mixed-signal design with Verilog top configuration See Running Mixed-Signal Fault Simulation for Verilog Top Configuration .	Custom Fault and VCS PrimeSim AMS
<code>cf_mixed-signal_spiceTop</code>	Mixed-signal design with SPICE top configuration See Running Mixed-Signal Fault Simulation for SPICE Top Configuration .	Custom Fault and VCS PrimeSim AMS

Both `cf_mixed-signal_verilogTop` and `cf_mixed-signal_spiceTop` contain individual run scripts to run the Custom Fault tool with VCS PrimeSim AMS (using any of its analog simulator engines: PrimeSim XA, FineSim, or PrimeSim).

Before starting the individual tutorials, make sure that all the necessary tools are installed correctly (this includes Verdi® as it is needed to output the digital FSDB file).

Note:

Before attempting to run the tutorial with the Custom Fault tool, make sure that your mixed-signal simulation can run standalone. This can be done by invoking:

```
% runVcsAms (for running VCS PrimeSim AMS using the PrimeSim XA
simulator engine)

% runVcsFsim (for running VCS PrimeSim AMS using the FineSim simulator
engine)

% runVcsPsim (for running VCS PrimeSim AMS using the PrimeSim simulator
engine)
```

Running Mixed-Signal Fault Simulation for Verilog Top Configuration

In the following tutorial (`/mixed-signal_flow/cf_mixed-signal_verilogTop`), you run analog fault simulations in the mixed-signal design for Verilog top configuration and use waveform comparison for accuracy checking.

If the output signal of the simulation does not match with the digital reference waveform file provided, then a violation is detected.

This tutorial uses a Verilog testbench at the top level that instantiates a 4-bit adder circuit in SPICE with three inputs, `a`, `b`, and `cin` (carry-in), and two outputs `s` (sum of `a`, `b`, and `cin`) and `cout` (carry-out).

Table 29 *Input Files for Running Custom Fault With VCS PrimeSim AMS for Verilog Top Configuration*

File	Description
<code>run_cf_vcsams_fsdbcheck</code> <code>run_cf_vcsfsim_fsdbcheck</code> <code>run_cf_vcspsim_fsdbcheck</code>	Run scripts to run mixed-signal fault simulation for waveform comparison checking.
<code>fsdbcheck_xa.json</code> <code>fsdbcheck_fs.json</code> <code>fsdbcheck_ps.json</code>	Custom Fault configuration files for waveform comparison checking.
<code>vcsAD_xa.init</code> <code>vcsAD_fs.init</code> <code>vcsAD_ps.init</code>	Mixed-signal control files for waveform comparison checking.
<code>golden_waveform_xa.fsdb</code> <code>golden_waveform_fs.fsdb</code> <code>golden_waveform_ps.fsdb</code>	FSDB waveform files containing the digital signals used as a reference for accuracy checking.
<code>addr4.spi</code>	SPICE netlist for the PrimeSim XA and FineSim simulator engines.
<code>addr4_psim.spi</code>	SPICE netlist for the PrimeSim simulator engine.
<code>adder.v</code>	Verilog file for the 4-bit adder.
<code>testbench.v</code>	Verilog testbench.
<code>xhost/xhost_sh_launch</code>	Host file for distributed processing.
<code>signal_list</code>	Signal file for waveform comparison checking.
<code>xa.cfg</code>	PrimeSim XA command file.

Table 29 *Input Files for Running Custom Fault With VCS PrimeSim AMS for Verilog Top Configuration (Continued)*

File	Description
CLEAN_XA	Cleans all simulation generated files.
CLEAN_FS	
CLEAN_PS	

Note:

The file names that contain `vcxsams` or `xa` are required for running the Custom Fault tool with VCS PrimeSim AMS using the PrimeSim XA simulator engine.

The file names that contain `vcfsim` or `fs` are required for running the Custom Fault tool with VCS PrimeSim AMS using the FineSim simulator engine.

The file names that contain `vcpsim` or `ps` are required for running the Custom Fault tool with VCS PrimeSim AMS using the PrimeSim simulator engine.

The remaining input files are common across both simulators.

The following sections describe how to run the mixed-signal fault simulation flow with the Custom Fault tool and the PrimeSim XA, FineSim, or PrimeSim tools in batch mode.

- [Creating the Configuration File to Run Mixed-Signal Fault Simulation With FSDB Comparison Check](#)
- [Running Mixed-Signal Fault Simulation](#)
- [Generating Mixed-Signal Fault Simulation Output](#)

Creating the Configuration File to Run Mixed-Signal Fault Simulation With FSDB Comparison Check

The accuracy checking criteria is set up to compare the simulation waveform signals to the reference signals provided in FSDB waveform format.

This simulation has the simulation tag name of `sim_fsdbcheck_xa`, `sim_fsdbcheck_fs`, or `sim_fsdbcheck_ps`. It reads in the `fsdbcheck_xa.json`, `fsdbcheck_fs.json`, or `fsdbcheck_ps.json` configuration file. The output files are directed to the `CUSTOMFAULT_RESULT_XA`, `CUSTOMFAULT_RESULT_FS`, or `CUSTOMFAULT_RESULT_PS` directory with the prefix of `cf_fsdbcheck_xa.*`, `cf_fsdbcheck_fs.*`, or `cf_fsdbcheck_ps.*`, respectively.

The content of `fsdbcheck_xa.json` is:

```
{
  "cpulimit" : 3600,
```

```

    "cosim_setup" : {
      "cosim_info" : {
        "init" : "./vcsAD_xa.init",
        "compilation" : "vcs -nc -full64 -sverilog testbench.v adder.v
-debug_access+all -ad=vcsAD_xa.init -l vcs.log",
        "simv_cmdline" : "simv -nc +COMPARE -l simv.log",
        "fsdb_check" : {
          "signal_file" : "signal_list",
          "golden_file" : "golden_waveform_xa.fsdb"
        }
      }
    },
    "case_dir" : ".",
    "dpconfig_file" : "xhost/xhost_sh_launch",
    "faults" : {
      "defaults" : {
        "short" : {
          "res" : 0.01,
          "cap" : null
        },
        "open" : {
          "res" : 1e6,
          "cap" : null
        }
      }
    }
  }
}

```

The content of `fsdbcheck_fs.json` is similar to `fsdbcheck_xa.json` except it differs with respect to the following fields:

```

"init" : "./vcsAD_fs.init"
"compilation" : "vcs -nc -full64 -sverilog testbench.v adder.v
-debug_access+all -ad=vcsAD_fs.init -l vcs.log",
"golden_file" : "golden_waveform_fs.fsdb"

```

The content of `fsdbcheck_ps.json` is similar to `fsdbcheck_xa.json` except it differs with respect to the following fields:

```

"init" : "./vcsAD_ps.init"
"compilation" : "vcs -nc -full64 -sverilog testbench.v adder.v
-debug_access+all -ad=vcsAD_ps.init -l vcs.log",
"golden_file" : "golden_waveform_ps.fsdb"

```

The `"cosim_setup"` command generates the defect universe for running mixed-signal simulations.

The `"cosim_info" : { ... }` argument specifies the necessary information needed to run the mixed-signal fault simulation flow in the Custom Fault tool, including the FSDB comparison for accuracy check.

The "fsdb_check" : { ... } argument specifies the reference FSDB waveform file ("golden_file" : " golden_waveform_xa.fsdb") and the signal file ("signal_file" : "signal_list") for FSDB comparison. The signal file contains the list of signals to be checked and provide control to each specified signal (see [Digital Waveform Comparison in the Mixed-Signal Flow](#)).

The content of the signal file (signal_list) is:

```
`tdelay 0.2n
top.cout,,1
top.s [3:0],,1
```

The signal_list file specifies cout node and s bus to be checked for accuracy comparison. The `tdelay command specifies 2ns as the threshold for time difference during accuracy comparison between the reference signal and the simulated signal. If the simulation output matches the expected output in the reference file (golden_waveform_*.fsdb), then there is no violation. Otherwise, a violation is reported in the *.log file. If a violation is reported, the defect is covered.

This is an example of the *.log file of the individual fault simulation when a violation is reported:

```
Error: [MSV-CF-002] CustomFault: {"code": "CW", "type": "FSDB Violation",
  "signal": "top.s[3:0]", "value": "1001", "violation_cond": "1001 should
  be 0001", "time": "0"}
```

The individual *.err file can be found in the CUSTOMFAULT_RESULT_*/sim_fsdbcheck_*/ directory.

Running Mixed-Signal Fault Simulation

The run script for the simulation (Custom Fault and VCS PrimeSim AMS using the PrimeSim XA engine) is run_cf_vcsams_fsdbcheck and the content is:

```
customfault -c fsdbcheck_xa.json -o CUSTOMFAULT_RESULT_XA/cf_fsdbcheck_xa
-t sim_fsdbcheck_xa -k 20
```

To run the script, specify:

```
% run_cf_vcsams_fsdbcheck
```

The run script for the simulation (Custom Fault and VCS PrimeSim AMS using the FineSim engine) is run_cf_vcsfsim_fsdbcheck and the content is:

```
customfault -c fsdbcheck_fs.json -o CUSTOMFAULT_RESULT_FS/cf_fsdbcheck_fs
-t sim_fsdbcheck_fs -k 20
```

To run the script, specify:

```
% run_cf_vcsfsim_fsdbcheck
```

The run script for the simulation (Custom Fault and VCS PrimeSim AMS using the PrimeSim engine) is `run_cf_vcspsim_fsdbcheck` and the content is:

```
customfault -c fsdbcheck_ps.json -o CUSTOMFAULT_RESULT_PS/cf_fsdbcheck_ps
-t sim_fsdbcheck_ps -k 20
```

To run the script, specify:

```
% run_cf_vcspsim_fsdbcheck
```

Generating Mixed-Signal Fault Simulation Output

The output files are directed to the `CUSTOMFAULT_RESULT_*/` directory (where `*` denotes `xa`, `fs`, or `ps`) as shown in [Table 30](#).

Table 30 Mixed-Signal Flow Output Files

File	Description
<code>sim_fsdbcheck_*/</code>	Output directory that contains all the intermediate files from the simulation. The name of the directory is determined by the simulation tag name specified by the <code>-t</code> command-line option.
<code>cf_fsdbcheck_*.fdb</code>	Fault simulation database of analog fault simulation, <code>sim_fsdbcheck_*</code> .
<code>cf_fsdbcheck_*.report</code>	Coverage report of analog fault simulation, <code>sim_fsdbcheck_*</code> .
<code>cf_fsdbcheck_*.log</code>	Log file of analog fault simulation, <code>sim_fsdbcheck_*</code> .

[Figure 40](#) shows a summary section of the coverage report (`cf_fsdbcheck_*.report`). Note that the coverage percentage may be slightly different based on the version of the PrimeSim XA, FineSim, or PrimeSim tool you use.

Figure 40 Summary of Mixed-Signal Coverage Report

```
*DEFECT UNIVERSE SUMMARY

      mos_open           656
      mos_short          864

      TOTAL              1520

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
      Type              sim_fsdbcheck_xa

      mos_open           205/656
      mos_short          747/864
      TOTAL              952/1520

Weighted test coverage:
      sim_fsdbcheck_xa    62.63%

TOTAL Coverage:
      Non-Sampled         62.63%

Total weight of fault universe: 1520.00

*DETAILED COVERAGE LISTING
```

The coverage report shows the following information:

- The simulation tag name is `sim_fsdbcheck_xa`.
- There are 656 `mos_open` defects and 864 `mos_short` defects.
- 205 out of 656 `mos_open` defects and 747 out of 864 `mos_short` defects are covered.
- The coverage percentage is 62.63%.

Running Mixed-Signal Fault Simulation for SPICE Top Configuration

In this tutorial (`/mixed-signal_flow/cf_mixed-signal_spiceTop`), you run analog fault simulations in the mixed-signal design for SPICE top configuration and use waveform comparison for accuracy checking.

If the output signal of the simulation does not match with the digital reference waveform file provided, then a violation is detected.

This tutorial uses a top-level SPICE subckt that instantiates a 4-bit adder circuit with three inputs, `a`, `b`, and `cin` (carry-in), and two outputs `s` (sum of `a`, `b`, and `cin`) and `cout` (carry-out).

Table 31 *Input Files for Running Custom Fault With VCS PrimeSim AMS for SPICE Top Configuration*

File	Description
run_cf_vcsams_fsdbcheck run_cf_vcsfsim_fsdbcheck run_cf_vcspsim_fsdbcheck	Run scripts to run mixed-signal fault simulation for waveform comparison checking.
run_compile_xa run_compile_fs run_compile_ps	Compile scripts that hold <code>vlogan</code> and <code>vcs</code> commands and are used by <code>run_cf_vcsams_fsdbcheck</code> , <code>run_cf_vcsfsim_fsdbcheck</code> , or <code>run_cf_vcspsim_fsdbcheck</code> .
fsdbcheck_xa.json fsdbcheck_fs.json fsdbcheck_ps.json	Custom Fault configuration files for waveform comparison checking.
vcsAD_xa.init vcsAD_fs.init vcsAD_ps.init	Mixed-signal control files for waveform comparison checking.
golden_waveform_xa.fsdb golden_waveform_fs.fsdb golden_waveform_ps.fsdb	FSDB waveform files containing the digital signals used as a reference for accuracy checking.
top.spi	SPICE netlist for the PrimeSim XA and FineSim simulator engines.
top_psim.spi	SPICE netlist for the PrimeSim simulator engine.
adder4.v	Verilog file for the 4-bit adder.
adder.vec	Input vector file.
xhost/xhost_sh_launch	Host file for distributed processing.
signal_list	Signal file for waveform comparison checking.
xa.cfg	PrimeSim XA command file.
CLEAN_XA CLEAN_FS CLEAN_PS	Cleans all simulation generated files.

Note:

The file names that contain `vcsams` or `xa` are required for running the Custom Fault tool with VCS PrimeSim AMS using the PrimeSim XA simulator engine.

The file names that contain `vcsfssim` or `fs` are required for running the Custom Fault tool with VCS PrimeSim AMS using the FineSim simulator engine.

The file names that contain `vcspssim` or `ps` are required for running the Custom Fault tool with VCS PrimeSim AMS using the PrimeSim simulator engine.

The remaining input files are common across both simulators.

The Custom Fault configuration file `fsdbcheck_*.json` used for SPICE top is very similar to the one used for Verilog top configuration (see [Creating the Configuration File to Run Mixed-Signal Fault Simulation With FSDB Comparison Check](#)) except that it shows how you can also use the cosimulation 3-step flow, that is, `vlogan` and `vcs` used for compilation and `simv` used for simulation. So the `vlogan` and `vcs` options can be embedded in a compile script such as `run_compile_xa`, `run_compile_fs`, or `run_compile_ps` and can then be passed to the Custom Fault tool as:

```
"cosim_setup" : {
  "cosim_info" : {
    "init" : "./vcsAD_xa.init",
    "compilation" : "vcs -nc -full64 -sverilog testbench.v adder.v
-debug_access+all -ad=vcsAD_xa.init -l vcs.log",
    "simv_cmdline" : "simv -nc +COMPARE -l simv.log",
    "fsdb_check" : {
      "signal_file" : "signal_list",
      "golden_file" : "golden_waveform_xa.fsdb"
    }
  }
}
```

In contrast to the Verilog top configuration tutorial, the mixed-signal control file `vcsAD_*.init` in this case has an extra command as: `spice_top;` which informs the tool that the design is a SPICE top configuration.

The content of the signal file (`signal_list`) is:

```
snps_sptop.x_dut.x_sp.s[3:0],,1
snps_sptop.x_dut.x_sp.cout,,1
```

The `signal_list` file specifies `snps_sptop.x_dut.x_sp.cout` node and `snps_sptop.x_dut.x_sp.s` bus to be checked for accuracy comparison. If the simulation output matches the expected output in the reference file (`golden_waveform_*.fsdb`), then there is no violation. Otherwise, a violation is reported in the `*.log` file. If a violation is reported, the defect is covered.

This is an example of the `*.log` file of the individual fault simulation when a violation is reported.

```
Error: [MSV-CF-002] CustomFault: {"code": "CW", "type": "FSDB
Violation", "signal": "snps_sptop.x_dut.x_sp.s[3:0]", "value": "1000",
"violation_cond": "1000 should be 0000", "time": "3e-09"}
```

The individual *.err file can be found in the CUSTOMFAULT_RESULT_*/sim_fsdbcheck_*/ directory.

To run the mixed-signal fault simulation for SPICE top configuration, the same run scripts used for the Verilog top case (see [Running Mixed-Signal Fault Simulation](#)) can be also be used.

To run the script for Custom Fault and VCS PrimeSim AMS using the PrimeSim XA simulator engine, specify:

```
% run_cf_vcsams_fsdbcheck
```

To run the script for Custom Fault and VCS PrimeSim AMS using the FineSim simulator engine, specify:

```
% run_cf_vcsfsim_fsdbcheck
```

To run the script for Custom Fault and VCS PrimeSim AMS using the PrimeSim simulator engine, specify:

```
% run_cf_vcspsim_fsdbcheck
```

The output files for the SPICE top configuration are generated in the same way as for the Verilog top configuration (see [Generating Mixed-Signal Fault Simulation Output](#)) and are directed to the CUSTOMFAULT_RESULT_*/ directory (where * denotes xa, fs, or ps).

10

Custom Fault Post-Layout Flow

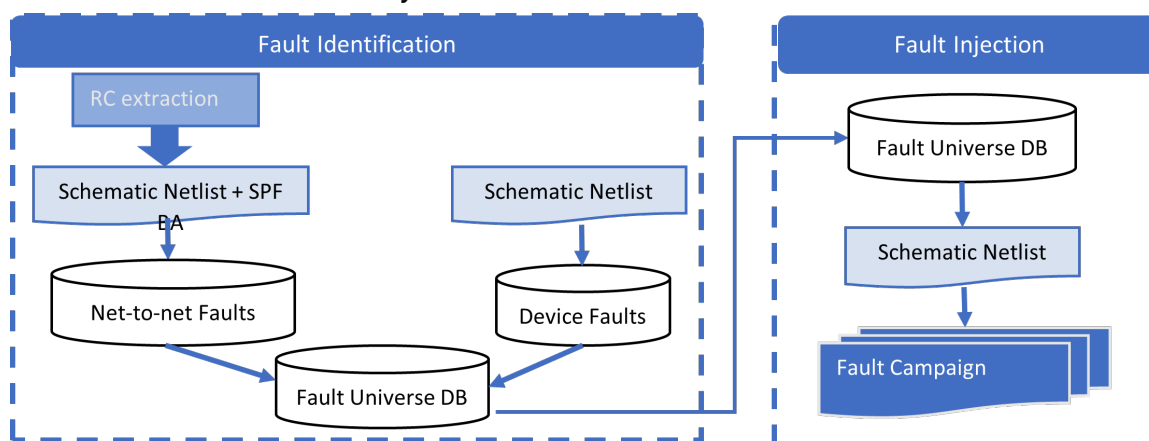
This chapter covers the post-layout flow for analog fault simulation.

- [Overview of the Post-Layout Flow](#)
- [Post-Layout Flow Tutorial](#)
- [Post-Layout Mixed-Signal Flow Tutorial](#)

Overview of the Post-Layout Flow

In the post-layout flow, `net_to_net_short` defects are detected using the SPF file information. The Custom Fault tool injects `net_to_net_short` defects in any nets that have a coupling capacitance above the `cap_net_to_net` threshold value (as defined using the `faults` command under `id_thresholds`). Such nets are detected using the SPF file information, and the defects are injected in the pre-layout (or schematic) netlist as shown in [Figure 41](#).

Figure 41 Custom Fault Post-Layout Flow



To invoke the post-layout flow for analog-only simulations, use the `postl_id_cmdline` argument of the `analog_setup` command and specify the simulator command line that loads the SPF file.

For example:

```
"analog_setup" : {  
    "postl_id_cmdline" : "xa top_ba.sp",  
    "sim_cmdline" : "xa top.sp"  
},
```

The SPF file must be specified in the netlist with the PrimeSim XA command `load_ba_file`. In this example, the `top_ba.sp` file contains:

```
.option xa_cmd="load_ba_file add4.spf"
```

Note:

To invoke the post-layout flow for mixed-signal simulations, use the `postl_id` argument of the `cosim_setup` command.

Post-Layout Flow Tutorial

This tutorial shows how to use the Custom Fault tool to find the `net_to_net_short` defects in the post-layout SPF file and inject the defects in the pre-layout netlist.

The tutorial described in this section demonstrates the following:

- [Running the Custom Fault Post-Layout Flow With PrimeSim XA](#)
- [Running the Custom Fault Post-Layout Flow With FineSim](#)
- [Running the Custom Fault Post-Layout Flow With PrimeSim](#)

Before starting the tutorial, make sure that all tools are installed correctly.

[Table 32](#) lists the files needed for this tutorial. These files are in the `install_directory/doc/tutorials/postlayout` directory.

Table 32 *Post-Layout Flow Tutorial Files*

File	Description
<code>add4_vddhd2.sp</code>	SPICE description of the adder.
<code>add4_vddhd2.spf</code>	SPF file for the adder.
<code>top.sp</code>	SPICE schematic (pre-layout) netlist of the circuit.
<code>top_ba.sp</code>	SPICE netlist of the circuit including the SPF file added with <code>load_ba_file</code> command.
<code>xhost/xhost_sh_launch</code>	Host file for distributed processing.
<code>config_xa.json</code>	Custom Fault configuration file for the PrimeSim XA simulator.

Table 32 Post-Layout Flow Tutorial Files (Continued)

File	Description
<code>config_fs.json</code>	Custom Fault configuration file for the FineSim simulator.
<code>config_ps.json</code>	Custom Fault configuration file for the PrimeSim simulator.
<code>run_cf_xa</code>	Run script to run the tutorial with the PrimeSim XA simulator.
<code>run_cf_fs</code>	Run script to run the tutorial with the FineSim simulator.
<code>run_cf_ps</code>	Run script to run the tutorial with the PrimeSim simulator.
<code>README</code>	README file with some information on the tutorial.

Note:

You can edit the `xhost/xhost_sh_launch` file and modify it to use your own Distributing Processing.

Running the Custom Fault Post-Layout Flow With PrimeSim XA

To run the post-layout flow and inject `net_to_net_short` defects on the nets coupled with large capacitors, you must specify the following in the JSON configuration file:

- PrimeSim XA command line using the schematic netlist in the `sim_cmdline` argument of the `analog_setup` command
- PrimeSim XA command line using the back-annotated netlist in the `postl_id_cmdline` argument of the `analog_setup` command.

In this tutorial, the configuration file to use the PrimeSim XA tool as a simulator shows:

```
"analog_setup" : {
  "postl_id_cmdline" : "xa top_ba.sp",
  "sim_cmdline" : "xa top.sp"
},
```

The `top_ba.sp` file specifies the name of the SPF file for `net_to_net_short` defect generation as follows:

```
.option xa_cmd="load_ba_file -file ./add4_vddhd2.spf"
```

You can limit the `net_to_net_short` defects to nets that are coupled with large capacitances using the `cap_net_to_net` argument of the `faults` command under `id_thresholds`. The configuration file used in the tutorial shows:

```
"id_thresholds" : {  
  "cap_net_to_net": 1e-29  
}
```

The `run_cf_xa` script contains the command to run the Custom Fault tool:

```
mv RESULTS RES_tbd  
\rm -rf RES_tbd &  
customfault -c config_xa.json -t ba_xa -o RESULTS/ba_xa -k 8
```

After the simulation completes, the `RESULTS/ba_xa.report` file shows the coverage report including the `net_to_net_short` defects:

```
*DEFECT UNIVERSE SUMMARY  
  
              mos_open              836  
              mos_short             1039  
          net_to_net_short             231  
  
              TOTAL                 2106  
  
*COVERAGE SUMMARY  
  
Defects covered - weighted (covered/simulated):  
              Type                  ba_xa  
  
              mos_open              99/836  
              mos_short             363/1039  
          net_to_net_short             83/231  
              TOTAL                 545/2106  
  
Weighted test coverage:  
              ba_xa                 25.88%  
  
TOTAL Coverage:  
              Non-Sampled           25.88%
```


Running the Custom Fault Post-Layout Flow With FineSim

To run the post-layout flow and inject `net_to_net_short` defects on the nets coupled with large capacitors, you must specify the following in the JSON configuration file:

- FineSim command line using the schematic netlist in the `sim_cmdline` argument of the `analog_setup` command
- FineSim command line using the back-annotated netlist in the `postl_id_cmdline` argument of the `analog_setup` command.

In this tutorial, the configuration file to use the FineSim tool as a simulator shows:

```
"analog_setup" : {  
  "postl_id_cmdline" : "finesim top_ba.sp",  
  "sim_cmdline" : "finesim top.sp"  
},
```

The `top_ba.sp` file specifies the name of the SPF file for `net_to_net_short` defect generation as follows:

```
.option xa_cmd="load_ba_file -file ./add4_vddhd2.spf"
```

You can limit the `net_to_net_short` defects to nets that are coupled with large capacitances using the `cap_net_to_net` argument of the `faults` command under `id_thresholds`. The configuration file used in the tutorial shows:

```
"id_thresholds" : {  
  "cap_net_to_net": 1e-29  
}
```

The `run_cf_fsim` script contains the command to run the Custom Fault tool:

```
mv RESULTS RES_tbd  
\rm -rf RES_tbd &  
customfault -c config_fs.json -t ba_fsim -o RESULTS/ba_fsim -k 8
```

After the simulation completes, the `RESULTS/ba_fsim.report` file shows the coverage report including the `net_to_net_short` defects:

```
*DEFECT UNIVERSE SUMMARY  
  
                mos_open                836  
                mos_short               1039  
                net_to_net_short         231  
  
                TOTAL                   2106  
  
*COVERAGE SUMMARY  
  
Defects covered - weighted (covered/simulated):
```

Type	ba_fsim
mos_open	71/836
mos_short	352/1039
net_to_net_short	80/231
TOTAL	503/2106
Weighted test coverage:	
ba_fsim	23.88%
TOTAL Coverage:	
Non-Sampled	23.88%

Running the Custom Fault Post-Layout Flow With PrimeSim

To run the post-layout flow and inject `net_to_net_short` defects on the nets coupled with large capacitors, you must specify the following in the JSON configuration file:

- PrimeSim command line using the schematic netlist in the `sim_cmdline` argument of the `analog_setup` command
- PrimeSim command line using the back-annotated netlist in the `postl_id_cmdline` argument of the `analog_setup` command.

In this tutorial, the configuration file to use the PrimeSim tool as a simulator shows:

```
"analog_setup" : {
  "postl_id_cmdline" : "primesim top_ba.sp",
  "sim_cmdline" : "primesim top.sp"
},
```

The `top_ba.sp` file specifies the name of the SPF file for `net_to_net_short` defect generation as follows:

```
.option xa_cmd="load_ba_file -file ./add4_vddhd2.spf"
```

You can limit the `net_to_net_short` defects to nets that are coupled with large capacitances using the `cap_net_to_net` argument of the `faults` command under `id_thresholds`. The configuration file used in the tutorial shows:

```
"id_thresholds" : {
  "cap_net_to_net": 1e-29
}
```

The `run_cf_psim` script contains the command to run the Custom Fault tool:

```
mv RESULTS RES_tbd
\rm -rf RES_tbd &
customfault -c config_ps.json -t ba_psim -o RESULTS/ba_psim -k 8
```

After the simulation completes, the `RESULTS/ba_psim.report` file shows the coverage report including the `net_to_net_short` defects:

```
*DEFECT UNIVERSE SUMMARY

              mos_open              836
              mos_short             1039
          net_to_net_short             231
              TOTAL                 2106

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
              Type                  ba_psim
              mos_open              71/836
              mos_short             352/1039
          net_to_net_short             80/231
              TOTAL                 503/2106

Weighted test coverage:
              ba_psim                23.88%

TOTAL Coverage:
              Non-Sampled            23.88%
```

Post-Layout Mixed-Signal Flow Tutorial

This tutorial shows how to use the Custom Fault tool to find the `net_to_net_short` defects in the post-layout SPF file and inject the defects in the pre-layout netlist for the analog portion of the mixed-signal design.

The tutorial described in this section demonstrates the following:

- [Running the Custom Fault Post-Layout Mixed-Signal Flow Tutorial With PrimeSim XA](#)
- [Running the Custom Fault Post-Layout Mixed-Signal Flow Tutorial With FineSim](#)
- [Running the Custom Fault Post-Layout Mixed-Signal Flow Tutorial With PrimeSim](#)

Before starting the tutorial, make sure that all tools are installed correctly.

[Table 33](#) lists the files needed for this tutorial. These files are in the `install_directory/doc/tutorials/mixed-signal_flow/cf_mixed-signal_post_layout` directory.

Table 33 *Post-Layout Mixed-Signal Flow Tutorial Files*

File	Description
add4_vddhd2.sp	SPICE description of the adder.
add4_vddhd2.spf	SPF file for the adder.
adder.v	Verilog file for the 4-bit adder.
top.sp	SPICE schematic (pre-layout) netlist of the circuit.
top_ba.sp	SPICE netlist of the circuit including the SPF file added with <code>load_ba_file</code> command.
xhost/xhost_sh_launch	Host file for distributed processing.
config_xa.json config_fs.json config_ps.json	Custom Fault configuration files for post-layout mode defect detection.
run_cf_vcsxa_ba run_cf_vcsfsim_ba run_cf_vcspsim_ba	Run scripts to run post-layout mixed-signal fault simulation using FSDB waveform comparison.
run_compile_xa run_compile_fs run_compile_ps	Compile script that holds <code>vlogan</code> and <code>vcs</code> commands, and uses a non-SPF netlist.
run_compile_ba_xa run_compile_ba_fs run_compile_ba_ps	Compile script that holds <code>vlogan</code> and <code>vcs</code> commands, and uses an SPF netlist.
vcsAD_xa.init vcsAD_fs.init vcsAD_ps.init	Mixed-signal control files that use a pre-layout netlist.
vcsAD_ba_xa.init vcsAD_ba_fs.init vcsAD_ba_ps.init	Mixed-signal control files that use a post-layout netlist.
golden_waveform_xa.fsdb golden_waveform_fs.fsdb golden_waveform_ps.fsdb	FSDB waveform files containing the digital signals used as a reference for accuracy checking.
signal_list	Signal file used for waveform comparison checking.
xa.cfg	PrimeSim XA command file.
README	README file with detailed information on the tutorial.

Table 33 Post-Layout Mixed-Signal Flow Tutorial Files (Continued)

File	Description
CLEAN_XA	Cleans all simulation generated files.
CLEAN_FS	
CLEAN_PS	

Note:

You can edit the `xhost/xhost_sh_launch` file and modify it to use your own Distributing Processing.

Running the Custom Fault Post-Layout Mixed-Signal Flow Tutorial With PrimeSim XA

To run the post-layout flow and inject `net_to_net_short` defects on the nets coupled with large capacitors, you must specify the following in the JSON configuration file:

- The `cosim_info` argument under the `cosim_setup` command makes use of the compile script `run_compile_xa` which compiles mixed-signal design using the SPICE netlist *without* any SPF.
- The `postl_id` argument under the `cosim_setup` command makes use of the compile script `run_compile_ba_xa` which compiles mixed-signal design using the SPICE netlist *with* SPF.

This tutorial makes use of FSDB waveform comparison to detect the `net_to_net_short` defects.

Example

```
"cosim_setup" : {
  "cosim_info" : {
    "init" : "vcsAD_xa.init",
    "compilation" : "run_compile_xa",
    "simv_cmdline" : "simv -nc -l simv.log",
    "fsdb_check" : {
      "signal_file" : "signal_list",
      "golden_file" : "golden_waveform_xa.fsdb"
    }
  },
  "postl_id" : {
    "init": "vcsAD_ba_xa.init",
    "compilation": "run_compile_ba_xa",
    "simv_cmdline": "simv_ba -nc -l simv_ba.log"
  }
}
```

An additional `-Mdir=temporary_compiled_directory` argument is used in the `run_compile_ba_xa` run script to separate out the default compiled directory results (such as, `csrc`) generated by the other `run_compile_xa` run script. For example, `-Mdir=csrc_ba` is used in the `vlogan` and `vcs` commands in the `run_compile_ba_xa` run script.

The `top_ba.spf` file specifies the name of the SPF file for `net_to_net_short` defect generation as follows:

```
.option xa_cmd="load_ba_file -file ./add4_vddhd2.spf"
```

You can limit the `net_to_net_short` defects to nets that are coupled with large capacitances using the `cap_net_to_net` argument of the `faults` command under `id_thresholds`.

The configuration file used in the tutorial shows the following:

```
"id_thresholds" : {
  "cap_net_to_net": 1e-29
}
```

The `run_cf_vcsxa_ba` script contains the command to run the Custom Fault tool:

```
customfault -c config_xa.json -o CUSTOMFAULT_RESULT_XA/ba_xa -t ba_xa -k 20
```

After the simulation completes, the `CUSTOMFAULT_RESULT_XA/ba_xa.report` file shows the coverage report including the `net_to_net_short` defects:

```
*DEFECT UNIVERSE SUMMARY

net_to_net_short                253
TOTAL                           253

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
Type                                ba_xa

net_to_net_short                250/253
TOTAL                           250/253

Weighted test coverage:
ba_xa                            98.81%

TOTAL Coverage:
Exhaustive                       98.81%

Total weight of defect universe: 253.00

Simulation Summary:
```

Tag	Completed	Failed
ba_xa	253	0

Running the Custom Fault Post-Layout Mixed-Signal Flow Tutorial With FineSim

To run the post-layout flow and inject `net_to_net_short` defects on the nets coupled with large capacitors, you must specify the following in the JSON configuration file:

- The `cosim_info` argument under the `cosim_setup` command makes use of the compile script `run_compile_fs` which compiles mixed-signal design using the SPICE netlist *without* any SPF.
- The `postl_id` argument under the `cosim_setup` command makes use of the compile script `run_compile_ba_fs` which compiles mixed-signal design using the SPICE netlist *with* SPF.

This tutorial makes use of FSDB waveform comparison to detect the `net_to_net_short` defects.

Example

```
"cosim_setup" : {
  "cosim_info" : {
    "init" : "vcsAD_fs.init",
    "compilation" : "run_compile_fs",
    "simv_cmdline" : "simv -nc -l simv.log",
    "fsdb_check" : {
      "signal_file" : "signal_list",
      "golden_file" : "golden_waveform_fs.fsdb"
    }
  },
  "postl_id" : {
    "init": "vcsAD_ba_fs.init",
    "compilation": "run_compile_ba_fs",
    "simv_cmdline": "simv_ba -nc -l simv_ba.log"
  }
}
```

An additional `-Mdir=temporary_compiled_directory` argument is used in the `run_compile_ba_fs` run script to separate out the default compiled directory results (such as, `csrc`) generated by the other `run_compile_fs` run script. For example, `-Mdir=csrc_ba` is used in the `vlogan` and `vcs` commands in the `run_compile_ba_fs` run script.

The `top_ba.sp` file specifies the name of the SPF file for `net_to_net_short` defect generation as follows:

```
.option xa_cmd="load_ba_file -file ./add4_vddhd2.spf"
```

You can limit the `net_to_net_short` defects to nets that are coupled with large capacitances using the `cap_net_to_net` argument of the `faults` command under `id_thresholds`.

The configuration file used in the tutorial shows the following:

```
"id_thresholds" : {  
    "cap_net_to_net": 1e-29  
}
```

The `run_cf_vcsfsim_ba` script contains the command to run the Custom Fault tool:

```
customfault -c config_fs.json -o CUSTOMFAULT_RESULT_FS/ba_fs -t ba_fs -k 20
```

After the simulation completes, the `CUSTOMFAULT_RESULT_FS/ba_fs.report` file shows the coverage report including the `net_to_net_short` defects:

```
*DEFECT UNIVERSE SUMMARY  
  
net_to_net_short                253  
  
TOTAL                           253  
  
*COVERAGE SUMMARY  
  
Defects covered - weighted (covered/simulated):  
Type                           ba_fs  
  
net_to_net_short                220/253  
TOTAL                           220/253  
  
Weighted test coverage:  
ba_fs                           86.96%  
  
TOTAL Coverage:  
Exhaustive                      86.96%  
  
Total weight of defect universe: 253.00  
  
Simulation Summary:  
Tag                             Completed  Failed  
ba_fs                           253       0
```


Running the Custom Fault Post-Layout Mixed-Signal Flow Tutorial With PrimeSim

To run the post-layout flow and inject `net_to_net_short` defects on the nets coupled with large capacitors, you must specify the following in the JSON configuration file:

- The `cosim_info` argument under the `cosim_setup` command makes use of the compile script `run_compile_ps` which compiles mixed-signal design using the SPICE netlist *without* any SPF.
- The `postl_id` argument under the `cosim_setup` command makes use of the compile script `run_compile_ba_ps` which compiles mixed-signal design using the SPICE netlist *with* SPF.

This tutorial makes use of FSDB waveform comparison to detect the `net_to_net_short` defects.

Example

```
"cosim_setup" : {
  "cosim_info" : {
    "init" : "vcsAD_ps.init",
    "compilation" : "run_compile_ps",
    "simv_cmdline" : "simv -nc -l simv.log",
    "fsdb_check" : {
      "signal_file" : "signal_list",
      "golden_file" : "golden_waveform_ps.fsdb"
    }
  },
  "postl_id" : {
    "init": "vcsAD_ba_ps.init",
    "compilation": "run_compile_ba_ps",
    "simv_cmdline": "simv_ba -nc -l simv_ba.log"
  }
}
```

An additional `-Mdir=temporary_compiled_directory` argument is used in the `run_compile_ba_ps` run script to separate out the default compiled directory results (such as, `csrc`) generated by the other `run_compile_ps` run script. For example, `-Mdir=csrc_ba` is used in the `vlogan` and `vcs` commands in the `run_compile_ba_ps` run script.

The `top_ba.sp` file specifies the name of the SPF file for `net_to_net_short` defect generation as follows:

```
.option xa_cmd="load_ba_file -file ./add4_vddhd2.spf"
```

You can limit the `net_to_net_short` defects to nets that are coupled with large capacitances using the `cap_net_to_net` argument of the `faults` command under `id_thresholds`.

The configuration file used in the tutorial shows the following:

```
"id_thresholds" : {  
  "cap_net_to_net": 1e-29  
}
```

The `run_cf_vcspsim_ba` script contains the command to run the Custom Fault tool:

```
customfault -c config_ps.json -o CUSTOMFAULT_RESULT_PS/ba_ps -t ba_ps -k 20
```

After the simulation completes, the `CUSTOMFAULT_RESULT_PS/ba_ps.report` file shows the coverage report including the `net_to_net_short` defects:

```
*DEFECT UNIVERSE SUMMARY  
  
net_to_net_short                253  
  
TOTAL                          253  
  
*COVERAGE SUMMARY  
  
Defects covered - weighted (covered/simulated):  
Type                           ba_ps  
  
net_to_net_short                220/253  
TOTAL                          220/253  
  
Weighted test coverage:  
ba_ps                          86.96%  
  
TOTAL Coverage:  
Exhaustive                      86.96%  
  
Total weight of defect universe: 253.00  
  
Simulation Summary:  
Tag                             Completed Failed  
ba_ps                          253      0
```

11

Custom Fault Parametric Defect Injection

This chapter describes how to use the Custom Fault tool to perform Parametric Defect Injection.

- [Parametric Defect Injection Flow](#)
- [Preparing the Netlist for PDI](#)
- [Generating a Template File for PDI](#)
- [Defining Parameter Variations for PDI](#)

Parametric Defect Injection Flow

The Custom Fault tool supports a Parametric Defect Injection (PDI) flow for both analog and mixed-signal simulation.

PDI allows you to specify parameter variation in terms of sigma for any or all random parameters in the circuit netlist.

The overall flow steps are as follows:

1. Set up the netlist model libraries to include variation effects by including the appropriate libraries (see [Preparing the Netlist for PDI](#)).
2. (Optional) Create a template parameter file (see [Generating a Template File for PDI](#)).
3. Define parameter variations using the `parameter_variation` command in the Custom Fault configuration file (see [Defining Parameter Variations for PDI](#)).
4. Run the fault campaign.

Preparing the Netlist for PDI

Perform the following netlist preparation steps to enable Parametric Defect Injection (PDI):

1. Configure the SPICE model libraries.

You must select the library sections that enable parametric variations or Monte Carlo analysis. Refer to your PDK documentation for details on how to configure your model libraries to enable parametric variation through Monte Carlo analysis.

2. Add `.tran sweep monte=N` to your SPICE netlist.

During [Generating a Template File for PDI](#), the N value is used, and the template file contains N sample indexes. During defect injection simulations, the N value is ignored. Each index in the parameter file results in one simulation.

Generating a Template File for PDI

The parameter definitions are provided in a standard PrimeSim HSPICE-compatible `.mc0` file format. The `.mc0` file defines the values of all random variables in the circuit netlist. There may be hundreds or thousands of variables in a circuit, so generating a template as a basis for further modification is a convenient method for defining the parametric defects for simulation.

As described in [Preparing the Netlist for PDI](#), set the number of Monte Carlo sweeps to the number of sample indexes needed for the template file and add the following `parameter_variation` template generation command to the Custom Fault configuration file:

```
"parameter_variation" : {  
    "template" : true  
}
```

Run the Custom Fault tool as follows:

```
$> customfault -c template.json -t nom -o cf -k 1
```

This produces the template `.mc0` file, `cf.mc0`, in the run directory.

You can copy or edit the template file to set the defect parameter values. The random variable values in the `.mc0` file are in units of sigma. If you want a random variable to use its nominal value, assign 0 to it. Specify a three-sigma change by assigning 3 to the random variable, and so on.

Note:

In mixed-signal simulation, the template `.mc0` file contains only one sample index regardless of the value of N in the `.tran sweep monte=N` command in the netlist.

Defining Parameter Variations for PDI

To execute the parametric defect injection, add the `parameter_variation` command as follows to the Custom Fault configuration file and run the Custom Fault tool:

```
"parameter_variation" : {  
    "inject" : true,  
    "parameter_file" : "path/to/mc0/or/JSON/parameter/file"  
}
```

The tool runs one simulation for each index defined in the `.mc0` file. The coverage report is provided in the `.parametric_report` file in the output directory.

Note:

The `.mc0` file with index 1 is reserved for the nominal sample and is not run with PDI. Nominal (defect-free) simulation is specified through the `-n yes` command-line option.

The parametric defects are also listed in a parametric defect list file, `.pdl.json`. The parametric defect list file is a JSON format file defining the variation parameters. It can be used in place of the `.mc0` file to define the defects.

Probing Signals in PDI

In regular defect injection, you can control signal probing by adding appropriate `.PROBE` statements to the SPICE netlist. This general principle applies to PDI, as well.

The PDI flow makes use of the Monte Carlo external sampling flow so you must be aware of the simulator rules for probing with Monte Carlo.

For the PrimeSim XA tool, waveform probing under Monte Carlo simulation is disabled unless the following command is used:

```
set_monte_carlo_option -dump_waveforms 1
```

The FineSim and PrimeSim tools do not prevent waveform probing under Monte Carlo simulation, so no additional commands are required.

12

Using Save-and-Restore With Custom Fault

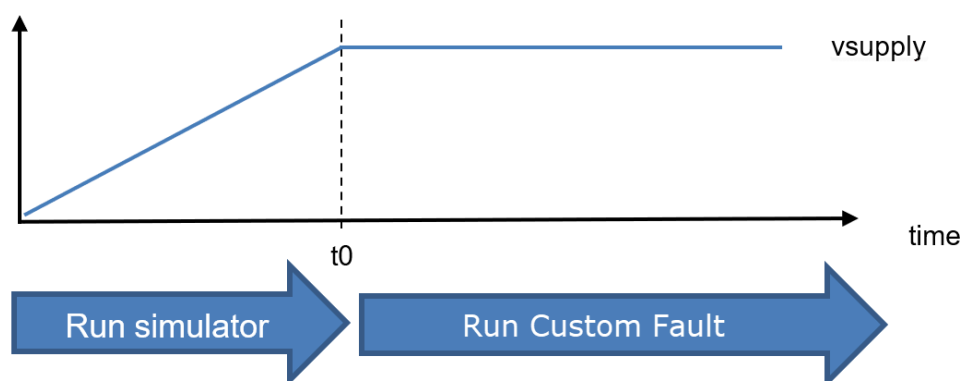
This chapter describes how to use the PrimeSim XA, FineSim, or PrimeSim Save-and-Restore feature with the Custom Fault tool to inject defects at a time different from 0.

- [When to Use Save-and-Restore With Custom Fault](#)
- [Save-and-Restore Using the PrimeSim XA Simulator](#)
- [Save-and-Restore Using the FineSim Simulator](#)
- [Save-and-Restore Using the PrimeSim Simulator](#)
- [Save-and-Restore Known Limitations](#)
- [Save-and-Restore With the Custom Fault Mixed-Signal Flow](#)

When to Use Save-and-Restore With Custom Fault

As the supply of a circuit is ramped up, it does not make sense to inject defects in the circuit until the supply is stable. For example, if the supply is ramping up from time 0 to time t_0 , you do not want to inject defects during that period of time. Defects are injected after time t_0 .

Figure 42 Defects are Injected When the Supply is Stable

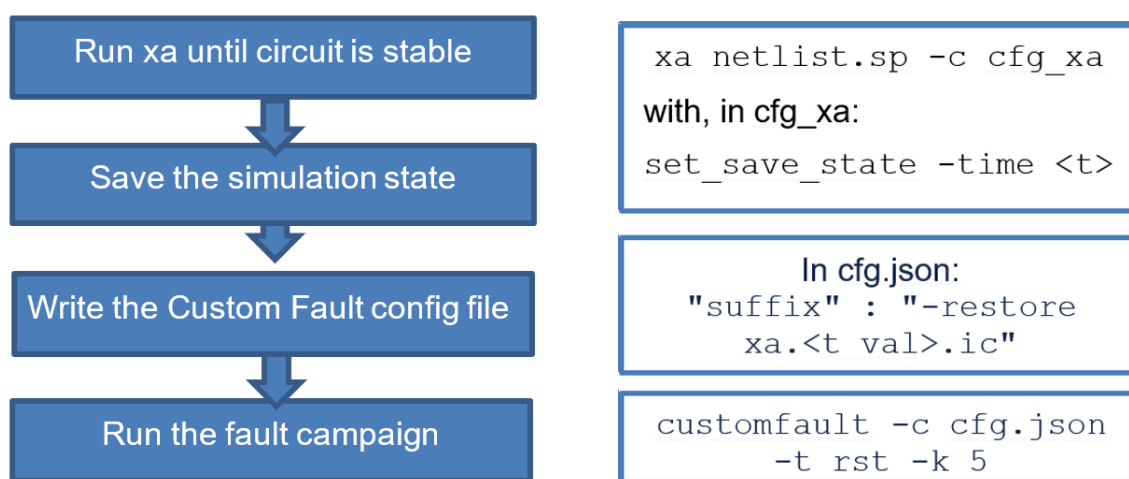


Save-and-Restore Using the PrimeSim XA Simulator

The flow for the PrimeSim XA tool (see [Figure 43](#)) consists of the following steps:

1. Run the PrimeSim XA simulator until the circuit is stable.
2. Save the simulation state using the PrimeSim XA `set_save_state` command.
3. Run the fault campaign after using the PrimeSim XA `-restore` option to inject defects at the beginning of the restore phase.

Figure 43 Flow Description Using the PrimeSim XA Options



The Save-and-Restore flow for the PrimeSim XA tool is detailed in the following sections:

- [Saving the PrimeSim XA Simulation when the Circuit is Stable](#)
- [Running the Fault Campaign With PrimeSim XA Save Results](#)
- [Example — Save-and-Restore With PrimeSim XA](#)

Saving the PrimeSim XA Simulation when the Circuit is Stable

You must run the transient simulation with the PrimeSim XA tool at least until the circuit is stable. The time at which the circuit is stable is called t_0 . In the PrimeSim XA commands, add the following command to save the simulation at time t_0 :

```
set_save_state -time t0
```

Then run the PrimeSim XA tool, for example:

```
xa netlist.sp -c xacfg -o Results/rampup
```

After the simulation is saved, you can start the fault campaign at time t_0 .

Running the Fault Campaign With PrimeSim XA Save Results

To be able to run the fault campaign with the PrimeSim XA save results, add the following command to the Custom Fault `config.json` file:

```
"compilation_cmdline": "xa command line without -restore option",
```

Note:

This command is usually used when running the Custom Fault tool with the FineSim or PrimeSim tool, however, it is also required if you want to run the Custom Fault tool with the PrimeSim XA tool and the `-restore` option:

In addition, specify the `-restore` option in the `"sim_cmdline"` argument as follows:

```
"analog_setup": {  
  "sim_cmdline": "xa netlist -c cmd -restore <Name of the saved .ic  
file>"  
}
```

With these two options, the PrimeSim XA simulations are run with the `-restore` option, and the defects are injected after the circuit is stable.

Example — Save-and-Restore With PrimeSim XA

The supply ramps up from 0ns to 10ns in the netlist: `top.spi`

```
vsu vdd gnd pw1 0 0 10n vsupply
```

1. Specify for the PrimeSim XA tool to save the simulation at 11ns (after the supply is stable) by adding the following command in `xacfg`:

```
set_save_state -time 11n
```

2. Run the PrimeSim XA tool:

```
xa top.spi -c xacfg -o Nom/rampup
```

The tool saves the simulation at 11ns and creates the `.ic` and `.ic.sup` files under the `Nom/` directory:

```
Unix> ls Nom/  
rampup.1.1e-08.ic  
rampup.1.1e-08.ic.sup
```

3. Set the required commands in the `config.json` file for defect analysis:

```
"analog_setup": {  
  "sim_cmdline": "xa netlist -restore Nom/rampup.1.1e-08.ic",
```



```
    "compilation_cmdline": "xa netlist"
}
```

4. Run the Custom Fault fault campaign as usual.

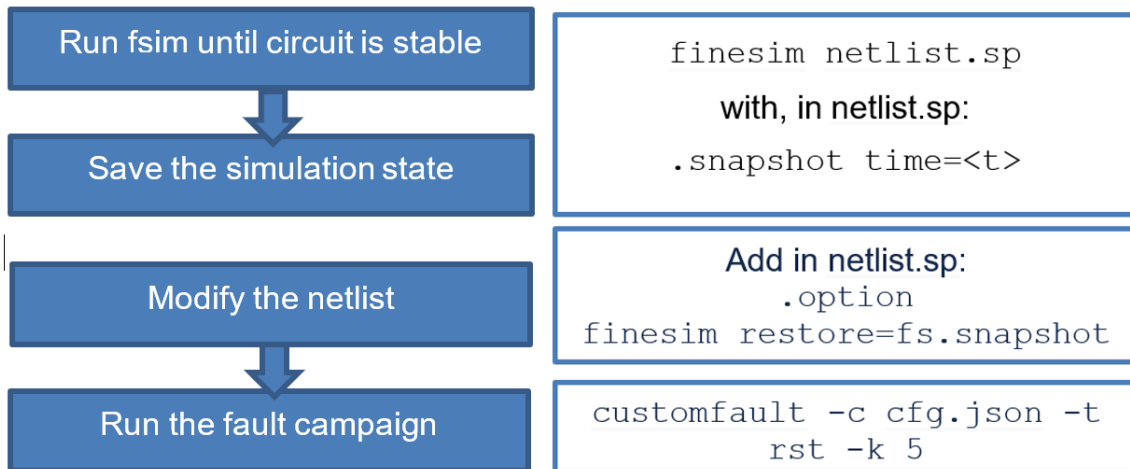
The defects are injected and the simulations are run using the `-restore` command.

Save-and-Restore Using the FineSim Simulator

The flow for the FineSim tool (see [Figure 44](#)) consists of the following steps:

1. Run the FineSim simulator until the circuit is stable.
2. Save the simulation state using the FineSim `.snapshot` command.
3. Run the fault campaign after using the FineSim `finesim_restore` option to inject defects at the beginning of the restore phase.

Figure 44 Flow Description Using the FineSim Options



The Save-and-Restore flow for the FineSim tool is detailed in the following sections:

- [Saving the FineSim Simulation when the Circuit is Stable](#)
- [Running the Fault Campaign With FineSim Save Results](#)
- [Example — Save-and-Restore With FineSim](#)

Saving the FineSim Simulation when the Circuit is Stable

You must run the transient simulation with the FineSim tool at least until the circuit is stable. The time at which the circuit is stable is called t_0 . In the netlist, add the following command to save the simulation at time t_0 :

```
.snapshot time= $t_0$ 
```

Then run the FineSim tool, for example:

```
finesim netlist.sp -o Results/rampup
```

After the simulation is saved, you can start the fault campaign at time t_0 .

Running the Fault Campaign With FineSim Save Results

Before starting the fault campaign with the FineSim save results, add the following FineSim option to the netlist:

```
.option finesim_restore=<fs_out>_ $t_0$ .snapshot
```

Then run the fault campaign as usual. The FineSim simulations each start at the time of the snapshot and the defects are injected at that same time.

Example — Save-and-Restore With FineSim

The supply ramps up from 0ns to 10ns in the netlist: `top.spi`

```
vsu vdd gnd pw1 0 0 10n vsupply
```

1. Specify for the FineSim tool to save the simulation at 11ns (after the supply is stable) by adding the following command to the netlist:

```
.snapshot time=11ns
```

2. Run the FineSim tool:

```
finesim top.spi -o Nom/rampup
```

The tool saves the simulation at 11ns and creates the `.snapshot` file under the `Nom/` directory:

```
Unix> ls Nom/  
rampup_11ns.snapshot
```

3. Add the following option to the netlist:

```
.option finesim_restore = Nom/rampup_11ns.snapshot
```

4. Run the fault campaign as usual.

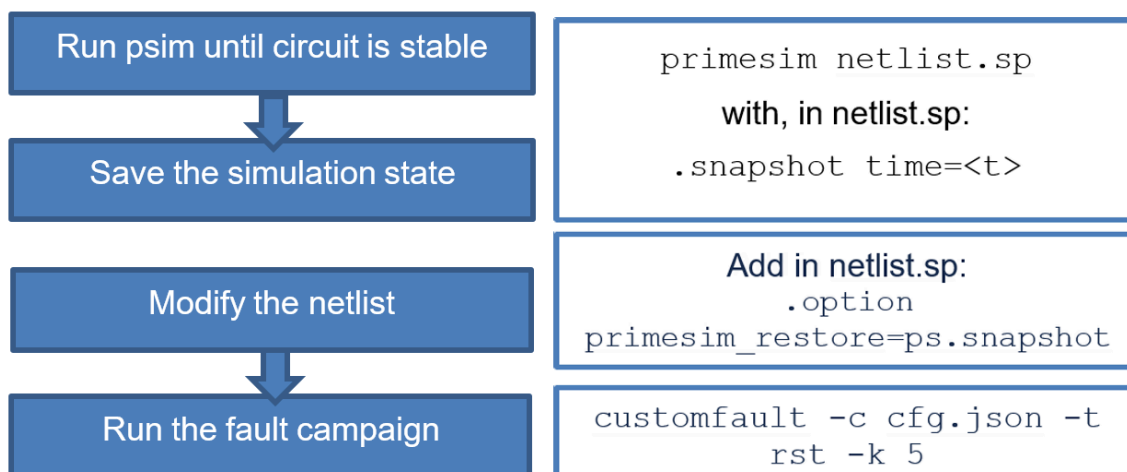
The defects are injected and the simulations are run using the `finessim_restore` option.

Save-and-Restore Using the PrimeSim Simulator

The flow for the PrimeSim tool (see [Figure 45](#)) consists of the following steps:

1. Run the PrimeSim simulator until the circuit is stable.
2. Save the simulation state using the PrimeSim `.snapshot` command.
3. Run the fault campaign after using the PrimeSim `primesim_restore` option to inject defects at the beginning of the restore phase.

Figure 45 Flow Description Using the PrimeSim Options



The Save-and-Restore flow for the PrimeSim tool is detailed in the following sections:

- [Saving the PrimeSim Simulation when the Circuit is Stable](#)
- [Running the Fault Campaign With PrimeSim Save Results](#)
- [Example — Save-and-Restore With PrimeSim](#)

Saving the PrimeSim Simulation when the Circuit is Stable

You must run the transient simulation with the PrimeSim tool at least until the circuit is stable. The time at which the circuit is stable is called t_0 . In the netlist, add the following command to save the simulation at time t_0 :

```
.snapshot time= $t_0$ 
```

Then run the PrimeSim tool, for example:

```
primesim netlist.sp -o Results/rampup
```

After the simulation is saved, you can start the fault campaign at time t_0 .

Running the Fault Campaign With PrimeSim Save Results

Before starting the fault campaign with the PrimeSim save results, add the following PrimeSim option to the netlist:

```
.option primesim_restore=<ps_out>_ $t_0$ .snapshot
```

Then run the fault campaign as usual. The PrimeSim simulations each start at the time of the snapshot and the defects are injected at that same time.

Example — Save-and-Restore With PrimeSim

The supply ramps up from 0ns to 10ns in the netlist: `top.spi`

```
vsu vdd gnd pw1 0 0 10n vsupply
```

1. Specify for the PrimeSim tool to save the simulation at 11ns (after the supply is stable) by adding the following command to the netlist:

```
.snapshot time=11ns
```

2. Run the PrimeSim tool:

```
primesim top.spi -o Nom/rampup
```

The tool saves the simulation at 11ns and creates the `.snapshot` file under the `Nom/` directory:

```
Unix> ls Nom/  
rampup_11ns.snapshot
```

3. Add the following option to the netlist:

```
.option primesim_restore = Nom/rampup_11ns.snapshot
```

4. Run the fault campaign as usual.

The defects are injected and the simulations are run using the `primesim_restore` option.

Save-and-Restore Known Limitations

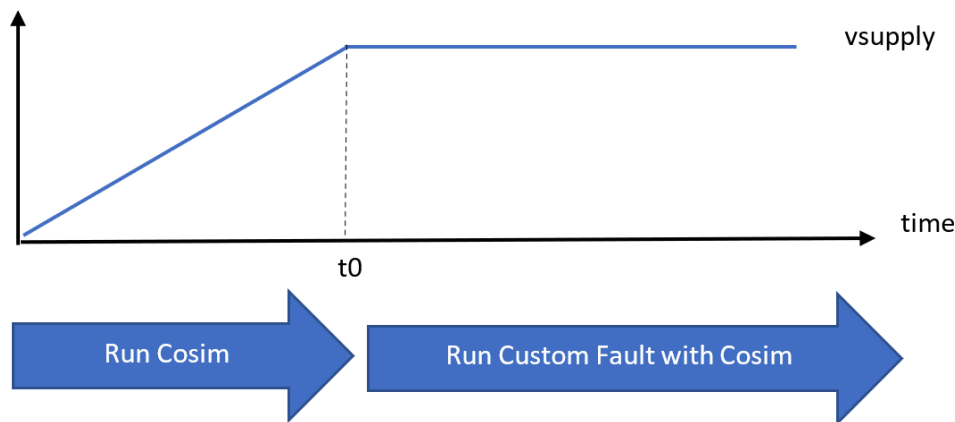
The following are known limitations of the Save-and-Restore feature:

- When restoring, DC convergence may occur.
- When restoring with the FineSim or PrimeSim tool, restoring may not work if GMI/Verilog-A or frequency dependent elements are present.

Save-and-Restore With the Custom Fault Mixed-Signal Flow

You can use both VCS image-based save-and-restore along with `ace reread` features for Custom Fault mixed-signal simulation with support for the VCS PrimeSim AMS tool (using any of its analog simulator engines: PrimeSim XA, FineSim, or PrimeSim).

Figure 46 Defects are Injected When the Supply is Stable or Test Conditions are Met



Similar to the analog simulation save-and-restore flow, you can choose to run only mixed-signal simulation (also called cosimulation) when the supply is getting ramped up. Then when the supply is stable or test conditions are met, you can run the Custom Fault tool with cosimulation to inject defects in the circuit. For example, if the supply is ramping up from time 0 to time t_0 , you do not want to inject defects during that period. Defects are injected after time t_0 .

For more details on how to set up the Save-and-Restore flow in mixed-signal, see Saving and Restoring Simulation State in the *VCS® PrimeSim™ AMS User Guide*.

The flow consists of the following steps:

1. [Running and Saving Standalone Mixed-Signal Simulation Until the Circuit is Stable or Test Conditions Are Met](#)
2. [Restoring the Saved Simulation and Running the Fault Campaign](#)

Running and Saving Standalone Mixed-Signal Simulation Until the Circuit is Stable or Test Conditions Are Met

Run the standalone mixed-signal transient simulation until the circuit is stable or test conditions are met at time t_0 , then save an image of the simulation using the VCS UCLI `save` command provided in the `ucli` control file:

```
run t0
save save_snapshot
```

This saves an image of the simulation up to time t_0 into the `save_snapshot` files. After the simulation is saved, start the fault campaign at time t_0 .

Restoring the Saved Simulation and Running the Fault Campaign

To run the fault campaign with the saved results, first restore the simulation using the VCS UCLI `restore` command provided in the `ucli` control file:

```
restore save_snapshot
```

There is no need to recompile the design, so the existing `simv` from the saved simulation is used to continue with the restored simulation. As a result, there is no need to specify the `compilation` options in the Custom Fault `config.json` file. After the simulation is restored, the tool automatically injects defects as specified the `config.json` file.

13

Custom Fault Output Files

This chapter covers the Custom Fault output files.

- [Test Coverage Report \(Single Testbench\)](#)
- [Test Coverage Report \(Multi-Testbench\)](#)
- [Diagnostic Coverage Report](#)
- [Not-covered Defect Analysis](#)

Test Coverage Report (Single Testbench)

The Custom Fault tool generates a file with the extension, `.report`. The file can be divided into the following sections:

- [Header](#)
- [Defect Universe Summary](#)
- [Coverage Summary](#)
- [Detailed Coverage Listing](#)

Each report section is explained using a single-testbench report. See [Test Coverage Report \(Multi-Testbench\)](#) for differences in a multi-testbench report.

Header

The header section of the test coverage report lists the product name and version, the simulation tag name, and sampling information, as shown in the following example:

```

PrimeSim (TM) Custom Fault

Version T-2022.06-1 for linux64 - Jul 03, 2022 15:49:09 7687798

Tags          Sample-Size
lab3xa        Exhaustive
  
```

When using AWRS (see [Using Adaptive Weighted Random Sampling \(AWRS\)](#)), the Custom Fault tool reports the sample size as follows:

Tags	Sample-Size
xa	26

When using WSS (see [Using Weight Sorted Sampling \(WSS\)](#)), the Custom Fault tool reports the specified maximum error percentage as follows:

Tags	Sample-Size
wss	WSS Max Error = 20%

Defect Universe Summary

The Defect Universe Summary section lists the number of identified defect model types in the design, as shown in the following example:

```
*DEFECT UNIVERSE SUMMARY
```

mos_open	656
mos_short	864
TOTAL	1520

Coverage Summary

As shown in the following example, the Coverage Summary section lists the number of detected (or covered) defects for each defect model type and the coverage percentage of each defect simulation by tag name. It also provides simulation summary by tag name.

```
*COVERAGE SUMMARY
```

Defects covered - weighted (covered/simulated):	
Type	lab3xa
mos_open	394/656
mos_short	696/864
TOTAL	1090/1520
TOTAL_OPEN	394/656
TOTAL_SHORT	696/864
Weighted test coverage:	
lab3xa	71.71%
lab3xa_open	60.06%
lab3xa_short	80.56%
TOTAL Coverage:	
Exhaustive	71.71%
Total weight of fault universe: 1520.00	
Simulation Summary:	

Tag	Completed	Failed
lab3xa	954	0

When using AWRS, the Custom Fault tool outputs details on sample size and the 95% confidence interval, as shown in the following example:

```

Weighted test coverage:
  sim_sample          67.87% (sample size: 221) (95% Confidence
Interval: (61.54%,73.76%))
  sim_sample_open     62.96% (sample size: 27) (95% Confidence
Interval: (NA,NA))
  sim_sample_short    72.00% (sample size: 25) (95% Confidence
Interval: (NA,NA))

TOTAL Coverage:
  Sampled             67.87% (sample size: 221) (95% Confidence
Interval: (61.54%,73.76%))

```

When using WSS, the Custom Fault tool outputs the `WSS-Sampled` keyword along with an absolute error percentage, as shown in the following example:

```

Weighted test coverage:
  wss                 56.17% Error = +/- 19.99%
  wss_open            52.03% Error = +/- 21.57%
  wss_short           59.80% Error = +/- 18.60%

Total Coverage:
  wss_sampled         56.17% Error = +/- 19.99%

```

Detailed Coverage Listing

The Detailed Coverage Listing section provides a list of all defects along with details like coverage code, instance name, port names, net names, and so on. The following codes are used for this listing:

Code	Description
CU	Covered by Upper boundary of measurement.
CL	Covered by Lower boundary of measurement.
CI	Covered by Inside boundaries of measurement.
CW	Covered by Waveform comparison.
CV	Covered by Vector comparison.
CF	Covered by Failed measure (for example, a flat-lined signal).
CE	Covered by WV expression.
CC	Covered by CCK expression.

Code	Description
CM	Covered by detection Message.
CX	Covered - not categorizable.
NC	Not Covered.
NS	Not Simulated.
FI	Not Covered - Invalid .measure statement.
F	Not Covered - Simulation failed.

The coverage listing can be reported in a `simple` format (see [Simple Format Report](#)) or a `detailed` format (see [Detailed Format Report](#)).

Use the `report` command argument `"test_coverage" : "simple"|"detailed"` to define the reporting style.

Simple Format Report

The simple report uses one line per defect, as follows:

Id	Defect	lab3xa	Simulation Data	Group Weight
Instance/Node(s)	Port(s)	Net	Name(s)	
A1.1	mos_open d	CV	fault_mos_open_1_1_1/mos_open_1.*	1.00E+00 x1.mn2
A2.2	mos_open g	CV	fault_mos_open_2_2_2/mos_open_2.*	1.00E+00 x1.mn2
		x1.k		

The following table lists the simple report's columns:

Column	Description
Id	Defect ID. The ID format is as follows: <i>Agroup_id.defect_id</i>
Defect	Fault Model.
lab3xa	Simulation Tag.
Simulation Data	Path to simulation data.
Weight/Group Weight	Weight for each defect or Group Weight for a collapsed defect (that is, all defects collapsed together form a group). The Group Weight is the sum of weights of every defect that belong to the group.

Column	Description
Instance/Node(s)	Defects injected on devices have an instance name. Defects injected on nodes have a node name. A net-to-net short is a node-based defect.
Port(s)	The device/instance terminal ports.
Net Name(s)	Nets to which the ports are connected.

The following is an example snipped from a report file showing collapsed defects and group weights:

Id	Defect	psim	Simulation Data	Group
Weight	Instance/Node(s)	Port(s)	Net Name(s)	
A378.378	mos_short NC	fault_mos_short_378_378_378/mos_short_378.*		
2.000000e+00	x4.x5.mn1	g, d	a[0], x4.an	
A380.380	mos_short CV	fault_mos_short_380_380_380/mos_short_380.*		
1.000000e+00	x4.mn2	d, g	n3, x4.k	

You can tell the defects are collapsed as there are skips in the numbers – the ID changes from 378 to 380, skipping 379. Group 378 has weight of 2 because it has two defects, each with a default weight of 1.

Detailed Format Report

The detailed report provides more verbose output which uses multiple lines per defect, as follows:

```
Fault ID: A1.1
Defect Type: mos_open
Weight: 1.000000e+00
Instance: x1.mn2
Port: d
Net Name: cout

Tag: lab3xa
Covered: yes
Data: /remote/FAULT_RESULT_DET/lab3xa.1/fault_mos_open_1_1_1/mos_open_1.*

Code: CV
Type: Output comparison error
Signal: cout
Violation-Condition: L expected
Violation-Time: 0.653us

Fault ID: A2.2
Defect Type: mos_open
Weight: 1.000000e+00
Instance: x1.mn2
Port: g
Net Name: x1.k

Tag: lab3xa
Covered: yes
```

```
Data: /remote/FAULT_RESULT_DET/lab3xa.1/fault_mos_open_2_2_2/mos_open_2.*  
  
Code: CV  
Type: Output comparison error  
Signal: cout  
Violation-Condition: H expected  
Violation-Time: 0.33us
```

Test Coverage Report (Multi-Testbench)

You can run the [Custom Fault Multi-Testbench Flow](#) on the same design with different stimuli to improve the output coverage. Each testbench goes through a separate fault campaign. The output `.report` file for all fault campaigns, except the first, reports coverage details for the current campaign in addition to the coverage from previous campaigns.

The following is an example of output report of an incremental fault campaign with only not-covered defects simulated in it:

Tags	Sample-Size	
sim_1st_vec_xa	Exhaustive	
sim_2nd_vec_xa	Exhaustive	
*DEFECT UNIVERSE SUMMARY		
mos_open	656	
mos_short	864	
TOTAL	1520	
*COVERAGE SUMMARY		
Defects covered - weighted (covered/simulated):		
Type	sim_1st_vec_xa	sim_2nd_vec_xa
Total (Union)		
mos_open	399/656	8/257
407/656		
mos_short	783/864	2/81
785/864		
TOTAL	1182/1520	10/338
1192/1520		
TOTAL_OPEN	399/656	8/257
407/656		
TOTAL_SHORT	783/864	2/81
785/864		
Weighted test coverage:		
sim_1st_vec_xa	77.76%	
+ sim_2nd_vec_xa	78.42%	
+ sim_2nd_vec_xa_open	62.04%	
+ sim_2nd_vec_xa_short	90.86%	
TOTAL Coverage:		
Exhaustive	78.42%	

Total weight of defect universe: 1520.00

Simulation Summary:

Tag	Completed	Failed
sim_1st_vec_xa	954	0
sim_2nd_vec_xa	289	0

*DETAILED COVERAGE LISTING

```

...
Id      Defect      sim_1st_vec_xa  sim_2nd_vec_xa  Simulation Data
  Group Weight  Instance/Node(s)  Port(s)  Net Name(s)

A1.1  mos_open  CV              NS              fault_mos_open_1_1_1/mos_open_1.*
      1.000000e+00 x1.mn2          d          cout
A2.2  mos_open  CV              NS              fault_mos_open_2_2_2/mos_open_2.*
      1.000000e+00 x1.mn2          g          x1.k
A3.3  mos_open  CV              NS              fault_mos_open_3_3_3/mos_open_3.*
      1.000000e+00 x1.mn2          s          0
A4.4  mos_open  NC              NC              fault_mos_open_4_4_4/mos_open_4.*
      1.000000e+00 x1.mn2          b          0
  
```

The [Header](#) section contains multiple tags because it is a multi-testbench report.

The [Coverage Summary](#) section shows coverage for previous fault campaigns and the current campaign.

In the preceding example, the first testbench coverage is 77.76% (1182/1520) . The second testbench is simulating only the defects that are not covered by the first campaign (that is, $1520 - 1182 = 338$). It is able to cover 10 defects out of these 338 defects, effectively making the total coverage as 1192/1520 (that is, 78.42%).

The [Detailed Coverage Listing](#) shows coverage information of individual defects from both testbenches as a [Simple Format Report](#). In this example, notice that the defects that are covered by `sim_1st_vec_xa` are not simulated (NS) by `sim_2nd_vec_xa`

The coverage listing can be reported in a [Detailed Format Report](#), as well. An example of such listing is as follows:

```

Fault ID: A1.1
Defect Type: mos_short_gds
Weight: 1.000000e+00
Instance: xtop.xinvla.mn
Ports: s d
Net Names: 0 out1

Tag: tb1
Covered: yes
Data: /remote/path/cf.mtbxa/tb1.1/fault_mos_short_1_1_1/mos_short_1.*

Code: CL
Type: meas
Name: out1
Value: 1.569220e-06
Violation-Condition: < 0.95
Violation-Time: 1.500000e-08
  
```

```
Fault ID: A2.2
Defect Type: mos_short_gds
Weight: 1.000000e+00
Instance: xtop.xinv2a.mn
Ports: s d
Net Names: 0 out2

Tag: tb1
Covered: no
Data: /remote/path/cf.mtbxa/tb1.1/fault_mos_short_2_2_2/mos_short_2.*

Code: NC
Type: not_covered

Tag: tb2
Covered: yes
Data: /remote/path/cf.mtbxa/tb2.1/fault_mos_short_2_2_2/mos_short_2.*

Code: CL
Type: meas
Name: out2
Value: 1.569220e-06
Violation-Condition: < 0.95
Violation-Time: 1.50000
```

The details of a defect in the next fault campaign are appended to the previous campaigns. In the preceding example, the defect with ID A2.2 is not covered in tb1, but is covered in tb2.

Diagnostic Coverage Report

The diagnostic coverage report is contained in the file with a .diagnostic_report extension. This file contains the [Header](#), [Defect Universe Summary](#), and [Coverage Summary](#) similar to a test coverage report. The diagnostic coverage report details are printed as follows:

DIAGNOSTIC COVERAGE OF RESIDUAL FAULT REPORT		
ID:	FM001	
Label:	OverVoltage	
Tag:	xa	Total
Sampled:	N	N
Collapsed:	Y	Y
Faults Injected:	159.38	159.38
Failed:	0.00	0.00
Safe Detected:	53.94	53.94
Safe Undetected:	38.70	38.70
Dangerous Detected:	66.75	66.75
Dangerous Undetected:	0.00	0.00
Failed Dangerous Undetected:	0.00	0.00
Defect Tolerance:	159.38	159.38
Diagnostic Coverage:	100%	100%

Item	Description
ID:	The <code>id_report_name</code> is reported after ID:
Label:	The <code>id_report_label</code> is reported after Label:
Tag:	The tag specified after <code>-t</code> in the command line is reported after Tag:
Sampled: Y or N	Indicates whether exhaustive defect injection or fault sampling was used.
Collapsed: Y or N	Indicates whether collapsing of equivalent defects was enabled or disabled.
Faults Injected:	The total weight of all injected defects.
Safe Detected:	The weight of safe detected defects. These are defects that did not trigger functional outputs but did trigger the safety mechanism output.
Safe Undetected:	The weight of safe undetected defects. These defects did not trigger the functional outputs nor the safety mechanism outputs
Dangerous Detected:	The weight of dangerous detected defects. These defects triggered both the functional and safety mechanism observation points.
Dangerous Undetected:	The weight of dangerous undetected defects. These defects triggered the functional output observation points but did not trigger the safety mechanism observation points.
Defect Tolerance:	The weight of safe detected plus dangerous detected defects.
Diagnostic Coverage:	The percentage of dangerous detected defects over all dangerous defects.
Conf Interval (95%)	The 95% confidence interval for Diagnostic Coverage. This Conf Interval is only reported when the Custom Fault tool has been run with sampling. For more information on 95%CI, see Using Adaptive Weighted Random Sampling (AWRS) .

When using AWRS (see [Using Adaptive Weighted Random Sampling \(AWRS\)](#)), the Custom Fault tool reports the 95% Confidence Interval as follows:

DIAGNOSTIC COVERAGE OF RESIDUAL FAULT REPORT

ID:	FM001	
Label:	OverVoltage	
Tag:	xa	Total
Sampled:	Y	Y

Collapsed:	Y	Y
Faults Injected:	30.99	30.99
Failed:	0.00	0.00
Safe Detected:	11.44	11.44
Safe Undetected:	8.47	8.47
Dangerous Detected:	11.07	11.07
Dangerous Undetected:	0.00	0.00
Failed Dangerous Undetected:	0.00	0.00
Defect Tolerance:	30.99	30.99
Diagnostic Coverage:	100%	100%
Conf Interval (95%):	100%-100%	100%-100%

When using WSS (see [Using Weight Sorted Sampling \(WSS\)](#)), the Custom Fault tool reports the sampling error as follows:

DIAGNOSTIC COVERAGE OF RESIDUAL FAULT REPORT

D = Weight of dangerous detected faults
 U = Weight of dangerous undetected faults
 N = Weight of not simulated faults
 $\text{Diagnostic Coverage} = 0.5 * (2 * D + N) / (D + U + N)$
 $\text{Error} = 0.5 * N / (D + U + N)$

ID:	FM001	
Label:	OverVoltage	
Tag:	xa	Total
Sampled:	N	N
Collapsed:	Y	Y
Faults Injected:	147.76	147.76
Failed:	0.00	0.00
Safe Detected:	49.45	49.45
Safe Undetected:	32.11	32.11
Dangerous Detected:	66.20	66.20
Dangerous Undetected:	0.00	0.00
Not Simulated:	11.62	11.62
Failed Dangerous Undetected:	0.00	0.00
Defect Tolerance:	147.76	147.76
Diagnostic Coverage:	92.53%	92.53%
WSS Sampling Error:	7.467%	7.467%

The Detailed Defect Listing appears next as shown in the following example:

```
Fault ID: A2.2
Defect Type: mos_mac_open
Weight: 1.000000e+00
Instance: xbandgap.xi62.xn22
Port: d
Net Name: xbandgap.pgate

Tag: xa
Data: <path>/xa.1/fault_subckt_open_2_2_2/subckt_open_2.*
Failure modes:
  ID: FM001
  Classification: safe_detected
  ID: FM002
  Classification: dangerous_detected
Details:
  Code: CU
```



```
Type: meas
Name: psm_alarm
Id: do_over_voltage
Value: 1.000000e+00
Violation-Condition: > 0.8
Violation-Time: 7.111580e-07

Code: CL
Type: meas
Name: vbg_min
Id: fo_under_voltage
Value: 3.195170e-04
Violation-Condition: < 0.6
Violation-Time: 5.100000e-06
```

The detailed report is followed by the failure mode distribution report. The failure mode distribution is a function of any defect that causes any functional output to be triggered. It is possible that a defect triggers two or more failure modes; in that case, a proportion of that defect is assigned to each failure mode. For example, if the defect triggers two failure modes, it contributes to 0.5 of each failure mode; if the defect triggers three failure modes, it contributes to 1/3rd of each failure mode.

For each failure mode, the total weight of defects causing that failure is reported together with the weight of defects causing that failure over the sum of the weights of all defects. Any defect that does not trigger any failure mode is reported as potentially safe.

```
Tag: xa
```

	Id	Id_label	Fault Weight	Percentage
	FM001	OverVoltage	11.07	35.73
	FM002	UnderVoltage	11.44	36.93
Potentially Safe			8.472	27.34

Not-covered Defect Analysis

The Custom Fault tool produces a *Not-covered Defect Analysis* report to aid in assessing defects that are not detected and covered. By default, the tool creates the `.nc_analysis.dat` and `.nc_analysis.csv` files in the output directory at the completion of a fault campaign.

The `.nc_analysis.dat` report file contains a clustering of not-covered defects. The defects are clustered by fault properties, their instance names, and Detectable Fault Analysis (DFA) information, if applicable. The instance names of any defect in a cluster only differ by one word. This clustering attempts to group like defects in the circuit structure by the instance name. For example, a circuit may have many identical subcircuits performing similar functions that differ only in their name. The observability of their defects may be very similar. For example, the following defects are grouped in a cluster:

```
xtop.xgain.xinv1.mn1
xtop.xgain.xinv2.mn1
xtop.xgain.xinv1.mpl
xtop.xgain.xinv2.mpl
```

The report is sorted by decreasing cluster weight, a representative defect for each cluster is indicated, and all defects within the cluster are sorted by decreasing weight. You can then analyze the representative defect. You must assess whether the tool's clustering allows any conclusion about the representative defect to be applied to the remaining defects.

For example, your analysis of the representative defect could conclude that the device was not sensitized and was therefore unobservable requiring a change in the test vector, or you might conclude that the defect is indeed undetectable. Such an analysis conducted in defect cluster order can help to improve the test coverage efficiently by addressing the highest weight clusters first.

The following is an example of a `.nc_analysis.dat` file:

```
Not-covered Fault Analysis
Defects belonging to each cluster -- ID & inject_fault command

Representative ID 845 (Cluster Weight = 16.000000):
845      inject_fault -inst x2.x5.mp1 -type short -node s d -res_val
2.000000e+02 -weight 4.000000e+00 -fault_type mos_short
979      inject_fault -inst x3.x5.mp1 -type short -node s d -res_val
2.000000e+02 -weight 4.000000e+00 -fault_type mos_short
1047     inject_fault -inst x2.x5.mn1 -type short -node s d -res_val
2.000000e+02 -weight 4.000000e+00 -fault_type mos_short
1111     inject_fault -inst x4.x5.mp1 -type short -node s d -res_val
2.000000e+02 -weight 4.000000e+00 -fault_type mos_short

Representative ID 1165 (Cluster Weight = 6.000000):
1165     inject_fault -inst x4.x2.mp1 -type short -node b s -res_val
2.000000e+02 -weight 6.000000e+00 -fault_type mos_short

Representative ID 1028 (Cluster Weight = 6.000000):
1028     inject_fault -inst x2.x1.x2.mn1 -type short -node s g -res_val
2.000000e+02 -weight 6.000000e+00 -fault_type mos_short

Representative ID 922 (Cluster Weight = 6.000000):
922      inject_fault -inst x1.x7.mn1 -type short -node s g -res_val
2.000000e+02 -weight 6.000000e+00 -fault_type mos_short

Representative ID 1207 (Cluster Weight = 4.000000):
1207     inject_fault -inst x3.x2.mn1 -type short -node b s -res_val
2.000000e+02 -weight 4.000000e+00 -fault_type mos_short

Representative ID 36 (Cluster Weight = 3.000000):
36       inject_fault -inst x1.x1.x3.mp1 -type open -node b -res_val
1.000000e+07 -weight 1.000000e+00 -fault_type mos_open
48       inject_fault -inst x1.x1.x4.mp1 -type open -node b -res_val
1.000000e+07 -weight 1.000000e+00 -fault_type mos_open
200      inject_fault -inst x2.x1.x3.mp1 -type open -node b -res_val
1.000000e+07 -weight 1.000000e+00 -fault_type mos_open
```

In the preceding example, the first cluster has a weight of 16 and consists of four source drain short defects. The defect with ID 845 is chosen as a representative defect and its analysis and conclusion could be applied to other defects within that cluster.

The second cluster with ID 1165 consists of only one defect, but it is the next highest weight uncovered defect.

Note:

The `.nc_analysis.csv` file provides the same information, as well as the defect count of each cluster, but in a `.csv` format.

14

Using Custom Fault for Functional Safety

This chapter describes how to use the Custom Fault tool to measure diagnostic coverage for functional safety.

- [Overview of Functional Safety](#)
- [ISO 26262 Basics](#)
- [Measuring the Diagnostic Coverage of Residual Faults](#)
- [Diagnostic Coverage of Residual Faults Tutorial](#)
- [Measuring the Diagnostic Coverage of Latent Faults](#)
- [Diagnostic Coverage of Latent Faults Tutorial](#)
- [Direct SPFM Calculation](#)

Overview of Functional Safety

ICs targeted for automotive functional safety applications have a requirement to demonstrate that the design is safe and free of unacceptable risk through qualitative and quantitative analysis. Standards such as ISO 26262 describe these requirements. One requirement described in ISO 26262 is the [Single Point Fault Metric \(SPFM\)](#), a measure of the robustness of a design against failure due to single faults. SPFM is typically calculated in a Failure Mode Effect and Diagnostic Analysis (FMEDA). This calculation requires a value for the [Diagnostic Coverage \(DC\)](#) which is a measure of the circuit's ability to self-detect single point faults and maintain a safe state of the system.

The Custom Fault tool can be applied to aid in the quantitative analysis of a circuit's robustness and behavior in the presence of transistor faults. The circuit behavior in the presence of the fault is analyzed to determine whether the circuit violates the electrical specification and whether the circuit's safety mechanism can detect the dangerous condition.

ISO 26262 Basics

ISO 26262 is a standard for the functional safety of road vehicles. It defines requirements for functional safety throughout the lifecycle of all automotive electronic and electrical mission circuit systems. Functional safety is the absence of risk that is judged to be unacceptable by societal norms to hazards caused by the failure or unintended behavior of E/E systems.

The following sections introduce some concepts from ISO 26262 that are related to evaluating diagnostic coverage:

- [Automotive Safety Integrity Level \(ASIL\)](#)
- [Single Point Fault Metric \(SPFM\)](#)
- [Single Point Faults and Residual Faults](#)
- [Diagnostic Coverage \(DC\)](#)
- [Calculating Diagnostic Coverage](#)
- [Failure Mode](#)
- [Latent Fault Metric](#)

Automotive Safety Integrity Level (ASIL)

The Automotive Safety Integrity Level (ASIL) defines an item's necessary ISO 26262 requirements and safety measures for avoiding unacceptable risk. There are four levels, A, B, C, and D, with D being the most stringent. An automotive system whose failure has the highest risk is classified as ASIL D.

Single Point Fault Metric (SPFM)

The Single Point Fault Metric (SPFM) is defined in ISO 26262-5:2018(E) Annex C in equation C.7. It is the sum of all mission circuit single point faults and residual faults failure rates over the sum of all mission circuit failure rates.

Equation 1 Single Point Fault Metric

$$SPFM = 1 - \frac{\sum_{SR,HW} (\lambda_{SPF} + \lambda_{RF})}{\sum_{SR,HW} \lambda}$$

The SPFM value is a quantitative measure of the design's robustness against single point failures. A high value is recommended.

ISO 26262 provides the following table as a possible source for the derivation of the target SPFM for different ASILs.

Table 34 SPFM for ASIL Levels

	ASIL B	ASIL C	ASIL D
SPFM	$\geq 90\%$	$\geq 97\%$	$\geq 99\%$

Single Point Faults and Residual Faults

ISO 26262 defines a single point fault as a hardware fault in an element that leads directly to the violation of a safety goal and no fault in that element is covered by any safety mechanism.

A residual fault is defined as a fault in a hardware element that leads to the violation of a safety goal where that portion of the fault is not controlled by a safety mechanism.

Single point faults and residual faults are similar in that they are the result of a single random hardware fault. The difference is by definition only. If a safety mechanism is not present, the fault is a single point fault. If a safety mechanism is present, but that safety mechanism does not detect or control the fault, it is a residual fault.

Figure 47 shows an example of a single point fault. A mission circuit element does not have any safety mechanism. Any fault in that element which violates a safety goal is a single point fault.

Figure 47 Single Point Fault

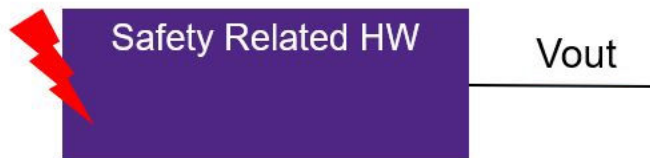
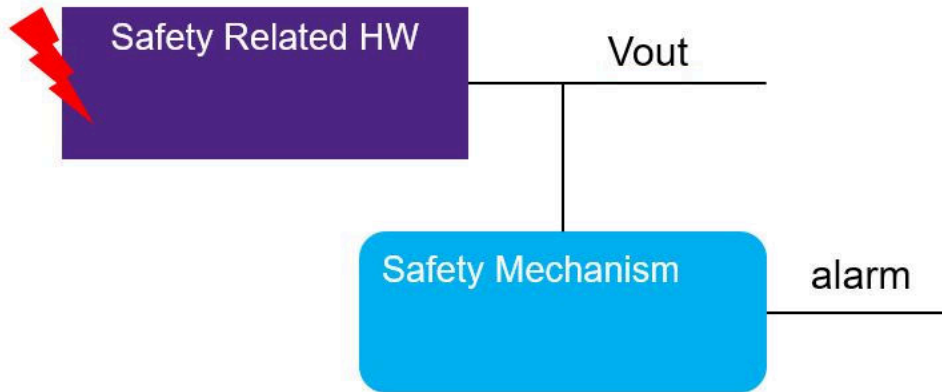


Figure 48 shows an example of a residual fault. Here a safety mechanism circuit is monitoring the output of the mission circuit item. If the fault in the mission circuit causes a

violation of a safety goal and the safety mechanism cannot detect or control the fault, this fault is defined as a residual fault.

Figure 48 Residual Fault



Diagnostic Coverage (DC)

ISO 26262 defines diagnostic coverage (DC) as the percentage of the failure rate of a hardware element or the percentage of the failure rate of a failure mode that is detected or controlled by the implemented safety mechanism. DC is a measure of the effectiveness of the safety mechanism.

Revisiting [Figure 48](#), if the fault in the mission circuit is detected and controlled by the safety mechanism, the fault is a covered fault and contributes to the diagnostic coverage.

Calculating Diagnostic Coverage

Diagnostic coverage is normally expressed as the percentage of dangerous faults detected or controlled by the safety mechanism over all dangerous faults in the element.

Equation 2 Diagnostic Coverage

$$DC = \frac{Dangerous_Detected}{Dangerous_Detected + Dangerous_Undetected}$$

[Equation 1](#) above shows that SPFM is a function of the failure rate of single point and residual faults. In practice, most mission circuit items require a safety mechanism to meet the ASIL requirements, so when safety mechanisms are present, an uncontrolled fault is a residual fault. Fortunately, ISO 26262 allows for the residual failure rate to be conservatively estimated from the diagnostic coverage.

Equation 3 Residual Fault Failure Rate

$$\lambda_{RF} \leq \lambda_{RF,est} = \lambda \times \left(1 - \frac{K_{DC,RF}}{100\%} \right)$$

In Equation 3 above, $K_{DC,RF}$ (also known as DC_{RF}) is the diagnostic coverage with respect to residual faults, expressed as a percentage.

When failure mode distributions and failure mode coverage are known, λ_{RF} can be expressed as:

Equation 4 Residual Fault Failure Rate With Known Failure Mode Distribution and Coverage

$$\lambda_{RF} = \sum_{alli} \lambda \times D_{F_{Mi},SR} \times (1 - F_{F_{Mi},safe}) \times F_{F_{Mi},PSVG} \times (1 - K_{F_{Mi},RF})$$

The Custom Fault analog fault injection tool can be used to measure the DC of residual faults for a mission circuit item. The DC value can then be input to a FMEDA to calculate the SPFM metrics.

Failure Mode

The ISO 26262 document defines the failure mode as the manner in which an element or an item fails. The Custom Fault tool allows you to define multiple measurements and also multiple pass/fail criteria for the same measurement.

In the Custom Fault tool, a failure mode is defined with functional and detection output results. The output results can be measurement checks or vector comparison or digital waveform comparison.

For example, a first failure mode for a bandgap corresponds to the bandgap output voltage being higher than the expected maximum value, while a second failure mode corresponds to the bandgap output voltage being lower than the expected minimum value.

Latent Fault Metric

The Latent Fault Metric (LFM) is defined in ISO 26262-5:2018(E) Annex C in equation C.8. It is the sum of all mission circuit multi-point faults that are latent over the sum of all safety related hardware failure rates less the failure rates of single point and residual faults as shown in the following equation.

$$LFM = 1 - \frac{\sum_{SR,HW} \lambda_{MPF,L}}{\sum_{SR,HW} (\lambda - \lambda_{SPF} - \lambda_{RF})}$$

The LFM value is a quantitative measure of the design robustness against latent failures. A high value is recommended.

ISO 26262 provides the following table as a possible source for the derivation of the target latent fault metric for different ASILs.

	ASIL B	ASIL C	ASIL D
LFM	$\geq 60\%$	$\geq 80\%$	$\geq 90\%$

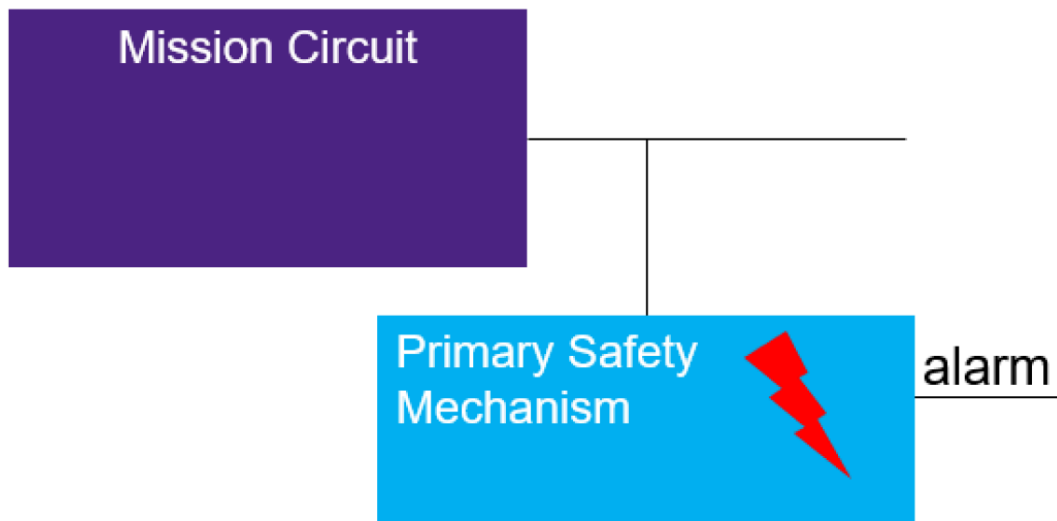
A latent fault is a multi-point fault whose presence is not detected by a safety mechanism. A multi-point fault is an individual fault that in combination with another independent fault leads to a multi-point failure.

ISO 26262 allows that multi-point faults of order three to be considered safe. This means that in most scenarios only dual-point faults (multi-point faults of order two) must be considered. Typically, a latent fault is a failure in the safety mechanism that goes undetected. If the safety mechanism has failed, the safety goal is violated if the mission circuit also fails.

Figure 49 shows an example of a latent fault. A mission circuit performing a safety function is protected by a primary safety mechanism. Assume that a random hardware failure occurs in the safety mechanism such that it can no longer detect a failure of the mission circuit. The safety mechanism does not assert its alarm signal.

The safety goal is not yet violated because the mission circuit is still providing its safety function. If the mission circuit were also to fail, then the safety goal is violated, and a dangerous condition exists. Because there is no latent safety mechanism to detect the failure of the primary safety mechanism that fault in the primary safety mechanism is a latent fault.

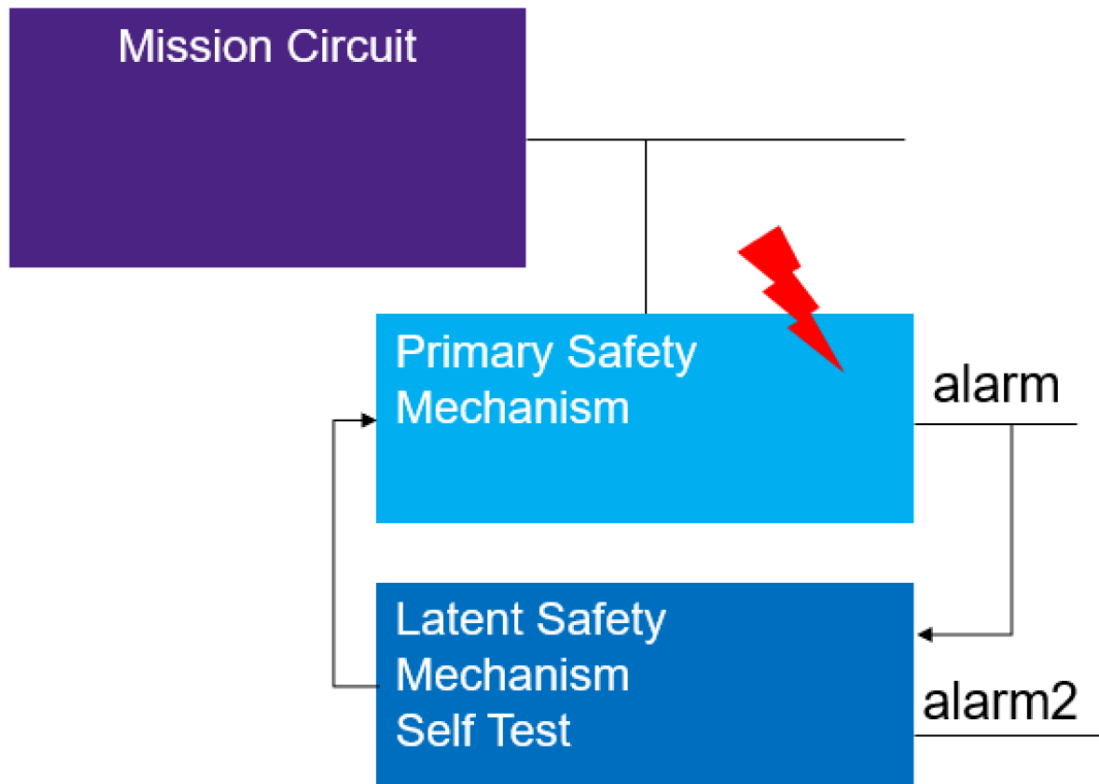
Figure 49 Example of a Latent Fault



A latent safety mechanism (see [Figure 50](#)) can be used to detect the presence of faults in the safety mechanism. A latent safety mechanism can be implemented as a self-test circuit to exercise and evaluate that the primary safety mechanism to ensure that it is functioning as expected.

To satisfy the ASIL requirements, analysis must show a sufficiently small number of latent faults in the system.

Figure 50 Latent Fault Safety Mechanism



Calculating Diagnostic Coverage for Latent Faults

Diagnostic coverage for latent faults is calculated in a similar manner as shown in [Calculating Diagnostic Coverage](#).

Equation 5 Diagnostic Coverage of Latent Faults

$$K_{DC,MPF,L} = \frac{\text{Dual Point Faults Prevented From Being Latent}}{\text{All Dual Point Faults}}$$

You can use the Custom Fault tool for analog fault injection too to measure the diagnostic of residual faults and latent faults for a safety related item.

Measuring the Diagnostic Coverage of Residual Faults

The following sections describe how to use the Custom Fault tool to measure the diagnostic coverage of residual faults:

- [Defining Observation Points](#)
- [Defining Thresholds for Fault Detection](#)
- [Diagnostic Coverage Command](#)

Defining Observation Points

The purpose of the observation point is to define how the tool determines whether the effect of an injected defect is observable. The observation point is also used to define whether the circuit safety mechanism can detect a failure of the circuit.

The Custom Fault tool supports the following mechanisms for defining observation points:

- `.MEASURE` Statement
- Vector Checking
- Digital Waveform Comparison

Note:

This section only describes the `.MEASURE` statement mechanism. More information on vector checking and digital waveform compare can be found in [Using a Vector Check](#) and [Digital Waveform Comparison in the Mixed-Signal Flow](#), respectively.

To evaluate diagnostic coverage, you must define the following sets of observation points:

- Functional Observation points

These are observation points on the mission circuit block that assess whether the circuit is functioning within specification or out-of-specification and potentially violating a safety goal.

- Detection Observation points

These are observation points on the safety mechanism and are used to determine whether the safety mechanism has detected or controlled a fault in the mission circuit block.

You should write the `.MEASURE` statements to check the critical circuit parameters at an appropriate time. For example, if the circuit simulation requires 10us for the circuit to get to its steady or operating state, the measurement should be made after that time.

In the following example, the signal `vout` is the output of the mission circuit block and the `.MEASURE` statements are used to test the maximum and minimum voltage of a signal in a specific time window. Assume that the associated safety mechanism has an output signal called `alarm` that asserts if `vout` is outside of its safe operating range. A `.MEASURE` statement is also required on the `alarm` signal.

```
.meas tran vout_max max v(vout) from=15u to=25u
.meas tran vout_min min v(vout) from=15u to=25u
.meas tran sm_alarm max v(alarm) from=15u to=25u
```

Now that observation points have been defined, the next step is to define the criteria for which a defect is detected.

Defining Thresholds for Fault Detection

Each observation point defined with a `.MEASURE` statement must have criteria to allow the tool to determine whether that observation point was triggered. Perform this using the `set_sim_stop` command.

To analyze diagnostic coverage, you must assign an ID to each `set_sim_stop` command using the option `-id id_name`. The `id_name` is used in the [Diagnostic Coverage Command](#) to associate this observation point as the functional output of the mission circuit or as the detection output of the safety mechanism. The `id_name` can be any user-defined string and it is not required to be unique. The `id_name` is always converted and processed in lowercase. The `diagnostic_coverage` command also converts and processes the `id_name` in lowercase.

For diagnostic coverage, you must also set the `-stop` option to 0 so that the fault simulation continues to the end of the netlist's defined transient time regardless of whether a detection event occurs.

PrimeSim XA Example:

```
.opt xa_cmd="set_sim_stop -check meas -name vout_max -max 0.9 -stop 0 -id func1"
.opt xa_cmd="set_sim_stop -check meas -name vout_min -min 0.7 -stop 0 -id func1"
.opt xa_cmd="set_sim_stop -check meas -name sm_alarm -max vdd/2 -stop 0 -id sm1"
```

FineSim or PrimeSim Example:

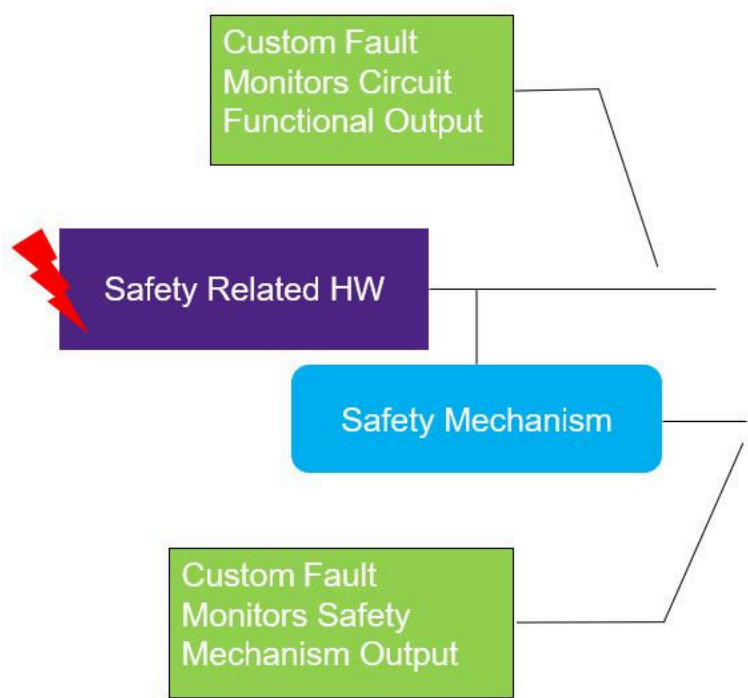
```
.set_sim_stop check=meas name=vout_max max=0.9 stop=0 id=func1
.set_sim_stop check=meas name=vout_min min=0.7 stop=0 id= func1
.set_sim_stop check=meas name=sm_alarm max=vdd/2 stop=0 id=sm1
```

Diagnostic Coverage Command

After you have defined observation points and criteria for triggering a detection at each observation point, you must use the `diagnostic_coverage` command to associate them with either the mission circuit as a functional output or with the safety mechanism as a

detection output. For more information, see [diagnostic_coverage](#) (for Residual Faults and Direct SPFM Calculation).

Figure 51 Diagnostic Coverage



The `diagnostic_coverage` command allows the tool to classify each injected defect into one of four categories based on which observation points are triggered.

Figure 52 Diagnostic Coverage Fault Classification

Fault Classification		Safety Mechanism Output	
		Undetected	Detected
Functional Output	Safe	Safe & Undetected	Safe & Detected
	Dangerous	Dangerous & Undetected (Residual Fault)	Dangerous & Detected (Diagnostic Coverage)

When the fault campaign is concluded, each fault is classified according to the table in [Figure 52](#). The diagnostic coverage is calculated as shown in [Equation 2](#).

The following sections provide more detail on the `diagnostic_coverage` command:

- [Diagnostic Coverage Command Example](#)
- [Analyze Fault Campaign Results](#)
- [Diagnostic Coverage Output Example](#)

Diagnostic Coverage Command Example

You must place the `diagnostic_coverage` commands in the input configuration file; see [diagnostic_coverage \(for Residual Faults and Direct SPFM Calculation\)](#). You can define multiple failure modes in a "failure_mode" block, however each mode must have a specific "id" and "id_label".

For this example, assume that the netlist contains the following `.MEASURE` and `set_sim_stop` commands:

```
.meas tran vout_max max v(vout) from=15u to=25u
.meas tran vout_min min v(vout) from=15u to=25u
.meas tran sm_alarm max v(alarm) from=15u to=25u

** for PrimeSim XA
.opt xa_cmd="set_sim_stop -check meas -name vout_max -max 0.9 -stop 0 -id fail_max"
.opt xa_cmd="set_sim_stop -check meas -name vout_min -min 0.7 -stop 0 -id fail_min"
.opt xa_cmd="set_sim_stop -check meas -name sm_alarm -max vdd/2 -stop 0 -id alarm"

** for FineSim or PrimeSim
.set_sim_stop check=meas name=vout_max max=0.9 stop=0 id=fail_max
.set_sim_stop check=meas name=vout_min min=0.7 stop=0 id=fail_min
.set_sim_stop check=meas name=sm_alarm max=vdd/2 stop=0 id=alarm
```

The `vout_max` and `vout_min` are the observation points for the mission circuit and the `sm_alarm` is the observation point for the safety mechanism.

The `diagnostic_coverage` command is as follows:

```
"diagnostic_coverage" : {
  "failure_mode" : [
    {
      "id" : "Bandgap max",
      "id_label" : "Bandgap_Max_Lbl",
      "functional_output" : [{ "type" : "meas", "id" : "fail_max" }],
      "detection_output" : [{ "type" : "meas", "id" : "alarm" }]
    },
    {
      "id" : "Bandgap min",
      "id_label" : "Bandgap_Min_Lbl",
      "functional_output" : [{ "type" : "meas", "id" : "fail_min" }],
      "detection_output" : [{ "type" : "meas", "id" : "alarm" }]
    }
  ]
}
```

Analyze Fault Campaign Results

See [Diagnostic Coverage Report](#).

Diagnostic Coverage Output Example

The following is an excerpt of a `.diagnostic_report` file when two failure modes, `BGFMunder` and `BGFMover`, have been defined in the config file with the following syntax:

```
"diagnostic_coverage" : {  
  "failure_mode": [  
    {  
      "id" : "BGFMunder",  
      "id_label" : "under_voltage",  
      "functional_output" : [ {"type" : "meas", "id" : "fo_under"} ],  
      "detection_output" : [ {"type" : "meas", "id" : "do"} ]  
    },  
    {  
      "id" : "BGFMover",  
      "id_label" : "over_voltage",  
      "functional_output" : [ {"type" : "meas", "id" : "fo_over"} ],  
      "detection_output" : [ {"type" : "meas", "id" : "do"} ]  
    }  
  ]  
},
```

[Figure 53](#) shows the resulting diagnostic coverage and detailed reports, while [Figure 54](#) shows the failure mode distribution report.

Chapter 14: Using Custom Fault for Functional Safety Measuring the Diagnostic Coverage of Residual Faults

Figure 53 Example — Diagnostic Coverage and Detailed Reports

```

TOTAL Coverage:
  Exhaustive                23.57%

Total weight of fault universe: 789.00
[]

DIAGNOSTIC COVERAGE OF RESIDUAL FAULT REPORT

      ID:      BGMunder
      Label:   under_voltage

      Tag:      fault_d1      Total
      Sampled:  N            N
      Collapsed: Y            Y
      Faults Injected: 789.00 789.00
      Safe Detected:   140.00 140.00
      Safe Undetected: 531.00 531.00
      Dangerous Detected: 92.00 92.00
      Dangerous Undetected: 26.00 26.00
      Defect Tolerance: 763.00 763.00
      Diagnostic Coverage: 77.97%

      ID:      BGMover
      Label:   over_voltage

      Tag:      fault_d1      Total
      Sampled:  N            N
      Collapsed: Y            Y
      Faults Injected: 789.00 789.00
      Safe Detected:   169.00 169.00
      Safe Undetected: 552.00 552.00
      Dangerous Detected: 63.00 63.00
      Dangerous Undetected: 5.00 5.00
      Defect Tolerance: 784.00 784.00
      Diagnostic Coverage: 92.65%

DETAILED REPORT

CU: Covered by Upper boundary
CL: Covered by Lower boundary
CW: Covered by Waveform comparison
CV: Covered by Vector comparison
CF: Covered by Failed measure - e.g. flat-lined signal
CE: Covered by WV expression
CX: Covered - not categorizable
NI: Not Covered - Invalid ,measure statement
NC: Not Covered
NS: Not Simulated
F: Not Covered - Simulation failed

Fault ID: A1.1
  Defect Type: mac_mos_open
  Weight: 1.000000e+00
  Instance: xbg.xml
  Port: s

  Tag: fault_d1
  Data: /slowfs/bandgap/AF/fault_d1/fault_subckt_open_1_1_1.*
  Failure modes:
    ID: BGMunder
    Classification: safe_detected
    ID: BGMover
    Classification: safe_detected
  Details:
    Code: CL
    Type: meas
    Name: bg
    Id: do
    Value: 2.790360e-09
    Violation-Condition: < 1.0

```


Figure 54 Example — Failure Mode Distribution Report

FAILURE MODE FAULT WEIGHT DISTRIBUTION REPORT				
Tag: run1				
	Id	Id_label	Fault Weight	Percentage
	FM001	OverVoltage	43	35.25
	FM002	UnderVoltage	35	28.69
Potentially Safe			44	36.07

Diagnostic Coverage of Residual Faults Tutorial

This tutorial shows you how to use the Custom Fault tool to measure diagnostic coverage of residual faults in a mission circuit block in a circuit.

The circuit used in the tutorial is a bandgap with a comparator used as the safety mechanism. The safety mechanism detects whether the bandgap fails to provide the expected output voltage. The bandgap expected voltage is defined as a voltage range within a minimum and maximum value.

In the tutorial, two failure modes are defined: one is when the bandgap output is below the expected voltage, and the other is when the bandgap output is above the expected voltage. Diagnostic coverage is reported for both failure modes.

The following sections describe how to perform the diagnostic coverage tutorial:

- [Tutorial Files and Commands for the PrimeSim XA Simulator](#)
- [Tutorial Files and Commands for the FineSim Simulator](#)
- [Tutorial Files and Commands for the PrimeSim Simulator](#)
- [Output Results](#)

Tutorial Files and Commands for the PrimeSim XA Simulator

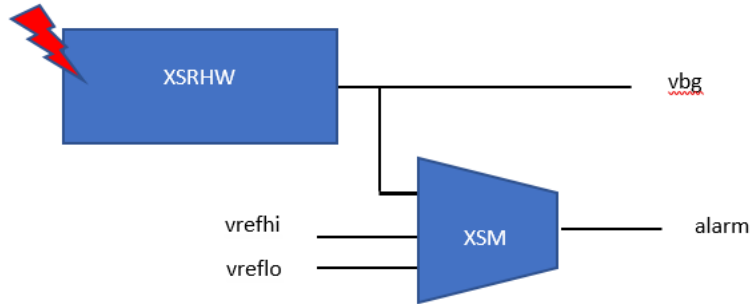
This section outlines how to run the tutorial with the PrimeSim XA tool. The tutorial files are:

- `af-xa.json`: Input configuration file for the Custom Fault tool
- `bandgap.sp`: SPICE netlist of the circuit
- `dpconfig`: CDPL configuration file
- `run_cf_xa`: Run script to run the Custom Fault tool

The test case in the `bandgap.sp` tutorial is a bandgap with a comparator circuit. As shown in [Figure 55](#), the bandgap instance is XSRHW and is the mission circuit. The comparator instance is an XSM_comparator and is the safety mechanism. The functional output signal

is the output of the bandgap: $v(bg)$, and the safety mechanism signal is the comparator output, $v(alarm)$.

Figure 55 Bandgap Circuit for Tutorial



Observation Points and Thresholds for Fault Detection

The functional outputs are measured with:

```
.meas tran vbg_max max v(vbg) from=1u
.meas tran vbg_min min v(vbg) from=1u
```

The detection output is measured with:

```
.meas tran psm_alarm max v(alarm) from=1u
```

The thresholds are specified with the following `set_sim_stop` commands. The maximum value allowed for $v(bg)$ is 0.64V, and its minimum value is 0.62V. The safety mechanism $v(alarm)$ signal must be less than 0.8V.

```
.opt xa_cmd="set_sim_stop -check meas -name vbg_max
-max 0.64 -stop 0 -id fo_over_voltage"
```

```
.opt xa_cmd="set_sim_stop -check meas -name vbg_min
-min 0.62 -stop 0 -id fo_under_voltage"
```

```
.opt xa_cmd="set_sim_stop -check meas -name psm_alarm
-max 0.8 -stop 0 -id do_over_voltage"
```

All of these commands are included in the `bandgap.sp` netlist.

Diagnostic Coverage Commands

The [diagnostic_coverage \(for Residual Faults and Direct SPFM Calculation\)](#) commands are specified in the `af-xa.json` file. This file specifies two failure modes: one for over voltages when the functional output is above the "FM001" maximum threshold, and one

for under voltages when the functional output is below the "FM002" minimum threshold. In both failure modes, the same safety mechanism is used.

```
"diagnostic_coverage" : {
  "failure_mode" : [
    {
      "id" : "FM001",
      "id_label" : "OverVoltage",
      "functional_output" : [
        { "type" : "meas", "id" : "fo_over_voltage" }
      ],
      "detection_output" : [
        { "type" : "meas", "id" : "do_over_voltage" }
      ]
    }, // end FM001
    {
      "id" : "FM002",
      "id_label" : "UnderVoltage",
      "functional_output" : [
        { "type" : "meas", "id" : "fo_under_voltage" }
      ],
      "detection_output" : [
        { "type" : "meas", "id" : "do_over_voltage" }
      ]
    } // end FM002
  ]
}, // End DC
```

Running the Custom Fault Tutorial With PrimeSim XA

- 1. Make sure both the PrimeSim XA and Custom Fault tools are installed and available.
- 2. Edit the `dpconfig` file and modify it to use your system jobs distribution (LSF, SGE, and so on).
- 3. Edit the `run_cf_xa` script and modify the number of keys to match the number of licenses you can run. In the tutorial script, the number of keys is set to 20 with `-k 20`.
- 4. Source the `run_cf_xa` script.

Output Files

When the run completes, the following files are generated in the `CF/` directory.

Table 35 *Output Files using the PrimeSim XA Simulator*

File Name	Description
<code>xa/</code>	This directory contains all of the intermediate files and results of each PrimeSim XA simulation. The name of this directory is specified with the <code>-t</code> option in the Custom Fault command line.

Table 35 *Output Files using the PrimeSim XA Simulator (Continued)*

File Name	Description
<code>xa.config.json</code>	Configuration file.
<code>xa.diagnostic_report</code>	Diagnostic coverage report for the Custom Fault simulation.
<code>xa.fdb</code>	Internal database.
<code>xa.FM001.dangerous_detected</code>	List of the dangerous_detected faults for the FM001 failure mode.
<code>xa.FM001.dangerous_undetected</code>	List of the dangerous_undetected faults for the FM001 failure mode.
<code>xa.FM001.safe_detected</code>	List of the safe_detected faults for the FM001 failure mode.
<code>xa.FM001.safe_undetected</code>	List of the safe_undetected faults for the FM001 failure mode.
<code>xa.FM002.dangerous_detected</code>	List of the dangerous_detected faults for the FM002 failure mode.
<code>xa.FM002.dangerous_undetected</code>	List of the dangerous_undetected faults for the FM002 failure mode.
<code>xa.FM002.safe_detected</code>	List of the safe_detected faults for the FM002 failure mode.
<code>xa.FM002.safe_undetected</code>	List of the safe_undetected faults for the FM002 failure mode.
<code>xa.log</code>	Log file of the Custom Fault simulation.
<code>xa_weights.cpp</code>	Weight template file automatically generated by the Custom Fault tool.

Tutorial Files and Commands for the FineSim Simulator

This section outlines how to run the tutorial with the FineSim tool. The tutorial files are:

- `af-fs.json`: Input configuration file for the Custom Fault tool
- `bandgap.sp`: SPICE netlist of the circuit
- `dpconfig`: CDPL configuration file
- `run_cf_fs`: Run script to run the Custom Fault tool

The circuit and the observation points are identical to those used in the PrimeSim XA tutorial. They are specified with the `.set_sim_stop` command. The maximum value

allowed for $v(bg)$ is 0.64V, and its minimum value is 0.62V. The $v(alarm)$ safety mechanism signal must be less than 0.8V.

```
.set_sim_stop check=meas name=vbg_max max=0.64 stop=0 id=fo_over_voltage
.set_sim_stop check=meas name=vbg_min min=0.62 stop=0 id=fo_under_voltage
.set_sim_stop check=meas name=psm_alarm max=0.8 stop=0 id=dö_over_voltage
```

The [Diagnostic Coverage Commands](#) are the same.

Running the Custom Fault Tutorial With FineSim

1. Make sure both the FineSim and Custom Fault tools are installed and available.
2. Edit the `dpconfig` file and modify it to use your system jobs distribution (LSF, SGE, and so on).
3. Edit the `run_cf_fs` script and modify the number of keys to match the number of licenses you can run. In the tutorial script, the number of keys is set to 20 with `-k 20`.
4. Source the `run_cf_fs` script.

Output Files

The output files are described in [Table 36](#).

Table 36 *Output Files using the FineSim Simulator*

File Name	Description
<code>fsim/</code>	This directory contains all the intermediate files and results of each FineSim simulation. The name of this directory is specified with the <code>-t</code> option in the Custom Fault command line.
<code>fsim.config.json</code>	Configuration file.
<code>fsim.diagnostic_report</code>	Diagnostic coverage report for the Custom Fault simulation.
<code>fsim.fdb</code>	Internal database.
<code>fsim.FM001.dangerous_detected</code>	List of the <code>dangerous_detected</code> faults for the FM001 failure mode.
<code>fsim.FM001.dangerous_undetected</code>	List of the <code>dangerous_undetected</code> faults for the FM001 failure mode.
<code>fsim.FM001.safe_detected</code>	List of the <code>safe_detected</code> faults for the FM001 failure mode.
<code>fsim.FM001.safe_undetected</code>	List of the <code>safe_undetected</code> faults for the FM001 failure mode.
<code>fsim.FM002.dangerous_detected</code>	List of the <code>dangerous_detected</code> faults for the FM002 failure mode.

Table 36 *Output Files using the FineSim Simulator (Continued)*

File Name	Description
<code>fsim.FM002.dangerous_undetected</code>	List of the dangerous_undetected faults for the FM002 failure mode.
<code>fsim.FM002.safe_detected</code>	List of the safe_detected faults for the FM002 failure mode.
<code>fsim.FM002.safe_undetected</code>	List of the safe_undetected faults for the FM002 failure mode.
<code>fsim.log</code>	Log file of the Custom Fault simulation
<code>fsim_weights.cpp</code>	Weight template file automatically generated by the Custom Fault tool.

Tutorial Files and Commands for the PrimeSim Simulator

This section outlines how to run the tutorial with the PrimeSim tool. The tutorial files are:

- `af-ps.json`: Input configuration file for the Custom Fault tool
- `bandgap_ps.sp`: SPICE netlist of the circuit
- `dpconfig`: CDPL configuration file
- `run_cf_ps`: Run script to run the Custom Fault tool

The circuit and the observation points are identical to those used in the PrimeSim XA tutorial. They are specified with the `.set_sim_stop` command. The maximum value allowed for `v(bg)` is 0.64V, and its minimum value is 0.62V. The `v(alarm)` safety mechanism signal must be less than 0.8V.

```
.set_sim_stop check=meas name=vbg_max max=0.64 stop=0 id=fo_over_voltage
.set_sim_stop check=meas name=vbg_min min=0.62 stop=0 id=fo_under_voltage
.set_sim_stop check=meas name=psm_alarm max=0.8 stop=0 id=do_over_voltage
```

The [Diagnostic Coverage Commands](#) are the same.

Running the Custom Fault Tutorial With PrimeSim

1. Make sure both the PrimeSim and Custom Fault tools are installed and available.
2. Edit the `dpconfig` file and modify it to use your system jobs distribution (LSF, SGE, and so on).
3. Edit the `run_cf_ps` script and modify the number of keys to match the number of licenses you can run. In the tutorial script, the number of keys is set to 20 with `-k 20`.
4. Source the `run_cf_ps` script.

Output Files

The output files are described in [Table 37](#).

Table 37 *Output Files using the PrimeSim Simulator*

File Name	Description
psim/	This directory contains all the intermediate files and results of each PrimeSim simulation. The name of this directory is specified with the <code>-t</code> option in the Custom Fault command line.
psim.config.json	Configuration file.
psim.diagnostic_report	Diagnostic coverage report for the Custom Fault simulation.
psim.fdb	Internal database.
psim.FM001.dangerous_detected	List of the dangerous_detected faults for the FM001 failure mode.
psim.FM001.dangerous_undetected	List of the dangerous_undetected faults for the FM001 failure mode.
psim.FM001.safe_detected	List of the safe_detected faults for the FM001 failure mode.
psim.FM001.safe_undetected	List of the safe_undetected faults for the FM001 failure mode.
psim.FM002.dangerous_detected	List of the dangerous_detected faults for the FM002 failure mode.
psim.FM002.dangerous_undetected	List of the dangerous_undetected faults for the FM002 failure mode.
psim.FM002.safe_detected	List of the safe_detected faults for the FM002 failure mode.
psim.FM002.safe_undetected	List of the safe_undetected faults for the FM002 failure mode.
psim.log	Log file of the Custom Fault simulation
psim_weights.cpp	Weight template file automatically generated by the Custom Fault tool.

Output Results

After running the Custom Fault tool with either the PrimeSim XA, FineSim, or PrimeSim simulators, a file with a `.diagnostic_report` extension is generated. The first section in the `.diagnostic_report` file reports the total fault coverage and the next section reports the diagnostic coverage (see [Figure 56](#)), while the last section reports the details of the faults (see [Figure 57](#)).

Note:

The example output shown in [Figure 56](#) and [Figure 57](#) is from a FineSim simulator run. Output from a PrimeSim XA or PrimeSim simulator run includes all the same information although the exact results may vary.

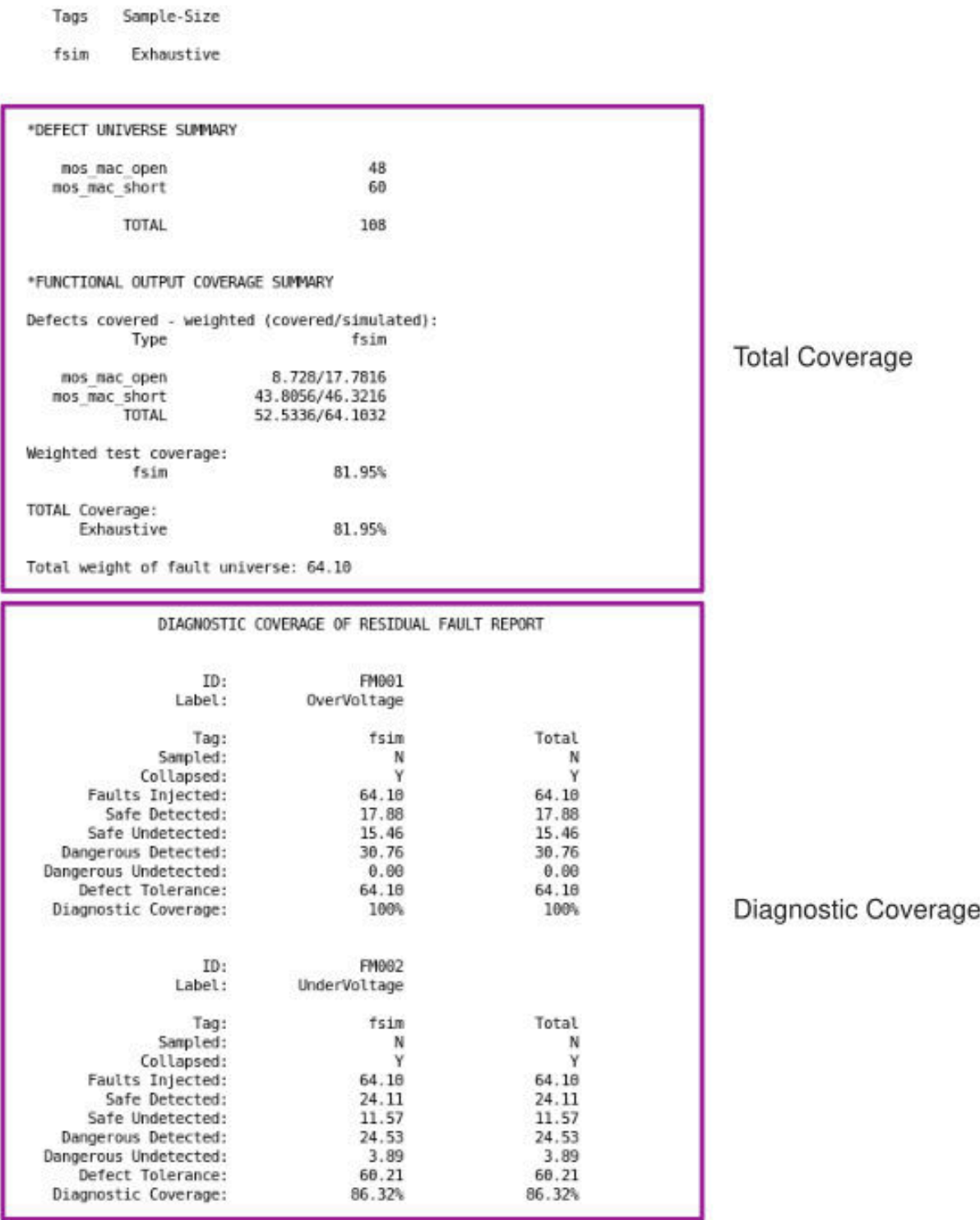
The total coverage for the bandgap circuit is 81.95%.

The diagnostic coverage is calculated as shown in [Equation 2](#).

For the first failure mode (FM001), the number of dangerous_detected faults is 30.76 and the number of dangerous_undetected faults is 0, so the diagnostic coverage is 100%.

For the second failure mode (FM002), the number of dangerous_detected faults is 24.53 and the number of dangerous_undetected faults is 3.89, so the diagnostic coverage is 86.32%.

Figure 56 Tutorial — Excerpt of the .diagnostic_report File



DIAGNOSTIC COVERAGE OF RESIDUAL FAULT REPORT

ID: FM001

Label: OverVoltage

Tag:	fsim	Total
Sampled:	N	N
Collapsed:	Y	Y
Faults Injected:	64.10	64.10
Safe Detected:	17.80	17.80
Safe Undetected:	15.46	15.46
Dangerous Detected:	30.76	30.76
Dangerous Undetected:	0.00	0.00
Defect Tolerance:	64.10	64.10
Diagnostic Coverage:	100%	100%

ID: FM002

Label: UnderVoltage

Tag:	fsim	Total
Sampled:	N	N
Collapsed:	Y	Y
Faults Injected:	64.10	64.10
Safe Detected:	24.11	24.11
Safe Undetected:	11.57	11.57
Dangerous Detected:	24.53	24.53
Dangerous Undetected:	3.89	3.89
Defect Tolerance:	60.21	60.21
Diagnostic Coverage:	86.32%	86.32%

Diagnostic Coverage

Figure 57 Tutorial — Excerpt of the Detailed Report in the .diagnostic_report File

```
DETAILED REPORT

CU: Covered by Upper boundary
CL: Covered by Lower boundary
CW: Covered by Waveform comparison
CV: Covered by Vector comparison
CF: Covered by Failed measure - e.g. flat-lined signal
CE: Covered by WV expression
CX: Covered - not categorizable
NI: Not Covered - Invalid .measure statement
NC: Not Covered
NS: Not Simulated
F: Not Covered - Simulation failed

Fault ID: A1.1
  Defect Type: mos_mac_short
  Weight: 1.104000e-01
  Instance: xsrhwxn1
  Ports: s g

  Tag: fsm
  Data: /tutorial_DC/CF/fsm/fault_subckt_short_1_1_1.*
  Failure modes:
    ID: FM001
    Classification: safe_undetected
    ID: FM002
    Classification: safe_undetected
  Details:
    Code: NC
    Type: not_covered

Fault ID: A3.3
  Defect Type: mos_mac_short
  Weight: 2.692800e+00
  Instance: xsrhwxm0
  Ports: s d

  Tag: fsm
  Data: /tutorial_DC/CF/fsm/fault_subckt_short_3_3_3.*
  Failure modes: _
```

Measuring the Diagnostic Coverage of Latent Faults

A latent fault is a multi-point fault that goes undetected by a safety mechanism. A multi-point fault is an individual fault that, when combined with another independent fault, leads to the violation of a safety goal. Typically, this means assessing the circuit's ability to self-detect failures of the primary safety mechanism.

If the dual-point faults of the safety mechanism are known these faults can be injected while exercising the latent safety mechanism to determine which dual-point faults are detected. It is assumed that when the latent safety mechanism detects a dual-point fault it asserts a signal that allows the system to prevent the fault from being latent (such as illuminating a warning lamp in the vehicle's dashboard). Dual-point faults that are not detected remain latent in the system. The diagnostic coverage can then be calculated by the equation described previously in the [Latent Fault Metric](#) section.

If the dual-point faults in the safety mechanism are not known you can use the following four-step procedure to determine the dual-point faults and determine if they are detected by the latent safety mechanism.

1. Perform diagnostic coverage of residual faults on the mission circuit. This step identifies failures of the mission circuit that the safety mechanism must detect.
2. Inject faults into the safety mechanism to determine safe faults of the safety mechanism. These are the faults that cause the safety mechanism to assert its detection output. These are false alarms, but still safe faults. These safe faults are excluded from the next steps.
3. This step determines the potentially latent faults in the safety mechanism. Perform dual-fault injection by simultaneously injecting each covered fault from step 1 with each of the remaining safety mechanism faults from step 2. The fault from step 1 should be detected by the safety mechanism. If the mission circuit fault is detected by the safety mechanism, then the safety mechanism fault did not cause the safety mechanism to fail and is not a dual-point fault. If the fault is not detected by the safety mechanism, then the safety mechanism fault caused it to fail. This is a dual-point fault that must be prevented from being latent.
4. This step measures the diagnostic coverage of the latent faults. Each of the safety mechanism's dual-point faults found in step 3 are injected one at a time to the safety mechanism while exercising the latent safety mechanism. Each detected fault is considered prevented from being latent and contributes to the diagnostic coverage of latent faults. Faults that are not detected are latent faults.

Steps 1, 2 and 3, make use of the diagnostic coverage of residual fault command because the mission circuit functionality and safety mechanism's ability to detect faults must be assessed.

Step 4 also requires an observation point and detection criteria. The observation point should be set on the signals that indicate that the latent safety mechanism has detected the failure of the safety mechanism. This is accomplished using the same methods as with residual faults.

Defining Observation Points and Thresholds for Fault Detection

The observation points and thresholds must be defined the same way as for residual faults. See the [Measuring the Diagnostic Coverage of Residual Faults](#) section.

To evaluate diagnostic coverage of latent faults an observation point must be set to define when the latent safety mechanism detects a dual-point fault.

Diagnostic Coverage Latent Faults Command Example

You must place the `diagnostic_coverage` commands in the input configuration file; see [diagnostic_coverage \(for Latent Faults\)](#).

Assume that the netlist is exercising a built-in self-test of the safety mechanism. When the self-test detects a failure of the safety mechanism it asserts the `sm_failure` signal. The observation point and criteria can be defined with the following commands

```
.meas tran sm_failure max v(sm_failure) from=15u to=25u
** for FineSim or PrimeSim
.set_sim_stop check=meas name=sm_failure max=vdd/2 stop=0 id=lsml
** for PrimeSim XA
.opt xa_cmd="set_sim_stop -check meas -name sm_failure -max vdd/2 -stop 0
-id lsml"
```

The diagnostic coverage command is:

```
"diagnostic_coverage" : {
  "failure_mode" : [
    "id" : "LFM1",
    "id_label" : "Voltage Monitor latent faults",
    "latent_detection_output" : [
      { "type" : "meas", "id" : "lsml" }
    ]
  ]
}
```

Analzying Fault Campaign Results

The latent diagnostic coverage report is contained in the `cf.latent_diagnsotic_report` file. This file contains the following data.

Sampled	Indicates if exhaustive fault injection was used or if fault sampling was used.
Collapsed	Indicates if collapsing of equivalent faults was enabled or disabled,
Faults Injected	Weight of all injected faults.
Detected by Latent SM	Weight of all injected faults detected by the latent safety mechanism.
Defect Tolerance	Count/weight of safe detected plus dangerous detected faults.
Diagnostic Coverage Latent Fault	Percentage of latent faults detected over the latent faults injected as a percentage.

Example of Diagnostic Coverage of Latent Faults Report

The following example shows a latent diagnostic report. The top portion of the file reports the latent diagnostic coverage for each defined failure mode. After the summary a detailed report of each injected fault is provided.

Figure 58 Latent Faults Report

DIAGNOSTIC COVERAGE OF LATENT FAULT REPORT		
ID:	LFM001	
Label:	Voltage Monitor Dual Point Faults	
Tag:	xa.mpfl.step4.all	Total
Sampled:	N	N
Collapsed:	Y	Y
Faults Injected:	24.25	24.25
Detected by Latent SM:	24.25	24.25
Diagnostic Coverage Latent Fault:	100%	100%

Diagnostic Coverage of Latent Faults Tutorial

This tutorial shows you how to use the Custom Fault tool to measure diagnostic coverage of latent faults. The circuit in the tutorial is a bandgap with comparators used as the safety mechanism. The safety mechanism detects if the bandgap fails to provide the expected output voltage. The bandgap expected voltage is defined as a range within a minimum and maximum voltage value. A built-in self-test mode is used to check the safety mechanism for latent faults. Under the test mode the input to the comparators is set to a high and low test voltage to check if the comparators are correctly detecting out of range voltages.

Two failure modes are defined: one mode is when the bandgap output is below the expected voltage, and the other mode is when the bandgap output is above the expected voltage. Diagnostic coverage is reported for both modes.

The Custom Fault tool can run with the PrimeSim XA, FineSim, or PrimeSim simulators. This tutorial explains how to use any simulator.

Note:

Before you run this tutorial, you must be familiar with the usage of the Custom Fault tool for diagnostic coverage of residual faults. Run the [Diagnostic Coverage of Residual Faults Tutorial](#) before you run this tutorial.

Tutorial Overview

The following sections describe the tutorial:

- [Input, Netlist, and Script Files](#)
- [Circuit Description](#)
- [Observation Points and Thresholds for Fault Detection](#)

Input, Netlist, and Script Files

This tutorial uses the following files.

Table 38 *Input, Netlist, and Script Files*

File Name	Description
af-xa-*.json	Input configuration files to use with the PrimeSim XA simulator
af-fs-*.json	Input configuration files to use with the FineSim simulator
af-ps-*.json	Input configuration files to use with the PrimeSim simulator
tb_dcrf.sp	SPICE netlist of the circuit for residual faults (used in Performing Diagnostic Coverage of Residual Faults , step 1. in Running the Diagnostic Coverage of Latent Faults Tutorial .)
tb_dcmpfl.sp	SPICE netlist of the circuit for latent faults (used in Diagnostic Coverage of Latent Faults , step 4. in Running the Diagnostic Coverage of Latent Faults Tutorial .)
tb_dcrf_ps.sp	PrimeSim SPICE netlist of the circuit for residual faults (used in Performing Diagnostic Coverage of Residual Faults , step 1. in Running the Diagnostic Coverage of Latent Faults Tutorial .)
tb_dcmpfl_ps.sp	PrimeSim SPICE netlist of the circuit for latent faults (used in Diagnostic Coverage of Latent Faults , step 4. in Running the Diagnostic Coverage of Latent Faults Tutorial .)
dpconfig	CDPL configuration file
run-xa-*	Run scripts to run with the PrimeSim XA simulator
run-fs-*	Run scripts to run with the FineSim simulator
run-ps-*	Run scripts to run with the PrimeSim simulator

Circuit Description

The `bandgap.sp` circuit in the tutorial is a bandgap with a comparator circuit. The bandgap instance is `XBandgap`, which is the safety-related hardware. The `XSM_comparator` comparator instance is the safety mechanism. The functional output signal is the output of the bandgap is `v(bg)` and the safety mechanism signal is the comparator output `v(alarm)`.

When analyzing latent faults, the circuit operates in a self-test mode driving a middle, high, and low voltage to the safety mechanism. The safety mechanism should respond by asserting its output during the high and low input ranges.

Observation Points and Thresholds for Fault Detection

The functional outputs are measured with:

```
.meas tran vbg_max max v(vbg) from=1u
.meas tran vbg_min min v(vbg) from=1u
```

The detection output is measured with:

```
.meas tran psm_alarm max v(alarm) from=1u
```

The thresholds are specified with the `set_sim_stop` command. The maximum value allowed for `v(bg)` is 0.64V, and its minimum value is 0.60V. The safety mechanism signal `v(alarm)` must be less than 0.8V. For example:

```
.opt xa_cmd="set_sim_stop -check meas -name vbg_max -max 0.64 -error 0
-id fo_over_voltage"
.opt xa_cmd="set_sim_stop -check meas -name vbg_min -min 0.60 -error 0
-id fo_under_voltage"
.opt xa_cmd="set_sim_stop -check meas -name psm_alarm -max 0.8 -error 0
-id do_over_voltage"
```

All these commands are included in the `tb_dcrf.sp` or `tb_dcrf_ps.sp` netlist.

Running the Diagnostic Coverage of Latent Faults Tutorial

To correctly measure the diagnostic coverage of latent faults, the dual-point faults of the safety mechanism must be known. In this tutorial a four-step procedure is applied to first determine the dual-point faults of the safety mechanism and then assess if they are prevented from being latent.

The steps are:

1. Performing Diagnostic Coverage of Residual Faults

The diagnostic coverage of residual fault results using the $\pm 200\text{mV}$ criteria is used as the basis for assessing latent faults.

2. Analyzing Defects in the Safety Mechanism

3. Running Dual-Fault Injection

Discover which faults cause the safety mechanism to fail.

4. Diagnostic Coverage of Latent Faults

Inject the safety mechanism dual-point faults discovered in step 3. and measure the diagnostic coverage of the latent faults.

Performing Diagnostic Coverage of Residual Faults

To run diagnostic coverage of residual faults, do the following steps.

1. Make sure both the Custom Fault tool and any of PrimeSim XA, FineSim, or PrimeSim tools are installed and available.
2. Edit the `dpconfig` file and modify it to use your system jobs distribution (LSF, SGE, and so on).
3. Edit the `run-xa-dcrf`, `run-fs-dcrf`, or `run-ps-dcrf` script and modify the number of keys to match the number of licenses you can run. In the tutorial script, the number of keys is set to 20 with `-k 20`.
4. Run the Custom Fault tool with one of the simulators.
 - To run the Custom Fault tool with the PrimeSim XA simulator, source the `run-xa-dcrf` script.
 - To run the Custom Fault tool with the FineSim simulator, source the `run-fs-dcrf` script.
 - To run the Custom Fault tool with the PrimeSim simulator, source the `run-ps-dcrf` script.

The results of these simulations are in the `xa.dcrf`, `fs.dcrf`, or `ps.dcrf` directories, respectively, as shown in .

Table 39 Diagnostic Coverage of Residual Faults Output Files

File Name	Description
<code>xa.dcrf/</code> <code>fs.dcrf/</code> <code>ps.dcrf/</code>	Directory that contains all the intermediate files and results of each PrimeSim XA, FineSim, or PrimeSim simulation. The name of this directory is specified with the <code>-t</code> option on the <code>customfault</code> command line.
<code>.config.json</code>	Configuration file.
<code>.diagnostic_report</code>	Diagnostic coverage report of the Custom Fault simulation.

Table 39 *Diagnostic Coverage of Residual Faults Output Files (Continued)*

File Name	Description
.fdb	Internal fault database file.
.dangerous_detected	List of the <code>dangerous_detected</code> faults for a given failure mode.
.dangerous_undetected	List of the <code>dangerous_undetected</code> faults for a given failure mode.
.safe_detected	List of the <code>safe_detected</code> faults for given failure mode.
.safe_undetected	List of the <code>safe_undetected</code> faults for given failure mode.
.dangerous_detected.failure_margin	List of the dangerous detected faults and their failure margin over the specified fault detection criteria.
.log	Fault simulation log file.
.cpp	Weight template file automatically generated by the Custom Fault tool.

Analyzing Defects in the Safety Mechanism

In this step, defects are injected into the safety mechanism. The purpose is to find defects that cause false assertions of the safety mechanism. These defects cause safe faults. They are not dual-point faults and thus cannot be latent faults. So they must be excluded from further analysis. The process of finding and removing these defects at this stage reduces the defect universe size in the next step when dual-fault injection is performed.

The fault injection scope must be changed from the bandgap to the safety mechanism, `xbandgap` to `xsm_comparator`. These changes can be seen in the `af-xa-mpfl-step2.json` or `af-fs-mpfl-step2` files.

The `diagnostic_coverage` command for residual faults from the previous step in the tutorial is used again in this step. The reason is that the functional and detection output criteria remain the same.

To analyze the defects in the safety mechanism, do one of the following:

- To run the Custom Fault tool with the PrimeSim XA simulator, source the `run-xa-mpfl-step2` script.
- To run the Custom Fault tool with the FineSim simulator, source the `run-fs-mpfl-step2` script.
- To run the Custom Fault tool with the PrimeSim simulator, source the `run-ps-mpfl-step2` script.

The output files of this step are same as those produced by diagnostic coverage of residual faults. The `*.diagnostic_report` file provides a detailed accounting of each fault injected in the safety mechanism (see the failure mode fault list files).

The faults listed in the `*.safe_detected` and `*.dangerous_detected` sections are excluded from analysis in the subsequent steps. These faults are safe faults with the assumption that when the safety mechanism asserts its output the system is put into a safe state. They are not latent faults.

Dual-Fault Injection

The purpose of dual-fault injection is to discover the dual-point faults of the safety mechanism. The fault pair consists of one fault that was dangerous detected in diagnostic coverage of residual faults. This first fault of the pair is known to cause the mission circuit to fail and was detected by the fault-free safety mechanism. The second fault of the dual-fault pair is a defect in the safety mechanism. If the fault pair is assessed as dangerous and undetected, the safety mechanism defect caused the safety mechanism to fail and is a dual-point fault.

The `diagnostic_coverage` command from the analysis of residual faults and from [Analyzing Defects in the Safety Mechanism](#), step 2. in [Running the Diagnostic Coverage of Latent Faults Tutorial](#), is used again in this section without modification.

The number of dual-fault pairs can be prohibitively large because the dangerous detected faults from diagnostic coverage of residual faults are multiplied with the safety mechanism faults. This dual-fault list can be generated using either of the following:

- `generate_dual_fault` command
- `gen_dual_fault.sh` script included in the Custom Fault installation

Both take the `.fdb` file and `.dangerous_detected.failure_margin` files from [Performing Diagnostic Coverage of Residual Faults](#), step 1. in [Running the Diagnostic Coverage of Latent Faults Tutorial](#), and the `.fdb` file from the [Analyzing Defects in the Safety Mechanism](#) step as input. Both also take an argument to specify whether all dangerous faults uncovered in the [Performing Diagnostic Coverage of Residual Faults](#) step or just a subset are used for creating the dual fault list.

The syntax to invoke the script is:

```
gen_dual_fault.sh path_and_prefix_to_perm-diag-cov_fdb path_and_prefix_to_analyze-defects_fdb fault_select
```

Table 40 `gen_dual_fault` Script `fault_select` Option

<code>fault_select</code> value	Description
0	Smallest margin

Table 40 *gen_dual_fault Script fault_select Option (Continued)*

fault_select value	Description
1	Largest margin
2	Both smallest and largest margins
3	All

For this tutorial, execute one of the following scripts to generate the dual-fault lists for the PrimeSim XA, FineSim, or PrimeSim tools, respectively.

```
gen_step3_xa_faults.sh
gen_step3_fs_faults.sh
gen_step3_ps_faults.sh
```

These scripts call the `gen_dual_fault.sh` utility one time for each value of the fault selected and produce four dual-fault list files.

Table 41 *Dual-Fault List Files (sim is one of: xa, fs, or ps)*

File Name	Number of Dual-Point Faults
step3.sim.small.dfl	162
step3.sim.large.dfl	162
step3.sim.both.dfl	324
step3.sim.all.dfl	~4,000

As shown in [Table 41](#), injecting all dual point faults becomes impractical for large circuits. A more practical approach is to use a subset of representative faults. You should apply engineering judgment to determine which faults from [Performing Diagnostic Coverage of Residual Faults](#), step 1. in [Running the Diagnostic Coverage of Latent Faults Tutorial](#), should be applied, which can be excluded.

In this tutorial the `both` option is used to apply the smallest and largest margin failures from [Performing Diagnostic Coverage of Residual Faults](#) step. This results in 324 fault simulations for [Dual-Fault Injection](#), step 3. in [Running the Diagnostic Coverage of Latent Faults Tutorial](#). Note that there might be slight differences among the PrimeSim XA, FineSim, and PrimeSim simulation results and a slight change in the number of dual-point faults.

Running Dual-Fault Injection

Now you are ready to run dual-fault injection with the PrimeSim XA, FineSim, or PrimeSim tool.

To use the PrimeSim XA tool as the fault simulator, specify the following script:

```
run-xa-mpfl-step3-both
```

To use the FineSim tool as the fault simulator, specify the following script:

```
run-fs-mpfl-step3-both
```

To use the PrimeSim tool as the fault simulator, specify the following script:

```
run-ps-mpfl-step3-both
```

Table 42 *List of Output Files for Dual-Fault Injection*

File Name	Description
<code>.diagnostic_report</code>	Provides a detailed accounting of the fault injection. The dangerous undetected faults for each failure mode identify the dual-point faults of the safety mechanism.
<code>.failure_mode_id.dual_point_failure</code>	There is a file for each failure mode defined in the <code>diagnostic_coverage</code> command. It lists all the dual-point faults where a safety mechanism failure was detected for the mission circuit failure mode.
<code>.forLDC4thstep.fdb</code>	Fault database version of the potential dual-point faults in the safety mechanism is used as an input in Diagnostic Coverage of Latent Faults , step 4. in Running the Diagnostic Coverage of Latent Faults Tutorial .
<code>.potential_latent_faults</code>	Lists the fault injection command for the list of unique safety mechanism dual-point faults detected in this step. These faults should be the union of the <code>.failuremode_id.dual_point_failure</code> files.

Diagnostic Coverage of Latent Faults

Now that the dual-point faults of the safety mechanism have been identified, diagnostic coverage of latent faults can be evaluated.

In this step the testbench and netlist are configured to operate the circuit in its self-test mode. During the self-test phase, the input to the safety mechanism is switched from the mission circuit to a high, medium, and low reference voltage. The high and low reference voltage should cause the safety mechanism to assert its output, alarm.

The following measure statements observe the safety mechanism output. They are defined in the `./tb_dcmpfl.sp` or `./tb_dcmpfl_ps.sp` file:

```
* Latent fault measures
.meas vmid_alarm    max v(alarm) from=2.1u to=2.9u $expect 0
.meas vhigh_alarm  max v(alarm) from=3.1u to=3.9u $expect 1
.meas vlow_alarm   max v(alarm) from=4.1u to=4.9u $expect 1

.opt xa_cmd="set_sim_stop -check meas -name vmid_alarm -max 0.1 -stop 0"
.opt xa_cmd="set_sim_stop -check meas -name vhigh_alarm -min '0.9*vdd'
-stop 0 -id latent_over_voltage"
.opt xa_cmd="set_sim_stop -check meas -name vlow_alarm -min '0.9*vdd'
-stop 0 -id latent_under_voltage"

.set_sim_stop check=meas name=vmid_alarm max=0.1 stop=0
.set_sim_stop check=meas name=vhigh_alarm min='0.9*vdd' stop=0
id=latent_over_voltage
.set_sim_stop check=meas name=vlow_alarm min='0.9*vdd' stop=0
id=latent_under_voltage
```

The measure statements are serving as the detection of the self-test mode. If the safety mechanism fails to assert alarm during the high or low input voltage, or does assert its output during the mid-voltage, the input the safety mechanism has failed.

The faults being injected are the known dual-point faults of the safety mechanism, and therefore it is expected that the safety mechanism fails. If the safety mechanism does not fail, then the dual-point fault that was injected was not detected and is therefore latent. The `set_sim_stop` command is written to detect the failures of the safety mechanism to indicate coverage of latent faults.

The dual-point faults determined in the [Dual-Fault Injection](#) step are most easily input to this step through the `fault_list_file` command in the configuration file.

For the PrimeSim XA tool:

```
"fault_list_file" :
"xa.mpfl.step3.both/xa.mpfl.step3.both.forLDC4thstep.fdb",
```

For the FineSim tool:

```
"fault_list_file" :
"fs.mpfl.step3.both/fs.mpfl.step3.both.forLDC4thstep.fdb",
```

For the PrimeSim tool:

```
"fault_list_file" :
"ps.mpfl.step3.both/ps.mpfl.step3.both.forLDC4thstep.fdb",
```

Note that the apply fault model section of the configuration file is ignored if a fault list file is provided.

To use the PrimeSim XA tool as the fault simulator, specify the following script:

```
run-xa-mpfl-step4-both
```

To use the FineSim tool as the fault simulator, specify the following script:

```
run-fs-mpfl-step4-both
```

To use the PrimeSim tool as the fault simulator, specify the following script:

```
run-ps-mpfl-step4-both
```

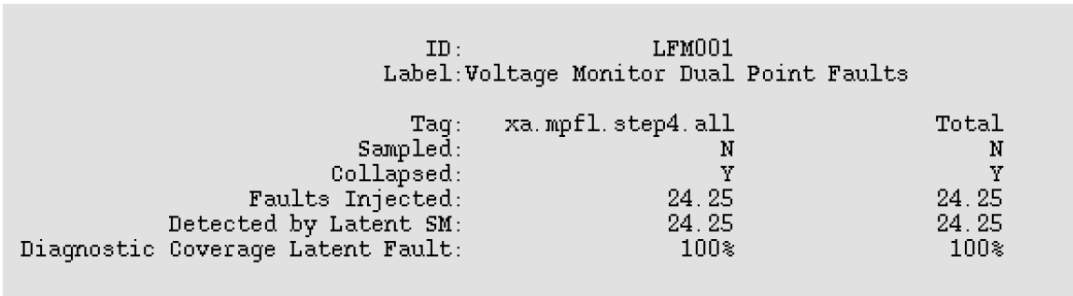
The fault campaign results are in the `xa.mpfl.step4.both`, `fs.mpfl.step4.both`, or `ps.mpfl.step4.both` directories. [Table 43](#) lists the output files from this step.

Table 43 List of Output Files for Diagnostic Coverage of Latent Faults

File Name	Description
<code>.latent_diagnostic_report</code>	The primary diagnostic coverage of latent faults output report. It lists the coverage of each failure mode and provides details for each fault.
<code>.failure_mode_id.latent_detected</code>	List of fault injection commands for each detected fault for the given failure mode.
<code>.failure_mode_id.latent_undetected</code>	List of fault injection commands for each undetected fault for the given failure mode.

The following is an example of the `.latent_diagnostic_report_file` file.

Figure 59 Diagnostic Coverage of Latent Faults Report



Direct SPFM Calculation

You can use the Custom Fault tool to measure the diagnostic coverage of residual faults (see [Measuring the Diagnostic Coverage of Residual Faults](#)). The diagnostic coverage

data, in addition to other inputs, are entered into a Failure Mode Effect and Diagnostic Analysis (FMEDA) to calculate the ISO 26262 Single Point Fault Metric (SPFM) for a chip.

In certain simpler cases, the SPFM can be calculated directly from the result of a fault campaign with the Custom Fault tool. This following sections describe those cases and the commands to use this feature.

- [SPFM Definition](#)
- [Limitations of Direct SPFM Calculation](#)
- [Command Specification](#)
- [SPFM Calculation](#)
- [SPFM Output](#)

SPFM Definition

As shown in [Single Point Fault Metric \(SPFM\)](#), the formula (see [Equation 1](#)) for calculating SPFM is defined in ISO 26262-5:2018(E) Annex C in equation C.7.

In this application of the Custom Fault tool, a safety mechanism is used in the circuit, therefore the analysis considers only residual faults and λ_{SPF} is assumed to be 0.

ISO 26262 also provides the formula (see [Equation 4](#)) for calculating λ_{RF} when the failure modes are known.

The Custom Fault tool can measure the failure mode distribution and safe faults, as well as the diagnostic coverage or residual faults. $F_{F_{Mi},PSVG}$ is assumed to be 1. This equation can be employed automatically by the tool.

The Failure in Time (FIT) of the design, λ , is calculated based on process parameters and circuit device counts and area. The Custom Fault tool assumes $\lambda = 1$, but as can be seen in the SPFM equation, the FIT cancels out of the equation if all the needed failure modes can be assessed within a single fault campaign.

Limitations of Direct SPFM Calculation

You can use the Custom Fault tool to calculate and report the ISO 26262 SPFM directly from a fault campaign. However, it is your responsibility to understand the following limitations of this feature and apply the data appropriately:

- You must set the fault campaign scope appropriately to ensure that the relevant portion of the circuit's FIT is assessed and that the FIT term cancels out of the SPFM equation for the circuit being analyzed.
- You must represent in the fault campaign and define with the `diagnostic_coverage` command all failure modes for the circuit identified in the FMEA/FMEDA.
- You must determine and provide to the tool via command the failure mode distribution using one of the following methods:
 - All failure modes are considered equally likely. For two failure modes, the distribution is 50% each; for three failure modes, the distribution is 33.33% each; and so on.
 - The failure mode distribution is measured from the actual fault campaign and the failure modes are mutually exclusive.

Command Specification

Use the `report_spfm` argument within the `diagnostic_coverage` command to enable direct calculation and reporting of SPFM.

The `fSAFE_method` and `failure_mode_distribution_method` arguments also affect the calculation of SPFM.

For more information, see [diagnostic_coverage \(for Residual Faults and Direct SPFM Calculation\)](#).

SPFM Calculation

The residual FIT for each failure mode is calculated as:

$$\text{Residual FIT} = 1 * D_{fm} * (1 - F_{SAFE}) * (1 - K_{rf})$$

The residual FITs of each failure mode are then summed.

The SPFM is calculated as:

$$\text{SPFM} = 1 - \text{sum}(\text{failure mode residual FIT}) * 100\%$$

SPFM Output

The SPFM output report is contained in the `.diagnostic_report` file.

The following is an example report:

```
SINGLE POINT FAULT METRIC REPORT

Failure Mode Distribution Calculation Method = 1
Fsafe Calculation Method = 1
Residual FIT = 1 * Dfm * (1-Fsafe) * (1-Krf)

Tag: nom
Failure Mode   Weight   Dfm   Fsafe   Krf   Residual FIT
FM001          5       0.5    0.2     0.4      0.24
FM002          3       0.5    0.2     0.666667 0.133333
Safe           2
Sum Residual FIT 0.373333
SPFM           62.6667
```

15

Using Custom Fault for Detectable Fault Analysis

This chapter describes how to use the Custom Fault tool to perform Detectable Fault Analysis.

- [Overview of Detectable Fault Analysis](#)
- [Invoking DFA](#)
- [DFA Output Results](#)
- [DFA Waveform Comparison Utility \(cfutil\)](#)
- [DFA Tutorial](#)
- [DFA Mixed-Signal Tutorial](#)

Overview of Detectable Fault Analysis

In a circuit, some defects may be undetectable for multiple reasons: dummy blocks or spare blocks are by definition undetectable; when several resistors are connected in series, shorting one of the resistors in the chain may be invisible from the observation point. The Custom Fault Detectable Fault Analysis (DFA) feature analyzes the circuit and reports the defects that are undetectable from the observation point you specified.

DFA is a methodology to gain insight to improve test coverage. It analyzes the circuit and identifies:

- Defects that do not have a path to the observation points
- Defects that do not propagate outside of the local block
- Defects belonging to digital gates and MOSCAP
- Spare circuitry

It also provides a local stimulus that sensitizes defects and provides grouping and clustering of similar defects.

Note:

The DFA feature is a voltage-based analysis. The automatically-generated Channel-connected block (CCB) level stimuli are voltage signals and the detectability conditions are also evaluated based on voltage signals.

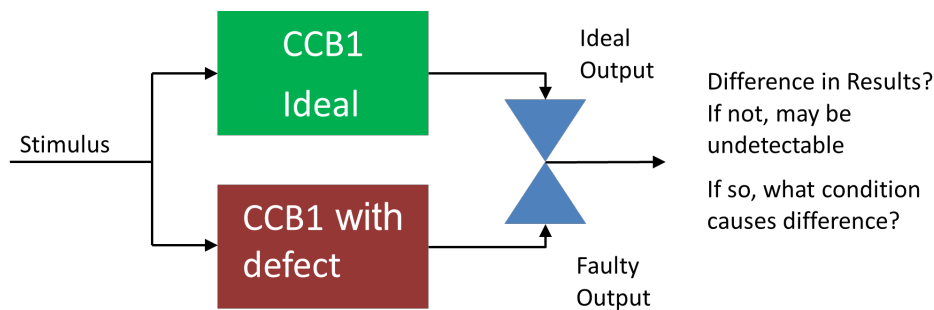
The PrimeSim simulator is not supported.

Invoking DFA

Invoke the Detectable Fault Analysis (DFA) feature using the `detectability` command in the Custom Fault configuration file. The command lists all the observation points, so specifying meaningful observation points is very important.

DFA is vector-independent and it drives each Channel-Connected Block (CCB) with its own stimuli. For each CCB, DFA runs one defect-free simulation and one simulation for each injected defect. DFA then compares the CCB output results between the defect-free and defect-injected simulations. If the results match, the defect is classified as potentially undetectable; if the results do not match, then the defect is classified as detectable (see [Figure 60](#)).

Figure 60 How DFA works



See [DFA Tutorial](#) for more information.

Recommended Use Flow

It is recommended to first run `detectability` with the following additional command:

```
"identify_only" : true,
```

This generates the list of undetectable faults in a file with the `.dfa_report` extension. More details are reported in the `dfa` directory under the tag directory.

You can then analyze the DFA results and make any changes you want in your stimuli and/or observation points. To run the fault campaign together with detectability, comment out the `"identify_only" : true`, command or set it to `false`.

Running static analysis

You can use DFA to run a static analysis to provide information using `detectability` with the following additional command:

```
"detectability":  
  
{ "static_only": true|false, "observation_nodes": [""] }
```

The default for `static_only` is `false` and simulations are executed. When it is set to `true`, a static analysis is executed and DFA reports the list of spare/dummy, list of MOSCAPs, list of digital gates, and any undetectable defects that can be found out by a static analysis.

Using DFA With Mixed-Signal Simulation

You can use DFA with mixed-signal simulation for both Verilog-Top and SPICE-Top flows.

The usage is the same as with analog-only simulation, however, the observation nodes must be analog as it is not possible to specify Verilog digital signals as observation nodes.

The following is an example of observation nodes for Verilog-top mixed-mode simulation:

```
"detectability": {  
  "observation_nodes": ["top.dut.cout", "top.dut.s_3", "top.dut.s_2",  
    "top.dut.s_1", "top.dut.s_0"]  
},
```

The tolerance can be set the same way as for analog-only simulation. When local tolerance is used, the `"name"` must be an analog signal name, as shown in the following example:

```
"detectability": {  
  "observation_nodes": ["top.dut.cout", "top.dut.s_3", "top.dut.s_2",  
    "top.dut.s_1", "top.dut.s_0"],  
  "tolerance": [  
    {"err": "3%"},  
    {  
      "name": "top.dut.x1.x1.*",  
      "err": "1%"  
    }  
  ]  
}
```

See [DFA Mixed-Signal Tutorial](#) for more information.

DFA Output Results

When the Detectable Fault Analysis (DFA) feature is running, you can follow the progress on the stdout/stderr (see [Figure 61](#)).

Figure 61 DFA progress

```
Deriving fault universe ...
Processing multi fault collapsing.
Processing fault collapsing.
IDENTIFY FAULTS=>Peak Virtual Memory: 324,992M,
Launching DFA

Statistics: fault: 3895, device-fault: 0, subckt-fault: 3895, loaded/total fault
: 3895/3895, loaded/total group: 3028/3028

CDPL: running: 28, pending: 2517, finished: 572
```

After analysis is complete, the DFA feature generates a `.dfa_report` file (see [Figure 62](#)).

Figure 62 Excerpt of the `.dfa_report` file

PrimeSim (TM) Custom Fault									
*DEFECT UNIVERSE SUMMARY									
Defects - unweighted	undetectable/total	DT	DF	UP	UN	US	SC	DG	MC
mos_mac_open_G	197/797	600	0	57	140	0	26	328	83
mos_mac_open_S	199/797	598	0	59	140	0	26	328	83
mos_mac_short_DS	145/714	569	0	20	125	0	9	328	0
mos_mac_short_GS	73/787	714	0	1	72	0	24	328	76
r_open	213/412	199	0	50	163	0	162	0	0
r_short	215/248	33	0	215	0	0	0	0	0
TOTAL	1042/3795	2533	0	402	640	0	247	1312	242

How to Read the `.dfa_report` File

As shown in [Figure 62](#), the `.dfa_report` file is organized into ten columns, as follows (from left-to-right):

1. List of all applied defect models
2. Number of undetectable defects on top of the total number of defects
3. DT (Detectable defects), excluding the defects with a failed simulation
4. DF (Detectable Failed): Defects that cause the simulation to fail
5. UP (Undetectable Potentially): Defects that are undetectable because the faulty output match the ideal output (as shown in [Figure 60](#))
6. UN (Undetectable No path): Defects that are undetectable because they have no path to the defined observation nodes

7. US (Undetectable Structurally): Defects that can be marked undetectable without any simulation, for example:
 - Short between the same net
 - Short of (more than 20) series devices (resistors/capacitors with the same value)
 - Open of (more than 20) parallel devices
8. SC (Spare Circuitry): Defects into blocks that have the word '*spare*' or '*dummy*' in their names (block name or net name). The keyword for spare circuitry can be changed using the "`spare_inst`" argument of the `detectability` command.
9. DG (Digital Gate): Defects in a transistor belonging to a digital gate. A digital gate is an inverter or dual-MOS (like NAND or NOR)
10. MC (MosCaps): Defects in MOSCAPs. A MOSCAP is a MOS device with drain and source connected to the same net.

Following the table, the details are reported in the same format as the usual Custom Fault `.report` file (see [Figure 63](#)).

Figure 63 Details of the `.dfa_report` file

Id	Defect	Detectable	Weight	Instance/Node(s)	Port Name(s)
A1.2	mos_nac_short_GS	UN	1.000000e+00	Xtop.Xlido,Mp0	Xtop.Xlido,vschmit, Xtop.Xlido,I9,net67
A11.12	mos_nac_short_GS	DT,DG	1.000000e+00	Xtop.Xlido,M1	vdd_ctrl, Xtop.Xlido,dis_ov_dynb_in
A81.82	mos_nac_short_GS	DT	1.000000e+00	Xtop.Xlido,M30	vss, Xtop.Xlido,passdev_off_in
A219.219	mos_nac_open_S	DT,SC,MC	1.000000e+00	Xtop.Xlido,MSPARE_CLMP	vss
A301.301	mos_nac_open_G	UP,MC	1.000000e+00	Xtop.Xlido,M31	Xtop.Xlido,Ibias,pbias
A371.371	mos_nac_open_G	UN,SC,DG	1.000000e+00	Xtop.IINVSPARE<0>,M0	Xtop.Xlido,tie_low_vsupply
A3019.3020	mos_nac_short_GS	UN,DG	1.000000e+00	Xtop.Xlido,sp1<0>,M0	vss, Xtop.Xlido,I0VUV_COMP,I0VUVCMP,net057
A3121.3124	mos_nac_short_GS	DT,SC,DG	1.000000e+00	Xtop.IINVSPARE<2>,M1	vssup, Xtop.Xlido,tie_low_vsupply
A3288.3288	r_open	UP	1.000000e+00	Xtop.Xlido,R3	Xtop.Xlido,net_0149

In the example report shown in [Figure 63](#), defects (identified by their IDs) are described as follows:

- Defect with ID A1.2 is UN: Potentially undetectable because it has no path to output
- Defect with ID A11.12 is DT,DG: Detectable and is injected into a Digital Gate
- Defect with ID A81.82 is DT: Detectable
- Defect with ID A219.219 is DT,SC,MC: Detectable and injected into a MOSCAP, and which is also a '*spare*' block because the instance name contains the string, SPARE:Ctop.Xlido.MSPARE_CLMP
- Defect with id A301.301 is UP,MC: Potentially undetectable and is injected into a MOSCAP

DFA generates a `.spare.list` in the json format that you can use. The spare list is generated automatically. Any instance with keyword "spare", "dummy", "dum" and "dmy" in the name is considered as spare. The listed hierarchical name stops at the level where the signal

word, for example, “spare”, “dummy”, appears. If you want to exclude these instances from fault injection and simulation, you need to copy and paste the content of the file under 'except_inst' in "apply_fault_model" in config.json.

DFA Waveform Comparison Utility (cfutil)

The Detectable Fault Analysis (DFA) `cfutil` utility generates and displays the waveforms of the CCB simulations. It compares the results of the defect-free simulation with the results of the defect's simulation. The syntax is as follows:

```
cfutil fdb_file_name 1st_defect_id 2nd_defect_id
```

Note:

Make sure the WaveView display tool is available in your shell `PATH` variable before running `cfutil`.

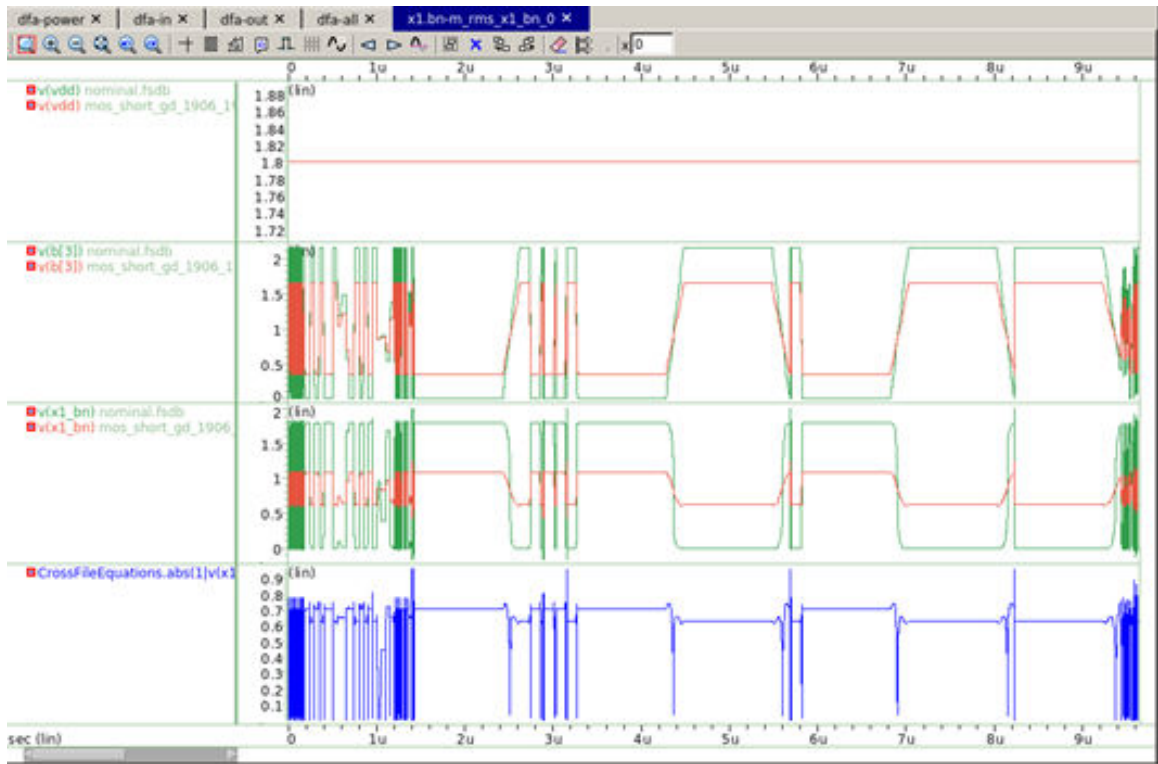
Example

The following example displays the waveforms for comparison of defect ID 1906 and defect ID 0 (the nominal defect-free simulation):

```
$> cfutil CF_RESULT/fault1.fdb 1906 0
```

As shown in [Figure 64](#), the WaveView tool opens and displays the overlapped waveforms for defect ID 1906 and nominal simulations. In this example, the waveforms do not match which means the defect is detectable.

Figure 64 Waveform comparison



DFA Tutorial

This tutorial shows how to use the Custom Fault tool to run Detectable Fault Analysis (DFA).

- [Running the Tutorial Using the PrimeSim XA Simulator](#)
- [Electrical Collapsing](#)
- [Improving Coverage](#)

Before starting the tutorial, make sure that all tools are installed correctly. Be aware the WaveView waveform display must be available to have access to the [DFA Waveform Comparison Utility \(cfutil\)](#).

[Table 44](#) lists the files needed for this tutorial.

Table 44 DFA Tutorial Files

File	Description
adder.vec	Vector file containing both input and output vectors

Table 44 *DFA Tutorial Files (Continued)*

File	Description
<code>adder_newstim.vec</code>	Vector file with longer vectors for better coverage
<code>top_xa.spi</code>	Top-level netlist of a 4-bit adder for the PrimeSim XA tool.
<code>top_xa_newstim.spi</code>	Top-level netlist of a 4-bit adder with a longer transient time to cover longer vectors
<code>xhost/xhost_sh_launch</code>	Host file for distributed processing
<code>config_xa.json</code>	Custom Fault configuration file for the PrimeSim XA simulator
<code>config_xa_s4.json</code>	Custom Fault configuration file for the PrimeSim XA simulator with one extra observation node
<code>config_elect_collapse.json</code>	Custom Fault configuration file to apply electrical collapsing
<code>run_cf_xa</code>	Run script to run DFA with the PrimeSim XA simulator
<code>run_cf_s4_xa</code>	Run script to run DFA with the PrimeSim XA simulator and one extra observation node
<code>run_cf_ec_xa</code>	Run script to run DFA and electrical collapsing with the PrimeSim XA simulator
<code>run_cf_xa_newstim</code>	Run script to run DFA with the PrimeSim XA simulator and new stimuli to improve the coverage
<code>README</code>	README file with some information on the tutorial
<code>cmd_file</code>	PrimeSim XA configuration file using the <code>adder.vec</code> vector file
<code>cmd_file_newstim</code>	PrimeSim XA configuration file using the <code>adder_newstim.vec</code> vector file

Edit the `xhost/xhost_sh_launch` file and modify it to use your own distributed processing.

Running the Tutorial Using the PrimeSim XA Simulator

The DFA flow is invoked using the `detectability` command. You must specify the observation nodes in the `observation_nodes` argument. The observation nodes are important to detect whether the defect is propagated until the observation point.

Excerpt of `config_xa.json`:

```
"detectability": {
  "observation_nodes": ["cout", "s[3]", "s[2]", "s[1]", "s[0]"]
},
```

DFA is run before the fault campaign starts, and the fault campaign is run after DFA has completed. To run the fault campaign and generate the DFA report and the coverage report, use the `run_cf_xa` script.

DFA Report

The DFA report is generated in the `RESULT` directory: `RESULT/xa_defect.dfa_report` file, as follows:

```
*DEFECT UNIVERSE SUMMARY
Defects - unweighted undetectable/total    DT  DF  UP  UN  US   SC   DG  MC
      mos_open              44/824      780   0   0  44   0  164  440   4
      mos_short             45/1083    1038   0   0  45   0  216  560   3
      TOTAL                  89/1907    1818   0   0  89   0  380 1000   7
```

In the `xa_defect.dfa_report` file, the defects are classified with the following categories:

- DT: Detectable
- DF: Detectable - simulation Failed
- UP: Undetectable Potentially
- UN: Undetectable potentially - No path to output
- US: Undetectable Structurally
- SC: Spare Circuitry
- DG: Digital Gate MOS
- MC: MosCap

In this tutorial, there are no defects in the DF (simulation failed), UP (Undetectable Potentially), or US (Undetectable Structurally) categories. There are defects in all other categories. As expected, there are defects that are detectable (it is the biggest number) because most defects are detectable. There are 89 defects are classified UN (No path to output), 380 are classified as Spare Circuitry (SC), 1000 defects are in digital gates (DG), and seven defects belong to a MOSCAP device (MC).

Examples of UN Defects

89 defects are classified UN (No path to output). You can check all of the UN defects and see that they are all injected either in `mcap` or in `xspare` instances. There is no path to output for these defects because the `s[4]` signal is not part of the observation nodes.

As shown in the following example, the `mos_open` defect on port `s[4]` of instance `mcap` is undetectable because there is no path for this defect to be propagated to the observation nodes (reminder: the observation nodes are `["cout", "s[3]", "s[2]", "s[1]", "s[0]"]`).

Id Name(s)	Defect	Detectable	Weight	Instance/Node(s)	Port(s)	Net
A2.2	<code>mos_open</code>	UN,MC	1.000000e+00	<code>mcap</code>	<code>g s[4]</code>	

As a test, you can modify the `config_xa.json` file and add `s[4]` in the list of observation nodes:

```
"detectability": {
  "observation_nodes": ["cout", "s[3]", "s[2]", "s[1]", "s[0]", "s[4]"]
},
```

With `s[4]` added to the list, there is no more UN defect in the `.dfa_report`. Launch the `run_cf_s4_xa` script to run DFA with the extra observation point `s[4]`. The script is only running DFA and not defect simulations; the results are in `RESULT/xa_s4.dfa_report`.

Examples of SC Defects

380 defects are classified as Spare Circuitry (SC). You can verify that they all belong to the `xspare` instance. The detection of Spare Circuitry is based on the instance or subcircuit name, so all defects injected into an instance or a subcircuit containing either the word 'dummy' or the word 'spare' are marked SC.

As shown in the following example, the `mos_open` defect on port `s[4]` of the `xspare.x2.mn1` instance is marked SC because the defect is injected in a device that has the word 'spare' in its name. You can note that the defect is also marked UN because there is no path to output for this defect.

Id Name(s)	Defect	Detectable	Weight	Instance/Node(s)	Port(s)	Net
A57.57	<code>mos_open</code>	UN,SC	1.000000e+00	<code>xspare.x2.mn1</code>	<code>d s[4]</code>	

Examples of DG Defects

1000 defects are in digital gates. The Custom Fault tool can recognize digital gates (in the current version only inverters and dual CMOS are recognized). Any defect belonging to such a digital structure is marked DG.

As shown in the following example, the `mos_open` defect on port `x1.a` of `x1.x1.x2.mn1` is marked DG because `x1.x1.x2` is an inverter.

Id Name(s)	Defect	Detectable	Weight	Instance/Node(s)	Port(s)	Net
A190.190	<code>mos_open</code>	DT,DG	1.000000e+00	<code>x1.x1.x2.mn1</code>	<code>g x1.a</code>	

You can verify it by looking at the `top_xa.spi` netlist, or using PrimeSim XA interactive mode, as follows:

```
XA> iprint_elem x1.x1.x2

Elem=x1.x1.x2 Type=suckt subckt=inv
PORT 1 port=in - x1.a (42) v=1.18576e-06
PORT 2 port=out - x1.x1.n3 (46) v=1.8
```

Examples of MC Defects

Seven defects belong to a MOSCAP device. As shown in the following example, the `mos_short` defect between net 0 and net `s[4]` of the `mcap` device is marked MC.

Id Name(s)	Defect	Detectable	Weight	Instance/Node(s)	Port(s)	Net
A926.926 s[4]	mos_short	UN,MC	1.000000e+00	mcap	d, g	0,

The MOSCAP `mcap` device can be seen in the `top_xa.spi` netlist as follows:

```
Mcap gnd s[4] gnd gnd n w=5.00u l=1.00u
```

Coverage Report

In the `RESULT/xa_defect.report`, the coverage is reported including all defect simulations as usual, where the coverage is 47.98%:

```
*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
Type                                xa_defect

    mos_open                        327/824
    mos_short                       588/1083
    TOTAL                          915/1907

Weighted test coverage:
xa_defect                          47.98%
```

The coverage is also reported excluding the undetectable defects, which increases the coverage to 50.33%:

```
*DETECTABLE COVERAGE SUMMARY (exclude undetectable)

Detectable defects covered - weighted (covered/simulated):
Type                                xa_defect

    mos_open                        327/780
    mos_short                       588/1038
    TOTAL                          915/1818

Detectable weighted test coverage:
xa_defect                          50.33%
```

The coverage report also excludes the undetectable defects plus the 'spare' circuitry, which increases the coverage to 59.06%:

```
*DETECTABLE COVERAGE SUMMARY (exclude undetectable, spare)
```

```
Detectable defects covered - weighted (covered/simulated):
```

Type	xa_defect
mos_open	323/656
mos_short	572/864
TOTAL	895/1520

```
Detectable weighted test coverage:
```

xa_defect	59.06%
-----------	--------

Electrical Collapsing

Electrical collapsing helps to improve the total runtime. The tool collapses the defects that are electrically identical or similar and runs only one simulation for such defects.

Use the `run_cf_ec_xa` script to run DFA together with electrical collapsing. You can compare the `RESULT/xa_defect.report` (run without electrical collapsing) and the `RESULT/xa_ec_defect.report` (run with electrical collapsing). The coverage is identical however the number of simulations run is not the same:

- Without electrical collapsing, 1192 simulations are run.
- With electrical collapsing, 1173 simulations are run.

When electrical collapsing is run, the `.dfa_report` file is generated and it includes the Electrical Collapse Summary, as follows:

```
*DETAILED ELECTRICAL COLLAPSE SUMMARY
```

Before	1192
After	1173
Reduced	19

Improving Coverage

You can improve coverage using one of the following methods:

- [Using New Stimuli](#)
- [Using the DFA Waveform Comparison Utility](#)

Using New Stimuli

A lower-than-expected coverage can be caused by stimuli that do not exercise the totality of the circuit, so you can change the stimuli to activate more devices in the circuit.

Invoke the Custom Fault tool with the new stimuli (new netlist), and run the `run_cf_xa_newstim` script to verify the coverage is improved when more devices are activated.

```
Defects covered - weighted (covered/simulated):
      Type                      newstim
mos_open                      368/824
mos_short                     640/1083
TOTAL                         1008/1907
```

```
Weighted test coverage:
      newstim                   52.86%
```

```
TOTAL Coverage:
      Exhaustive                52.86%
```

Total weight of defect universe: 1907.00

*DETECTABLE COVERAGE SUMMARY (exclude undetectable)

```
Detectable defects covered - weighted (covered/simulated):
      Type                      newstim
mos_open                      368/780
mos_short                     640/1038
TOTAL                         1008/1818
```

```
Detectable weighted test coverage:
      newstim                   55.45%
```

```
Detectable TOTAL Coverage:
      Exhaustive                55.45%
```

Total weight of detectable defect universe: 1818.00

*DETECTABLE COVERAGE SUMMARY (exclude undetectable, spare)

```
Detectable defects covered - weighted (covered/simulated):
      Type                      newstim
mos_open                      364/656
mos_short                     621/864
TOTAL                         985/1520
```

```
Detectable weighted test coverage:
      newstim                   65.03%
```

```
Detectable TOTAL Coverage:
      Exhaustive                65.03%
```

Using the DFA Waveform Comparison Utility

Even with the improved vector file, some defects are marked detectable but are not covered by the Custom Fault tool. This is the case of the defect with ID 1906. Open

RESULT/newstim.report and you can see this `mos_short` defect is marked detectable (DT) and is not covered (NC):

```
A1906.1906 mos_short DT,DG NC fault_mos_short_1906_1906_1906/mos_short_1906.*  
2.000000e+00 x1.x3.mn1 g,d b[3],x1.bn
```

Run the [DFA Waveform Comparison Utility \(cfutil\)](#) to compare the defect ID 1906 with a nominal defect-free simulation (ID 0):

```
$> cfutil RESULT/newstim.fdb 1906 0
```

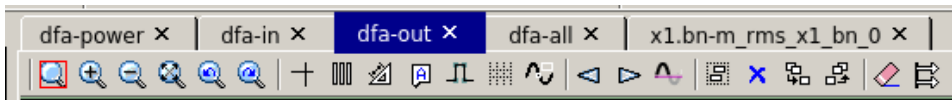
The waveform display opens with the waveforms of interest, that is the nominal waveform, `nominal.fsdb`, and the defect waveform, `mos_short_gd_1906.fsdb`.

You can see two sessions of the WaveView tool open: one shows the DFA results (all window names starting with **dfa-**) and the other one shows the Custom Fault results (all window names starting with **cf-**).

Multiple windows appear as shown in [Figure 65](#):

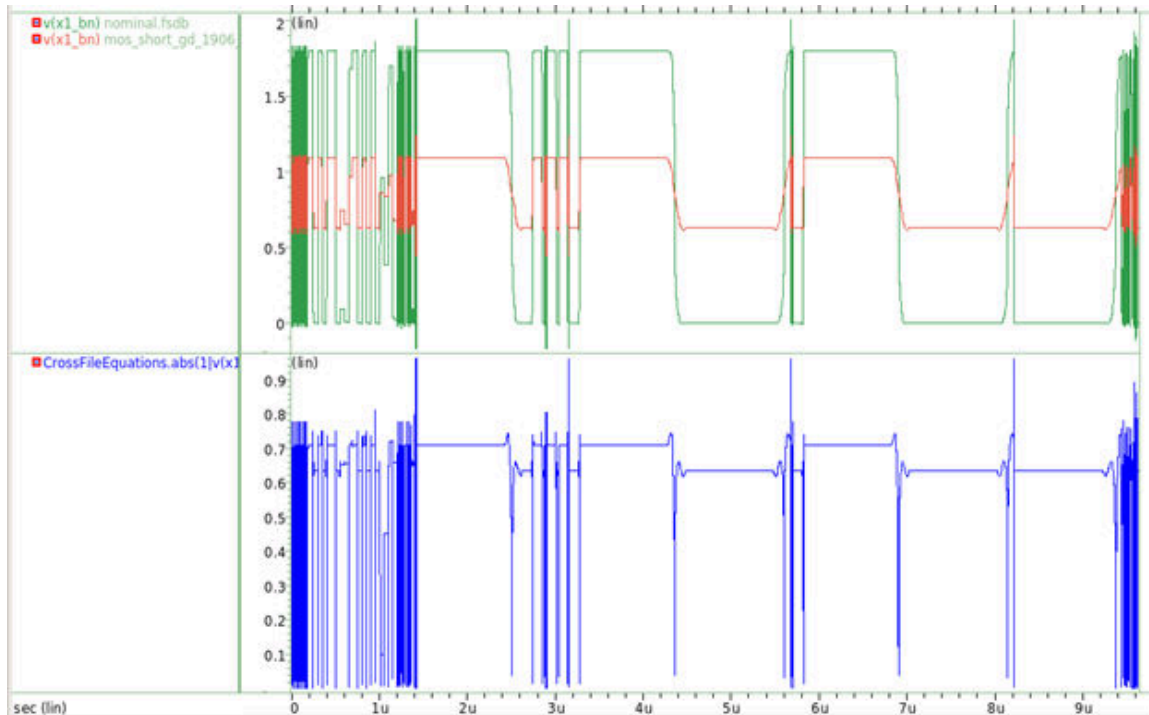
- The **dfa-power** window displays the power signals (`vdd`).
- The **dfa-in** window displays the input signal of the CCB (`b[3]`).
- The **dfa-out** window displays the output signal of the CCB (`x1_bn`).
- The **dfa-all** window displays all signals.
- The last window displays the signals whose results do not match the fault-free results. There might be more than one such window. In the present case, there is one signal reported: `x1.bn`. The syntax of the window's header is: *signal_name-meas_name*.

Figure 65 WaveView windows



Look at the signals in the **dfa-out** window as shown in Figure 66:

Figure 66 DFA waveforms (expected nominal signal is in green)



The output signal of the CCB is `x1_bn`, and the two waveforms of this signal—the nominal result in green and the defect result in brown—are overlapped. Below the signal waveforms, the blue signal that is displayed is the difference between the nominal and the defect waveforms.

It is clear the defect is detectable, the `x1_bn` signal has an amplitude which is much less than the expected amplitude, so why is it not detected by the Custom Fault tool?

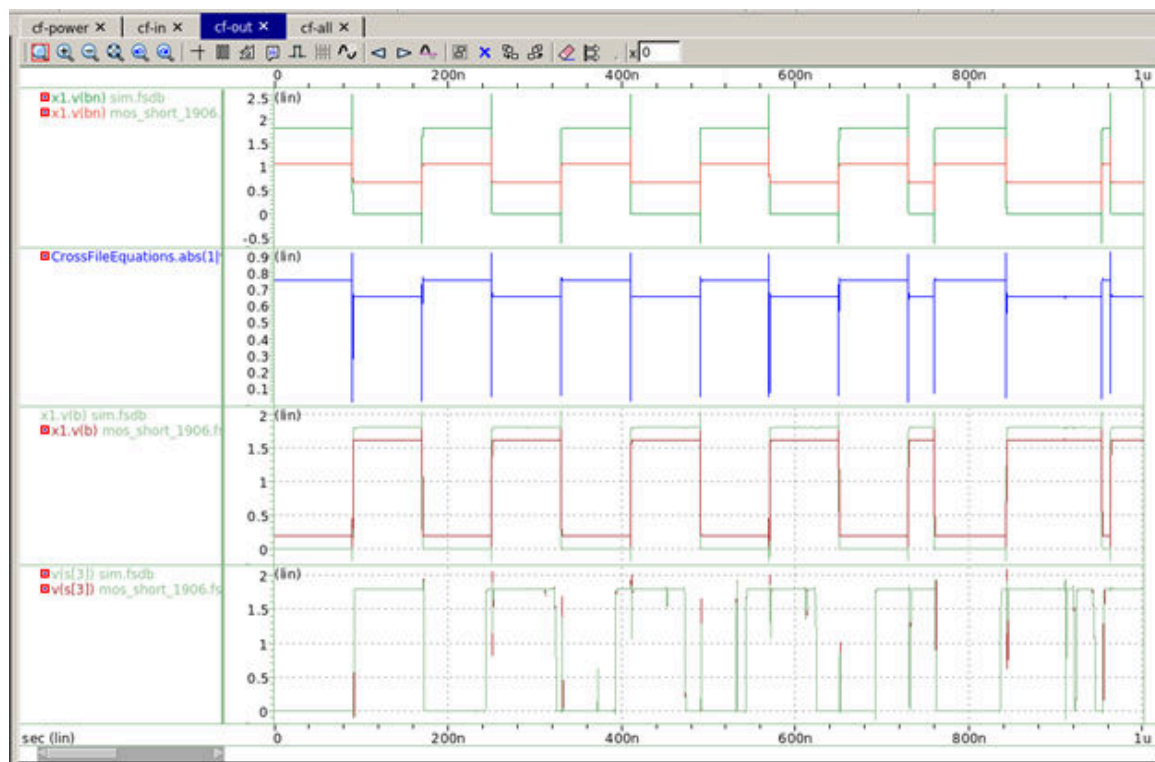
The defect is a `mos_short` between `b[3]` and `x1.bn`. Look at the following netlist excerpt and you can see that signal `x1.bn` is the output of the inverter `x1.x3` and the input of the inverter `x1.x4`.

```
.subckt addr ain bin cout sum cin
x1 b a p xor2
x2 cin p sum xor2
x3 bin bn inv20
x4 bn b inv20
x5 ain an inv20
x6 an a inv20
...
```


Even if the signal `x1.bn` is degraded by the `mos_short` defect, it is good enough to have the `x1.x4` inverter recover it, `x1.b` is correct, and the final output signal of the adder is not perturbed by the `b[3]-x1.bn` short defect.

To confirm this assumption, look at the second waveform display opened by `cfutil`. You can see the nominal result (`sim.fsdb`) and the Custom Fault result are overlapped for the `x1.bn` signal, and you can see the signals do not match. In that waveform display, you can open the `x1.b` and the `s[3]` signals for both the nominal and Custom Fault runs (Figure 67). You can see that `x1.b` recovers and the final signal, `s[3]`, which is the observation point, can match the nominal value.

Figure 67 Waveforms from the Custom Fault simulation of defect 1906



In conclusion, the defect is detectable locally at the `x1.x3` inverter output but not detectable at the output of the circuit. The next step would be to figure out how the short defect can be detected, should more measurements be added, should the stimuli be modified, and so on.

To ensure the signals of interest are probed, you can use the probe file generated by DFA. In the `RESULT/newstim.1/dfa` directory, you can find a probe file, `dfa.probe` (see the following excerpt of the file). Include this file in your netlist before running the fault campaign.

```
* probe nodes

.probe v(0)
.probe v(vdd)
.probe v(s[4])
.probe v(a[3])
.probe v(b[3])
.probe v(cout)
.probe v(s[3])
.probe v(n1)
.probe v(a[2])
.probe v(b[2])
.probe v(s[2])
.probe v(n2)
...
```

DFA Mixed-Signal Tutorial

This tutorial shows how to use the Custom Fault tool to run Detectable Fault Analysis (DFA) with VCS PrimeSim AMS mixed-signal simulation.

- [Running the Tutorial Using the VCS PrimeSim AMS Simulator](#)
- [Electrical Collapsing](#)
- [Using the DFA Waveform Comparison Utility](#)

Before starting the tutorial, make sure that all tools, PrimeSim Custom Fault, PrimeSim XA, VCS, Verdi, and WaveView, are installed correctly. Be aware that the WaveView waveform display must be available to have access to the [DFA Waveform Comparison Utility \(cfutil\)](#).

Table 1 lists the files needed for this tutorial.

Table 45 *DFA Mixed-Signal Tutorial Files*

File	Description
adder.v	Verilog description of the adder
addr4.spi	SPICE description of the adder
testbench.v	Verilog description of the testbench
xhost/xhost_sh_launch	Host file for distributed processing
config.json	Custom Fault configuration file for the VCS PrimeSim AMS simulator

Table 45 DFA Mixed-Signal Tutorial Files (Continued)

File	Description
config_s4.json	Custom Fault configuration file for the VCS PrimeSim AMS simulator with an extra observation point
config_elect_collapse.json	Custom Fault configuration file to apply electrical collapsing
vcsAD_xa.init	Mixed-signal control file
golden_waveform_xa.fsdb	Digital waveform files containing the golden results for waveform comparison
signal_list	List of signals to compare with the golden waveform file
run_dfa	Run script to run DFA with the VCS PrimeSim AMS simulator
runVcsAms	Run script to run the circuit with VCS PrimeSim AMS simulator
run_dfa_s4	Run script to run DFA with VCS PrimeSim AMS simulator and an extra observation point
run_cf_ec	Run script to run DFA with electrical collapsing

Edit the `xhost/xhost_sh_launch` file and modify it to use your own distributed processing.

Running the Tutorial Using the VCS PrimeSim AMS Simulator

The DFA flow is invoked using the `detectability` command. You must specify the observation nodes in the `observation_nodes` argument. The observation nodes are important to detect whether the defect is propagated until the observation point.

Note:

In the mixed-signal flow, the observation nodes must be analog. It is not possible to specify Verilog digital signals as observation nodes.

Excerpt of `config.json`:

```
"detectability": {
  "observation_nodes": ["top.dut.cout", "top.dut.s_3", "top.dut.s_2",
    "top.dut.s_1", "top.dut.s_0"]
},
```

DFA is run before the fault campaign starts, and the fault campaign is run after DFA has completed. To run the fault campaign and generate the DFA report and the coverage report, use the `run_dfa` script.

DFA Report

The DFA report is generated in the `RESULT` directory: `RESULT/dfa.dfa_report` file, as follows:

```
*DEFECT UNIVERSE SUMMARY
Defects - unweighted undetectable/total    DT  DF  UP  UN  US  SC  DG  MC
cap_open                20/20             0   0   0   0  20   0   0   0
cap_short              20/20             0   0   0   0  20   0   0   0
mos_open_dgs          33/618           585   0   0  33   0  123  330   3
mos_short_dgs         24/617           593   0   0  24   0  123  330   2
res_open              25/25             0   0   0  25   0   0   0   0
res_short             25/25             0   0   0   3  22   0   0   0

TOTAL                  147/1325       1178   0   0  105  42  246  660   5
```

In the `xa_defect.dfa_report` file, the defects are classified with the following categories:

- DT: Detectable
- DF: Detectable - simulation Failed
- UP: Undetectable Potentially
- UN: Undetectable potentially - No path to output
- US: Undetectable Structurally
- SC: Spare Circuitry
- DG: Digital Gate MOS
- MC: MosCap

In this tutorial, there are no defects in the DF (simulation failed) or UP (Undetectable Potentially) categories. There are defects in all other categories. As expected, there are defects that are detectable (it is the biggest number) because most defects are detectable. In this case, there are 105 defects that are classified UN (No path to output), 42 are classified US (Undetectable Structurally), 246 are classified as Spare Circuitry (SC), 660 defects are in digital gates (DG), and five defects belong to a MOSCAP device (MC).

Examples of UN Defects

105 defects are classified UN (No path to output). You can check all of the UN defects and see that they are all injected either in `mcap`, in `xspare`, in `xrchain` or `xpcap` instances. There is no path to output for the defects in `mcap` and in `xspare` because the `s_4` signal is not part of the observation nodes.

As shown in the following example, the `mos_open` defect on port `s_4` of instance `top.dut.mcap` is undetectable because there is no path for this defect to be propagated to the observation nodes (reminder: the observation nodes are `["top.dut.cout", "top.dut.s_3", "top.dut.s_2", "top.dut.s_1", "top.dut.s_0"]`).

Id	Defect	Detectable	Weight	Instance/Node(s)	Port(s)	Net
A27.27	mos_open_dgs	UN,MC	1.000000e+00	top.dut.mcap	g	top.dut.s_4

As a test, you can modify the `config.json` file and add `s_4` in the list of observation nodes:

```
"detectability": {
  "observation_nodes": ["top.dut.cout", "top.dut.s_3", "top.dut.s_2",
    "top.dut.s_1", "top.dut.s_0", "top.dut.s_4"]
},
```

With `s_4` added to the list, there are fewer UN defects in the `.dfa_report`. Launch the `run_dfa_s4` script to run DFA with the extra observation point, `s_4`. The script is only running DFA and is not running defect simulations (note this tutorial uses `"identify_only": true`, in the `config_s4.json` file); the results are in `RESULT/dfa_s4.dfa_report`.

Examples of US Defects

42 defects are classified as US (Undetectable Structurally). They are all either in the `xrchain` or `xpcap` instances. When N (more than 20) identical resistors are in series, or N (more than 20) identical capacitors are in parallel, a short on one of them may not be detectable. In that case, the open or short defect is undetectable because of the circuit's structure.

Examples of SC Defects

246 defects are classified as Spare Circuitry (SC). You can verify that they all belong to the `xspare` instance. The detection of Spare Circuitry is based on the instance or subcircuit name, so all defects injected into an instance or a subcircuit containing either the word `'dummy'` or the word `'spare'` are marked SC.

As shown in the follow example, the `mos_open` defect on port `top.dut.s_4` of the `top.dut.xspare.x2.mn1` instance is marked SC because the defect is injected in a device that has the word `'spare'` in its name. You can note that the defect is also marked UN because there is no path to output for this defect.

Id	Defect	Detectable	Weight	Instance/Node(s)	Port(s)
A560.560	mos_open_dgs	UN,SC	1.000000e+00	top.dut.xspare.x2.mn1	d
				top.dut.s_4	

Examples of DG Defects

660 defects are in digital gates. The Custom Fault tool can recognize digital gates (in the current version only inverters and dual CMOS are recognized). Any defect belonging to such a digital structure is marked DG.

As shown in the follow example, the `mos_open` defect on port `top.dut.x1.a` of `top.dut.x1.x1.x2.mn1` is marked DG because `top.dut.x1.x1.x2` is an inverter.

Id	Defect	Detectable	Weight	Instance/Node(s)	Port(s)
Net Name(s)					
A45.45	mos_open_dgs	DT,DG	1.000000e+00	top.dut.x1.x1.x2.mn1	g
top.dut.x1.a					

You can verify it by looking at the netlist `addr4.spi`, or using the VCS PrimeSim AMS interactive mode, as follows:

```
$> ./simv -ucli
Ucli% ace iprint_elem top.dut.x1.x1.x2
Elem=top.dut.x1.x1.x2 Type=suckt subckt=inv
PORT 1 port=in - x1.a (42) v=1.18576e-06
PORT 2 port=out - x1.x1.n3 (46) v=1.8
```

Examples of MC Defects

Five defects belong to a MOSCAP device. As shown in the following example, the `mos_short` defect between net 0 and net `top.dut.s_4` of the `top.dut.mcap` device is marked MC.

Id	Defect	Detectable	Weight	Instance/Node(s)	Port(s)	Net
Name(s)						
A768.768	mos_short_dgs	UN,MC	1.000000e+00	top.dut.mcap	d, g	0,
top.dut.s_4						

The MOSCAP `mcap` device can be seen in the `addr4.spi` netlist as follows:

```
Mcap gnd s_4 gnd gnd n w=5.00u l=1.00u
```

Coverage Report

In the `RESULT/dfa.report`, the coverage is reported including all defect simulations as usual, where the coverage is 51.77%:

```
*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
      Type                                dfa
      cap_open                            0/20
      cap_short                           0/20
      mos_open_dgs                        297/618
      mos_short_dgs                      389/617
      res_open                            0/25
      res_short                           0/25
      TOTAL                              686/1325

Weighted test coverage:
      dfa                                51.77%
```

The coverage is also reported excluding the undetectable defects, which increases the coverage to 58.23%:

```
*DETECTABLE COVERAGE SUMMARY (exclude undetectable)
```

```

Detectable defects covered - weighted (covered/simulated):
      Type                                dfa
      cap_open                            0/0
      cap_short                           0/0
      mos_open_dgs                         297/585
      mos_short_dgs                       389/593
      res_open                             0/0
      res_short                            0/0
      TOTAL                              686/1178

Detectable weighted test coverage:
      dfa                                58.23%

```

The coverage report also excludes the undetectable defects plus the 'spare' circuitry, which increases the coverage to 64.74%:

```

*DETECTABLE COVERAGE SUMMARY (exclude undetectable, spare)

Detectable defects covered - weighted (covered/simulated):
      Type                                dfa
      cap_open                            0/0
      cap_short                           0/0
      mos_open_dgs                       280/492
      mos_short_dgs                     357/492
      res_open                             0/0
      res_short                            0/0
      TOTAL                              637/984

Detectable weighted test coverage:
      dfa                                64.74%

```

Electrical Collapsing

Electrical collapsing helps to improve the total runtime. The tool collapses the defects that are electrically identical or similar and runs only one simulation for such defects.

Use the `run_cf_ec` script to run DFA together with electrical collapsing. You can compare the `RESULT/dfa.report` (run without electrical collapsing) and the `RESULT/ec_defect.report` (run with electrical collapsing). The coverage is identical however the number of simulations run is not the same:

- Without electrical collapsing, 1024 simulations are run.
- With electrical collapsing, 964 simulations are run.

When electrical collapsing is run, the `RESULT/ec_defect.dfa_report` file is generated, and it includes the Electrical Collapse Summary, as follows:

```

*DETAILED ELECTRICAL COLLAPSE SUMMARY
Before          1024
After           964
Reduced         60

```

Using the DFA Waveform Comparison Utility

Some defects are marked detectable but are not covered by the Custom Fault tool. This is the case of the defect with ID 903. Open `RESULT/ec_defect.report` and you can see this `mos_short` defect is marked detectable (DT) and is not covered (NC):

```
A762.762 mos_short_dgs DT,DG NC fault_mos_short_762_762_762/mos_short_762.*
3.000000e+00 top.dut.x1.x8.mp1 s,d vdd,top.dut.x1.x8.n1
```

Run the [DFA Waveform Comparison Utility \(cfutil\)](#), to compare the defect ID 903 with a nominal defect-free simulation (ID 0):

```
$> cfutil RESULT/ec_defect.fdb 762 0
```

The waveform display opens with the waveforms of interest, that is the nominal waveform, `nominal.fsdb`, and the defect waveform, `mos_short_ds_762.fsdb`.

You can see two sessions of WaveView open: one shows the DFA results (all tab names start with **dfa-**) and the other one shows the Custom Fault results (all tab names start with **cf-**).

The following tabs open (as shown in [Figure 68](#)):

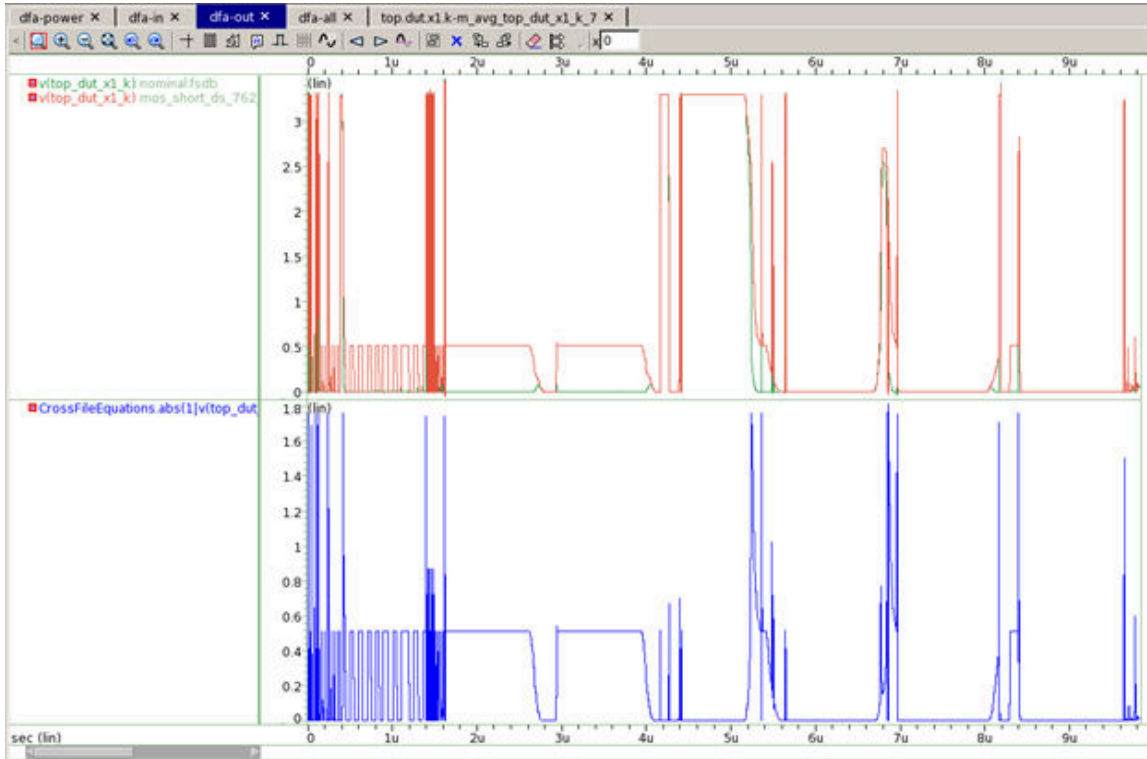
- The **dfa-power** tab displays the power signals (`vdd`).
- The **dfa-in** tab displays the input signals of the CCB (`top_dut_x1_b` and `top_dut_x1_a`).
- The **dfa-out** tab displays the output signal of the CCB (`top_dut_x1_k`).
- The **dfa-all** tab displays all signals.
- The last tab displays the signals whose results do not match the fault-free results. There might be more than one such tab. In the present case, there is one signal reported: `top_dut_x1_k`. The syntax of the tab's header is *signal_name-meas_name*.

Figure 68 WaveView tabs



Look at the signals in the **dfa-out** tab as shown in [Figure 69](#):

Figure 69 DFA waveforms (expected nominal signal is in green)



The output signal of the CCB is `top_dut_x1_k`, the two waveforms of this signal—the nominal result in green color and the defect result in brown color—are overlapped. Below the signal waveforms, the blue signal that is displayed is the difference between the nominal and the defect waveforms.

It is clear the defect is detectable; the `top_dut_x1_k` signal with the injected defect does not match the nominal defect-free signal, so why is it not detected by the Custom Fault tool?

The defect is a `mos_short` between `vdd` and `top.dut.x1.x8.n1`. Look at the following netlist excerpt and you can see the short defect is shorting source and drain on the `mp1` MOS device, forcing `n1` to be pulled up to `vdd`.

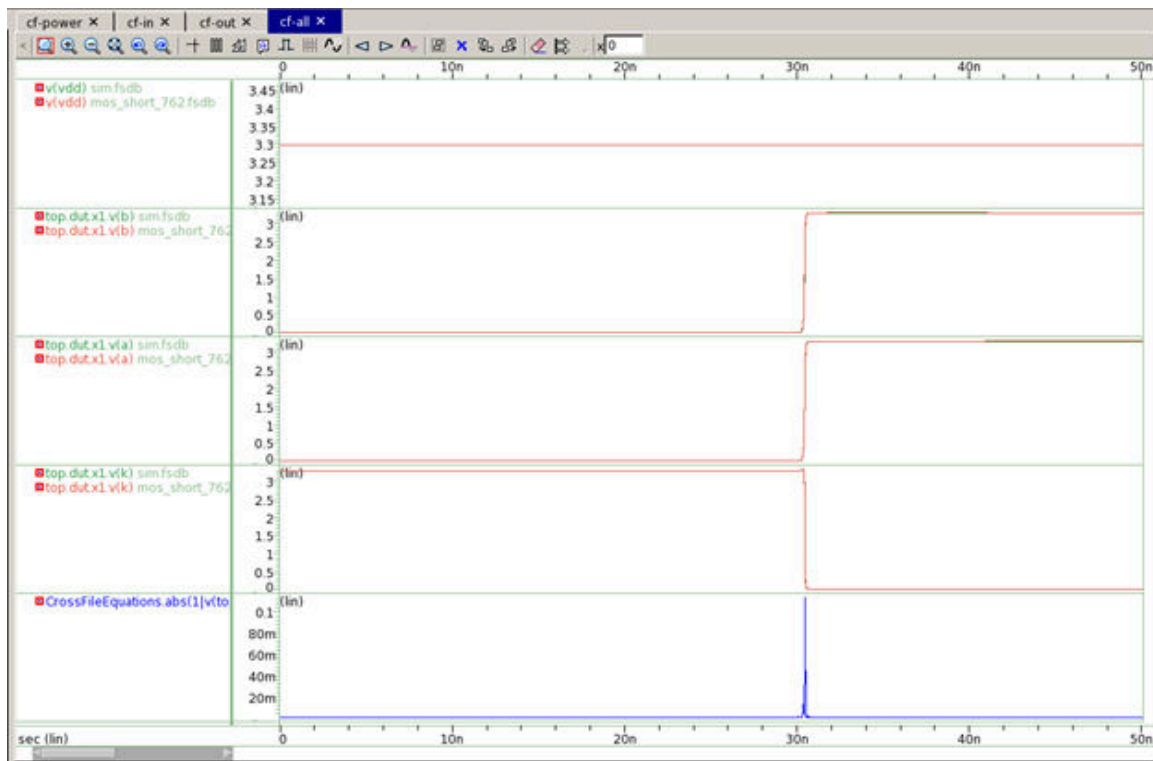
```
.subckt nor2 in1 in2 out
mp1 n1 in1 vdd vdd p w=10.00u l=0.35u
mp2 out in2 n1 vdd p w=10.00u l=0.35u
mn2 out in2 gnd gnd n w=5.00u l=0.35u
mn1 out in1 gnd gnd n w=5.00u l=0.35u
.ends

.subckt addr ain bin cout sum cin
```

```
x1 b a p xor2
x2 cin p sum xor2
x3 bin bn inv20
x4 bn b inv20
x5 ain an inv20
x6 an a inv20
x7 n1 n2 inv
x8 b a k nor2
x9 p k n1 nor2
mn2 cout k gnd gnd n w=5.00u l=0.35u
mp1 cout n2 vdd vdd p w=10.00u l=0.35u
mn1 cout p cin gnd n w=5.00u l=0.35u
.ends
```

Now look at the second waveform display opened by `cfutil`, the DFA waveforms, and select the **cf-all** tab (Figure 70). You can see the inputs of the `nor2` subcircuit are simultaneously 0 or 3.3V (`vdd`), so even if the `mp1` MOS device is shorted and `n1` pulled up to `vdd`, that never changes the output signal (`k`) value.

Figure 70 PrimeSim Custom Fault waveforms



In conclusion, the defect is detectable however it cannot be detected with the current stimuli. The next step would be to modify the stimuli to be able to detect the short defect.

To ensure the signals of interest are probed, you can use the probe file generated by DFA. In the `RESULT/ec_defect.1/dfa` directory, you can find a probe file, `dfa.probe` (see the following excerpt of the file). Include this file in your netlist before running the fault campaign if you intend to verify the results.

```
* probe nodes
```

```
.probe v(0)
.probe v(vdd)
.probe v(top.dut.a_3)
.probe v(top.dut.a_2)
.probe v(top.dut.a_1)
.probe v(top.dut.a_0)
.probe v(top.dut.b_3)
.probe v(top.dut.b_2)
.probe v(top.dut.b_1)
.probe v(top.dut.b_0)
.probe v(top.dut.cin)
...
```

16

Running the PrimeSim Custom Fault Application

This chapter outlines how to run the PrimeSim Custom Fault application (App), the graphical user interface (GUI) for driving the PrimeSim Custom Fault engine. This App is available in both the Custom Compiler and PrimeWave Design Environment for Reliability Analysis applications. Currently, the Custom Fault app supports the manufacturing test and functional safety test application modes.

- [Tool Setup](#)
- [Launching the Custom Fault App](#)
- [Starting a New Session](#)
- [Importing Testbenches](#)
- [Manufacturing Test Testbench Setup](#)
- [Functional Safety Analysis Testbench Setup](#)
- [Silicon Failure Analysis Testbench Setup](#)
- [Defect Model Settings](#)
- [Defect Universe Definition](#)
- [Launching the Defect Campaign](#)
- [Coverage Reports](#)
- [Analyzing Defect Campaign Results](#)
- [Fault Database Viewer](#)

Tool Setup

The following tools must be installed and set up in the `PATH` variable. All tools should be of the same versions.

- Custom Compiler or PrimeWave Design Environment for Reliability Analysis applications
- PrimeWave Design Environment
- One of the following supported simulators:
 - PrimeSim
 - PrimeSim XA
 - FineSim
- VCS (and VCS PrimeSim AMS) for mixed-signal simulation
- PrimeSim Custom Fault

Launching the Custom Fault App

The Custom Fault App can be launched from either the Custom Compiler tool or the PrimeWave Design Environment for Reliability Analysis applications.

If the Custom Fault or PrimeWave Design Environment for Reliability Analysis tool has been properly added to the `PATH` variable, the Custom Fault App appears on the GUI home screen.

```
$> custom_compiler
```

or

```
$> primewave_reliability
```

Chapter 16: Running the PrimeSim Custom Fault Application

Launching the Custom Fault App

Figure 71 Launching the Custom Fault App



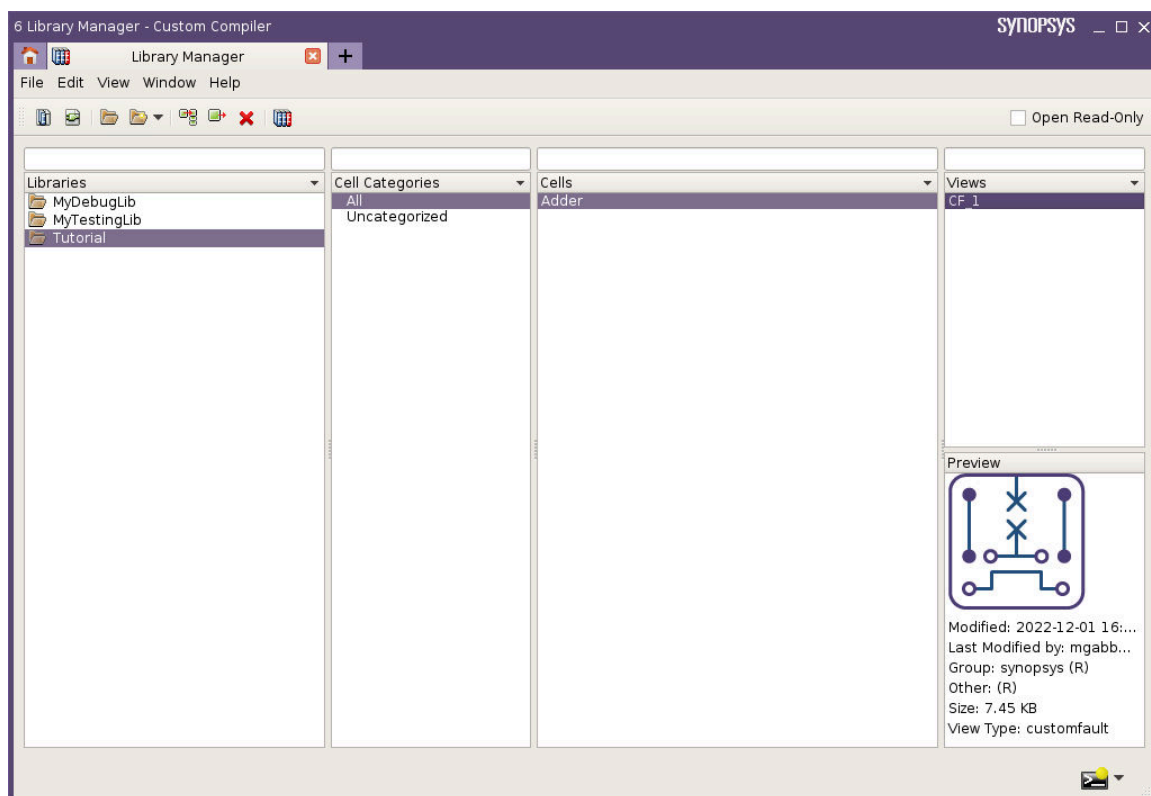
PrimeSim Custom Fault App Entry Points

The Custom Fault App has the following available entry points:

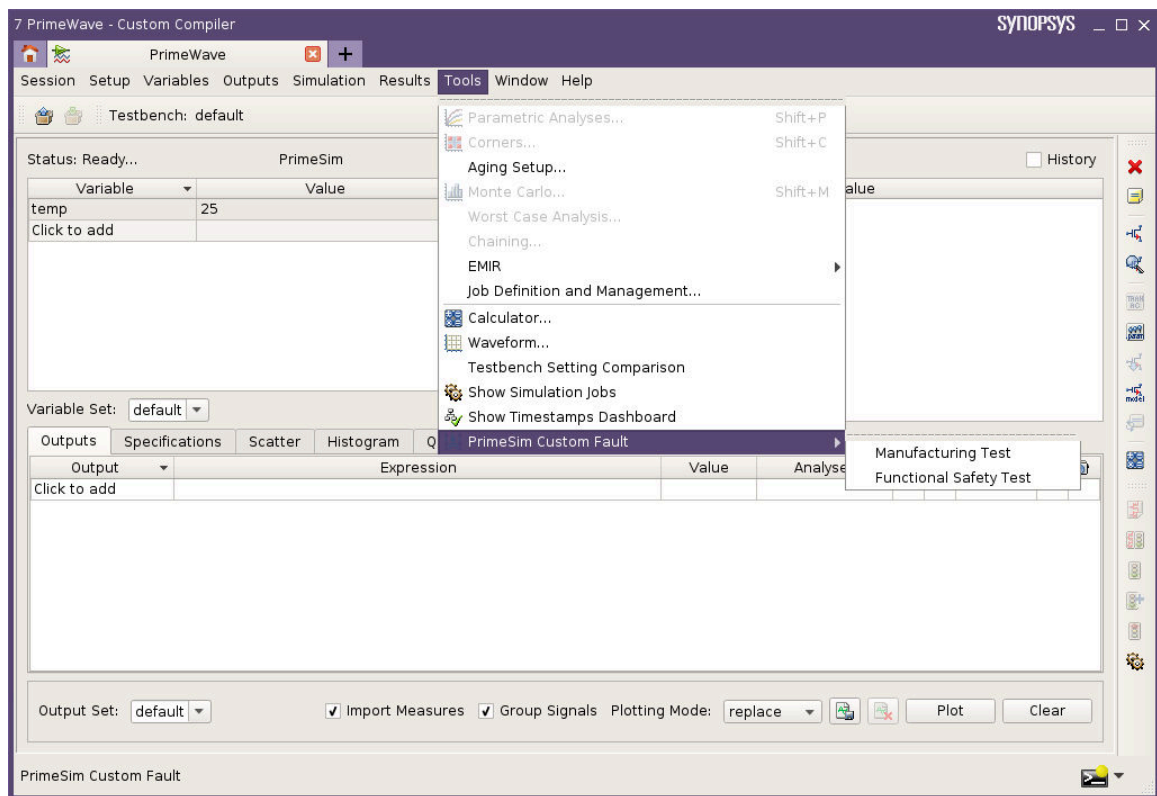
- Custom Fault App Icon

Click the Custom Fault App icon on the Custom Compiler or PrimeWave Design Environment for Reliability Analysis applications home screens as shown in [Figure 71](#).

- Custom Fault cell view in the Library Manager:



- From the PrimeWave Design Environment **Tools** menu:

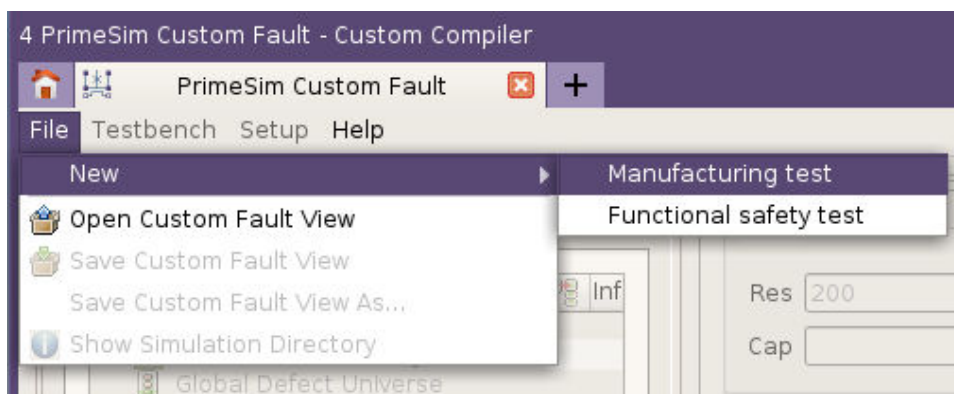


Starting a New Session

As described in [Launching the Custom Fault App](#), use one of the following methods to start a new session of the Custom Fault App:

- Launch the Custom Fault App from a home screen icon.
- Launch a new session from an existing session in the Custom Fault App.
- Launch the Custom Fault App from PrimeWave Design Environment.

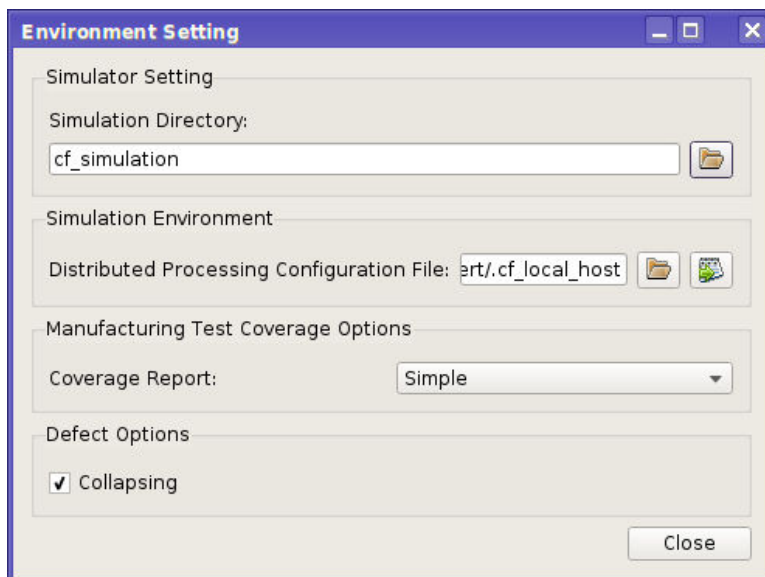
When a new session is launched from the PrimeWave Design Environment, you must choose the application mode simultaneously. For the other methods, you must choose the application mode from the **File** menu.



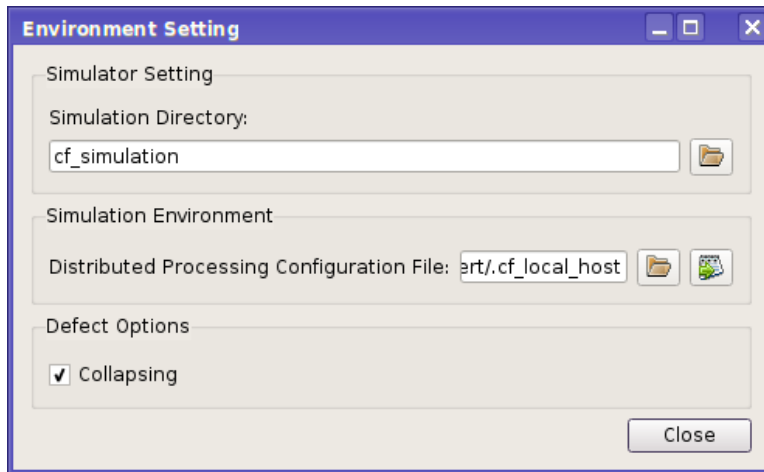
The windows, dialog boxes, and features within the App are tailored when you choose the application mode. After you choose the application mode, it cannot be changed for the current session. That is, If if you have already chosen the application mode and you subsequently choose **File > New > Manufacturing test** or **File > New > Functional safety test** again, a new tab opens in the environment with a new session of the Custom Fault App.

Environment Settings

In Manufacturing test mode, choose **Setup > Environment** to open the **Environment Setting** dialog box.



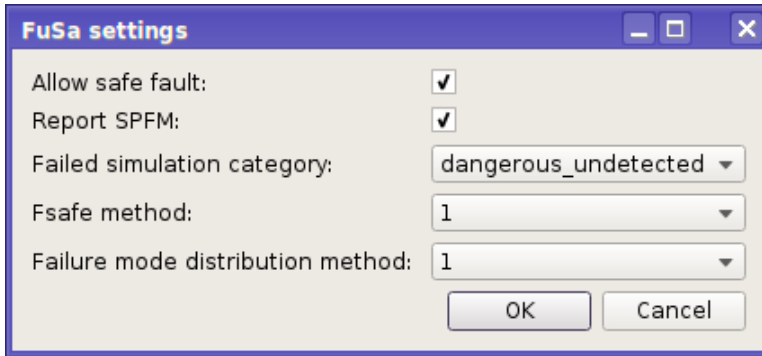
In Functional safety test mode, choose **Setup > Environment** to open the **Environment Setting** dialog box.



Field	Description
Simulation Directory	By default, the Custom Fault tool creates a <code>cf_simulation</code> directory from the location where the environment and Custom Fault App was launched.
Distributed Processing Configuration File	Path to the distributed processing configuration file (see Common DP Library Information). You can click Load  to load a file, or Edit  to open the file in a text editor.
Coverage Report	Specifies the output format for reporting test coverage results: <code>Simple</code> (default) produces the one-line-per-defect report shown in Setting Up the Custom Fault Tool. <code>Detailed</code> produces a verbose report similar to that shown in Detailed Format Report. Corresponds to the <code>test_coverage</code> argument of the <code>report</code> command. Note: This field is only displayed for Manufacturing test mode environment settings.
Collapsing	Specifies whether to collapse defects: - If this field is checked, the tool runs a single simulation for equivalent defects. - If this field is unchecked, the tool runs a simulation for each defect. Corresponds to the <code>enable_fault_collapse</code> command.

Functional Safety Settings

In Functional safety test mode only, choose **Setup > Functional Safety** to open the **FuSa settings** dialog box.



Note:

The fields in this dialog box correspond to arguments for the `diagnostic_coverage` command. See [diagnostic_coverage \(for Residual Faults and Direct SPFM Calculation\)](#) for more information.

Field	Description
Allow safe fault	The default is <code>true</code>. When <code>false</code>, no injected defect is classified as safe. All injected defects are classified as either dangerous detected or dangerous undetected.
Report SPFM	The default is <code>false</code>. When <code>true</code>, an additional report is printed at the end of the <code>.diagnostic_report</code> file. For details, see SPFM Output.
Failed simulation category	Specifies that a failed simulation is classified as one of the following: • <code>dangerous_undetected</code> (default) • <code>dangerous_detected</code> • <code>safe_undetected</code> • <code>safe_detected</code> Note: This field has priority over the Allow safe fault field.
Fsafe method	Defines how the percentage of Fsafe (safe faults) for a failure mode is calculated. A fault counted as safe does not contribute to any failure mode and most FMEDAs do not have a row for safe faults. Safe faults are allocated to one of the defined failure modes. For details, see the <i>f_{safe} method Settings</i> table in diagnostic_coverage (for Residual Faults and Direct SPFM Calculation).

Field	Description
Failure mode distribution method	Determines the failure mode distribution for the SPFM calculation. Weights of defects allocated as safe are not included in the failure mode distribution calculation. The sum of all failure mode distributions should be 1 or 100%. For details, see the <i>failure_mode_distribution_method Settings</i> table in diagnostic_coverage (for Residual Faults and Direct SPFM Calculation) .

Importing Testbenches

This section describes how to import design data to run a defect analysis campaign. The following flows are supported for importing data.

- The first flow uses the PrimeWave Design Environment to fully define the testbench simulations, circuit measurements, and defect detection criteria. This data set is managed fully within the Custom Compiler or PrimeWave Design Environment for Reliability Analysis applications.

See [Importing Testbenches in the PrimeWave Design Environment Flow](#).

- The second flow is a netlist input flow. In this flow, you are generating the simulation netlist, measurements, and defect detection criteria through other means into the netlist files. The Custom Fault App is pointed at these files to execute the defect analysis campaigns.

See [Importing Analog-Only Testbenches in the Netlist Flow](#) and [Importing Mixed-Signal Testbenches in the Netlist Flow](#).

Importing Testbenches in the PrimeWave Design Environment Flow

Use the PrimeWave Design Environment to set up and run simulations as follows:

- Define a testbench or test suite in the PrimeWave Design Environment and import it into the Custom Fault App.
- Define the library and cell from which the simulation netlist is generated.
- Specify the simulator, any simulator options, netlist parameters, and circuit measurements.
- Define analog-only or mixed-signal testbenches.

See the *PrimeWave™ Design Environment User Guide* for more information.

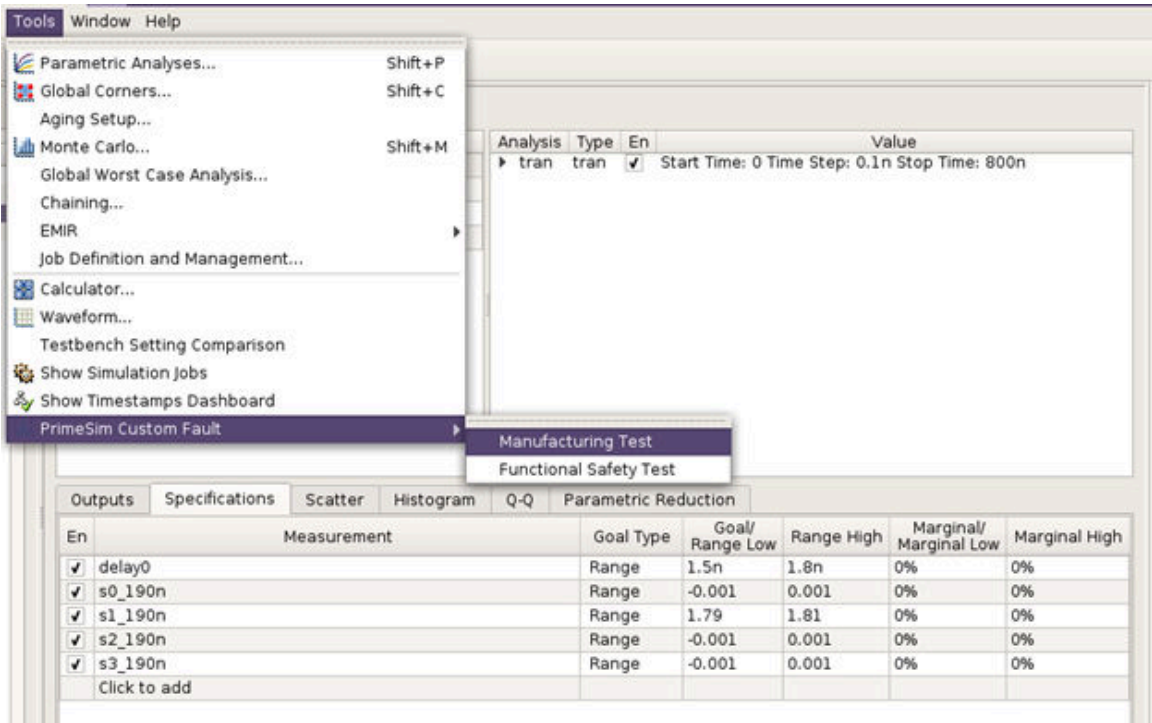
The circuit measurements defined in the PrimeWave Design Environment **Outputs** pane are either SPICE measure statements or PrimeWave Design Environment waveform calculator expressions. Both can be used as the basis for defect detection.

- To designate such measurements as defect detection criteria, define a specification value for the output.

Outputs Specifications Scatter Histogram Q-Q Parametric Reduction						
En	Measurement	Goal Type	Goal/Range Low	Range High	Marginal/Marginal Low	Marginal High
✓	delay0	Range	1.5n	1.8n	0%	0%
✓	s0_190n	Range	-0.001	0.001	0%	0%
✓	s1_190n	Range	1.79	1.81	0%	0%
✓	s2_190n	Range	-0.001	0.001	0%	0%
✓	s3_190n	Range	-0.001	0.001	0%	0%

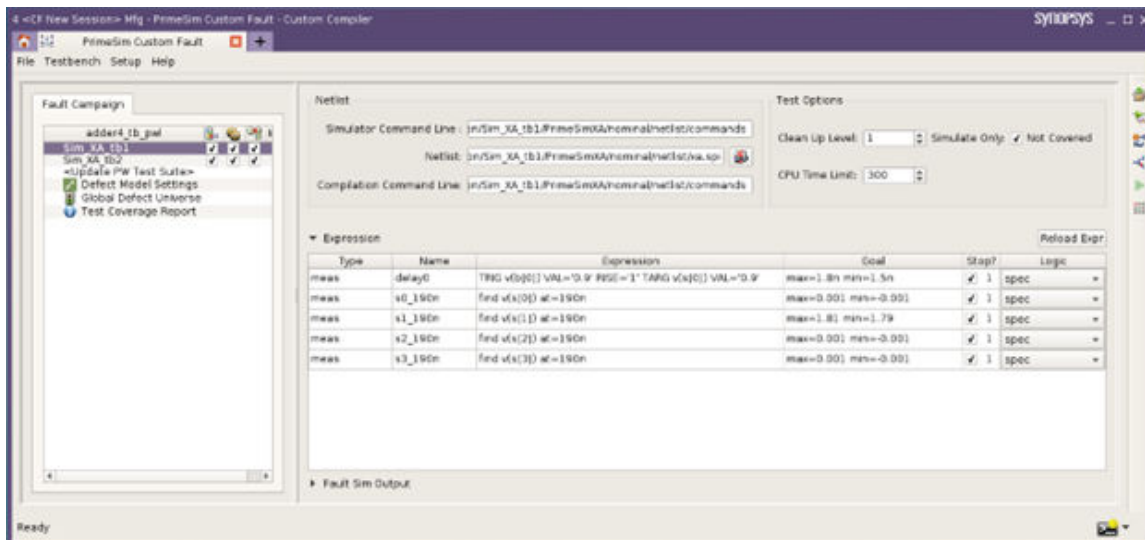
After the testbenches are prepared in the PrimeWave Design Environment, they are all imported to the Custom Fault App in a single step.

- From the PrimeWave Design Environment **Tools** menu, choose **PrimeSim Custom Fault**, then choose the appropriate application mode for the test to be performed.



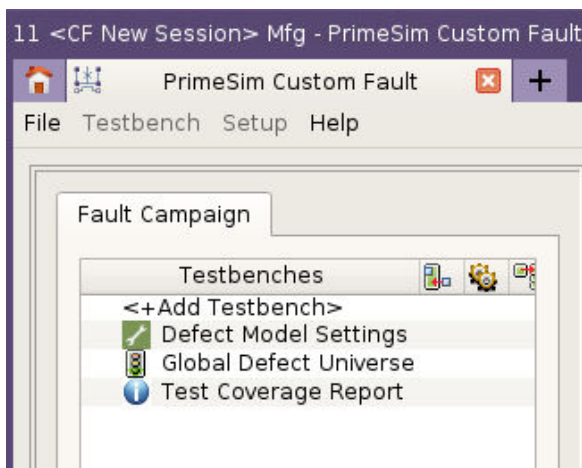
The Custom Fault App opens in a new tab in the selected mode. All the testbenches in the test suite appear in the campaign list. The **Netlist** section of the Custom Fault

App is automatically populated and any outputs with specifications are listed in the expression table.



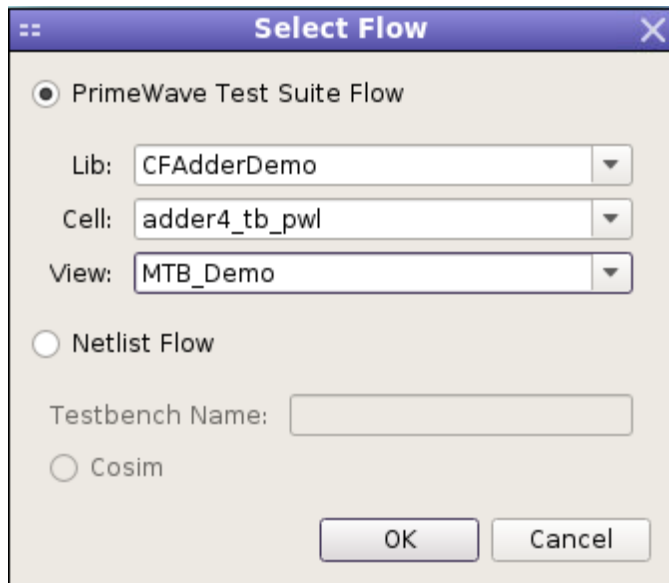
The PrimeWave Design Environment testbench or test suite can also be saved as a cell view in the environment and the Custom Fault App can import the testbench directly from the cell view.

1. After opening the Custom Fault App, choose a new manufacturing test or functional safety test session, then double-click **<+Add Testbench>** in the **Fault Campaign** tab.



The first time a testbench is added to the session, a **Select Flow** dialog box opens.

2. Click **PrimeWave Test Suite Flow** and then select the **Lib**, **Cell**, and **View** of the PrimeWave Test Suite.



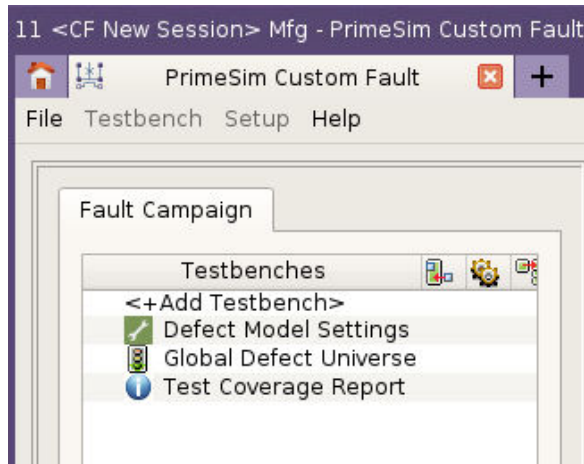
3. Click **OK**.

The Custom Fault App imports and displays the testbench data.

Importing Analog-Only Testbenches in the Netlist Flow

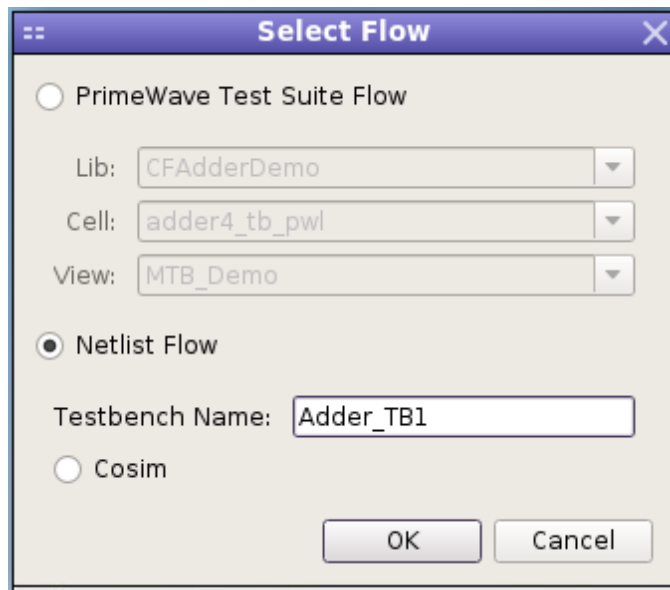
You can use the netlist import flow if the testbench simulation netlist is created outside of the PrimeWave Design Environment by some other means you manage. This can be common for complex simulation and verification flows in user environments. When using the netlist import flow, certain features of the Custom Fault App are not available, such as annotating instances on a schematic view.

1. After opening the Custom Fault App, choose a new manufacturing test or functional safety test session, then double-click **<+Add Testbench>** in the **Fault Campaign** tab.

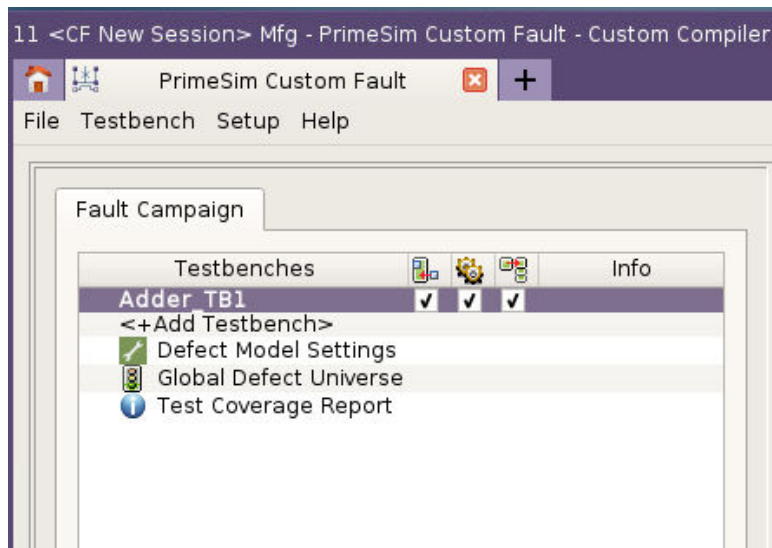


The first time a testbench is added to the session, a **Select Flow** dialog box opens.

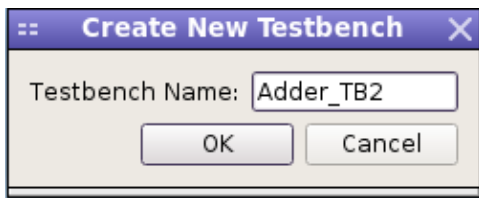
2. Click **Netlist Flow** and type a **Testbench Name**. The testbench name is a label for the newly added testbench.



3. Click **OK**. The Fault Campaign list is updated to show the new testbench.

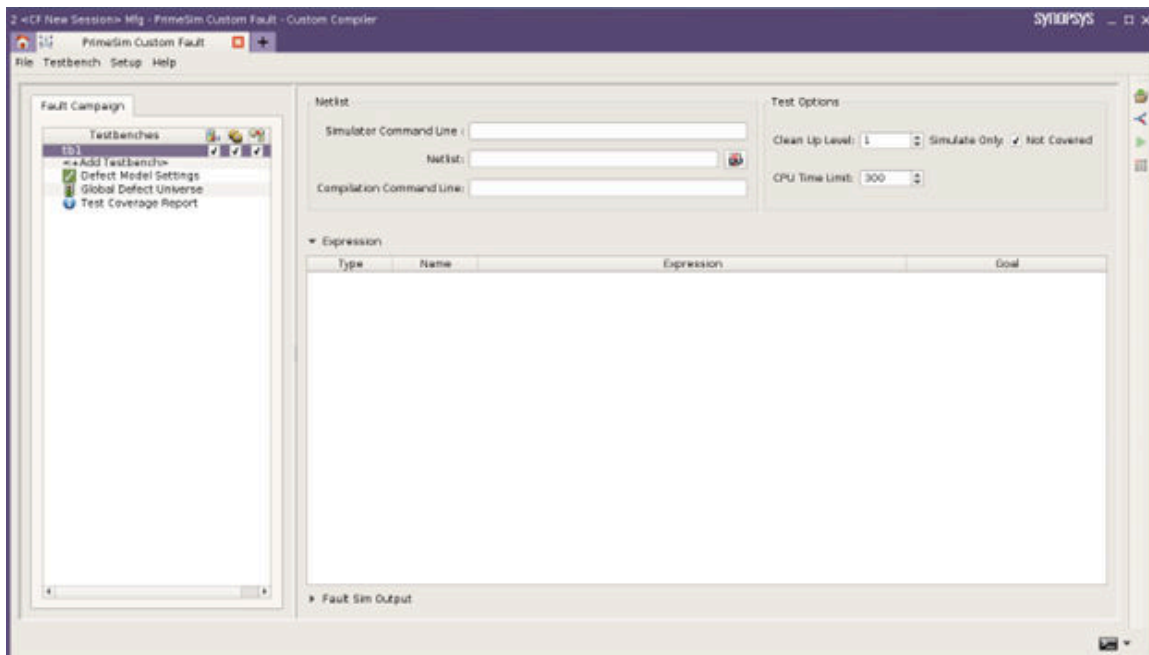


Each time an additional testbench is added, a **Create New Testbench** dialog box opens, prompting a new testbench name.



After a testbench has been added to the **Fault Campaign** list, the data for that testbench must be populated.

1. Click the testbench name in the **Fault Campaign** list. The right side of the display now shows the **Netlist** and **Test Options**, as well as the defect detection methods for the testbench.



2. Enter the **Simulator Command Line** used to execute a simulation of the testbench just as if that simulation was being launched in batch mode from a Linux terminal. For example:

For PrimeSim:

```
$> primesim test1.sp -mode prohd
```

For PrimeSim XA:

```
$> xa test1.sp -c xa.cfg
```

Omit the `-o` option to redirect simulator output because the Custom Fault tool automatically controls the output file locations.

Note:

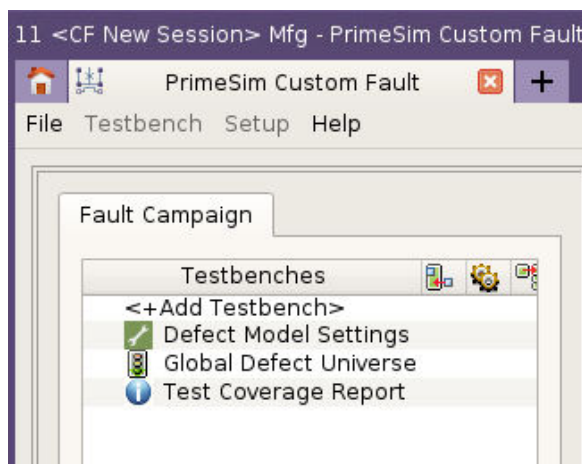
The **Simulator Command Line** field corresponds to the `sim_cmdline` argument of the `analog_setup` command.

After you provide the **Simulator Command Line**, the App updates **Netlist** field. You can view the netlist file by clicking the button beside the netlist name.

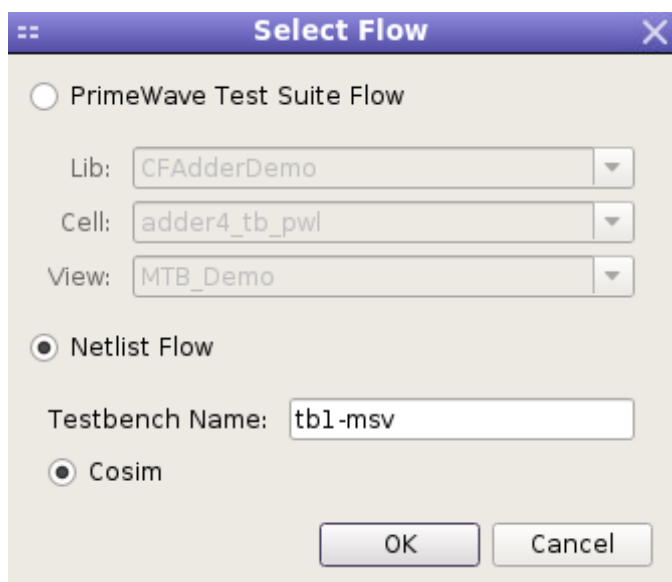
Importing Mixed-Signal Testbenches in the Netlist Flow

This section describes how to set up a mixed-signal testbench.

1. Double-click the **<+Add Testbench>** in the **Fault Campaign** tab to open the **Select Flow** dialog box.



2. Click **Netlist Flow**, type a **Testbench Name**, and click **Cosim**.

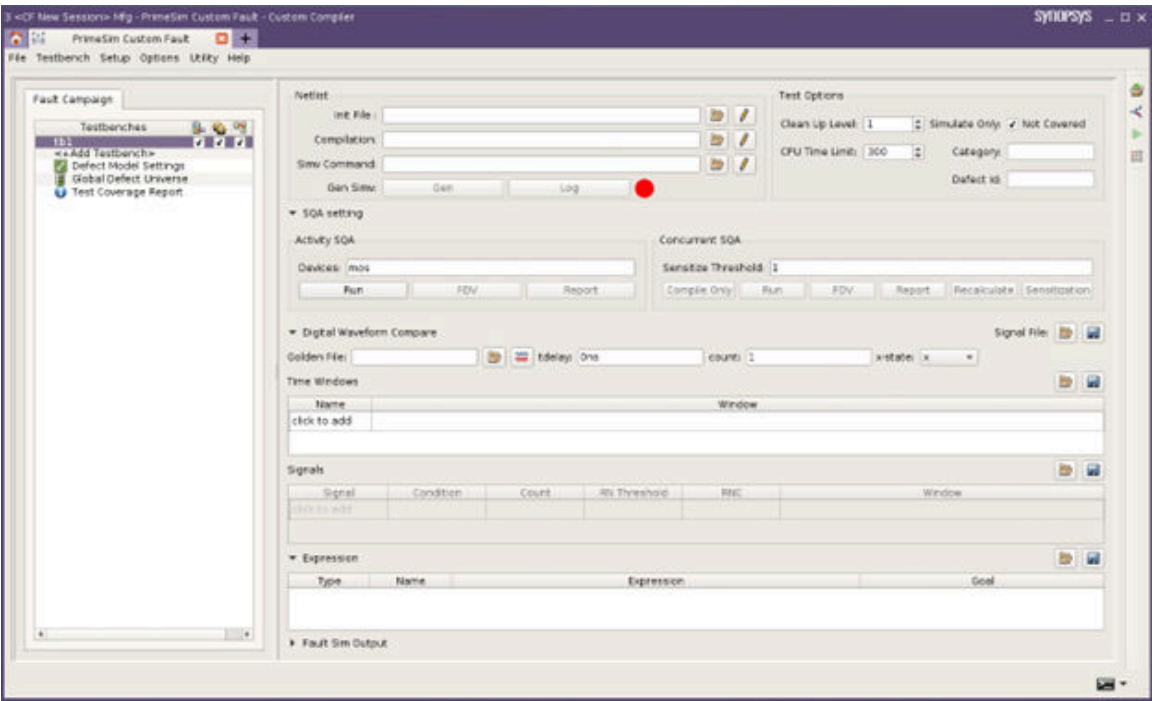


Note:

Once the **Cosim** selection is made, any further testbench added to the session must also be either cosim or non-cosim.

3. Click **OK**. The **Fault Campaign** tab is updated to show the new testbench.

4. Click the testbench name in the campaign list to display the testbench settings.



5. Provide the entries in the **Netlist** section as described in the following table:

Field	Description
Init File	(Required) Click Load  to select the mixed-signal control file. Corresponds to the <code>init</code> argument of the <code>cosim_setup</code> command.
Compilation	(Optional) If you want the Custom Fault tool to recompile the <code>simv</code> mixed-signal executable, click Load  to select the file containing the compilation script. Corresponds to the <code>compilation</code> argument of the <code>cosim_setup</code> command.
Simv Command	(Required) Click Load  to select the file containing the <code>simv</code> executable command. Corresponds to the <code>simv_comdline</code> argument of the <code>cosim_setup</code> command.
Gen Simv	Click Gen to execute the compilation script. Click Log to view the compilation log file. The status light is colored red if the <code>simv</code> executable cannot be found and changes to green if the <code>simv</code> executable is available.

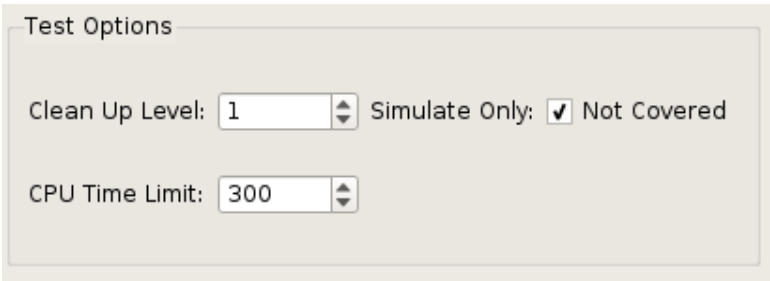
Manufacturing Test Testbench Setup

This section describes the various controls available for manufacturing test coverage analysis on a per-testbench basis.

- [Test Options](#)
- [Defect Detection Expressions](#)
- [Mixed-Signal Digital Waveform Compare Settings](#)

Test Options

The test options section provides controls that are applied on a per-testbench basis.



Test Options

Clean Up Level: 1 Simulate Only: ☒ Not Covered

CPU Time Limit: 300

Setting	Description
Clean Up Level	Sets the file cleanup level for defect simulation. See the <code>cleanup_level</code> command for more information.
CPU Time Limit	Sets the maximum CPU time limit for simulations of this testbench. If the CPU time is exceeded the simulation ends. See the <code>cpulimit</code> command for more information.
Simulate Only	The default setting for test coverage applications is to only simulate defects that are not already covered or detected by prior tests. To force simulation of the defect regardless of the coverage status, uncheck this box. Note: This option functions as the opposite of the <code>covered</code> argument of the <code>simulate_only</code> command.

Defect Detection Expressions

The **Expression** table lists defect detection expressions and their detection thresholds. These expressions cannot be edited in this table. The expressions and criteria can only be edited in the source PrimeWave Design Environment testbench or the source netlist.

Expression Reload Expr					
Type	Name	Expression	Goal	Stop?	Logic
meas	delay0	TRIG v(b[0]) VAL="0.9" RISE="1" TARG v(s[0]) VAL="0.9"	max=1.8n min=1.5n	☒ 1	spec
meas	s0_190n	find v(s[0]) at=190n	max=0.001 min=-0.001	☒ 1	spec
meas	s1_190n	find v(s[1]) at=190n	max=1.81 min=1.79	☒ 1	spec
meas	s2_190n	find v(s[2]) at=190n	max=0.001 min=-0.001	☒ 1	spec
meas	s3_190n	find v(s[3]) at=190n	max=0.001 min=-0.001	☒ 1	spec

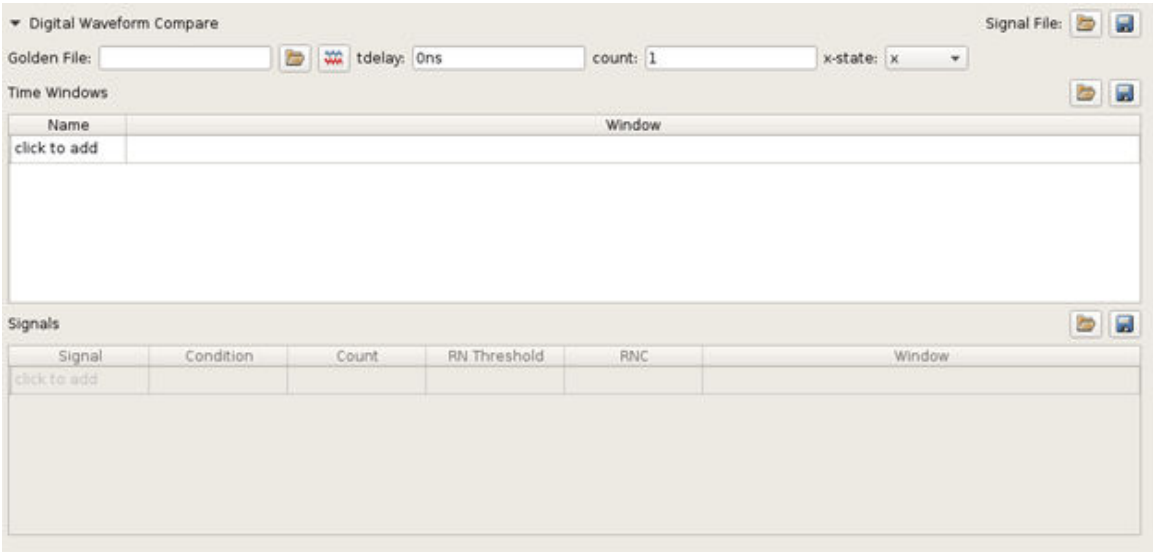
Column	Description
Type	Identifies the expression type.
Name	Identifies the name or label of the detection statement.
Expression	Provides the details of the defect detection measurement.
Goal	Displays the defect detection thresholds.
Stop?	In manufacturing test coverage, this setting defaults to 1 which stops the simulation as soon as a defect detection occurs. If set to 0, a triggered detection does not stop the simulation. If the expression is a PrimeWave Design Environment expression, it cannot be stopped on detection.
Logic	Determines how the Goal section is interpreted. spec: The goal defines the spec and any value outside of the spec is considered a defect detection. defect: The specified goal range defines the defect and a value within the threshold is considered a defect detection. The setting of defect is the logical inverse of the setting of spec.

Note:

In a netlist-based flow, only the top-level netlist file is parsed by the Custom Fault tool to display the expressions. Any `.MEASURE` statements associated with a `set_sim_stop` command are displayed in the expression table (see [Using a .MEASURE Statement](#) for more information). The expressions and criteria can only be edited from the source netlist file.

Mixed-Signal Digital Waveform Compare Settings

A digital waveform comparison feature is available for defect detection when running mixed-signal simulation.

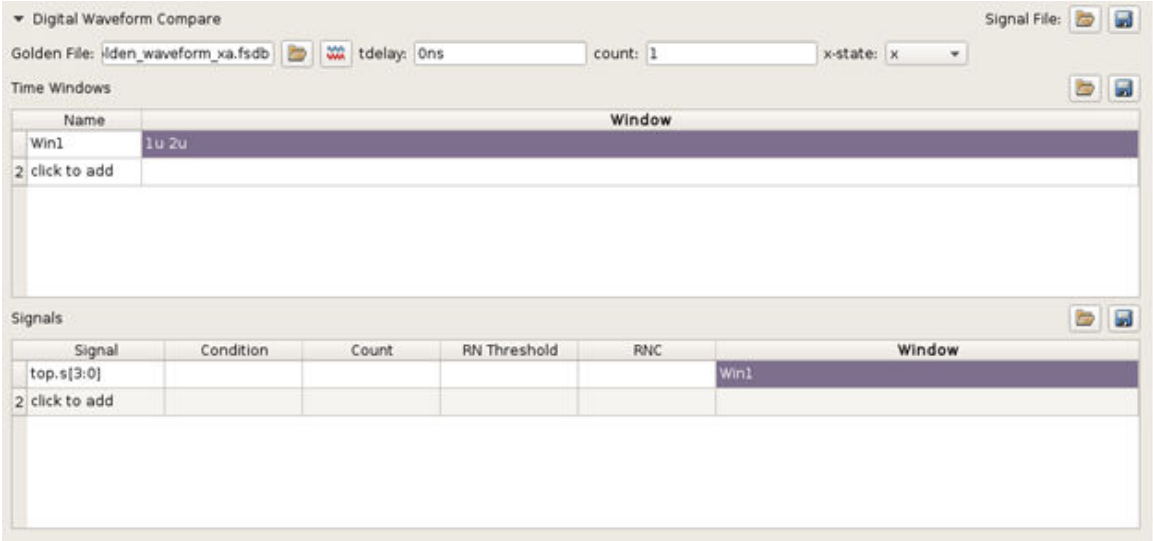


See [Digital Waveform Comparison in the Mixed-Signal Flow](#) for additional information on the digital waveform comparison feature.

Field	Description
Golden File	(Required) Click Load Golden File  to load the reference defect-free waveform file. It must be an FSDB format file. You can click Display Waveform  to open the golden file in the PrimeWave Design Environment waveform browser.
tdelay	Specifies the global threshold for time difference during comparison between the golden reference signal and the simulation signal. See the 'tdelay argument under Signal File Format for more information.
count	Specifies the global setting for the maximum number of violations allow per signal before the simulation is ended. See the 'count argument under Signal File Format for more information.
x-state	Defines if a logic x state is treated as X, 0, or 1. See the 'x argument under Signal File Format for more information.

Time windows are defined in the **Time Windows** table. The time windows can later be assigned to signals being checked in the **Signals** table. Click a row of the table to define

a new time window. The **Name** column is the time window ID. The **Window** is a space-separated list of start and stop times for the checking windows. See [Defining a Time Window](#) for more information.



The **Signals** displays the following fields (see [Signal File Format](#) for more information):

Column	Description
Signal	Specifies the name of the digital signal. It must match to a signal name available in the golden file.
Condition	Specifies a conditional signal name with a condition such that if the condition is satisfied, FSDB comparison takes place (see Conditional Comparison of Signals). Corresponds to <i>arg1</i> in Signal File Format .
Count	Specifies the maximum number of violations allowed for the specified signal. It takes precedence over the count field. Corresponds to <i>arg2</i> in Signal File Format .
RN Threshold	Specifies the threshold to use in case of a Comparison of Real Signals . Corresponds to <i>arg3</i> in Signal File Format .
RNC	Specifies on what basis to perform a Comparison of Real Signals : • <i>blank</i> - Instantaneous Difference (ID) • <i>rms</i> - Root Mean Square Difference (RMSD) Corresponds to <i>arg4</i> in Signal File Format .
Window	Specifies the time window name if you are using the time window based FSDB comparison feature (see Time Window Based FSDB Comparison). Corresponds to <i>arg5</i> in Signal File Format .

Functional Safety Analysis Testbench Setup

Setting up a functional safety analysis has additional requirements over the defect detection criteria. You must also define failure modes and associate the defect detection criteria with the failure modes. Additionally, you must assign the detection expression to either a mission circuit functionality detection or a safety mechanism detection.

- [Failure Modes](#)
- [Expression](#)
- [Mixed-Signal Functional Safety Setup for Digital Waveform Comparison](#)

Failure Modes

When in functional safety mode, the testbench screen contains a **Failure Modes** table.



Failure Modes	
ID	ID Label
FM01	Over voltage
2 FM02	Under voltage
3 click to add	

The **Failure Modes** table is arranged the following columns.

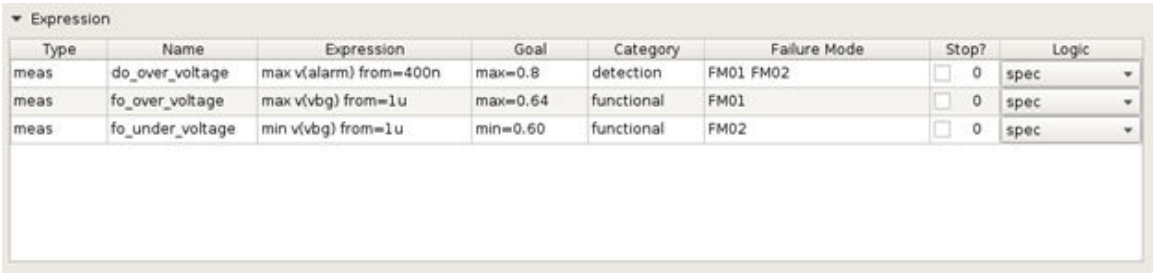
Column	Description
ID	The failure mode ID; this is normally a code that cross-references with a Failure Mode Effect and Diagnostic Analysis (FMEDA). See Overview of Functional Safety for more information.
ID Label	A verbose description of the failure mode; this is printed in the reports.

Update the **Failure Modes** table as follows:

- Click **click to add** to add additional failure modes.
- Click **Load Failure Mode**  to select a file with a previously-saved failure mode definition to automatically populate the table.
- Click **Save Failure Mode**  to save the failure mode definitions to a file.

Expression

In functional safety mode, the **Expression** table has the following additional columns.



Type	Name	Expression	Goal	Category	Failure Mode	Stop?	Logic
meas	do_over_voltage	max v(alarm) from=400n	max=0.8	detection	FM01 FM02	☐ 0	spec
meas	fo_over_voltage	max v(vbg) from=1u	max=0.64	functional	FM01	☐ 0	spec
meas	fo_under_voltage	min v(vbg) from=1u	min=0.60	functional	FM02	☐ 0	spec

Column	Description
Category	Set as follows: - functional if the expression is assigned to assessing functionality of the mission circuit. - detection if the expression is assigned to assessing the safety mechanism.
Failure Mode	Used to assign an expression to a failure mode using its ID .

See [Defect Detection Expressions](#) for the other columns.

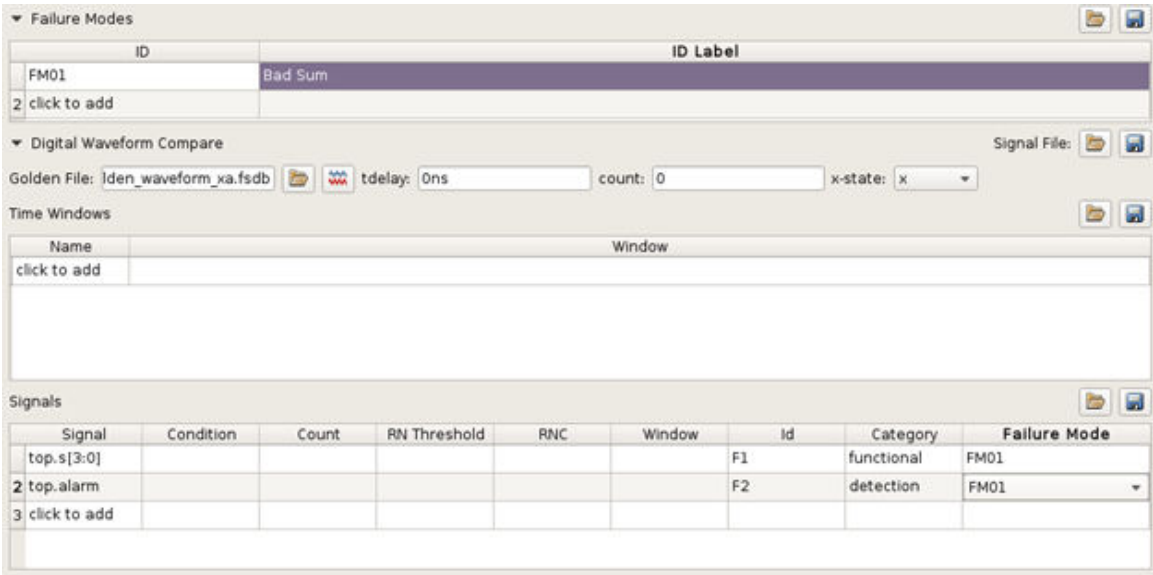
Note:

In functional safety mode, the **Stop?** setting defaults to 0 so that the simulation is not stopped on a detection event. This is the recommended setting for functional safety analysis.

Mixed-Signal Functional Safety Setup for Digital Waveform Comparison

As with manufacturing test coverage (see [Mixed-Signal Digital Waveform Compare Settings](#)), mixed-signal simulation allows for a digital waveform comparison method to be used for defect detection.

You must assign the digital waveform comparison signals as either a mission circuit functional check or as a safety mechanism check and you must assign them to a failure mode. In addition, you must assign each signal a unique **Id** in the **Signals** table as shown in the following figure.



Silicon Failure Analysis Testbench Setup

The PrimeSim Custom Fault App can be used for Silicon Failure Analysis. In the Silicon Failure Analysis, you can narrow down the list of defects that can be the root cause for a failure on silicon. The silicon failure analysis allows to run what-if analysis. Using what-if analysis you can modify measurements or modify the defect criteria and reevaluate the custom fault results without running any new simulation.

- [Requirements](#)
- [Launching the Custom Fault App](#)
- [Selecting the Netlist and Measurement Expression](#)
- [Creating the Defect Models](#)
- [Running Simulations](#)
- [Creating More Measurements and Running What-if Analysis](#)
- [Deleting original measurements](#)
- [Limitations](#)

Requirements

The following tools are required for Silicon Failure Analysis:

- PrimeSim Custom FaultV-2023.12
 - PrimeWave Design Environment for Reliability Analysis V-2023.12 and Custom Infrastructure V-2023.12 or Custom Compiler V-2023.12
 - PrimeWave Design Environment V-2023.12
-

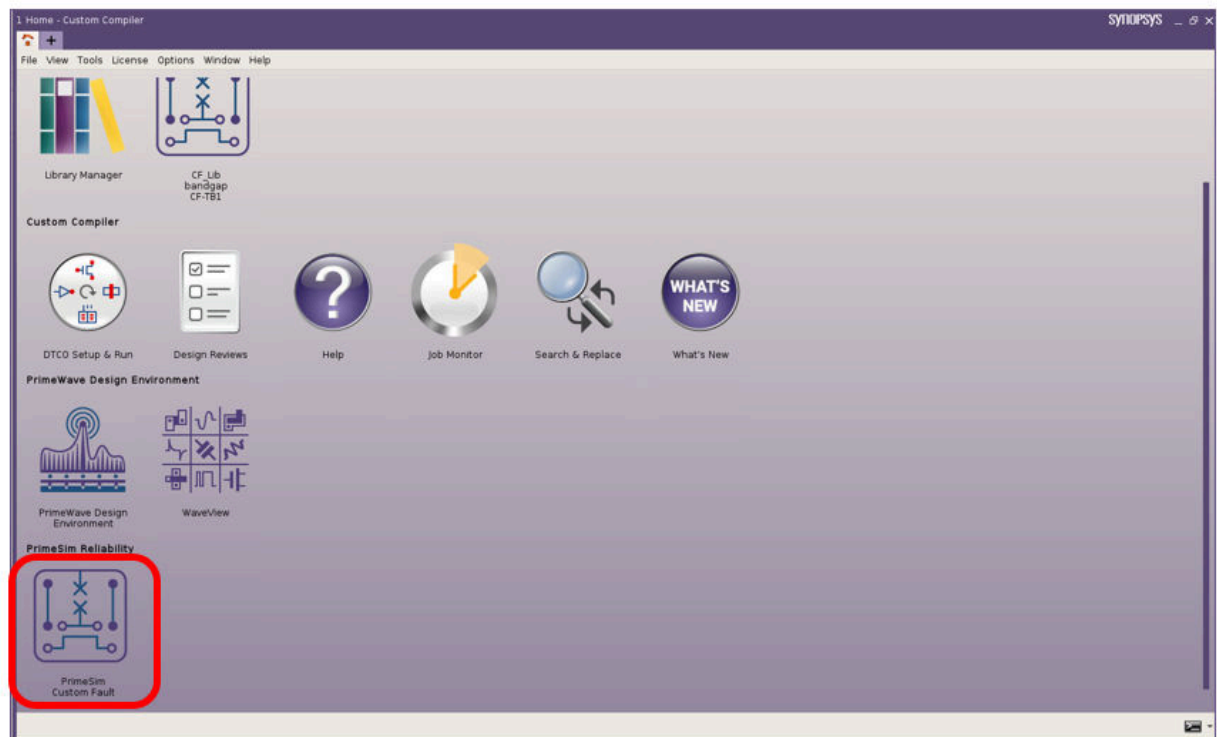
Launching the Custom Fault App

1. Open the Custom Fault in the PrimeWave Design Environment for Reliability Analysis or Custom Compiler. The Custom Fault App can be launched from either the Custom Compiler tool or the PrimeWave Design Environment for Reliability Analysis applications.

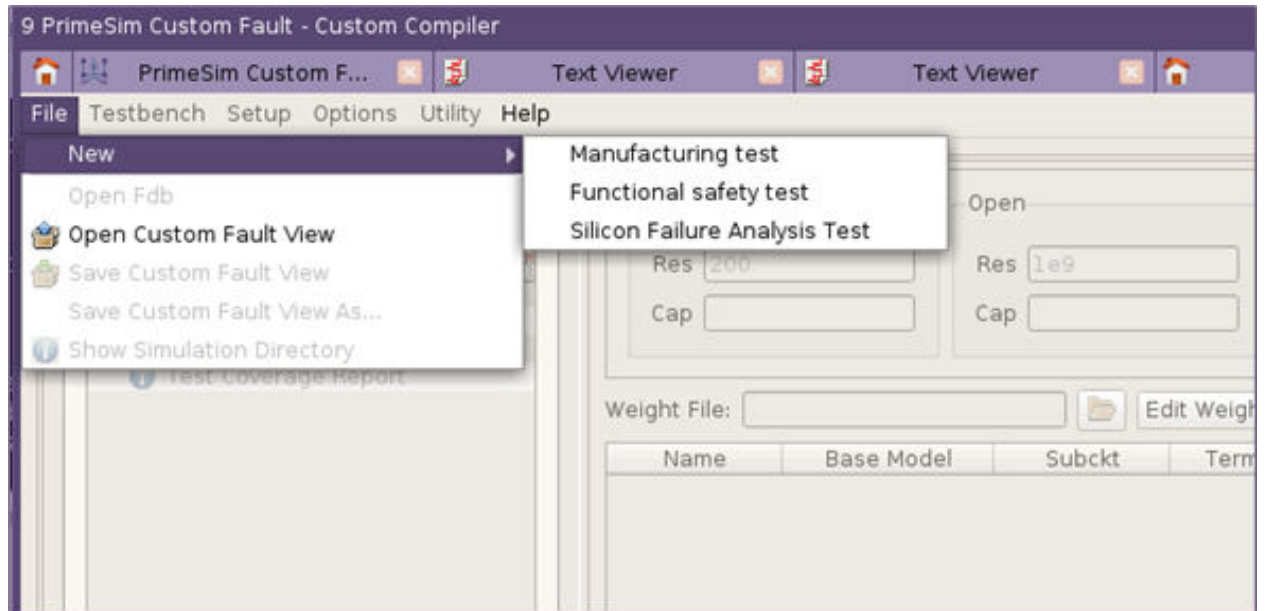
```
$> custom_compiler
```

or

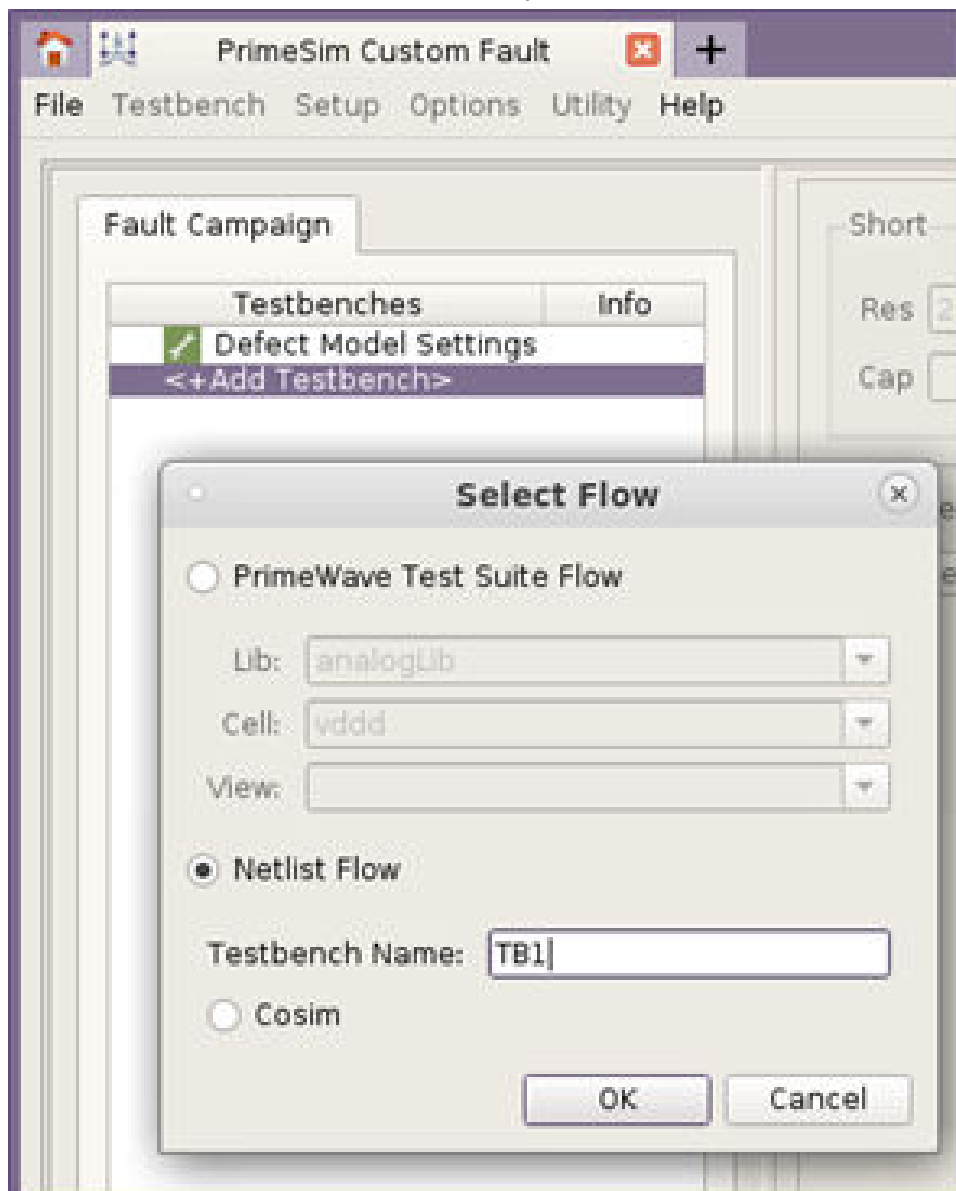
```
$> primewave_reliability
```



2. Click the PrimeSim Custom Fault App icon. The App opens, it is empty and disabled.
3. Choose **File > New > Silicon Failure Analysis Test**.



4. Create a new testbench:
 - a. Double-click **<+Add Testbench>** in the **Fault Campaign** tab. The first time a testbench is added to the session, the **Select Flow** dialog box opens.
 - b. Select **Netlist Flow** and type a **Testbench Name**, for example TB1. The **Testbench Name** is a label for the newly added testbench.

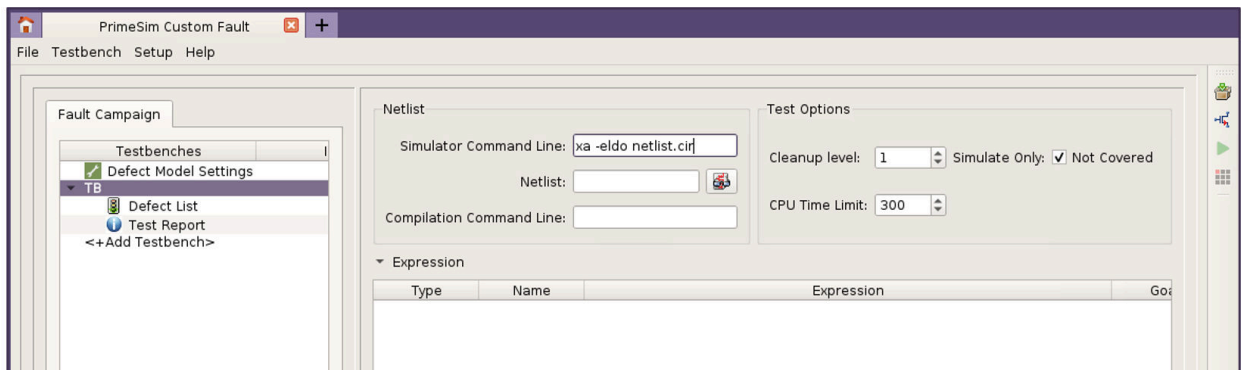


5. Click **OK**. Now you have access to all functionalities of the Custom FaultApp.

Selecting the Netlist and Measurement Expression

1. Ensure that TB1 is selected in the **Fault Campaign** tab, and enter your **Simulator Command Line**, for example:

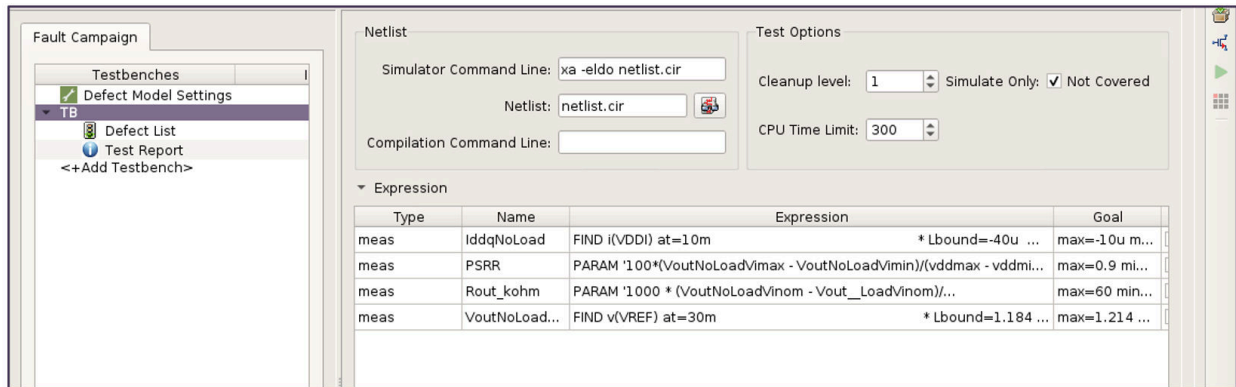
```
xa -eldo netlist.cir
```



After you provide the **Simulator Command Line**, the **Netlist** field and **Expressions** are populated. The expressions are copied from `netlist.cir`.

Note:

If the measurements are in the top netlist, they are considered (for the fault campaign) and displayed. If the measurements are in the included file, they are considered but not displayed.

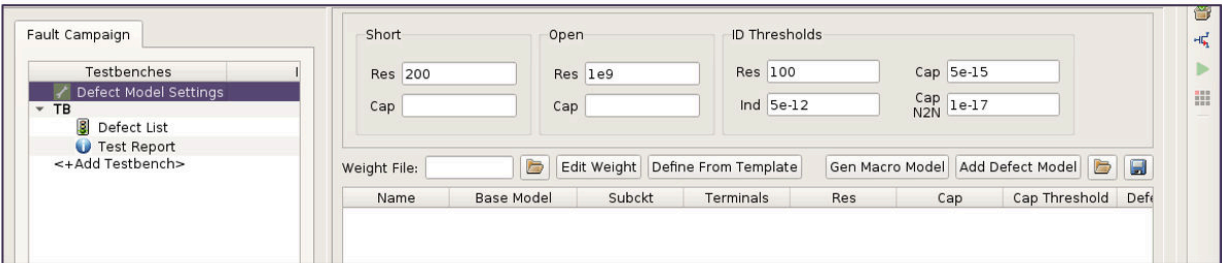


2. In the **Expressions** panel, select **stop=0** if you want to run the What-if analysis.

	Goal	Stop?	
.5*vdd,...	max=52 min=50	☐ 0	spec
	max=450e-12 min=400e-12	☐ 0	spec
	max=300e-12 min=220e-12	☐ 0	spec
	max=3.1 min=2.9	☐ 0	spec
	max=60e-9 min=50e-9	☐ 0	spec

Creating the Defect Models

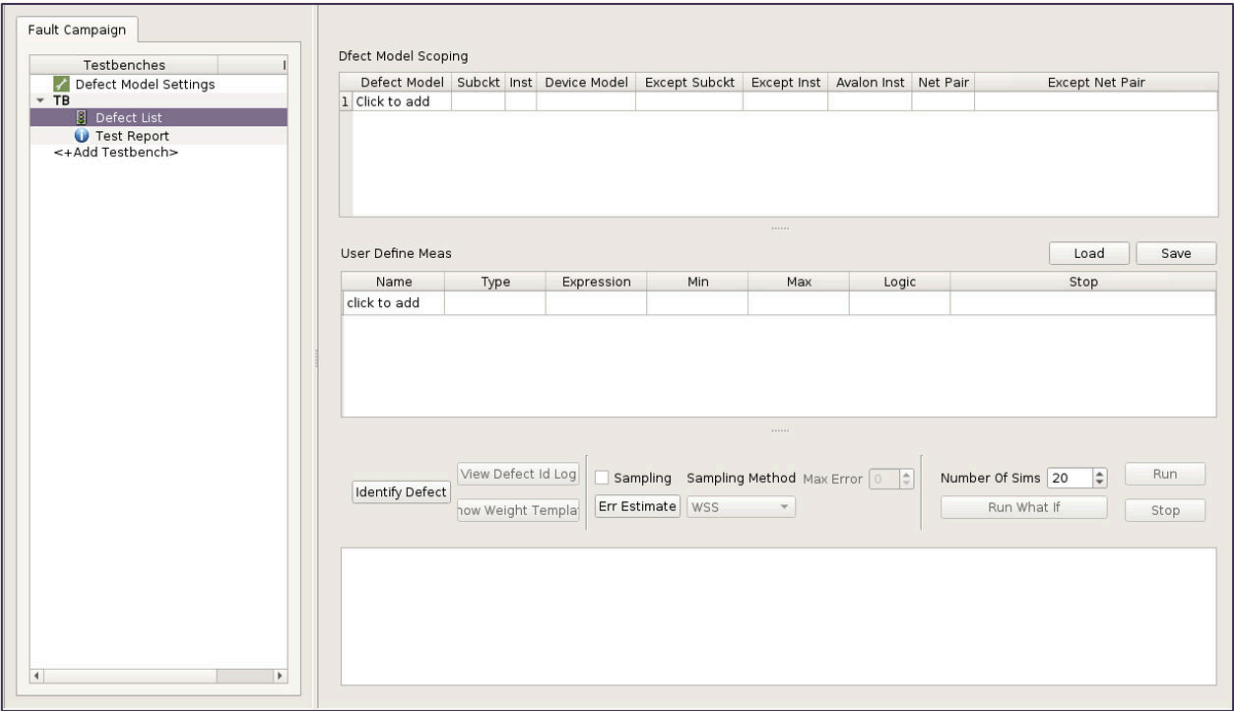
1. Select **Defect Model Settings** in the **Fault Campaign** tab and specify the default values for **Short** and **Open**.



2. Load your defect models (if you already have a JSON configuration file available) or create your defect models. For more details on Defect Model Settings, see the PrimeSim Custom Fault User Guide.

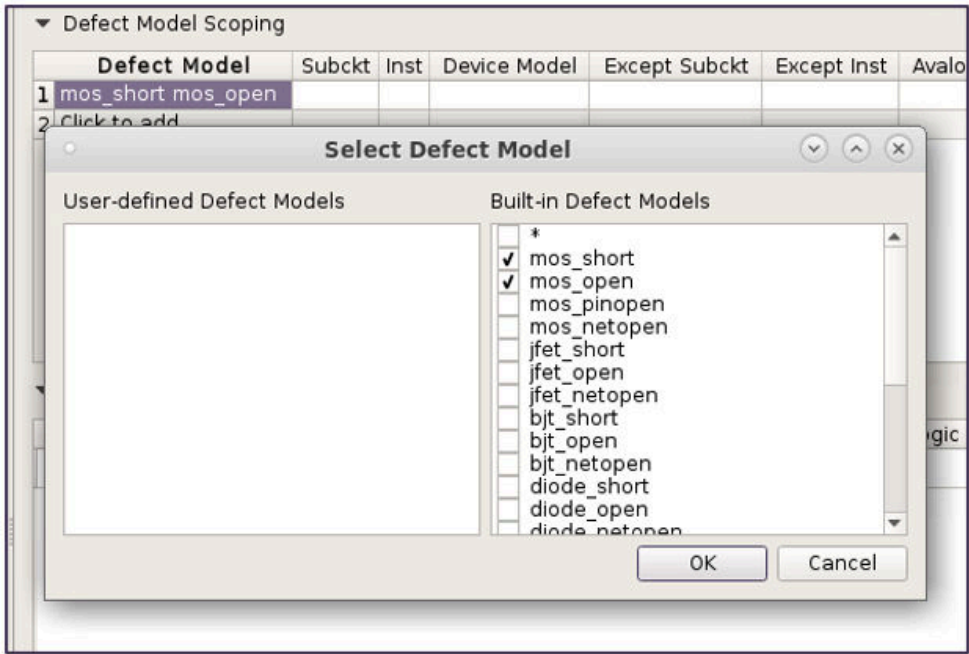
Running Simulations

1. Select **Defect List** in the **Fault Campaign** tab.

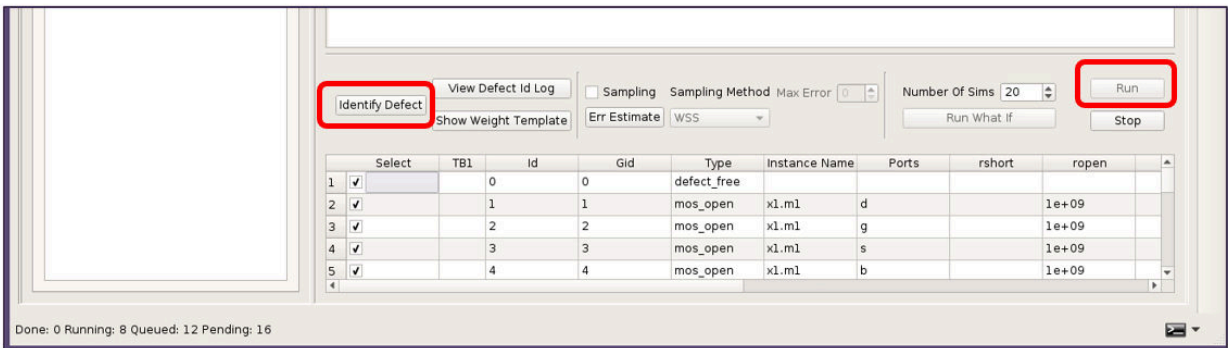


2. Select the models you want to apply.

For more details on Defect Model Settings, see the PrimeSim Custom Fault User Guide.



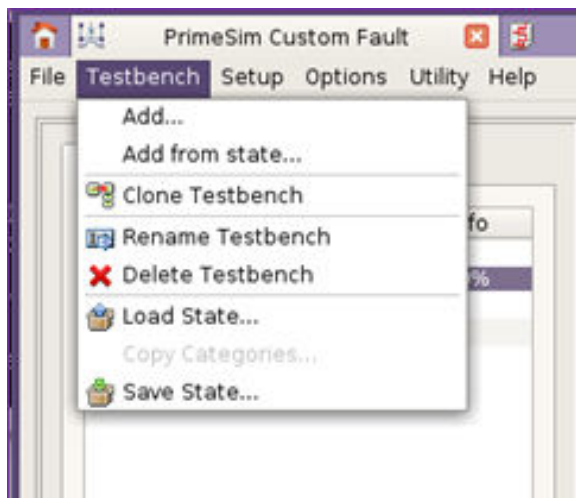
3. Click **Identify Defect** and after completed click **Run**.



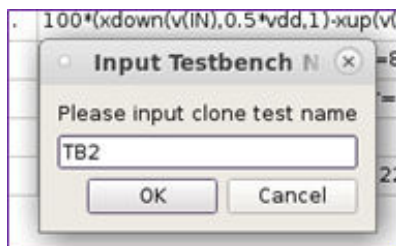
Creating More Measurements and Running What-if Analysis

1. Clone the testbench TB1 to TB2: Select TB1.

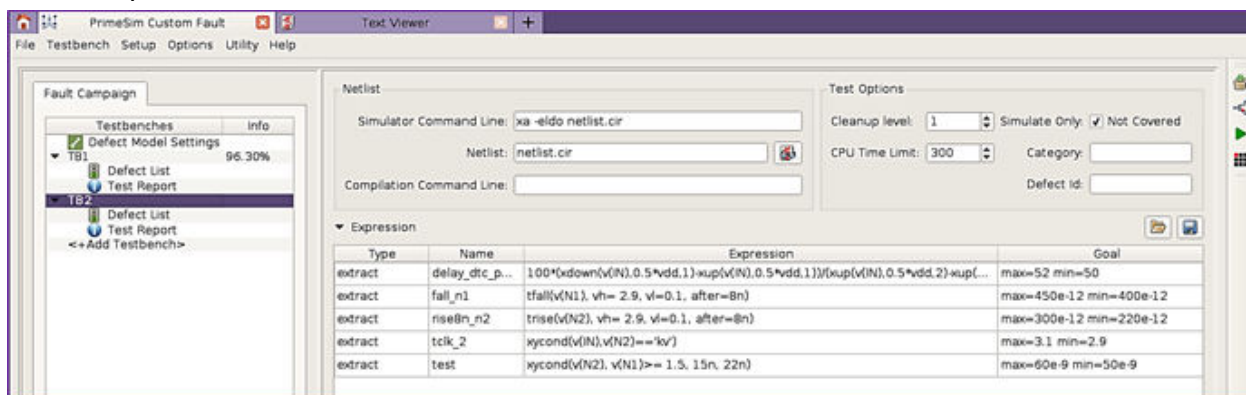
a. Choose **Testbench > Clone Testbench**.



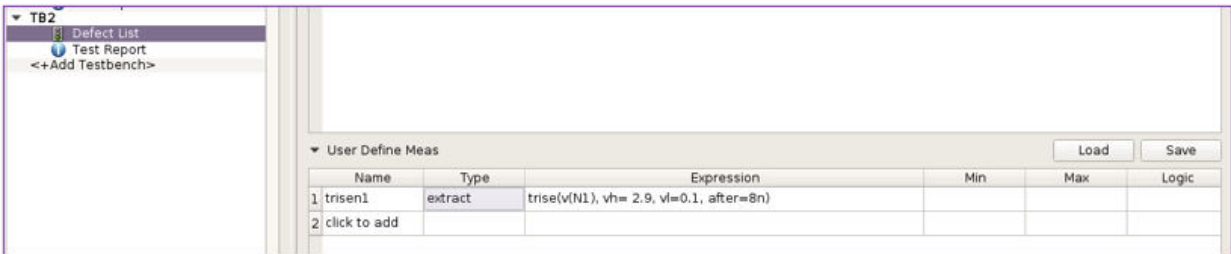
b. Enter a new testbench name, for example TB2.



c. TB1 is copied into TB2. Both testbenches are now visible:

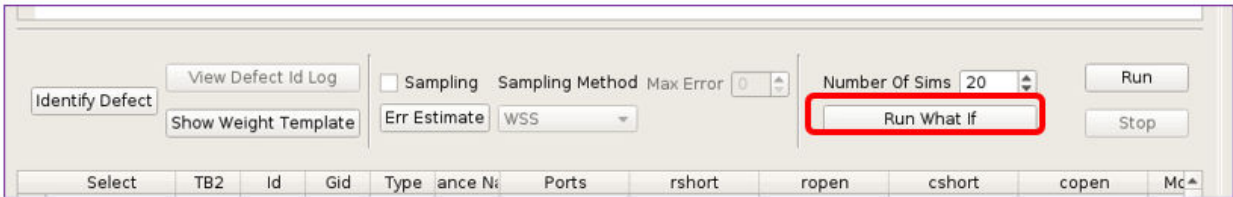


2. In TB2, select Defect List and enter a new measurement in the User Define Meas area.

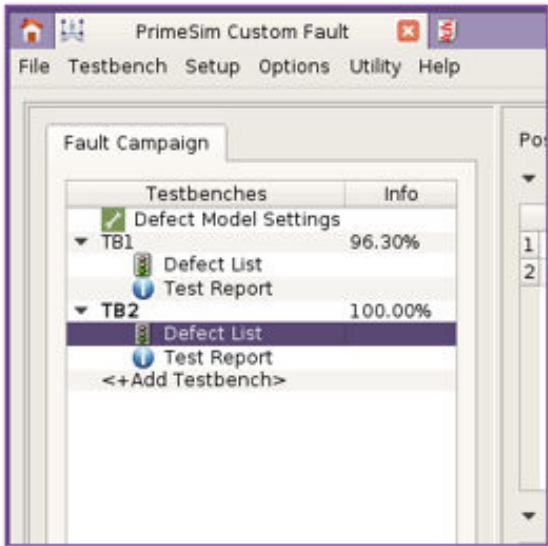


You can also use the **Load** button if you have new measurements in an already existing file. The **Save** button allows you to save the measurements in a file.

3. Click **Run What If**. You can also modify the number of licenses before clicking **Run What If**.

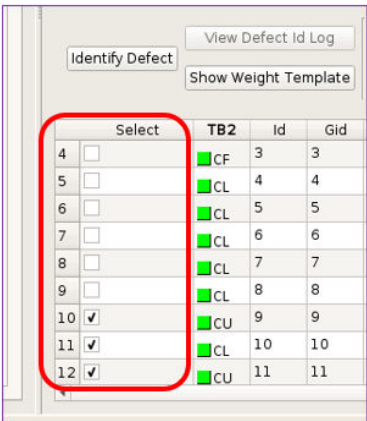


The Custom Fault tool runs the What-if analysis and displays the report. The Fault Campaign tab displays the coverage for both testbenches.

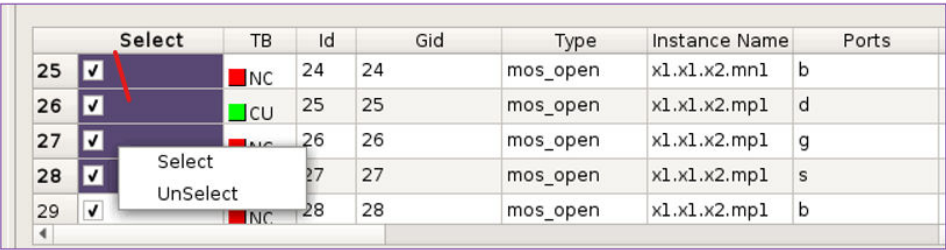


Selecting and Deselecting Defects

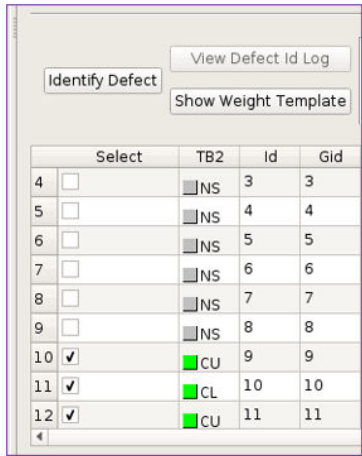
You can run only selected defects. You can ignore the defects by clearing the defect check box if you do not want to rerun.



You can also use the Shift and Ctrl keys. Select the defects using the mouse, Shift + left-click or Ctrl + left-click and when the defects are selected, right-click to choose the defects you want to keep (Select) or ignore (UnSelect).

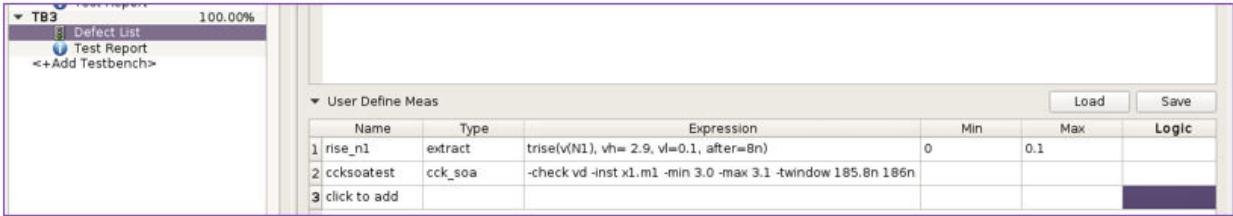


Run What If and the ignored defects are not simulated.



Iterative What-if Runs

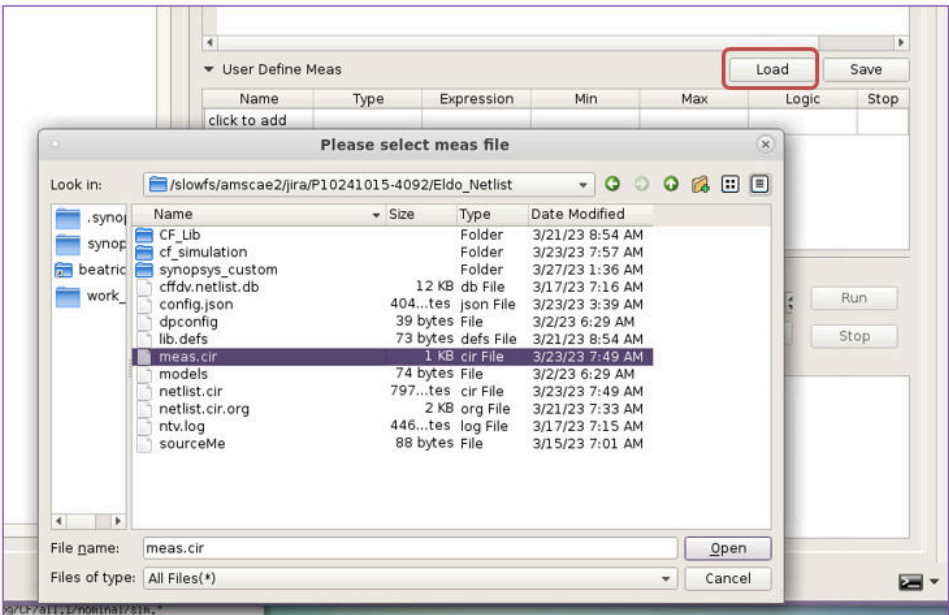
You can clone the second testbench and repeat the steps multiple times. You can also rerun the previous testbenches, select other defects or write new measurements. The following example shows a third testbench cloned with a cck_soa check:



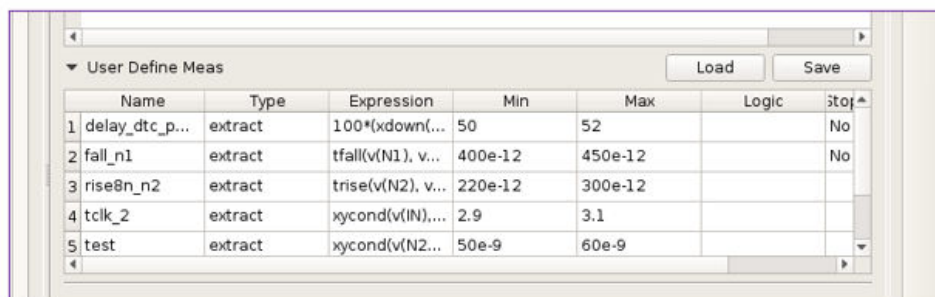
Deleting original measurements

In the Custom Fault App, it is not possible to delete measurements from the original netlist. In case you want to delete measurements, the proposed flow is as follows:

1. Create an external file with measurements. For example, meas.cir. Do not include the measurements and meas.cir in the netlist. It is included using the App.
2. Open PrimeWave Design Environment for Reliability Analysis and the App for Silicon Failure Analysis and write the command line as previously described. No measurement is displayed because there is no measurement in the netlist.

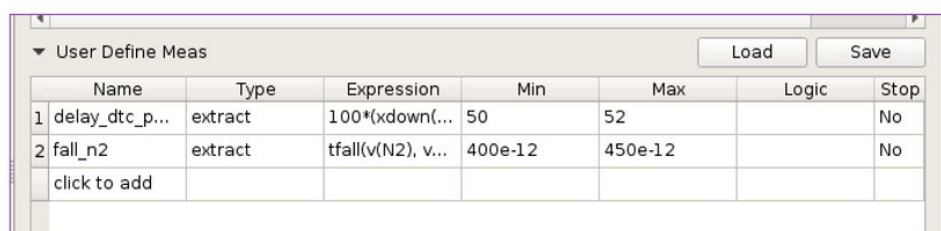


The measurements are displayed.



	Name	Type	Expression	Min	Max	Logic	Stop
1	delay_dtc_p...	extract	100*(xdown(...	50	52		No
2	fall_n1	extract	tfall(v(N1), v...	400e-12	450e-12		No
3	rise8n_n2	extract	trise(v(N2), v...	220e-12	300e-12		
4	tclk_2	extract	xycond(v(IN),...	2.9	3.1		
5	test	extract	xycond(v(N2...	50e-9	60e-9		

3. Identify defects and run Custom Fault as previously described.
4. Clone the testbench for what-if analysis. In the subsequent testbenches you can delete measurements and add new measurements.
 - a. To delete a measurement, double-click on the Name and delete the name, the whole line is deleted.
 - b. In the following example all measurements except one have been deleted, and a new measurement fall_n2 has been added.



	Name	Type	Expression	Min	Max	Logic	Stop
1	delay_dtc_p...	extract	100*(xdown(...	50	52		No
2	fall_n2	extract	tfall(v(N2), v...	400e-12	450e-12		No
	click to add						

5. Run what-if as previously described.

Limitations

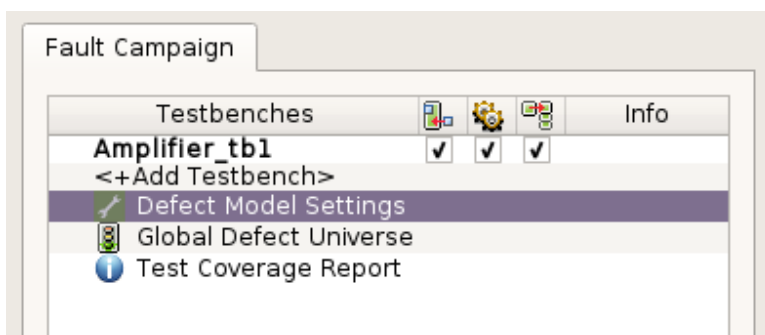
The following are the limitations of the Silicon Failure Analysis in the Custom Fault app:

- The Silicon Failure Analysis in the Custom Fault app is only available with the netlist based flow, it is not available with the OA based flow.
- The CustomFault-Avalon link is available; however, the Avalon trace file cannot be modified when running what-if analysis.
- The cck commands cannot be modified when running the what-if analysis.

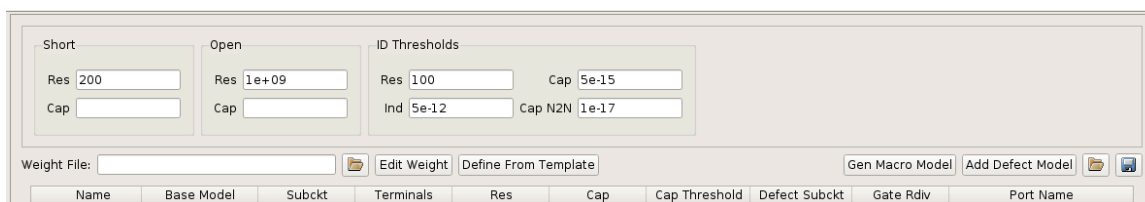
Defect Model Settings

The defect model settings section is used to set default parameters for defect models and to specify user-defined defect models for use in the defect injection campaign.

- Click the **Defect Model Settings** item in the **Fault Campaign** tab to view the defect model settings.



The right side of the application now displays the default settings for **Short** and **Open** defects, the **ID Thresholds**, weight settings, and defect model definition.



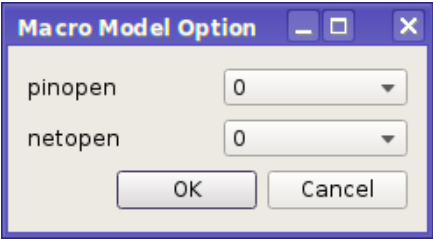
The **Short** and **Open** sections have fields for **Res** and **Cap**. These are the default resistance and capacitance for short and open defects. Leaving a field empty removes that element from the defect model. For example, to have an open defect model that is a complete net disconnect, remove all values from the **Open Res** and **Open Cap** fields. See [Customizing User-defined Defect Models](#) and the `faults` command for more information.

The **ID Thresholds** set the threshold value for identifying defects on primitive R, L, and C elements, as well as net-to-net short thresholds for coupling capacitors. Refer to [Overview of the Post-Layout Flow](#) and the `faults` command for more information.

Generating Defect Models for Macro Models

1. You can make the tool create macro models automatically by clicking the **Gen Macro Model** button. You can also choose whether you want **pinopen** and **netopen** macro models to be created.

This opens the **Macro Model Option** dialog box.



Field	Description
pinopen	Choose one of the following settings: ◦ 0: (default) No subckt_pinopen models are created, only subckt_short and subckt_open. ◦ 1: subckt_short and subckt_pinopen models are created. ◦ 2: subckt_short, subckt_open, and subckt_pinopen models are created.
netopen	Choose one of the following settings: ◦ 0: (default) no subckt_netopen models are created, only subckt_short and subckt_open. ◦ 1: subckt_short and subckt_netopen models are created. ◦ 2: subckt_short, subckt_open, and subckt_netopen models are created.

2. After you have completed the dialog box, click **OK** to add the new defect model to the table.

Adding a New Defect Model

1. Add a new user-defined defect model by clicking the **Add Defect Model** button.
This opens the **Add Defect Model** dialog box.

Add Defect Model

Name*

mos_short_gds_500

Base Model

mos_short

Subckt

Terminals

g d s

Res

500

Cap

Cap_th

Defect Model

Rdiv

False

Port Name

OK

Cancel

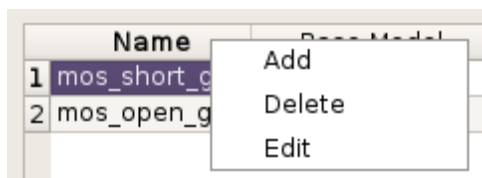
Field	Description
Name	The user-defined name for the defect model. A name must always be specified.
Base Model	Select the base model type from the drop-down list. The other fields in the dialog box enable or disable depending upon the base model type. See Supported Built-in Defect Models and Pin Open and Net Open Defect Models .
Subckt	If the base model is of the <code>subckt</code> type, then you must specify the <code>.subckt</code> name.
Terminals	The terminal names for the <code>primitive</code> or <code>subckt</code> are listed here. - For <code>primitive</code> MOSFETs, the terminal names are <code>g</code>, <code>d</code>, <code>s</code>, or <code>b</code>. - For <code>subckt</code> defect types, the terminal names from the <code>.subckt</code> definition must be used.
Res	Specifies the resistor value to be used in the open or short defect model. This field can be left empty to indicated that no resistor is used in the defect model.
Cap	Specifies the capacitor value to be used in the open or short defect model. This field can be left empty to indicated that no capacitor is used in the defect model.

Field	Description
Cap_th	This field is only enabled if the base model is <code>net_to_net_short</code> . It sets the coupling capacitor threshold for identifying the net-to-net-short.
Defect Model	If a custom SPICE defect model is to be applied, then this field must be set to the <code>.subckt</code> name of the defect model that is applied.
Rdiv	This field is only enabled if the base model type is <code>mos_open</code> or <code>subckt_open</code> . It applies the resistor divider gate open defect model (see Resistor Divider Gate Open Defect Model). For <code>subckt_open</code> types, you must define the Port Names for the gate, drain, and source of the subckt.
Port Name	This field is only enabled if the base model is <code>subckt_open</code> and Rdiv is <code>true</code> . You must define the <code>.subckt</code> port name of the gate connection of the subckt MOS.

- After you have completed the dialog box, click **OK** to add the new defect model to the table.

The defect models can be edited by clicking directly in a table cell or right-clicking and selecting **Edit**. Selecting edit reopens the **Add Defect Model** dialog box.

You can also delete a defect model by right-clicking in a table cell and selecting **Delete**.



After you have defined the defect models, you can save them to a file by clicking **Save Defect Model** . Similarly, saved defect model definitions can be loaded from a file by clicking **Load Defect Model**  and selecting the file.

Defect Weights

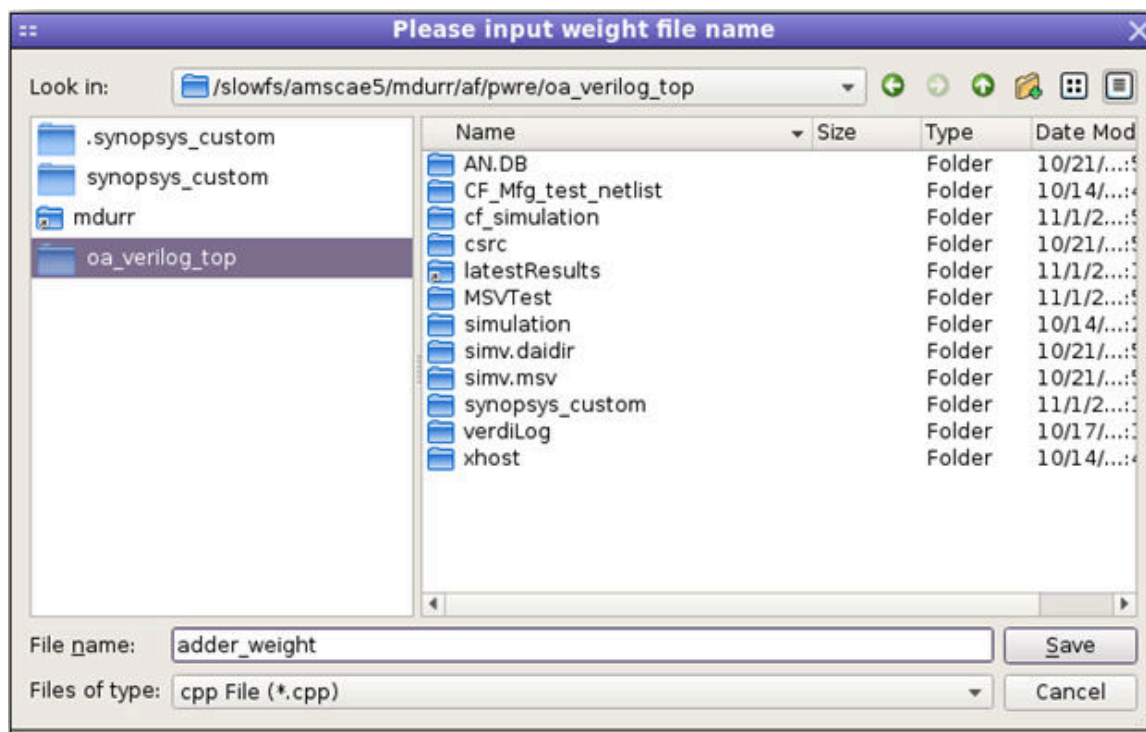
You can assign weights to each defect (see [Specifying Weights for Defects](#) for more information).

- Open an existing weight file by clicking **Load Weight File**  and selecting the weight definition file. After you have selected a weight file, you can view and edit it by clicking the **Edit Weight** button.

You can also define a new weight definition file from an automatically created template. To create a defect definition from a template:

1. Identify the defect universe (see [Defect Universe Definition](#)).
2. Click the **Define From Template** button.

This opens the following dialog box where you can specify a new **File name**.



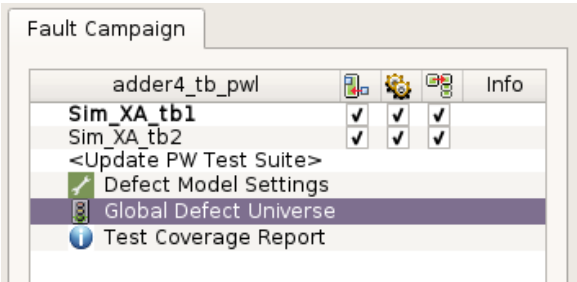
3. Enter a **File name** and click **Save**.

A text editor opens and the weight definition functions can be edited as needed (see [Specifying Weights for Defects](#)).

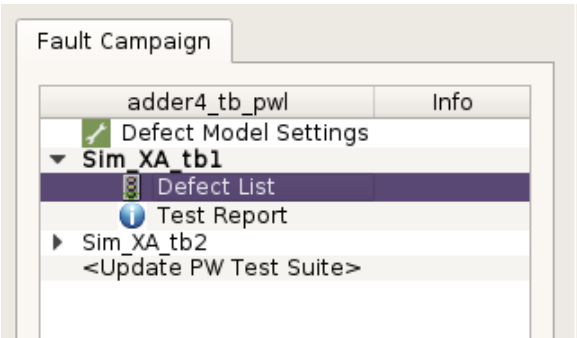
Defect Universe Definition

You must identify a defect universe from the circuit under test before defect injection and fault simulation can occur.

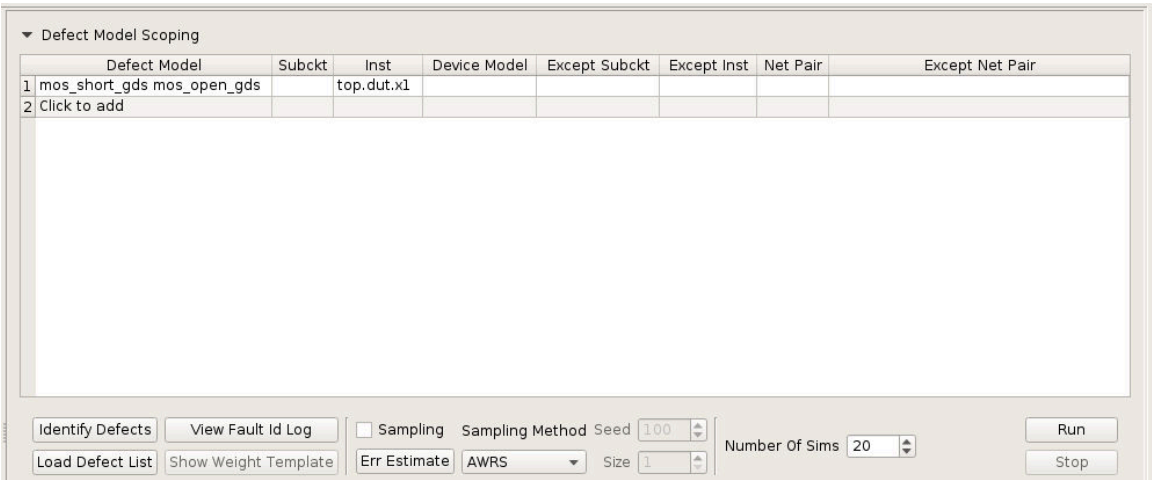
1. In manufacturing test applications, click **Global Defect Universe** in the **Fault Campaign** tab.



In functional safety applications, the defect list is subordinate to the testbench.



The right-hand side of the display shows the **Defect Model Scoping** table.

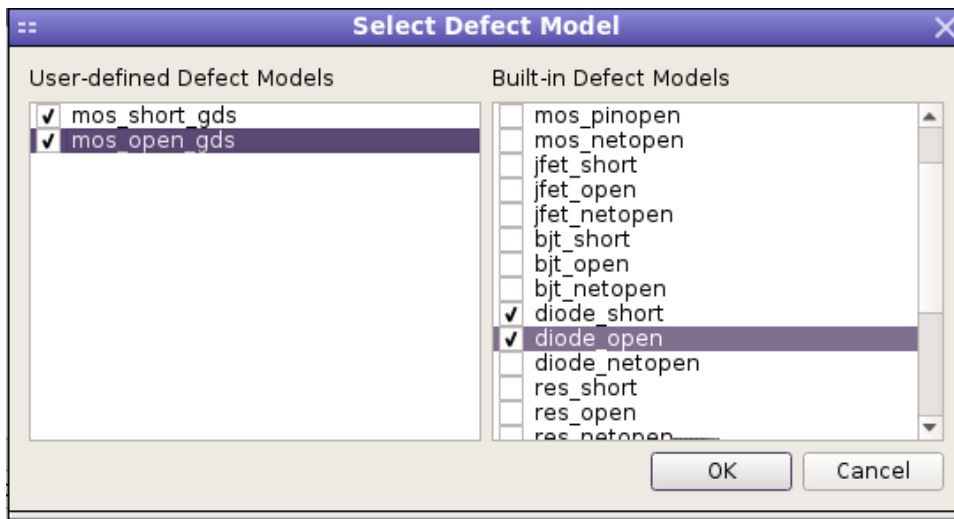


Note:

If you have already defined a list of defects for the defect universe, see [Manual Injection of Defects](#).

2. Click **Click to add** in the table to make a defect scoping definition.

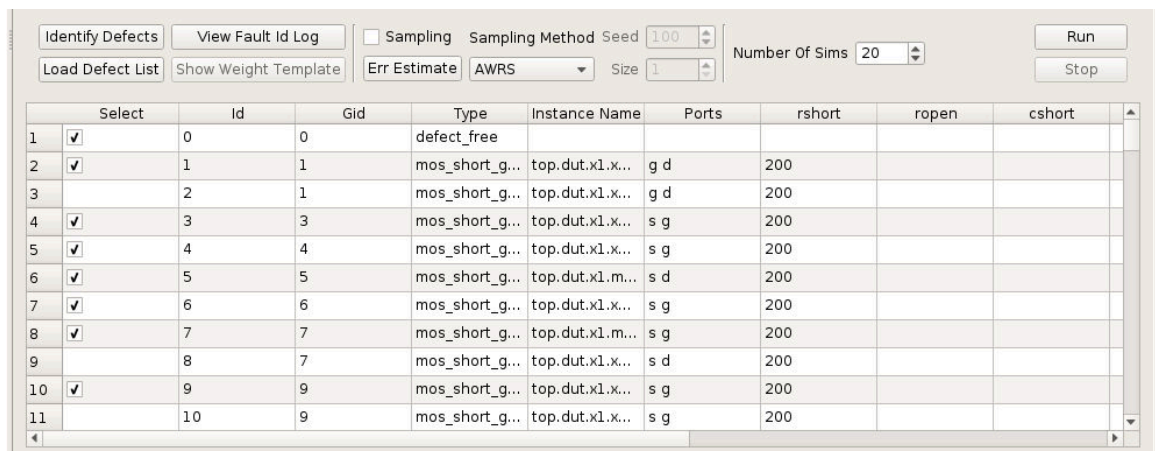
This opens the **Select Defect Model** dialog box to apply defect models.



3. You can select defect models from custom defect definitions defined on the [Defect Model Settings](#) screen or from the Custom Fault built-in default defect models.
4. Click **OK** to add these models to the table.

The circuit scope that these models are applied to can be specified by clicking in any of the table columns and providing the relevant identifiers.

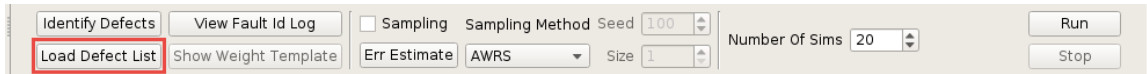
5. After all defect models and scopes are applied, you can identify and create the defect universe. Click the **Identify Defects** button below the **Defect Model Scoping** table to perform the identification step. The defect universe table is populated with all of the identified defects and their parameters.



The defect listing provides the details of each defect in the defect universe. You can now proceed to [Launching the Defect Campaign](#).

Manual Injection of Defects

Instead of specifying defect models one by one to the **Defect Model Scoping** table and then clicking the **Identify Defects** button to populate the defect universe table, you can load a defect list you've already defined by clicking the **Load Defect List** button, then proceed to [Launching the Defect Campaign](#).



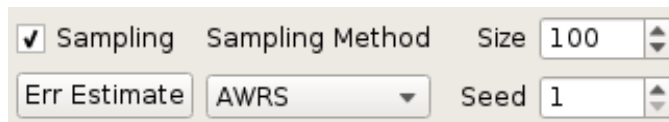
Launching the Defect Campaign

After you have identified the defect universe, you can launch the defect analysis campaign. You can conduct the campaign as *exhaustive*, simulating all defects in the universe, or you can select a sampling method (see [Comparison of Fault Campaign Methods](#)).

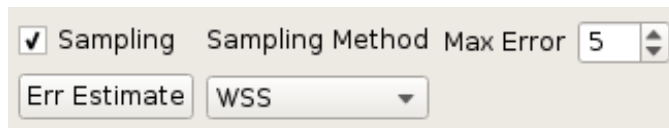
- [Defect Sampling](#)
- [Launching Simulations](#)

Defect Sampling

1. Click the **Err Estimate** button to view the Adaptive Weighted Random Sampling (AWRS) and Weight Sorted Sampling (WSS) estimated error graphs (see [Estimating Error and Sample Size](#) for more information).
2. Click the **Sampling** check box to enable sampling and then choose the sampling method:
 - For AWRS, you can set the sample **Size** and Random Number Generator (RNG) **Seed**:



- For WSS, you can specify the **Max Error** as a percentage:



Launching Simulations

1. Select the maximum number of parallel simulations to execute during the defect analysis campaign by setting the **Number Of Sims**.



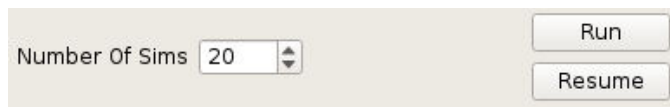
A screenshot of a control panel with a light beige background. On the left, the text "Number Of Sims" is followed by a text box containing the number "20" and a small up/down arrow icon. To the right of this are two buttons: "Run" on top and "Stop" on the bottom, both with a light gray gradient and rounded corners.

Note:

The number of parallel simulations that can be executed is limited by tool license mode and availability.

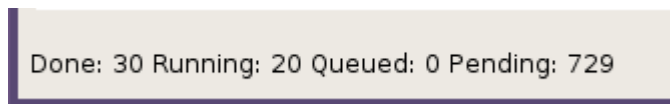
2. Click the **Run** button to launch the defect analysis campaign.

The campaign can be halted by clicking the **Stop** button. A stopped campaign can be resumed by clicking the **Resume** button.



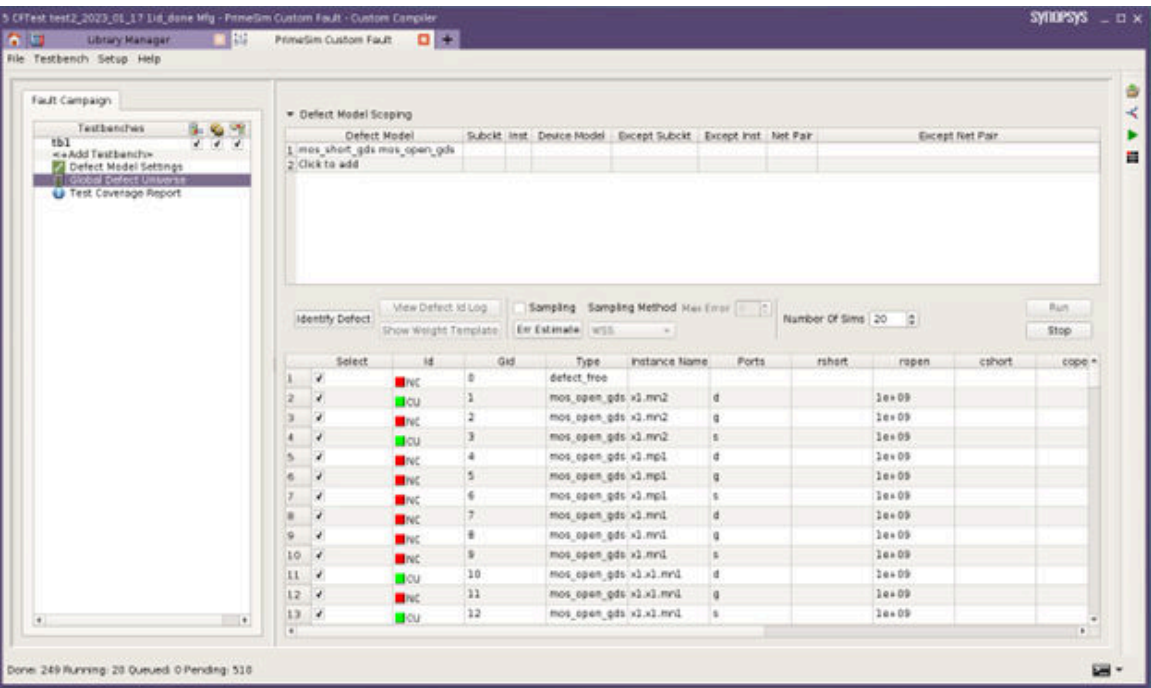
A screenshot of a control panel similar to the one above, but with a different set of buttons. It has the "Number Of Sims" text box with "20" and an up/down arrow. To the right are two buttons: "Run" on top and "Resume" on the bottom, both with a light gray gradient and rounded corners.

The status of the running defect analysis campaign is displayed in the status bar at the bottom of the display.



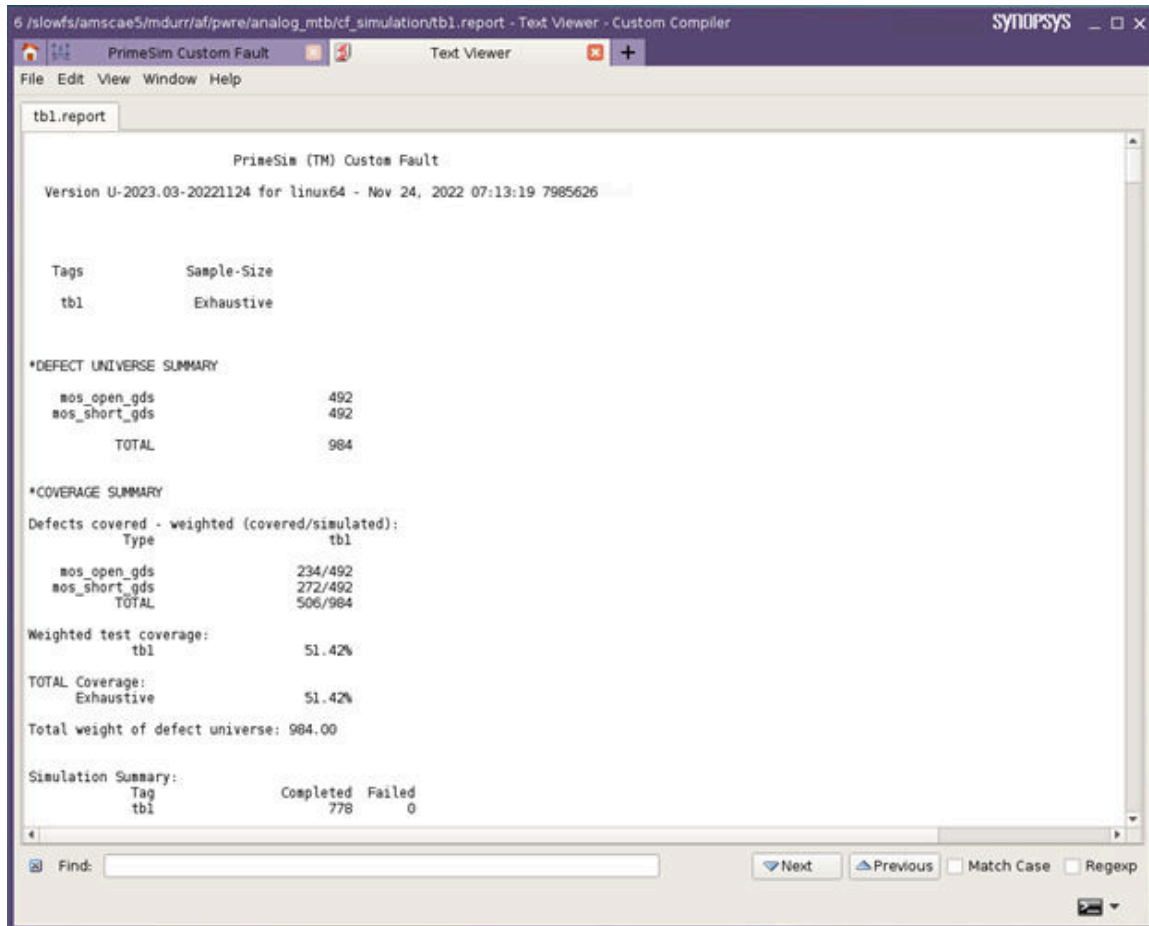
A screenshot of a status bar with a light beige background and a dark purple border. It contains the text "Done: 30 Running: 20 Queued: 0 Pending: 729" in a dark gray font.

The defect list dynamically updates as the campaign progresses.



Coverage Reports

When the defect analysis campaign completes, a new text viewer tab opens in the environment. This text viewer displays the contents of the test coverage report file or diagnostic coverage report file. This file is identical to the coverage report file generated from a batch mode execution of the Custom Fault tool (see [Custom Fault Output Files](#))



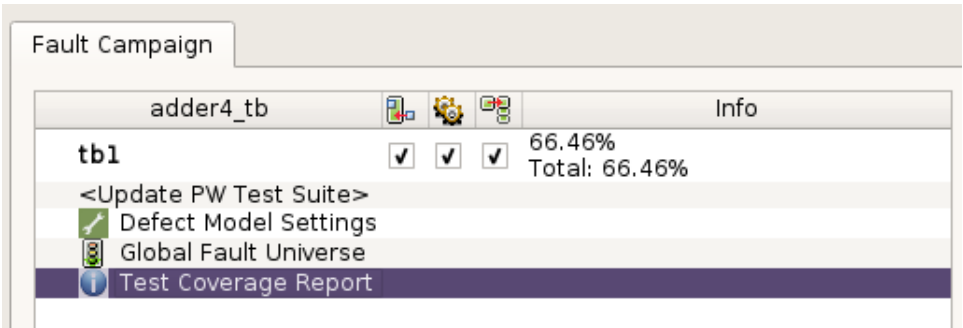
Analyzing Defect Campaign Results

How you analyze defect campaign results depends on which application mode you used in the defect campaign.

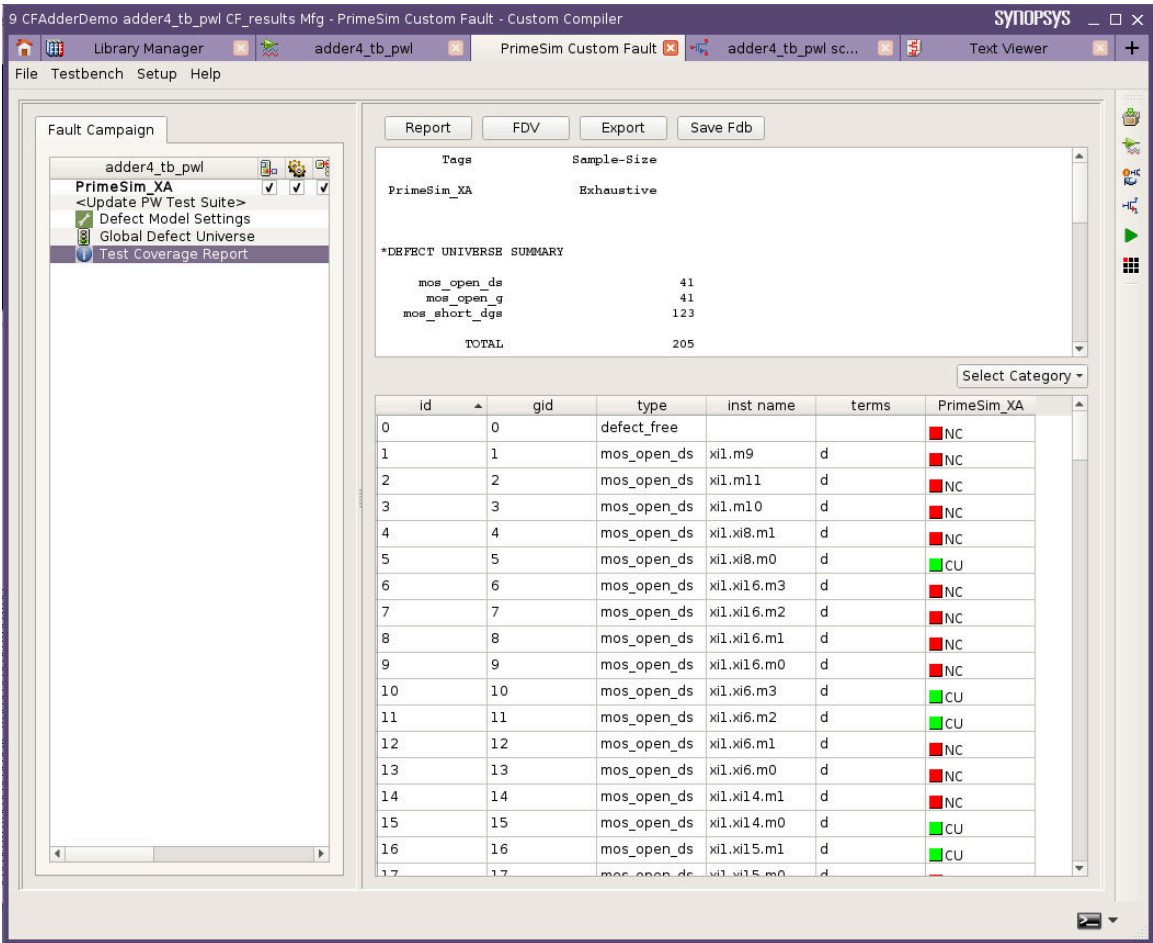
Analyzing Defect Campaign Results in Manufacturing Test Mode

In manufacturing test mode, you can analyze the fault campaign results in more detail on the **Test Coverage Report** window.

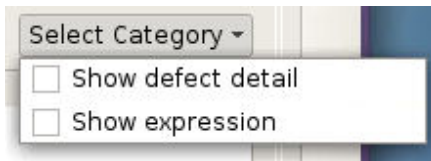
To view this screen, click the **Test Coverage Report** in the **Fault Campaign** tab.



The right side of the screen displays the test coverage report summary. This is the same summary that appears in the `.report` file (see [Test Coverage Report \(Single Testbench\)](#)).



You can use the **Select Category** menu to choose which columns to display.

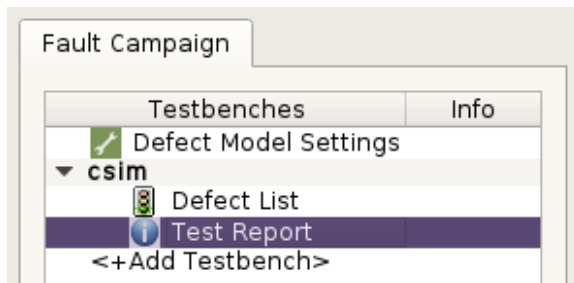


- If neither category is selected, only the **id**, **gid**, and session name columns are displayed.
- If **Show defect detail** is selected, the **type**, **inst name**, and **terms** columns are also displayed.
- If **Show expression** is selected, the extra expression columns of all sessions are also displayed.

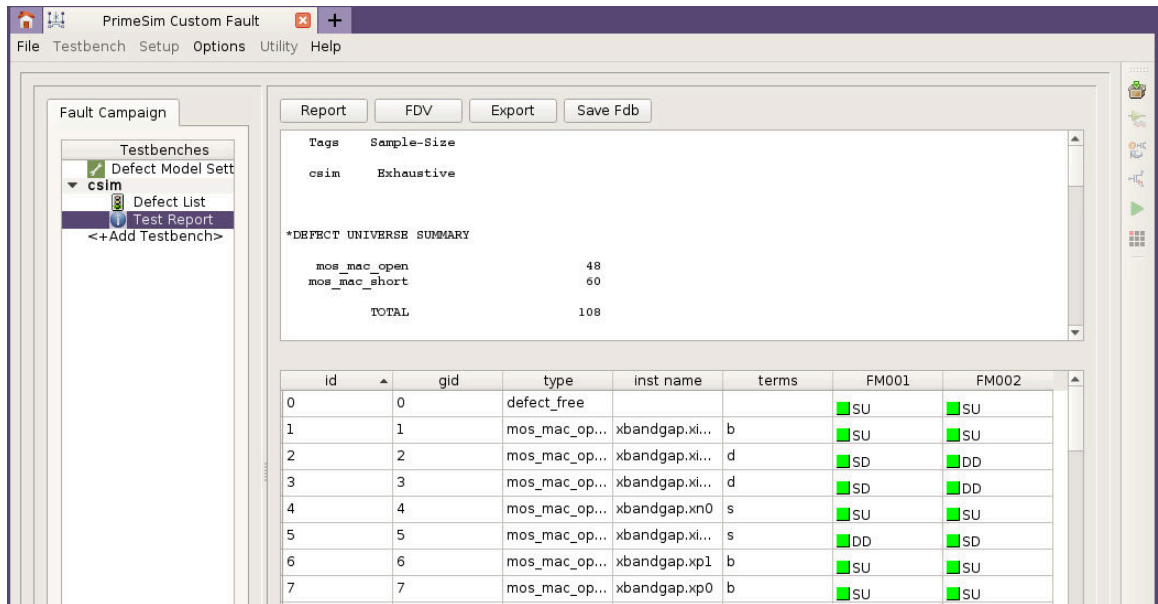
Analyzing Defect Campaign Results in Functional Safety Test Mode

In functional safety test mode, you can analyze the fault campaign results in more detail on the **Diagnostic Coverage Report** window.

To view this screen, click the **Test Report** in the **Fault Campaign** tab.



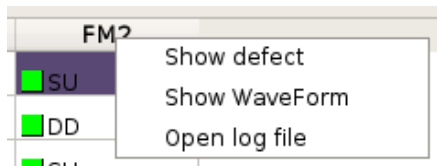
The right side of the screen displays the diagnostic coverage report summary. This is the same summary that appears in the `.diagnostic_report` file (see [Diagnostic Coverage Report](#)).



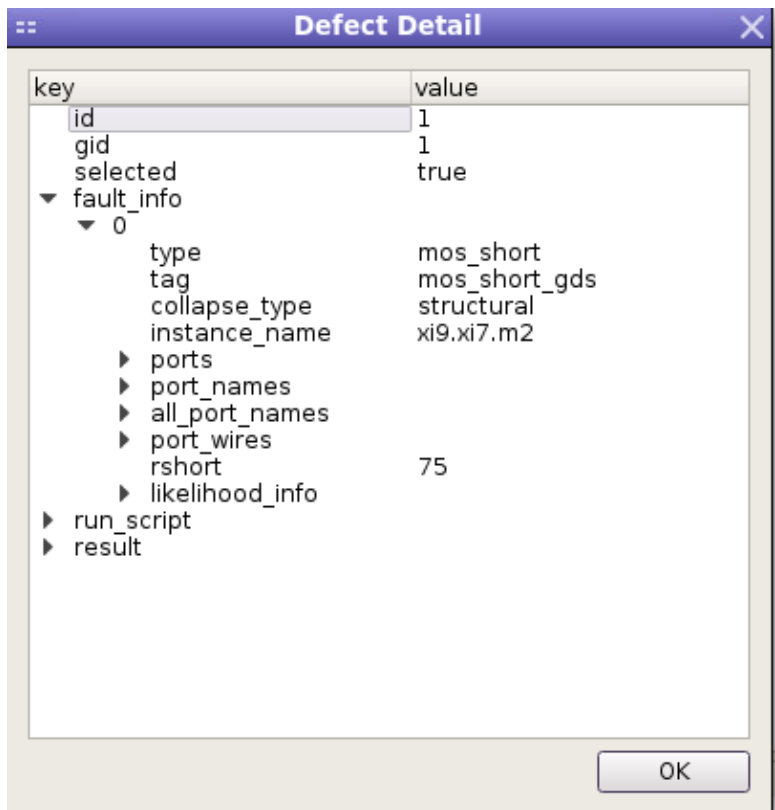
The report summary screens provide the following buttons:

Button	Function
Report	Displays the complete text report file (see Coverage Reports). The bottom portion of the screen shows a detailed results table with the coverage status of each simulated defect.
FDV	Launches the Fault Database Viewer .
Export	Exports the defect campaign results to a file in CSV (Comma Separated Value) format.
Save Fdb	Saves the defect campaign results in FDB (Fault Database) format.
Open NCA	(Manufacturing test only) Displays the Not-covered Defect Analysis file.

You can view the details of each defect by right-clicking a defect in the table and selecting **Show defect**.



This opens a **Defect Detail** dialog box.

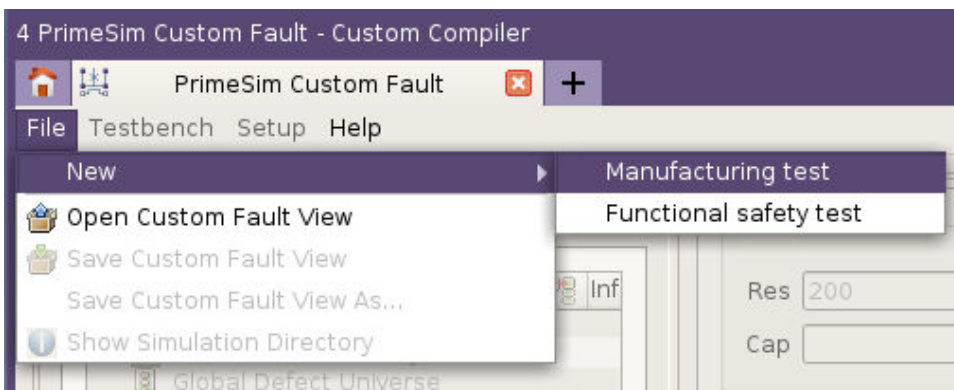


You can view the log file for each defect by right-clicking a defect in the table and selecting **Open log file**. This opens a **Text Viewer** tab.

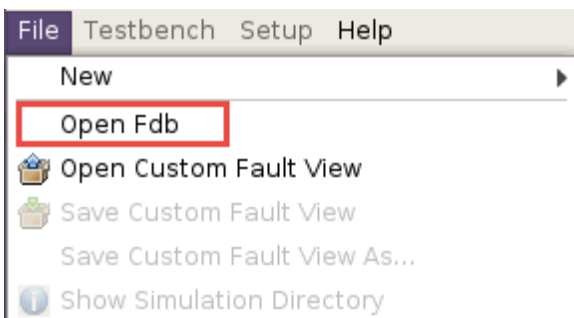
Loading an Existing Fault Database

If you have already run a fault campaign in Custom Fault batch mode and want to analyze the results using the Custom Fault App, you can load the fault database (.fdb) file into the App as follows.

1. Open the App and choose **File > New > Manufacturing test** or **File > New > Functional safety test** as appropriate.



2. Choose **File > Open Fdb** to load the existing fault database file.



3. Analyze the results as described in [Analyzing Defect Campaign Results](#).

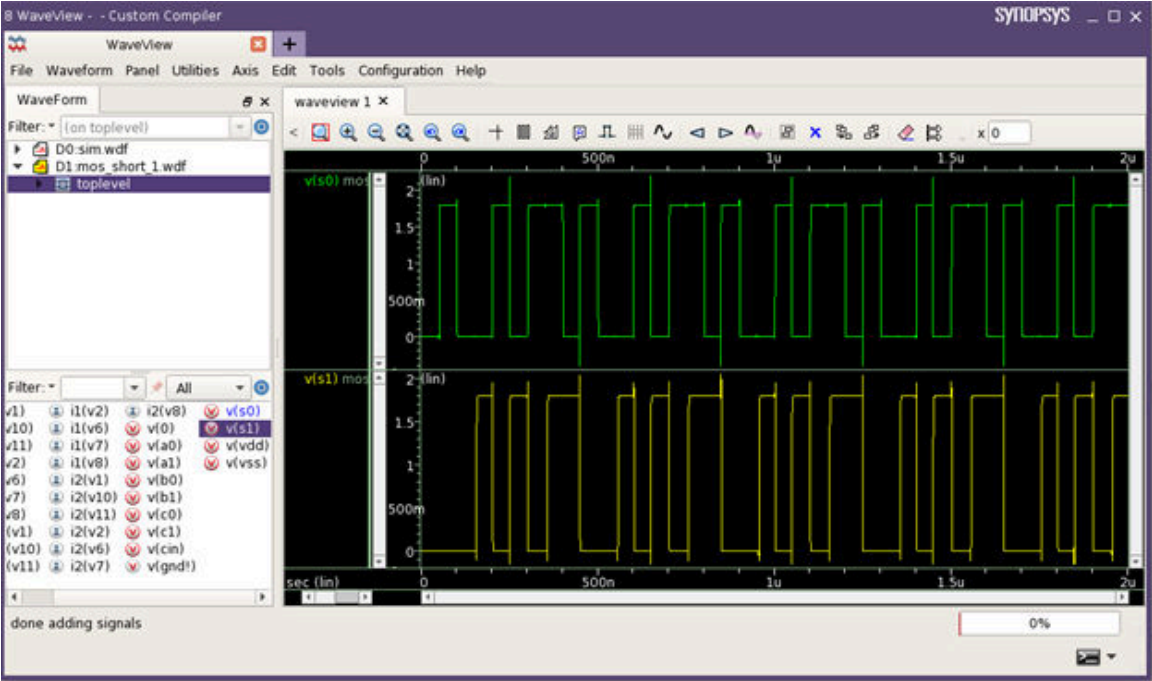
Note:

It is not recommended to re-run the defect campaign when an existing fault database file has been loaded. The feature is only for analyzing the results.

Viewing Defect Simulation Waveforms

If waveforms were saved during the fault campaign, you can view them for each defect.

- ▶ Right-click in the testbench column on the row for the defect of interest and select **Show Waveform**. This opens a WaveView window with the defect's waveform file and defect_free waveform file (if available).

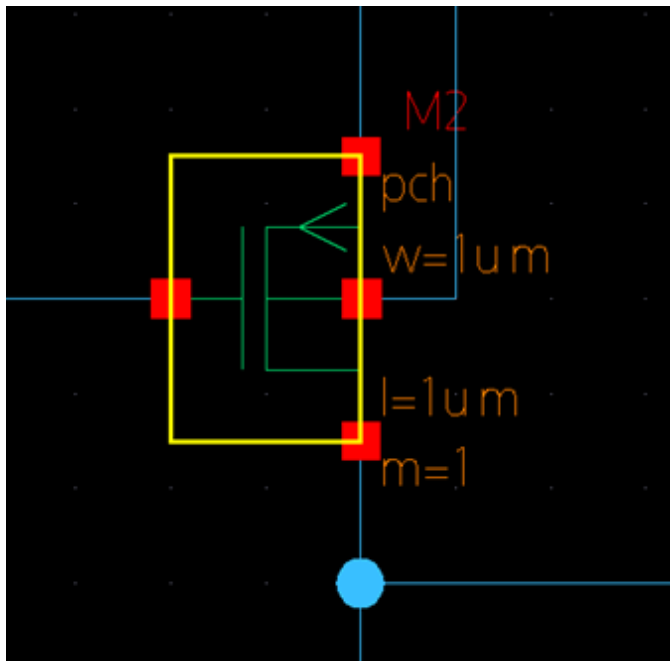


Schematic Highlighting

- ▶ To see the instance associated with any defect in the Schematic Editor, double-click the defect row in the results table.

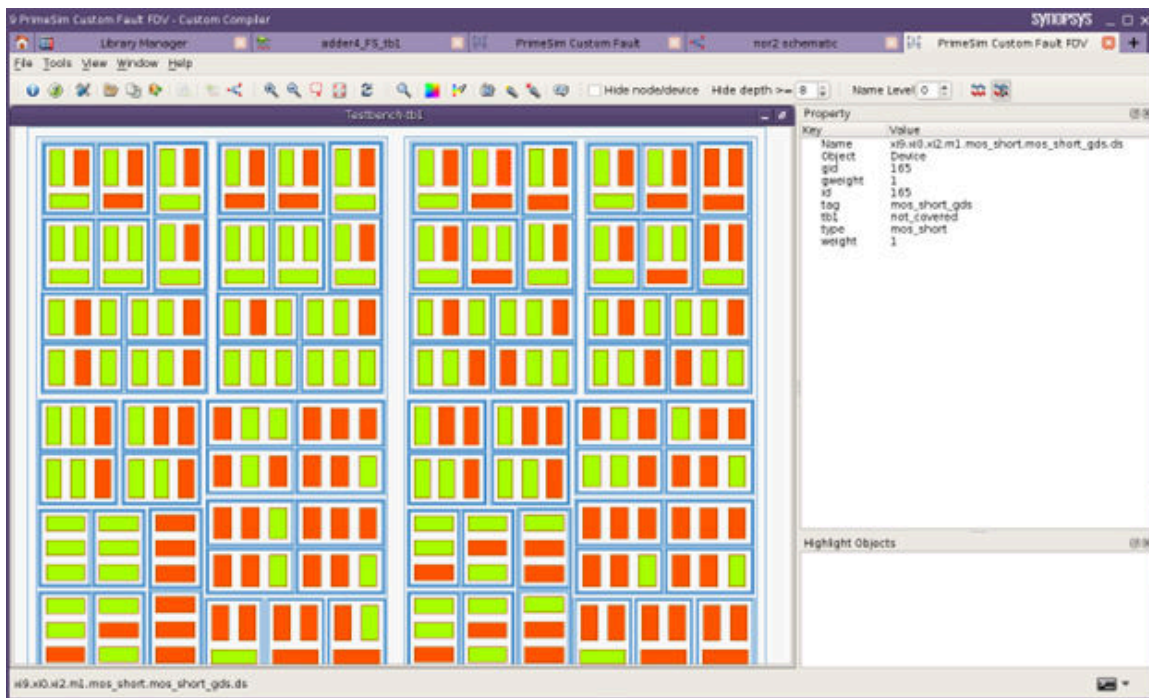
	id	gid	type	inst name	terms	tbl
1	0	0	fault_free			■ NC
2	1	1	mos_short_g...	xi9.xi7.m2	d s	■ NC
3	2	2	mos_short_g...	xi9.xi5.m0	d g	■ CU
4	4	4	mos_short_g...	xi9.xi0.xi2.m0	g d	■ CU
5	6	6	mos_short_g...	xi9.xi6.m2	g s	■ NC
6	7	7	mos_short_g...	xi9.xi4.m0	d g	■ CU
7	9	9	mos_short_g...	xi9.xi0.xi3.m0	g d	■ CU

The schematic window opens with a highlight box placed on the associated instance.



Analyzing Coverage With the Fault Database Viewer

- You can launch the Fault Database Viewer (FDV) by clicking the **FDV** button at the top of the report summary screen. FDV opens in a separate tab.



See [Fault Database Viewer](#) for specific instructions on operating FDV for defect analysis. You can also invoke schematic highlighting and waveform browsing from FDV.

Fault Database Viewer

Use the Fault Database Viewer (FDV) to display Custom Fault results graphically. You can invoke the FDV from the Custom Fault App after the defect campaign has completed.

The FDV is described further in the following sections:

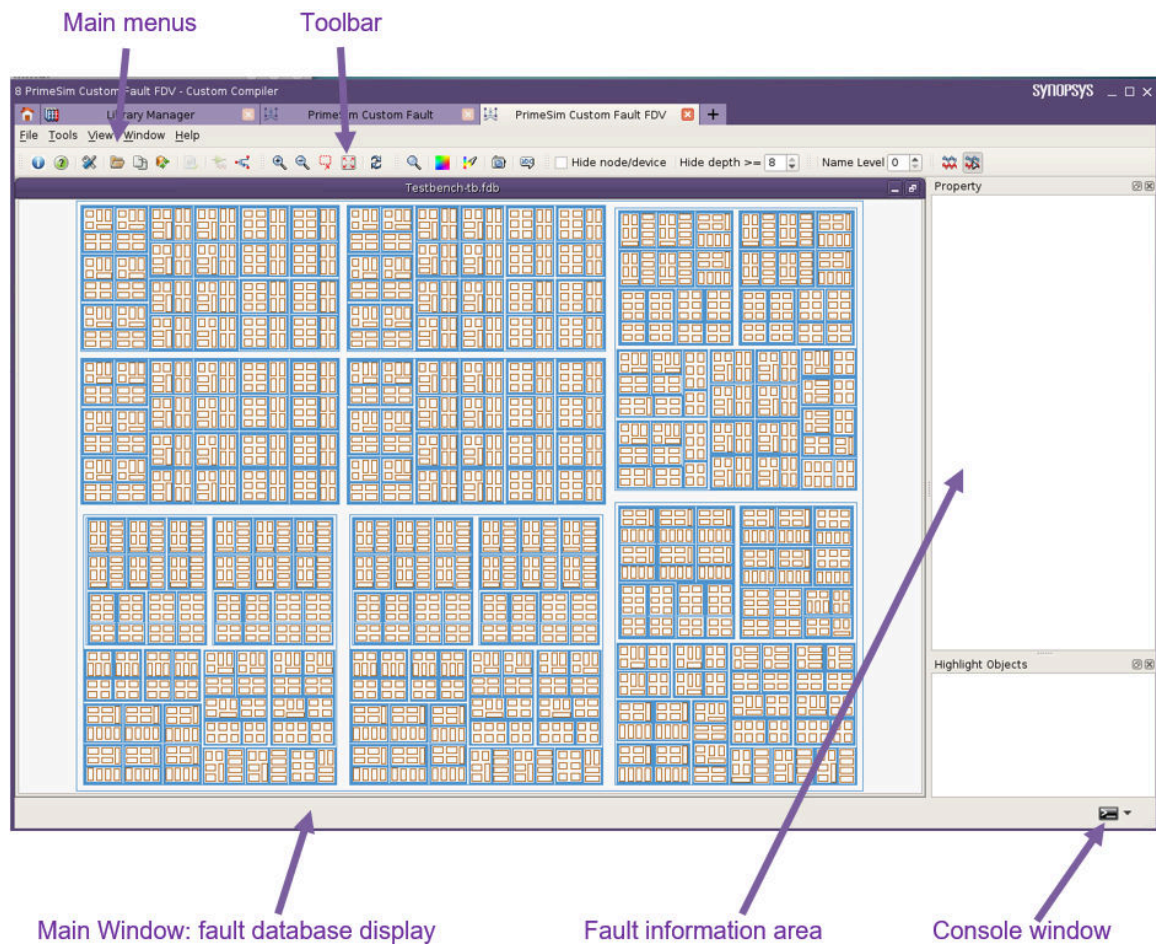
- [Invoking the FDV](#)
- [Displaying Defects](#)
- [Outputting Results](#)
- [Windows Manipulation](#)

- [Other Display Options](#)
- [Displaying Waveforms](#)

Invoking the FDV

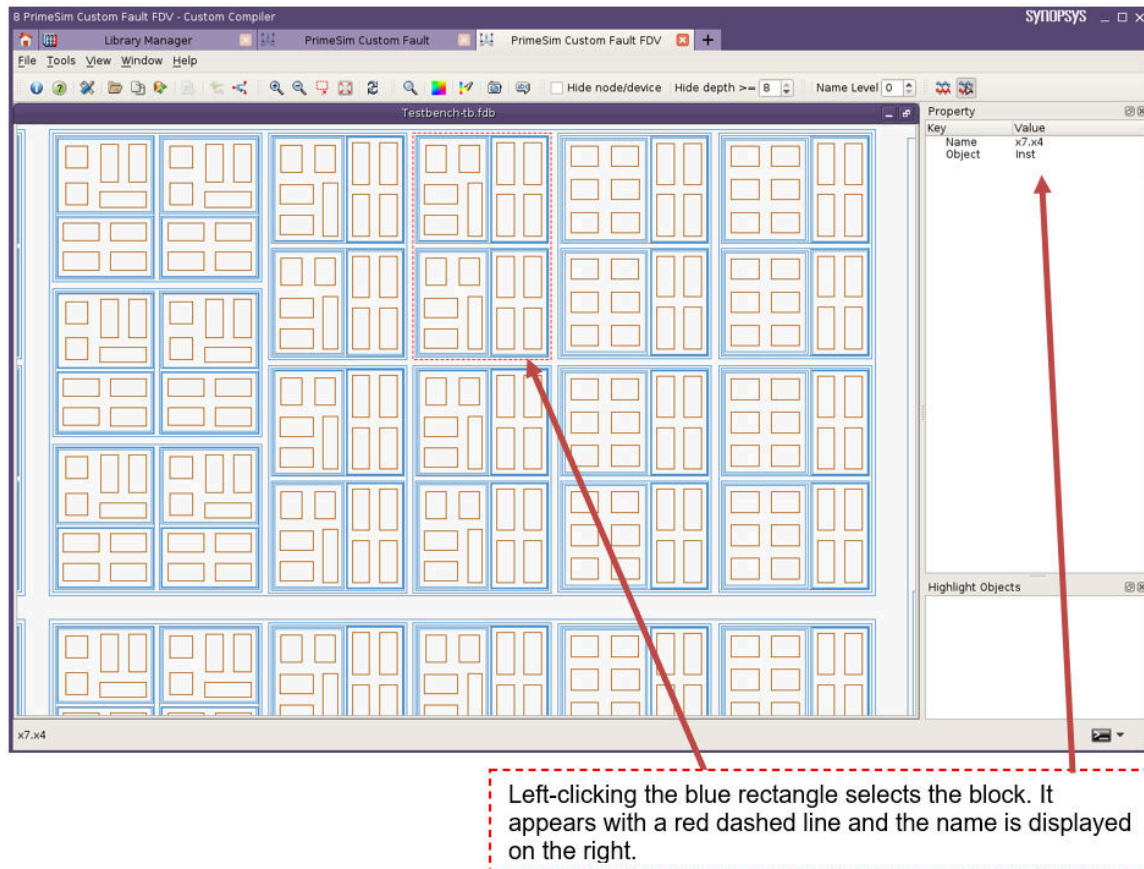
- Click the **FDV** button displayed at the top of the either the **Test Coverage Report** or **Diagnostic Coverage Report** window (see [Analyzing Defect Campaign Results](#)).

FDV displays a flat view of the defects. The left side shows a flat view of the Custom Fault database while the **Property** window lists details of the selection being displayed.



Each blue rectangle represents an instance of the circuit. All instances of the same subcircuit are grouped within the same blue rectangle. The top blue rectangle is the top level of your netlist, then the next level of blue rectangles is the first level of hierarchy, and so on, until the lowest level which is the device itself.

Left-click a blue line to highlight a blue rectangle with a red dashed line and display information about the instance in the **Property** window.



The following topics describe basic functionality of the FDV interface:

- [Information and Help](#)
- [Zooming In and Out](#)
- [Changing the Colors](#)

Information and Help

The first two toolbar icons provide the following functions:

-  – Information about the tool version
-  – Help on the tool usage

You can also choose the same functions from the **Help** menu:

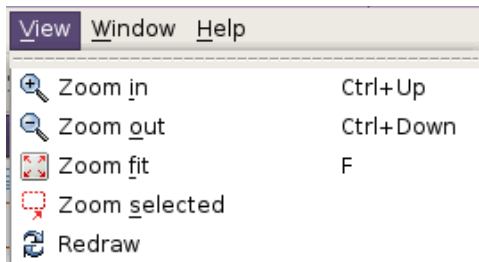


Zooming In and Out

► You can zoom in and out by clicking the icons in the top banner, or you can zoom in by right-click-dragging to select the zoom area.

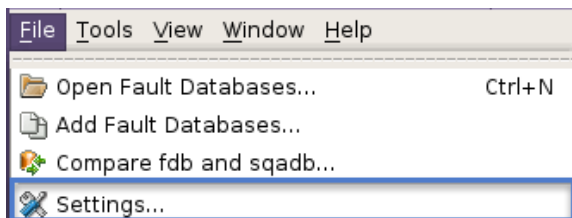
-  – Zoom in
-  – Zoom out
-  – Zoom in on the selected area
-  – Return to the full view (or type the **F** key)

You can also choose all of the same commands from the **View** menu:

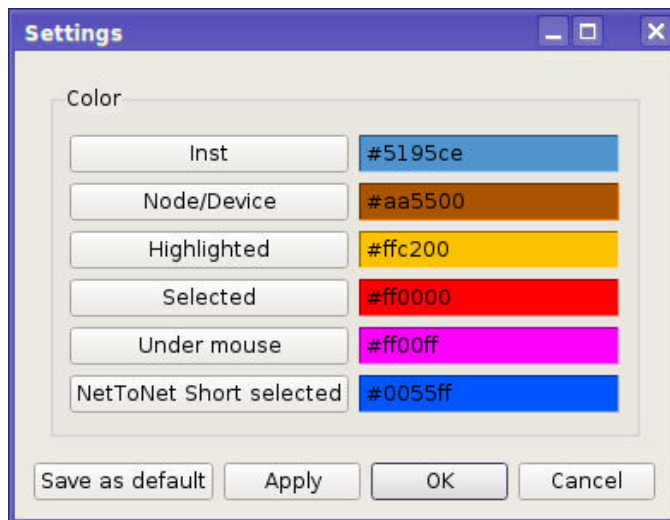


Changing the Colors

1. You can change the default colors by clicking **Settings**  or choosing **File > Settings**:



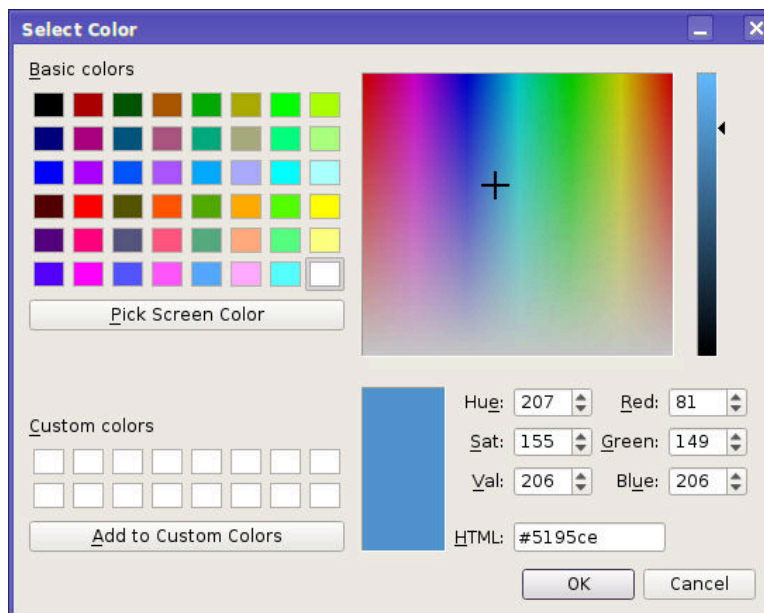
This displays the **Settings** dialog box.



2. Click one of the following and modify the color:

- **Inst** to change the default blue color
- **Selected** to change the default red color used when you select a block
- **Highlighted** to change the default golden color used to highlight defects

Clicking any of these displays the **Select Color** dialog box.



3. Select the desired color and click **OK**, then click **OK** again in the **Settings** dialog box.

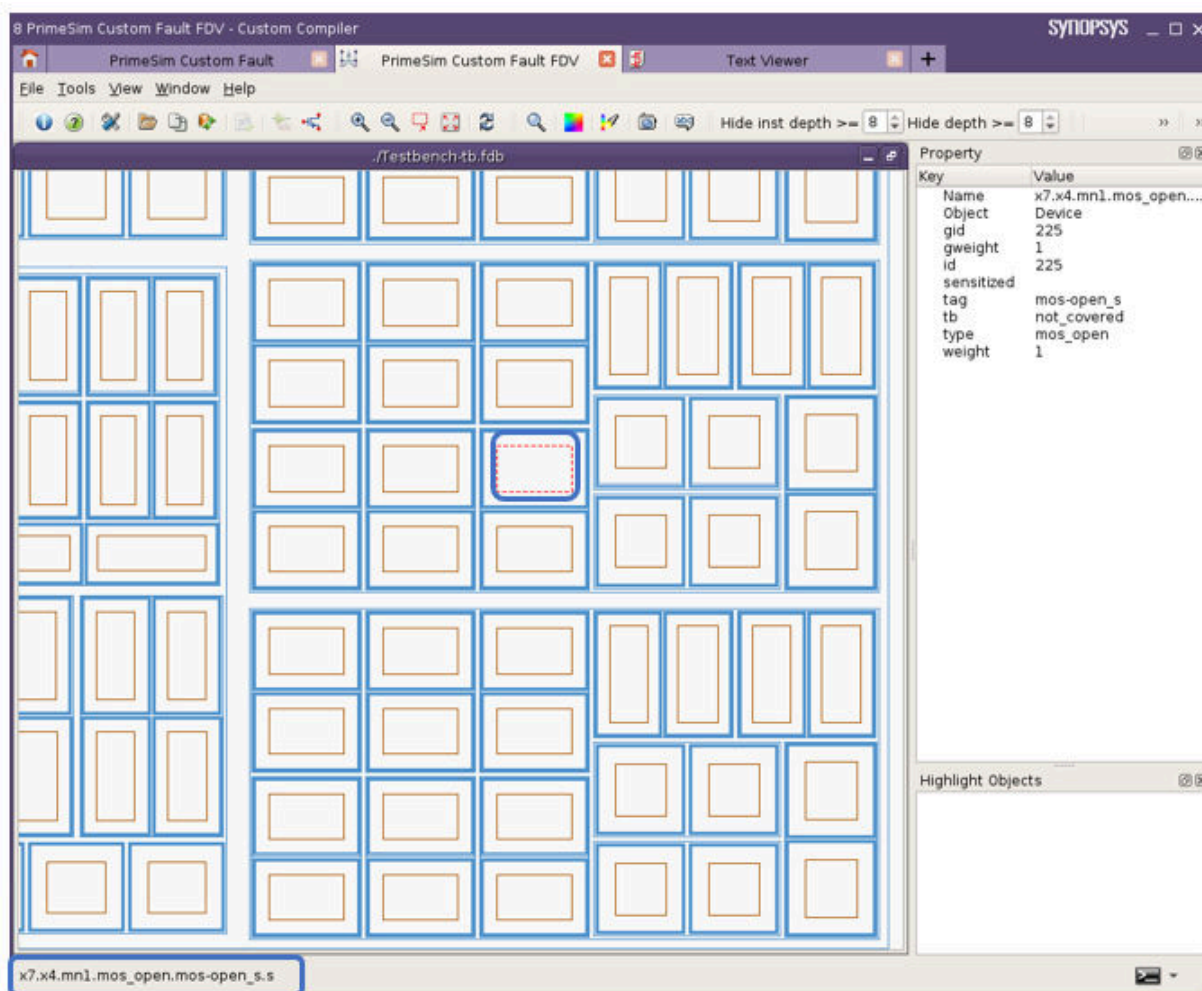
The new color is only applied when you click **Refresh**  to refresh the FDV display.

Displaying Defects

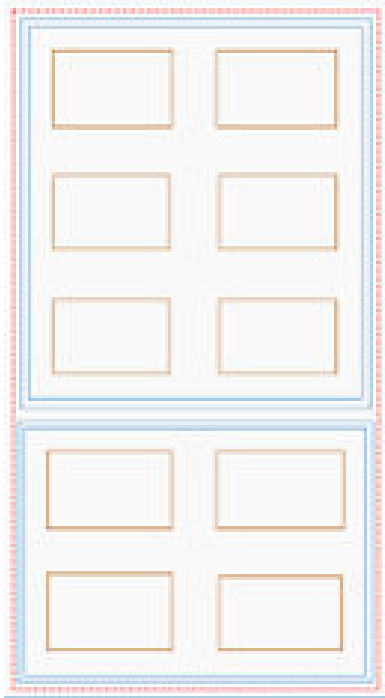
By default, the FDV represents defects as brown/gold rectangles.

In the following screen capture, the source `mos_open` defect on MOS `x7.x4.mn1` is highlighted with a red dashed line. The defect name is displayed in the **Property** window on the right and also at the bottom

Figure 72 Displaying defects

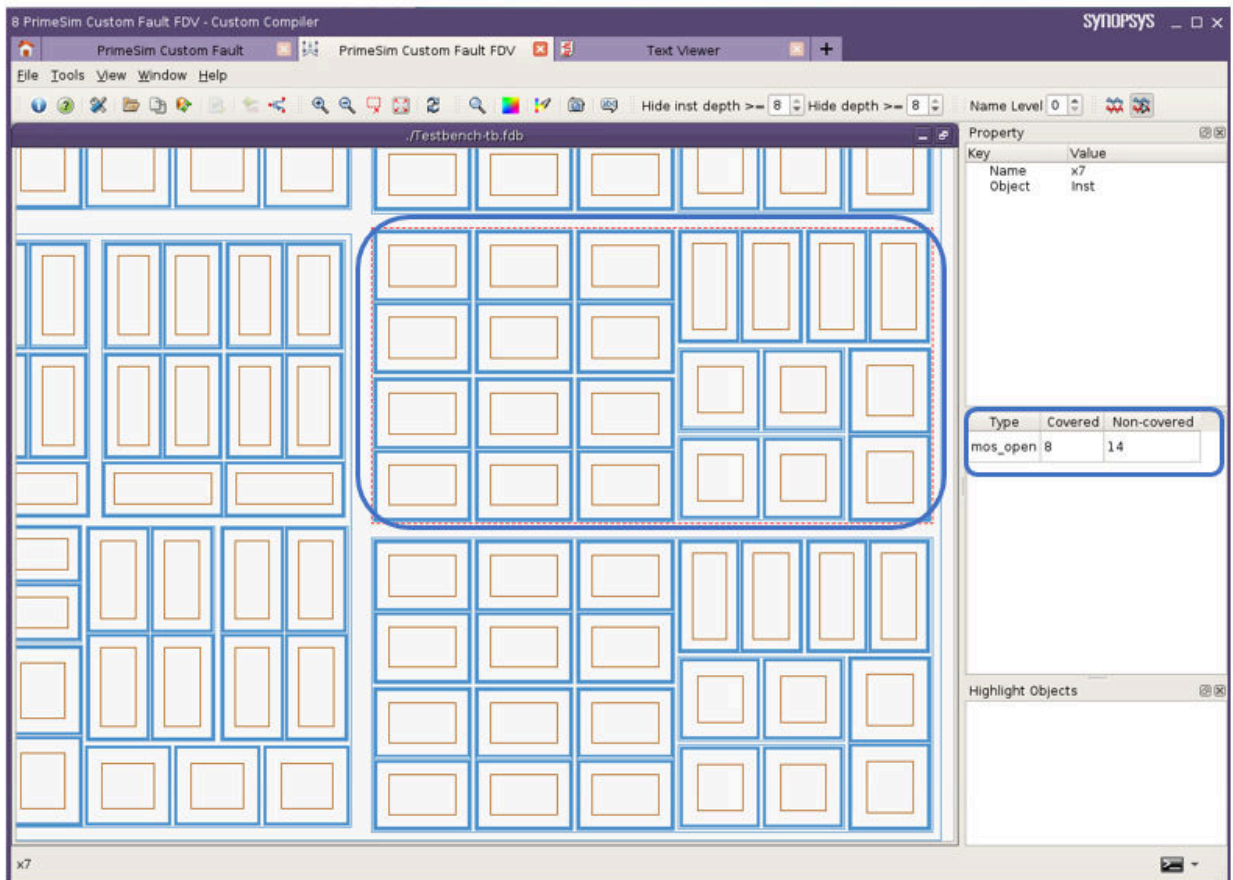


The following shows 10 defects in a MOS device: the six golden rectangles at the top are the six short defects and the four golden rectangles at the bottom are the four open defects.



If you select an instance, the coverage for that instance is displayed in the right side of the Viewer:

Figure 73 Displaying coverages



The following topics provide more information about displaying defects:

- [Highlighting Covered or Non-covered Defects](#)
- [Using an Expression to Display Defects](#)
- [Highlighting Defects for Functional Safety](#)
- [Displaying a Heat Map of Weights \(or Weight Map\)](#)

Highlighting Covered or Non-covered Defects

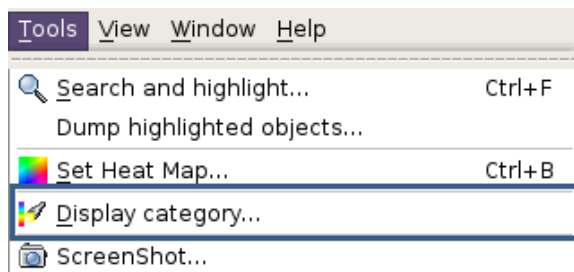
You can quickly highlight covered and not_covered defects in the same display, or you can highlight the defects separately.

- [Quick Highlight](#)
- [Highlighting the Defects Separately](#)

Quick Highlight

1. You can highlight both covered and not_covered defects at the same time by clicking

Display category  in the toolbar or choosing **Tools > Display category**:



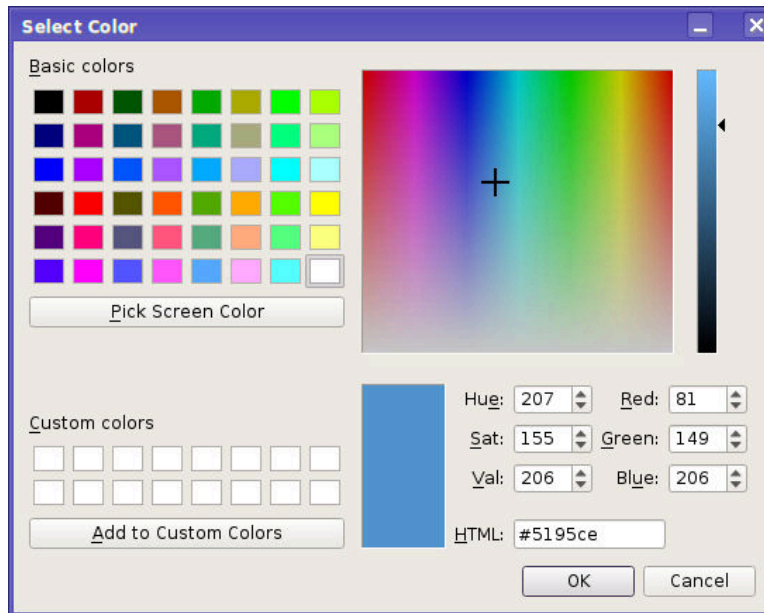
This displays the **Category Coloring** dialog box, where you can change the colors for the defect categories.



The 'Category Coloring' dialog box is used to assign colors to different categories of faults. It features three main sections: 'Coverage', 'Failure Mode', and 'Type'. Each section contains a list of categories with corresponding color swatches and hex codes. The 'Coverage' section has 'covered' (yellow, #aaff00) and 'not_covered' (orange, #ff5500). The 'Failure Mode' section includes an 'id' dropdown and four categories: 'Dangerous Detected' (teal, #9ad3bc), 'Dangerous Undetected' (orange, #f5b461), 'Safe Detected' (yellow, #f3eac2), and 'Safe Undetected' (red, #ec524b). The 'Type' section has four categories: 'mos_open_covered' (purple, #d5c6f2), 'mos_open_not_covered' (blue, #5162b5), 'mos_short_covered' (purple, #b56a96), and 'mos_short_not_covered' (pink, #ef82da). At the bottom are 'Apply' and 'Clear' buttons.

Category	Color	Hex Code
covered	Yellow	#aaff00
not_covered	Orange	#ff5500
Dangerous Detected	Teal	#9ad3bc
Dangerous Undetected	Orange	#f5b461
Safe Detected	Yellow	#f3eac2
Safe Undetected	Red	#ec524b
mos_open_covered	Purple	#d5c6f2
mos_open_not_covered	Blue	#5162b5
mos_short_covered	Purple	#b56a96
mos_short_not_covered	Pink	#ef82da

2. Click the defect type (covered, not_covered, Dangerous Detected, and so on) to display the **Select Color** dialog box.

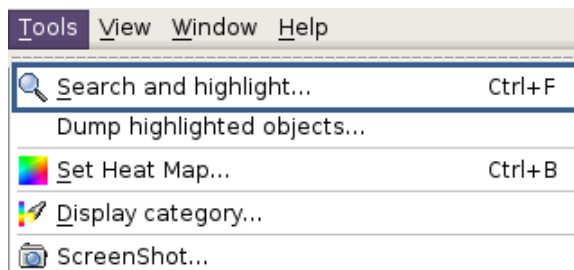


3. Select the desired color and click **OK**, then click **Apply** again in the **Category Coloring** dialog box.

The new color is only applied when you click **Refresh**  to refresh the FDV display.

Highlighting the Defects Separately

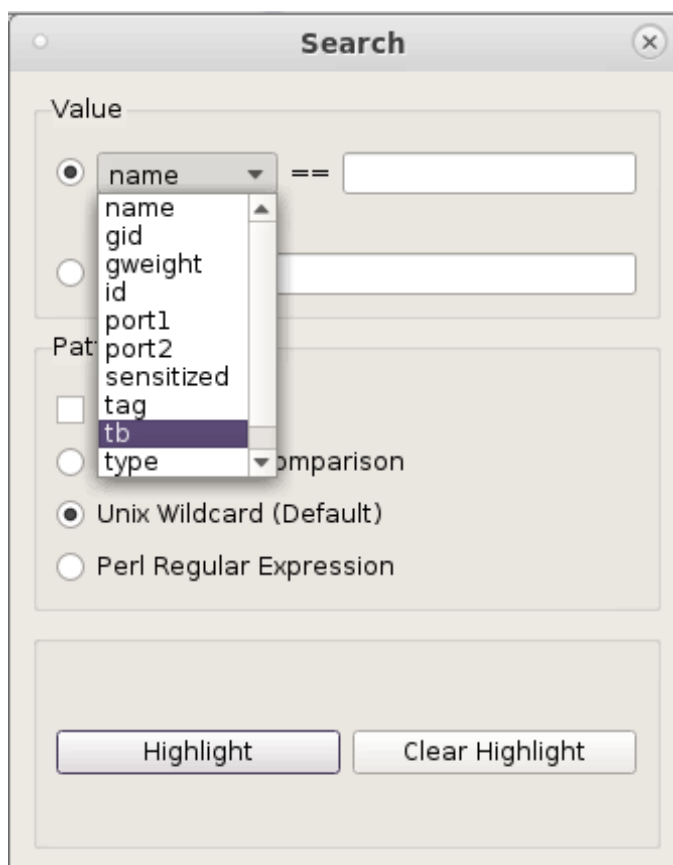
1. To highlight covered or not_covered defects, click **Search**  in the toolbar or choose **Tools > Search and highlight**:



This opens the **Search** dialog box.

The image shows a 'Search' dialog box with a title bar containing the word 'Search' and a close button (X). The dialog is divided into two main sections. The first section, titled 'Value', contains a radio button selected next to a dropdown menu showing 'name', followed by an equals sign and an empty text input field. Below this, the word 'string' is displayed. A second radio button is selected next to a text input field containing the word 'Expression'. The second section, titled 'Pattern Matching', contains four options: 'Case Sensitive' (unchecked checkbox), 'Exact String Comparison' (unselected radio button), 'Unix Wildcard (Default)' (selected radio button), and 'Perl Regular Expression' (unselected radio button). At the bottom of the dialog, there are two buttons: 'Highlight' and 'Clear highlight'.

2. Click the left half of the **Value** menu to display a list of what you can highlight.



As shown in the preceding figure, all the displayed options (name, gid, gweight, id, port1, port2, tag, type, and weight) are keywords except one: **tb**, the testbench name.

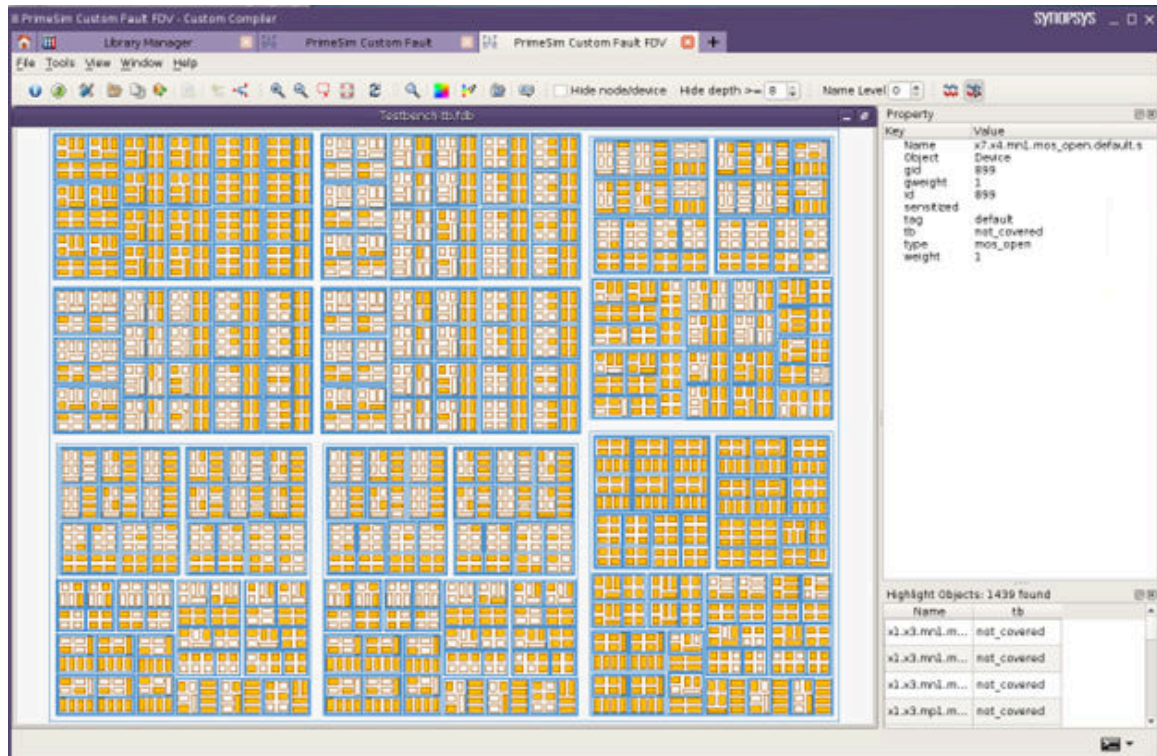
3. Choose **tb** in the left half of the **Value** menu.

The right half of the **Value** menu is populated with a list: covered and not_covered.



4. Select the defects you want to display (either covered or not_covered), then click **Highlight** to highlight the defects in a golden color on the FDV display.

The tool also lists the not_covered (or covered depending on which you chose) defects and the total number of not_covered defects in the **Highlight Objects** window.



When you click one defect in the list of **Highlight Objects**, the defect is zoomed in and selected in the main window.

In the **Highlight Objects** window, you can click the column header (**Name** for example) to sort the data (names) in descending or ascending order (as indicated by an arrow that appears in the column header).

To clear a highlight, click **Clear highlight** in the **Search** window.

Using an Expression to Display Defects

In the **Search** dialog box, instead of using the populated list of values, you can enter an expression in the **Expression** field.

The rules to write an expression are:

- Supported operators from high-to-low priority are:

```
( )
abs
* / %
+ -
```

```
>=
== !=
&&
||
```

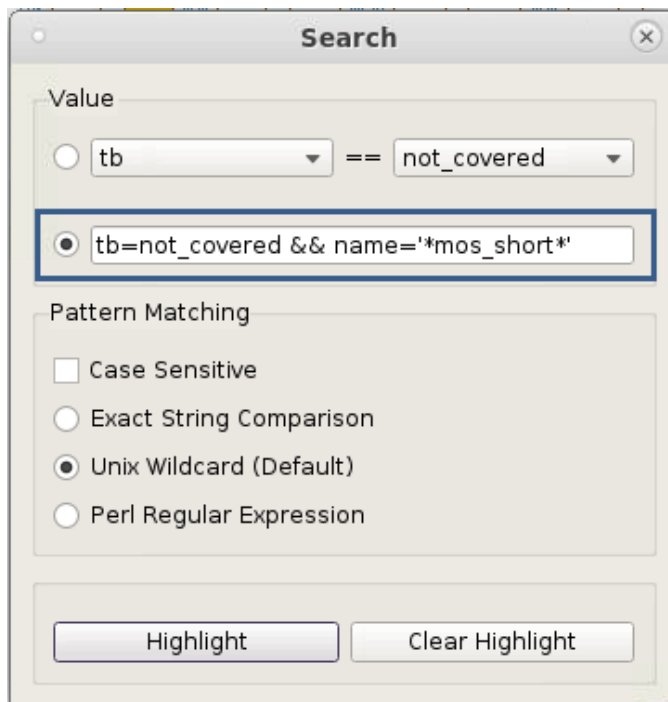
- Supported operators for string are == and !=.
- Strings which contain the following characters or "abs(" must be enclosed by quotation marks (""):

```
( ) * / % + -
```

- Case-insensitive

For example, if you want to display the defects that are of `mos_short` type and are `not_covered`, you can enter the following expression:

```
tb=not_covered && name="*mos_short"
```

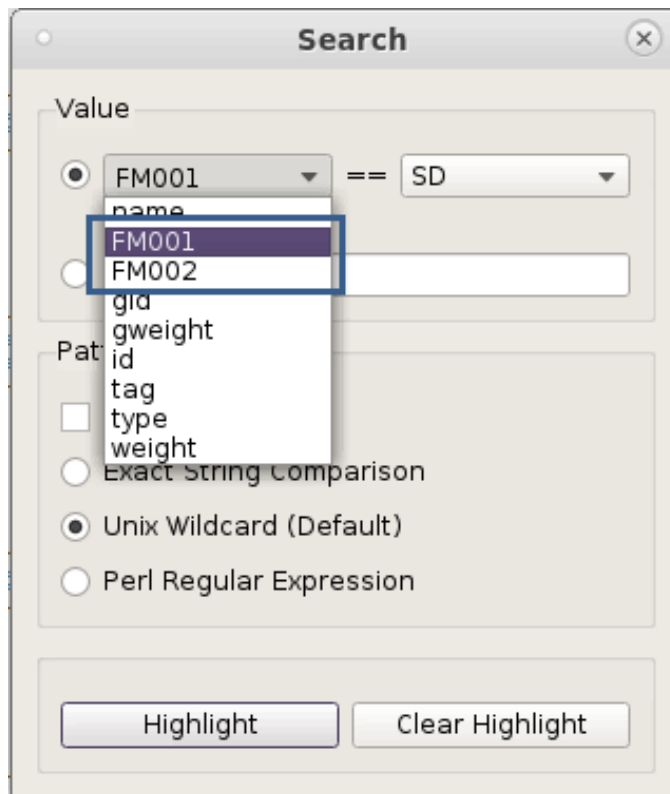


You can change the rules for pattern matching by modifying the defaults rules in the **Pattern Matching** area of the dialog box.

Highlighting Defects for Functional Safety

1. To highlight the Failure Modes detected and undetected defects, click **Search**  in the toolbar or choose **Tools > Search and highlight** to open the **Search** dialog box.

The failure modes are displayed as shown as in the following example: FM001 and FM002.



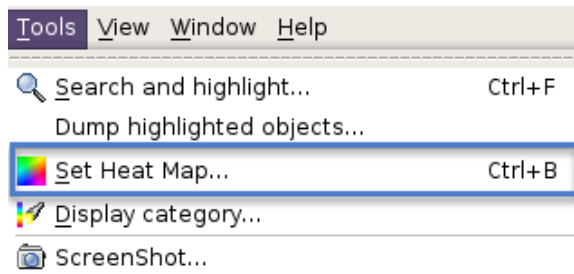
2. Pick one of the Failure Modes, and choose the defects you want to highlight:
 - DD: Dangerous Detected
 - DU: Dangerous Undetected

- SD: Safe Detected
- SU: Safe Undetected

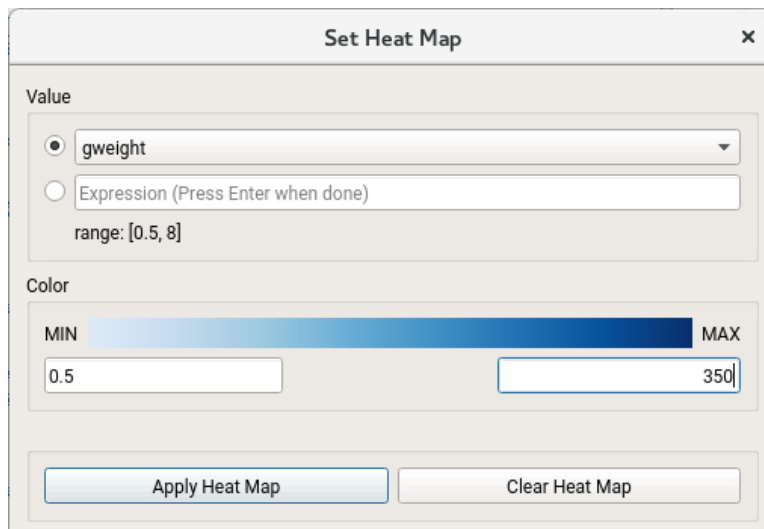


Displaying a Heat Map of Weights (or Weight Map)

1. You can display the heat map of weights by clicking **Set Heat Map**  in the toolbar or choosing **Tools > Set Heat Map**:

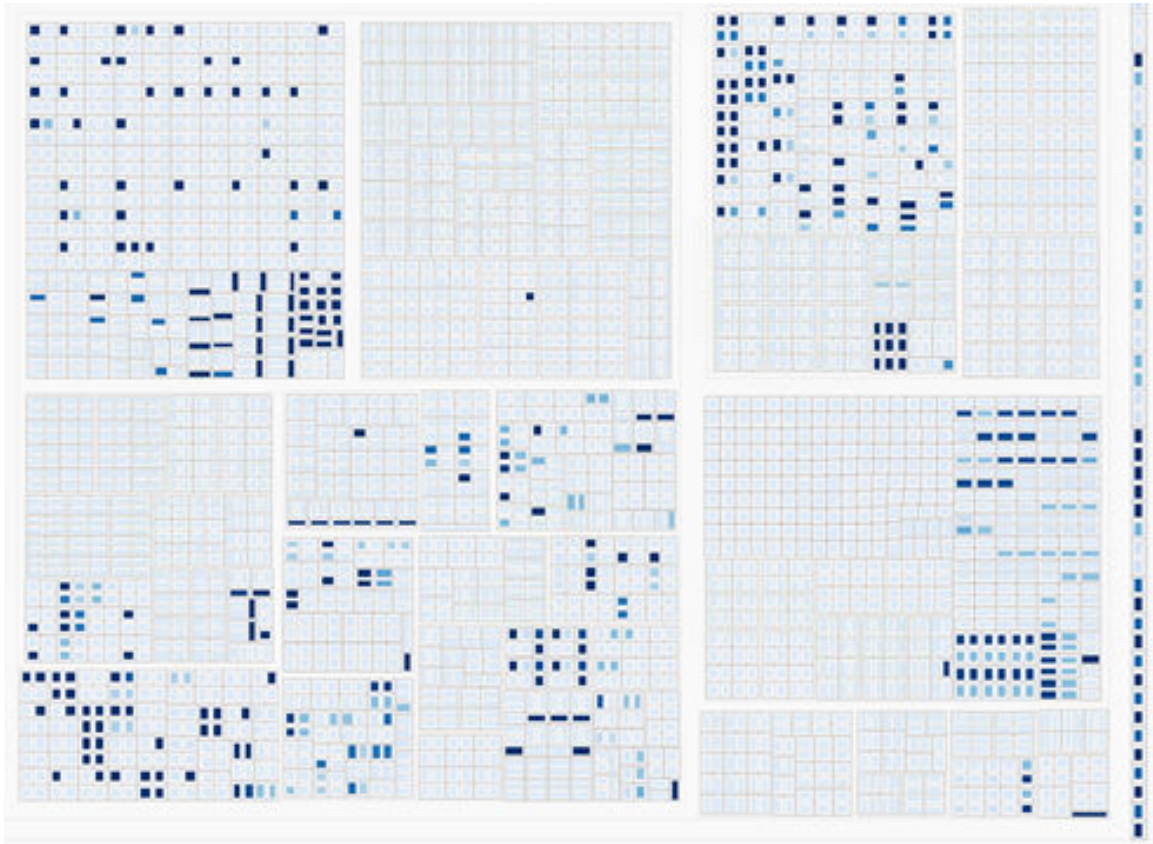


This opens the **Set Heat Map** dialog box.



2. Choose gweight or weight in the **Value** field, change the **MIN** and **MAX** of the color scale at the bottom, then click **Apply Heat Map**.

The database display is refreshed with darker colors on the defects with higher weights and lighter colors on the defects with lower weights.



You can also use an expression to define the weight range you want to highlight. Instead of using the slide bar to define **MIN** and **MAX**, you can use an expression:

You can clear the heat map by clicking **Clear Heat Map**.

Note:

The heat map can also be used to display id and gid.

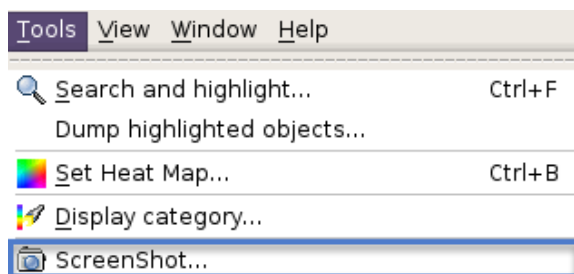
Outputting Results

You can output the FDV results either in the form of an image using a screenshot or in the form of a text file.

- [Screenshot](#)
- [Text File](#)

Screenshot

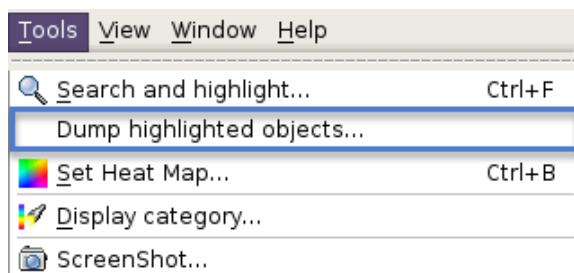
- You can take a snapshot of the main window display by clicking **Screenshot**  in the toolbar or choosing **Tools > Screenshot**:



A dialog box opens asking for a file name. The .png extension is added to the file name.

Text File

- When you highlight defects, you can output the results to a text file by choosing **Tools > Dump highlighted objects**:

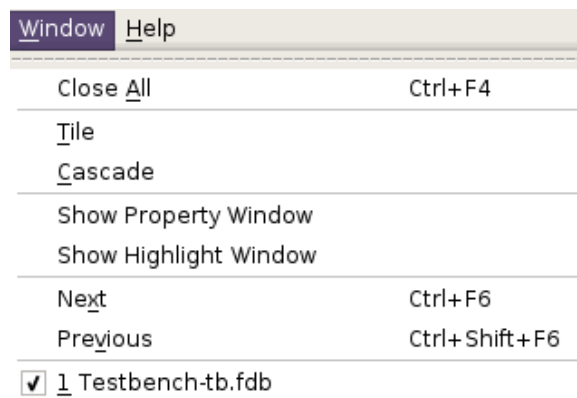


Windows Manipulation

The FDV window can be resized, and all the windows inside the FDV can also be resized. You can unpin and move the **Property** window and the **Highlight Objects** window by clicking the  icon. Clicking the  icon again puts it back to its original position.

You can close the **Property** window and the **Highlight Objects** window by clicking the  icon at the upper right of the window.

To reopen the closed window, choose **Window > Show Property Window** to reopen the **Property** window or **Window > Show Highlight Window** to reopen the **Highlight Objects** window.



Other Display Options

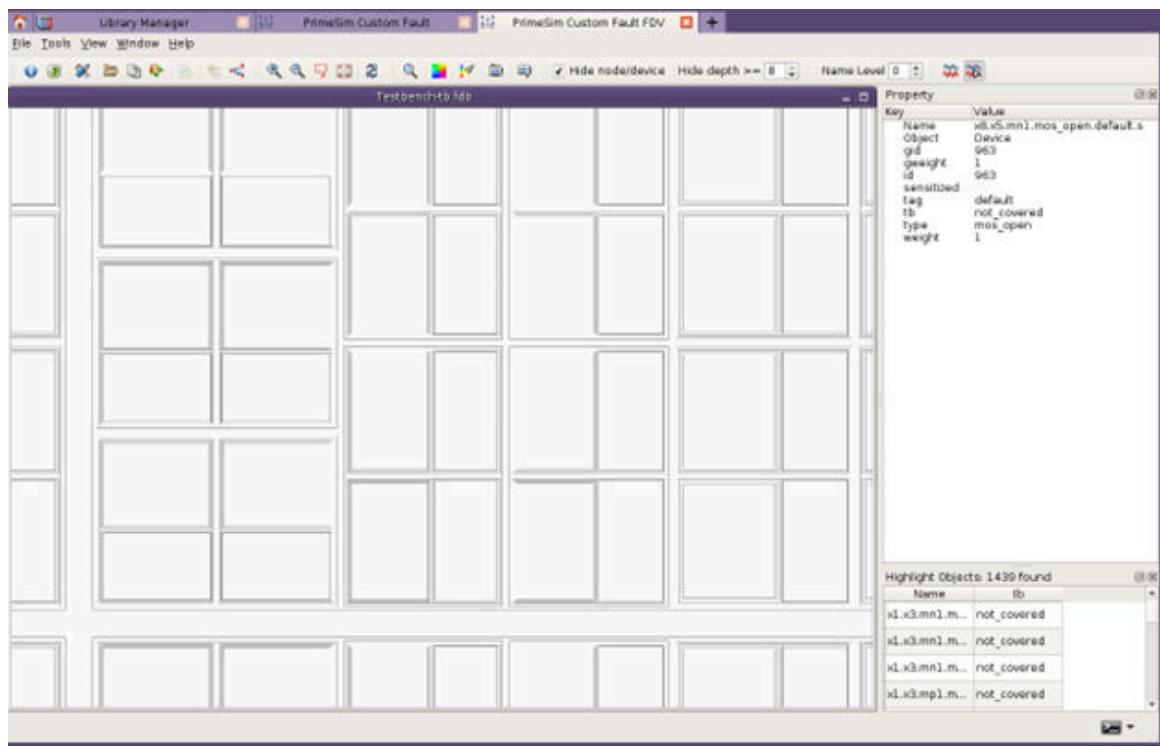
The FDV supports the following additional display options:

- [Hide Nodes and Devices](#)
- [Hide Depth](#)
- [Display Instance Names](#)

Hide Nodes and Devices

You can hide the defects and display only the instances as blue rectangles.

- Select **Hide node/device** to display only the instance rectangles.

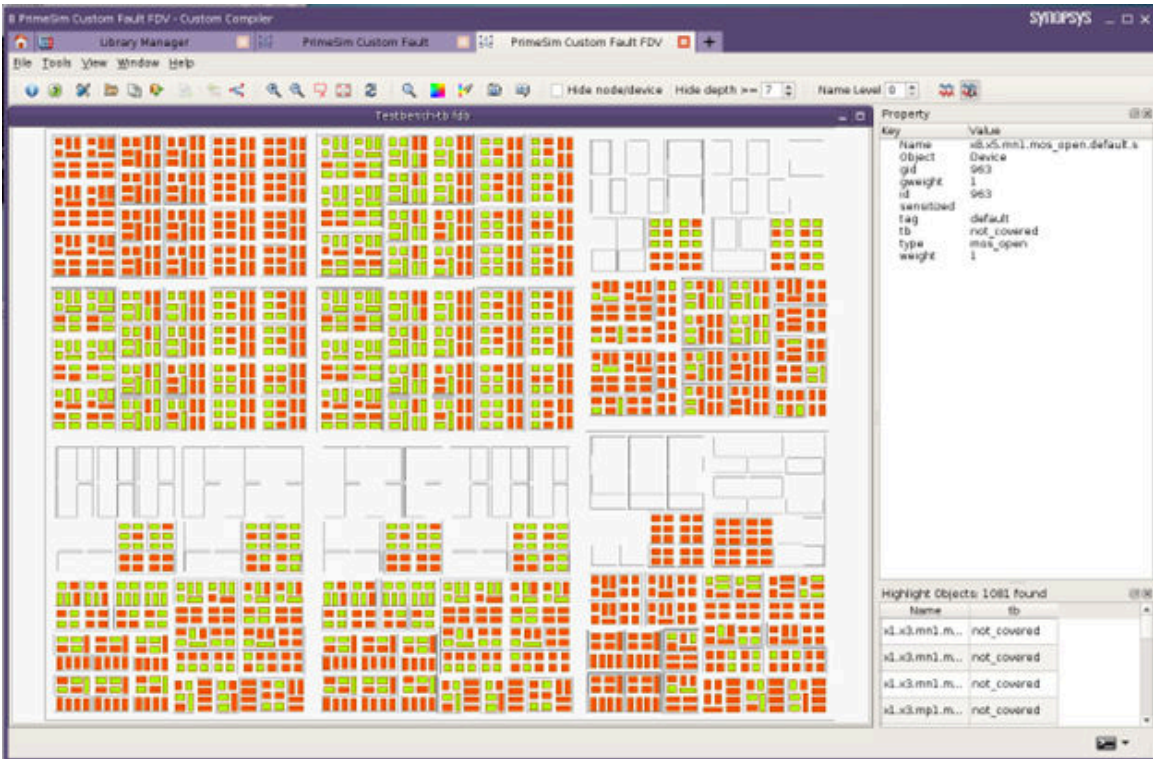


Hide Depth

You can limit the hierarchy display to a given level using the **Hide depth** control:

Hide depth >=

Only the defects down to the specified hierarchy level are displayed.

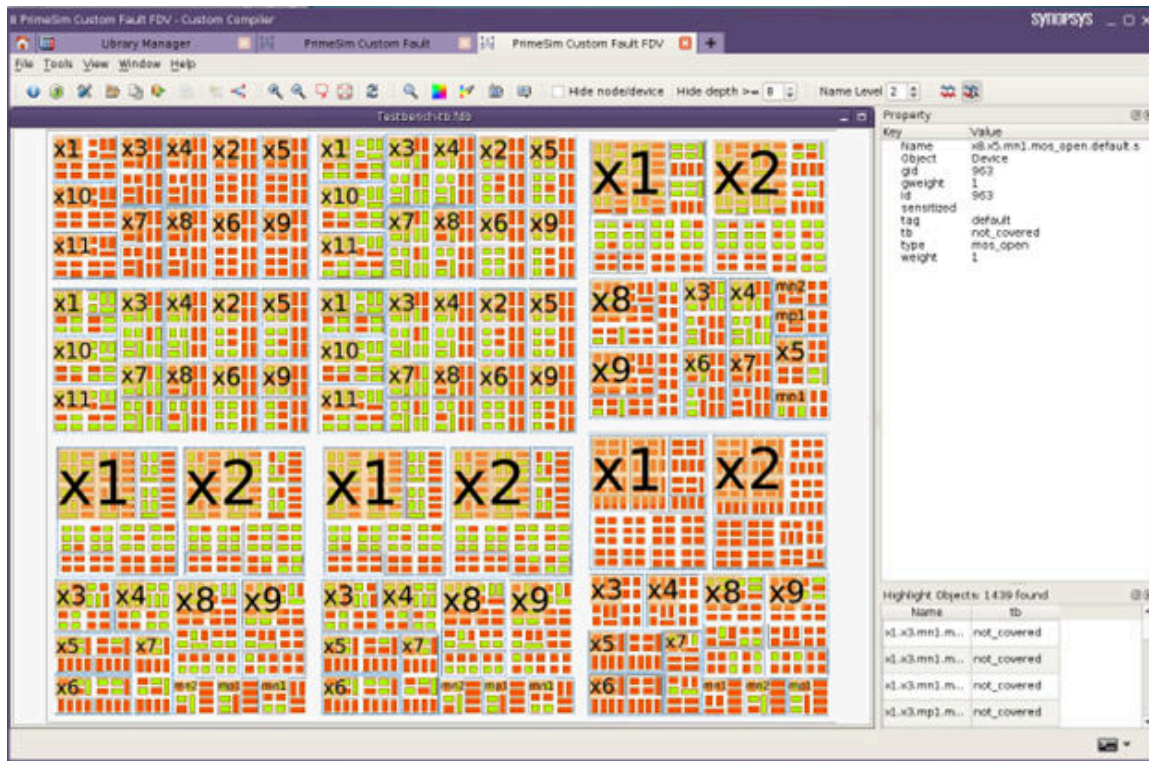


Display Instance Names

You can display the instance names at a given hierarchy level. Choose the hierarchy level to display the instance name with the **Name Level** control:

Name Level

The instance names corresponding to that level are displayed.

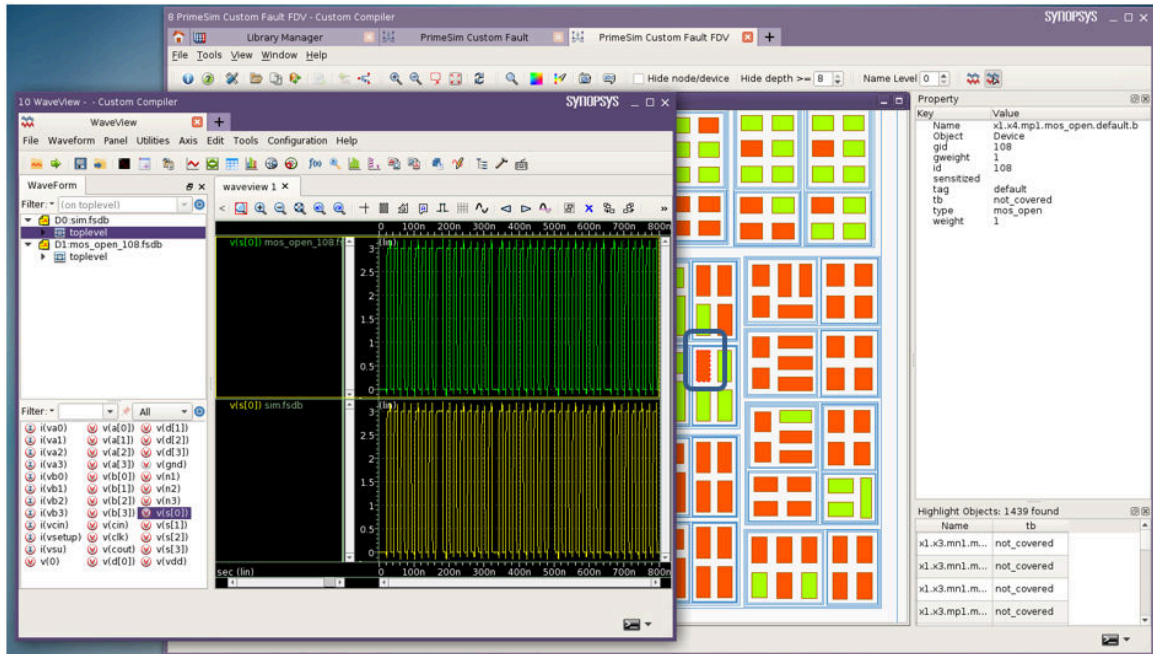


Displaying Waveforms

You can display the defect waveforms from the FDV.

- To display the waveforms of a specific defect, select the defect in FDV and click **Load Waveform File** .

The WaveView opens and loads the waveforms corresponding to the defect.



If **Nominal Wave**  is enabled, then the nominal waveforms are displayed together with the defect waveforms.

17

Configuration File Command Reference

This chapter describes the Custom Fault input and defect model configuration file commands.

- [Input Commands](#)
- [Defect Model Commands](#)

Input Commands

This section lists the Custom Fault input commands.

- [analog_setup](#)
- [case_dir](#)
- [circuit_device_count](#)
- [cleanup_level](#)
- [collapsing](#)
- [collapsing](#)
- [cosim_setup](#)
- [cpulimit](#)
- [create_fault_files](#)
- [dc_operating_point](#)
- [defect_weight](#)
- [detectability](#)
- [diagnostic_coverage](#) (for Latent Faults)
- [diagnostic_coverage](#) (for Residual Faults and Direct SPFM Calculation)
- [dpconfig_file](#)

- [enable_fault_collapse](#)
- [enable_stackmos_fault_collapse](#)
- [fault_list_file](#)
- [exclude_fault_list_file](#)
- [fault_sim_output](#)
- [generate_dual_fault](#)
- [grouped_waveform](#)
- [identify_only](#)
- [nc_analysis](#)
- [outdir](#)
- [parameter_variation](#)
- [report](#)
- [resume_data](#)
- [sampling](#)
- [simulate_only](#)
- [sqa](#)
- [what_if](#)

analog_setup

Description

Generates the defect universe for running transistor-only simulations with the PrimeSim XA, FineSim, or PrimeSim tools.

Note:

When running mixed-signal simulations with the VCS PrimeSim AMS tool, use the [cosim_setup](#) command.

Syntax

```
"analog_setup" : {  
  "sim_cmdline" : "simulator_command_line",  
  "postl_id_cmdline" : "postlayout_id_cmdline",  
  "compilation_cmdline" : "compilation_cmdline"  
}
```

Arguments

Argument	Description
<code>"sim_cmdline" :</code> <code>"simulator_command_line"</code>	Specifies the simulator command line. It must be a valid command line for the simulator including the top netlist name and the required command-line options. For the PrimeSim XA tool, you can specify the following command-line options: • <code>-netlist_format</code> • <code>-mt</code> • <code>-ssl2</code> • <code>-c</code> • <code>-wavefmt</code> • <code>-top</code> For the FineSim tool, you can specify the following command-line options: • <code>-netlist_format</code> • <code>-spice</code> • <code>-mt</code> • <code>-mode</code> For the PrimeSim tool, you can specify the following command-line options: • <code>-netlist_format</code> • <code>-spice</code> • <code>-spice_lic</code> • <code>-mt</code> Note: For <code>-netlist_format</code>, see the respective fault simulator documentation for command-line options specifying PrimeSim HSPICE or 3rd-party netlist formats.
<code>"postl_id_cmdline" :</code> <code>"postl_id_cmdline"</code>	Specifies the simulator command line to run a post-layout netlist. Note: The post-layout netlist is used only to identify the <code>net_to_net_short</code> defects. The simulations are still run using the <code>sim_cmdline</code> argument.
<code>"compilation_cmdline" :</code> <code>"compilation_cmdline"</code>	(Optional) Specifies the simulator command line to generate the defect universe when running the Custom Fault tool with the FineSim or PrimeSim tools as the transient simulator.

Examples

The examples in this section illustrate setting up transistor-only simulation both without and with back-annotation for net-to-net short defect identification.

Example 1

In this example, the PrimeSim XA is used and the netlist file may be schematic, flat-extracted, or a back-annotation setup.

```
"analog_setup": {  
  "sim_cmdline": "xa netlist1.sp -c cfg.xa -mt 4"  
}
```

Example 2

In this example, the FineSim tool is used and the netlist file may be schematic, flat-extracted, or a back-annotation setup.

```
"analog_setup": {  
  "sim_cmdline": "finesim netlist1.sp -mt 4"  
}
```

Example 3

In this example, the PrimeSim tool is used and the netlist file may be schematic, flat-extracted, or a back-annotation setup.

```
"analog_setup": {  
  "sim_cmdline": "primesim netlist1.sp -mt 4"  
}
```

Example 4

In this example, the PrimeSim XA tool is used and the Eldo format netlist file may be schematic, flat-extracted, or a back-annotation setup.

```
"analog_setup" : {  
  "sim_cmdline" : {"xa -eldo netlist1.cir -mt 4"}  
}
```

Example 5

In this example, the PrimeSim XA tool is used and the back-annotation commands are placed in the `cfg_ba.xa` file.

```
"analog_setup": {  
  "sim_cmdline": "xa netlist1.sp -c cfg.xa -mt 4",  
  "postl_id_cmdline": "xa netlist1.sp -c cfg_ba.xa -mt 4"  
}
```

Example 6

In this example, the PrimeSim tool is used and the back-annotation commands are placed in the `netlist_ba.sp` file.

```
"analog_setup": {  
  "sim_cmdline": "primesim netlist_sch.sp -mt 4 -mode prohd",
```

```
} "postl_id_cmdline": "primesim netlist_ba.sp -mt 4 mode prohd"
```

case_dir

Specifies the directory in which to find the data and run the simulation.

Syntax

```
"case_dir" : "dir"
```

Description

You can specify a relative or full path. When running mixed-signal analog fault simulation, compilation might happen in a different directory. See [Overview of the Mixed-Signal Fault Simulation Flow](#) for more information.

circuit_device_count

Description

Counts and reports the number of devices of user-specified models or subcircuits at all hierarchical levels of a design at and below the user-specified top instance.

This command writes the results to the following files:

- A `.device_count` formatted text file containing a table listing the counts of devices at each level and the sum below the reported instance.
- A `.csv` file that reports only the devices at the reported instance and which is formatted such that it can be imported to VC Functional Safety Manager.

Syntax

```
"circuit_device_count" :
{
  "top_inst"      : [ "top1_hier_name", "top2_hier_name", ... ],
  "report_depth" : depth,
  "stop_subckt"  : [ list_of_subckt ]
  "vc_fsm_file"  : "filename.csv",
  "column"       : [
    {
      "name"      : "user_defined_name",
      "count"     : [
        {
          "device_name" : [ "devname_1", ... , "devname_n" ],
          "parameters"  : [ "parm_1", "parm_2", ... , "parm_n" ],
          "scale_factor" : number
        }
      ]
    }
  ] // end count
```

```
        } // end column1
    ] // end columns
} // end command
```

Arguments

Argument	Description
<code>"top_inst" : ["top1_hier_name", "top2_hier_name", ...]</code>	Specifies one or more top-level instances for which the device count is tallied. The <i>topn_hier_name</i> is a hierarchical instance name from the design netlist and can include the "*" wildcard character.
<code>"report_depth" : depth</code>	(Optional) Indicates how many levels of hierarchy are reported under the top-level instance (level 0). <i>depth</i> can be 1, 2, 3, and so on. The default depth is to the bottom leaf cells of the hierarchy. This argument can be used together with <i>stop_subckt</i> .
<code>"stop_subckt" : [list_of_subckt]</code>	(Optional) Defines the bottom of the hierarchy that is reported. Hierarchy below the stop list subcircuits is counted, but reported at the <i>stop_subckt</i> level. If this keyword is not provided, the hierarchy is reported down to the <i>count</i> devices.
<code>"vc_fsm_file" : "filename.csv"</code>	(Optional) Specifies the filename for the .csv file produced by the command. If this argument is not provided, the VC FSM file is named <i>prefix.top_inst.devcnt.csv</i> , where <i>prefix</i> is specified by the tool -o command-line option and the top-level instance is specified by the <i>top_inst</i> argument of this command.
<code>"column" : []</code>	Defines columns with "name" and "count".
<code>"name" : "user_defined_name"</code>	A user-defined name explaining what kind of devices are counted for the number of count value such as Flops, Latches, RAM_BITS, ROB_BITS, FETs, BJTs, CAPs, and RESs. This forms the column name in the output file.
<code>count : []</code>	An array that indicates which subcircuits or models should be counted and what subcircuit or model parameters can modify the count.

Argument	Description
"device_name" : ["list-of-devnames"]	Specifies the name of a subcircuit or primitive that should be counted, for example, "nfet", "pfet". At least one device name must be provided.
"parameters" : ["list-of-parameters"]	(Optional) Multiplied to finalize the count value, for example, "nfin", "nf", "m". If no parameters are provided, then each instance of the device is counted as 1. The scale_factor can still be applied.
"scale_factor" : number	(Optional) Multiplied to finalize the count value, for example: 1, 10, 1e+06, or 1e-06. If not specified, it takes a default value of 1.

Examples

The examples in this section use the following circuit definition:

```
.subckt nfet d g s b nf=1 nfin=1 mult=1
mn d g s b nch nf=nf nfin=nfin m=mult
.ends

.subckt pfet d g s b nf=1 nfin=1 mult=1
mp d g s b pch nf=nf nfin=nfin m=mult
.ends

.subckt myres n p b
R1 n p b resmodel
.ends

.subckt block3
Xmn1 a b c d nfet nfin=6                $6
.ends

.subckt block2
Xmn1 a b c d nfet nfin=4 nf=1 mult=1      $4
XL3 a b c d block3                        $6
.ends

.subckt block1
XMN1 a b c d nfet nfin=4 nf=2 mult=2      $16
XMP1 a b c d pfet nfin=8 nf=2 mult=2      $32
XMN2 a b c d nfetlvt nfin=8 nf=1 mult=4   $32
XR1 e f g myres w=10u l=100u
XL2 a b c d block2                        $10
.ends

.subckt top
```



```
Xi100 n1 n2 n3 n4      block1          $90
Xi101 n5 n6 n7 n8      block1          $90
Xi201 rn1 rn2 rn3      myres w=1u l=4u
Xi202 rn4 rn5 rn6      myres w=1u l=4u
XMn1 d g s b nfet nfin=10 nf=2 mult=4  $80
.end
```

Example 1

The default report depth is used and no `stop_subckt` list is used, so the report descends to the count devices.

Xtop top

```
"circuit_device_count" :
{
  "top_inst" : [ "xtop" ],

  "vc_fsm_file" : "xtop.devcnt.csv"
  "column" : [
    {
      "name" : "FETs",
      "count" : [
        {
          "device_name" : [ "nfet", "pfet" ],
          "parameters" : [ "nfin", "nf", "m" ],
          "scale_factor" : 1
        }
      ]
    },
    {
      "name" : "RES",
      "count" : [
        {
          "device_name" : [ "myres" ],
          "parameters" : [ "w", "l" ],
          "scale_factor" : 1e12
        }
      ]
    }
  ] // end columns
} // end report
```

The output text file report `$ODIR/$ONAME.device_count` contains:

Instance	subckt	Per Level:	FETs	RES	Sum FETs	Sum RES
Xtop	top		80	8	260	2008
Xtop.xi100	block1		80	1000	90	1000
Xtop.xi100.XL2	block2		4	0	10	0
Xtop.xi100.XL2.XL3	block3		6	0	6	0
Xtop.xi101	block1		80	1000	90	1000
Xtop.xi101.XL2	block2		4	0	10	0
Xtop.xi101.XL2.XL3	block3		6	0	6	0

The .csv file \$ODIR/\$ONAME.xtop.devcnt.csv (for direct import to VC FSM) contains:

```
"HierScope", "RealScope", "FETs", "RES"
Xtop, Xtop, 80, 8
Xtop.xi100, Xtop.xi100, 80, 1000
Xtop.xi100.xl2, Xtop.xi100.xl2, 4, 0
Xtop.xi100.xl2.xl3, Xtop.xi100.xl2.xl3, 6, 0
Xtop.xi200, Xtop.xi200, 80, 1000
Xtop.xi200.xl2, Xtop.xi200.xl2, 4, 0
Xtop.xi200.xl2.xl3, Xtop.xi200.xl2.xl3, 6, 0
```

Example 2

In this example, the report depth is set to 2, limiting the reporting to two levels of hierarchy. All FETs in block2 and block3 are reported as block2.

Xtop top

```
"circuit_device_count" :
{
  "top_inst" : [ "xtop" ],
  "report_depth" : 2,
  "vc_fsm_file" : "xtop.devcnt.csv"
  "column" : [
    {
      "name" : "FETs",
      "count_subckt" : [
        {
          "device_name" : [ "nfet", "pfet" ],
          "parameters" : [ "nfin", "nf", "mult" ],
          "scale_factor" : 1
        }
      ]
    },
    {
      "name" : "RES",
      "count" : [
        {
          "device_name" : [ "myres" ],
          "parameters" : [ "w", "l" ],
          "scale_factor" : 1e12
        }
      ]
    }
  ] // end columns
} // end report
```

The output text file report \$ODIR/\$ONAME.device_count contains:

Instance	subckt	Per Level:	FETs	RES	Sum FETs	Sum RES
Xtop	top		80	8	260	2008
Xtop.xi100	block1		80	1000	90	1000
Xtop.xi100.XL2	block2		10	0	10	0

Xtop.xi101	block1	80	1000	90	1000
Xtop.xi101.XL2	block2	10	0	10	0

The .csv file \$ODIR/\$ONAME.xtop.devcnt.csv (for direct import to VC FSM) contains:

```
"HierScope", "RealScope", "FETs", "RES"
Xtop,Xtop,80,8
Xtop.xi100,Xtop.xi100,80,1000
Xtop.xi100.xl2,Xtop.xi100.xl2,10,0
Xtop.xi200,Xtop.xi200,80,1000
Xtop.xi200.xl2,Xtop.xi200.xl2,10,0
```

Example 3

In this example, the stop_subckt command is set, limiting the reporting to block1. All FETs in block2 and block3 are reported as block2.

```
Xtop top
"circuit_device_count" :
{
  "top_inst" : [ "xtop" ],
  "stop_subckt" : [ "block1" ],
  "vc_fsm_file" : "xtop.devcnt.csv",
  "column" : [
    {
      "name" : "FETs",
      "count" : [
        {
          "device_name" : [ "nfet", "pfet" ],
          "parameters" : [ "nfin", "nf", "mult" ],
          "scale_factor" : 1
        }
      ]
    },
    {
      "name" : "RES",
      "count" : [
        {
          "device_name" : [ "myres" ],
          "parameters" : [ "w", "l" ],
          "scale_factor" : 1e12
        }
      ]
    }
  ] // end columns
} // end report
]
```

The output text file report \$ODIR/\$ONAME.device_count contains:

Instance	subckt	Per Level:	FETs	RES	; Sum	FETs	Sum RES
Xtop	top		80	8		260	2008

```
Xtop.xi100          block1          90 1000          90 1000
Xtop.xi101          block1          90 1000          90 1000
```

The `.csv` file `$ODIR/$ONAME.xtop.devcnt.csv` (for direct import to VC FSM) contains:

```
"HierScope", "RealScope", "FETs", "RES"
Xtop, Xtop, 80, 8
Xtop.xi100, Xtop.xi100, 90, 1000
Xtop.xi200, Xtop.xi200, 90, 1000
```

cleanup_level

Description

Controls which intermediate files to keep after the simulation.

Syntax

```
"cleanup_level" : value
```

Arguments

Argument	Description
"cleanup_level" : value	Choose one of the following options to specify which intermediate files to keep. • 0: Keep all intermediate files. • 1: (default) Keep all *.csh, *.tcl, *.stderr, *.log, .fsdb (waveform), and measurements files, and subdirectories. • 2: Keep all *.csh, *.stderr, *.log, and measurements files. • 3: Keep all *.stderr, *.log, and .fsdb (waveform) files. • 4: Keep all *.stderr and *.log files.

collapsing

Description

Specify `true` to compress the fault database (FDB) file with gzip to create `prefix.fdb.gz`. The default is `false`.

Syntax

```
"compress_fdb" : true|false
```

collapsing

Description

Specify `false` to disable structural or electrical collapsing. The default is `true`.

The Detectable Fault Analysis (DFA) technology not only investigates the detectability of faults but also provides grouping of faults that have approximately the same electrical effect. This is estimated by simulation and controlled by the tolerance values.

Syntax

```
"collapsing": {  
  "structural": true|false,  
  "electrical": true|false,  
  "tolerance": [  
    {  
      "name" : "string",  
      "err" : "string",  
      "abs" : "string"  
    },  
    ...  
  ]  
}
```

Arguments

Argument	Description
<code>"structural": true false,</code>	Enables the structural collapsing.
<code>"electrical": true false,</code>	Enables the electrical collapsing.
<code>"tolerance": [{ "name" : "string", "err" : "string", "abs" : "string" }]</code>	The tolerance can be modified by the user. The default for tolerance is 3% for "err" and 30mV is for "abs".

Examples

In the following example, both structural and electrical collapsing are enabled. For electrical collapsing, a tolerance error of 5% is applied.

```
"collapsing": {  
  "structural": true,  
  "electrical": true,  
  "tolerance": [{  
    "err": "5%"  }]  
}
```

cosim_setup

Description

Generates the defect universe for running mixed-signal simulations with the VCS PrimeSim AMS tool.

Note:

When running transistor-only simulations with the PrimeSim XA, FineSim, or PrimeSim tools, use the [analog_setup](#) command.

Syntax

```
"cosim_setup" : {  
  "cosim_info" : {  
    "init" : "init_file",  
    "compilation" : "compile_opt",  
    "simv_cmdline" : "simv_cmd",  
    "fsdb_check" : {  
      "signal_file" : "signal_file",  
      "golden_file" : "golden_file"  
    },  
    "msv_message_limit" : total_number_of_messages,  
    "dump_digital_waveform" : 0|1|2  
  },  
  "postl_id" : {  
    "init" : "postl_init_file",  
    "compilation" : "postl_compile_opt",  
    "simv_cmdline" : "postl_simv_cmd"  
  }  
}
```

Arguments

Argument	Description
"cosim_info" : { ... }	Provides information for running mixed-signal fault simulation.
"init" : "init_file"	Specifies the path name and file name to the mixed-signal control file. You must specify the <code>report_option msv_dir=spice</code> command in the <code>init_file</code> . This is a mandatory argument.

Argument	Description
<code>"compilation" : "compile_opt"</code>	(Optional) Specifies the command-line option or the name of a run script to compile a mixed-signal design. If the <code>fsdb_check</code> is specified, <code>compile_opt</code> must have the <code>-debug_access</code> option. It only supports the Verilog-top and SPICE-top configurations. The VHDL-top configuration is not supported and the Verilog-top wrapper must be created. This is an optional command; it can be dropped if the mixed-signal design is already compiled and if all fault simulations are run on the same type of machine and OS that was used for compilation.
<code>"simv_cmdline" : "simv_cmd"</code>	Specifies the command line for mixed-signal simulation. This is a mandatory argument.
<code>"fsdb_check" : {...}</code>	Enables waveform comparison on the digital signals for accuracy checking when running the mixed-signal analog fault simulation. It supports the FSDB file format.
<code>"signal_file" : "signal_file"</code>	Specifies the name of the signal file that contains the list of the digital signals for accuracy checking. See Signal File Format for details.
<code>"golden_file" : "golden_file"</code>	Specifies the name of the reference waveform file. The tool compares the specified digital signals in the simulated waveform file to the reference waveform file. If the simulated signal does not match the reference signal, a violation is reported in the log file. If the <code>fsdb_check</code> command is specified, <code>compile_opt</code> must have the <code>-debug_access</code> option. It only supports the Verilog-top, SPICE-top configuration. The VHDL-top configuration is not supported and the Verilog-top wrapper must be created.
<code>"msv_message_limit" : total_number_of_messages</code>	(Optional) Specifies the limit of the total number of mixed-signal messages generated by the Custom Fault tool during FSDB check. • Not specified: This is the default behavior; there is no limit to the number of messages generated by the fault simulator and all messages are displayed. • -1: There is no limit to the number of messages generated by the fault simulator and all messages are displayed. • 0: The total number of messages generated by the fault simulator is limited to 0, so none are displayed. • N: Limits the number of messages generated by the fault simulator and displayed to a positive integer value. For example, 10 sets the limit to 10 messages, 1000 sets the limit to 1000 messages, and so on.

Argument	Description
"dump_digital_waveform" : 0 1 2	(Optional) Specifies whether to output digital waveforms. 0: (default) Do not add extra output commands, but honor user-specified output commands. 1: Add one extra output command for all digital signals. 2: Disable all digital signals output, including user-specified output commands.
"postl_id" : { ... }	Provides the information for running mixed-signal simulation with back-annotation to identify the <code>net_to_net_short</code> defects.
"init" : "postl_init_file"	Specifies the path name and file name to the post-layout mixed-signal control file. You must specify the <code>report_option msv_dir=spice</code> command in the <code>postl_init_file</code> . This is a mandatory argument. Note: The post-layout mixed signal control file must have a name different from the pre-layout mixed signal control file.
"compilation" : "postl_compile_opt"	(Optional) Specifies the command-line option or the name of a run script to perform back-annotation on a mixed-signal design. If the <code>fsdb_check</code> is specified, <code>postl_compile_opt</code> must have the <code>-debug access</code> option. It only supports the Verilog-top and SPICE-top configurations. The VHDL-top configuration is not supported and the Verilog-top wrapper must be created. This is an optional command; it can be dropped if the mixed-signal design is already compiled and if all fault simulations are run on the same type of machine and OS that was used for compilation. Note: The post-layout compilation command must include the <code>-o</code> and <code>-Mdir</code> options to not overwrite the pre-layout data.
"simv_cmdline" : "postl_simv_cmd"	Specifies the command line for mixed-signal simulation. This is a mandatory argument.

Examples

The examples in this section illustrate setting up VCS PrimeSim AMS mixed-signal simulation both without and with back-annotation for `net-to-net short` defect identification.

Example 1

In this example, fault simulations are run using the `xa.init` control file. The mixed-signal compilation command line is specified in the `compilation` argument. The digital results are compared with the signals specified in `sig.txt` and output to `golden.fsdb`.

```
"cosim_setup": {
  "cosim_info" : {
    "init" : "xa.init",
    "compilation" : "vcs -ad_new_dr addr4.v -debug_access
      + all -ad=xa.init -full64",
    "simv_cmdline" : "./simv",
    "fsdb_check" : {
      "signal_file" : "sig.txt",
      "golden_file" : "golden.fsdb"
    },
    "msv_message_limit" : 50
  }
}
```

Example 2

In this example, fault simulations are run using the `xaVCS.init` control file. The compilation run script is specified in the `compilation` argument. The digital results are compared with the signals specified in `fsdb.chk` and output to `mygold.fsdb`.

```
"cosim_setup": {
  "cosim_info": {
    "init": "xaVCS.init"
    "compilation": "./compile_msv"
    "simv_cmdline": "./runsimv"
    "fsdb_check": {
      "signal_file": "fsdb.chk",
      "golden_file": "mygold.fsdb"
    }
  }
}
```

Example 3

In this example, fault simulations are run using the `xaVCS.init` control file. The compilation run script is specified in the `compilation` argument. The digital results are compared with the signals specified in `fsdb.chk` and output to `mygold.fsdb`. The `net_to_net_short` defects are identified using the `postl_id` argument information

```
"cosim_setup": {
  "cosim_info": {
    "init": "xaVCS.init"
    "compilation": "./compile_msv"
    "simv_cmdline": "./runsimv"
    "fsdb_check": {
      "signal_file": "fsdb.chk",
      "golden_file": "mygold.fsdb"
    }
  }
}
```

```
    },
    "postl_id" : {
        "init": "xaVCS_ba.init"
        "compilation": "./compile_msv_ba"
        "simv_cmdline": "./runsimv_ba"
    }
}
```

Example 4

In the following example, the `net_to_net_short` defects are identified using the `vcsAD_ba.init` control file. The post-layout compilation command is run with the `-o simvl -Mdir=csrcl` command-line options to ensure the pre-layout files are not overwritten when running `net_to_net_short` defects identification.

```
"cosim_setup" : {
    "cosim_info" : {
        "init": "./vcsAD.init",
        "compilation": "./run_comp",
        "simv_cmdline": "simv -nc -ucli -do vcs.do",
        "dump_digital_waveform" : true
    },
    "postl_id" : {
        "init": "vcsAD_ba.init",
        "compilation": "vcs -ad=vcsAD_ba.init -debug_access
            + all -full64 -lca -o simvl -Mdir=csrcl",
        "simv_cmdline": "simvl -ucli -do vcs_ba.do"
    }
}
```

cpulimit

Specifies the maximum runtime (in seconds) allowed per simulation. The purpose of the `cpulimit` command is to prevent the overall fault campaign from becoming unresponsive due to very long fault simulations. If a simulation requires more time than the `cpulimit`, that simulation is terminated so that the fault campaign can continue.

Note:

This is a mandatory command.

Syntax

```
"cpulimit" : cpulimit
```

Description

A recommended setting for the `cpulimit` command is 2x the defect-free simulation's CPU time. When a defect is inserted, it has the potential to cause circuit problems or simulation conditions that are very time intensive to evaluate. For example, an injected defect could cause a circuit oscillation to start, which greatly increases the simulation runtime. Or an

injected defect could cause wild circuit behavior leading to numerical instability. This, too, can cause the simulation time to greatly increase. If any individual fault simulation exceeds the `cpulimit`, the tool terminates that fault simulation.

The `cpulimit` is the CPU time limit, and not the wall clock time or elapsed time. When running with a single core, the CPU time is close to the elapsed time, however when running with multi-core, the CPU time can be much greater than the elapsed time. Refer to the nominal run CPU time to pick a meaningful `cpulimit` value.

Examples

For example, if using the PrimeSim XA tool, check the log file for the *Grand Total Time*, which is the PrimeSim XA CPU time. As noted previously, for multi-core simulations, the CPU time is generally a multiple of the elapsed time.

For this example, if the elapsed time for the defect-free simulation is one hour with four cores (`-mt 4`), the CPU time or Grand Total Time reported in the PrimeSim XA log file should be around four hours. In this case, set the `cpulimit` to $4 \times 60 \times 60 \times 2 = 28800$ seconds, as follows:

```
"cpulimit" : 28800
```

create_fault_files

Description

Use this command only when you set `identify_only` to `true`. It is ignored if `identify_only` is not specified or is set to `false`.

By default, the `create_fault_files` value is `false` when `identify_only` is `true`, which means the Custom Fault tool does not create fault directories when you use it to identify defects rather than run simulations.

When set to `true`, the Custom Fault tool creates fault directories when you use it to identify defects.

Syntax

```
"create_fault_files" : true|false
```

dc_operating_point

Description

Specifies whether to force simulations to skip DC initialization or to generate initial values from nominal simulations and use these initial values in the fault simulations.

Syntax

```
"dc_operating_point" : {  
  "dc_method" : dc_method_value,  
  "dc_cpu_limit" : dc_cpu_limit_value,  
  "dc_cpu_limit_action" : "abort"|"continue"  
}
```

Arguments

Argument	Description
"dc_method" : dc_method_value	Specifies the method of DC initialization to use. Choose one of the following settings: • 0: (default) No change to DC initialization. • 1: Force DC analysis (and initialization) to be skipped. • 2: Use the nominal defect-free simulation operating point as .nodeset in the defect injection simulation. • 3: Use the nominal defect-free simulation operating point as .ic in the defect injection simulation Note: When this argument is set to 0, 2, or 3, the tool runs a nominal simulation even if the -n command-line option is not used, and even if -n no is used. If this argument is set to 1, the tool does not run a nominal simulation.
"dc_cpu_limit" : dc_cpu_limit_value	Specifies the maximum CPU time in seconds. If DC initialization takes longer than this value, the simulator exits DC initialization and then acts based on the setting of the dc_cpu_limit_action argument.
"dc_cpu_limit_action" : "abort" "continue"	Specifies the action to take if DC initialization exceeds the maximum CPU time set by the dc_cpu_limit argument. Choose one of the following settings: • abort: After exiting DC initialization, end the simulation run. • continue: (default) After exiting DC initialization, begin transient analysis.

defect_weight

Description

Specifies the weight file to read in when you run analog fault simulation.

Syntax

```
"defect_weight" : "weight_file_name"
```

Arguments

Argument	Description
"defect_weight" : " <i>weight_file_name</i> "	Specifies the weight file name that contains the C++ functions to define the weight of each defect type. If you do not specify this command, the default weight for all defect types is set to 1.

detectability

Description

Invokes the Detectable Fault Analysis (DFA) feature.

For more information, see [Using Custom Fault for Detectable Fault Analysis](#).

Syntax

```
"detectability": {
  "observation_nodes" : ["obsv1", "obsv2", "obsv3", ...],
  "tolerance" : [
    { "err" : "error_value%",
      {
        "name" : "node_name",
        "err" : "error_value%"
      }
    ],
  "spare_inst" : ["inst_name1", "inst_name2", ...]
},
```

Arguments

Argument	Description
"observation_nodes" : ["obsv1", "obsv2", "obsv3", ...]	Specifies one or more observation points, that is, the signals under observation. The observation nodes must be in the analog domain. It is not possible to specify Verilog digital signals as observation nodes. The detectability feature checks whether the voltage on these nodes can vary when a fault is injected. If the voltage cannot vary, then the fault is flagged as undetectable.

Argument	Description
<pre>"tolerance" : [{ "err" : "error_value%"}, { "name" : "node_name", "err" : "error_value%" }]</pre>	(Optional) Specifies an <i>ideal versus defect</i> voltage comparison. Default value is 5%. The tolerance can be global and apply to all signals, or you can define a tolerance local to specified nets. If specified, the tolerance must be followed by a percent (%) sign.
<pre>"spare_inst" : ["inst_name1", "inst_name2", ...]</pre>	Modifies the spare instance names. By default, any block that has the word '<i>spare</i>' or the word '<i>dummy</i>' in its name is assumed to be a spare instance. You can change the names for spare circuitry using "spare_inst". Using a wildcard is supported.

Examples

In the following example, the observation nodes are `alarm` and `vout`. A global tolerance of 3% is applied to the whole circuit, which means the signal values are assumed to be matching if they differ by less than 3%. A tighter tolerance of 1% is applied to all nodes belonging to the instance `Itop.Ildo`. All instances that have '*dummy*' or '*no_use*' in their name, and only those, are assumed to be spare circuitry and are reported with the SC category.

```
"detectability": {
  "observation_nodes": ["alarm", "vout"],
  "tolerance": [
    { "err": "3%"},
    {
      "name": "Itop.Ildo.*",
      "err": "1%"
    }
  ],
  "spare_inst": ["*dummy*", "*no_use*"]
},
```

diagnostic_coverage (for Latent Faults)

Description

The following syntax is used for [Measuring the Diagnostic Coverage of Latent Faults](#).

Syntax

```
"diagnostic_coverage" : {
  "failure_mode" : [
    "id" : "failure_mode_id" ,
    "id_label" : "failure_mode_label",
    "latent_detection_output" : [
```

```
        {"type" : "meas", "id" : "idname"},
        {"type" : "vec", "signal" : "sname"},
        {"type" : "dwc", "signal" : "sname"}
    ]
}
```

Arguments

Argument	Description
"id" : "failure_mode_id"	Identifier for a diagnostic coverage set. The <i>failure_mode_id</i> is reported in the <i>.latent_diagnostic_coverage</i> file. The <i>failure_mode_id</i> can be any user-defined string.
"id_label" : "failure_mode_label"	The <i>failure_mode_label</i> can be any user-defined string. It defines the header in the <i>.latent_diagnostic_report</i> output file.
"latent_detection_output" : []	Declares all the latent safety mechanism observation points for the diagnostic coverage of latent faults. The <i>latent_detection_output</i> cannot be contained in the diagnostic coverage command at the same time as functional output and detection output.
"type" : "meas" "vec" "dwc"	Declares the type of observations point. Measures must have an associated <i>set_sim_stop</i> command identified by the <i>id</i> .
"id" : "idname"	Identifies a set of <i>set_sim_stop</i> commands to associate with the observation point. Note: The <i>idname</i> is always converted and processed in lowercase. If an <i>idname</i> contains any uppercase characters, the Custom Fault tool converts it to lowercase and issues a warning in the log file. Note also that the <i>set_sim_stop</i> command's <i>-id id_name</i> option is also processed in lowercase only.
"signal" : "sname"	Declares the hierarchical signal name used in the VEC/VCD or digital waveform comparison.

diagnostic_coverage (for Residual Faults and Direct SPFM Calculation)

The following syntax is used for [Measuring the Diagnostic Coverage of Residual Faults](#) and for [Direct SPFM Calculation](#).

Syntax

```
"diagnostic_coverage" : {
  "allow_safe_fault" : true|false,
  "failed_sim" : "assigned_category",
  "failure_mode_distribution_method" : 1|2,
  "fsafe_method" : 1|2,
  "report_spfm" : true|false,
  "failure_mode" : [
    {
      "id" : "id_report_name",
      "id_label" : "failure_mode_label",
      "functional_output" : [
        { "type" : "meas", "id" : "idname" },
        { "type" : "vec", "signal" : "sname" },
        { "type" : "dwc", "signal" : "sname" },
        { "type" : "dwc", "id" : "tag1" }
      ],
      "detection_output" : [
        { "type" : "meas", "id" : "idname" },
        { "type" : "vec", "signal" : "sname" },
        { "type" : "dwc", "signal" : "sname" },
        { "type" : "dwc", "id" : "tag2" }
      ]
    }
  ]
}
```

Arguments

Argument	Description
"allow_safe_fault" : true false	The default for allow_safe_fault is true. When false, no injected defect is classified as safe. All injected defects are classified as either dangerous detected or dangerous undetected.
"failed_sim" : "assigned_category"	Specifies that a failed simulation is classified as one of the following: - dangerous_undetected (default) - dangerous_detected - safe_undetected - safe_detected Note: This argument has priority over the allow_safe_fault argument.

Argument	Description
<code>"failure_mode_distribution_method" : 1 2</code>	The <code>failure_mode_distribution_method</code> argument determines the failure mode distribution for the SPFM calculation. Weights of defects allocated as safe are not included in the failure mode distribution calculation. The sum of all failure mode distributions should be 1 or 100%. For details, see Table 46.
<code>"fsafe_method" : 1 2</code>	The <code>fsafe_method</code> argument defines how the percentage of Fsafe (safe faults) for a failure mode is calculated. A fault counted as safe does not contribute to any failure mode and most FMEDAs do not have a row for safe faults. Safe faults are allocated to one of the defined failure modes. For details, see Table 47.
<code>"report_spfm" : true false</code>	The default for <code>report_spfm</code> is <code>false</code>. When <code>true</code>, an additional report is printed at the end of the <code>.diagnostic_report</code> file. For details, see SPFM Output.
<code>"id" : "id_report_name"</code>	This is an identifier for a diagnostic coverage set. The <code>failure_mode_id</code> is reported in the <code>.latent_diagnostic_coverage</code> file. The <code>failure_mode_id</code> can be any user-defined string.
<code>"id_label" : "failure_mode_label"</code>	The <code>failure_mode_label</code> can be any user-defined string. It defines the header in the <code>.diagnostic_report</code> output file.
<code>"functional_output" : []</code>	Declares all the functional output observation points.
<code>"detection_output" : []</code>	Declares all the safety mechanism observation points
<code>"type" : "meas" "vec" "dwc"</code>	Declares the type of each observation point. Measures must have an associated <code>set_sim_stop</code> command identified by the <code>id</code>.

Argument	Description
<code>"id" : "idname"</code>	Identifies a set of <code>set_sim_stop</code> commands to associate with the observation point. Note: The <code>idname</code> is always converted and processed in lowercase. If an <code>idname</code> contains any uppercase characters, the Custom Fault tool converts it to lowercase and issues a warning in the log file. Note also that the <code>set_sim_stop</code> command's <code>-id id_name</code> option is also processed in lowercase only.
<code>"signal" : "sname"</code>	Declares the hierarchical signal name used in the VEC/VCD or Digital Waveform Comparison. Note: If you specify <code>"signal"</code> for Digital Waveform Comparison, there is no need to specify <code>"id"</code>.
<code>"id" : "tagN"</code>	(Optional) Declares the time windows (<code>tag1</code> for functional output and <code>tag2</code> for detection output) used in Digital Waveform Comparison. Note: If you specify <code>"id"</code>, there is no need to specify <code>"signal"</code> for Digital Waveform Comparison.

Description

The following tables describe the settings for the `failure_mode_distribution_method` (Table 46) and `fsafe_method` (Table 47) arguments and how they interact to determine the equation for `Fsafe` (Table 48).

Table 46 *failure_mode_distribution_method* Settings

Setting	Description
1	The failure mode distribution (Dfm) is divided equally among defined failure modes. $Dfm_i = 1 / (\text{number of defined failure modes})$
2	(Default) The failure mode distribution is derived from the fault campaign, that is, the weight of defects causing that failure mode divided by the sum of the weight of all defects contributing to a failure mode. $Dfm_i = \text{Weight}_i / \text{sum}(\text{failure_mode_weights})$ Failure modes should be mutually exclusive. If a defect causes multiple failure modes, the weight is split equally to each associated failure mode.

Table 47 *fSAFE_method Settings*

Setting	Description
1	FSAFE weight is allocated equally to each failure mode.
2	(default) FSAFE weight is allocated proportionally to each failure mode according to the failure mode distribution method.

Table 48 *FSAFE Equation based on failure_mode_distribution_method and fSAFE_method Arguments*

FMD Method	FSAFE Method	FSAFE Equation
1 (equal)	1 (equal)	$\frac{\text{safe_weight}}{\left(\frac{\text{safe_weight}}{\text{\#failure_modes}} + \frac{\text{sum(failure_mode_weights)}}{\text{\#failure_modes}} \right)} = \text{safe_weight} / \text{total_weight}$
1 (equal)	2 (proportional)	$\frac{\text{safe_weight} * \text{Dfm_i}}{\text{sum(failure_mode_weights)} * \text{Dfm_i}} = \text{safe_weight} / \text{total_weight}$
2 (measured)	1 (equal)	$\frac{\text{safe_weight}}{\left(\frac{\text{safe_weight}}{\text{\#failure_modes}} + \text{Weight_Failure_Mode_i} \right)}$
2 (measured)	2 (proportional)	$\frac{\text{safe_weight} * \text{Dfm_i}}{\text{Weight_i}} = \text{safe_weight} / \text{total_weight}$

Note:

Three of the settings in [Table 48](#) simplify to the same equation:

$$\text{FSAFE} = \text{safe_weight} / \text{total weight}$$

dpconfig_file

Description

Specifies the Hosts Information file for distributed parallel processing capability. See [Common DP Library Information](#) for more details. This is a mandatory command.

Syntax

"dpconfig_file" : "hostfile"

enable_fault_collapse

Description

Specify `false` to disable fault collapse. The default is `true`.

Syntax

```
"enable_fault_collapse" : true|false
```

enable_stackmos_fault_collapse

Specify `true` to enable fault collapsing of an array of MOSFETs or *stackmos*. The default setting is `false`.

Syntax

```
"enable_stackmos_fault_collapse" : true|false
```

Description

An array of identically-sized transistors called a *stackmos* is sometimes used to build larger effective transistors. With normal defect identification, most transistors in the array are considered independent because they do not share common nodes. However, this array structure allows for fault collapsing because the effects of many defects are the same. When stackmos fault collapsing is enabled, the Custom Fault tool assigns the stackmos defects to three primary groups: the drain edge, the interior, and the source edge. The defects in each group are assumed to be equivalent.

Examples

The following example illustrates the difference between standard fault collapsing and stackmos fault collapsing.

The stackmos shown in [Figure 74](#) has 20 MOSFETs in the circuit netlist. This example assumes that no short or open defects use the bulk terminal. It also assumes that all MOSFETs in the stackmos have identical geometries and therefore identical weight.

Figure 74 Stackmos Example

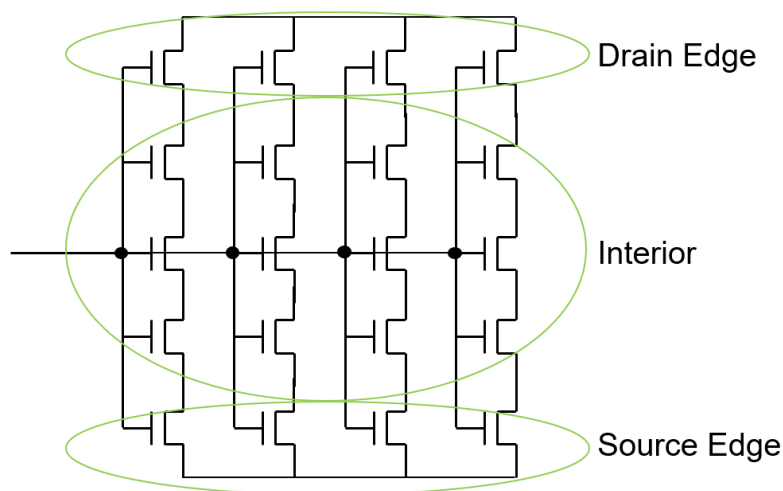


Table 49 lists the defects identified when using standard identification.

Table 49 Standard Identification and Fault Collapsing

Defect	Fault Simulations	Group Defect Weight After Collapsing Equivalent Defects	Total Weight of Defects
Terminal open (drain, gate, source)	60	$W_o \times 1$	$60W_o$
Gate to Drain Short Drain Edge	1	$W_s \times 4$	$4W_s$
Gate to Drain/Source of interior nodes	16	$W_s \times 2$	$32W_s$
Gate to Source short at source edge	1	$W_s \times 4$	$4W_s$
Drain to Source short at drain edge	1	$W_s \times 4$	$4W_s$
Drain to Source short interior nodes	12	$W_s \times 1$	$12W_s$
Drain to Source short at source edge	1	$W_s \times 4$	$4W_s$
Total	92		$60W_o + 60W_s$

Table 50 lists the defects identified when using stackmos identification.

Table 50 *Stackmos Identification and Fault Collapsing*

Defect	Fault Simulations	Group Defect Weight After Collapsing Equivalent Defects	Total Weight of Defects
Gate open at drain edge	1	$W_o \times 4$	$4W_o$
Drain open at drain edge	1	$W_o \times 4$	$4W_o$
Gate open at interior MOS	1	$W_o \times 12$	$12W_o$
Drain/Source open at interior MOS	1	$W_o \times 32$	$32W_o$
Gate open at source edge	1	$W_o \times 4$	$4W_o$
Source open at source edge	1	$W_o \times 4$	$4W_o$
Gate Drain short at drain edge	1	$W_s \times 4$	$4W_s$
Drain Source short at Drain edge	1	$W_s \times 4$	$4W_s$
Gate to Drain/Source short at interior nodes	1	$W_s \times 32$	$32W_s$
Drain Source short at interior MOS	1	$W_s \times 12$	$12W_s$
Gate to Source short at source edge	1	$W_s \times 4$	$4W_s$
Drain to Source short at source edge	1	$W_s \times 4$	$4W_s$
Total	12		$60W_o + 60W_s$

In the example, there is a significant reduction in the number of fault simulations required: 92 for standard identification and fault collapsing compared to only 12 when stackmos identification and fault collapsing is enabled. This reduction in the number of simulations increases as the size of the stackmos increases.

Stackmos fault collapsing has the following limitations:

- Any defect in a stack column is assumed to be identical to the matching defect of all other stack columns and have the same defect detection status.
- Defects in the interior of the stack are also assumed to have the same defect status. There are scenarios where, either by circuit design or the threshold set, the defect detection status of the interior MOSFETs is not identical. This leads to different reported values for the coverage between regular defect identification and stackmos identification.

These limitations must be considered and engineering judgement applied before using stackmos fault collapsing for test coverage, post silicon failure analysis, or functional safety applications.

fault_list_file

Description

Provides a defect list file with user-defined defect models to be simulated.

Syntax

```
"fault_list_file" : "file_name"
```

Arguments

Argument	Description
"fault_list_file" : "file_name"	Specifies the user-defined defect list file name or the fault database file name. The user-defined defect list file contains the list of user-defined defect models to be injected for analog fault simulation. See the Defect List File Format section for details. The fault database file can also be an input to the tool. See Overview of the Multi-Testbench Flow for details. If both the <code>fault_list_file</code> command and <code>-f</code> command-line option are specified, the <code>-f</code> command-line option take precedence.

exclude_fault_list_file

Description

Excludes a list of defects from simulation. When using this command, two coverage numbers are reported in CustomFault report file. One coverage totally excludes the count of the faults listed, another coverage counts the excluded faults as simulated but not covered.

Syntax

```
"exclude_fault_list_file" : "file_name"
```

Arguments

Argument	Description
"exclude_fault_list_file" : "file_name"	Specifies the user-defined defect list file name to be excluded from simulation. See the Defect List File Format section for details. The fault database file can also be an input to the tool. See Overview of the Multi-Testbench Flow for details. If both the <code>exclude_fault_list_file</code> command and <code>-f</code> command-line option are specified, the <code>-f</code> command-line option take precedence.

fault_sim_output

Description

Provides some control and flexibility to the processing of the fault simulation output.

For example, if your custom output from a Verilog/VHDL or Verilog-A file contains error or other messages that the Custom Fault tool should ignore, use this command with the `ignore_message` argument.

Additionally, if there are certain warnings or other messages from the fault simulator that you want to promote and read as an error, use this command with the `error_message` argument.

As noted in [Using Simulation Output Messages](#), you can use the `detection_message` argument to scan the fault simulation output for specific messages using regular expressions. If these specific messages are found, the defect simulated in that simulation can be considered as detected.

The `detection_message` argument can detect most arbitrary messages. It can also be used to extract numeric real values from a message. If those values violate user-defined thresholds, then the simulated defect is considered detected.

Each fault simulation has its STDOUT and STDERR redirected to a `.stderr` file in the fault simulation directory, where the file is scanned for regular expressions that you define.

Note:

The regular expressions must not contain any of the following patterns, as they are already specially handled by the Custom Fault tool before any attempt to match the detection message regular expressions:

- `Error:`
- `ERROR!`

- `Error-[`
- `Warning: measure`
- `WARNING! measure`

The regular expressions must use the regular expression syntax of the C++ 11 `std::regex` library, but this is consistent with the regular expression syntax of most other languages using regular expressions.

The regular expression can contain up to four subgroups. These subgroups must be interpreted as real numbers. The following regular expression can be used to capture real number patterns such as 12.34, -56.78, 6e-10, 4e+3, and so on.

```
( [-+] ? [0-9] * \\. ? [0-9] + (?: [eE] [-+] ? [0-9] + ) ? )
```

This complex pattern can be substituted with a macro, `#cfnum`. Wherever `#cfnum` appears in the regular expression pattern, it is substituted with the preceding regular expression. Refer to the examples for more details on how to use this feature.

The regular expression pattern is provided in the Custom Fault configuration file in JSON syntax, so any special character for JSON must be escaped. This most commonly occurs when the regular expression pattern must match a literal dot, plus sign, question mark, parenthesis, and so on. In regular expression syntax, these symbols must be escaped to make the match literal. The escape character, `\`, must also be escaped one time for the JSON parsing of the configuration file. For example, to match a literal dot, the following expression must be used: `"\\. "`

Syntax

```
"fault_sim_output" : {
  "ignore_message" : [ "pat1", "pat2", ... ],
  "error_message" : [ "epat1", "epat2", ... ],
  "detection_message" : [
    {
      "regex" : "regular_expression",
      "id" : "id_name",
      "groups" : [
        {
          "group" : group_number,
          "id" : "id",
          "min" : min_threshold,
          "max" : max_threshold
        }
      ]
    }
  ]
}
```

Arguments

Argument	Description
<code>"ignore_message" : ["pat1", "pat2", ...]</code>	Lines of the fault simulation <code>stderr</code> file that contain the ignore string patterns are skipped and not processed.
<code>"error_message" : ["epat1", "epat2", ...]</code>	Lines of the fault simulation <code>stderr</code> file that contain the error string patterns are interpreted as a simulator error.
<code>"detection_message" : []</code>	Lines of the fault simulation <code>stderr</code> file that contain the detection string patterns can be used to treat the effect of a defect as being observed and detected.
<code>"regex" : "regular_expression"</code>	Defines the regular expression pattern. It may contain up to four groups. Note: The regular expression escape character requires an extra escape to pass Custom Fault tool parsing.
<code>"id" : "id_name"</code>	Required if the regular expression does not contain groups; it can be used to reference this regular expression detection in the <code>diagnostic_coverage</code> command.
<code>"groups" : []</code>	Required only if the regular expression captures groups of real numbers that must be checked against a threshold. If multiple groups are defined, satisfying any group results in the defect being considered detected for test coverage. In diagnostic coverage, each group is considered separately. Thus, a functional output and a detection output can be captured from a single output message.
<code>"group" : group_number</code>	Must be a non-zero integer to associate the data in this block to a captured grouping of the regular expression.
<code>"id" : "id"</code>	Must be unique and can be used to reference this check in the <code>diagnostic_coverage</code> command.
<code>"min" : min_threshold,</code> <code>"max" : max_threshold</code>	If the captured real number group falls below the minimum threshold or above the maximum threshold, the defect is considered detected.

Examples

The examples in this section illustrate the use of the `detection_message` argument.

Example 1

In this example, a simple message is output by the fault simulator when the circuit operation is incorrect and is considered a defect detection.

```
"fault_sim_output" : {  
  "detection_message": [  
    {  
      "regex": "^Circuit block 1 output is wrong$",  
      "id": "block1_error"  
    },  
    {  
      "regex": "^Circuit block 2 output is wrong$",  
      "id": "block2_error"  
    }  
  ]  
}
```

Example 2

In this example, a simple message is output by the fault simulator when the circuit operation is incorrect, but the message includes some data or instance path that varies. For this example, assume that the following messages indicate defect detection:

Output on channel 4 is incorrect.

Output on channel 17 is incorrect.

where the numbers 4 and 17 could be other numbers that also indicate a defect detection.

```
"fault_sim_output" : {  
  "detection_message": [  
    {  
      "regex": "^Output on channel #cfnum is incorrect\\.\\.\"",  
      "id": "channel_error"  
    }  
  ]  
}
```

Example 3

In this example, a numeric value from the message is captured and compared to a threshold.

```
"fault_sim_output" : {  
  "detection_message": [  
    {  
      "regex": "Info: the output value is #cfnum",  
      "groups" : [  
        {  
          "id" : 1,  
          "min" : 10.0,  
          "max" : 15.4  
        }  
      ]  
    }  
  ]  
}
```

If the simulation output contains:

Info: the output value is 9.3

it is considered a detected and covered defect because 9.3 is below the minimum threshold..

If the simulation output contains:

Info: the output value is 13.8

it is not considered a detected defect because 13.8 is between the minimum and maximum thresholds.

Example 4

In this example, a simple message is output by the fault simulator when the circuit operation is incorrect and is considered a functional defect detection. A detection message is also detected. Both are then linked to the `diagnostic_coverage` command.

```
"fault_sim_output" : {  
  "detection_message": [  
    {  
      "regex": "^Circuit block 1 output is wrong$",  
      "id": "block1_error"  
    },  
    {  
      "regex": "^Circuit block 2 output is wrong$",  
      "id": "block2_error"  
    }  
  ],  
  {  
    "regex": "Safety Mechanism has detected error condition",  
    "id": "SM_detection"  
  }  
],  
  "diagnostic_coverage": {  
    "failure_mode": [  
      {  
        "id": "FM001",  
        "id_label": "Block 1 failure",  
        "functional_output": [  
          {"type": "regex", "id": "block1_error" }  
        ],  
        "detection_output": [  
          {"type": "regex", "id": "SM_detection" }  
        ]  
      },  
      {  
        "id": "FM002",  
        "id_label": "Block 2 failure",  
        "functional_output": [  
          {"type": "regex", "id": "block2_error" }  
        ],  
      }  
    ]  
  }  
}
```

```

        "detection_output": [
            {"type": "regex", "id": "SM_detection" }
        ]
    }
}
]
}

```

Example 5

This example shows how numeric values can be captured and then used with the `diagnostic_coverage` command. The fault simulator outputs a message such as:

The circuit value = 2.34V; monitor reports 0

If the circuit value is less than 2 or more than 2.5, it is considered a functional failure. The voltage monitor reports 0 when it believe the circuit value is in range and 1 otherwise.

```

"fault_sim_output" : {
  "detection_message": [
    {
      "regex": "^The circuit value = #cfnumV; monitor reports #cfnum",
      "groups" : [
        {
          "group" : 1,
          "id" : "fo1",
          "min" : 2.0,
          "max" : 2.5
        },
        {
          "group" : 2,
          "id" : "do1",
          "min" : -0.5,
          "max" : 0.5
        }
      ]
    }
  ]
},
"diagnostic_coverage": {
  "failure_mode": [
    {
      "id": "FM001",
      "id_label": "circuit value out of range",
      "functional_output": [
        {"type": "regex", "id": "fo1" }
      ],
      "detection_output": [
        {"type": "regex", "id": "do1" }
      ]
    }
  ]
}

```

generate_dual_fault

Generates a dual-fault list file that can be used in step 3 of latent fault analysis (see [Measuring the Diagnostic Coverage of Latent Faults](#)).

Syntax

```
"generate_dual_fault" : [  
    // multiple dual-fault lists can be specified in a single command.  
    {  
        "mode" : mode_val,  
        "mission_fdb" : "path_to_mission_fdb_file",  
        "safety_fdb" : "path_to_safety_mechanism_fdb_file",  
        "output_file" : "name_of_output_file"  
    }  
]
```

Arguments

Argument	Description
"mode" : mode_val	mode and mode_val must be provided. Valid values of mode_val are: • 0 – Take the smallest margin of failure from each mission failure mode's dangerous detected defects and combine it with all possible defects of the safety mechanism. • 1 – Take the largest margin of failure from each mission failure mode's dangerous detected defects and combine it with all possible defects of the safety mechanism • 2 – Take both the smallest and largest failure from each mission failure mode's dangerous detected failure and combine it with all safety mechanism defects. • 3 – Take all mission dangerous detected defects and combine them with all safety mechanism defects.
"mission_fdb" : "path_to_mission_fdb_file"	Specifies the .fdb file from the fault campaign which defines the mission circuit dangerous detected defects.
"safety_fdb" : "path_to_safety_fdb_file"	Specifies the .fdb file from the fault campaign which defines the safety mechanism undetected defects.
"output_file" : "filename"	Specifies the output file name to contain the dual-fault list. It is placed in the -o directory.

Description

In order to generate the dual-fault list file, steps 1 and 2 of latent fault analysis must first be performed. Step 1 is the identification of dangerous detected defects in the mission circuit. Step 2 is the exclusion of safe faults from the safety mechanism. The .fdb and

`.failure_margin` files for each failure mode of step 1 and step 2 must be available to use this command.

The command can generate the dual-fault list for any or all of the smallest, largest, smallest and largest, or all of the mission circuit dangerous detected defects.

Examples

The following example shows how to generate a dual-fault list for all four modes simultaneously. Four output files are written to the tool output directory with the names specified by the `output_file` option.

```
{
  "generate_dual_fault" : [
    {
      "mode" : 0,
      "mission_fdb" : "/project/cf.krf/cf.krf.fdb",
      "safety_fdb" : "/project/cf.smstep2/cf.smstep2.fdb",
      "output_file" : "small.dfl"
    },
    {
      "mode" : 1,
      "mission_fdb" : "/project/cf.krf/cf.krf.fdb",
      "safety_fdb" : "/project/cf.smstep2/cf.smstep2.fdb",
      "output_file" : "large.dfl"
    },
    {
      "mode" : 2,
      "mission_fdb" : "/project/cf.krf/cf.krf.fdb",
      "safety_fdb" : "/project/cf.smstep2/cf.smstep2.fdb",
      "output_file" : "both.dfl"
    },
    {
      "mode" : 3,
      "mission_fdb" : "/project/cf.krf/cf.krf.fdb",
      "safety_fdb" : "/project/cf.smstep2/cf.smstep2.fdb",
      "output_file" : "all.dfl"
    }
  ]
}
```

grouped_waveform

Description

Specify `true` to generate grouped waveforms. A grouped waveform is generated only for FSDB format. There is one grouped waveform per defect category.

If you run the Custom Fault tool using the `-n yes` command-line option to enable a nominal (defect-free) run, the nominal FSDB is added to the grouped waveform files.

The grouped waveform files are named `output_prefix.fault_category.fsdb.grp`, where `output_prefix` is determined based on the `-o|--out` command-line option.

The default is `false` where no grouped waveforms are generated.

Note:

Large grouped waveforms can take minutes to load in a waveform viewer, so setting this option to `true` is not recommended when many defects are injected.

Syntax

```
"grouped_waveform" : true|false
```

identify_only

Description

Specify `true` if you want to identify the defects and not run simulations. Otherwise specify `false` to identify and run the simulations.

Syntax

```
"identify_only" : true|false
```

nc_analysis

Description

Specify `true` (default) to run a [Not-covered Defect Analysis](#) which produces `.nc_analysis.dat` and `.nc_analysis.csv` files in the output directory. Specify `false` to not run the analysis.

Syntax

```
"nc_analysis" : true|false
```

outdir

Specifies the name of output directory to store all the intermediate files and simulation results generated by the tool.

Syntax

```
"outdir" : "outdirname"
```

Description

If it is not specified, the default is equivalent to the output directory specified by `-o` command-line option. A directory with the tag name specified with `-t` command-line option

is created under this output directory, which contains the intermediate files and simulation results of all fault simulations. You can use `cleanup_level` command to choose the files to keep.

parameter_variation

Description

Use this command to perform Parametric Defect Injection (PDI).

For more information, see [Custom Fault Parametric Defect Injection](#).

Syntax

```
"parameter_variation" : {  
    "template" : true | false,  
    "inject" : true | false,  
    "parameter_file" : "path/to/mc0/or/JSON/parameter/file",  
    "missing_values_nominal" : true | false  
}
```

Arguments

Argument	Description
"template" : true false	Defaults to <code>false</code> . If set to <code>true</code> , the tool generates a template parameter file in <code>.mc0</code> format for the circuit. Note: The <code>template</code> and <code>inject</code> arguments cannot be set to <code>true</code> at the same time.
"inject" : true false	Defaults to <code>false</code> . If set to <code>true</code> , the tool runs a defect injection campaign. Note: The <code>template</code> and <code>inject</code> arguments cannot be set to <code>true</code> at the same time.
"parameter_file" : "path/to/mc0/or/JSON/parameter/f ile"	Defines the parametric defects to inject during the fault simulation. This must be provided if <code>inject</code> is set to <code>true</code> . The file can be either <code>.mc0</code> format or JSON format.
"missing_values_nominal" : true false	Defaults to <code>true</code> . The input <code>.mc0</code> file only has to define the independent random variables (IRV) from the netlist that you want to set explicitly. All other IRVs from the netlist can be set to their nominal value, 0, or to a random value according to each IRV's defined distribution. Setting the value of this command to <code>true</code> assigns a value of 0 to each undefined variable and setting it to <code>false</code> causes a random value to be assigned to each variable.

report

Description

Specifies whether to report failed measurements as covered (CF) or not covered (NC) defects, how often to perform an incremental update of the `.fdb` and `.report` files, and controls other report-related settings.

Syntax

```
"report": {
  "failed_measure" : "CF"|"NC"
  "flush_period" : {
    "hour"|"hours" : value_hours,
    "min"|"mins" : value_minutes
  }
  "skip_zero_weight" : true|false,
  "test_coverage" : "simple"|"detailed",
  "block_coverage" : hier_level_value,
  "violation_info" : true|false,
  "violation_sort" : true|false,
  "test_order" : true|false,
  "run_times" : true|false,
  "collapse_info" : true|false,
  "measure_summary" : true|false,
  "covered_node" : true|false,
  "avalon" : true|false,
  "yaml" : 0|1
}
```

Arguments

Argument	Description
"failed_measure" : "CF" "NC"	When a measurement fails, it is reported as "failed" in the measurement output file. By default, the Custom Fault tool assumes the defect is covered and assigns it a CF (covered by failed measure) status. If you want the failed measurement to be a not-covered defect, use this command to set "failed_measure" to "NC".
"flush_period" : {...}	By default, the <code>.fdb</code> and <code>.report</code> files are updated incrementally every 15 minutes. Use the "hour" and "min" arguments to change the period.
"hour" "hours" : value_hours	Specifies an unsigned integer to set the number of hours between incremental updates of the <code>.fdb</code> and <code>.report</code> files.

Argument	Description
<code>"min" "mins" : <i>value_minutes</i></code>	Specifies an unsigned integer to set the number of minutes between incremental updates of the <code>.fdb</code> and <code>.report</code> files.
<code>"skip_zero_weight" : true false</code>	If set to <code>false</code>, the Custom Fault tool simulates <code>weight=0</code> defects. When <code>true</code> (default), the Custom Fault tool does not simulate <code>weight=0</code> defects. If defects are collapsed and the group weight is 0, they are skipped, but if any defect in the collapsed group has a non-zero weight, it is simulated.
<code>"test_coverage" : "simple" "detailed"</code>	Specifies the output format for reporting test coverage results: <code>simple</code> (default) produces the one-line-per-defect report shown in Simple Format Report. <code>detailed</code> produces a verbose report similar to that shown in Detailed Format Report. Note: When using the <code>diagnostic_coverage</code> command, the report format is always <code>detailed</code> and cannot be changed to <code>simple</code>.
<code>"block_coverage" : <i>hier_level_value</i></code>	Reports the coverage for each subcircuit in the design by adding an extra section with sub-block coverage. The <code>cfs.report</code> file shows the coverage per defect type and the total coverage. In the case of the multi-testbench flow, the coverage per testbench is reported. Choose one of the following options to specify the hierarchy level for the sub-block coverage report: 0 – (default) Report only the total coverage. 1 – Report the first level of hierarchy together with total coverage. 2 – Report the first and second levels of hierarchy together with total coverage. 3 – Report the first, second, and third levels of hierarchy together with total coverage. See Reporting Sub-block Coverage for details.
<code>"violation_info" : true false</code>	Specify <code>true</code> to generate a new file with all the violation information. The extension for the new file is <code>.tag_name.violation_info</code>. The default is <code>false</code>.
<code>"violation_sort" : true false</code>	Specify <code>true</code> to sort in time order the information output to the <code>.violation_info</code> file. The default is <code>false</code>.

Argument	Description
<code>"test_order": true false</code>	Creates a file that estimates the number and the names of tests that must be run to obtain the total coverage. This is useful in the multi-testbench flow. When set to <code>true</code>, the Custom Fault tool estimates the number and the names of the tests that must be run to obtain the total coverage. This feature is used to limit the number of simulations required to reach the total coverage. The estimation is done based on the defects being covered by each test and their overlap or non-overlap. When <code>test_order</code> is set to <code>true</code>, the Custom Fault tool generates an output file with a <code>.test_order</code> extension. The default is <code>false</code>.
<code>"run_times" : true false</code>	Specify <code>true</code> to generate a new file with all the runtime information—CPU time, wall clock time, and peak virtual memory—for each simulation (except the nominal simulation if <code>-n yes</code> is used) and to include the sums of all simulation CPU times and wall clock times in the log file. The extension for the new file is <code>.tag_name.runtime</code>. The default is <code>false</code>.
<code>"collapse_info" : true false</code>	Specify <code>true</code> to generate an exhaustive list of instance/node/port names, including those from collapsed defects. The default is <code>false</code>. When defects are collapsed, the group weight is the sum of their individual weights and the information for only one instance/node/port name of the group is reported..
<code>"measure_summary" : true false</code>	Specify <code>true</code> to generate a summary of defect detections by <code>.MEASURE</code> statements. See Measurement Detection Summary for details. The default is <code>false</code>.
<code>"covered_node" : true false</code>	Specify <code>true</code> to generate a CSV format report file listing only nodes from covered defects, sorted by count. The extension for the report file is <code>.covered_node.list</code>. The default is <code>false</code>.
<code>"avalon" : true false</code>	Specify <code>true</code> to generate an additional <code>cf.avalon</code> file. The <code>cf.avalon</code> file can be read by the Avalon tool to identify the specified instances in the Avalon schematic and layout view. This file is identical to the test coverage report with the following exceptions: • Only covered defects are reported in the file. Uncovered defects are not reported. • All lines other than the defect lines are prefixed with a pound sign (#).

Argument	Description
"yaml" : 0 1	Specify 1 to output a .yaml file. See YAML Report Format for details. The default is 0.

Reporting Sub-block Coverage

The coverage of a sub-block is the "sum of detected defects in the sub-block multiplied by weight" divided by the total number of defects injected in that same sub-block.

Note:

[Net to Net Short Defects](#) are excluded from the sub-block coverage.

Consider a sub-block x1.x5. The current .report file shows:

```

A7.7 net_to_net_short NC
fault_net_to_net_short_7_7_7/sim.* 1.000000e+00 x1.x5.n1, x1.n3
A9.9 mos_short NC fault_mos_short_9_9_9/sim.*
4.000000e+00 x1.x5.mpl a[3], vdd
A78.78 mos_short NC fault_mos_short_78_78_78/sim.*
4.000000e+00 x1.x5.mpl vdd, x1.an
A263.263 mos_short CL fault_mos_short_263_263_263/sim.*
6.000000e+00 x1.x5.mn1 a[3], x1.an
A1440.1440 mos_open CU fault_mos_open_1440_1440_1440/sim.*
5.000000e-01 x1.x5.mn1 x1.an
A1441.1441 mos_open NC fault_mos_open_1441_1441_1441/sim.*
5.000000e-01 x1.x5.mn1 a[3]
A1442.1442 mos_open NC fault_mos_open_1442_1442_1442/sim.*
5.000000e-01 x1.x5.mn1 0
A1443.1443 mos_open NC fault_mos_open_1443_1443_1443/sim.*
5.000000e-01 x1.x5.mn1 0
A1444.1444 mos_open NC fault_mos_open_1444_1444_1444/sim.*
5.000000e-01 x1.x5.mpl x1.an
A1445.1445 mos_open NC fault_mos_open_1445_1445_1445/sim.*
5.000000e-01 x1.x5.mpl a[3]
A1446.1446 mos_open NC fault_mos_open_1446_1446_1446/sim.*
5.000000e-01 x1.x5.mpl vdd
A1447.1447 mos_open NC fault_mos_open_1447_1447_1447/sim.*
5.000000e-01 x1.x5.mpl vdd

```

The total number of injected defects (multiplied by their weight) is:

$$4+4+6+0.5*8 = 18$$

The number of covered defects is 6 (covered with CL) + 0.5 (covered with CU).

The coverage of sub-block x1.x5 is:

$$6.5/18 = 36.11\%$$

Note:

The `net_to_net_short` defect, A7.7, is not counted because `net_to_net_short` defects are excluded from the sub-block coverage.

To report the hierarchical coverage down to the second level of hierarchy, add the following command in the `config.json` file:

```
"report": {
  "block_coverage" : 2
},
```

In the `cfs.report` file, the sub-block coverage is highlighted below:

```
*DEFECT UNIVERSE SUMMARY

      mos_open           1008
      mos_short          1331
net_to_net_short         1
      TOTAL             2340

*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
      Type              test
      mos_open          157/504
      mos_short         1019/1331
net_to_net_short         0/1
      TOTAL             1176/1836

Weighted test coverage:
      test              64.09%

Sub-block coverage:
      M1                0.00%
      M0                0.00%
      X1                27.04%
      X1.mp0            0.02%
      X1.X1             12.01%
      X1.x2             2.00%
      X1.x3             13.01%
      X2                13.02%
      X2.x1             10.01%
      X2.x2              3.01%
      X3                24.03%

TOTAL Coverage:
      Non-Sampled       64.09%
```

When running with the multi-testbench flow, the report is in the `.report` files if the `block_coverage` argument is used. It is recommended that all testbenches use the same hierarchy level value for `block_coverage`.

If you use:

```
"report": {
  "block_coverage" : 1
},
```

the `.report` of the first run shows, for example:

```
*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
      Type          sim_1st_vec
mos_open           405/656
mos_short          788/864
TOTAL             1193/1520

Weighted test coverage:
sim_1st_vec        78.49%

Sub-block coverage:    sim_1st_vec
M1                    0.00%
M0                    0.00%
X1                    27.04%
X2                    13.02%
X3                    38.43%

TOTAL Coverage:
Non-Sampled          78.49%
```

while the `.report` of the second and last run shows, for example:

```
COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
      Type          sim_1st_vec          sim_2nd_vec          Total (Union)
mos_open           405/656           20/251           425/656
mos_short          788/864           1/76           789/864
TOTAL             1193/1520           21/327           1214/1520

Weighted test coverage:
sim_1st_vec        78.49%
+ sim_2nd_vec      79.01%

Sub-block coverage:  sim_1st_vec          sim_2nd_vec          Total (Union)
M1                    0.00%           0.00%           0.00%
M0                    0.00%           0.00%           0.00%
X1                    27.04%           2.05%           27.15%
X2                    13.02%           1.30%           13.20%
X3                    38.43%           3.07%           39.52%

TOTAL Coverage:
Non-Sampled          79.87%
```

Measurement Detection Summary

In test coverage applications, use the Measurement Detection Summary (a summary of defect detections by `.MEASURE` statements) to minimize the number of checkpoints on the test equipment by eliminating redundant `.MEASURE` statements from your testbenches.

The Measurement Detection Summary is a `.csv` file, `outfile.tag.meas_summary.csv`, where `outfile` is specified by the `-o|--out` command-line option and `tag` is specified by the `-t|--tag` command-line option.

For example, if the `.report` includes the following:

```
Fault ID: A1.1
  Defect Type: mos_short_ds
  Weight: 1.000000e+00
  Instance: x0.mpl
  Ports: s d
  Port Names: vdd x0.d1

  Tag: tb1
  Covered: no

Data: /
sortedsample/multitb2/cf.cfg1.json/tb1.1/fault_mos_short_1_1_1/mos_short_
1.*

Code: NC
Type: not_covered

Fault ID: A16.16
  Defect Type: net_to_net_short
  Weight: 1.000000e+00
  Ports: 3 20

  Tag: nom
  Covered: yes
  Data: /
passive/cf.cfg1.json/nom.1/fault_net_to_net_short_16_16_16/net_to_net_sho
rt_16.*

Code: CU
Type: meas
Name: meas1
Value: 6.000000e-01
Violation-Condition: > 0.51
Violation-Time: 1.000000e-07

Fault ID: A81.81
  Defect Type: mos_short_ds
  Weight: 1.000000e+00
```



```

Instance: x15.mp1
Ports: s d
Port Names: vdd x15.d1

Tag: tb1
Covered: yes

Data: /
sortedsample/multitb2/cf.cfg1.json/tb1.1/fault_mos_short_81_81_81/mos_sho
rt_81.*

Code: CU
Type: meas
Name: f15
Id: f15
Value: 9.781170e-01
Violation-Condition: > 0.5
Violation-Time: 1.000000e-08

```

The corresponding Measurement Detection Summary .csv file contains the following:

```

Defect, f15, meas1
A1.1_mos_short_ds_x0.mp1_s_d, 0, 0
A16.16_net_to_net_short_3_20, 0, 1
A81.81_mos_short_ds_x15.mp1_s_d, 1, 0

```

YAML Report Format

When the `yaml` argument is set to 1, a .yaml file is created with the following information for each defect:

```

fault_info:
- type:
  tag:
  instance_name:
  ports:
    - <list of ports>
  net_names:
    - <list of nets>
  likelihood_info:
    weight:
    gweight:
result:
- tag:
  covered:
  details:
    - code:
      id:
      type:

```

resume_data

Lets you re-simulate a subset of a completed fault campaign according to the relevant sections listed in the `.diagnostic_report` file

Syntax

```
"resume_data" : {  
  "coverage_category" : ["category_1", "category_2", "category_3",  
    ...],  
  "fault_id" : ["id_1", "id_2", "id_3", ...] ,  
  "failure_mode_classification" : ["class_1", "class_2", "class_3", ...  
}
```

Arguments

Argument	Description
"coverage_category" : ["category_1", "category_2", "category_3", ...	Lets you re-simulate a subset of a completed fault campaign according to the defect categories listed in the <code>.diagnostic_report</code> file, such as F, NC, CL, CU, and so on. For example: "resume_data" : { "coverage_category" : ["F", "NC", "CL", "CU"]},
"fault_id" : ["id_1", "id_2", "id_3", ...	Lets you re-simulate a subset of a completed fault campaign according to the fault Id's listed in the <code>.diagnostic_report</code> file, such as A1.1, A2.2, A3.3, and so on. For example: "resume_data" : { "fault_id" : ["A1.1", "A2.2", "A3.3"]},
"failure_mode_classification" : ["class_1", "class_2", "class_3", ...	Lets you re-simulate a subset of a completed defect universe according to the classifications listed in the <code>.diagnostic_report</code> file, such as dangerous_detected, dangerous_undetected, safe_detected, safe_undetected, and so on. For example: "resume_data" : { "failure_mode_classification" : ["dangerous_undetected"]},

Description

You use the `resume_data` command reduce unnecessary effort when you want to re-simulate a small portion of a completed fault campaign. You use this command with

`--resume` command-line option. If you do not specify this command-line option, the `resume_data` command is ignored. You also must specify:

- The name of the FDB file from the completed simulation with the `-f` command-line option.
- The same configuration files you used to rerun the completed simulation, except with the `resume_data` command in the configuration file with the `-c` command-line option.
- The `-t` command-line option with the exact same tag value as in the completed run.
- The number of licenses you want to use. You can change the number of licenses to use when resuming a simulation (with the `-k` command-line option). If you do not specify the `-k` option, the tool uses one license. You should also keep the `-t` option with the exact same tag name you want to resume.

sampling

Provides the sampling controls.

Syntax

```
"sampling" : {  
  "estimate_error" : true|false,  
  "error_sample_sizes" : [size1, size2,..., size10]  
  "seed" : seed_value,  
  "size" : #sample,  
  "method" : "awrs"|"wss",  
  "max_error" : max_error_value  
}
```

Arguments

Argument	Description
"estimate_error" : true false	Controls the printing of the estimated coverage error bound (%) report. If "method" : "wss" is specified, both the Weight Sorted Sampling (WSS) estimated error and the Adaptive Weighted Random Sampling (AWRS) estimated error are generated. The default is <code>false</code> to disable the printing of the report. You can specify it as <code>true</code> to enable the printing of the report. For more information, see Estimating Error and Sample Size.

Argument	Description
<code>"error_sample_sizes" : [size1, size2,..., size10]</code>	(Optional) If <code>estimate_error</code> is set to <code>true</code>, use this argument to specify an array of up to 10 sample sizes. By default, 10 sample sizes are chosen based on a percentage of the total number of defects: 2%, 4%, 8%, 16%, 32%, 50%, 60%, 70%, 80%, and 90%. So if there are 1000 defects, the sample sizes are: 20, 40, 80, 160, 320, 500, 600, 700, 800, and 900. For more information, see Estimating Error and Sample Size.
<code>"seed" : seed_value</code>	Specifies the seed value to enable the tool to select the same samples for each fault simulation. By default, the seed value is 123456789. You can specify another integer to use as the seed value, or specify the string <code>"random"</code> (including the quotation marks) to set a random seed value.
<code>"size" : #sample</code>	Specifies the sample size for Adaptive Weighted Random Sampling (AWRS). If this command is not specified, the tool runs all the identified defects. You can also specify the sample size using <code>-s</code> command-line option.
<code>"method" : "awrs" "wss"</code>	Specifies the sampling method: <code>awrs</code> – See Using Adaptive Weighted Random Sampling (AWRS) <code>wss</code> – See Using Weight Sorted Sampling (WSS)
<code>"max_error" : max_error_value</code>	Sets the target error for a Weight Sorted Sampling (WSS) fault campaign. As soon as the error is reduced to less than the target error, the fault campaign stops. The default is 0 which is functionally equivalent to Exhaustive Fault Simulation (see Comparison of Fault Campaign Methods). Note: This argument is ignored unless <code>"method" : "wss"</code> is specified.

Description

You can also specify AWRS sampling and the sample size using a command-line option:

```
$> customfault ... -s #sample
```

The command-line option sample size takes precedence over the configuration file sampling size command. The `-s` command-line option cannot be used together with a configuration file specifying WSS sampling.

simulate_only

Controls the types of defects to be re-simulated in the multi-testbench flow (see [Custom Fault Multi-Testbench Flow](#)).

Syntax

```
"simulate_only" : {  
    "covered" : true|false,  
    "coverage_category" : [coverage_category] ,  
    "fault_id" : [list_of_fault_ids]  
}
```

Arguments

Argument	Description
"covered" : true false	When set to <code>false</code>, prevents any defect that has already been detected as covered from being re-simulated. See the Multi-Testbench Flow Tutorial for examples. The default is <code>true</code> and specifies that all defects are re-simulated. Note: This argument cannot be used at the same time as the <code>coverage_category</code> or <code>fault_id</code> arguments.
"coverage_category" : [coverage_category]	Specifies the type of defects to be re-simulated when running test coverage in the multi-testbench flow. The supported test coverage categories are: • CU: Covered by upper boundary • CL: Covered by lower boundary • CW: Covered by waveform comparison • CV: Covered by vector comparison • CF: Covered by failed measure (flat-lined signal) • CE: Covered by WV expression • CX: Covered - not categorized • NI: Not covered - Invalid <code>.measure</code> statement • NC: Not covered • NS: Not simulated • F: Not covered - Simulation failed You can specify more than one test coverage category. Every test coverage category has to be inside quotation marks ("") and separated by a comma (,).
"fault_id" : [list_of_fault_ids]	Specifies the fault ID to be re-simulated, which you can find in the <code>*.report</code> file. The fault ID always starts with the <code>A</code> character. The <code>***</code> wildcard character is supported. You can specify more than one fault ID. Every fault ID has to be inside quotation marks ("") and separated by a comma (,).

Description

The `simulate only` command has the following mutually-exclusive uses:

- The `"covered":false` argument is used in the multi-testbench flow to prevent re-simulating any defect that has already been detected as covered. This improves the throughput of the fault campaign.
- The `coverage_category` and `fault_id` arguments are used when certain specific defects must be re-simulated to provide some incremental update to the campaign result. It is a common strategy to change the testbench and the defect detection criteria to re-simulate specified defects to increase the test coverage. When used with these arguments, the `simulate only` command provides flexibility and control to select the defect criteria to be re-simulate.

The multi-testbench flow allows you to run the same design with multiple testbenches. The coverage report from each simulation contains the current simulation results and all the previous simulation results. If a specific defect model has multiple coverage categories from different simulations, the Custom Fault tool can re-simulate the defect models that match the specified coverage categories and the fault IDs in the `*.report` file.

Note:

The `simulate only` command is only for test coverage. The multi-testbench flow is not supported for diagnostic coverage.

In addition, the `"covered":false` argument cannot be used at the same time as the `coverage_category` or `fault_id` arguments.

Examples

For examples of the `simulate only` command using the `"covered":false` argument, see the [Multi-Testbench Flow Tutorial](#).

The following example illustrates the `simulate only` command using the `coverage_category` and `fault_id` arguments.

The table shows the report of the coverage category of simulation tag1 and tag2.

id	tag1	tag2
A1.1	CE	NS
A1.2	CU	NS
A1.3	NC	CE
A1.4	NC	NC

id	tag1	tag2
A1.5	CL	NS

If the `simulate_only` command in the configuration file is:

```
"simulate_only" : {
  "coverage_category" : ["CE","CU" ] ,
  "fault_id" : [ "A3.3" ]
}
```

If either coverage category of tag1 or tag2 matches with the specified criteria, then the Custom Fault tool re-simulates it. In this example, it re-simulates A1.1, A2.2, A3.3.

The next example runs two simulations. The first simulation runs with all the defects. The tag name for this simulation is `run_tag_i1` and the outputs are directed to `FAULT_RESULT/xa1.*`. The command line for the first run is:

```
$> customfault -c config_file1 -o FAULT_RESULT/xa1 -t run_tag_i1 -k 50
```

The second simulation uses the multi-testbench flow to re-simulate the failed defects (F) and the faults with IDs A1.1 and A10.10 with the following `simulate_only` command. This simulation has the tag name of `run_tag_i2` and the outputs are directed to `FAULT_RESULT/xa2.*`. The syntax of the `simulate_only` command is:

```
"simulate_only" : {
  "coverage_category" : ["F" ] ,
  "fault_id" : [ "A.1.1","A10.10" ]
}
```

The command line for the second run is:

```
$> customfault -c config_file2 -o FAULT_RESULT/xa2 -t run_tag_i2 -k 50 -f
FAULT_RESULT/xa1.fdb
```

The following coverage summary section of `FAULT_RESULT/xa2.report` contains the detail of both `run_tag_i1` and `run_tag_i2`. It shows you the individual test coverage as well as the total coverage.

```
*COVERAGE SUMMARY

Defects covered - weighted (covered/simulated):
  Type                run_tag_i1                run_tag_i2                Total (Union)
  bjt_open             0/28                0/0                0/28
  bjt_short            0/26                0/0                0/26
  cap_open             0/128               0/0                0/128
  cap_short            54/128              0/0                54/128
  mos_open             26/948              0/0                26/948
  mos_short            776/1242             4/15               780/1242
  res_open             45/102              0/0                45/102
  res_short            3/102               0/0                3/102
  TOTAL                904/2704             4/15               908/2704

Weighted test coverage:
  run_tag_i1           33.76%
  run_tag_i2           26.67%

TOTAL Coverage:
  Non-Sampled          33.91%

Total weight of fault universe: 2678.00
```

sqa

Description

Enables the Stimulus Quality Assessment (SQA) feature, for both Active SQA (ASQA) and Concurrent SQA (CSQA).

Note:

Both ASQA and CSQA can be used in the same run.

For more information, see [Custom Fault Stimulus Quality Assessment](#).

Syntax

```
"sqa" : {
  "asqa" : {
    "devices" : [ {
      "type" : "mos"
    } ],
    "csqa" : {
      "compile_only" : true|false,
      "defect_sensitization_threshold" : comparison_threshold_value
    }
  },
}
```


Arguments

Argument	Description
<code>"asqa" : { "devices" : [{ "type": "mos" }] }</code>	Enables Active SQA (ASQA) which is currently only supported for MOSFET devices. The following commands must also be included to run ASQA: • <code>case_dir</code> • <code>cpulimit</code> • <code>dpconfig_file</code> • <code>analog_setup</code>
<code>"csqa" : {...}</code>	Enables Concurrent SQA (CSQA). Unlike ASQA, CSQA runs a defect campaign. All commands required for a defect campaign are required for CSQA. Note: When you run CSQA, make sure waveform output is not disabled.
<code>"compile_only" : true false</code>	(Optional) Specify <code>true</code> if you want the Custom Fault tool to perform defect identification to determine the level of concurrency that can be achieved before choosing to actually run simulations.
<code>"defect_sensitization_threshold" : comparison_threshold_value</code>	(Optional) Specify if you want to change the comparison threshold. The default value is 1 volt. You can use the <code>.csqa.csqasense.png</code> picture (see Figure 37) to help you define the threshold.

Examples

The examples in this section demonstrate both ASQA and CSQA usage.

Example 1

The following is an example configuration file for running ASQA:

```
{
  "case_dir" : "./",
  "cpulimit" : 100,
  "dpconfig_file" : "xhost/xhost_sh_launch",
  "analog_setup" : {
    "sim_cmdline": "xa -hspice case.spi -c cmd_file"
  }
  "sqa" : {
    "asqa" : {
      "devices" : [ {
        "type" : "mos"
      }
    ]
  }
}
```

```
    } ]
  } }
}
```

Example 2

The following example shows the configuration file and the Custom Fault command line used to invoke CSQA.

Configuration file:

```
{  "cpulimit" : 3600,
   "case_dir" : ".",
   "analog_setup" : {
     "sim_cmdline" : "xa testbench.sp -ssl2"
   },
   "enable_fault_collapse": false,
   "dpconfig_file" : "./my_hosts",
   "sqa" : {
"csqa" : { }
   },
   "faults" : {
     "defaults" : {
       "short" : {
         "res" : 0.01,
         "cap" : null
       },
       "open" : {
         "res" : 1e12,
         "cap" : null
       }
     }
   }
}
```

Command line:

```
$> customfault -c config.json -t conc_sqa -o CF_RESULTS/conc_sqa -n yes -k 5
```

Example 3

The following example shows the CSQA command that specifies a sensitization threshold of 0.1.

```
"sqa" : {
  "csqa" : {
    "defect_sensitization_threshold": 0.1 }
},
```

Example 4

The following example shows how to run with the CSQA `compile_only` argument.

```
"sqa" : {
  "csqa" : {
    "compile_only": true }
},
```

The `.sqa_report` file reports the concurrency summary as follows:

*DEFECT UNIVERSE SUMMARY	
mos_open-nobulk	756
mos_short-nobulk	754
TOTAL	1510
*CONCURRENCY SUMMARY	
Defects Per Simulation	Simulations
11	9
8	12
7	7
6	4
5	11
4	93
3	67
2	84
1	145
TOTAL	432

what_if

Enables recalculation of coverage and diagnostic coverage reports without rerunning defect simulation.

Syntax

```
"what_if" : {  
    "change_weight_only" : true|false,  
    "recalculate_coverage" : true|false  
}
```

Arguments

Argument	Description
"change_weight_only" : true false	When set to <code>true</code> , recalculates the test coverage or diagnostic coverage report using a new weight definition from an existing fault campaign result without running additional fault simulations. See change_weight_only for more information.
"recalculate_coverage" : true false	When set to <code>true</code> , recalculates the test coverage or diagnostic coverage report using a new netlist (with different <code>.MEASURE</code> , <code>.set_sim_stop</code> , and any dependent <code>.PARAM</code> statements) and a new weight definition from an existing fault campaign result without running additional fault simulations. See recalculate_coverage for more information.

Description

This command groups the following related functionality for what-if style analysis.

change_weight_only

When enabled, a `defect_weight` file and a prior fault campaign FDB file must be provided. The prior fault campaign FDB file is provided using either the `fault_list_file` command or the `-f|--fdb` command-line option.

For exhaustive and Weighted Sorted Sampling (WSS) fault campaigns, the coverage report can be updated directly.

If the prior fault campaign was generated via Adaptive Weighted Random Sampling (AWRS), the new coverage report is recalculated using WSS methods. AWRS samples are chosen based on the defined defect weight distribution while the coverage and confidence interval are calculated based on the total defect universe weight distribution. The samples chosen for a given weight definition cannot be used to calculate the coverage and confidence interval for a different weight distribution. If a confidence interval result is required after a defect weight change, the AWRS fault campaign should be rerun.

By default, defects with a weight of 0 are not simulated (see the `skip_zero_weight` argument of the `report` command). If a defect having a `weight=0` is assigned a weight greater than 0 for recalculation, there is no existing simulation to assign a coverage. This defect then contributes to the error term just as any other unsimulated defect per a WSS fault simulation.

Note:

The `-t|--tag` command-line option is ignored when recalculating weight.

recalculate_coverage

When enabled, allows you to perform a what-if analysis updating the defect detection criteria without rerunning defect simulations. This feature is supported for both manufacturing test flows and diagnostic coverage flows. The following inputs are required:

- The FDB file of a completed fault campaign
- A netlist with the following:
 - All the `.MEASURE` statements required for defect detection
 - All the `.set_sim_stop` statements required for defect detection
 - Any `.PARAM` statements required by the `.MEASURE` and `.set_sim_stop` commands

- The FSDB or WDF waveform file from the previously completed fault campaign. All the signals referenced by the measure statement must be present in the FSDB or WDF file from the original fault campaign.
- The `-t|--tag` command-line option that identifies which result to recalculate. This allows for recalculation in multi-testbench flows.

Recalculating coverage is only supported for `.MEASURE` with `.set_sim_stop` defect detection methods.

The regular fault simulation flow is still exercised when performing recalculation so the `-k|--keys` or `--dp` argument is still required on the command line. The Custom Fault tool uses its own utility instead of the fault simulator (PrimeSim or PrimeSim XA) to perform the post-processing of the waveform file to recalculate the result. These processes are still distributed to the compute farm.

When recalculating the coverage, it is also possible to change the weight function for defects of exhaustive or WSS fault campaigns. Changing coverage cannot be supported for AWRs campaigns, because changing weight almost certainly changes which defects are sampled and therefore no previous simulation result is available from which to recalculate the coverage status of a defect and the 95% CI cannot be calculated.

Using `recalculate_coverage` with Mixed-Signal

If the original fault campaign was conducted using mixed-signal simulation, the coverage recalculation can be performed. However, the normal mixed-signal flow cannot be used. The configuration file for the recalculation must be set up using the `analog_setup` command and the netlist provided only has to contain the `.MEASURE`, `.set_sim_stop`, and any dependent `.PARAM` statements.

Examples

`change_weight_only` Example

The following example uses the fault campaign results in `weight1.fdb` to derive the fault coverage status, but applies the weight definition in `weight2.cpp` to generate an updated coverage or diagnostic coverage report file.

```
{
...
"what_if" : {
    "change_weight_only" : true
}
"fault_list_file" : "/data/cf/weight1.fdb"
"defect_weight" : "weight2.cpp"
...
}
```

recalculate_coverage Example

The following example shows the steps for running a multi-testbench test coverage campaign followed by a `recalculate_coverage` analysis. The original netlist files are `tb1.sp` and `tb2.sp` and use `.MEASURE` and `.set_sim_stop` statements to perform defect detection.

1. Run the test coverage campaign:

```
$> customfault -c tb1.json -t tb1 -o cf.tb1 --dp 10
$> customfault -c tb2.json -t tb2 -o cf.tb2 --dp 10
```

2. Recalculate the coverage of the first test, `tb1`, as follows:

- a. Copy the netlist file, `tb1.sp`, to a new file, `tb1-b.sp`.
- b. Edit the new netlist file, `tb1-b.sp`, to change `.MEASURE`, `.PARAM`, and/or `.set_sim_stop` statements.
- c. Copy the file, `tb1.json`, to a new file, `tb1-b.json`.
- d. Edit the new file, `tb1-b.json`, to point at the new netlist file, `tb1-b.sp`.
- e. Add the following `what_if` command to the configuration file:

```
"what_if" : {
  "recalculate_coverage" : true
}
```

- f. Perform the recalculation run. Use the `-f|--fdb` option to point at the prior result and the `-t|--tag` option to indicate which testbench to recalculate. The output must be directed to a new file using the `-o|--out` option because the prior result cannot be overwritten.

```
$> customfault -c tb1-b.json -t tb1 -f cf.tb2.fdb -o cf.tb1-b --dp
10
```

The updated coverage report, `cf.tb1-b.report`, contains the updated results for `tb1`, the prior results for `tb2`, and the new overall combined coverage.

Defect Model Commands

This section lists the Custom Fault defect model commands.

- [apply_fault_model](#)
- [faults](#)

apply_fault_model

Description

Enables you to perform the following actions:

- Specify the circuit scope where the defects are to be identified
- Specify which defect models are to be used in the specified scope

If the `apply_fault_model` command is not specified, the Custom Fault tool still applies defects to the entire netlist by the following rules:

1. If no user-defined defect models are defined, then the [Supported Built-in Defect Models](#) are applied by default.
2. If user-defined defect models are defined, then only the user-defined defect models are applied by default. User-defined defect models are specified using the `faults` command.

If the `apply_fault_model` command is specified with the argument `"fault_model" : ["*"]`, the wildcard matches to only built-in models or only to user-defined models per rules 1 and 2.

Most modern PDKs use subcircuit macro models to define actual design devices, so it is expected that user-defined defect models are required in most situations.

The defect models defined in the `"fault_model"` argument are applied to the scope specified by the `"inst"`, `"subckt"`, `"except_subckt"`, and `"except_subckt"` arguments.

If you specify both `"inst"` and `"subckt"`, `"inst"` is relative to `"subckt"`.

If you specify `"except_inst"` and `"except_subckt"`, the options are independent of each other.

The `"device_model"` argument applies to the specified scope (specified in `"inst"` and/or `"subckt"`).

A wildcard character is supported for `"inst"` and `"subckt"`. `"inst"` supports `*` and `?`, and `"subckt"` supports `*`:

- `*` to match all levels of hierarchy
- `?` to match a single character

Syntax

```
"apply_fault_model" : [ {  
  "fault_model" : [list_of_fault_models],  
  "net_pair" : [{"net1","net2"}, [{"net3","net4"}, ...],
```

```
"except_net_pair" : [{"net1","net2"}, [{"net3","net4"}, ...],
"subckt" : [list_of_subckts],
"inst" : [list_of_insts],
"device_model" : [list_of_device_models],
"except_subckt" : [list_of_subckts],
"except_inst" : [list_of_insts]
},
{
  "fault_model" : [list_of_fault_models],
  "subckt" : [list_of_subckts],
  "inst" : [list_of_insts],
  "device_model" : [list_of_device_models],
  "except_subckt" : [list_of_subckts],
  "except_inst" : [list_of_insts],
  "avalon_inst" : "trace_file"
},
"multi_fault": [
  {
    "fault_model": "built-in_model_or_user-defined_model",
    "inst":["inst1", "inst2",...],
    "terminal": ["terminal_name", "terminal_name"]
  },
  {
    "fault_model": "built-in_model_or_user-defined_model",
    "inst":["inst1", "inst2",...],
    "terminal": ["terminal_name", "terminal_name"]
  }, ...]
{
...} ]
```

Arguments

Argument	Description
"apply_fault_model" : {...}	Scopes the defect models to the specified blocks. Only the specified blocks within the scope are considered for defect identification.

Argument	Description
<pre>"fault_model" : [<i>list_of_fault_models</i>]</pre>	Specifies the defect models to apply. If this command is not specified, the defect models applied by default are only the built-in defect models (see Table 3) unless user-defined defect models are defined. If user-defined defect models are specified using the faults command, then only user-defined defect models are applied by default. Explicitly defining the defect models to be applied allows any combination of built-in or user-defined defect models to be applied. The syntax for <i>list_of_fault_models</i> is as follows: <code>"<i>fault_model_name</i>", "<i>fault_model_name</i>", ...</code> More than one defect model name is allowed. Every defect model name has to be inside "" and separated by a comma.
<pre>"net_pair" : [{"<i>net1</i>", "<i>net2</i>"}, {"<i>net3</i>", "<i>net4</i>"}, ...]</pre>	Applies a Net to Net Short Defect between the specified net pairs. When you specify a net pair with the <i>net_pair</i> argument, the <i>cap_net_to_net</i> argument value in the faults command is ignored. You can specify a wildcard character, but the format must still be a pair. For example, if you want to apply a <i>net_to_net_short</i> defect to all nets inside the XBG instance, you cannot specify: <pre>"net_pair": ["xbg.*"]</pre> You must specify: <pre>net_pair": [{"xbg.*", "xbg.*"}]</pre> You must use the wildcard character in the same scope, which means the nets must belong to the same instance.
<pre>"except_net_pair" : [{"<i>net1</i>", "<i>net2</i>"}, {"<i>net3</i>", "<i>net4</i>"}, ...]</pre>	Specifies the net pairs to be excluded from a Net to Net Short Defect. The <i>net_to_net_short</i> defect is not applied to the net pairs specified in the <i>except_net_pair</i> command even if the coupling capacitance between the nets is greater than the <i>cap_net_to_net</i> argument value specified in the faults command.
<pre>"subckt" : [<i>list_of_subckts</i>]</pre>	Scopes the defect models to the specified subcircuits. All instances within these subcircuits are included for defect identification. The syntax for <i>list_of_subckts</i> is as follows: <code>"<i>subckt_name</i>", "<i>subckt_name</i>", ...</code> More than one subcircuit name is allowed. Every subcircuit name has to be inside "" and separated by a comma.

Argument	Description
<code>"inst" : [list_of_insts]</code>	Scopes the defect models to the specified instances in which defects are identified. The syntax for <i>list_of_insts</i> is as follows: <code>"inst_name","inst_name",...</code> More than one instance name is allowed. Every instance name has to be inside "" and separated by a comma.
<code>"device_model" : [list_of_device_models]</code>	Scopes the defect models to the specified device models in which defects are identified. The syntax for <i>list_of_device_models</i> is as follows: <code>"device_model_name","device_model_name",...</code> More than one device model name is allowed. Every device model name has to be inside "" and separated by a comma.
<code>"except_subckt" : [list_of_subckts]</code>	Specifies the subcircuits to be excluded. All instances from these subcircuits are excluded from defect identification. The syntax for <i>list_of_subckts</i> is as follows: <code>"subckt_name","subckt_name",...</code> More than one subcircuit name is allowed. Every subcircuit name has to be inside "" and separated by a comma.
<code>"except_inst" : [list_of_insts]</code>	Specifies the instances to be excluded from defect identification. The instance names must be an absolute hierarchy. The syntax for <i>list_of_insts</i> is as follows: <code>"inst_name","inst_name",...</code> More than one instance name is allowed. Every instance name has to be inside "" and separated by a comma.
<code>"avalon_inst" : "trace_file"</code>	<i>trace_file</i> is generated from the Avalon schematic view (<code>trace.shl</code>) or mask view (<code>trace.hl</code>). The Custom Fault tool attempts to identify the instances from the trace file in the simulation netlist database to perform defect identification.
<code>"multi_fault" : {...}</code>	Applies multiple defects in the same simulation by specifying an array of defects. Each "multi_fault" array represents the defects that must be injected together and uses the following arguments: • <code>"fault_model"</code> – Defines the defect model to be injected. • <code>"inst"</code> – The defects are injected in the terminal of the listed instances. • <code>"terminal"</code> – The defect is injected in the specified terminal(s). For open defects, only one terminal is specified. For short defects, two terminals are specified.

Examples

In Examples 1-4, no user-defined defects are defined (that is the `faults` command does not include the `"models"` argument).

Example 1

```
"apply_fault_model" : [ {  
  "fault_model" : ["*"]  
} ]
```

This is equivalent to the default setting. All built-in defect models are applied to the entire netlist.

Example 2

```
"apply_fault_model" : [ {  
  "fault_model" : ["mos_short", "mos_open"]  
} ]
```

Only the built-in models for `mos_short` and `mos_open` are applied to the entire netlist.

Example 3

```
"apply_fault_model" : [ {  
  "inst" : ["xtop"]  
} ]
```

All built-in defect models are applied only to the instance `xtop`.

Example 4

```
"apply_fault_model" : [ {  
  "fault_model" : ["mos_short", "mos_open"]  
  "inst" : ["xtop"]  
  "except_inst" : ["xdummy*"]  
} ]
```

The built-in models `mos_short` and `mos_open` are applied to all primitive MOSFETs in the instance `xtop` except for any instance starting with `xdummy`.

Example 5

The following example applies only `mos_open` and `mos_short` to `nchlvt` and `pchlvt`, and `diode_open` to `dfn`.

```
"faults" : {  
  "defaults" : {  
    "short" : {  
      "res" : 1e-6,  
      "cap" : 1e-20  
    },  
    "open" : {
```

```

        "res" : 1e6,
        "cap" : 1e-20
    }
},
"apply_fault_model" : [ {
    "fault_model" : ["mos_open", "mos_short", "diode_open"],
    "device_model" : ["nchlvt", "pchlvvt", "dfn"]
} ]

```

In Examples 6-8, the following additional defect models have been defined:

```

"faults" : {
    "defaults" : {
        "short" : {
            "res" : 200,
            "cap" : null,
        },
        "open" : {
            "res" : 1e6,
            "cap" : null,
        },
    },
},
"models" : [
    {
        "name" : "macmos_short_dgs",
        "base_model" : "subckt_short",
        "subckt" : ["nch_mac", "pch_mac" ],
        "terminals" : ["d", "g", "s"]
    },
    {
        "name" : "macmos_open_dgs",
        "base_model" : "subckt_open",
        "subckt" : ["nch_mac", "pch_mac" ],
        "terminals" : ["d", "g", "s"]
    }
]
}

```

Example 6

```

"apply_fault_model" : [ {
    "fault_model" : ["*"]
} ]

```

This is equivalent to the default setting. Because user-defined defect models have been specified, only the user-defined models `macmo_short_dgs` and `macmos_open_dgs` are applied to the entire netlist. None of the built-in defect models are applied.

Example 7

```
"apply_fault_model" : [ {
  "fault_model" : ["macmos_short_dgs"]
} ]
```

In this example, only `macmos_short_dgs` is applied to the entire netlist. The user-specified `macmos_open_dgs` model is not applied and neither are any built-in defect models.

Example 8

```
"apply_fault_model" : [ {
  "fault_model" : ["macmos_short_dgs", "macmos_open_dgs", "res_short",
    "res_open"],
  "inst" : ["xtop.xanalog.xbandgap"]
},
{
  "fault_model" : ["macmos_short_dgs", "macmos_open_dgs"],
  "inst" : ["xtop.xanalog.xcore"]
}
]
```

In this example, two blocks are used within the `apply_fault_model` command. The first block applies the user-defined models for `macmos_short_dgs` and `macmos_open_dgs` and the built-in models for `res_short` and `res_open` to the instance `xtop.xanalog.xbandgap`. The second block applies only the user-defined defects for `macmos_short_dgs` and `macmos_open_dgs` to `xcore`.

Example 9

In the following example, the `net_to_net_short` is only applied between the nets `in1` and `a2` that are either in the `SUB1` subcircuit or in the `SUB2` subcircuit (ignoring the `cap_net_to_net` argument value in the `faults` command).

```
"apply_fault_model" : [ {
  "fault_model" : ["net_to_net_short"],
  "net_pair": ["in1", "a2"],
  "subckt" : ["SUB1", "SUB2"]
} ]
```

Example 10

In the following example, the `net_to_net_short` is applied on all nets (provided the coupling capacitance is greater than the `cap_net_to_net` argument value of the `faults` command), except between `in1` and `a2`, even if the coupling capacitance between `in1` and `a2` is greater than `cap_net_to_net`.

```
"apply_fault_model" : [ {
  "fault_model" : ["net_to_net_short"],
  "except_net_pair": ["in1", "a2"]
} ]
```

Example 11

In the following example, five `mos_open` defects are injected at the same time.

```
"apply_fault_model": [
  { "multi_fault": [
    {
      "fault_model": "mos_open",
      "inst":["x1.mn1", "x1.mp2"],
      "terminal": ["d"] },
    {
      "fault_model": "mos_open",
      "inst":["x2.mn1"],
      "terminal": ["s"] },
    {
      "fault_model": "mos_open",
      "inst":["x3.mn1", "x3.mp3"],
      "terminal": ["g"] }
  ] }
]
```

This is similar to using the following Tcl command in the manual flow:

```
inject_fault -inst x1.mn1 -type open -node d -res_val 1.000000e+09
-cap_val 1.000000e-20, inject_fault -inst x1.mp2 -type open
-node d -res_val 1.000000e+09 -cap_val 1.000000e-20, inject_fault
-inst x2.mn1 -type open -node s -res_val 1.000000e+09 -cap_val
1.000000e-20, inject_fault -inst x3.mn1 -type open -node g -res_val
1.000000e+09 -cap_val 1.000000e-20, inject_fault -inst x3.mp3 -type
open -node g -res_val 1.000000e+09 -cap_val 1.000000e-20
```

Example 12

In the following example, two multi-defects are injected. Two `mos_open` defects are injected at the same time, and two `mos_short` are injected at the same time in the next simulation of the same fault campaign.

```
"apply_fault_model": [
  { "multi_fault": [
    {
      "fault_model": "mos_open",
      "inst":["x1.mn1"]
      "terminal": "d" },
    {
      "fault_model": "mos_open",
      "inst":["x2.mn1"],
      "terminal": "s" }
  ],
  "multi_fault": [
    {
      "fault_model": "mos_short",
      "inst":["x1.mp2"],
      "terminal": ["d", "g"] },
    {
      "fault_model": "mos_short",
      "inst":["x2.mp1"],
```

```
        "terminal": ["d", "s"] }
    ] }]
```

This is similar to using the following Tcl commands in the manual flow:

```
inject_fault -inst xtop.x1.mn1 -type open -node d -res_val 1.000000e+09
-cap_val 1.000000e-20, inject_fault -inst xtop.x2.mn1 -type open -node s
-res_val 1.000000e+09 -cap_val 1.000000e-20
inject_fault -inst xtop.mp2 -type short -node d g -res_val 1.000000e+02
-cap_val 1.000000e-20, inject_fault -inst xtop.x2.mp1 -type short -node
d s -res_val 1.000000e+02 -cap_val 1.000000e-20
```

Example 13

In the following example, two user-defined defects and one `mos_open` defect are injected at the same time.

```
"apply_fault_model": [
  { "multi_fault": [
    {
      "fault_model": "my_gate_open",
      "inst":["x1top.xml", "x2top.xml"],
      "terminal": ["g"] },
    {
      "fault_model": "mos_open",
      "inst":["x2top.mn1"],
      "terminal": ["s"] }
  ] }]
```

This is similar to using the following Tcl commands in the manual flow:

```
inject_fault -subckt_inst xtop.xml1 -type open -node g -rdiv 1,
inject_fault -subckt_inst xtop.xml2 -type open -node g -rdiv 1,
inject_fault -inst xtop.x2.mn1 -type open -node s -res_val 1.000000e+09
-cap_val 1.000000e-20
```

faults

Defines the defect models.

Syntax

```
"faults" : {
  "defaults" : {
    "short" : {
      "res" : value1r,
      "cap" : value1c
    },
    "open" : {
      "res" : value2r,
      "cap" : value2c
    },
  },
}
```

```
"id_thresholds" : {
    "res" : value3r,
    "cap" : value3c,
    "ind" : value3l,
    "cap_net_to_net" : value3n
},
"wire_threshold": wvalue
},
"models" : [ {
    "include_files" : "file_name ..."
    "name" : "user_defined_name",
    "base_model" : "base_model_name",
    "subckt" : [list_of_subckts],
    "defect_subckt" : "defect_subckt_name",
    "terminals" : [list_of_terminals],
    "terminal_name" : [list_of_terminals],
    "terminal_index" : [list_of_indices],
    "res" : rval,
    "cap" : cval,
    "cap_net_to_net" : cthreshval,
    "gate_rdiv" : true|false,
},
{
    "name" : "user_defined_name",
    "base_model" : "base_model_name",
    "subckt" : [list_of_subckts],
    "terminals" : [list_of_terminals],
    "res" : rval,
    "cap" : cval,
    "cap_net_to_net" : cthreshval
},
{
    "name" : "user_defined_name",
    "base_model" : "base_model_name",
    "inst_param" : "parameter_name",
    "inst_param_abs_values" : [absval1,absval2,...],
    "inst_param_abs_sweep" : [start,stop,step],
    "inst_param_abs_delta" : [absdel1,absdel2,...],
    "inst_param_rel_delta" : [relval1,relval2,...],
    "inst_param_rel_sweep" : [start,stop,step]
},
{
    ...
} ]
}
```

Arguments

Argument	Description
"faults" : {...}	Defines the built-in and customized defect models.

Argument	Description
"defaults" : {...}	Defines the properties for the build-in defect models.
"short" : {...}	Defines the resistor and capacitor values modeling the build-in short defect models.
"res" : <i>value1r</i>	Specifies a resistor of <i>value1r</i> ohm injected between two nodes when a short defect is identified. This is a mandatory field. If it is not specified or set to <i>null</i> , the tool errors out.
"cap" : <i>value1c</i>	Specifies a capacitor of <i>value1c</i> Farad injected between two nodes when a short defect is identified. This is a mandatory field. You can set it to <i>null</i> to omit injection of the capacitor.
"open" : {...}	Defines the resistor and capacitor values modeling the build-in open defect models.
"res" : <i>value2r</i>	Specifies a resistor of <i>value2r</i> ohm to open a connection when an open defect identified. The default when it is not specified is <i>null</i> , which means no resistor is injected.
"cap" : <i>value2c</i>	Specifies a capacitor of <i>value2c</i> ohm to open connection when an open defect is identified. This is a mandatory field. You can set it to <i>null</i> to omit injection of the capacitor.
"id_thresholds" : {...}	Specifies the threshold values at which the defects are only injected to resistors, capacitors, and inductors when their values are higher than the threshold values.
"res" : <i>value3r</i>	Specifies the threshold value for resistors. Only resistors larger than <i>value3r</i> are injected with a defect model. The default is 100.
"cap" : <i>value3c</i>	Specifies the threshold value for capacitors. Only capacitors where $C > \text{value3c}$ are injected with a short or open defect model. The default is $5e-15$.
"ind" : <i>value3l</i>	Specifies the threshold value for inductors. Only inductors larger than <i>value3l</i> are injected with a defect model. The default is $5e-12$.
"cap_net_to_net" : <i>value3n</i>	Specifies the threshold value for coupling capacitors. Only coupling capacitors where $\text{value3n} \leq C < \text{value3c}$ (from "cap") are injected with a <i>net_to_net_short</i> defect model. The default is $1e-17$. Capacitors where $C < \text{value3n}$ are not injected with a defect model.

Argument	Description
<code>"wire_threshold": wvalue</code>	Specifies the wire threshold. Any primitive resistor less than <i>wvalue</i> is considered shorted for the purpose of automatic defect identification. The default value is 10u ohm. Note: The wire threshold does not impact resistor defect identification. For more information, see Wire Threshold.
<code>"models" : [{...}]</code>	Defines the properties for the customized defect models.
<code>"include_files" :</code> <code>"file_name ..."</code>	Lets you include files in a "models" : [{...}] command. See the Description section for more information.
<code>"name" : "user_defined_name"</code>	Defines the name of the customized model.
<code>"base_model" : "base_model_name"</code>	Specifies the base model name for the customer defect model. See Table 3 for the supported base model names. For pin open and net open defects, see the list of supported base model types in Pin Open and Net Open Defect Models. For soft defects, see the list of supported base model types in Instance Parameter Variation Defect Models.
<code>"subckt" : [list_of_subckts]</code>	Specifies the subcircuit names if the base model is <code>subckt_open</code> or <code>subckt_short</code>. This can be used to specify a macromodel. The defect cannot be injected inside the specified subcircuit. The <i>list_of_subckts</i> syntax is: <code>"subckt_name","subckt_name"...</code> More than one subcircuit name is allowed. Every subcircuit name must be inside "" and separated by a comma.
<code>"defect_subckt" :</code> <code>"defect_subckt_name"</code>	Specifies the subcircuit in a SPICE netlist that defines a custom SPICE defect model. See Defining Custom SPICE Defect Models for more information.

Argument	Description
<pre>"terminals" : [<i>list_of_terminals</i>]</pre>	Specifies the terminal names. If none of <code>terminals</code>, <code>terminal_name</code>, or <code>terminal_index</code> is provided, open/shorts are applied to all terminals. The <code>list_of_terminals</code> syntax is: <code>"terminal_name","terminal_name", ...</code> More than one terminal name is allowed. Every terminal name must be inside <code>"</code> and separated by a comma. Use <code>[]</code> to specify short defects between specific terminals in the <code>list_of_terminals</code> syntax as follows: <code>["terminal_name1","terminal_name2"], ...</code> For example, <pre>{ "terminals": [["port1","port2"], ["port3","port4"], ...] }</pre> applies a short defect between <code>port1</code> and <code>port2</code> and another between <code>port3</code> and <code>port4</code>.
<pre>"terminal_name" : [<i>list_of_terminals</i>]</pre>	Specifies the terminal names. Note: This argument can only be used on user-defined subcircuit open/short type defect models. If none of <code>terminals</code>, <code>terminal_name</code>, or <code>terminal_index</code> is provided, open/shorts are applied to all terminals. The <code>list_of_terminals</code> syntax is: <code>"terminal_name","terminal_name", ...</code> More than one terminal name is allowed. Every terminal name must be inside <code>"</code> and separated by a comma.
<pre>"terminal_index" : [<i>list_of_indexes</i>]</pre>	Specifies the terminal indexes. Note: This argument can only be used on user-defined subcircuit open/short type defect models. If none of <code>terminals</code>, <code>terminal_name</code>, or <code>terminal_index</code> is provided, open/shorts are applied to all terminals. The <code>list_of_indexes</code> syntax is: <code>"index_name","index_name", ...</code> More than one index is allowed. Every index must be inside <code>"</code> and separated by a comma.
<pre>"res" : <i>rval</i></pre>	Specifies a resistor of <code>rval</code> ohm injected for the customized model. You can set <code>rval</code> to null to omit the injection of a resistor. If this value is not specified, the default value is the same as the value specified in the <code>"defaults" : {...}</code> argument.

Argument	Description
<code>"cap" : cval</code>	Specifies a capacitor of <i>cval</i> Farad injected for the customized model. You can set <i>cval</i> to null to omit the injection of a capacitor. If this value is not specified, the default value is the same as the value specified in the <code>"defaults" : {...}</code> argument.
<code>"cap_net_to_net" : cthreshold</code>	Specifies a threshold value for coupling capacitors. Only coupling capacitors larger than <i>cthreshval</i> are injected with a <code>net_to_net_short</code> defect model. By default, coupling capacitors with value larger than $1e-17$ are injected with a <code>net_to_net_short</code> defect model.
<code>"gate_rdiv" : true false</code>	Set to <code>true</code> when defining a resistor divider gate model. The default is <code>false</code> . For more information, see Resistor Divider Gate Open Defect Model .
<code>"inst_param" : parameter_name</code>	Specifies the instance parameter to vary in the soft defect flow. See Instance Parameter Variation Defect Models for more information.
<code>"inst_param_abs_values" : [absval1, absval2, ...]</code>	Specifies a list of actual parameter values.
<code>"inst_param_abs_sweep" : [start, stop, step]</code>	Specifies a sweep of actual parameter values.
<code>"inst_param_abs_delta" : [absdel1, absdel2, ...]</code>	Specifies a list of absolute parameter delta values.
<code>"inst_param_rel_delta" : [relval1, relval2, ...]</code>	Specifies a list of percentage change values.
<code>"inst_param_rel_sweep" : [start, stop, step]</code>	Specifies a sweep of percentage change values.

Description

The Custom Fault tool can read defect models in included files in the configuration file. If you include multiple files, the Custom Fault tool reads all the files and creates the defect universe with user-specified files.

Specify single or multiple included files with the `"include_files": [...]` argument inside the `"models": [...]` command

Single Include File Example

This section shows a configuration file example that includes files at a single level.

```
"models": [
{
```

```
    "include_files": [  
        "./CF_inc/cap_mac.inc",  
        "./CF_inc/mos_stuckoff.inc",  
    ] } ]
```

"./CF_inc/cap_mac.inc" File

The following example shows the contents of the ./CF_inc/cap_mac.inc file.

```
{ "models " : [  
  {  
    "name": "macro_cfmom_wo_short",  
    "base_model" : "subckt_short",  
    "subckt": ["cfmom_wo"],  
    "terminals": ["a", "b"]  
  },  
  {  
    "name": "macro_cfmom_wo_open",  
    "base_model" : "subckt_open",  
    "subckt": ["cfmom_wo"],  
    "terminals": ["a", "b"]  
  },  
]
```

"./CF_inc/mos_stuckoff.inc" File

The following example shows the contents of the ./CF_inc/mos_stuckoff.inc file.

```
{ "models " : [  
  {  
    "name": "macromos_nch_stuck_off",  
    "base_model" : "subckt_short",  
    "subckt": ["nch_mac"],  
    "terminals": ["n2", "n4"]  
  },  
  {  
    "name": "macromos_pch_stuck_off",  
    "base_model" : "subckt_short",  
    "subckt": ["pch_mac"],  
    "terminals": ["n2", "n4"]  
  },  
]
```

Multiple Include Files Example

This section shows a configuration file example that includes files at a multiple levels.

```
"models": [ {  
  "include_files": [  
    "./CF_inc/rc.inc",  
    "./CF_inc/mos.inc"  
  ]  
} ]
```

"./CF_inc/rc.inc" File

The following example shows the contents of the `./CF_inc/rc.inc` file.

```
{ "models": [
{   "include_files": ["cap_mac.inc", "res_mac.inc"]
}] }
```

"cap_mac.inc" File

The following example shows the contents of the `cap_mac.inc` file.

```
{ "models " : [
{
  "name": "macro_cfmom_wo_short",
  "base_model" : "subckt_short",
  "subckt": ["cfmom_wo"],
  "terminals": ["a", "b"]
},
{
  "name": "macro_cfmom_wo_open",
  "base_model" : "subckt_open",
  "subckt": ["cfmom_wo"],
  "terminals": ["a", "b"]
},
}
```

"res_mac.inc" File

The following example shows the contents of the `res_mac.inc` file.

```
{ "models " : [
{
  "name": "macro_rnodwo_short",
  "base_model" : "subckt_short",
  "subckt": ["rnodwo"],
  "terminals": ["n1", "n2"]
},
{
  "name": "macro_rnodwo_open",
  "base_model" : "subckt_open",
  "subckt": ["rnodwo"],
  "terminals": ["n1", "n2"]
},
}
```

Note:

The `"include_files": [...]` argument is not allowed in other parts of configuration file. It is only allowed inside the `"models": [...]` command.

Terminal Index Example

Given the following subcircuit definition:

```
.subckt pch_mac a b c d w=w l=l
mp a b c d pch w=w l=l
```

```
.model pch pmos level=49
.ends
```

The following model definition can be used to inject drain source short:

```
"faults" : {
  "models" : [
    {
      "name" : "pchmac_short_ds_by_index",
      "base_model" : "subckt_short",
      "subckt" : ["pch_mac"],
      "terminal_index" : [1, 3]
    }
  ]
}
```

Wildcard Support

The "*" wildcard character cannot be used by itself, such as:

```
"models": [{ "subckt" : ["*"]}]
```

However, it can be used with other characters, for example:

```
"models" : [{"name": "sub_open",
"base_model": "subckt_open", "subckt": ["in*"]}]
```

or

```
"models" : [{"name": "sub_open",
"base_model": "subckt_open", "subckt": ["*nv"]}]
```

or

```
"models" : [{"name": "sub_open",
"base_model": "subckt_open", "subckt": ["*nv*"]}]
```

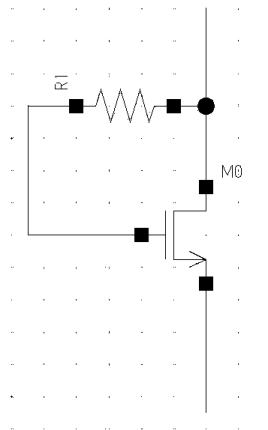
Wire Threshold

Because the "wire_threshold": *wvalue* argument is used to decide whether two nodes separated by a resistor can be considered to be the same node, both defect identification and defect collapsing can be affected by this value, as follows:

- **Defect identification:** If there is a resistor less than *wvalue* between two nodes, then for the purpose of defect identification, these two nodes are considered the same and a short defect between them is not identified.

As shown in the example in [Figure 75](#), if there is a resistor R1 between drain and gate of a transistor with a resistance less than *wvalue*, then a short defect between the drain and the gate node is not identified.

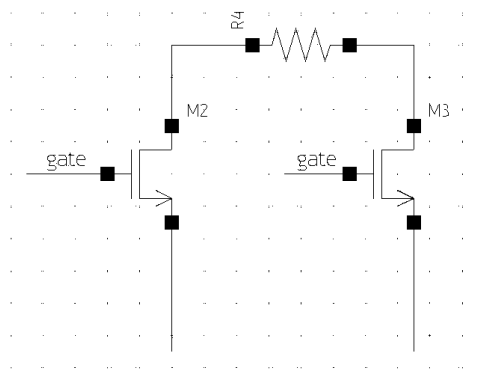
Figure 75 Wire Threshold: Defect Identification



- **Defect collapsing:** When identifying equivalent defects that may be collapsed, two nodes that are separated by resistance less than *wvalue* are considered to be the same node.

As shown in the example in [Figure 76](#), if the resistor R4 is less than *wvalue*, then R4 is considered a short and the drains of M2 and M3 are the same node. The gate-drain short of M2 is collapsed with the gate-drain short of M3 resulting in only one fault simulation. But if resistor R4 is greater than *wvalue*, there are two unique gate-drain short defects identified that are not collapsed, resulting in two fault simulations.

Figure 76 Wire Threshold: Defect Collapsing



Note:

The "wire_threshold": *wvalue* argument should only be increased from its default if the netlist contains suitably small parasitic resistors that should be ignored for the purpose of defect identification.

A

Frequently Asked Questions (FAQ)

This appendix answers some frequently asked questions (FAQ) about the Custom Fault tool.

This FAQ is organized into the following sections:

- [General](#)
- [License](#)
- [Defect Models and Simulation](#)
- [Weight and Sampling](#)
- [Mixed-Signal Simulation](#)
- [Manufacturing Test](#)
- [Functional Safety](#)

General

Question:

Is the Custom Fault tool's configuration file case-sensitive? Is there any option to control the case-sensitivity in the configuration file?

Answer:

The Custom Fault tool's configuration file is based on JavaScript Object Notation (JSON) format. JSON format is a case-sensitive format. There is no option to make the configuration file case-insensitive. For example, `"myid" : true` and `"Myid" : true` are two different statements.

Question:

Is the Custom Fault tool supported in the Synopsys container?

Answer:

The Custom Fault tool is enabled to access the Synopsys container but not when running in mixed-signal simulation with VCS PrimeSim AMS. See the link <https://www.synopsys.com/support/licensing-installation-computeplatforms/compute-platforms/containers.html>.

To run the Custom Fault tool in the Synopsys container, use one of the following methods:

- Use the `customfault ... -container` command when you invoke the tool.
- Set the `SNPS_CONTAINER` environment variable to 1 before running the tool.

For details, see the *Synopsys PrimeSim Custom Fault Release Notes*.

Question:

I would like to run the defect-free (or nominal) simulation to check the defect detection checking criteria and transient simulation command setting. Can I use the Custom Fault tool to run a defect-free simulation?

Answer:

Starting with the P-2019.06-SP1 release, the Custom Fault tool can run a defect-free (or nominal) simulation. By default, the Custom Fault tool does not run the defect-free simulation. To enable it, specify `-n only` or `-n yes` on the command line.

When `-n only` is specified, the tool runs only defect-free simulation. When `-n yes` is specified, the defect-free simulation is simulated as part of the fault campaign. The status of the nominal run can be found in the log file.

If the nominal run passes the defect detection checking, then the following message is printed. If `-n yes` is specified, the fault simulation campaign continues.

```
Nominal Simulation Passed!
```

If the nominal run fails, then the following message is printed. The fault simulation campaign is suspended.

```
Aborting due to nominal simulation failure.
```

Note:

Make sure the defect-free simulation passes before running the fault simulation campaign.

Question:

I use the Custom Fault tool to run a fault simulation campaign on different blocks of my circuit. Is there a method to merge all the reports from different blocks into one report?

Answer:

The Custom Fault tool supports functionality to merge the many database files (*.fdb) into one database file and it also generates a merged report file. See [Methodology for Merging FDBs](#) for details.

The merging functionality does not check for duplicated defects, so you must use the proper scope feature for common circuit defect injection.

The following example merges the database files of FAULT_RESULT1/fa1.fdb and FAULT_RESULT2/fa2.fdb into FAULT_MERGE/fa_merge.fdb and generates a combined report, FAULT_MERGE/fa_merge.report.

```
&> customfault -m FAULT_RESULT1/fa1.fdb -m FAULT_RESULT2/fa2.fdb -o  
    FAULT_MERGE/fa_merge
```

License

Question:

If I specify `-k 100` on the command line, but I only need 50 license keys for my fault campaign simulation, what happens to the other 50 license keys?

Answer:

The Custom Fault tool does not start unless it can check out the number of keys requested by the user specified by the `-k` command-line option.

In this scenario, 100 keys are needed to start the tool. If the number of defects to simulate is only 200, the tool checks out 100 keys at the beginning of the fault simulation. Because only 50 keys are needed to run 200 simulation (50 keys * 4 simulations per key), the unneeded 50 keys are checked back into the license server as soon as the 200 defects simulations are launched.

Defect Models and Simulation

Question:

When I run the Custom Fault tool on a large defect universe, the fault simulation campaign is interrupted due to some machine, CPU limit, disk-space or setup issue. Can I resume the fault simulation campaign from where it was interrupted?

Answer:

You can resume a partially completed fault simulation campaign using the `-r` command-line option (or `--resume`) together with the `-f` command-line option to load the fault database file. The same simulation tag specified by the `-t` command-line option must be

used to resume the interrupted fault simulation campaign. See [Rerunning an Interrupted Fault Simulation](#) for details.

Question:

The Custom Fault tool supports collapsing as a feature. What are the criteria for collapsing faults? How does this affect my coverage results?

Answer:

Fault collapsing is the merging of equivalent defects. For example, assume that you have two MOSFETs connected in parallel as follows:

```
mp1 d g s b pch
mp2 d g s b pch
```

A short between `mp1.d` and `mp1.s` is the same electrical circuit as if `mp2.d` and `mp2.s` were shorted. The Custom Fault tool does not need to simulate both of these defects. The tool collapses these defects into a defect group.

The weight of the defect group is the sum of the weights of each defect. Because the weight of the collapsed defect group accounts for both defects, the collapsing of faults does not impact the measured coverage percentage. By default, only one defect from a defect group is simulated to reduce the turnaround time, you could enable the tool to simulate all defects in a defect group using the `"enable_fault_collapse" : false` command.

Question:

Can I inject the digital equivalent defects of stuck-at-0 and stuck-at-1 defects using the Custom Fault tool?

Answer:

There are multiple ways you can specify stuck-at-0 and stuck-at-1 defects using the Custom Fault tool. The recommended way is to apply the `net_to_net_short` defect model by connecting the node to VDD for stuck-at-1 defect or GND for stuck-at-0 defect.

The `cap_net_to_net` threshold must be set to 0 if there are no coupling capacitors in the netlist, as follows:

```
"faults" : {
  "defaults" : {
    "id_thresholds" : {
      "cap_net_to_net" : 0
    }
  }
}
```

You can use the `"net_pair"` argument of the `apply_fault_model` command as follows:

```
"apply_fault_model" : [ {
```

```
"fault_model" : ["net_to_net_short"],  
"net_pair" : [["net1","vdd"], ["net3", "gnd"]]  
} ]
```

This example connects the `net_to_net_short` defect model from `net1` to `vdd`, and from `net3` to `gnd`.

Question:

The Custom Fault tool supports both `cap_short` defect and `net_to_net_short` as built-in defect types. What is the difference between `cap_short` and `net_to_net_short`?

Answer:

The `cap_short` defect model applies to design capacitors and `net_to_net_short` applies to parasitic capacitors. The Custom Fault tool determines whether a capacitor is a design capacitor or a parasitic capacitor based on its value. By default, a capacitor with value equal to or larger than 5e-15 farad is tagged as design capacitor. You can change the default using the following command:

```
"faults" : {  
  "defaults" : {  
    "id_thresholds" : {  
      "cap" : value  
    }  
  }  
}
```

The Custom Fault tool ignores the parasitic capacitor when its value is less than 1e-17 farad. You can change the value by creating a customized model and apply the customized model to your fault simulation campaign. The command to create a customized model based on the `net_to_net_short` defect is as follows:

```
"faults" : {  
  "models" : [ {  
    "name" : "customized_name",  
    "base_model" : "net_to_net_short",  
    "cap_net_to_net" : value  
  } ]  
}
```

Question:

If I know which block of the circuit is faulty—for example, the safety-related section of the circuit—and I would like to inject defects only into those blocks, how do I enable this in the Custom Fault tool?

Answer:

The Custom Fault tool supports the `apply_fault_model` command to scope the defect models to specific blocks of the circuit. The command allows you to scope any built-in

defect model or any customized defect model to specific subcircuits, instances, and device models. It also allows you to exclude specific subcircuits or instances.

Question:

Does the Custom Fault tool support parametric defect? If so, what type of parametric defect models are supported.

Answer:

Parametric defect is only supported in the manual injection flow. You must use the `inject_fault` command and provide a list of parametric defects to be injected in a file. The Custom Fault tool reads in the file and injects parametric defect for fault simulation. Currently, only the following MOSFET models and parameters are supported for parametric defect. See [Parametric Defect](#) for more information.

MOSFET Model	Parameters
BSIM3v3	W, L, Vth, Ids, Ig
Bsim4	W, L, Vth, Ids, Ig
CustomCMI	W, L, Vth, Ids, Ig
BSIMCMG	Vth, Ids, Ig

Question:

Does the Custom Fault tool support injecting dual defects at the same time?

Answer:

The Custom Fault tool supports dual defects injection only in the manual injection flow. You can manually inject two defects at the same time by using the `inject_fault` command to specify two defects in single line separated by a comma (,) with the following format:

```
inject_fault ... , inject_fault ...  
inject_fault ... , inject_fault ...  
...
```

This example injects the `cap_short` defect model on `xi0.c18` and `xi0.c17`.

```
inject_fault -inst xi0.c18 -type short -res_val 5 -fault_type cap_short, inject_fault  
-inst xi0.c17 -type short -res_val 5 -fault_type cap_short
```

You can include the file using the "`fault_list_file`" command or the `-f` command-line option.

Weight and Sampling

Question:

Must I use weight for defects?

Answer:

Yes, all defects always use a defect weight. All defects are assigned a default weight of 1.0, but you can assign different weights to different defects.

The Custom Fault tool uses defect weight to calculate test coverage, diagnostic coverage, and to select samples during defect sampling. The defect's weight is the probability that the defect will occur in a real circuit.

Question:

How do I assign weights to a defect?

Answer:

The Custom Fault tool has a powerful and flexible capability to assign defect weight. Instructions for defining defect weights are described in [Weighting and Sampling for Defects](#).

Question:

How does defect weight affect test coverage?

Answer:

The defect weight represents the probability that a defect will occur in a real circuit. If a certain defect has 10X the weight of another defect, it contributes 10X more to the test coverage than the other defect.

Consider the following simple example:

Defect	Weight	Coverage Contribution
1	1.0	1/36
2	1.0	1/36
3	1.0	1/36
4	1.0	1/36
5	1.0	1/36
6	1.0	1/36

Defect	Weight	Coverage Contribution
7	5.0	5/36
8	5.0	5/36
9	10.0	10/36
10	10.0	10/36

The defect universe of the preceding example contains ten defects with a total weight of 36.0. The coverage contribution of each defect, if detected, is its own weight over the sum of all defect weights. Defects 1 through 6 each contribute 2.78% coverage, while defects 7 and 8 each contribute 13.89%, and defects 9 and 10 each contribute 27.78% to the total coverage.

Defect weight has the same impact in diagnostic coverage.

Question:

Must I use weight with defect sampling?

Answer:

All defects have a weight with and without sampling. The default weight of any defect is 1.0, but you can assign different weights to different defects.

Question:

The report file provides group weight for each defect. What is group weight and how is it different from weight?

Answer:

Each defect has a weight. If two defects are electrically equivalent, they are collapsed together. When defects are collapsed, the group weight is the sum of their individual weights. Only one defect of the group is simulated because all defects in the group are electrically equivalent.

Question:

I've set higher weight on some defects, but when I run sampling, the defects are not simulated. Why?

Answer:

The goal of the sampling algorithm is to provide a coverage number that is close to the actual coverage number with an associated 95% confidence interval. To achieve this goal, the algorithm selects whichever defect is necessary. The higher the weight of a defect, the higher the chances that it is sampled (because defects with higher weights make a

bigger contribution to the coverage). But depending on the weight distribution of the defect universe and the sample size, one defect, regardless of its weight, may or may not be sampled by the algorithm.

To reiterate, the reported coverage and the 95% confidence interval holds true regardless of which defects are sampled. The sampling algorithm is proprietary to Synopsys and its details cannot be revealed. You can evaluate the accuracy of the sampling methodology by comparing the sampled coverage with actual coverage.

Question:

What weight value should I set for a defect?

Answer:

The value for defect weight is up to you to decide. You can rely on your own engineering judgement for your specific application.

The IEEE P2427 working group is creating a standard for analog fault simulation has proposed a variety of defect weight schemes. These IEEE schemes can be used as a guide for setting defect weights. These proposed defect weight schemes are functions of device geometry and the engineering judgement of group members.

Question:

How does Adaptive Weighted Random Sampling (AWRS) work? How is the 95% confidence interval calculated? How can I verify the sampling result?

Answer:

The AWRS algorithm and 95% confidence interval calculation use computational statistics and are proprietary. The methodology to verify the sampling results is as follows:

1. Run exhaustive fault simulation without sampling to obtain the true coverage value for the circuit and testbench.
2. Run sampling with a variety of sample sizes.
3. Compare the coverage value and confidence interval to the true coverage value.
 - 95% of the time, the confidence interval overlaps the true coverage.
 - As the number of samples increases, the confidence interval should reduce.

Question:

I have some formulas for calculating the weights of individual defects based on device instance parameters. How do I define the weights in the Custom Fault tool?

Answer:

The weights must be defined in the weight file. The weight file uses C++ functions to define weight for each defect type and can be read in using "defect_weight" by the Custom Fault tool to calculate the weight for each defect.

You can use the Custom Fault tool to generate a template weight file named *_weights.cpp in the output directory. By default, all weights are set to 1. You can modify that template weight file with your customized weights and then include it for the fault simulation campaign with the "defect_weight" command.

Question:

I saw fault_port0, fault_port_name0, and port_name0 in the template weight file? What are those variables? And how can I use those variables?

```
double mos_short_weight(int scout, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    // (fault_type: mos_short_sd) (model: p) model_name, w, l, gate_cap,
    m, instance_name, fault_port0, fault_port1, fault_port_name0,
    fault_port_name1, port_name0, port_name1, port_name2, port_name3
    GET_DOUBLE_VAR(gate_cap);
    GET_DOUBLE_VAR(l);
    GET_DOUBLE_VAR(m);
    GET_DOUBLE_VAR(w);
    GET_STRING_VAR(fault_port0);
    GET_STRING_VAR(fault_port1);
    GET_STRING_VAR(fault_port_name0);
    GET_STRING_VAR(fault_port_name1);
    GET_STRING_VAR(fault_type);
    GET_STRING_VAR(instance_name);
    GET_STRING_VAR(model_name);
    GET_STRING_VAR(port_name0);
    GET_STRING_VAR(port_name1);
    GET_STRING_VAR(port_name2);
    GET_STRING_VAR(port_name3);
    //////////// CUSTOM WEIGHT CODE GOES HERE ////////////
    double weight=1.0;
    return weight;
}
```

Answer:

fault_portX are the terminal names of the device model or subcircuit model at or between which the defect is being injected. For short defect, there are fault_port0 and fault_port1. For open defect, there is only fault_port0.

The following example assigns the weight of a `mos_short` defect to 5 if there is a short between the source terminal and drain terminal; otherwise, the weight is 1.

```
double mos_short_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    ...
    GET_STRING_VAR(fault_port0);
    GET_STRING_VAR(fault_port1);
    if (!strcmp(fault_port0,"d") || !strcmp(fault_port0,"s")) {
        weight = 5;
    } else if (!strcmp(fault_port0,"s") || !strcmp(fault_port1,"d")) {
        weight = 5;
    } else {
        weight = 1;
    }
}
```

`fault_port_nameX` are the hierarchical port names of the device instance or subcircuit instance where the defects are injected. For short defect, there are `fault_port_name0` and `fault_port_name1`. For open defect, there is only `fault_port_name0`.

The following example assigns the weight of a `mos_short` defect to 10 if one of the port names of a defect is connected to `xtop.x1.x6.a`; otherwise, the weight is 1.

```
double mos_short_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    ...
    GET_STRING_VAR(fault_port_name0);
    GET_STRING_VAR(fault_port_name1);
    if (!strcmp(fault_port_name0," xtop.x1.x6.a")
    || !strcmp(fault_port_name1," xtop.x1.x6.a")) {
        weight = 10;
    } else {
        weight = 1;
    }
}
```

`port_nameX` are all the hierarchical port names of a particular device. If the device is a MOSFET, there are four ports, so there should be `port_name0`, `port_name1`, `port_name2`, and `port_name3`.

The following example assigns the weight of a `mos_short` defect to 5 if the first port name of the MOSFET or the third port name of the MOSFET is `xtop.xdut.s_3`; otherwise, the weight is 1.

```
double mos_short_weight(int scount, char ** skey, char ** sval,
    int dcount, char ** dkey, double * dval)
{
    ...
    GET_STRING_VAR(port_name0);
```

Appendix A: Frequently Asked Questions (FAQ)

Weight and Sampling

```

GET_STRING_VAR(port_name1);
GET_STRING_VAR(port_name2);
GET_STRING_VAR(port_name3);
if (!strcmp(port_name0,"xtop.xdut.s_3")
|| !strcmp(port_name2,"xtop.xdut.s_3")) {
    weight = 5;
} else {
    weight = 1;
}
return weight;
}
}

```

Question:

I have been recommended to set a larger weight value on short defect compared to open defect. Is there any scientific basis for this?

Answer:

The following is stated in “Using Mixed-Signal Defect Simulation to Close the Loop Between Design and Test”, by Sunter, Jurga, and Laidler, *IEEE Transactions on Circuits and Systems-I*, Vol. 63(12), Dec 2016, pages 2313-2322. <https://ieeexplore.ieee.org/document/7755906>

“Defect likelihoods can be estimated for circuit elements in a schematic netlist, without having a layout, because larger capacitances occupy more area in a layout, as do larger resistances (on same mask layer) and larger transistors. For many passive elements, mask layers are implied by the model. Fabrication process statistics are often available for at least a few relative likelihoods [20]: shorts are typically 3 ~ 10 times more likely than opens in today’s mixed-signal CMOS processes, and defect likelihood is proportional to circuit area and inversely proportional to proximity.”

Question:

If I enable sampling in the Custom Fault tool, how many samples may be sufficient for my error target. How do I get this information?

Answer:

See [Estimating Error and Sample Size](#).

Question:

The log file says that the total number of simulations is 100 but the report file says that the total defects simulated is 125. What is the correct number?

Answer:

The total real number of simulation is what is reported in the log file next to "Total Number of Simulations:". If you do not use fault collapsing, this should be the same

as the simulation count reported in the *.report file. If fault collapsing is enabled, then even though the tool is doing 100 real simulations, the effective number of simulations can be higher due to group defect, sometimes considerably higher than the number of actual simulations.

Question:

The Custom Fault tool provides a 95% confidence interval for the coverage result when the sampling is enabled. How do we know whether the number is accurate?

Answer:

Synopsys does not divulge the method we use to calculate the 95% confidence interval. A 95% confidence interval means that the interval provided is correct approximately 95 out of 100 times if we repeat the same sampling experiment with different random seeds.

The following is a simple experiment to verify whether the method used to calculate 95% confidence interval reported by the Custom Fault tool is accurate:

1. Take a test case and run an exhaustive fault campaign to get a reference on the coverage percentage. The case must be small enough to run exhaustively.
2. Run the same case but with a fixed sample size and repeat the run 100 times. Do not fix the sampling seeds.
3. Collect the confidence interval of all the runs. The confidence interval should include the reference coverage derived from the exhaustive run approximately 95% of the time, that is, in approximately 5% of the experiments, the 95% confidence interval does not overlap the exhaustive coverage value.

Question:

My weight definition is using a less-than comparison for a transistor length. My MOSFET length is equal to the compared value but the `if` statement is not resolving correctly. For example, my weight code is using `if (l < 120e-9) {weight=20.0} else {weight=5.0}`. My netlist has a MOSFET with `l=120n`. The weight code is returning 20, but I expect a weight of 5 because 120nm is not less than 120e-9. Why is incorrect weight being assigned?

Answer:

This is a floating-point precision issue. Most floating-point numbers are not represented exactly with the finite precision available in computing systems. It is recommended that a small factor, *epsilon*, be added to the threshold of any value being compared. If the minimum device geometry resolution of the process is 1nm, then adding/subtracting 0.5nm to any comparison should prevent this problem. For example:

```
double epsilon=0.5e-9;  
// 119nm will be weight 20  
// 120nm will be weight 5
```

```
if ( 1 < 120e-9 - epsilon) {  
    weight = 20.0;  
} else {  
    weight = 5.0;  
}
```

Mixed-Signal Simulation

Question:

Does the Custom Fault tool supported mixed-signal fault simulation? How do I set up a mixed-signal fault simulation campaign using VCS PrimeSim AMS (using any of its analog simulator engines: PrimeSim XA, FineSim, or PrimeSim)?

Answer:

The Custom Fault tool does support mixed-signal fault simulation using VCS PrimeSim AMS. To run a fault simulation on a mixed-signal design, the following factors must be considered:

- Defects can only be injected in the analog parts of the design (and not digital).
- The defect detection checking criteria (or observation points in the case of Functional Safety (FuSa)) can be either in the analog or digital domain.
- If any of the defect detection checking criteria (or observation points in the case of FuSa) are in the analog domain, the analog methods of `.measure` statement, or vector check can be used for defect detection criteria.

If any of the defect detection checking criteria (or observation points in the case of FuSa) are in the digital domain, FSDB file compare can be used for defect detection to compare the digital signals. It is highly recommended that the golden FSDB file is generated using the defect-free run from mixed-signal simulation.

- Make sure the following mixed-signal command is included in the mixed-signal control file `vcsAD.init`:

```
report_option msv_dir=spice;
```

This command makes sure that the simulation outputs are placed in the output directory specified by the PrimeSim XA, FineSim, or PrimeSim simulator, and not the one specified by VCS. Not doing so could lead to output file clobbering when parallel mixed-signal simulations are launched from the same `simv` executable.

Manufacturing Test

Question:

What is the recommended flow to analyze the order of testbenches using the Custom Fault tool?

Answer:

In the manufacturing test scenario, many testbenches are created to achieve greatest test coverage possible with the least amount of testbenches. Some testbenches provide better coverage than the others. The Custom Fault tool supports the "test_order" command to estimate the order of testbenches for achieving the test coverage.

You must use the multi-testbench flow to enable the test ordering feature. All defects must be simulated in each iteration. At the end of the simulations, you obtain a *.test_order file to show you an estimated test order to achieve the test coverage.

An example of the *.test_order file below shows the order of testbenches to achieve the maximum test coverage.

```
Test Vector Ordering
Total number of test vectors: 6

Test vector budget: 1
Cumulative weighted coverage: 9.72%
-----
tb1

Test vector budget: 2
Cumulative weighted coverage: 14.56%
-----
tb1
tb2

Test vector budget: 3
Cumulative weighted coverage: 56.35%
-----
tb3
tb1
tb2

Test vector budget: 4
Cumulative weighted coverage: 67.61%
-----
tb3
tb1
tb4
tb2

Test vector budget: 5
```

```
Cumulative weighted coverage: 77.96%
```

```
-----  
tb3  
tb1  
tb4  
tb5  
tb2
```

```
No increase in coverage with additional test vectors.
```

Functional Safety

Question:

I would like to derive diagnostic coverage numbers for my design which has many safety-related analog hardware components. How can I use the Custom Fault tool to derive these?

Answer:

The Custom Fault tool can be used to measure diagnostic coverage of residual faults and of latent faults starting in the P-2019.06-SP2 release. These diagnostic coverage values can then be input to a FMEDA to calculate the important ISO-26262 metrics, SPFM and LFM. For details on using Custom Fault to measure diagnostic coverage, see [Using Custom Fault for Functional Safety](#).